Chapter 1

Introduction to sampling

1.1 Introduction

The fundamental objective of Bayesian statistics is to compute the posterior distribution of a random variable $\theta \in \Theta$ given data $\mathcal{D}$. In the context of supervised learning, where the data $\mathcal{D} = \{X, Y\}$ comprises features/inputs X and responses/labels Y, Bayes' formula for the parameter posterior distribution takes the following form

$$p(\theta \mid \mathcal{D}) = \frac{p(Y \mid \theta, X)p(\theta)}{\int_\Theta p(Y \mid \theta, X)p(\theta)\mathrm{d}\theta}\,, \tag{1.1}$$

where $p(Y \mid \theta, X)$ denotes the likelihood of N observations for the target $Y = \{y_1, \ldots, y_N\}$ given inputs $X = \{x_1, \ldots, x_N\}$, $p(\theta)$ is the density of the prior distribution for θ, and

$$p(\mathcal{D}) = \int_\Theta p(Y \mid \theta, X)p(\theta)\mathrm{d}\theta, \tag{1.2}$$

denotes the *model evidence* of the data. Note that, in general the posterior is often only known up-to a normalizing constant, because, aside from simple models, calculating the integral in the model evidence is not feasible for high-dimensional θ nor is it often available in closed form. Therefore, it is more common to work with the unnormalized posterior $p(\theta \mid \mathcal{D}) \propto p(Y \mid \theta, X)p(\theta)$.

We note that the ultimate goal in solving (1.1) is estimating the *posterior predictive* for new inputs x^*, i.e. the pushforward of the posterior through the test likelihood:

$$p(y^* \mid x^*, \theta)\,, \quad \text{for} \quad \theta \sim p(\theta \mid \mathcal{D})\,. \tag{1.3}$$

This is a distributed quantity that incorporates both *epistemic and aleatoric uncertainty* (uncertainty caused by a lack of data or knowledge, respectively), but often we content ourselves with its expectation as a point estimator $\hat{p}$:

$$\hat{p}(y^* \mid x^*, \mathcal{D}) = \mathbb{E}_{\theta|\mathcal{D}}\left[p(y^* \mid x^*, \theta)\right] = \int_\Theta p(y^* \mid x^*, \theta)p(\theta \mid \mathcal{D})\mathrm{d}\theta\,. \tag{1.4}$$

Simulating or sampling from $p(y^* \mid x^*, \theta)$, or from $\hat{p}(y^* \mid x^*)$, or $p(y^* \mid x^*, \hat{\theta})$ for some point estimator $\hat{\theta}$ is not considered explicitly in this chapter, although it is worth noting that the posterior predictive is often better behaved than the parameter posterior. One primary reason for this may be that it eliminates the non-identifiability arising from exchangeability of the weights on a given layer.

For the remainder of this chapter, we will consider sampling from a generic target distribution $\pi(\theta) = \kappa(\theta)/Z$, where $Z = \int_\Theta \kappa(\theta)\mathrm{d}\theta$ may or may not be known. The reader can think of $\pi(\theta) := p(\theta \mid \mathcal{D})$ as the posterior already introduced in (1.1), but where we drop the data $\mathcal{D}$ for notational convenience.

1.2 Monte Carlo

The most fundamental sampling algorithm is the Monte Carlo algorithm [1406], and it is the basis for many other sampling schemes. Monte Carlo works as follows. Suppose we have a random variable $\theta \in \Theta$ from a distribution which is continuous on $\mathbb{R}^P$. Let π be the probability density function for θ and assume that we are able to simulate

independent and identically distributed (i.i.d.) samples from π. Note that we may refer to π as the distribution itself. For some function of interest h, which is integrable with respect to π, we wish to estimate an expectation

$$\mathbb{E}[h(\theta)] = \int h(\theta)\pi(\theta)\mathrm{d}\theta, \tag{1.5}$$

which is often not available in closed form. Quadrature methods [375] are suitable in low parameter dimension $P \leq 5$, and with tricks up to $P \approx 20$ [606] or even $P \approx 100$ [219]. However, for large P, Monte Carlo reigns supreme due to its remarkable ability to achieve a mean square error (MSE), defined as $\text{MSE} = \mathbb{E}\left[(\hat{h}_M - \mathbb{E}[h(\theta)])^2\right]$, that decays at the canonical rate $\mathcal{O}(1/M)$ in the number of samples M, regardless of dimension or smoothness of the quantity of interest h.

The Monte Carlo method is extremely simple to use. We wish to generate M independent samples $\theta_1, \theta_2, \ldots, \theta_M$ as shown in Algorithm 1.

Algorithm 1: Monte Carlo Algorithm

Inputs : target density π
Output: $\{\theta_i\}_{i=1}^M$
for $i = 1$ **to** M **do**
  simulate $\theta_i \sim \pi$ independently
return $\{\theta_i\}_{i=1}^M$

Using the Monte Carlo samples $\{\theta_i\}_{i=1}^M$ from π, we can estimate the expectation as follows

$$\mathbb{E}[h(\theta)] \approx \hat{h}_M := \frac{1}{M}\sum_{i=1}^{M} h(\theta_i)\,. \tag{1.6}$$

This beautiful equation has remarkable convergence properties, and it is this gold standard that we aim to achieve with other sampling algorithms. Since the samples $\theta_1, \ldots, \theta_M$ are i.i.d. from π, and writing $h_i = h(\theta_i)$, we have

$$\mathbb{E}[\hat{h}_M] = \mathbb{E}\Big[\tfrac{1}{M}\sum_{i=1}^{M} h_i\Big] = \tfrac{1}{M}\sum_{i=1}^{M}\mathbb{E}[h_i] = \mathbb{E}[h(\theta)]. \tag{1.7}$$

Thus $\hat{h}_M$ is an *unbiased* estimator of the target integral. Moreover, we can increase the accuracy of our estimator by increasing the number of samples M. By the strong law of large numbers, if $\mathbb{E}|h(\theta)| < \infty$, we can guarantee that the Monte Carlo estimator converges almost surely to the true value as the number of samples grows without bound. Formally,

$$\frac{1}{M}\sum_{i=1}^{M} h(\theta_i) \;\to\; \mathbb{E}[h(\theta)] \quad \text{almost surely as } M \to \infty. \tag{1.8}$$

That is, with probability one the estimator will eventually get arbitrarily close to the truth. If we invoke the central limit theorem, then we can also quantify how accurate $\hat{h}_M$ is for a given finite M. Define the variance

$$V \;=\; \int \{h(\theta) - \mathbb{E}[h(\theta)]\}^2\,\pi(\theta)\,\mathrm{d}\theta, \tag{1.9}$$

and assume $V < \infty$. Then as $M \to \infty$,

$$\sqrt{M}\left(\frac{\hat{h}_M - \mathbb{E}[h(\theta)]}{\sqrt{V}}\right) \;\xrightarrow{\mathsf{D}}\; \mathcal{N}(0,1). \tag{1.10}$$

Equivalently, for large M,

$$\hat{h}_M \;\sim\; \mathcal{N}\Big(\mathbb{E}[h(\theta)],\, V/M\Big), \tag{1.11}$$

so that the standard error of the Monte Carlo estimator is $\sqrt{V/M}$ and its MSE decays at the canonical rate $O(M^{-1})$ (i.e. root-MSE $O(M^{-1/2})$). This $M^{-1/2}$ convergence is independent of the dimension P or smoothness of h, and is the benchmark against which all more advanced sampling methods are compared.

Often, a challenge is that we cannot sample from our target probability distribution π directly, and can only evaluate or sample from a non-negative unbiased estimator of κ. Addressing these challenges has led to the development of the algorithms presented in the remainder of this chapter.

1.3 Rejection Sampling

Suppose that $q(\theta)$ is a probability density function from which we *can* sample relatively easily and that $\pi(\theta)$ is the target density from which we would like to sample. Suppose in addition that there exists a constant, $c > 0$, such that

$$\frac{\pi(\theta)}{q(\theta)} \leq c, \tag{1.12}$$

whenever $\pi(\theta) > 0$. Then, we can formulate an algorithm for simulating M samples from $\pi(\theta)$ as given in Algorithm 2.

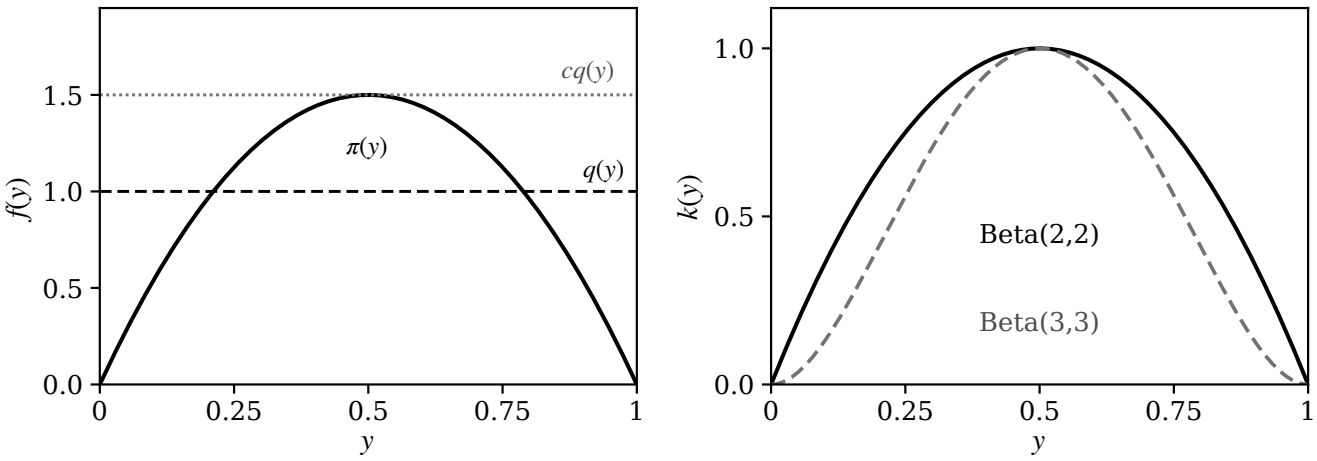


Figure 1.1: Rejection sampling from a Beta$(2, 2)$ target $\pi(y)$ using a uniform proposal $q(y)$. Left: q alone does not dominate π, but the envelope cq with $c = 3/2$ does. Right: the acceptance probability $k(y) = \pi(y)/(cq(y))$, shown also for a Beta$(3, 3)$ target ($c = 15/8$), for which acceptance is lower.

This algorithm is referred to as *rejection sampling*. We can see that this recipe will be very computationally efficient if the densities $q(\theta)$ and $\pi(\theta)$ are approximately equal and the constant c can be chosen close to 1. In this case, the acceptance probability will generally be close to 1. On the other hand, if $\pi(\theta)$ and $q(\theta)$ are very different, then many runs of the algorithm may be required before a value is finally accepted.

Example 1.1. *We illustrate the use of rejection sampling to sample from the Beta(2, 2) distribution. In this case* $\pi(\theta) = 6x(1-x), 0 \leq x \leq 1$. *Suppose that we select* $q(\theta)$ *to be the uniform density on* $(0, 1)$. *Then we can see that* $\pi/q \leq 3/2$ *and the conditions for rejection sampling hold. However, we can see that the acceptance probability will often be small, for example if* $\theta < 0.25$ *or* $\theta > 0.75$ *(something that will happen 1/2 of the time) then acceptance happens less than* 75% *of the time, meaning that we will require significantly more samples than in the ideal i.i.d. case. For the Beta(3, 3) distribution, the rejection rate would be even higher. This can be seen from the right panel of Figure 1.1.*

Despite its attraction of ease of implementation and simulation, the efficiency of rejection sampling relies on (i) having a good proposal distribution and (ii) a bound c. Note that it can easily be modified to work with only κ, but these difficulties remain.

1.4 Importance Sampling

The importance sampling estimator of $\mathbb{E}_\pi[h(\theta)]$ begins by re-writing the integral as follows

$$\int_\Theta h(\theta)\pi(\theta)\mathrm{d}\theta = \int \frac{\pi(\theta)}{q(\theta)} h(\theta)q(\theta)\mathrm{d}\theta\,, \tag{1.13}$$

where q is a density such that $q(\theta) > 0$, whenever $\pi(\theta)h(\theta) \neq 0$. We now produce i.i.d. samples $(\theta_1, \ldots, \theta_M)$ from q, and estimate

$$\hat{h} = \frac{1}{M}\sum_{i=1}^{M} \frac{\pi(\theta_i)}{q(\theta_i)} h(\theta_i) = \frac{1}{M}\sum_{i=1}^{M} w(\theta_i)h(\theta_i)\,. \tag{1.14}$$

Algorithm 2: Rejection Sampling

Inputs : target density π, proposal density q, constant $c > 0$
Output: $\{\theta_i\}_{i=1}^M$
$i \leftarrow 1$
while $i \leq M$ **do**
  simulate $\theta' \sim q$ and $u \sim U(0,1)$
  if $u < \frac{\pi(\theta')}{cq(\theta')}$ **then**
    $\theta_i \leftarrow \theta'$
    $i \leftarrow i + 1$
return $\{\theta_i\}_{i=1}^M$

We call this procedure *importance sampling*. The density q is called the proposal or instrumental density and $w(\theta_i) = \frac{\pi(\theta_i)}{q(\theta_i)}$ are the *importance weights*. Note that $\hat{h}$ is an unbiased Monte Carlo estimator of $\mathbb{E}_\pi[h(\theta)]$, as shown in Equation (1.7). There are two reasons why we might be interested in performing importance sampling:

1. Sampling from $\pi(\theta)$ is not possible or too expensive.
2. The function $h(\theta)$ has a large variance, so the conventional unbiased estimator has large Monte Carlo error.

Recalling the results of Section 1.2, the effective test function is now $(\pi/q)h$, and so its regularity dictates the convergence behaviour. Hence, if possible, we should choose a proposal density q such that the variance of $(\pi/q)h$ is small. It can be shown that the optimal proposal distribution is $q \propto \pi|h|$, however that does little good by itself, since we are back to the same problem of approximating q. However, it can provide useful guidance.

Example 1.2. *Let $\pi = \mathcal{N}(0,1)$ and $h = \mathbb{1}(\theta > 4)$, where $\mathbb{1}$ is the indicator function taking the value 1 if the argument is true and 0 otherwise. Since we can simulate i.i.d. from π, it is tempting to do so, i.e. let $\theta_1, ..., \theta_M \sim \pi$ i.i.d. and set*

$$\hat{h} = \frac{1}{M} \sum_{i=1}^{M} h(\theta_i) \,. \tag{1.15}$$

The number of samples M would have to be very large to get a non-zero estimator. An alternative is to produce an i.i.d. sample from an exponential *random variable with rate 1 translated to the right by 4. In this case, we obtain samples $\theta > 4$ such that*

$$w(\theta) = \frac{\pi(\theta)}{q(\theta)} = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{\theta^2}{2} + (\theta - 4)\right) . \tag{1.16}$$

Suppose that we only know how to evaluate κ. It is easy to see that an unbiased estimator of the normalizing constant can be built using the following identity

$$Z = \int_\Theta \kappa(\theta) \mathrm{d}\theta = \int_\Theta \frac{\kappa(\theta)}{q(\theta)} q(\theta) \mathrm{d}\theta \,. \tag{1.17}$$

It is left as an exercise to the reader to build a consistent (but biased) self-normalized importance sampling estimator using only κ. Unlike rejection sampling, importance sampling does not waste samples through the rejection step, which could be a high amount depending on the application at hand.

As alluded to already, a major disadvantage of importance sampling is that it can result in high variance. There are numerous methods beyond what has been introduced here to alleviate such issues, including the *sequential importance sampling and resampling* methods which will be introduced later in Section 1.6.

1.5 Markov chain Monte Carlo

The preceding sections have introduced Monte Carlo integration and two classical approaches—rejection sampling and importance sampling—for drawing samples from a target distribution $\pi(\theta)$, typically the posterior in Bayesian inference. While these methods are simple to use and theoretically well-founded, they suffer significant limitations in high-dimensional settings. Rejection sampling becomes increasingly inefficient as the dimension grows, due to the difficulty of finding proposal distributions that tightly bound the target. Importance sampling, although

more flexible, tends to suffer from high variance unless the proposal distribution is well matched to the target [7, 261]—a condition that is notoriously hard to satisfy in practice, especially in the multimodal, heavy-tailed, and high-dimensional posteriors encountered in Bayesian deep learning [1292].

Markov Chain Monte Carlo (MCMC) provides a powerful and general framework to overcome these challenges. Rather than attempting to generate independent samples from $\pi(\theta)$, MCMC constructs a *Markov chain* whose stationary distribution is π itself. By simulating a trajectory through parameter space in such a way that, asymptotically, the empirical distribution of visited states converges to the target, MCMC transforms the task of independent sampling into one of designing a suitable transition mechanism that ensures both *ergodicity* and *invariance* with respect to π [1608].

In practical terms, MCMC methods produce a correlated sequence of samples $\{\theta_1, \theta_2, \dots, \theta_M\}$, which, under suitable conditions, can still be used to approximate expectations under π using ergodic averages. The canonical estimator takes the form:

$$\hat{h}_M = \frac{1}{M} \sum_{i=1}^{M} h(\theta_i), \tag{1.18}$$

which, despite the dependence between samples, converges to the true posterior expectation $\mathbb{E}_\pi[h(\theta)]$ under mild regularity assumptions [613].

In what follows, we introduce several fundamental MCMC algorithms, starting with the *Metropolis–Hastings algorithm*, which provides a generic recipe for constructing π-invariant Markov chains from arbitrary proposal distributions [713, 1169]. We then examine key special cases—random-walk Metropolis, the Metropolis-adjusted Langevin algorithm (MALA), and Hamiltonian Monte Carlo (HMC)—that are particularly relevant in the context of high-dimensional posterior inference in Bayesian deep learning. Throughout, we pay close attention to algorithmic trade-offs, scaling behaviour [509, 1408, 1409], and diagnostic tools for assessing convergence and sampling efficiency [211].

1.5.1 Metropolis–Hastings

Markov chain Monte Carlo (MCMC) proceeds by generating a Markov chain $\theta_1 \to \theta_2 \to \dots$ with transition kernel $\mathcal{M}$, described by a density $p(\theta, \theta')$ on Θ^2, which is invariant with respect to the target posterior π:

$$\int_\Theta \pi(\theta')\, p(\theta', \theta)\, \mathrm{d}\theta' = \pi(\theta). \tag{1.19}$$

The Metropolis–Hastings (MH) algorithm achieves this invariance by first choosing an arbitrary proposal density $q(\theta, \theta')$ that proposes θ' given the current iterate of the Markov chain θ, and then accepting or rejecting each proposed move so that detailed balance—and hence (1.19)—holds.

Concretely, we can define the MH transition kernel as

$$p(\theta, \theta') = q(\theta, \theta')\alpha(\theta, \theta'), \tag{1.20}$$

where the acceptance probability is

$$\alpha(\theta, \theta') = \min\left\{1,\ \frac{\pi(\theta')\, q(\theta', \theta)}{\pi(\theta)\, q(\theta, \theta')}\right\}. \tag{1.21}$$

Because $\pi(\theta) \propto \kappa(\theta)$, any unknown normalisation cancels and only the unnormalized density κ is needed. See Algorithm 3 for a full implementation.

Proposition 1.1. *The Metropolis-Hastings algorithm satisfies the equation* (1.19).

Proof. Let us first assume the proposal is accepted. Recalling the definition of $p(\theta, \theta')$ we have

$$p(\theta, \theta') = q(\theta, \theta') \times \alpha(\theta, \theta') = \min\left\{q(\theta, \theta'), \frac{\pi(\theta')q(\theta', \theta)}{\pi(\theta)}\right\}. \tag{1.22}$$

It follows that

$$\pi(\theta)p(\theta, \theta') = \min\{\pi(\theta)q(\theta, \theta'), \pi(\theta')q(\theta', \theta)\} = \pi(\theta')p(\theta', \theta)\,, \tag{1.23}$$

by symmetry. The symmetry of the rejection case is left as an exercise. This property is called *detailed balance* and is a sufficient condition for invariance (1.19). □

Algorithm 3: Metropolis-Hastings Algorithm

Inputs : unnormalized target density κ, proposal distribution q
Output: $\{\theta_i\}_{i=1}^M$
Generate initial sample $\theta_0 \sim p$, with $\pi \ll p$ (absolute continuity)
for $i = 0$ **to** $M - 1$ **do**
  propose new state $\theta' \sim q(\theta_i, \cdot)$
  compute the acceptance probability

$$\alpha(\theta_i, \theta') = \min\left\{1, \frac{\pi(\theta')q(\theta', \theta_i)}{\pi(\theta_i)q(\theta_i, \theta')}\right\} = \min\left\{1, \frac{\kappa(\theta')q(\theta', \theta_i)}{\kappa(\theta_i)q(\theta_i, \theta')}\right\}$$

  draw $u \sim \mathcal{U}[0, 1]$
  if $u \leq \alpha(\theta_i, \theta')$ **then**
    $\theta_{i+1} \leftarrow \theta'$
  else
    $\theta_{i+1} \leftarrow \theta_i$
return $\{\theta_i\}_{i=1}^M$

Invariance with respect to π is a necessary condition for building convergent estimators from a Markov chain. The chain also needs to have certain other stability properties, collectively known as *ergodicity*, to ensure empirical averages (1.6) built from its output converge. A desirable and achievable property is called *geometric ergodicity*, which essentially guarantees linear convergence, i.e. the dependence on the initial condition vanishes geometrically.

Random walk Metropolis

A particularly simple—and historically important—special case of the Metropolis–Hastings algorithm is the *random–walk Metropolis* (RWM) proposal, in which one sets

$$q(\theta, \theta') = q(\theta' - \theta), \tag{1.24}$$

where q is any symmetric density, $q(\theta) = q(-\theta)$. The canonical choice is

$$\theta' = \theta + \epsilon, \qquad \epsilon \sim \mathcal{N}\big(0,\ \lambda^2 I_P\big), \tag{1.25}$$

so that $\theta' \sim \mathcal{N}\left(\theta, \lambda^2 I_P\right)$, i.e. $q(\theta, \cdot) = \mathcal{N}\left(\theta, \lambda^2 I_P\right)$. Due to symmetry, the Metropolis acceptance probability simplifies to

$$\alpha(\theta, \theta') = \min\left\{1,\ \frac{\kappa(\theta')}{\kappa(\theta)}\right\}. \tag{1.26}$$

Local moves and tuning. RWM proposals are inherently *local*: each candidate θ' lies in a neighbourhood of θ. If λ is too small, the chain makes only tiny steps and mixes very slowly; if λ is too large, most proposals fall in low-density regions and are rejected. Optimal scaling results show that, for a wide class of P-dimensional targets, the stationary acceptance rate tends to ≈ 0.234 as $P \to \infty$ when $\lambda \propto P^{-1/2}$ [1408]. In practice, one therefore adapts λ so that the empirical acceptance rate lies between about 20% and 30%.

Preconditioning. When components of θ have very different scales or strong correlations, one can replace $\lambda^2 I_P$ by $\lambda^2 V$, where V is a positive-definite "preconditioning" matrix (often an estimate of the posterior covariance). This *preconditioned RWM* retains symmetry and thus the same simplified acceptance probability, but can yield orders-of-magnitude improvements in computational efficiency, i.e. less samples required to sample the target, because the proposed parameters θ' will be more closely aligned with the posterior distribution, and thus less likely to be rejected.

Gibbs Sampler

Given a multi-variate $\theta = (\theta_1, \theta_2 \dots \theta_n)$, it may be intractable to sample all at once. However if the full conditional distributions for each coordinate are known, then it becomes possible to apply *Gibbs sampling*.

The Gibbs sampling algorithm is fairly straightforward. Given the proposals are known, the Gibbs algorithm updates the sample by updating each coordinate one at a time, i.e. $\theta_i' \sim P(\theta_i \mid \theta_{\neg i})$, where $\theta_{\neg i}$ denotes all coordinates except i. While it may be tempting to update all coordinates at the same time, this would not properly preserve the joint distribution as all coordinates would be updated independently. We include a simple formulation in Algorithm 4.

Algorithm 4: Gibbs Sampler

Inputs : initial state $\theta = (\theta_1, \theta_2 \dots \theta_n)$; full conditional distributions $P(\theta_i \mid \theta_{\neg i})$ for $i \in \{1, \dots, n\}$
Outputs: sample set $\mathcal{S}$
$\mathcal{S} \leftarrow \emptyset$
for *each iteration* **do**
  for $i \in \{1, \dots, n\}$ **do**
    $\theta_i' \sim P(\theta_i \mid \theta_{\neg i})$
    $\theta \leftarrow (\theta_1, \theta_2, \dots \theta_i', \dots \theta_n)$
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{\theta\}$

Given that the Gibbs sampler is drawing from the correct proposal, the typical Metropolis-Hastings acceptance step is not needed in this case. However, this sampler suffers from slow and highly correlated samples due to the nature of the single coordinate update approach.

It is possible to extend the initial Gibbs sampler to a block-style approach in the case where the joint distribution over multiple coordinates is known. Instead of the single-coordinate marginal, if we know the joint distribution over one-set of coordinates conditioned on another set, it is possible to quickly sample by alternative sampling from each joint distribution. This is known as the Block-Gibbs sampler. More formally, if we have two sets of coordinates I, J, the block Gibbs sampler proceeds by sampling $\theta_I' \sim P(\theta_I \mid \theta_J)$, and then sampling $\theta_J' \sim P(\theta_J \mid \theta_I')$. This sampling algorithm avoids the pitfalls of the typical Gibbs sampler. However, it is only possible to use this in cases where the conditional distributions over blocks of coordinates is known.

Metropolis-adjusted Langevin Algorithm

The random-walk Metropolis ignores gradient information about π. The Metropolis–Adjusted Langevin Algorithm (MALA) rectifies this by using an Euler–Maruyama discretisation of the overdamped Langevin diffusion

$$\mathrm{d}\theta_t = \tfrac{1}{2}\nabla \log \pi(\theta_t)\,\mathrm{d}t + \mathrm{d}W_t, \tag{1.27}$$

where $\{W_t\}_{t\geq 0}$ is standard Brownian motion on $\mathbb{R}^P$, and whose stationary law is π. Concretely, the MALA proposal is

$$\theta' | \theta \sim \mathcal{N}\left(\theta + \tfrac{\lambda^2}{2}\,\nabla \log \kappa(\theta), \lambda^2 I_P\right). \tag{1.28}$$

This drift term $\nabla \log \kappa(\theta)$ biases proposed moves towards regions of higher posterior density, enabling larger step-sizes than RWM and better acceptance rates. The MALA proposal also forms the basis for many other wonderful Monte Carlo algorithms, including the stochastic gradient Langevin dynamics (SGLD) [1737] and all other stochastic gradient MCMC (SG-MCMC) algorithms, which will be the focus of the next chapter.

Gradient-based tuning. Under regularity conditions, choosing $\lambda \propto P^{-1/6}$ yields an optimal stationary acceptance rate of approximately 0.574 as $P \to \infty$ [1409]. In applications, one often adapts λ to target an acceptance rate of about 50% to 60%. This can be done by monitoring the acceptance rate and adjusting λ accordingly, or by using more sophisticated adaptive schemes that adjust the step size based on the empirical gradient information.

Preconditioned MALA and robustness. Analogous to RWM, one can introduce a symmetric preconditioning matrix V, replacing I_P by V both in the covariance and in the drift:

$$\theta' \sim \mathcal{N}\left(\theta + \tfrac{\lambda^2}{2}\,V\,\nabla \log \kappa(\theta), \lambda^2 V\right). \tag{1.29}$$

Preconditioning often dramatically accelerates convergence on anisotropic posteriors. However, because MALA relies on $\nabla \log \kappa$, it can be sensitive to regions where the gradient is large or ill-behaved (e.g. lighter-than-Gaussian tails). Common mitigations include gradient clipping or fallback to RWM outside of a "trust region."

Monitoring Performance

Assessing the quality of an MCMC run is essential to ensure that empirical averages $\hat{h}_M = \frac{1}{M}\sum_{i=1}^{M} h(\theta_i)$ reliably approximates $\mathbb{E}_\pi[h(\theta)]$. Three widely used diagnostics are:

1. Autocorrelation and Integrated Autocorrelation Time. For a scalar summary of $h(\theta)$, define the lag-k autocorrelation

$$\rho_k = \frac{\mathrm{Cov}\big[h(\theta_i),\, h(\theta_{i+k})\big]}{\mathrm{Var}_\pi\big[h(\theta)\big]}. \tag{1.30}$$

As the Markov chain is stationary, $\rho_0 = 1$, and $\rho_k \to 0$ as $k \to \infty$. The slower ρ_k decays, the more correlated is the chain. One can show that, via the Markov-chain central limit theorem,

$$\sqrt{M}\big(\hat{h}_M - \mathbb{E}_\pi[h]\big) \xrightarrow{\mathcal{D}} \mathcal{N}\big(0, V_{\text{eff}}\big), \quad \text{where} \quad V_{\text{eff}} = \mathrm{Var}_\pi[h]\left(1 + 2\sum_{k=1}^{\infty}\rho_k\right). \tag{1.31}$$

The quantity $\tau_{\text{int}} = 1 + 2\sum_{k=1}^{\infty}\rho_k$ is the *integrated autocorrelation time*, and the *effective sample size* $\text{ESS} = M/\tau_{\text{int}}$ measures the number of independent draws represented by M correlated samples. Figure 1.3 illustrates autocorrelation decay. A related metric is the *expected squared jumping distance* (ESJD) $\mathbb{E}[\|\theta_{i+1} - \theta_i\|^2]$, which balances move size against acceptance rate. Larger moves will reduce the correlation between elements of the Markov chain, however, larger moves are also less likely to be accepted, which means that the chain does not evolve. Therefore, the challenge for practitioners, is to balance between small moves (high correlation and high acceptance rate) and large moves (low correlation and low acceptance rate).

2. Trace Plots. Plotting the sequence $\{h(\theta_i)\}_{i=1}^{M}$ versus iteration i provides a simple-to-interpret visual representation of mixing and stationarity. Well-mixed chains rapidly traverse the bulk of the posterior, showing no apparent trends or "stickiness" (see Left Panel Figure 1.2). Conversely, slow mixing or multimodality is signalled by long periods of drift or entrapment in subregions of the posterior (see Right Panel Figure 1.2). Trace plots are a useful visual tool for diagnosing the efficiency of an MCMC algorithm, however, they tend to be most useful for low-dimensional posteriors as it is easier to visualise all of the parameters θ, whereas, for high-dimensional posteriors, visualising all of the parameters in the Markov chain individually is often impractical.

3. Potential Scale Reduction (Gelman–Rubin) Diagnostic. When multiple chains $\{\theta_i^{(c)}\}$ are run from overdispersed starting points, one can compare the *between-chain variance* B to the *within-chain variance* W. The *potential scale reduction factor*

$$\widehat{R} = \sqrt{\frac{\frac{M-1}{M}W + \frac{1}{M}B}{W}} \tag{1.32}$$

tends to 1 as all chains converge to the same target distribution [600]. Values of $\widehat{R} < 1.1$ are commonly taken to indicate satisfactory convergence, after which samples from all chains may be pooled to produce an approximation to the posterior π.

In practice, one would usually employ a combination of these diagnostics: trace plots for qualitative assurance, autocorrelation and ESS for quantitative accuracy, and $\widehat{R}$ to guard against failure to explore multimodal targets.

Example 1.3. *We present a Metropolis-Hastings experiment aiming to sample from a target $\pi(\theta) = N(0,1)$. Our proposal is based on a random walk scheme (Section 1.5.1) $\theta' = \theta + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \lambda^2 I)$ is additive Gaussian noise. We run two different experiments, with different choices of step size parameter $\lambda \in \{0.01, 0.9\}$. Our simulations are plotted below in Figures 1.2 and 1.3. For the larger choice of λ, we notice that the chain explores the space well as the iterations progress, with the ACF decorrelating rapidly. However, for the smaller value of λ, the chain wanders slowly and mixes poorly, and the ACF decreases much slower, remaining close to one at moderate lags.*

1.5.2 Hamiltonian Monte Carlo

Random–walk Metropolis (RWM) requires proposal steps of size $\lambda = O(d^{-1/2})$ to maintain a non–degenerate acceptance rate in dimension d, whereas MALA can take steps of size $O(d^{-1/6})$. *Hamiltonian Monte Carlo* (HMC) goes a step further: with an integration step size $\epsilon = O(d^{-1/4})$ it still enjoys an acceptance rate bounded away from 0 while exploring much larger regions of parameter space between consecutive accept–reject decisions [140, 315, 456, 1229]. The key is to embed the d-dimensional parameter θ in a $2d$-dimensional *phase space* and simulate approximate Hamiltonian dynamics that preserve a surrogate "total energy".

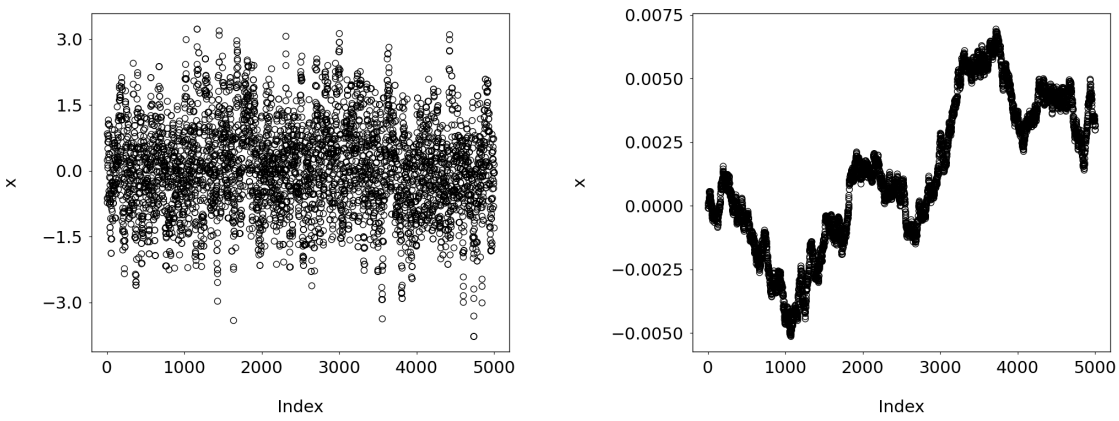


Figure 1.2: Trace plots for $\lambda = 0.9$ (left) and $\lambda = 0.01$ (right).

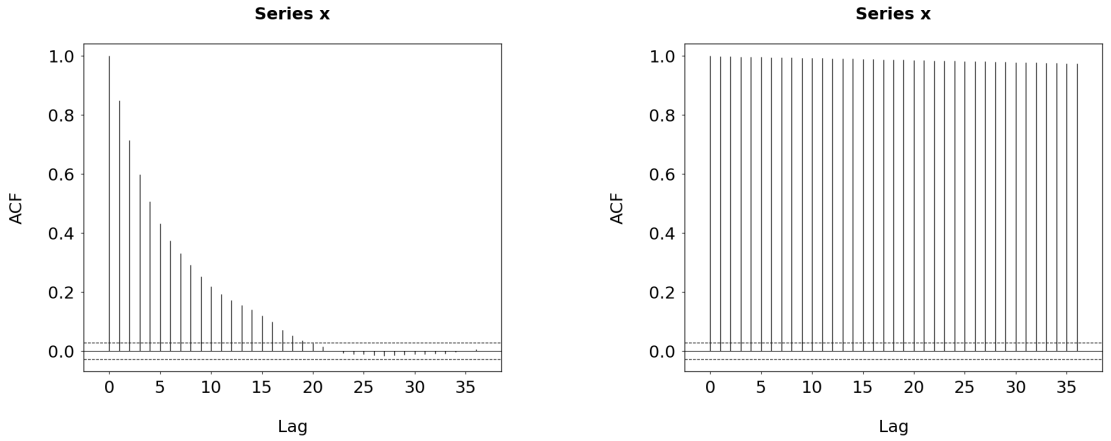


Figure 1.3: Corresponding ACFs for $\lambda = 0.9$ (left) and $\lambda = 0.01$ (right).

Extended target and Hamiltonian. Introduce an auxiliary *momentum* variable $z \in \mathbb{R}^d$ and a positive–definite *mass matrix* M_0. Define the Hamiltonian

$$H(\theta, z) = U(\theta) + K(z) := -\log \pi(\theta) + \tfrac{1}{2} z^\top M_0^{-1} z, \tag{1.33}$$

so that the joint density $\widetilde{\pi}(\theta, z) \propto \exp[-H(\theta, z)]$ has marginal $\pi(\theta)$ and independent $z \sim \mathcal{N}(0, M_0)$.

Hamiltonian dynamics. The continuous equations of motion are

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = M_0^{-1} z, \qquad \frac{\mathrm{d}z}{\mathrm{d}t} = \nabla_\theta \log \pi(\theta), \tag{1.34}$$

which conserve H exactly and therefore move along surfaces of constant joint density. Exact simulation is impossible for general targets π, so HMC uses the second–order, symplectic *leapfrog* integrator [1011] with step

size Δt:

$$\begin{aligned} z_{t+\frac{1}{2}\Delta t} &= z_t + \tfrac{\Delta t}{2}\nabla_\theta \log \pi(\theta_t), \\ \theta_{t+\Delta t} &= \theta_t + \Delta t\, M_0^{-1} z_{t+\frac{1}{2}\Delta t}, \\ z_{t+\Delta t} &= z_{t+\frac{1}{2}\Delta t} + \tfrac{\Delta t}{2}\nabla_\theta \log \pi(\theta_{t+\Delta t}). \end{aligned} \tag{1.35}$$

One repeats this map L times, with $\epsilon = \Delta t$ and $T = L\,\epsilon$ is the *integration time*.

HMC kernel. Starting from θ_{i-1}:

1. Draw momentum $z \sim \mathcal{N}(0, M_0)$.
2. Apply L leapfrog steps (1.35) to obtain (θ', z').
3. Accept (θ', z') with probability
$$\alpha = \min\Big\{1,\, \exp\big[H(\theta_{i-1}, z) - H(\theta', z')\big]\Big\}. \tag{1.36}$$
4. Set $\theta_i = \theta'$ if accepted, otherwise $\theta_i = \theta_{i-1}$; discard z'.

Because the leapfrog map is volume–preserving (i.e. the Jacobian $= 1$) and symmetric (its inverse is obtained by $(\theta, z) \mapsto (\theta, -z)$ and running the L steps backward), the Metropolis correction guarantees $\widetilde{\pi}$ is invariant; the posterior π is therefore stationary for the marginal chain on θ.

Algorithm 5: Hamiltonian Monte Carlo Kernel

Inputs : initial state θ_0, target distribution π, mass matrix M_0
Output: $\{\theta_i\}_{i=1}^M$
for $i = 1$ **to** M **do**
  generate initial momentum $z \sim \mathcal{N}(0, M_0)$
  for $l = 1$ **to** L **do**
    generate $\theta_{l\Delta t}$ and $z_{l\Delta t}$ from (1.35) applied at $(\theta_{(l-1)\Delta t}, z_{(l-1)\Delta t})$, with $\theta_0 = \theta_{i-1}$ and $z_0 = z$
  let $(\theta', z') \leftarrow (\theta_{L\Delta t}, z_{L\Delta t})$ and generate $u \sim U[0,1]$
  if $u \le \min\{1, \exp[H(\theta, z) - H(\theta', z')]\}$ **then**
    $\theta_i \leftarrow \theta'$
  else
    $\theta_i \leftarrow \theta_{i-1}$
return $\{\theta_i\}_{i=1}^M$

High-dimensional scaling. For product targets $\pi(\theta) = \prod_{i=1}^d f(\theta^i)$ with $M_0 = I_d$, one can show [509] that to keep a well-behaved acceptance rate, one must scale $\epsilon = O(d^{-1/4})$, hence $L = O(d^{1/4})$. The computational cost per effectively independent draw is therefore $O(d^{1/4})$, compared with $O(d^{1/3})$ for MALA and $O(d)$ for RWM.

Practical tuning.

- **Step size ϵ.** During warm–up, adapt ϵ (e.g. with dual-averaging) to target an acceptance rate in the 60–80% range; theory suggests 65% is asymptotically optimal [140].
- **Trajectory length $T = L\,\epsilon$.** Too small T gives RWM-like behaviour; too large wastes computation and may return to near the starting point. Popular heuristics randomise L or employ the No-U-Turn criterion [762].
- **Mass matrix M_0.** Setting M_0 to an estimate of the posterior covariance (or its diagonal) acts like the preconditioning matrices used with RWM and MALA, dramatically improving convergence when parameters are on different scales.

Because HMC combines long, momentum-driven proposals with high acceptance probabilities, it has become the default sampler in many Bayesian deep-learning applications, where posteriors are high-dimensional, strongly correlated and feature narrow valleys that thwart simpler MCMC methods.

1.6 Sequential Monte Carlo

Sequential Monte Carlo (SMC) methods are a set of Monte Carlo algorithms designed to sample from a sequence of target probability densities $\{\pi_n(\theta_{1:n})\}$ of increasing dimension, where each one is defined on the product space $\Theta^n = \prod_{i=1}^n \Theta$ [401, 447]. This is referred to as the *smoothing distribution* in the context of state-space models. Often one may be interested only in the time n marginal $\pi_n(\theta_n)$, which is known as the *filtering distribution*. Suppose

$$\pi_n(\theta_{1:n}) = \frac{\kappa_n(\theta_{1:n})}{Z_n}\,, \tag{1.37}$$

where the un-normalized density admits the decomposition

$$\kappa_n(\theta_{1:n}) = \kappa_n(\theta_n|\theta_{1:n-1})\kappa_{n-1}(\theta_{1:n-1})\,, \tag{1.38}$$

and $\kappa_n(\theta_n|\theta_{1:n-1})$ and $\kappa_1(\theta_1)$ can be evaluated, and Z_n is the unknown normalizing constant given by

$$Z_n = \int \kappa_n(\theta_{1:n})\mathrm{d}\theta_{1:n}. \tag{1.39}$$

Note that we write

$$\int \mathrm{d}\theta_{1:n} \quad \text{which is equivalent to} \quad \int_\Theta \int_\Theta \cdots \int_\Theta \mathrm{d}\theta_1 \mathrm{d}\theta_2 \cdots \mathrm{d}\theta_n.$$

SMC provides an approximation of $\pi_1(\theta_1)$ and an estimate of Z_1 at time 1 then an approximation of $\pi_2(\theta_{1:2})$ and an estimate of Z_2 at time 2 and so on. The approximations are done sequentially in time:

- At time step 1, one samples N samples $\{\theta_1^{(i)}\}_{i=1}^N$ from some given function $q_1(\theta_1)$ and then approximates $\pi_1(\theta_1)$ and Z_1 using these samples;
- At time step 2, sample $\{\theta_2^{(i)}\}_{i=1}^N$ from some function $q_2(\theta_2|\theta_1)$, and use the cumulative samples to approximate $\pi_2(\theta_{1:2})$ and Z_2.

Consider another probability density $q_n(\theta_{1:n}) = q_n(\theta_n|\theta_{1:n-1})q_{n-1}(\theta_{1:n-1})$ that is easy to sample from, and such that the support of $q_n(\theta_{1:n})$ includes the support of $\pi_n(\theta_{1:n})$. Then, we have the following importance sampling (IS) identities

$$\pi_n(\theta_{1:n}) = \frac{\kappa_n(\theta_{1:n})}{Z_n} = \frac{w_n(\theta_{1:n})q_n(\theta_{1:n})}{Z_n}, \tag{1.40}$$

$$Z_n = \int w_n(\theta_{1:n})q_n(\theta_{1:n})\mathrm{d}\theta_{1:n}, \tag{1.41}$$

where $w_n(\theta_{1:n})$ is the unnormalized weight function and in light of (1.38) we have the following recursive definition of the weights

$$w_n(\theta_{1:n}) = \frac{\kappa_n(\theta_{1:n})}{q_n(\theta_{1:n})} = \frac{\kappa_n(\theta_n|\theta_{1:n-1})}{q_n(\theta_n|\theta_{1:n-1})} w_{n-1}(\theta_{1:n-1}) =: \alpha(\theta_n|\theta_{1:n-1})w_{n-1}(\theta_{1:n-1})\,. \tag{1.42}$$

Assume we draw, sequentially in time, M independent samples $\theta_{1:n}^{(i)}$, $i = 1, \cdots, M$, from $q_n(\theta_{1:n})$ (see Algorithm 6). We have the following unbiased estimator of Z_n:

$$\widehat{Z}_n = \frac{1}{M}\sum_{i=1}^M w_n(\theta_{1:n}^{(i)}). \tag{1.43}$$

A consistent self-normalized importance sampling estimator of $\mathbb{E}_n(h_n)$, for an arbitrary function $h_n : \Theta^n \to \mathbb{R}$, is given by

$$\widehat{\mathbb{E}_n}(h_n) = \frac{\sum_{i=1}^N w_n(\theta_{1:n}^{(i)})h_n(\theta_{1:n}^{(i)})}{\sum_{i=1}^N w_n(\theta_{1:n}^{(i)})} = \sum_{i=1}^M W_n^{(i)}\ h_n(\theta_{1:n}^{(i)})\,,$$

where

$$W_n^{(i)} = \frac{w_n(\theta_{1:n}^{(i)})}{\sum_{i=1}^M w_n(\theta_{1:n}^{(i)})}\,. \tag{1.44}$$

Algorithm 6: Sequential Importance Sampling (SIS) to estimate $\mathbb{E}_n(\theta_n)$ for $n \geq 1$

Inputs : number of particles M, proposal distributions q_n, adjustment multipliers α_n
Outputs: estimates $\widehat{\theta}_n$ and $\widehat{Z}_n$ for $n \geq 1$
for $i = 1$ **to** M **do**
  initialize weights $w_1^{(i)} \leftarrow 1/M$
$n \leftarrow 1$
repeat
  for $i = 1$ **to** M **do**
    sample $\theta_n^{(i)} \sim q_n(\theta_n \mid \theta_{1:n-1}^{(i)})$
    update unnormalized weight

$$w_n(\theta_{1:n}^{(i)}) \leftarrow \alpha_n(\theta_n^{(i)} \mid \theta_{1:n-1}^{(i)})\, w_{n-1}(\theta_{1:n-1}^{(i)})$$

    normalize weight

$$W_n^{(i)} \leftarrow \frac{w_n(\theta_{1:n}^{(i)})}{\sum_{j=1}^M w_n(\theta_{1:n}^{(j)})}$$

  compute estimates

$$\widehat{\theta}_n \leftarrow \sum_{i=1}^M W_n^{(i)} \theta_n^{(i)}, \qquad \widehat{Z}_n \leftarrow \tfrac{1}{M} \sum_{i=1}^M w_n(\theta_{1:n}^{(i)})$$

  $n \leftarrow n + 1$
until *termination criterion met*

These are the ingredients of the *sequential importance sampling* (SIS) algorithm, given in Algorithm 6. In Algorithms 6 and 7 we take $h_n(\theta_{1:n}) = \theta_n$.

As n grows the weights will degenerate, in the sense that one will dominate the rest. This can be monitored by the effective sample size, which is related to the variance of the weights:

$$\mathsf{ESS} = \frac{\left(\sum_{i=1}^M w_n^{(i)}\right)^2}{\sum_{i=1}^M (w_n^{(i)})^2}\,. \tag{1.45}$$

One way to mitigate this somewhat is by resampling, i.e. for $i = 1, \ldots, M$, let

$$\theta_{1:n}^{(i)} \leftarrow \theta_{1:n}^{(j)}\,, \quad \text{for} \quad j \sim (W_n^{(1)}, \ldots, W_n^{(M)})\,, \quad \text{and} \quad w_n^{(i)} \leftarrow 1/M\,. \tag{1.46}$$

Resampling slightly increases the variance of the estimator, but the particles are rejuvenated at the filtering end, and under suitable conditions the filter can even remain stable online for infinite time [401]. Note that this branching of the particles forward in time means coalescence backward in time, so the particles still degenerate for small times, which is problematic for the smoother. SIS along with resampling is called sequential importance resampling (SIR) or sequential Monte Carlo (SMC). See Algorithm 7. In the context of a state space model, q_n is often chosen as the forward evolution kernel of the hidden process, and in the filtering context this specific choice is often called the *bootstrap particle filter*. In practice, one resamples when the ESS is less than a threshold (e.g. $M/2$ or $M/4$). See Chopin and Papaspiliopoulos [297] for a recent comprehensive introduction.

Algorithm 7: Sequential Monte Carlo (SMC) to estimate $\mathbb{E}(\theta_n)$ for $n \geq 1$

Inputs : number of particles M, proposals q_n, adjustment multipliers α_n
Outputs: estimates $\widehat{\theta}_n$ and $\widehat{Z}_n$ for $n \geq 1$
`// Initialization at` $n = 1$
for $i = 1$ **to** M **do**
  sample $\theta_1^{(i)} \sim q_1(\theta_1)$
  compute $w_1(\theta_1^{(i)}) \leftarrow \kappa_1(\theta_1^{(i)}) / q_1(\theta_1^{(i)})$

$$W_1^{(i)} \leftarrow \frac{w_1(\theta_1^{(i)})}{\sum_{j=1}^{M} w_1(\theta_1^{(j)})} \quad \text{for } i = 1, \ldots, M, \qquad \widehat{Z} \leftarrow 1$$

`// Iterate for subsequent times`
$n \leftarrow 1$
repeat
  `// Effective sample size at time` n

$$ESS_n \leftarrow \left(\sum_{i=1}^{M} \left(W_n^{(i)} \right)^2 \right)^{-1}$$

  `// Optional resampling at time` n
  if $ESS_n \leq M/2$ **then**
    `// Multinomial resampling with probabilities` $W_n^{(1:M)}$
    for $i = 1$ **to** M **do**
      draw $j \sim \text{Categorical}\left(W_n^{(1)}, \ldots, W_n^{(M)}\right)$
      $\theta_{1:n}^{(i)} \leftarrow \theta_{1:n}^{(j)}$
      $w_n^{(i)} \leftarrow 1/M$

$$\widehat{Z} \leftarrow \left(\frac{1}{M} \sum_{i=1}^{M} w_n(\theta_{1:n}^{(i)}) \right) \widehat{Z}$$

  `// Advance to time` $n + 1$
  $n \leftarrow n + 1$
  for $i = 1$ **to** M **do**
    sample $\theta_n^{(i)} \sim q_n(\theta_n \mid \theta_{1:n-1}^{(i)})$
    update unnormalized weight

$$w_n(\theta_{1:n}^{(i)}) \leftarrow \alpha_n(\theta_n^{(i)} \mid \theta_{1:n-1}^{(i)}) w_{n-1}(\theta_{1:n-1}^{(i)})$$

$$W_n^{(i)} \leftarrow \frac{w_n(\theta_{1:n}^{(i)})}{\sum_{j=1}^{M} w_n(\theta_{1:n}^{(j)})} \quad \text{for } i = 1, \ldots, M$$

  `// Estimates at time` n

$$\widehat{\theta}_n \leftarrow \sum_{i=1}^{M} W_n^{(i)} \theta_n^{(i)}, \qquad \widehat{Z}_n \leftarrow \frac{1}{M} \sum_{i=1}^{M} w_n(\theta_{1:n}^{(i)}) \, \widehat{Z}$$

until *termination criterion met*

1.7 Summary

This chapter introduced the Monte Carlo algorithm and a number of methods that develop it. Each addresses the same underlying difficulty: simulating from, or estimating expectations under, a target distribution $\pi(\theta)$ that in most applications can only be evaluated up to an unknown normalising constant.

When independent samples from π are available, Monte Carlo integration (Section 1.2) applies directly. When

they are not, rejection sampling (Section 1.3) and importance sampling construct estimators from an accessible proposal distribution. Both are exact in principle, but become inefficient in high dimension unless the proposal closely matches the target.

Markov chain Monte Carlo (Section 1.5) removes the need for independent draws by constructing a Markov chain whose stationary distribution is the target. The Metropolis–Hastings algorithm (Section 1.5.1) provides the general recipe, with the random-walk Metropolis algorithm (Section 1.5.1), the Gibbs sampler, the Metropolis-adjusted Langevin algorithm (Section 1.5.1), and Hamiltonian Monte Carlo (Section 1.5.2) as the special cases treated here. Since the samples are correlated, step-size tuning and the convergence diagnostics of Section 1.5.1 are essential in practice.

Sequential Monte Carlo (Section 1.6) extends importance sampling to a sequence of targets on nested product spaces, as in filtering. Sequential importance sampling with resampling propagates weighted particles forward in time, with resampling when the effective sample size drops.

The methods play complementary roles: rejection and importance sampling suit low-dimensional or well-matched problems, MCMC is the standard tool for high-dimensional Bayesian computation, and sequential Monte Carlo is suited to inference along a sequence. These ideas are taken further in the chapters that follow. For example, stochastic gradient MCMC is covered in Chapter 2, and sequential Monte Carlo *samplers*, which applies SMC to an artificial sequence of intermediate targets with MCMC mutations, with tempering and multilevel extensions for neural network posteriors, in Chapter 4.

Chapter 2

Stochastic Gradient MCMC

Deep learning models extensively use stochastic gradients (i.e., mini-batches) to reduce the memory requirements of model training [192, 809]. Under suitable assumptions, stochastic gradients computed from individual samples or minibatches can be used in place of full-batch gradients while retaining almost-sure convergence guarantees. [1482]. In this chapter, we introduce stochastic gradient MCMC (SG-MCMC) methods, in which stochastic gradients make MCMC suitable for modern deep learning models.

2.1 Introduction

Standard MCMC algorithms are asymptotically exact samplers under suitable conditions [713]; Chapter 1 provides an introduction to MCMC and HMC. For Langevin-based methods, an appropriate decaying step-size schedule yields asymptotic convergence to the target distribution [266]. However, using the entire dataset to compute each update makes full-batch MCMC impractical for deep learning models, which commonly have tens of millions of parameters. Inspired by stochastic gradient descent (SGD), stochastic gradient MCMC (SG-MCMC) uses stochastic gradients to approximate full-batch MCMC updates. Similar ideas have been transferred to mini-batch Metropolis–Hastings (MH) algorithms [917, 1357, 1853]. However, mini-batch MH often relies on strong assumptions, such as bounded log-likelihoods or bounded gradients, which are not guaranteed for deep learning models. SG-MCMC [273, 430, 1737] addresses these practical concerns under fewer assumptions and with substantially lower memory requirements, thereby bridging the gap between MCMC and Bayesian neural networks (BNNs).

2.2 Classic SG-MCMC Algorithms

In this section, we introduce two concrete examples of SG-MCMC algorithms that are widely used for deep learning models.

2.2.1 Notation

Let θ be the parameters of a deep learning model, and let $\mathcal{S} = \{\theta_s\}_{s=1}^{M}$ be a sample set collected during the MCMC process. Due to memory constraints, we can collect only a finite number of samples to characterise the posterior distribution:

$$p(\theta \mid \mathcal{D}) \approx \frac{1}{M} \sum_{s=1}^{M} \delta(\theta - \theta_s), \tag{2.1}$$

where $\delta(\cdot)$ is the Dirac delta function (also known as the unit impulse) [839]. At inference time, the sample set is used for Bayesian model averaging (BMA) [279], in which the predictive probabilities of all collected samples contribute to the final prediction:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \mathbb{E}_{\theta \sim p(\theta \mid \mathcal{D})}[p(y \mid x, \theta)] \approx \arg\max_{y \in \mathcal{Y}} \frac{1}{M} \sum_{s=1}^{M} p(y \mid x, \theta_s). \tag{2.2}$$

The goal of each SG-MCMC algorithm is to sample from the posterior distribution and collect such a sample set to reconstruct an approximate posterior distribution.

2.2.2 Stochastic Gradient Langevin Dynamics

Stochastic Gradient Langevin Dynamics (SGLD) [1737] is the first well-known SG-MCMC algorithm. The full-batch energy function is $U(\theta) = -\sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \log p(\boldsymbol{y} \mid \boldsymbol{x}, \theta) - \log p(\theta)$. Its mini-batch estimate is

$$\widetilde{U}(\theta) = -\frac{|\mathcal{D}|}{|\boldsymbol{\Xi}|} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\boldsymbol{\Xi}} \log p(\boldsymbol{y} \mid \boldsymbol{x}, \theta) - \log p(\theta), \tag{2.3}$$

where $\boldsymbol{\Xi} \subset \mathcal{D}$ is the sampled batch. The sampling process for SGLD follows the Langevin dynamics:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla_{\theta_t} \widetilde{U}(\theta_t) + \sqrt{2\alpha_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{2.4}$$

where α_t is the step size at iteration t. The complete SGLD framework is outlined in Algorithm 8. Compared with standard SGD, SGLD introduces an additional random-noise term $\sqrt{2\alpha_t}\,\boldsymbol{\epsilon}_t$. As noted by Welling and Teh [1737], SGLD approaches standard unadjusted Langevin dynamics [1410] when the step size α_t is sufficiently small, as the injected noise dominates the stochastic-gradient noise. Under standard regularity conditions, convergence is guaranteed by a decaying step-size schedule [193, 1097, 1737]. Any schedule satisfying i) $\sum_{t=1}^{\infty} \alpha_t = \infty$ and ii) $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$ allows the sampler to explore the target distribution while controlling its asymptotic error [279]. Notably, mini-batch noise in SGD without externally injected noise can produce behaviour qualitatively similar to SGLD, but converges to a biased approximate posterior [1128].

Algorithm 8: Stochastic Gradient Langevin Dynamics (SGLD)

Inputs : dataset $\mathcal{D}$, initial sample $\theta_0 \in \Theta$, step-size schedule $\{\alpha_t\}_{t\geq 0}$
Outputs: collected samples $\mathcal{S} \subset \Theta$
$\theta \leftarrow \theta_0$; $\mathcal{S} \leftarrow \emptyset$
for *each iteration* **do**
  $\boldsymbol{\Xi} \leftarrow$ a mini-batch sampled from $\mathcal{D}$
  $\widetilde{U} \leftarrow -\frac{|\mathcal{D}|}{|\boldsymbol{\Xi}|} \log p(\boldsymbol{\Xi} \mid \theta) - \log p(\theta)$ `// Compute energy`
  $\theta \leftarrow \theta - \alpha_t \nabla_\theta \widetilde{U} + \sqrt{2\alpha_t}\boldsymbol{\epsilon}$ // $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
  $\mathcal{S} \leftarrow \mathcal{S} \cup \{\theta\}$

2.2.3 Stochastic Gradient Hamiltonian Monte Carlo

Full-batch Hamiltonian Monte Carlo (HMC) has achieved gold-standard performance in Bayesian inference [826]. HMC incorporates a kinetic-energy term characterised by a set of auxiliary "momentum" variables. To adapt HMC to deep learning models and reduce its memory requirements, stochastic gradient HMC (SG-HMC) [273] eliminates the need for full-batch gradients and removes the MH correction used by full-batch HMC. Specifically, naive SG-HMC replaces the full-batch energy in the Hamiltonian with its mini-batch estimate:

$$\widetilde{H}(\theta, \boldsymbol{r}) = \widetilde{U}(\theta) + \frac{1}{2}\boldsymbol{r}^\top \boldsymbol{M}^{-1}\boldsymbol{r} = -\frac{|\mathcal{D}|}{|\boldsymbol{\Xi}|} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\boldsymbol{\Xi}} \log p(\boldsymbol{y} \mid \boldsymbol{x}, \theta) - \log p(\theta) + \frac{1}{2}\boldsymbol{r}^\top \boldsymbol{M}^{-1}\boldsymbol{r}, \tag{2.5}$$

where $\boldsymbol{r}$ is the momentum and $\boldsymbol{M}$ is a positive-definite mass matrix. Despite its intuitive appeal and simplicity, naive SG-HMC does not preserve the target posterior [273]: stochastic-gradient noise makes the desired joint distribution $\pi(\theta, \boldsymbol{r}) \propto \exp(-H(\theta, \boldsymbol{r}))$, where $H(\theta, \boldsymbol{r}) = U(\theta) + \frac{1}{2}\boldsymbol{r}^\top \boldsymbol{M}^{-1}\boldsymbol{r}$, non-invariant. To mitigate this problem, Chen et al. [273] introduces a "friction" term $C\boldsymbol{M}^{-1}\boldsymbol{r}$. If $\widehat{B}$ estimates the stochastic-gradient noise coefficient and $C \geq \widehat{B}$, the practical dynamics are

$$\mathrm{d}\theta = \boldsymbol{M}^{-1}\boldsymbol{r}\,\mathrm{d}t \quad \text{and} \quad \mathrm{d}\boldsymbol{r} = -\nabla_\theta \widetilde{U}(\theta)\,\mathrm{d}t - C\boldsymbol{M}^{-1}\boldsymbol{r}\,\mathrm{d}t + \sqrt{2(C - \widehat{B})}\,\mathrm{d}\boldsymbol{W}_t. \tag{2.6}$$

Here, $\boldsymbol{W}_t$ is standard Brownian motion. The friction and injected-noise terms counteract the stochastic-gradient noise. These modified dynamics are commonly known as second-order Langevin dynamics [1707]. The complete SG-HMC framework is outlined in Algorithm 9.

Algorithm 9: Stochastic Gradient Hamiltonian Monte Carlo (SG-HMC)

Inputs : dataset $\mathcal{D}$, initial sample $\theta_0 \in \Theta$, initial momentum $\boldsymbol{r}_0$, step-size schedule $\{\alpha_t\}_{t\geq 0}$, inner steps m, friction coefficient C, noise estimate $\widehat{B}$

Outputs: collected samples $\mathcal{S} \subset \Theta$

$\theta \leftarrow \theta_0$; $\boldsymbol{r} \leftarrow \boldsymbol{r}_0$; $\mathcal{S} \leftarrow \emptyset$

for *each iteration* **do**

- $\boldsymbol{r} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{M})$ `// Optionally resample momentum`
- $(\theta^{(0)}, \boldsymbol{r}^{(0)}) \leftarrow (\theta, \boldsymbol{r})$
- **for** $i = 1$ **to** m **do**
  - $\boldsymbol{\Xi} \leftarrow$ a mini-batch sampled from $\mathcal{D}$
  - $\widetilde{U} \leftarrow -\frac{|\mathcal{D}|}{|\boldsymbol{\Xi}|} \log p(\boldsymbol{\Xi} \mid \theta^{(i-1)}) - \log p(\theta^{(i-1)})$ `// Compute energy`
  - $\theta^{(i)} \leftarrow \theta^{(i-1)} + \alpha_t \boldsymbol{M}^{-1} \boldsymbol{r}^{(i-1)}$
  - $\boldsymbol{r}^{(i)} \leftarrow \boldsymbol{r}^{(i-1)} - \alpha_t \nabla_\theta \widetilde{U} - \alpha_t C \boldsymbol{M}^{-1} \boldsymbol{r}^{(i-1)} + \sqrt{2\alpha_t (C - \widehat{B})} \boldsymbol{\epsilon}_t$ `//` $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
- $(\theta, \boldsymbol{r}) \leftarrow (\theta^{(m)}, \boldsymbol{r}^{(m)})$ `// No Metropolis-Hastings step`
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{\theta\}$

2.3 Practical Concerns and Recent Improvements

SG-MCMC methods have been shown to be effective on many small-scale deep learning models [273, 430, 1737]. However, their application to large-scale deep learning is hindered by several practical concerns. In this section, we discuss these concerns and selected attempts to address them. Chapter 5 presents a complementary perspective on scalable, low-precision sampling.

2.3.1 Sampling from Diverse Local Minima

The loss surface (or energy landscape) of deep learning models is highly non-convex and multimodal [1029, 1854], which makes it difficult for sampling algorithms to move between modes. Moreover, a loss surface typically contains multiple local minima that may all represent good solutions. However, traditional SG-MCMC samplers commonly use a decaying step size and converge to only one local minimum, reducing their ability to explore the entire loss surface.

To mitigate this problem, Zhang et al. [1854] proposed cyclical stochastic gradient MCMC (cSG-MCMC), which uses the cyclical step-size schedule shown in Figure 2.1. The schedule is defined as

$$\alpha_t = \frac{\alpha_0}{2} \left[\cos\left(\frac{\pi \bmod (t-1, \lceil T/K \rceil)}{\lceil T/K \rceil} \right) + 1 \right], \tag{2.7}$$

where α_0 is the initial step size, K is the number of cycles, and T is the total number of iterations. The cyclical step-size schedule enables the sampler to leave the current mode when the step size increases. It can therefore explore and characterise multiple modes while retaining asymptotic convergence to the target distribution.

2.3.2 Sampling from Wide and Robust Local Minima

Distribution shift between training and test data creates a generalisation challenge for SG-MCMC algorithms [91]. Local minima identified using the training data may not remain local minima under the test data, especially when they are sharp [82, 263]. To mitigate this problem, Li and Zhang [1023] introduces Entropy-MCMC (EMCMC), which incorporates flatness-aware optimisation methods [163, 263, 544] into the SG-MCMC framework. Specifically, EMCMC introduces local entropy into the posterior distribution of model parameters:

$$p(\widetilde{\theta} \mid \mathcal{D}) = p(\theta, \theta_a \mid \mathcal{D}) \propto \exp\left\{ -f(\theta) - \frac{1}{2\eta} \|\theta - \theta_a\|^2 \right\}, \tag{2.8}$$

where $f(\theta) = U(\theta)$ is the negative log-posterior energy, $\eta > 0$ is a coupling constant, and θ_a is an auxiliary parameter vector that locally explores the neighbourhood of θ. This posterior induces an extended energy whose gradient is

$$\nabla_{\widetilde{\theta}} U(\widetilde{\theta}) = \begin{bmatrix} \nabla_\theta U(\widetilde{\theta}) \\ \nabla_{\theta_a} U(\widetilde{\theta}) \end{bmatrix} = \begin{bmatrix} \nabla_\theta f(\theta) + \frac{1}{\eta}(\theta - \theta_a) \\ \frac{1}{\eta}(\theta_a - \theta) \end{bmatrix}. \tag{2.9}$$

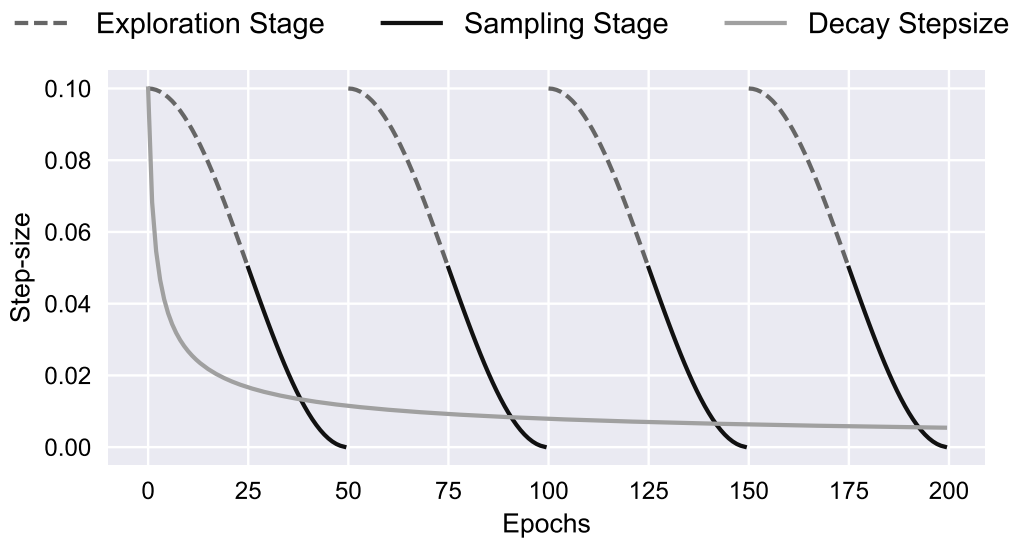


Figure 2.1: Comparison of cyclical and decaying step-size schedules. Adapted from [1854] with the permission of Ruqi Zhang.

This form provides a clear interpretation of the EMCMC sampling process: i) θ seeks low-energy regions of the energy landscape, and ii) θ_a acts as a "direction-correction force" that pulls θ away from sharp minima. Figure 2.2 illustrates this process. EMCMC frames flatness-aware sampling as a standard MCMC process on an extended energy function without introducing an additional correction step. It also has convergence guarantees and can converge faster than previous flatness-aware methods [263, 483].

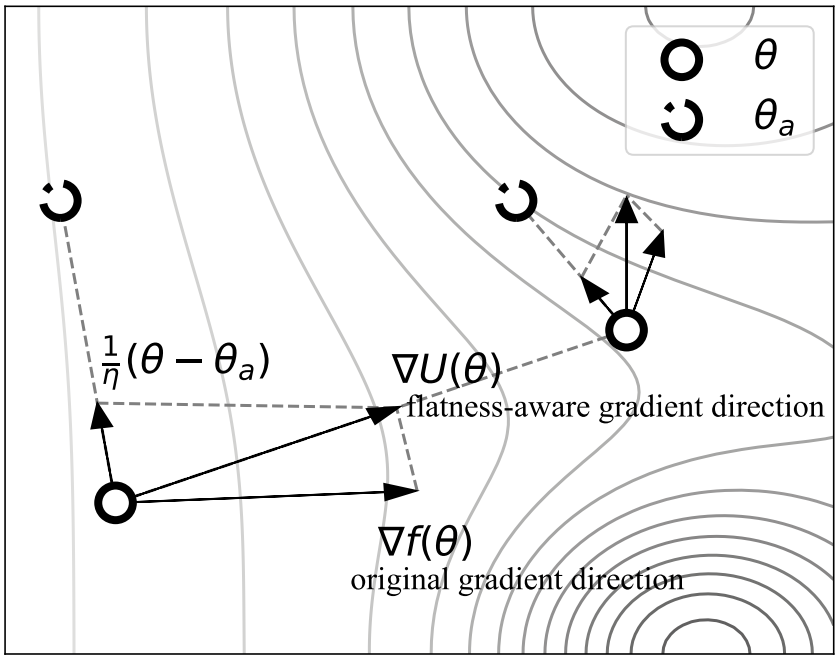


Figure 2.2: The sampling dynamics of Entropy-MCMC, showing how the guiding variable θ_a pulls θ toward flat regions in the energy landscape. Adapted from Li and Zhang [1023] with the permission of Bolian Li.

Chapter 3

Simulation-based inference

Simulation-based inference (SBI) [344, 400, 1520] has the same goal as Bayesian inference: Given observed data and a set of beliefs, apply Bayes' rule to update those beliefs accordingly. Unlike other Bayesian inference methods such as Markov chain Monte Carlo (MCMC) or variational inference, SBI does not rely on evaluations of the likelihood, but only on samples thereof. This makes SBI applicable to any kind of black-box function that takes in parameters and simulates observational data, a so-called forward simulator.

As an illustrative example, imagine that you are an astronomer interested in the study of planets that orbit distant stars, i.e., exoplanets. Exoplanets are usually obscured by the bright light emanating from their host stars. Astronomers observe the brightness and spectrum of these stars in a time series called a light curve, and aim to infer from these light curves properties of the exoplanets using physical models. Suppose we want to estimate the mass of an exoplanet θ_o from the light curve observations x_o of a distant star. Using relevant physical theory, we design a computer program that maps a hypothetical mass θ to a simulated light curve x (the "forward model"). By running such a simulator, we can obtain samples x from the likelihood $p(x \mid \theta)$, but evaluating the likelihood can be computationally expensive or impossible (e.g., if the simulator takes a long time to run, or if it contains many latent variables). *Simulation-based inference* computes the posterior distribution $p(\theta \mid x_o)$ while only relying on samples (or simulations) from the likelihood. To achieve this, recent SBI methods use neural networks to parameterise an "inference model" $q_\psi(\theta \mid x_o)$, where ψ denotes the neural-network weights.

3.1 Introduction

Classical approaches for Bayesian inference (e.g., MCMC; Chapter 1, or variational inference; Chapter 12) rely on repeated evaluations of the likelihood and prior in order to characterise the posterior distribution and to generate samples from it. While powerful, these approaches face limitations for some forward models: Many models in science and engineering are defined as stochastic simulators, which can efficiently be run in the forward direction (i.e., their likelihood can be *sampled* from), but whose likelihood (as well as its gradient with respect to parameters) cannot be evaluated, or only with great computational cost [1088, 1287, 1520]. For example, the simulator might sample a long series of latent variables $z_{1:N}$ before generating observations. Evaluating the likelihood of an observation $p(x_o \mid \theta)$ would require integrating out these latent variables via

$$p(x_o \mid \theta) = \int p(x_o, z_{1:N} \mid \theta) \, \mathrm{d}z_{1:N},$$

which can be computationally prohibitive even for a moderate number of latent variables. In addition, many classical Bayesian inference methods are expensive at test-time: Every time we want to perform inference on new data, we have to re-run the inference procedure, which might involve computationally expensive iterations (e.g., in MCMC sampling). As a result, it has been challenging to tackle high-throughput or real-time applications with Bayesian inference.

Simulation-based inference (SBI) methods overcome these limitations. The basic goal of SBI is to perform inference given only access to simulations (i.e., samples from the likelihood), without having to explicitly evaluate the likelihood, or any access to the internal workings of the forward model or its gradients. In addition, many SBI methods also *amortize* the cost of Bayesian inference: After an initial phase of simulation and training, they can perform inference for any new dataset within milliseconds, typically requiring only a single forward pass through a deep neural network at test time.

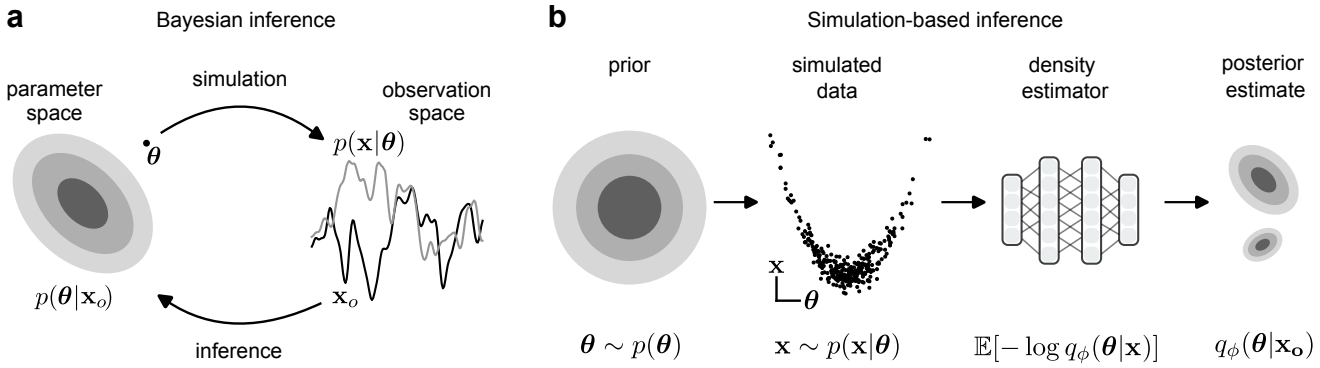


Figure 3.1: *a:* Simulators generate synthetic data x (right) given parameters θ (left). Bayesian inference computes the posterior distribution over parameters (left) given observations x_o (right). *b:* Simulation-based inference uses simulations (i.e., samples from the likelihood, but no likelihood evaluations) to perform Bayesian inference. A recent SBI method, Neural Posterior Estimation (NPE) uses neural networks to improve inference. NPE draws samples from the prior and runs the simulator to generate pairs of (θ, x). These data are then used to train an inference network: A conditional generative model that estimates the posterior distribution given any data x. After training, the inference network is evaluated at any observation in order to draw samples from the posterior, without requiring further simulations or training. Figure modified from Deistler et al. [400].

Early SBI methods were developed for cases in which the likelihood of the model cannot be evaluated, and are often referred to as *Approximate Bayesian Computation* methods [107, 1132, 1520]. Wasserstein ABC provides an example of comparing observed and simulated empirical distributions directly, avoiding the need to choose summary statistics [131]. More recently, the ability of SBI methods to perform inference on any black-box model, to massively parallelise model runs, and to amortize inference has expanded their scope. For example, even when the likelihood or model gradients can—in principle—be evaluated, this might require interfacing with code describing the forward model written across decades in outdated programming languages. In contrast, SBI methods entirely decouple the inference process from simulation runs, which makes it easy to apply SBI methods to simulators that are written in other programming languages or that have to be run on dedicated hardware (or, in principle, which might even correspond to a physical or living system generating the simulations from input). In other scenarios, the likelihood *can* be evaluated, but one needs fast inference for real-time or high-throughput analysis of observed phenomena, which is often out of reach for, e.g., MCMC sampling, but is possible with amortized SBI methods [378]. Finally, in some cases, large-scale training can also yield highly accurate and robust inference results [378, 399, 1092]. Likely due to its simplicity, the core idea of SBI—to use forward simulations to train a supervised machine learning model—has been used in modified ways across many domains, for example, for tabular foundation models [771].

The core insight of SBI is that performing Bayesian inference can be seen as a form of performing *conditional density estimation*. In recent years, a flurry of powerful conditional density estimation methods—such as normalizing flows—have been developed, often under the moniker of "Generative AI". This has enabled an explosion of powerful new SBI methods which build on these advances, and which have been successfully applied in a wide range of scientific disciplines, e.g., in neuroscience [325, 399, 644, 674, 712, 1426], cognitive science [177, 1686], biology [65, 920], and physics [99, 148, 324, 378, 379, 588, 700, 1012, 1188]. Here, we introduce the basic ideas and concepts of SBI. We focus on *Neural Posterior Estimation*, a conceptually simple yet powerful algorithm, but also review other approaches, diagnostic tools, and current developments. Two notable methods are *Neural Likelihood Estimation* [1291] and *Neural Ratio Estimation* [471, 741, 1175, 1601], which we explain later in the chapter.

3.2 Simulation-based inference with Neural Posterior Estimation

3.2.1 Neural Posterior Estimation

The goal of Bayesian inference is to calculate—at least approximately—the posterior distribution of the form $p(\theta \mid x)$. Via Bayes' rule, the posterior is proportional to the product of the prior and the likelihood $p(\theta \mid x) = \frac{1}{Z(x)} p(x \mid \theta) p(\theta)$, where $Z(x) = \int p(x, \theta)\, \mathrm{d}\theta = p(x)$ is the normalizing constant, also called the marginal likelihood or evidence. Calculating $Z(x)$ is often challenging, and many classical Bayesian inference algorithms such as MCMC sampling therefore aim to obtain samples from the posterior without having to evaluate it. However, even

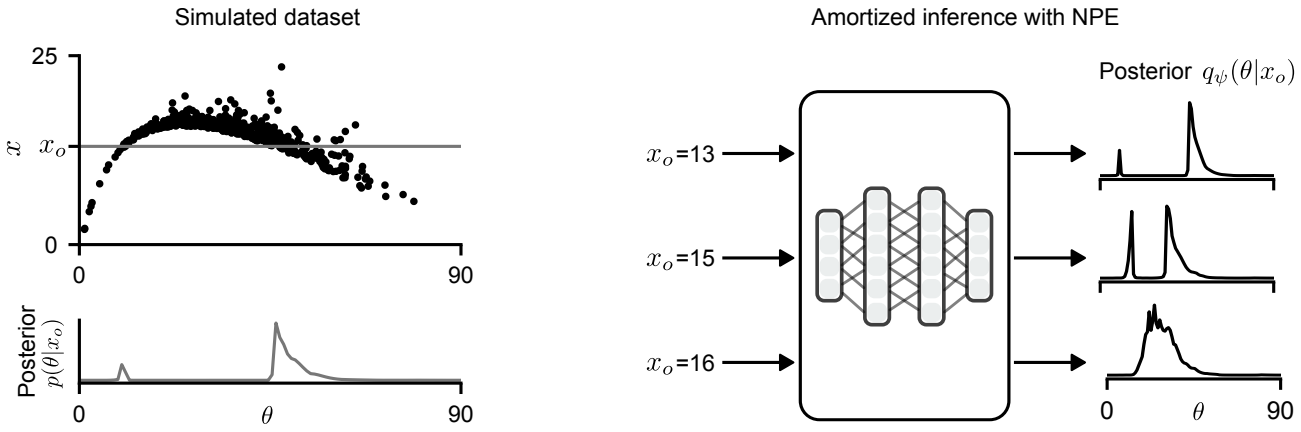


Figure 3.2: Framing Bayesian inference as a supervised learning problem. Left: A database of simulations (θ, x), sampled from the joint distribution $p(\theta, x)$ by drawing samples from the prior and running the simulator. The posterior distribution (left, bottom) is a horizontal cut through the joint distribution. Right: NPE trains an inference network on these simulations. After training, the network can be evaluated at any observation x_o and directly predicts the posterior distribution. Figure modified from Deistler et al. [400].

these algorithms require (typically many) evaluations of the likelihood $p(x \mid \theta)$. However, many models in scientific applications are defined through numerical simulators: One can easily generate *samples* from the likelihood $p(x \mid \theta)$ by running the simulator, but *evaluating* the likelihood can be expensive or even impossible. How can one perform Bayesian inference in such setting?

A key insight is that one does not need Bayes rule for Bayesian inference: The posterior distribution $p(\theta \mid x_o)$ at some particular $x = x_o$ is, by definition, a *conditional distribution*, i.e., a normalised slice through the joint distribution $p(\theta, x)$ at $x = x_o$ (Fig. 3.2, left). Thus, an alternative way to get the posterior is to generate a simulated dataset with—possibly many—paired samples $D = \{(\theta_i, x_i)\}_{i=1}^N$, and then to directly estimate this posterior distribution via conditional density estimation. We can generate such a simulated dataset by first sampling the θ_i from the prior, and then, for each θ_i, to run the simulator once to get the associated x_i, as $p(\theta, x) = p(\theta)p(x \mid \theta)$.

We want to learn a mapping from x to the distribution $p(\theta \mid x)$. One way to tackle this is to parameterise the posterior distribution by a functional form, for example, a Gaussian with mean $\mu(x)$ and covariance $\Sigma(x)$, and to use a neural network (with parameters ψ) to capture the mapping from x to associated posterior parameters $(\mu_\psi(x), \Sigma_\psi(x))$. One can learn ψ by maximising the log-likelihood of the parameters, i.e., by minimising the loss

$$\mathcal{L}(\psi) = -\frac{1}{N} \sum_{i=1}^{N} \log q_\psi(\theta_i \mid x_i), \tag{3.1}$$

where in this case, the predicted posterior probabilities $q_\psi(\theta \mid x)$ would be given by a Gaussian, $q_\psi(\theta \mid x) = \mathcal{N}(\theta \mid \mu_\psi(x), \Sigma_\psi(x))$ [172, 1288]. Thus—and provided that one can generate a large set of simulations D—we have turned Bayesian inference into a supervised learning problem.

Clearly, a Gaussian might be a poor approximation to the posterior, but this approach can readily be generalised to more flexible models. For example, one can use a Mixture of Gaussians [159, 1288], which requires the network to learn the mapping to multiple means $\mu_j(x)$, covariances $\Sigma_j(x)$ and associated mixture weights $\pi_j(x)$. In recent years, more flexible and easy-to-train neural conditional density estimators have been proposed, which can be used for estimating non-Gaussian posteriors. In particular, *Normalizing Flows* [1290] constitute a class of neural conditional densities estimators which have many favourable properties for SBI, as we will describe in the next section.

Once the conditional density estimator (sometimes referred to as *inference network*) has been trained on simulated data D, it can readily be *evaluated* on any new observation x_o to approximate the posterior $p(\theta \mid x_o) \approx q_\psi(\theta \mid x_o)$. Indeed, it can be shown that—provided that the density estimator is sufficiently flexible, that the model is well specified, and the optimisation identifies the minimum—it will yield the true posterior [1288]. To demonstrate this, we rewrite the loss function (in the limit of infinite training data, i.e., in expectation of samples

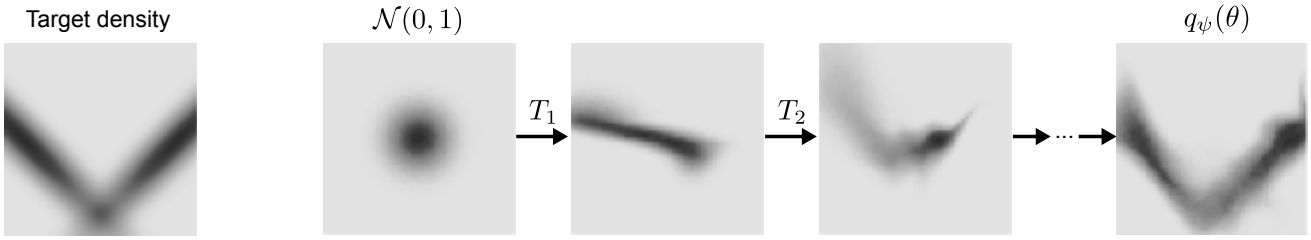


Figure 3.3: Normalizing flows. Left: A target distribution which we aim to estimate based on samples. Right: Normalizing flows define a base distribution (e.g., a multivariate Gaussian) and transform this distribution through multiple transformations T_i. After training, the transformed distribution $q_\psi(\theta)$ approximates the target distribution.

from $p(\theta, x) = p(\theta)p(x \mid \theta)$) as

$$\begin{aligned}
\mathcal{L} &= \mathbb{E}_{p(\theta)p(x|\theta)}[-\log q_\psi(\theta \mid x)] \\
&= \iint -p(\theta)p(x \mid \theta) \log q_\psi(\theta \mid x) \, \mathrm{d}\theta \, \mathrm{d}x \\
&= \iint -p(\theta) \frac{p(\theta \mid x)p(x)}{p(\theta)} \log q_\psi(\theta \mid x) \, \mathrm{d}\theta \, \mathrm{d}x \\
&= \int p(x) \int -p(\theta \mid x) \log q_\psi(\theta \mid x) \, \mathrm{d}\theta \, \mathrm{d}x.
\end{aligned}$$

The inner integral differs from the Kullback–Leibler divergence $\mathrm{D}_{\mathrm{KL}}(p(\theta \mid x) \| q_\psi(\theta \mid x))$ only by the conditional entropy of $p(\theta \mid x)$, which does not depend on the neural-network parameters ψ. Therefore, if the conditional density estimator is well specified, the population loss is minimised when $q_\psi(\theta \mid x) = p(\theta \mid x)$ for $p(x)$-almost every x.

One big advantage of these approaches is that—once the inference network has been trained—the posterior distribution for new observations x_o can be evaluated by just plugging it into the inference network to get $q_\psi(\theta \mid x)$. Thus, inference can be performed with a single forward pass through a neural network, i.e., it is *amortized* (Fig. 3.2, right). Conversely, a drawback of this approach is that our simulations need to cover the entire prior, which might be wasteful as we might generate many simulations which look very different to an observation x_o we are interested in, and which therefore do not help in constraining the relevant posterior. To address this issue (but which requires giving up on amortization), so-called *sequential* approaches have been developed, in which simulations are generated adaptively to target a posterior for a specific observation. We discuss these approaches in Section 3.3.

The approach described above is now commonly known as *Neural Posterior Estimation* (NPE). Given its conceptual simplicity and the many variations that it allows, it is challenging to track down its precise origins, and variants of it have likely been discovered independently in different application domains. Beaumont et al. [107] and Blum and François [172] introduced the idea of fitting a regression model from data to parameters to refine rejection-ABC approaches, which can be interpreted as NPE with a Gaussian inference network. Papamakarios and Murray [1288] showed that flexible density estimators can directly learn the posterior, when optimised with log-loss minimisation (also see Le et al. [991]), and introduced a sequential version. Lueckmann et al. [1090] (which, to our knowledge, introduced the term (S)NPE) introduced an alternative sequential formulation, and also embedding networks for learning summary statistics from time series data. Greenberg et al. [663] provided the first method that could use normalizing flows and is compatible with sequential learning, and used convolutional embedding nets. Ardizzone et al. [45] proposed using invertible neural networks for solving inverse problems, resulting in an algorithm very similar to flow-based NPE, and Radev et al. [1359] popularised the amortization property of NPE.

3.2.2 Conditional density estimation with normalizing flows

As described above, a core task in SBI is to estimate the conditional distribution of parameters given some observations. Normalizing flows [1290] have emerged as a popular conditional density estimator for SBI. They are simple and efficient to train, can be used to quickly generate samples, or to evaluate exact log-probabilities of the learned density. The latter can be useful for many downstream analyses such as maximum-a-posteriori estimation or visualisation of the posterior distribution. Finally, normalizing flows are (typically) trained by minimising the negative log-likelihood as loss function. This loss function is mass-covering, which, as we will discuss below, can

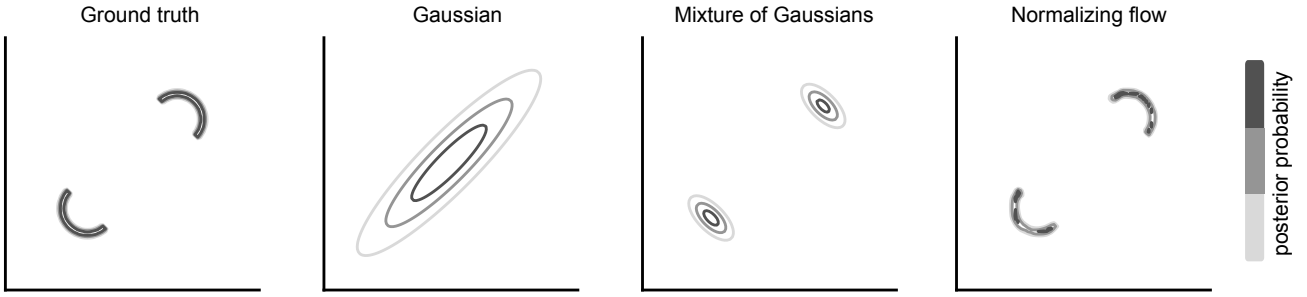


Figure 3.4: Expressiveness of different generative models for approximating the posterior distribution with NPE. The ground truth posterior (left) consists of two moons and has low posterior density in between. Using a Gaussian (second from left) as generative model produces a poor—albeit mass-covering—posterior distribution. A mixture of multiple Gaussians (second from right) improves accuracy, but fails to produce the moon shapes. Normalizing flows (right) enable NPE to accurately capture complex posterior distributions.

be beneficial for SBI.

Normalizing flows model a target density $q(\theta)$ by learning a transformation $T : Z \to \Theta$ such that a random variable Z, which is defined to follow a simple "base" distribution $p(z)$ (typically a multivariate standard Normal distribution) is transformed into a random variable Θ, such that the probability density of Θ is $q(\theta)$. The probability density modelled by such a transformation can be computed with the change of variables formula

$$q(\theta) = \left|\det\left(\frac{\partial T^{-1}}{\partial \theta}\right)\right| p_Z(T^{-1}(\theta)), \tag{3.2}$$

where T^{-1} is the inverse of the transformation T and $\det\left(\frac{\partial T^{-1}}{\partial \theta}\right)$ is the determinant of the Jacobian of the inverse transformation T^{-1}. Normalizing flows stack many transformations T in order to model complicated densities.

To evaluate Eq. 3.2 and to efficiently train normalizing flows, the transformation T must be a diffeomorphism (i.e., it must be differentiable and invertible). In addition, the Jacobian of T^{-1} is of size $N \times N$ (for densities of dimensionality N), and computing the determinant of such a matrix is typically of computational complexity $\mathcal{O}(N^3)$, which would be prohibitive for large N. As such, normalizing flows require that the transformation is constructed such that the determinant of the Jacobian can be evaluated efficiently. Normalizing flows differ in how they define the transformation T such that it fulfills these criteria. A popular class of transformations are autoregressive flows, and within SBI, masked autoregressive flows [1289] and neural spline flows [470] are particularly popular. These normalizing flows define bijective transformations for every individual dimension and then couple dimensions with neural networks.

Normalizing flows train the transformations T by minimising the negative log-likelihood of the data

$$\psi = \arg\min_{\psi} -\log \prod_{i=1}^{M} q_\psi(\theta_i).$$

In neural posterior estimation (NPE), normalizing flows are used to estimate the posterior probability density $p(\theta \mid x)$. To estimate a *conditional* probability density, NPE conditions that transformation T on data x, or an embedding $s(x)$ thereof (Sec. 3.2.3). This enables NPE to learn different posterior distributions for different data.

The normalizing flow is then trained over pairs of parameters and simulation outputs, where the parameters are sampled from the prior. NPE minimises the loss

$$\mathcal{L} = \mathbb{E}_{p(\theta)p(x|\theta)}[-\log q_\psi(\theta \mid x)]. \tag{3.3}$$

If the normalizing flow is well-specified, i.e., the true conditional posterior $p(\theta \mid x)$ belongs to the family of normalizing-flow models, then there exists some $\psi^* \in \Psi$ such that $q_{\psi^*}(\theta \mid x) = p(\theta \mid x)$ for $p(x)$-almost every x, where Ψ denotes the parameter space. In this case, the population loss is minimised at the true conditional posterior, assuming successful optimisation. After training, one can then sample from the posterior for any observation x_o by drawing a sample from the base distribution and transforming it with the series of transformations T (which are conditioned on x_o). Notably, samples from normalizing flows are i.i.d. and follow the estimated density exactly (i.e., normalizing flows do not require any approximations to draw samples from the modelled density). In addition, normalizing flows can also evaluate the density of parameters under the posterior via Eq. 3.2.

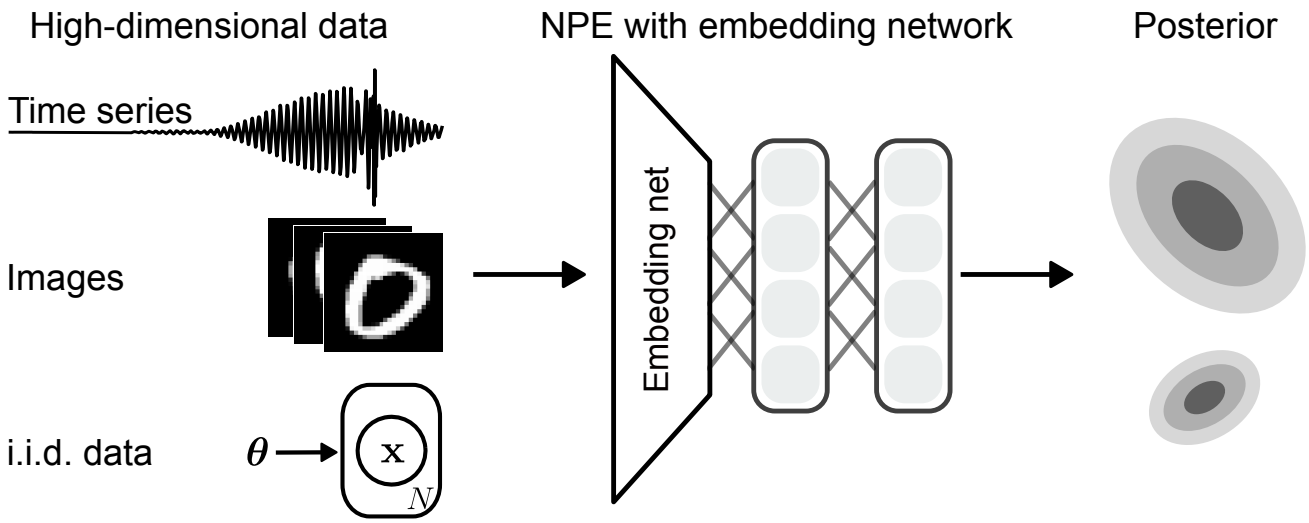


Figure 3.5: Neural Posterior Estimation (NPE) with embedding networks. Left: For many models, simulation outputs can be high-dimensional. For example, simulators might produce long time series, images, or i.i.d. data given a particular parameter set. Middle: In order to perform inference based on such observations, NPE can be combined with suitable embedding networks that efficiently reduce these data to lower-dimensional summary statistics. These summary statistics are then processed by the inference network, and both neural networks can be trained end-to-end. Right: Illustration of a posterior distribution underlying such observations.

An important property of this loss is that it tends to be mass-covering: it will strongly penalise the normalizing flow being too narrow (as $\log q(\theta \mid x)$ becomes a large negative number, leading to very high loss). As such, for limited training data or for imperfect convergence of the normalizing flow, $q(\theta \mid x)$ will tend towards being broader than the true posterior distribution $p(\theta \mid x)$. In many applications of SBI, this is a desirable feature: too narrow posteriors would indicate overly confident parameter estimates, which could lead to wrong claims of scientific discovery [742]. This mass-covering behaviour is in contrast to variational inference, which typically uses divergence objectives whose covering or seeking behaviour depends on the divergence direction, with common formulations tending to be mode-seeking (i.e., towards a too narrow posterior approximation) [1035]. Since we operate with finite samples, none of these are guarantees.

3.2.3 Learning from high-dimensional data with embedding networks

Many models in science and engineering produce high-dimensional simulation outputs such as images or long time series. In order to estimate parameters underlying such data, NPE can be combined with *embedding networks* (Fig. 3.5) [1090]. Embedding networks are neural networks which take as input high-dimensional data x and return a lower-dimensional summary statistic $s(x)$ of them. These summary statistics are then passed to the inference network, which uses the summary statistics to approximate the posterior. Ideally, these summary statistics will be (approximately) sufficient, i.e., still preserve all relevant information for the posterior, so that $p(\theta \mid x) \approx p(\theta \mid s(x))$ [280, 508].

In NPE, the embedding network can be trained end-to-end with the inference network, that is, with the log-likelihood loss specified in Eq. 3.3. Provided that the embedding network is sufficiently expressive, it will automatically learn appropriate summary statistics, without requiring separate data-compression methods [280] (although those might still be empirically useful in some cases [281]). The choice of embedding network depends on the type of data modelled by the simulator. For example, for images, a popular choice for the embedding net are convolutional neural networks (CNNs), and recurrent neural networks (RNNs) might be useful for time-series data [1090].

An important case is the setting in which the observation x_o does not constitute a single observation, but rather stands for a *set* of observations $x = \{x_1, \dots x_N\}$ which are thought to be sampled i.i.d. from the same parameter. In this case, the posterior distribution should be invariant to the ordering of the observations, which can be achieved by using a permutation invariant embedding network such as a set transformer [1003], as proposed by Chan et al. [255] for applications in population genetics. Radev et al. [1359] proposed and evaluated using such networks for datasets with varying size.

Instead of learning summary statistics end-to-end with an embedding network, one can also manually extract summary statistics from the data and perform inference based on these statistics. This requires domain knowledge and, if the choice of summary statistics is poor, might lead to a significant loss of information about parameters, but it can be beneficial if one is interested in inferring parameters given particular features of data. Indeed, in many cases, the simulator is not able to reproduce *all* features of the (experimentally) observed data. Manually defined summary statistics can avoid such misspecification by focusing only on properties of the data which can be modelled by the simulator.

3.3 Amortized and sequential inference

As described above, NPE *amortizes* the cost of inference: After training on a simulated dataset generated from prior predictives (i.e., simulation results based on parameters drawn from the prior distribution), they can perform inference for *any* new observed data x_o. In cases where one is interested in performing inference for many different observations, or where inference is time-critical, this can be highly beneficial. In cases where one only aims to perform inference for a single observation (or for few observations), NPE can be wasteful: By training a neural network on *prior* predictives, the neural network has to learn from a broad range of (simulated) data, even though it is eventually only evaluated at few observations.

Sequential methods have been developed to improve the simulation-efficiency of SBI in these cases. These methods draw parameters (which are then used to generate the training dataset) from a "proposal" distribution. This proposal distribution is chosen such that simulation outputs are expected to be closer to the observation than prior predictives. A popular choice for the proposal distribution is the posterior distribution obtained by running NPE with a limited number of simulations (but explicit active learning schemes have also been proposed [668]).

These methods are often called *sequential*, as the inference network is trained across multiple, sequentially simulated datasets, where the approximate posterior after each round guides generation of the next dataset. It has been demonstrated that sequential methods can improve simulation efficiency, often by an order of magnitude or more [398, 635, 1092].

While potentially reducing the number of required simulations, using sequential methods with NPE can have a drawback: Drawing parameters from a proposal distribution (instead of from the prior) biases the posterior distribution towards regions that were oversampled in the proposal (compared to the prior). Several methods have been developed to overcome this. Papamakarios and Murray [1288] proposed to train the neural network with the standard log-likelihood loss, and to then correct the posterior post-hoc. Lueckmann et al. [1090] suggested to importance-weight the loss, such that the density estimator directly approximates the posterior after training. Greenberg et al. [663] introduced a contrastive loss, which implicitly frames conditional density estimation as a classification problem [471], and which can be combined with arbitrary acquisition functions [668]. Finally, Blum and François [172] and Deistler et al. [398] proposed to draw parameter sets from a restricted region of the prior, such that no modifications of the loss function or post-hoc corrections are required. Notably, for many other SBI methods, such as Neural Likelihood Estimation, NLE [1091, 1291], or Neural Ratio Estimation, NRE [471, 741, 1175, 1601], parameters for the training dataset can be drawn from *any* distribution and the neural networks can nonetheless be trained with standard loss functions, as the prior is explicitly taken into account during inference with MCMC or variational inference.

3.4 Evaluating the correctness of SBI methods

3.4.1 The problem of evaluating SBI methods

After having obtained an approximate posterior with SBI methods, a central challenge is to evaluate the quality of this posterior. This is difficult because, for any real-world task, the ground-truth posterior is unavailable.

Several diagnostic tools have been developed to overcome this limitation. Note, however, that many of these tools do not provide *sufficient* conditions for posterior correctness. Instead, they provide *necessary* conditions for posterior correctness, and thus enable detection of inaccuracies in the posterior estimate. Other methods can, in principle, provide sufficient conditions for correctness, but rely on additional hyperparameters or the accuracy of other trained neural networks, which can compromise the accuracy of the diagnostic tool. Below, we focus on a set of diagnostic tools called coverage diagnostics, and we then briefly describe other methods.

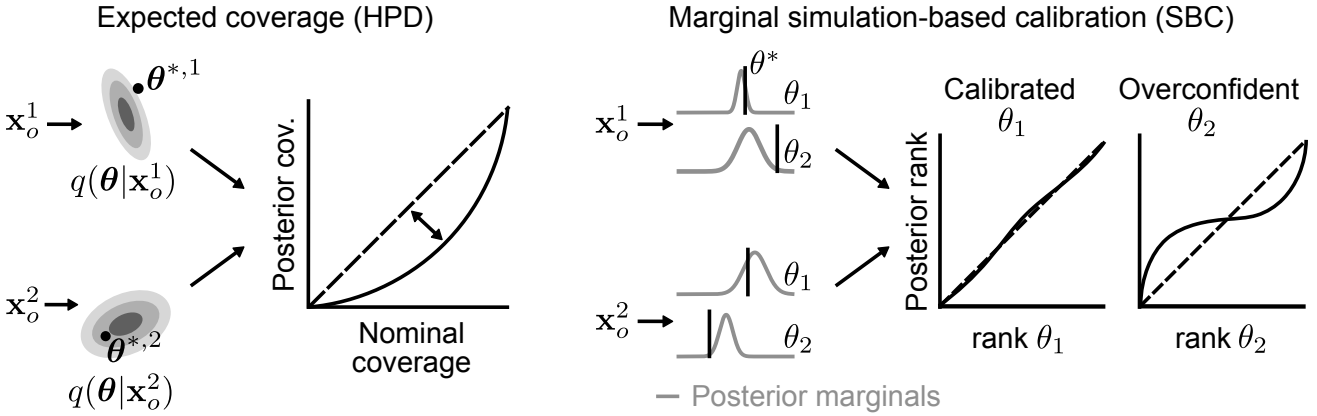


Figure 3.6: Coverage diagnostics for identifying inaccuracies in the approximate posterior. Left: Expected coverage based on the highest-probability density (HPD) region aims to detect whether the joint posterior distribution is over- or under-confident. Marginal simulation-based calibration (SBC) aims to detect inaccuracies in the marginals of the posterior distribution. Both of these methods produce rank distributions, which are optimal if the posterior lies on the diagonal, and indicate issues if it lies above or below the diagonal. Figure modified from Deistler et al. [400].

3.4.2 Coverage diagnostics

A set of diagnostic tools that aim to detect inconsistencies in the posterior distribution are *coverage diagnostics* [328, 1585]. These methods begin by generating a calibration dataset: They sample parameters from the prior and run the simulator to generate samples $\theta, x \sim p(\theta, x)$. For every x in the calibration dataset, these methods then draw samples from the posterior $\theta_{\text{post}} \sim q_\psi(\theta \mid x)$ obtained with an SBI method. For amortized SBI methods, this process is fast, as inference can be performed for any x without retraining or resimulating.

Coverage diagnostics then reduce posterior samples θ_{post} and the ground truth parameters θ to 1-dimensional quantities, via a reducing function $f(\cdot) : \mathbb{R}^N \to \mathbb{R}$, where N is the parameter dimensionality. The posterior approximation is *calibrated* if, for any reducing function $f(\cdot)$, the rank of $f(\theta)$ is distributed uniformly within $f(\theta_{\text{post}})$ (Fig. 3.6). Different diagnostic tools use different mappings $f(\cdot)$ to detect posterior inconsistencies. For example, $f(\theta)$ can compute the posterior log-probability $q_\psi(\theta \mid x)$ (often referred to as *expected coverage* based on the highest-probability density (HPD) region) [398, 742, 1174], or it can pick the 1D marginals of θ [328, 1585]. In the former case, coverage diagnostics can detect inconsistencies in higher-order moments of the posterior (Fig. 3.6, left). In the latter case, coverage diagnostics cannot detect inconsistencies in higher-order moments of the posterior, but they can provide intuition about which parameter is inconsistent (Fig. 3.6, right).

A core advantage of coverage diagnostics is that they can be run with relatively small calibration sets (typically, $\sim$ 200 simulations), and that they do not require further tuning of hyperparameters or training of a "diagnostic neural network". This makes them robust and easy to deploy. However, coverage diagnostics typically provide a necessary, but not a sufficient condition for posterior correctness. In particular, if the approximate posterior $q_\psi(\theta \mid x)$ always returns the prior distribution $p(\theta)$ (instead of an approximation to the posterior), then it passes the above described coverage diagnostics. This highlights the need for additional diagnostic tools.

3.4.3 Other methods

A simple tool for diagnosing the posterior approximation $q_\psi(\theta \mid x)$ is to perform *Posterior Predictive Checks* (PPCs). Given an observation, these methods draw samples from $q_\psi(\theta \mid x_o)$, simulate them, and then compare the simulation outputs (the posterior predictives) to the observation. For every posterior sample θ, it should be possible to obtain a simulation result that closely matches the observation. If posterior predictive checks fail (e.g., because all posterior predictives are far away from the observation, or if some parameter sets can *never* match the observation), then this indicates issues in the accuracy of $q_\psi(\theta \mid x)$. In particular it might be trained on too few simulations, or it might hint towards *model misspecification* (a case where the observation x_o cannot be matched by *any* parameter set, as discussed in Sec. 3.6).

Beyond coverage checks and PPCs, many other diagnostic tools for SBI have been developed over the past years. For example, some local calibration diagnostics aim to provide sufficient and necessary conditions for posterior correctness [1052, 1448]. However, these methods often require to train an additional neural network and hinge

on the accuracy of that additional trained neural network. Because of this, these diagnostic tools require larger calibration datasets and might themselves have issues (e.g., due to poor convergence of the additional trained neural network).

3.5 Alternative methods

Neural Posterior Estimation (NPE) directly estimates the posterior distribution. In addition to NPE, researchers have developed other methods for SBI. These approaches train neural networks to emulate either the likelihood $p(x \mid \theta)$ (Neural Likelihood Estimation, NLE) or the likelihood-to-evidence ratio $p(x \mid \theta)/p(x)$ (Neural Ratio Estimation, NRE) and then use traditional Bayesian inference methods (e.g., MCMC) to draw samples from the posterior. Finally, some recent methods also go beyond estimating only the posterior or the likelihood(-ratio). Below, we describe these alternative methods for SBI.

3.5.1 Likelihood estimation

Neural Likelihood Estimation (NLE) uses neural density estimators (such as conditional normalizing flows) to estimate the likelihood $p(x \mid \theta)$ [1291]. This estimate can be learned from samples of the joint distribution $\{(\theta_i, x_i)\}_{i=1}^N \sim p(\theta, x)$ by optimising the negative log likelihood of simulation outputs given parameters. NLE trains a conditional density estimator $\ell_\psi(x \mid \theta)$, such as a normalizing flow, by optimising ψ towards the objective

$$\psi^* \in \arg\min_\psi \left[-\frac{1}{N} \sum_{i=1}^{N} \log \ell_\psi(x_i \mid \theta_i) \right]. \tag{3.4}$$

After training, the trained model can be used to *emulate* the simulation process by drawing samples from the likelihood approximation $\ell_\psi(x \mid \theta)$.

For posterior inference, the trained model is multiplied by the prior to obtain the (unnormalised) posterior estimate

$$\hat{p}(\theta \mid x) \coloneqq \frac{1}{Z_\psi(x)} \ell_\psi(x \mid \theta) p(\theta), \qquad Z_\psi(x) \coloneqq \int \ell_\psi(x \mid \theta) p(\theta) \, \mathrm{d}\theta. \tag{3.5}$$

In order to draw samples from the (approximate) posterior distribution $\hat{p}(\theta \mid x_o)$, a method to draw samples from an unnormalised distribution is necessary, for example, rejection sampling or Markov chain Monte Carlo (Sec. 3.5.3). Normalizing flows enable (differentiable) evaluation of the likelihood approximation $\ell_\psi(x_o \mid \theta)$, making this possible, albeit being potentially computationally expensive. Variational inference has been shown to speed up high-dimensional sampling from NLE [635, 1774]. However, any of these sampling methods are typically slow and can introduce additional errors into the inference procedure. An additional disadvantage of NLE (compared to NPE) is that it cannot be combined with an embedding network that is trained end-to-end. One can embed x_o *before* training with a separate neural network, e.g., an autoencoder, then perform NLE on the embedded simulation outputs [280].

NLE can also be used in a sequential scheme to reduce the number of required simulations for inference given a particular observation. These sequential NLE methods focus on a specific observation x_o, generating data nearby x_o iteratively [1291]. A major advantage of NLE in that scenario is that it does not require modifications of the loss function: The neural density estimator will converge to the likelihood for any distribution $\tilde{p}(\theta)$ from which parameters are drawn.

3.5.2 Likelihood-to-evidence ratio estimation

The likelihood-to-evidence ratio $r(x \mid \theta) \coloneqq \frac{p(x|\theta)}{p(x)} = \frac{p(\theta,x)}{p(\theta)p(x)}$ is a natural estimation target for simulation-based inference because Bayes' rule gives $p(\theta \mid x) = r(x \mid \theta)p(\theta)$. Ratio estimators allow more flexible architectures than diffeomorphic normalizing flow layers with tractable determinants. However, density ratios can span many orders of magnitude and therefore be numerically difficult to approximate directly. For this reason, one typically estimates $\log r(x \mid \theta)$. A stable approach to estimating $\log r(x \mid \theta)$ converts the problem into a classification task. Several variants have been developed [343, 471, 1175, 1601]; here, we focus on the method of Hermans et al. [741].

NRE introduces a binary class label y and defines the joint distribution $\pi(\theta, x, y) \coloneqq \pi(\theta, x \mid y)\pi(y)$. We assign

equal class probabilities, $\pi(y=0) \coloneqq \pi(y=1) \coloneqq \frac{1}{2}$, and define the class-conditional distributions as

$$\pi(\theta, x \mid y) \coloneqq \begin{cases} p(\theta)p(x), & y=0, \\ p(\theta, x), & y=1. \end{cases} \tag{3.6}$$

$p(\theta)p(x)$ is called the product of marginals. It is best understood algorithmically: One takes two independent samples $\theta_a, \theta_b \sim p(\theta)$ and simulates one of them $x_b \sim p(x \mid \theta_b)$. Then the samples are distributed $(\theta_a, x_b) \sim p(\theta)p(x)$. Sampling from the joint $p(\theta, x)$ is achieved by first sampling the prior $\theta_c \sim p(\theta)$ and simulating $x_c \sim p(x \mid \theta_c)$. We then have a sample $(\theta_c, x_c) \sim p(\theta, x)$.

Bayes' rule shows why this classification problem identifies the desired ratio:

$$\frac{\pi(y=1 \mid \theta, x)}{\pi(y=0 \mid \theta, x)} = \frac{\pi(\theta, x \mid y=1)\pi(y=1)}{\pi(\theta, x \mid y=0)\pi(y=0)} = \frac{p(\theta, x)}{p(\theta)p(x)} = r(x \mid \theta).$$

Consequently, $\pi(y=1 \mid \theta, x) = \frac{r(x|\theta)}{1+r(x|\theta)} = \sigma(\log r(x \mid \theta))$, where σ denotes the sigmoid function. We therefore construct

$$\hat{\pi}(y=1 \mid \theta, x) \coloneqq \sigma(f_\psi(\theta, x)) \qquad \text{and} \qquad \hat{r}(x \mid \theta) \coloneqq \exp(f_\psi(\theta, x)),$$

where $\hat{\pi}(y=1 \mid \theta, x)$ is a classifier, with $\hat{\pi}(y=0 \mid \theta, x) = 1 - \hat{\pi}(y=1 \mid \theta, x)$, and $\hat{r}(x \mid \theta)$ is a likelihood-to-evidence ratio estimate. Both are parameterised by the neural network $f_\psi(\theta, x)$ with weights ψ. We optimise ψ by minimising the conditional KL divergence,

$$\psi^* \in \arg\min_\psi \mathrm{D}_{\mathrm{KL}}(\pi(y \mid \theta, x) \| \hat{\pi}(y \mid \theta, x)), \tag{3.7}$$

$$\mathrm{D}_{\mathrm{KL}}(\pi(y \mid \theta, x) \| \hat{\pi}(y \mid \theta, x)) = \iint \pi(\theta, x) \sum_{y \in \{0,1\}} \pi(y \mid \theta, x) \log \frac{\pi(y \mid \theta, x)}{\hat{\pi}(y \mid \theta, x)} \, \mathrm{d}x \, \mathrm{d}\theta. \tag{3.8}$$

Here, $\pi(\theta, x)$ is the marginal mixture induced by the constructed classification problem:

$$\pi(\theta, x) \coloneqq \pi(y=1)\pi(\theta, x \mid y=1) + \pi(y=0)\pi(\theta, x \mid y=0) = \frac{p(\theta, x) + p(\theta)p(x)}{2}. \tag{3.9}$$

The conditional KL divergence differs from the binary cross-entropy used to train the classifier only by the conditional entropy of $\pi(y \mid \theta, x)$, which is constant with respect to ψ. The two objectives therefore have the same minimiser. Assuming sufficient model capacity and successful optimisation, the conditional KL divergence is zero at the optimum, so $\pi(y=1 \mid \theta, x) = \sigma(f_{\psi^*}(\theta, x))$ and $f_{\psi^*}(\theta, x) = \log r(x \mid \theta)$ almost everywhere under $\pi(\theta, x)$.

We can now form the unnormalised posterior approximation $\hat{p}'(\theta \mid x) \coloneqq \hat{r}(x \mid \theta)p(\theta) = p(\theta)\exp(f_\psi(\theta, x))$. Unless the classifier recovers the exact ratio, $\hat{p}'$ need not integrate to one. Its normalised counterpart is

$$\hat{p}(\theta \mid x) \coloneqq \frac{\hat{p}'(\theta \mid x)}{Z_\psi(x)} = \frac{\hat{r}(x \mid \theta)}{Z_\psi(x)} p(\theta) = \frac{\exp(f_\psi(\theta, x))}{Z_\psi(x)} p(\theta), \tag{3.10}$$

with $Z_\psi(x) \coloneqq \int \exp(f_\psi(\theta, x))p(\theta) \, \mathrm{d}\theta$. At the exact optimum, $\exp(f_{\psi^*}(\theta, x)) = r(x \mid \theta)$ and hence $Z_{\psi^*}(x) = 1$. In practice, posterior behaviour is usually characterised using samples, and samplers for unnormalised distributions do not require $Z_\psi(x)$ to be computed.

One can draw samples from $\hat{p}(\theta \mid x)$ using any sampling method for unnormalised distributions, just like with NLE (Sec. 3.5.3). The ratio-based neural network $f_\psi(\theta, x)$ can be both more expressive and cheaper than the normalizing flow-based likelihood approximation from NLE. Flows require parameterising an efficient diffeomorphism with a tractable log determinant, whereas NRE permits arbitrary neural-network architectures. One limitation is that the standard NRE objective in (3.7) does not generally attain the same asymptotic efficiency as maximum-likelihood density estimation [295, 1392, 1824]. Alternative objectives have therefore been explored for ratio estimation [634, 1177].

3.5.3 Sampling from the approximate posterior using unnormalised inference models

While NPE approximates the posterior directly, NLE and NRE require an additional inference step to draw samples from the posterior. To this end, one can use general-purpose sampling methods (see Chapter 1) and adapt them

to sample from an unnormalised posterior estimate. Rejection sampling or reweighted sampling from the prior are natural choices for simple problems with low dimensionality due to straightforward embarrassingly parallel implementations [1211]. Both of these methods start by drawing N samples from the prior

$$\theta_i \sim p(\theta), \quad i \in \{1, 2, \ldots, N\}. \tag{3.11}$$

In *rejection sampling*, we choose a constant $M \geq 1$ such that $M \cdot p(\theta) \geq \hat{p}'(\theta \mid x_o)$ for all θ in the support of $\hat{p}'(\theta \mid x_o)$, where $\hat{p}'(\theta \mid x_o)$ denotes the unnormalised posterior density. For each of the N samples, we draw

$$u_i \sim \mathcal{U}(0, 1), \quad i \in \{1, 2, \ldots, N\}, \tag{3.12}$$

where we let $\mathcal{U}(a, b)$ denote the one-dimensional uniform distribution between a and b. This algorithm yields samples from the approximate posterior $\hat{p}(\theta \mid x_o)$ in the following way:

$$u_i < \frac{\hat{p}'(\theta_i \mid x_o)}{M \cdot p(\theta_i)} \implies \theta_i \sim \hat{p}(\theta \mid x_o). \tag{3.13}$$

Rejection sampling generates N/M samples, on average. It is efficient when $M \approx 1$, *i.e.,* in low dimensions and when the prior and approximate posterior have similar support. Rejection sampling is inefficient when the approximate posterior is narrow compared to the prior.

In *reweighted sampling*, we use the samples from the prior in (3.11), but assign them a self-normalised importance weight. The weights and self-normalised weights are defined as

$$\tilde{w}_i \coloneqq \hat{p}'(\theta_i \mid x_o), \qquad w_i \coloneqq \frac{\tilde{w}_i}{\sum_{j=1}^{N} \tilde{w}_j} = \frac{\hat{p}'(\theta_i \mid x_o)}{\sum_{j=1}^{N} \hat{p}'(\theta_j \mid x_o)}. \tag{3.14}$$

Histograms and moments of the distribution can be computed using weighted samples $w_i\theta_i$. The variance of the weights can be large, implying one sample takes most of the probability mass. This occurs in the same situations when rejection sampling is inefficient.

One sampling method that can be more robust to high dimensionality and approximate posteriors that make up only a small volume (compared to the prior) is *Markov chain Monte Carlo*. Rather than explaining further, we simply define the Metropolis-Hastings acceptance probability and refer the reader to Chapter 1. Given a fixed x and proposal distribution $\mathcal{T}$ with transition density $\mathcal{T}(\theta' \mid \theta)$ from θ to θ', the acceptance probability $\alpha(\theta' \mid \theta, x)$ in a Metropolis-Hastings step is

$$\alpha(\theta' \mid \theta, x_o) \coloneqq \min\left(1, \frac{\hat{p}(\theta' \mid x_o)}{\hat{p}(\theta \mid x_o)} \frac{\mathcal{T}(\theta \mid \theta')}{\mathcal{T}(\theta' \mid \theta)}\right) = \min\left(1, \frac{\hat{p}'(\theta' \mid x_o)/\cancel{Z_\psi(x_o)}}{\hat{p}'(\theta \mid x_o)/\cancel{Z_\psi(x_o)}} \frac{\mathcal{T}(\theta \mid \theta')}{\mathcal{T}(\theta' \mid \theta)}\right) \tag{3.15}$$

This acceptance probability does not depend on $Z_\psi(x_o)$ and can therefore draw samples from the approximate posterior even when $\hat{p}'(\theta \mid x_o)$ is not normalised. Designing an efficient transition kernel and identifying convergence is an open area of research.

3.5.4 Estimating multiple quantities at once

Finally, some recent methods go beyond estimating the posterior, the likelihood, or the likelihood-ratio and estimate several of these quantities, typically with separate models [635, 1360, 1774]. Recently, the Simformer [636] has been proposed to estimate the full joint distribution $p(\theta, x)$ as well as all of its conditionals and marginals in a single model, based on transformers and a diffusion model. Thereby, the Simformer improves the flexibility of SBI and it combines desirable features of NPE (e.g., to directly draw samples from the posterior) and NLE (e.g., to emulate the simulator).

3.6 Current developments

Neural Posterior Estimation (NPE), as well as alternative methods such as Neural Likelihood Estimation (NLE) or Neural Ratio Estimation (NRE), have been demonstrated as powerful tools for SBI. Over the past years, researchers have extended these methods in many ways, and have developed new SBI methods that go beyond estimating the posterior or likelihood(-ratio). We outline recent trends in using neural networks for SBI.

3.6.1 Better generative models

Normalizing flows have been a popular choice for NPE for several years, likely due to their robustness in training, their ability to quickly (and exactly) evaluate the log-probability of samples, and their mass-covering property. However, the fact that the transformation T trained by normalizing flows has to be invertible can constrain the flexibility of the normalizing flow. This may reduce the accuracy of posterior estimates of normalizing flows, especially for high-dimensional and structured parameter spaces (e.g., if the parameters are a time series or an image). To overcome this, recent methods have explored other conditional generative models for NPE. In recent years, diffusion models have been particularly popular due to their ability to estimate high-dimensional parameter spaces, and to maintain the ability to evaluate the log-probability via the probability flow ODE (albeit at significantly higher computational cost) [589, 636, 1505]. Many other generative models have been explored for NPE, ranging from flow-matching [1757] to consistency models [1471], generative adversarial networks [1375], energy-based models [634], and tabular foundation models [1677]. We expect that any major advances in (conditional) generative modelling will directly carry over to improvements in NPE.

3.6.2 Improving the flexibility of simulation-based inference

A core limitation of NPE with normalizing flows is that it requires the inference task to be known upfront. If, for example, the prior changes, when the simulator is modified, or when the data representation changes, NPE has to be re-run in order to perform inference (including running simulations and training the neural network).

To overcome this, many recent methods have improved the flexibility of NPE. First, in order to perform inference given any number of i.i.d. trials, permutation-invariant embedding networks can be used, as described above [1359]. Alternatively, diffusion models enable NPE to estimate the posterior distribution given any number of i.i.d. datapoints, even when the neural network was trained on pairs of parameters and a single simulation result [589, 637, 1053]. Second, in order to enable more flexible specification of prior or simulator, it has been proposed to amortize over hyperparameters of prior or simulator [489, 1209, 1558]. In that case, the inference network is conditioned not only on data, but also on additional values describing the prior or simulator. At inference time, one can then perform inference for any prior and for any simulator. Third, in some cases and only for some observations, some parameters may be known and should not be inferred, but instead should be kept fixed [399, 636]. In other cases, data may not always be complete, and parts of an observation might be missing. In order to flexibly perform amortized inference in these cases, several methods enable the estimation of any parameter conditional of NPE and systematically deal with missing data [399, 636, 1430, 1675].

3.6.3 Improving robustness to model misspecification

A fundamental challenge for SBI emerges when the simulator is *misspecified*, i.e., when it does not match reality. In the most extreme case, the observation x_o cannot be generated by the simulator (i.e., the combination of prior and simulator) with any set of parameters and with any level of simulator noise.

This can have a drastic impact on SBI: While one might expect that, for misspecified observations, the approximate posterior will have mass in parameter regions that generate predictives that are close to the observation, or that the approximate posterior should be very uncertain, this is not the case. Indeed, neural network-based methods for SBI have been shown to react erratically to the misspecified observations and can produce non-sensible posteriors [230].

Several methods have been proposed to improve the robustness of SBI in these scenarios, including adding noise to the simulation outputs [1720], learning statistics of the observations [791] such that they are not misspecified, training on unlabelled [1187] and labelled [1731] observations when available, or changing the target of inference [573].

3.6.4 Conservative and calibrated posterior approximations

Approximate SBI posteriors can be overconfident[1], particularly when simulations are limited, motivating methods that promote conservative or calibrated uncertainty estimates [742]. Balanced neural ratio estimation modifies the NRE training objective to favour conservative approximations, while preserving its Bayes-optimal solution [404], by effectively regularising the estimate to be closer to the prior. This balancing principle has also been extended to NPE and contrastive NRE [405]. Alternatively, differentiable relaxations of coverage error can be included directly in the training objectives of amortized SBI methods, encouraging calibrated posteriors with moderate

[1]This occurs when the approximate posterior is narrower than the ground truth.

additional computational cost [502]. However, expected coverage is a necessary rather than sufficient diagnostic: an approximation can exhibit nominal coverage while still differing substantially from the true posterior. Ratio coverage plots address some of these blind spots by comparing the approximate and true posteriors through an estimated density ratio [1055].

3.7 Discussion

Many processes in the natural sciences and in engineering are best described by complex, stochastic, and potentially non-differentiable simulators. For many such models, the likelihood can become excessively expensive or even impossible to evaluate. In order to perform inference with traditional Bayesian inference methods such as MCMC, scientists and engineers had to modify their model such that the likelihood can be efficiently evaluated, but which comes at the cost of reducing the fidelity of the simulator.

Simulation-based inference (SBI) makes Bayesian inference accessible to the widest class of models: It can be applied to any black-box simulator for which one can run forward simulations. In recent years, neural networks have largely improved the accuracy and applicability of SBI. These methods generate a database of parameters and corresponding simulation outputs, and then train neural networks to learn the statistical relationship between these quantities. After training, the neural network can be evaluated at any observation and enables us to infer the posterior distribution without further simulations or retraining.

SBI with neural networks has already been applied to perform inference in a wide range of disciplines in science and engineering. We expect that the continued progress in improving the accuracy, flexibility, and robustness of these approaches will have a profound impact on many fields, and we hope that it will enable new scientific discoveries.

Chapter 4

Sequential Monte Carlo samplers

This chapter develops sequential Monte Carlo (SMC) samplers as a population-based approach to inference in Bayesian deep learning. Building on the sequential Monte Carlo methods of Chapter 1, we focus on the two extensions that make them practical for neural network posteriors: multilevel SMC and scalable parallel SMC.

4.1 Motivation

Chapter 1 introduced SMC as a particle filter [446, 447]: a latent state evolving in time, tracked by a population of particles over a sequence of growing spaces. This chapter applies the same population-based approximation to a static target—a Bayesian posterior on a fixed parameter space. Without any dynamics to generate them, the intermediate distributions are now constructed by design, and the resulting methods are known as SMC samplers [402].

It is useful to place the neighbouring methods relative to this sampler. Tempering is the mechanism inside the sampler that builds the intermediate distributions, annealing the likelihood from the prior to the posterior [599, 612, 1228] (data-tempering [296] is a common alternative). Annealed importance sampling (AIS) is the reduced case in which resampling is omitted [1228], so it is best viewed as a simplified SMC sampler rather than a distinct method. The chapter's two main contributions are developments of the SMC sampler, along two different axes: multilevel SMC (MLSMC) improves statistical efficiency by combining a hierarchy of coarse-to-fine approximations in a telescoping estimator [142, 143], while parallel SMC improves scalability by running many samplers concurrently and weighting them by their normalizing-constant estimates, avoiding the heavy communication of island particle methods [1038, 1674, 1745].

These methods matter for Bayesian deep learning because neural network posteriors are high-dimensional, multi-modal with many symmetric modes, and costly to evaluate—the regime where importance sampling degenerates and single-chain MCMC mixes slowly. An SMC sampler anneals an entire population towards the posterior, provides an unbiased estimate of the model evidence, and parallelises naturally. The remainder of the chapter develops the two extensions that make this practical at scale: MLSMC with trace-class neural network priors [252, 1488], and scalable parallel SMC [1038].

4.2 Introduction

Monte Carlo based Bayesian algorithms of the type described in Chapter 1 typically require generating many samples from a target distribution which at best can be evaluated up to a normalizing constant. This can be expensive and cumbersome in high-dimensions, and so methods to improve complexity are paramount. As discussed in Chapter 1, the best computational complexity achievable by a Monte Carlo algorithm in general is cost $\propto 1/\text{MSE}$, i.e. achieving a mean square error (MSE) of $\mathcal{O}(\varepsilon^2)$ for $\varepsilon > 0$ will asymptotically require $\mathcal{O}(\varepsilon^{-2})$ samples. This is hence referred to as the *canonical rate*, as it holds for any intractable integral for which we can generate i.i.d. unbiased samples of the integrand.

The sequential Monte Carlo (SMC) sampler [297, 355, 402] was developed at the turn of the millennium [139, 296, 629, 837, 1228], and is well suited to this high-dimensional regime. Forcing the (successive) importance distributions to be close provides a *generic mechanism* to overcome the "curse-of-dimensionality", i.e. complexity *constant* scaling like e^P for dimension P [7, 261]. Under suitable high-dimensional stability assumptions, SMC samplers can avoid the exponential degradation of direct importance sampling and may achieve polynomial cost

scaling in dimension [141]. The SMC sampler also handles bad initialization or multi-modality on par with other population methods [997], and facilitates adaptive tuning of the Markov kernel [215], with comparable efficiency to MCMC approaches like the No-U-Turn Sampler (NUTS) [762]. These benefits are not shared by the particle filter, which does not utilize MCMC methodology. It also delivers an *unbiased* estimator of the normalizing constant, or model evidence, which can be useful in practice [297].

The simplest method for improving complexity is parallelization. Standard i.i.d. Monte Carlo simulation is "embarrassingly parallel", as the samples can all be simulated *independently* on different machines. If the cost of each simulation dominates the $\mathcal{O}(\varepsilon^{-2})$ communication and summation operations, which should both have extremely small scalar constants which are often invisible to the algorithm ($< 10^{-9}$ smaller than likelihood computations), then the constant and hence final time complexity can be reduced significantly in practice, even down to effectively $\mathcal{O}(1)$ when the merge is invisible. Monte Carlo methods for Bayesian simulation do not typically deliver i.i.d. samples, and so they are not a priori parallel. SMC samplers admit a certain degree of parallelism in their raw form, and we will show that they can also be provably "embarrassingly" parallelized beyond this [1038].

If the underlying problem we are simulating from additionally requires a discrete approximation of a function then the complexity compounds. For example, suppose that achieving a bias^2 of $\mathcal{O}(\varepsilon^2)$ demands a cost of $\mathcal{O}(\varepsilon^{-\xi})$. Then the MSE complexity compounds as $\mathcal{O}(\varepsilon^{-2-\xi})$. To overcome this issue, one remedy to reduce the computational cost is through a modified Monte Carlo estimator which is known as multilevel Monte Carlo (MLMC) [627, 628, 730]. MLMC introduces a hierarchy of intermediate convergent discretization levels and constructs an estimator as a telescopic sum of a coarse and cheap estimator with many samples and successive increment estimators with increasing cost and decreasing sample size, ultimately improving the complexity to $\mathcal{O}(\varepsilon^{-(2\wedge\xi)})$ instead of the product form, which can translate to huge scaling gains for small target MSE.

MLMC methodology in the Bayesian context was first developed for Markov chain Monte Carlo (MCMC) [434, 757] and sequential Monte Carlo (SMC) samplers [142]. We consider a multilevel sequential Monte Carlo (MLSMC) sampler based on the application of a function-space version of the latter algorithm [143] to a strongly convergent class of neural network models known as *trace-class neural networks* (TNN) [252, 1488]. These are Bayesian feedforward neural networks (BNN) which impose a *width decay* on the weights instead of a standard L^2 width decay prior. This simple and natural idea eliminates exchangeability and non-identifiability of the weights and delivers strong convergence with a tunable smoothness parameter. The limiting Gaussian measures are defined in function spaces with similar properties to Sobolev spaces [181]. The resulting method was originally developed in Chada et al. [252], where the authors were able to demonstrate the canonical complexity is achievable. We will use function evaluations as the fundamental unit of computational cost, which is also a convenient proxy for wall-clock time which is agnostic to particulars of the implementation or hardware.

The organization of this chapter is as follows. First, we introduce the SMC sampler in Section 4.3. In Section 4.4 we present the MLSMC sampler method for TNN. In Section 4.5 we present the parallel SMC method.

4.3 SMC sampler

Suppose that we aim to simulate from a target distribution

$$\pi(\theta) := p(\theta \mid \mathcal{D}) \propto p(Y \mid X, \theta)\, p(\theta)\,. \tag{4.1}$$

One option is to use an MCMC kernel $\mathcal{M}$ such that $p\mathcal{M} = p$ to simulate $\theta^{(k)} \sim \mathcal{M}(\theta^{(k-1)}, \cdot)$, but this method produces correlated local samples and is prone to getting stuck in a given basin of attraction. If that basin of attraction is in the set of indistinguishable basins that we want then it may not particularly matter in practice. Gradient-based MCMC methods such as HMC can reduce random-walk behaviour, but they can still mix poorly across isolated posterior modes or permutation-symmetric basins.

Population methods such as importance sampling (IS) are able to mitigate this issue, but they suffer from their own difficulties. In particular, in its raw form, IS suffers from a curse of dimensionality in both parameter and data [261], and as such it is not suitable for high dimensional problems and big data. The SMC sampler bypasses this problem by

(i) interpolating between an easy-to-sample-from initial distribution p_0 and the target distribution of interest p with several intermediate distributions p_t such that $p_T = p$, and

(ii) interleaving sequential importance sampling (SIS) with resampling and mutation by a sequence of appropriate MCMC kernels $\mathcal{M}_t$ such that $p_t\mathcal{M}_t = p_t$.

In the Bayesian context, typically the prior p is a suitable initial distribution. The sequence of intermediate targets can be built to gradually introduce the data, for example either from growing subsets of the data [296] or

Algorithm 10: SMC sampler (c = communication here)

Inputs : number of particles M, sequence of distributions π_t, MCMC kernels $\mathcal{M}_t$
Outputs: particle system $\{\theta_t^i\}_{i=1}^M$ and normalizing constant estimate Z_T^M
initialize $\theta_0^i \sim \pi_0$ for $i = 1, \dots, M$; set $Z_0^M \leftarrow 1$
for $t = 1$ **to** T *(in serial)* **do**
  `// (Optional) adapt annealing parameter`
  select λ_t such that $\mathsf{ESS} = \alpha M$
  `// (c) Update estimate of normalizing constant`
  $Z_t^M \leftarrow Z_{t-1}^M \cdot \frac{1}{M} \sum_{k=1}^M p(Y \mid \theta_{t-1}^k, X)^{\lambda_t - \lambda_{t-1}}$
  for $i = 1$ **to** M *(in parallel)* **do**
    define $w_t^i \propto p(Y \mid \theta_{t-1}^i, X)^{\lambda_t - \lambda_{t-1}}$
    `// (c) Selection step`
    sample ancestor index $I_t^i \sim \{w_t^1, \dots, w_t^M\}$
    `// Mutation step`
    sample $\theta_t^i \sim \mathcal{M}_t(\theta_{t-1}^{I_t^i}, \cdot)$

with a tempering schedule $0 = \lambda_0 < \lambda_1 < \cdots < \lambda_T = 1$ [1228]:

$$p_t(\theta) \propto p(Y \mid X, \theta)^{\lambda_t}\, p(\theta)\,.$$

Three quantities are tracked along this path. The normalizing constant at temperature λ_t is

$$Z_t = \int p(Y \mid X, \theta)^{\lambda_t}\, p(\theta)\, d\theta,$$

which at $\lambda_T = 1$ equals the model evidence $p(Y \mid X)$. Algorithm 10 maintains an unbiased estimate Z_t^M at each step, with Z_T^M returned as the estimate of the evidence. Successive targets are linked by the incremental importance weight that reweights particle i from π_{t-1} towards π_t,

$$w_t^i \propto p(Y \mid X, \theta_{t-1}^i)^{\lambda_t - \lambda_{t-1}}, \qquad \tilde{w}_t^i = \frac{w_t^i}{\sum_{k=1}^M w_t^k}.$$

How evenly these weights are spread is measured by the effective sample size (ESS),

$$\mathrm{ESS} = \frac{1}{\sum_{i=1}^M (\tilde{w}_t^i)^2} \in [1, M].$$

Values near M indicate balanced weights, whereas small values mean a few particles dominate. The ESS drives the algorithm in two ways: a low value motivates resampling, and in the adaptive variant the next temperature is chosen to preserve a target $\mathrm{ESS} = \alpha M$. Data-tempering and likelihood-tempering construction schemes for the intermediate targets can also be mixed and matched. We will consider another option in Section 4.4. See Algorithm 10 for the version with tempering for Bayesian inference. The steps which require communication during execution are denoted with (c). It is noteworthy that if we skip the resampling step then the algorithm is called annealed importance sampling (AIS) and was introduced in Neal [1228]. The normalizing constant then only needs to be computed once at the end. This version of the algorithm is convenient because it is embarrassingly parallel – communication is only required at resampling times. However, the purpose of resampling is to prevent degeneracy and improve stability, and this benefit often outweighs the added variance [296, 297].

4.4 MLSMC for TNN

In this section we will build up the MLSMC for TNN method. First we introduce the MLMC method, followed by the TNN. Then we present the MLSMC sampler and provide theoretical justification, and numerical results validating and extending the theory.

4.4.1 MLMC method

We shall begin with a short review of MLMC. MLMC is useful when accurate simulations are expensive but coarse approximations are cheap. Rather than estimating the finest-level expectation directly, one estimates a coarse expectation plus a sequence of corrections. If consecutive levels are strongly coupled, the correction variances decay rapidly, so fewer samples are needed at expensive fine levels. Let us assume that we are given a probability density p, on a state-space U. It is of interest to compute expectations of $p-$integrable functions, $\varphi : \mathsf{U} \to \mathbb{R}$; $p(\varphi) := \int_{\mathsf{U}} \varphi(u)p(u)du$. Now, we assume that we need to approximate p by a density p_l on a state-space $\mathsf{U}_l \subseteq \mathsf{U}$ such that:

1. $\lim_{l\to\infty} p_l(\varphi) = p(\varphi)$, for any integrable $\varphi : \mathsf{U} \to \mathbb{R}$.
2. The cost of computing with p_l is increasing in l.

Let $L \in \{1, 2, \dots\}$ be given, we have the telescoping sum identity

$$p_L(\varphi) = p_0(\varphi) + \sum_{l=1}^{L} [p_l - p_{l-1}](\varphi), \tag{4.2}$$

with $[p_l - p_{l-1}](\varphi) = p_l(\varphi) - p_{l-1}(\varphi)$. The idea behind MLMC is to leverage this identity to reduce the complexity of the approximation. This is achieved by constructing *couplings* of (p_l, p_{l-1}), $l \in \{1, \dots, L\}$: joint densities $\check{p}_l$, on $\mathsf{U}_l \times \mathsf{U}_{l-1}$ such that $\int_{\mathsf{U}_l} \check{p}_l(u_l, u_{l-1})du_l = p_{l-1}(u_{l-1})$ and $\int_{\mathsf{U}_{l-1}} \check{p}_l(u_l, u_{l-1})du_{l-1} = p_l(u_l)$.

The MLMC estimator can be constructed in the following way.

1. Sample $U_0^1, \dots, U_0^{M_0}$ i.i.d. from p_0, with $M_0 \in \mathbb{N}$ given.
2. For $l \in \{1, \dots, L\}$ independently of all other random variables, sample $(U_l^1, \tilde{U}_{l-1}^1), \dots, (U_l^{M_l}, \tilde{U}_{l-1}^{M_l})$ i.i.d. from $\check{p}_l$, with $M_l \in \mathbb{N}$ given.

Then, we have the approximation

$$p_L^{ML}(\varphi) := \frac{1}{M_0} \sum_{i=1}^{M_0} \varphi(U_0^i) + \sum_{l=1}^{L} \frac{1}{M_l} \sum_{i=1}^{M_l} \{\varphi(U_l^i) - \varphi(\tilde{U}_{l-1}^i)\}. \tag{4.3}$$

Using i.i.d. samples $U_L^1, \dots, U_L^M$ from p_L to approximate $p_L(\varphi)$, we have

$$p_L^{IID}(\varphi) := \frac{1}{M} \sum_{i=1}^{M} \varphi(U_L^i). \tag{4.4}$$

Both estimators are unbiased, so we have the general decomposition of an unbiased estimator $\hat{p}$

$$\mathbb{E}[(p_L^{\mathsf{ML}}(\varphi) - p(\varphi))^2] = \underbrace{\mathbb{V}\mathrm{ar}[p_L^{ML}(\varphi)]}_{\mathsf{variance}} + \underbrace{[p_L - p](\varphi)^2}_{\mathsf{bias}^2}, \tag{4.5}$$

with $\mathbb{V}\mathrm{ar}$ representing the variance operator. The bias is equal for both, and to optimize the complexity this should match the variance. The variance of (4.3) is

$$\mathbb{V}\mathrm{ar}[p_L^{\mathsf{ML}}(\varphi)] = \frac{\mathbb{V}\mathrm{ar}[\varphi(U_0^1)]}{M_0} + \sum_{l=1}^{L} \frac{\mathbb{V}\mathrm{ar}[\varphi(U_l^1) - \varphi(U_{l-1}^1)]}{M_l}, \tag{4.6}$$

whereas for (4.4)

$$\mathbb{V}\mathrm{ar}[p_L^{\mathsf{IID}}(\varphi)] = \frac{\mathbb{V}\mathrm{ar}[\varphi(U_L^1)]}{M}.$$

Now, if the couplings $\check{p}_l$ are constructed so that $\mathbb{V}\mathrm{ar}[\varphi(U_l^1) - \varphi(U_{l-1}^1)]$ falls sufficiently quickly with l then it can be possible to achieve an MSE (as in (4.5)) for the estimator (4.3) which is of the same order as (4.4) except for a cost that is lower. This is characterized in the following result.

Theorem 4.1 (Giles [627]). *Suppose that there exists constants $(\alpha, \beta, \gamma) \in \mathbb{R}_+^3$ with $\alpha \geq \frac{\min(\beta,\gamma)}{2}$ such that*

(i) $|p_l(\varphi) - p(\varphi)| = \mathcal{O}(2^{-\alpha l})$.

(ii) $\mathbb{V}\mathrm{ar}[\varphi(U_l^1) - \varphi(U_{l-1}^1)] = \mathcal{O}(2^{-\beta l})$.

(iii) $C_l = \mathcal{O}(2^{\gamma l})$.

Then for any $\varepsilon < 1$ *and* $L := \lceil \log(1/\varepsilon) \rceil$, *there exists* $(M_1, \ldots, M_L) \in \mathbb{N}^L$ *such that*

$$\mathrm{MSE} = \mathbb{E}[(p_L^{ML}(\varphi) - p(\varphi))^2] = \mathcal{O}(\varepsilon^2), \tag{4.7}$$

and

$$\mathrm{Cost(MLMC)} := \sum_{l=0}^{L} M_l C_l = \begin{cases} \mathcal{O}(\varepsilon^{-2}), & \text{if } \beta > \gamma, \\ \mathcal{O}(\varepsilon^{-2}(\log \varepsilon)^2), & \text{if } \beta = \gamma, \\ \mathcal{O}(\varepsilon^{-2-\frac{(\gamma-\beta)}{\alpha}}), & \text{if } \beta < \gamma, \end{cases} \tag{4.8}$$

where C_l *is the cost for one sample of* $\varphi(U_l^i) - \varphi(U_{l-1}^i)$.

Asymptotically the cost in (4.8) is smaller than that of using the estimator (4.4). It is simple to see that the corresponding optimal complexity of the single level estimator (4.4) is given by the product of $\mathcal{O}(\varepsilon^{-2})$ samples at a per sample complexity of $\mathcal{O}(\varepsilon^{-\gamma/\alpha})$, i.e. $\mathcal{O}(\varepsilon^{-2-\gamma/\alpha})$, which is larger than the MLMC complexity for $\beta > 0$. The case of $\beta > \gamma$ is referred to as the canonical rate of convergence, i.e. $\mathcal{O}(\varepsilon^{-2})$, in other words this is the best rate that one can obtain.

4.4.2 Trace class neural networks

First we set up the problem. Suppose we have data $\mathcal{D} = ((x_1, y_1), \ldots, (x_N, y_N))$, $N \in \mathbb{N}$, where, for $i \in \{1, \ldots, N\}$, $x_i \in \mathsf{X}$ and $y_i \in \mathsf{Y}$. The objective is to infer a predictive model $f : \mathsf{X} \to \mathsf{Y}$ based on the data. One way to do this is with a parametric model of the form $f : \mathsf{X} \times \Theta \to \mathsf{Y}$, with $\Theta \subseteq \mathbb{R}^P$. It is assumed that $x_{1:N}$ are deterministic, i.e. the discriminative vs. generative view of supervised learning.

Regression. If $\mathsf{Y} = \mathbb{R}^C$, then we will assume that for $i \in \{1, \ldots, N\}$

$$y_i = f(x_i, \theta) + \epsilon_i \,, \qquad \epsilon_i \overset{\text{ind}}{\sim} \mathcal{N}_C(0, \Sigma_i), \tag{4.9}$$

where ind denotes independence across the indices $i \in \{1, \ldots, N\}$ and $\mathcal{N}_C(\mu, \Sigma)$ denotes the $C-$dimensional Gaussian distribution with mean μ and covariance matrix Σ. The likelihood function in this case is

$$p(Y|\theta, X) = \prod_{i=1}^{N} \phi_C(y_i; f(x_i, \theta), \Sigma_i), \tag{4.10}$$

where $\phi_C(y; \mu, \Sigma)$ is the $C-$dimensional Gaussian density of mean μ, covariance Σ evaluated at y given.

Classification. If $\mathsf{Y} = \{1, \ldots, C\}$ for some $C \in \mathbb{N}$, for convenience of the present exposition we will define $f : \mathsf{X} \times \Theta \to \mathbb{R}^C$, with $f(x, \theta) = (f_1(x, \theta), \ldots, f_C(x, \theta))$, and then separately the *softmax* function as

$$S_k(x_i, \theta) := \frac{\exp\{f_k(x_i, \theta)\}}{\sum_{j=1}^{C} \exp\{f_j(x_i, \theta)\}} \,, \qquad k \in \mathsf{Y} \,. \tag{4.11}$$

We will assume that $y_i \sim S(x_i, \theta)$, independently for $i \in \{1, \ldots, N\}$, where $S(x, \theta) = (S_1(x, \theta), \ldots, S_C(x, \theta))$ denotes a categorical distribution on C outcomes, given input x. In this case, the likelihood is

$$p(Y|\theta, X) = \prod_{i=1}^{N} \prod_{k=1}^{C} S_k(x_i, \theta)^{\mathbb{I}[y_i = k]} \,. \tag{4.12}$$

The predictive S is Lipschitz, so conclusions about f translate immediately. We conclude this part by stating the key advantages of TNNs: one is able to have a stable infinite limit related to the width, as well as high-dimensional scalability, due to the fact that one does not need to define the covariance operator structure.

Bayesian neural networks

Define the element-wise activation function as σ. Let $\mathsf{X} = \mathbb{R}^D$ and $\mathsf{Y} = \mathbb{R}^C$. The DNN can be defined in the following way. Let $D \in \mathbb{N}$, $(D_0, \ldots, D_L) \in \mathbb{N}^{L+1}$ be given, with the constraint that $D_0 = D$ for the input layer and $D_L = C$ for the output layer. Now for weights $A_d \in \mathbb{R}^{D_{L'} \times D_{L'-1}}$ and biases $b_d \in \mathbb{R}^{D_d}$, $d \in \{1, \ldots, L\}$ also given, we use the notation $\theta := ((A_1, b_1), \ldots, (A_L, b_L))$ and so $\theta \in \Theta = \bigotimes_{L'=1}^{L} \{\mathbb{R}^{D_d \times D_{d-1}} \times \mathbb{R}^{D_d}\}$. Now set

$$\begin{aligned} g_0(x, \theta) &:= A_1 x + b_1, \\ g_{L'}(x, \theta) &:= A_{L'} \sigma(g_{L'-1}(x)) + b_{L'}, \qquad L' \in \{1, \ldots L-1\}, \\ f(x, \theta) &:= A_L \sigma(g_{L-1}(x)) + b_L, \end{aligned} \tag{4.13}$$

where $f(x, \theta)$ is the output of the final layer of the DNN. The BNN is given by placing a prior p on Θ.

TNN

We now introduce the trace class neural network (TNN) priors, which were first proposed in Sell and Singh [1488], and differ from standard BNN priors in two fundamental ways. The first is that the prior on the weights and biases depends on the rows and columns, as opposed to standard isotropic weight decay. This is referred to as *width-decay* and it eliminates exchangeability and the associated non-identifiability. The second thing is that nested approximations of the well-defined TNN function-space converge strongly to a non-Gaussian process in the limit of infinite width. This stands in contrast to standard isotropic priors, under which the appropriately rescaled network converges weakly to a Gaussian process [386, 1226], so that the infinite-width limit is a kernel method and the compositional structure of the prior is lost. This will be made precise below.

These priors were introduced to mimic Gaussian measure priors $p \sim \mathcal{N}(0, \mathcal{C})$ for inverse problems over function-space, where a common way to simulate Gaussian random fields is to use the Karhunen-Loève expansion

$$f_{\mathsf{KL}} = \sum_{j \in \mathbb{Z}^+} \sqrt{\lambda_j} \iota_j \Phi_j, \qquad \iota_j \sim \mathcal{N}(0, 1), \tag{4.14}$$

where $(\lambda_j, \Phi_j)_{j \in \mathbb{Z}^+}$ are the associated eigenbasis of the covariance operator $\mathcal{C}$, and $\{\iota_j\}_{j \in \mathbb{Z}^+}$ is Gaussian white noise. For a detailed description of the derivation of (4.14) and its application to stochastic numerical problems, we refer the reader to Lord et al. [1078]. An issue that can arise with using (4.14) is that these priors do not scale well with input dimension, which is the initial motivation for TNN priors, where the variances of the weights and biases are summarized in a trace-class diagonal covariance operator $\mathcal{C}$. See [1488, Theorem 1]. More precisely, the TNN prior is given by

$$A_{ij,d} \sim \mathcal{N}(0, (ij)^{-s}), \quad b_{i,d} \sim \mathcal{N}(0, i^{-s}). \tag{4.15}$$

The tuning parameter s controls how much information one believes concentrates on the first nodes, and as such also controls the smoothness, expressiveness, and flexibility of the prior in terms of the functions it can represent. In the case of $s > 1$, we refer to the prior as trace-class, which is the motivation for the name trace-class neural network prior.

A convenient property of these priors is strong convergence, which is made precise with the following proposition. The proof is given in Chada et al. [252]. This is the key result that enables application of the MLSMC algorithm to TNN. The advantage of TNN is that it allows one not to prespecify the covariance operator structure in the prior, and it is also capable of having an infinite-width limit.

Proposition 4.1. *Assume that for all $z \in \mathbb{R}$, $|\sigma(z)| \leq |z|$. Then for $s > 1/2$, the prior predictive output is square summable for the prior defined as in* (4.15)*. Furthermore, let $x \in \mathsf{X}$, and consider the TNN $f_l(x, \theta_l)$ truncated at $D_l = 2^l$ width, for $l \in \mathbb{N}$, with the limit denoted $f(x, \theta) = \lim_{l \to \infty} f_l(x, \theta_l)$, as described above. Then there is a $K(x) > 0$ such that*

$$\mathbb{E}\left[|f_l(x, \theta_l) - f(x, \theta)|^2\right] \leq K 2^{-(2s-1)l}. \tag{4.16}$$

Multilevel TNN

We now consider the question of choosing the dimension of θ, in particular $D_{L'}$ for $L' \in \{1, \ldots, L-1\}$ (since D_0 and D_L are fixed to the input and output dimensions). For simplicity, here we assume L is fixed and the NN width is the same for all layers other than the input or the output, i.e. $D_{L'} = D_{L''}$ for $L', L'' \in \{1, \ldots, L-1\}$, but dependent upon some resolution parameter $l \in \mathbb{N}$. We can now re-define $D_l = 2^l$, noting that $D_0 = D$ and $D_L = C$, so those variables are no longer needed. Denote the corresponding vector of parameters by $\theta_l :=$

$\big((A_1^l, b_1^l), \ldots, (A_D^l, b_D^l)\big) \in \Theta_l \subset \Theta$. The limiting infinite width NN output function $f(x, \theta)$ (4.13) approximated at finite resolution l is denoted by $f_l(x, \theta_l)$, the likelihood by $p_l(Y|\theta_l, X)$, and the posterior distribution by

$$\pi_l(\theta_l) := p_l(\theta_l|\mathcal{D}) \propto p_l(Y|\theta_l, X)p_l(\theta_l) =: \kappa_l(\theta_l)\,. \tag{4.17}$$

We view this posterior (and the corresponding $f_l(x, \theta_l)$) as a finite approximation of the posterior associated to the non-parametric limiting DNN as $l \to \infty$, assuming it exists.

4.4.3 MLSMC TNN method

We begin this section by firstly presenting our algorithm for approximating functional expectations, with respect to the posterior π, such as the posterior predictive $\mathbb{E}_\pi[f(x, \theta)]$, as well as some mathematical results which justify their implementation. The form of our results are such that they extend trivially to objective functions $\varphi \circ f$, for Lipschitz φ, which includes most conventional performance metrics, such as negative log likelihood and accuracy. The results will be presented first for the posterior predictive, and then extended as a corollary. Noting that $\mathbb{E}_\pi[f(x, \theta)] = \mathbb{E}_{\pi_L}[f_L(x, \theta_L)] + (\mathbb{E}_\pi[f(x, \theta)] - \mathbb{E}_{\pi_L}[f_L(x, \theta_L)])$, our main objective is to leverage the MLMC machinery for the following telescopic identity:

$$\mathbb{E}_{\pi_L}[f_L(x, \theta_L)] = \sum_{l=1}^{L} \left\{\mathbb{E}_{\pi_l}[f_l(x, \theta_l)] - \mathbb{E}_{\pi_{l-1}}[f_{l-1}(x, \theta_{l-1})]\right\} + \mathbb{E}_{\pi_0}[f_0(x, \theta_0)]\,. \tag{4.18}$$

We will show that the optimal computational cost for achieving a MSE equal to $(\mathbb{E}_\pi[f(x, \theta)] - \mathbb{E}_{\pi_L}[f_L(x, \theta_L)])^2$ by approximating the individual summands on the R.H.S. independently is lower than directly approximating $\mathbb{E}_{\pi_L}[f_L(x, \theta_L)]$.

Algorithm

The approach we construct follows that in Beskos et al. [142, 143] and in order to directly apply some of its results, we introduce additional notation. Throughout this exposition $(x, L) \in \mathsf{X} \times \mathbb{N}$ are fixed and given. Define the spaces constructively as $\Theta_l = \Theta_{l-1} \times \Delta_l$, for $l \in \{1, \ldots, L\}$, and $\Delta_0 = \Theta_0$; the significance of these spaces will be made clear below. Set $\theta_0 = \delta_0 \in \Theta_0$ and for $l \in \{1, \ldots, L\}$

$$\theta_l = (\theta_{l-1}, \delta_l) = (\delta_0, \ldots, \delta_l) \in \Theta_l. \tag{4.19}$$

The quantities $(\delta_1, \ldots, \delta_l)$ will be used to denote the increase in dimension between θ_{l-1} and θ_l say, which is pivotal to the algorithm that we will present. Let $q_l(\cdot|\theta_{l-1})$ be a positive probability density on Δ_l and $q_0(\theta_0)$ a positive probability density on Θ_0. Set $G_0(\theta_0) = \kappa_0(\theta_0)/q_0(\theta_0)$ and define weights as

$$G_l(\theta_l) = \frac{\kappa_l(\theta_l)}{\kappa_{l-1}(\theta_{l-1})q_l(\delta_l|\theta_{l-1})}\,. \tag{4.20}$$

Let $(\mathcal{K}_l)_{l\in\{1,\ldots,L-1\}}$ be a sequence of $(\pi_l)_{l\in\{1,\ldots,L-1\}}-$invariant Markov kernels and define for $l \in \{1, \ldots, L-1\}$

$$\mathcal{M}_l(\theta_l, d\theta'_{l+1}) = \mathcal{K}_l(\theta_l, d\theta'_l)q_{l+1}(\delta_{l+1}|\theta'_l)d\delta_{l+1}, \tag{4.21}$$

where $\theta'_{l+1} = (\theta'_l, \delta_{l+1})$ and $d\delta_l$ is the appropriate dimensional Lebesgue measure. This kernel is invoked in Step 4 of Algorithm 11.

Set $\eta_0(\theta_0) = q_0(\theta_0)$ and for $l \in \{1, \ldots, L\}$

$$\eta_l(\theta_l) = \pi_{l-1}(\theta_{l-1})q_l(\delta_l|\theta_{l-1}). \tag{4.22}$$

The algorithm to be presented will provide a sample-based approximation of these probability densities and in particular, associated expectations. In the subsequent exposition, given $(\theta_l^1, \ldots, \theta_l^M) \in \Theta_l^M$, the so-called $M-$empirical measure will be denoted η_l^M. In other words we will have access to $M \in \mathbb{N}$ samples $(\theta_l^1, \ldots, \theta_l^M) \in \Theta_l^M$ so that

$$\eta_l^M(\varphi_l) := \frac{1}{M}\sum_{i=1}^{M} \varphi_l(\theta_l^i) \to \eta_l(\varphi_l) \quad \text{a.s.} \tag{4.23}$$

Algorithm 11: Multilevel Sequential Monte Carlo Sampler for TNN

Inputs : highest resolution $L \in \mathbb{N}$; number of samples per level $(M_0, \ldots, M_L) \in \mathbb{N}^{L+1}$, with $+\infty > M_0 \geq M_1 \geq \cdots \geq M_L \geq 1$
Outputs: particles $(\theta_1^1, \ldots, \theta_1^{M_1}, \ldots, \theta_L^1, \ldots, \theta_L^{M_L})$, from which (4.27) is constructed
`// Initialization`
for $i = 1$ **to** M_0 **do**
  independently sample $\theta_0^i \sim q_0(\theta_0)d\theta_0$
$l \leftarrow 0$
`// Iteration across levels`
while $l < L$ **do**
  for $i = 1$ **to** M_{l+1} **do**
    sample $\theta_{l+1}^i \mid \theta_l^1, \ldots, \theta_l^{M_l}$ independently using

$$\sum_{j=1}^{M_l} \frac{G_l(\theta_l^j)}{\sum_{s=1}^{M_l} G_l(\theta_l^s)} \mathcal{M}_l(\theta_l^j, d\theta_{l+1})$$

  $l \leftarrow l + 1$

Recalling (4.18), we will approximate the increments as follows

$$\frac{\eta_l^{M_l}(G_l f_l)}{\eta_l^{M_l}(G_l)} - \eta_l^{M_l}(f_{l-1}). \tag{4.24}$$

Note that $\eta_l^{M_l}(f_{l-1})$ will converge in probability (as $M_l \to \infty$) to $\pi_{l-1}(f_{l-1})$ [401]. Then $\eta_l^{M_l}(G_l)$ will converge to

$$\int_{\Theta_l} \pi_{l-1}(\theta_{l-1}) q_l(\delta_l|\theta_{l-1}) \frac{\kappa_l(\theta_l)}{\kappa_{l-1}(\theta_{l-1}) q_l(\delta_l|\theta_{l-1})} d\theta_l = \frac{1}{Z_{l-1}} \int_{\Theta_l} \kappa_l(\theta_l) d\theta_l = \frac{Z_l}{Z_{l-1}}, \tag{4.25}$$

and $\eta_l^{M_l}(G_l f_l)$ converges to

$$\int_{\Theta_l} \pi_{l-1}(\theta_{l-1}) q_l(\delta_l|\theta_{l-1}) \frac{\kappa_l(\theta_l)}{\kappa_{l-1}(\theta_{l-1}) q_l(\delta_l|\theta_{l-1})} f_l(x, \theta_l) d\theta_l = \frac{1}{Z_{l-1}} \int_{\Theta_l} \kappa_l(\theta_l) f_l(x, \theta_l) d\theta_l \,. \tag{4.26}$$

Consequently, one can use the following approximation of $\pi_L(f_L)$:

$$\widehat{\pi}_L(f_L) = \sum_{l=1}^{L} \left\{ \frac{\eta_l^{M_l}(G_l f_l)}{\eta_l^{M_l}(G_l)} - \eta_l^{M_l}(f_{l-1}) \right\} + \frac{\eta_0^{M_0}(G_0 f_0)}{\eta_0^{M_0}(G_0)}. \tag{4.27}$$

Ordinarily we choose q_l so that for each $\theta_l \in \Theta_l$

$$p_l(\theta_l) = p_{l-1}(\theta_{l-1}) q_l(\delta_l|\theta_{l-1}). \tag{4.28}$$

This means that

$$G_l(\theta_l) = \frac{p_l(Y|\theta_l, X)}{p_{l-1}(Y|\theta_{l-1}, X)}. \tag{4.29}$$

The main result is given in Chada et al. [252].

Proposition 4.2. *Under suitable assumptions, an analog of Theorem 4.1 applies to MLSMC with rates determined by* (4.16) *for costs* $C_l \propto 2^{2l}$*, i.e. our effective* $2\alpha = \beta = 2s - 1$ *and* $\gamma = 2$*. Hence we are able to choose* $L, \{M_1, \ldots, M_L\}$ *appropriately to achieve MSE of* $\mathcal{O}(\varepsilon^2)$ *for the canonical complexity* $\mathcal{O}(\varepsilon^{-2})$*.*

4.4.4 Numerical experiments

Regression Problem

The first numerical experiment will be based on a well-specified Bayesian regression problem, as given in (4.9), i.e. the data is generated from the model. We let $\Sigma_i^2 = 0.01^2 I$, $\sigma(z) = \tanh(z)$, $D = 10$, $D_l = 2^l$, and $N = 200$.

The inputs are simulated as $x_i \sim \mathcal{N}(2, 0.5)$. We use a high-resolution parameter $l = 9$ for the ground truth. We compare SMC and MLSMC samplers, using 100 replications to compute MSE. We report the complexity rate ξ such that cost $\propto$ MSE$^{-\xi}$ for different values of $\beta = 2s - 1$. Recall that $C_l = \mathcal{O}(2^{2l})$ so one expects to attain the canonical rate of convergence $\xi = 1$ when $s > 1.5 \to \beta > 2$. We conduct our numerical experiment with levels $L \in \{3, 4, \ldots, 7\}$ and smoothness parameters $s \in \{3, 1.1\}$, delivering canonical and sub-canonical convergence behaviour. The results are presented in Figure 4.1, including credible sets around the MSE values, given by the thin blue and red curves. Further details on implementation of the MLSMC sampler in general can be found in Beskos et al. [143].

The MLSMC sampler shows computational benefits, where for a given MSE the cost associated with attaining that order of MSE is smaller than the SMC sampler. We can see that (i) the rates are different, with MLSMC approximately achieving the canonical 1/MSE for $s = 3$ and (ii) the difference in cost for the lowest MSE is approximately a factor of 10 for $s = 3$, so the asymptotic benefit is achievable at a finite resolution, and (iii) that benefit is smaller in the sub-canonical case $s = 1.1$.

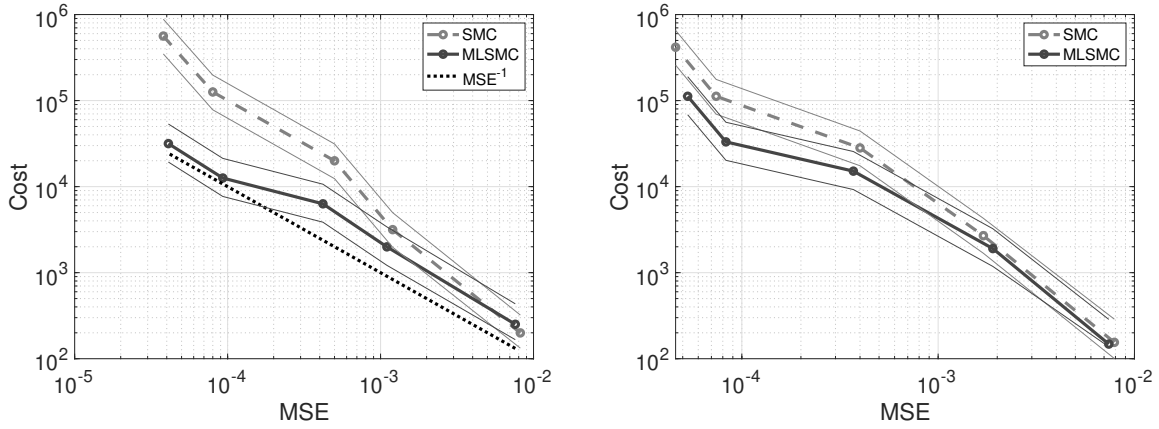


Figure 4.1: Regression problem: error vs cost plots for SMC and MLSMC using TNN priors. Left: $s = 3$. Right: $s = 1.1$. Credible intervals are provided by thin curves.

Binary MNIST Classification

As a second proof of concept, we consider a simplified binary MNIST [995] classification task using $D = 100$ principal components (PCs) of the original 28×28 image dataset, with $N = 400$. The results are similar to the regression case, and are presented in Figure 4.2.

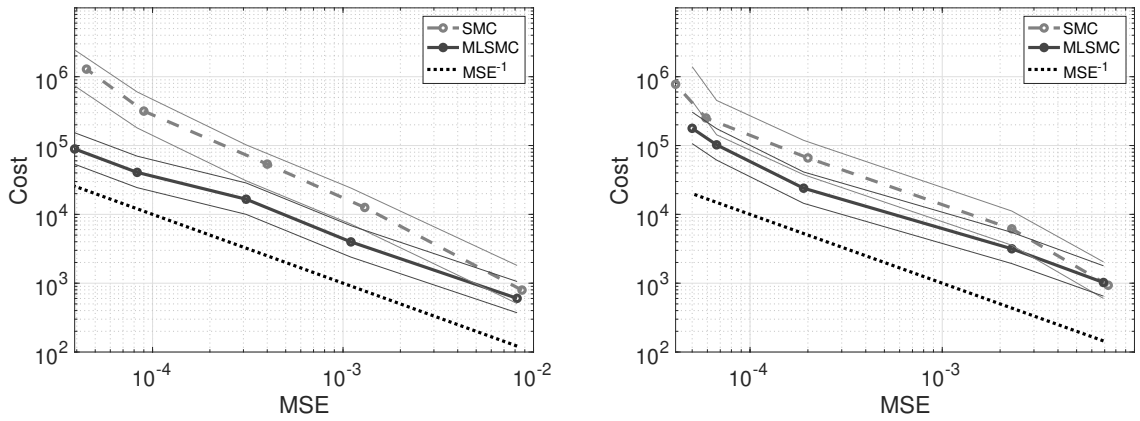


Figure 4.2: MNIST Classification problem: error vs cost plots for SMC and MLSMC, using TNN priors. Left: $s = 3$. Right: $s = 1.1$. Credible sets are provided in the thin curves.

4.5 Parallel SMC

For this section, we will assume a fixed and finite network architecture f of the standard form, as introduced in Section 4.4.2. This gives rise to a computable target distribution (up to a normalizing constant) $\pi = \kappa/Z$.

SMC is inherently parallel, as the MCMC moves for each particle can be executed concurrently [997, 1282, 1581]. This is where the likelihood computations happen, which are the most expensive and dominant contributors to the computational complexity. One must take care that sufficient memory per core is available or else this approach will hit the "memory wall" [822]. The resampling stage requires communication between all particles, which could also be a bottleneck depending on the communication bandwidth, but this may not be an issue for single-instruction-multiple-data (SIMD)-type architectures as found on a single multi-core CPU or GPU. This is the most common approach for leveraging parallelism in SMC, as it is very simple to implement and can sometimes deliver strong parallel scaling [997].

Algorithm 12: $\text{SMC}_\parallel$ sampler

Inputs : number of parallel runs R; number of particles M
Outputs: particle systems $\{\theta^{i,r}\}_{i=1}^M$ and normalizing constants $Z^{M,r}$ for $r = 1, \ldots, R$
for $r = 1$ **to** R **do**
  run Algorithm 10
  output $\{\theta^{i,r}\}_{i=1}^M$ and $Z^{M,r}$

Beyond this inter-SMC parallelism, the combination of parallel SMCs has recently been considered in Vergé et al. [1674], Whiteley et al. [1745]. The recommended approach typically involves communication between *all samples*, which improves stability but hinders scalability for large models which need to be distributed across many SIMD nodes, which may either be disconnected or have slower between-node inter-connect. The island particle model [1674] separates the total number of samples M into $R \leq R_{\max}$ SMC islands with M/R samples each. Without between-SMC interaction, naive (equal-weight) averaging results in an asymptotic bias penalty $(R/M)^2$ [346, 1674]. However, *by assigning appropriate weights to each SMC we can eliminate this penalty* [355, 1745]. The resulting scalable parallel SMC sampler is denoted by $\text{SMC}_\parallel$.

Algorithm 12 displays the $\text{SMC}_\parallel$ method, and we define the consistent (in R, for finite M suitably large) $\text{SMC}_\parallel$ ratio estimator as

$$\hat{\varphi}_{\text{SMC}_\parallel} = \sum_{r=1}^{R} \omega_r \pi^{M,r}(\varphi)\,, \quad \omega_r = \frac{Z^{M,r}}{\sum_{s=1}^{R} Z^{M,s}}\,. \tag{4.30}$$

This estimator is consistent, which is not the case for a naive unweighted *average.* See Proposition 4.3 and the discussion above.

4.5.1 Parallel MCMC

MCMC can also exploit parallel resources: one can retain a single sample from each of M parallel chains, each run for b warm-up epochs, achieving $\text{MSE} = O(e^{-b} + 1/M)$ [1131], which is indistinguishable from the single-chain result for $b \gg \log M$, at a parallel cost equivalent to SMC when the integrated autocorrelation time matches the total number of mutation epochs. However, unlike SMC, the resulting estimator is not consistent for finite b, which can potentially spoil convergence.

4.5.2 Theoretical Result

The convergence result is made rigorous in the following proposition. Suitable assumptions and proof are provided in Liang et al. [1038].

Proposition 4.3. *For suitable φ, m, M, T, there exists a $K > 0$, which depends on φ, m, T, such that for any $R \in \mathbb{N}$,*

$$\mathbb{E}[(\hat{\varphi}_{SMC_\parallel} - \pi(\varphi))^2] \leq \frac{K}{MR}\,. \tag{4.31}$$

4.5.3 Numerical experiments

As above, we will measure wall-clock time complexity in terms of function evaluations, as an implementation and hardware agnostic proxy. We will refer to a likelihood evaluation as an *epoch*, in analogy with stochastic

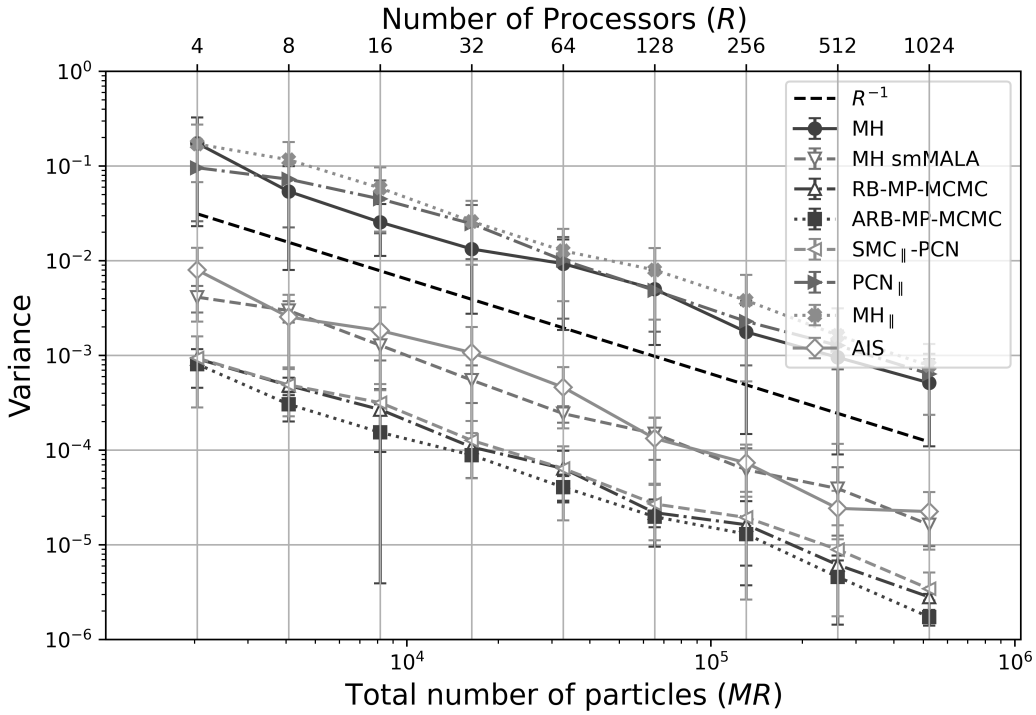


Figure 4.3: Empirical variance of posterior mean estimates for Bayesian logistic regression on Australian Credit ($m = 690$, $d = 15$), for SMC$_\|$-pCN, various MCMC$_\|$ methods, and AIS. SMC$_\|$-pCN and the MCMC$_\|$ baselines are compared at matched total sample size MR, with $M = 511$ particles per process; AIS is equivalent to $M = 1$ particle per process. SMC$_\|$-pCN uses pCN mutations with 30 mutation steps per tempering stage and adaptive tempering that keeps the effective sample size above $M/2$. Curves are averaged over 25 repetitions; vertical bars show ± 3 standard errors. Both axes are log-scaled.

gradient descent (SGD) [194, 1404] based optimization methods like Adam [894]. However, note that the time to compute an epoch of mini-batches depends heavily on the size of the problem and the details of the hardware and implementation. If all data fits on a single computational unit (either a node or a core), then a serial epoch of mini-batch likelihood computations will typically require much longer wall-clock time in comparison to a single likelihood evaluation. In contrast, if only a single mini-batch fits on a node, and interconnect is slow, then even data parallel may be slower than serial mini-batches. It is also worth noting that mini-batch gradients can deliver comparable gain to full gradient steps far from convergence, which can significantly accelerate progress as measured by epochs [195].

Proof of concept

First, we will consider logistic regression on the Australian Credit data [1355], illustrating comparable results to state-of-the-art (SOTA) parallel MC algorithms and the $1/R$ convergence guaranteed by Proposition 4.3. The dataset has $d = 15$ covariates and $m = 690$ data and Bayesian logistic regression is used. We compare SMC$_\|$-pCN (preconditioned Crank–Nicolson, pCN) with annealed importance sampling [1228], various MCMC$_\|$ methods, and recent synchronous-parallel MCMC [1479]. Figure 4.3 shows the empirical variance for posterior mean estimates of the proposed methods with respect to the number of samples and number of parallel processes, consistent with the $1/R$ convergence result of Proposition 4.3. SMC$_\|$-pCN attains the lowest empirical variance among the baselines shown in Figure 4.3 that do not require communication scaling with R. The MCMC$_\|$ data in Figure 4.3 is reproduced directly from Schwedes and Calderhead [1479]. SMC and AIS incur a per-sample overhead in cost ($\sum_t m_t$, where m_t are the mutations at tempering step t), but this is offset by intra-parallelism ($\sum_t m_t/M \ll 1$).

IMDb dataset

We now consider a large problem in semantic classification of natural language, using the IMDb dataset [1100] of 50,000 movie reviews, split evenly into training and test sets.

Architecture. Each review is mapped to a fixed feature vector by SBERT embeddings [1388] from the model `all-mpnet-base-v2` [1539][1], whose weights are *frozen* throughout: they are used only to produce the 768 dimensional [CLS] embedding and are never sampled. On top of these embeddings we place (i) one hidden layer with 128 neurons, (ii) ReLU activation, (iii) a final linear layer, and (iv) a softmax output. Inference is over *all* weights and biases of this network, of dimension $P = 98690$, and not merely the final classifier head. The whole train (25000 data) and test dataset (25000 data) are considered.

Sampling setup. The likelihood is the softmax (categorical cross-entropy) likelihood over the whole training set of 25000 reviews, and inference uses an adaptive likelihood-tempering SMC sampler with HMC mutations of length T. We compare $\mathrm{SMC}_{\|}$ against two baselines that share the same prior, likelihood, and frozen embeddings: $\mathrm{HMC}_{\|}$, which averages over MR independent HMC chains of length T, and a single serial HMC run of length MRT; in every method the HMC step size is adapted to a common acceptance-rate target, so the comparison is made at matched cost.

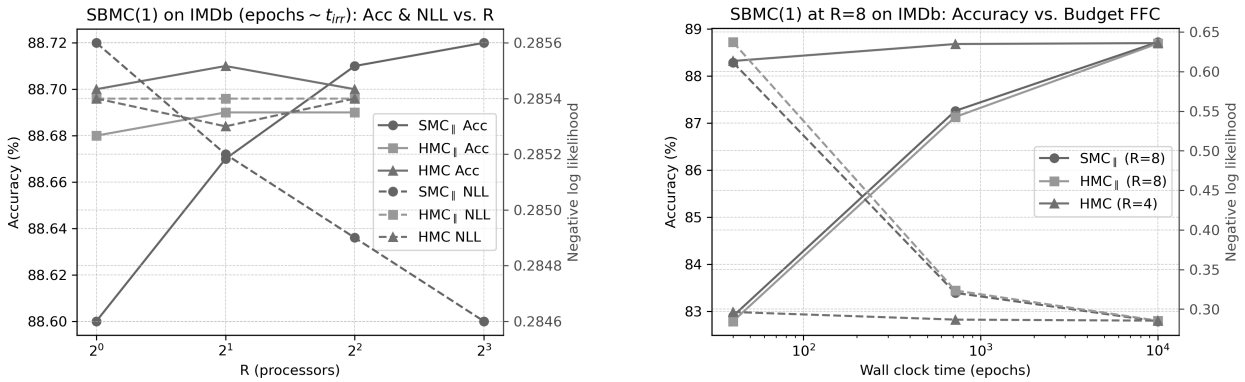


Figure 4.4: IMDb sentiment classification, with the full posterior over all $P = 98690$ network weights, using $\mathrm{SMC}_{\|}$ with $M = 32$ particles for each of R SMC runs, HMC mutations, and adaptive tempering that keeps the effective sample size above $M/2$. Left: test accuracy and negative log-likelihood (NLL) against the number of parallel runs R at a budget of about $\tau_{\mathrm{irr}} = 10^4$ epochs, alongside a serial HMC baseline given $MR\tau_{\mathrm{irr}}$ epochs and $\mathrm{HMC}_{\|}$ with MR chains with τ_{irr} epochs each. Right: converged values ($R = 8$) showing the catastrophic failure (collapsed accuracy and inflated NLL) once the budget drops below τ_{irr}. The R- and budget-axes are log-scaled; all curves are averaged over 5 realizations.

Experiment. Denote by m the number of evaluations of κ per mutation step, which we refer to as *epochs*, and by T the number of tempering-and-mutation steps. The product mT (or $\sum_{t=1}^{T} m_t$ for an adaptive schedule) is the total simulation cost, which we call the *irreducible serial time* τ_{irr}: the serial simulation the sampler needs in order to mix, which cannot be removed by adding more parallel runs. In practice τ_{irr} is read off empirically as the budget at which accuracy and NLL plateau, here $\tau_{\mathrm{irr}} \approx 10^4$ epochs. When m and/or T are too small, so that the budget falls well short of τ_{irr}, the sampler has not mixed and the method *catastrophically fails*: test accuracy collapses and NLL inflates. Figure 4.4 summarises this behaviour. The left panel shows performance as a function of R at τ_{irr}, where $\mathrm{SMC}_{\|}$ improves steadily with R and matches the far more expensive serial HMC baseline; the right panel fixes $R = 8$ and reduces the budget below τ_{irr}, exposing the catastrophic failure of $\mathrm{SMC}_{\|}$ and $\mathrm{HMC}_{\|}$ once the budget falls short of the irreducible time.

[1] https://huggingface.co/sentence-transformers/all-mpnet-base-v2

Chapter 5

Low-precision sampling

When running simulations for a dynamic system on a computer, the impact of round-off errors on number representations (e.g., floating point representations) can not be naïvely presumed to be negligible. It has been observed that a large deviation can occur between numerical behavior and the theoretical behaviour [684]. In Bayesian literature, the same question arises: how does the rounding error from computer simulation affect the convergence property of a Markov chain? Roberts et al. [1411] shows a rather alarming example that a well-behaved Markov chain (with Feller continuity and geometrical ergodicity) becomes transient (i.e., will not converge to any target distribution) after adding an arbitrarily small roundoff error. Hoffman [764] also warned that roundoff errors can severely degrade the performance of the Metropolis-Hastings sampling algorithm. To sum up, a common wisdom for conventional MCMC practice is to utilize high-precision numerical representations to limit the negative impact of roundoff errors. However, in this chapter, we will embrace the low-precision representations in the era of SG-MCMC samplers.

5.1 Low-Precision Deep Neural Network

The rise of large-scale deep neural networks (DNNs) and large language models (LLMs) has led to significant computational and memory costs of model training and inference. To improve efficiency, low-precision deep learning [800, 827, 1028, 1383, 1843, 1873] — which uses fewer bits to represent numbers in model parameters, activations, and gradients, and thus can drastically lower resource demands — has emerged as a promising direction. Precision formats such as 16-bit (FP16), 8-bit (INT8), 4-bit, and even binary (1-bit) offer memory reduction, energy savings, and speedup in inference, enabling deployment on edge devices and platforms, and specialized accelerators. On the other hand, well-designed low-precision optimization algorithms can maintain the model utility, such that the performance degradation from full-precision models to low-precision models is tolerable.

Modern foundations of low-precision deep learning include Gupta et al. [689], which leverages 16-bit wide fixed-point arithmetic and stochastic rounding to accelerate the training speed of DNNs, and Vanhoucke et al. [1666], which discusses the possibility of running DNN inference on x86 CPUs rather than GPUs using 8-bit fixed-point arithmetic. In general, there are two directions for achieving lower precision representations of DNN models: quantization-aware training (QAT) and post-training quantization (PTQ). The former simulates quantization during the training process, allowing the model to learn a robust solution that accounts for quantization errors induced by rounding; the latter, on the other hand, applies quantization to off-the-shelf pre-trained models. It is arguably true that QAT outperforms PTQ in terms of better accuracy, but incurs greater computational costs of model training. Other worth mentioning techniques of low-precision models are, but not limited to, non-uniform quantization and mixed precision training. Opposite to conventional quantization methods that map floating point values to an evenly spaced quantized target space, non-uniform quantization chooses an unevenly spaced quantized target space that can allocate more quantization levels where the values are dense. This unevenly spaced quantized target space can be of logarithmic scale [1191], or adaptively learned from the training process [1843]. Mixed-precision methods [440, 1171] assign different bit-widths to different layers of the neural networks or different computation steps of the training, leveraging the fact that different components of DNNs have varying sensitivities to quantization error. Theoretical investigations related to low-precision training are available in Li et al. [1028], Markov et al. [1134], Sa et al. [1445], Sakr et al. [1449].

Inevitably, low-precision optimization achieves better efficiency at the expense of performance sacrifice. One natural way to mitigate performance degradation caused by quantization error is low-precision Bayesian sampling. We argue that posterior sampling is particularly suited for low-precision arithmetic because of its intrinsic ro-

bustness to system noise. In particular: (1) Bayesian posterior explores weight space instead of converging to a single point, thus it should not require precise weights and be tolerant to non-full-precision models; and (2) it allows us to perform Bayesian model averaging during testing using an ensemble of models, which allows coarse representations of individual models to be compensated by the overall model average [1878].

5.2 Low-Precision SG-MCMC

5.2.1 Preliminary

The Bayesian counterpart of QAT directly samples low-precision models (i.e., low-precision weights of the neural network) from a target distribution (typically, a Bayesian posterior distribution). A straightforward idea is to adapt existing Monte Carlo sampling algorithms for low-precision sampling tasks. In the regime of deep learning, where the training data sets are usually massive, the SG-MCMC sampling algorithm (see Chapter 2) is the most suitable choice since it can avoid a full scan of the data set when calculating the likelihood terms.

Now we briefly recall two seminal SG-MCMC algorithms, the SGLD and SGHMC. Given some dataset $\mathcal{D}$, a model with parameters $\theta \in \mathbb{R}^P$, and a prior $p(\theta)$, we are interested in sampling from the posterior $p(\theta|\mathcal{D}) \propto \exp(-U(\theta))$, where the energy function is

$$U(\theta) = -\sum_{(x,y)\in\mathcal{D}} \log p(x, y|\theta) - \log p(\theta).$$

Stochastic Gradient Langevin Dynamics (SGLD) reduces the computational cost by using a stochastic gradient estimation $\nabla\tilde{U}$, which is an unbiased estimator of ∇U based on a subset of the dataset $\mathcal{D}$. Specifically, SGLD updates the parameter in the $(k+1)$-th step following the rule

$$\theta_{k+1} = \theta_k - \alpha\nabla\tilde{U}(\theta_k) + \sqrt{2\alpha}\xi_{k+1}, \tag{5.1}$$

where α is the stepsize and ξ_{k+1} is a standard Gaussian noise. Compared to the Stochastic Gradient Descent (SGD) update, the only difference is that SGLD adds an additional Gaussian noise in each step, which essentially enables SGLD to characterize the full distribution instead of converging to a single point. The close connection between SGLD and SGD makes it convenient to implement and run on existing deep learning tasks for which SGD is the typical learning algorithm.

Stochastic Gradient HMC [SGHMC, 273] is derived by replacing ∇U with a noisy gradient $\nabla\tilde{U} + \xi$ in the Hamiltonian Monte Carlo, and boils down to the unadjusted underdamped Langevin algorithm with stochastic gradient, where the underdamped Langevin dynamics can be defined as

$$\begin{aligned} \mathrm{d}v_t &= -\gamma v_t\,\mathrm{d}t - u\nabla U(\theta_t)\,\mathrm{d}t + \sqrt{2\gamma u}\,\mathrm{d}\mathbf{B}_t \\ \mathrm{d}\theta_t &= v_t\,\mathrm{d}t, \end{aligned} \tag{5.2}$$

where u, γ denote the hyperparameters of the inverse mass and friction, respectively. Furthermore, Cheng et al. [287] proposed the following discretization of underdamped Langevin dynamics (5.2) with stochastic gradient:

$$\begin{aligned} v_{k+1} &= v_k e^{-\gamma\eta} - u\gamma^{-1}(1 - e^{-\gamma\eta})\nabla\tilde{U}(\theta_k) + \xi_k^v \\ \theta_{k+1} &= \theta_k + \gamma^{-1}(1 - e^{-\gamma\eta})v_k + u\gamma^{-2}(\gamma\eta + e^{-\gamma\eta} - 1)\nabla\tilde{U}(\theta_k) + \xi_k^\theta, \end{aligned} \tag{5.3}$$

where ξ_k^v, ξ_k^θ are normal distributed in $\mathbb{R}^P$ satisfying that :

$$\begin{aligned} \mathbb{E}\xi_k^v(\xi_k^v)^\intercal &= u(1 - e^{-2\gamma\eta})\cdot\mathbf{I}, \\ \mathbb{E}\xi_k^\theta(\xi_k^\theta)^\intercal &= u\gamma^{-2}(2\gamma\eta + 4e^{-\gamma\eta} - e^{-2\gamma\eta} - 3)\cdot\mathbf{I}, \\ \mathbb{E}\xi_k^\theta(\xi_k^v)^\intercal &= u\gamma^{-1}(1 - 2e^{-\gamma\eta} + e^{-2\gamma\eta})\cdot\mathbf{I}. \end{aligned} \tag{5.4}$$

Before formally introducing the low-precision SG-MCMC algorithms, we review the basic concepts of low-precision training. To represent numbers, e.g., the weights of a DNN, in low-precision, one simple way is to use *fixed point*, which has been utilized in both theory and practice [689, 1028, 1797]. Specifically, suppose that we use W bits to represent numbers with F of those W bits to represent the fractional part. Then there is a distance between consecutive representable numbers, $\Delta = 2^{-F}$, which is called *quantization gap*. The representable numbers also have a lower bound l and an upper bound u, where

$$L = -2^{W-F-1}, \quad U = 2^{W-F-1} - 2^{-F}.$$

As shown above, when the number of bits decreases, the accuracy of number representation decreases. We use this type of number representation in our theoretical analysis and empirical demonstration following previous work [1028, 1797].

Another common type of number representation is *floating point*, where each number has its own exponent part. Between fixed point and floating point, there is *block floating point* which allows all numbers within a block to share the same exponent [1543]. We use block floating point for deep learning experiments since it has been shown to be more favourable for deep models [1797].

Having low-precision number representation in hand, we also need a quantization function Q to convert a real-valued number into a low-precision number. Such functions include *deterministic rounding* and *stochastic rounding*. Particularly, the deterministic rounding function Q^d quantizes a number to its nearest representable neighbour as follows:

$$Q^d(\theta) = \text{sign}(\theta) \cdot \text{clip}\left(\Delta \left\lfloor \frac{|\theta|}{\Delta} + \frac{1}{2} \right\rfloor, L, U\right),$$

where $\text{clip}(x, L, U) = \max(\min(x, U), L)$ and the above action is applied to the whole vector elementwisely. Instead, stochastic rounding Q^s quantizes a number with a probability based on the distance to its representable neighbour:

$$Q^s(\theta) = \begin{cases} \text{clip}\left(\Delta \left\lfloor \frac{\theta}{\Delta} \right\rfloor, L, U\right), & \text{w.p. } \left\lceil \frac{\theta}{\Delta} \right\rceil - \frac{\theta}{\Delta} \\ \text{clip}\left(\Delta \left\lceil \frac{\theta}{\Delta} \right\rceil, L, U\right), & \text{w.p. } 1 - \left(\left\lceil \frac{\theta}{\Delta} \right\rceil - \frac{\theta}{\Delta}\right). \end{cases}$$

An important property of Q^s is that $\mathbb{E}[Q^s(\theta)] = \theta$, which means the quantized number is unbiased. Q^s is generally preferred over Q^d in practice since it can preserve gradient information, especially when the gradient update is smaller than the quantization gap [689, 1708]. In what follows, when necessary, we use Q_W and Δ_W to denote the weights' quantizer and quantization gap, Q_G and Δ_G to denote gradients' quantizer and quantization gap.

To do the gradient update in low-precision training optimization, there are two common choices depending on whether we store an additional copy of full-precision weights. *Full-precision gradient accumulators* use a full-precision weight buffer to accumulate gradient updates and only quantize weights before computing gradients. SGD with full-precision gradient accumulators (SGDLP-F) updates the weights as follows,

$$\theta_{k+1} = \theta_k - \alpha Q_G\left(\nabla \tilde{U}(Q_W(\theta_k))\right),$$

where we use full-precision θ_{k+1} and θ_k in the update, and only quantize the weight for forward and backward propagation [334, 1028].

However, gradient accumulators have to be frequently updated during training. It would be ideal to also represent it in low-precision to further reduce the costs. To achieve it, we could instead do the update as follows,

$$\theta_{k+1} = Q_W\left(\theta_k - \alpha Q_G\left(\nabla \tilde{U}(\theta_k)\right)\right), \tag{5.5}$$

where θ is always represented in low-precision. This update of SGD is called using *low-precision gradient accumulators* (SGDLP-L). Both full- and low-precision gradient accumulators have been widely used: low-precision gradient accumulators are cheaper and faster because of having all numbers in low-precision, whereas full-precision gradient accumulators generally have better performance because of more precisely reflecting small gradient updates [334, 1028].

5.2.2 Low-Precision Stochastic Gradient Langevin Dynamics

In this section, we combine the low-precision rounding and the SGLD update (5.1). As shown in Equation (5.1), the update of SGLD is simply an SGD update plus a Gaussian noise. Therefore, the low-precision formulation for SGD in the previous section can be naturally extended to SGLD training. Similar to SGDLP-F, we can do low-precision SGLD with full-precision gradient accumulators (SGLDLP-F) as follows:

$$\theta_{k+1} = \theta_k - \alpha Q_G\left(\nabla \tilde{U}(Q_W(\theta_k))\right) + \sqrt{2\alpha}\xi_{k+1}, \tag{5.6}$$

which can also be viewed as SGDLP-F plus a Gaussian noise in each step. However, the Gaussian noise turns out to help counteract the noise introduced by quantization, making SGLDLP-F more robust to inaccurate number representation and converging better than SGDLP-F as shown in Theorem 5.1.

To facilitate the theoretical analysis, we assume that the target distribution is smooth and strongly log-concave, and the energy function has Lipschitz Hessian, i.e., $\forall\ \theta, \theta' \in \mathbb{R}^P$,

$$\begin{aligned} &U(\theta) - U(\theta') - \nabla U(\theta')^\intercal(\theta - \theta') \geq (m/2)\|\theta - \theta'\|_2^2, \\ &\|\nabla U(\theta) - \nabla U(\theta')\|_2 \leq M\|\theta - \theta'\|_2, \\ &\|\nabla^2 U(\theta) - \nabla^2 U(\theta')\|_2 \leq \Psi\|\theta - \theta'\|_2, \\ &\mathbb{E}\Big[\|\nabla\tilde{U}(\theta) - \nabla U(\theta)\|_2^2\Big] \leq \kappa^2. \end{aligned} \tag{5.7}$$

These assumptions are commonly used in the literature [e.g., 359, 1719, 1797, 1856].

Theorem 5.1. *We run SGLDLP-F under the above assumptions and with a constant stepsize $\alpha \leq 2/(m+M)$. Let π be the target distribution, μ_0 be the initial distribution and μ_K be the distribution obtained by SGLDLP-F after K iterations, then the 2-Wasserstein distance is*

$$\begin{aligned} W_2(\mu_K, \pi) \leq (1-\alpha m)^K W_2(\mu_0, \pi) + 1.65(M/m)(\alpha P)^{1/2} \\ + \min\left(\frac{\Psi\Delta_W^2 P}{4m}, \frac{M\Delta_W\sqrt{P}}{2m}\right) + \sqrt{\frac{(\Delta_G^2 + M^2\Delta_W^2)\alpha P + 4\alpha\kappa^2}{4m}}. \end{aligned}$$

This theorem shows that SGLDLP-F converges to the accuracy floor at

$$\min\left(\frac{\Psi\Delta_W^2 P}{4m}, \frac{M\Delta_W\sqrt{P}}{2m}\right) \tag{5.8}$$

given a large iteration number K and small stepsize α. Besides, if we further assume that the energy function is quadratic (i.e., $\Psi = 0$), then SGLDLP-F converges to the target distribution asymptotically. This matches the result of SGDLP-F on a quadratic function which converges to the optimum asymptotically [1028]. Our theorem also recovers the bound of full-precision SGLD (i.e., $\Delta_W = 0$) in Dalalyan and Karagulyan [359]. However, when the energy function is not quadratic, the convergence of SGLDLP-F to the target distribution has a $\mathcal{O}(\Delta_W^2)$ rate whereas SGDLP-F to the optimum has a $\mathcal{O}(\Delta_W)$ rate [1797]. Recall that $\Delta_W = 2^{-F}$ where F is the number of fractional bits. Our result suggests that asymptotically, SGLD only needs half the number of bits as SGD needs to achieve the same convergence accuracy! This comparison between SGLD and SGD also fits into the literature in comparing sampling and optimization convergence bounds [1098, 1586].

To further reduce the costs using low-precision gradient accumulators beyond SGLDLP-F, we mimic the update of SGDLP-L in Equation (5.5) and naturally get the following update rule for SGLD with low-precision gradient accumulators (SGLDLP-L),

$$\theta_{k+1} = Q_W\left(\theta_k - \alpha Q_G\left(\nabla\tilde{U}(\theta_k)\right) + \sqrt{2\alpha}\xi_{k+1}\right). \tag{5.9}$$

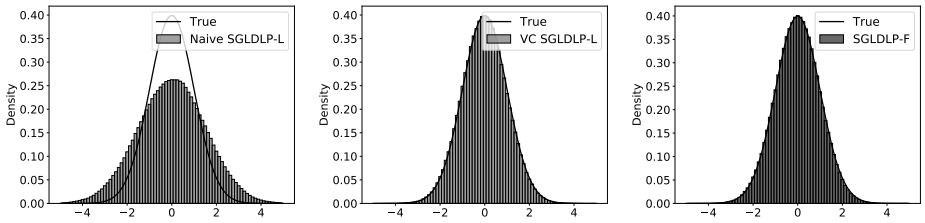
Surprisingly, despite the convergence of SGLDLP-L, the next theorem shows that it may diverge arbitrarily far away from the target distribution with small stepsizes.

Theorem 5.2. *We run SGLDLP-L under the same assumptions as in Theorem 5.1. Let μ_0 be the initial distribution and μ_K be the distribution obtained by SGLDLP-L after K iterations, then*

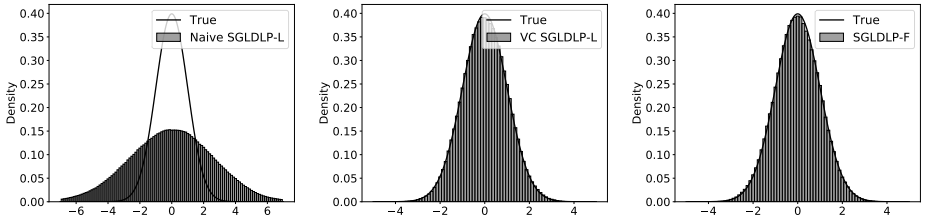
$$\begin{aligned} W_2(\mu_K, \pi) \leq (1-\alpha m)^K W_2(\mu_0, \pi) + 1.65(M/m)(\alpha P)^{1/2} + \min\left(\frac{\Psi\Delta_W^2 P}{4m}, \frac{M\Delta_W\sqrt{P}}{2m}\right) \\ + \sqrt{\frac{(\alpha\Delta_G^2 + \alpha^{-1}\Delta_W^2)P + 4\alpha\kappa^2}{4m}} + \left((1-\alpha m)^K + 1\right)\frac{\Delta_W\sqrt{P}}{2}. \end{aligned}$$

Note that one term on the right-hand side contains α^{-1}, this theorem implies that as the stepsize α decreases, W_2 distance between the stationary distribution of SGLDLP-L and the target distribution may increase. To see if this is the case, we empirically run SGLDLP-L on a standard Gaussian distribution in Figure 5.1. We use 8-bit fixed-point and assign 3 of them to represent the fractional part. The simulations verify that SGLDLP-L indeed diverges from the target distribution with small stepsizes. At the same time, SGLDLP-F always converges to the target distribution with different stepsizes, aligning with the result in Theorem 5.1.

One may choose a stepsize that minimizes the above W_2 distance to avoid divergence, however, getting that optimal stepsize is, in general, difficult since the constants are unknown in practice. Moreover, enabling a small



(a) Low-precision SGLD with $\alpha = 0.001$



(b) Low-precision SGLD with $\alpha = 0.0001$

Figure 5.1: Low-precision SGLD with varying stepsizes on a Gaussian distribution. Variance-corrected SGLD with low-precision gradient accumulators (VC SGLDLP-L) and SGLD with full-precision gradient accumulators (SGLDLP-F) converge to the true distribution, whereas naïve SGLDLP-L diverges and the divergence increases as the stepsize decreases.

stepsize in SGLD is often desirable, since it is needed to reduce the asymptotic bias of the posterior approximation [1737].

One key reason causing the divergence of SGLDLP-L is that the *variance* of each dimension of θ_{k+1} becomes larger due to using low-precision gradient accumulators. Specifically, given the stochastic gradient $\nabla\tilde{U}$, the update of full-precision SGLD can be viewed as sampling from a Gaussian distribution for each dimension i

$$\theta_{k+1,i} \sim \mathcal{N}\left(\theta_{k,i} - \alpha\nabla\tilde{U}(\theta_k)_i, 2\alpha\right), \text{for } i = 1, \cdots, P.$$

Since stochastic rounding is unbiased, using it as the weight quantizer Q_W and the gradient quantizer Q_G in SGLDLP-L gives us

$$\mathbb{E}\theta_{k+1,i} = \mathbb{E}Q_W\left(\theta_{k,i} - \alpha Q_G\left(\nabla\tilde{U}(\theta_k)\right)_i + \sqrt{2\alpha}\xi_{k+1,i}\right) = \theta_{k,i} - \alpha\nabla\tilde{U}(\theta_k)_i,$$

which has the same mean as θ_{k+1} in full-precision. However, the variance of $\theta_{k+1,i}$ is now essentially larger than needed. If we ignore the variance from Q_G and the stochastic gradient, since they are present and have been shown to work well in SGLDLP-F, the variance of $\theta_{k+1,i}$ is

$$\begin{aligned}
&\text{Var}(\theta_{k+1,i}) \\
&= \mathbb{E}\left[\text{Var}\left[Q_W\left(\theta_{k,i} - \alpha\nabla U(\theta_k)_i + \sqrt{2\alpha}\xi_{k+1,i}\right)\middle|\xi_{k+1,i}\right]\right] \\
&+ \text{Var}\left[\mathbb{E}\left[Q_W\left(\theta_{k,i} - \alpha\nabla U(\theta_k)_i + \sqrt{2\alpha}\xi_{k+1,i}\right)\middle|\xi_{k+1,i}\right]\right] \\
&= \frac{\Delta_W^2}{4}\chi_{k+1,i} + 2\alpha,
\end{aligned}$$

where $\chi_{k+1,i} \in [0, 1]$ depends on the distance of $\theta_{k+1,i}$ to its discrete neighbour. The above equation shows that the variance in SGLDLP-L update is larger than the right variance value 2α. This can be validated from Figure 5.1 where the naïve SGLDLP-L estimates the mean correctly but the variance wrongly.

To enable SGLD with low-precision gradient accumulators, we introduce a new quantization function Q^{vc}, denoting variance-corrected quantization, to fix the issue. The main idea of Q^{vc} is to directly sample from the discrete weight space instead of quantizing a real-valued Gaussian sample. First, we note that if we want a sample with mean $\mu \geq 0$ and variance $v \leq \Delta_W^2/4$, we could sample from the following categorical distribution over

$\{\Delta_W, -\Delta_W, 0\}$ to get it,

$$\mathrm{Cat}(\mu, v) = \begin{cases} \Delta_W, & w.p. \frac{v+\mu^2+\mu\Delta_W}{2\Delta_W^2} \\ -\Delta_W, & w.p. \frac{v+\mu^2-\mu\Delta_W}{2\Delta_W^2} \\ 0, & \text{otherwise} \end{cases} \tag{5.10}$$

providing that the probability values are valid (i.e., non-negative and smaller than 1). Now we show how to use this categorical distribution to preserve the correct mean and variance for quantized θ_{k+1}. We do so considering two cases: when the Gaussian variance 2α is larger than the largest possible stochastic rounding variance $\Delta_W^2/4$, Q^{vc} first adds a small Gaussian noise and uses a sample from Equation (5.10) to make up the remaining variance; in the other situation, Q^{vc} directly samples from Equation (5.10) to achieve the target variance. The full description of Q^{vc} is outlined in Algorithm 13 (where the use of $\mathrm{Cat}(\cdot,\cdot)$ sampler is valid). SGLDLP-L using Q^{vc}, denoting VC SGLDLP-L, follows the update rule:

$$\theta_{k+1} = Q^{\mathrm{vc}}\left(\theta_k - \alpha Q_G\left(\nabla\tilde{U}(\theta_k)\right), 2\alpha, \Delta_W\right). \tag{5.11}$$

Algorithm 13: Variance-Corrected Quantization Function Q^{vc}.

Input : (μ, v, Δ)
Output: $\theta \in \mathbb{R}^P$ with mean vector $\mu \in \mathbb{R}^P$ and coordinate variance $v \in \mathbb{R}$
$v_0 \leftarrow \Delta^2/4$ `// Δ²/4 is the largest variance due to stochastic rounding`
if $v > v_0$ **then**
  $x \leftarrow \mu + \sqrt{v - v_0}\,\xi$, where $\xi \sim \mathcal{N}(0, I_P)$
  $r \leftarrow x - Q^d(x)$
  foreach i **do**
    sample c_i from $\mathrm{Cat}(|r_i|, v_0)$ as in (5.10)
    `// c_i, r_i and θ_i are ith coordinates of c, r and θ, respectively`
  $\theta \leftarrow Q^d(x) + \mathrm{sign}(r) \odot c$
else
  $r \leftarrow \mu - Q^s(\mu)$
  foreach i **do**
    $v_s \leftarrow \left(1 - \frac{|r_i|}{\Delta}\right) r_i^2 + \frac{|r_i|}{\Delta}\left(-r_i + \mathrm{sign}(r_i)\Delta\right)^2$
    if $v > v_s$ **then**
      sample c_i from $\mathrm{Cat}(0,\, v - v_s)$ as in (5.10)
      $\theta_i \leftarrow Q^s(\mu)_i + c_i$
    else
      $\theta_i \leftarrow Q^s(\mu)_i$
clip θ if outside representable range
return θ

Our variance-corrected quantization function Q^{vc} always guarantees the correct mean, $\mathbb{E}(\theta_{k+1,i}) = \theta_{k,i} - \alpha\nabla\tilde{U}(\theta_k)_i$, and further guarantees the correct variance $\mathrm{Var}(\theta_{k+1,i}) = 2\alpha$ most of the time except when $v = 2\alpha < v_s$, where v_s is defined in Algorithm 13. However, that case rarely happens in practice, because the stepsize has to be extremely small. Besides, our quantization is simple to implement, and its computation cost is negligible compared to gradient computation. Although Q^{vc} only preserves the correctness of the first two moments (i.e., mean and variance), this does not affect the performance much in both theory and practice, as shown by Theorem 5.3 and later numerical evidence.

Theorem 5.3. *We run VC SGLDLP-L as in* (5.11)*. Besides the same assumptions in Theorem 5.1, we further assume the gradient is bounded* $\mathbb{E}\left[\left\|Q_G(\nabla\tilde{U}(\theta_k))\right\|_1\right] \le G$*. Let* $v_0 = \Delta_W^2/4$*. Then*

$$\begin{aligned} W_2(\mu_K, \pi) &\le (1-\alpha m)^K W_2(\mu_0, \pi) + 1.65(M/m)(\alpha P)^{1/2} \\ &+ \min\left(\frac{\Psi A}{m}, \frac{M\sqrt{A}}{m}\right) + \sqrt{\frac{\alpha\Delta_G^2 P + 4\alpha\kappa^2}{4m} + \frac{A}{\alpha m}} + \left((1-\alpha m)^K + 1\right)\sqrt{A}, \end{aligned}$$

	Test Errors (%)		ECE ↓(%)	
	CIFAR-10	CIFAR-100	CIFAR-10	CIFAR-100
32-bit Floating Point				
SGLDFP	4.65 ±0.06	22.58 ±0.18	1.11	3.92
SGDFP	4.71 ±0.02	22.64 ±0.13	2.53	4.97
8-bit Fixed Point				
VC SGLDLP-L	7.13 ±0.01	26.62 ±0.16	0.60	3.19
SGDLP-L	8.53 ±0.08	28.86 ±0.10	3.40	10.38
SGLDLP-F	5.12±0.06	23.30 ±0.09	1.12	4.42
SGDLP-F	5.20 ±0.14	23.84 ±0.12	3.05	6.80
8-bit Floating Point				
VC SGLDLP-L	5.51 ±0.01	25.22 ±0.18	0.60	5.82
SGDLP-L	5.86 ±0.18	26.19 ±0.11	4.23	12.97
SGLDLP-F	4.58 ±0.07	22.59 ±0.18	1.19	3.78
SGDLP-F	4.75 ±0.05	22.90 ±0.13	2.76	5.20

Table 5.1: Test Error and ECE on CIFAR dataset with the ResNet-18 model.

$$\textit{where } A = \begin{cases} 5v_0P, & \textit{if } 2\alpha > v_0 \\ \max\left(2\Delta_W \alpha G, 4\alpha P\right), & \textit{otherwise} \end{cases}$$

This theorem shows that when the stepsize $\alpha \to 0$, VC SGLDLP-L converges to the target distribution up to an error instead of diverging. Moreover, VC SGLDLP-L converges to the target distribution in $\mathcal{O}(\sqrt{\Delta_W})$, which is equivalent to the convergence rate of SGD with low-precision gradient accumulators to the optimum [1028, 1797].

Numerical Experiments We demonstrate the generalization accuracy and uncertainty estimation of low-precision SGLD with full-precision gradient accumulators (SGLDLP-F) and with variance-corrected low-precision gradient accumulators (VC SGLDLP-L) on the ResNet-18 model on the CIFAR datasets [933, 934]. We use the same quantization for weights, gradients, activations, and backpropagation errors unless otherwise stated. For all experiments, SGLD collects samples from the posterior of the model's weight and obtains the prediction on test data by Bayesian model averaging. We mainly compare low-precision SGLD with low-precision SGD, which is commonly used for frequentist low-precision deep learning [1577, 1578]. We use SGDFP, SGDLP-F, and SGDLP-L to denote full-precision SGD, low-precision SGD with full-precision gradient accumulators, and low-precision SGD with low-precision gradient accumulators, respectively. The numerical results are provided in Table 5.1, which reports average performance over 3 runs with the standard error.

1. Fixed Point: We use 8-bit fixed point for weights and gradients but full-precision for activations since we find low-precision activations significantly harm the performance. We find that, SGLDLP-F is better than SGDLP-F and VC SGLDLP-L significantly outperforms SGDLP-L across datasets and architectures. Notably, the improvement of SGLD over SGD becomes larger when using more low-precision arithmetic. For example, on CIFAR-100, VC SGLDLP-L outperforms SGDLP-L by 2.24%, SGLDLP-F outperforms SGDLP-F by 0.54% and SGLDFP outperforms SGDFP by 0.06%. This demonstrates that SGLD is particularly compatible with low-precision deep learning because of its natural ability to handle system noise.

2. Block Floating Point: We also consider block floating point (BFP) which is another common number type and is often preferred over fixed point on deep models due to less quantization error caused by overflow and underflow [1543]. Following the block design in Yang et al. [1797], we use *small-block* for ResNet. The Q^{vc} function naturally generalizes to BFP and only needs a small modification (see Zhang et al. [1856] for the detailed algorithm of Q^{vc} with BFP). By using BFP, the results of all low-precision methods improve over fixed point. SGLDLP-F can match the performance of SGLDFP with all numbers quantized to 8-bit except gradient accumulators. Again, SGLDFP-F and VC SGLDLP-L outperform their SGD counterparts on all tasks, suggesting the general applicability of low-precision SGLD with different number types.

3. Expected Calibration Error: Besides generalization performance, we further report the results of expected calibration error (ECE) [687] to demonstrate the uncertainty estimation of low-precision SGLD. In Table 5.1, we observe that SGLDLP-F and VC SGLDLP-L achieve almost the same or even lower ECE than full-precision SGLD, showing the ability of SGLD to give well-calibrated predictions does not degenerate due

to using low-precision. VC SGLDLP-L sometimes gives lower ECE than SGLDLP-F which may be due to the regularization effect of low-precision arithmetic. In contrast, the ECE of low-precision SGD increases significantly compared to full-precision SGD, implying that the quantization error makes the standard DNNs even more overconfident, which might lead to wrong decisions in real-world applications.

5.2.3 Low-Precision Stochastic Gradient Hamiltonian Monte Carlo

In this section, we combine low-precision arithmetic with SGHMC algorithm (5.3). Recall that SGHMC is equivalent to a discretized underdamped Langevin algorithm with stochastic gradient. The stationary distribution of underdamped Langevin dynamics is defined on the augmented probability space: $\pi(\theta, v) \propto \exp(-U(\theta) - \|v\|_2^2/2u)$. Therefore, our theoretical investigation will investigate the 2-Wasserstein distance[1] between the distribution of (θ_K, v_K) and $\pi(\theta, v)$.

Adopting the update rule in equations (5.3), we introduce low-precision SGHMC with full gradient accumulators (SGHMCLP-F) as follows:

$$\begin{aligned} v_{k+1} &= v_k e^{-\gamma\eta} - u\gamma^{-1}(1 - e^{-\gamma\eta})Q_G(\nabla\tilde{U}(Q_W(\theta_k))) + \xi_k^v \\ \theta_{k+1} &= \theta_k + \gamma^{-1}(1 - e^{-\gamma\eta})v_k + u\gamma^{-2}(\gamma\eta + e^{-\gamma\eta} - 1)Q_G(\nabla\tilde{U}(Q_W(\theta_k))) + \xi_k^\theta, \end{aligned} \tag{5.12}$$

where the distributions of ξ^v and ξ^θ follow (5.4). SGHMCLP-F keeps full-precision parameters v_k, θ_k at each iteration and quantizes them to low-precision representations before taking the gradient. Following the previous section, we still assume strong convexity of $U(\theta)$. In addition, we denote $\theta^* = \arg\min_\theta U(\theta)$ and $\rho = M/m$, and assume $\Delta_G = \Delta_W = \Delta$ for the simplicity of analysis presented in the following sections.

Theorem 5.4. *Under assumptions of Theorem 5.1, we assume that the minimum of $U(\theta)$ satisfies $\|\theta^*\|^2 < \mathcal{D}^2$. Given any sufficiently small $\epsilon > 0$, if we set the step size to be*

$$\eta = \min\left\{ \frac{\epsilon\rho^{-1}}{\sqrt{479232/5(P/m + \mathcal{D}^2)}}, \frac{\epsilon^2}{1440\rho u^2[(M^2+1)\frac{\Delta^2 P}{4} + \kappa^2]} \right\},$$

then after K steps starting with initial points $\theta_0 = v_0 = 0$, the distribution (θ_K, v_K) of the SGHMCLP-F (5.12), denoted by μ_K, satisfies

$$W_2(\mu_K, \pi) \le \tilde{\mathcal{O}}(\epsilon + \Delta),$$

for some K satisfying

$$K \le \frac{\rho}{\eta}\log\left(\frac{36\left(\frac{P}{m} + \mathcal{D}^2\right)}{\epsilon}\right) = \tilde{\mathcal{O}}\left(\epsilon^{-2}\log(\epsilon^{-1})(\Delta^2+1)\right),$$

where the big-O notation $\tilde{\mathcal{O}}$ gives explicit dependence on the quantization error Δ but hides multiplicative terms that polynomially depend on the other parameters (e.g., dimension P, friction γ, inverse mass u, and gradient variance κ^2).

Theorem 5.1 implies that for a strongly log-concave target distribution, the low-precision SGLDLP-F also achieves ϵ accuracy within $\tilde{\mathcal{O}}\left(\epsilon^{-2}\log(\epsilon^{-1})(\Delta^2+1)\right)$ iterations with learning rate of order $\epsilon^2/(\Delta^2+1)$. Thus, under strong convexity, SGHMCLP-F achieves no better convergence speed than SGLDLP-F. This is not surprising, since the quantization applied to the gradients in the full-precision gradient accumulator algorithm is equivalent to adding extra noise to the stochastic gradients. As theoretically shown by Cheng et al. [287] for a strongly-log-concave target distribution, SGHMC doesn't exhibit any advantage over the overdamped Langevin algorithm when stochastic gradients are used.

To reduce the computation cost, the SGHMC with low-precision gradient accumulators (SGHMCLP-L) can be trivially implemented as

$$\begin{aligned} v_{k+1} &= Q_W\left(v_k e^{-\gamma\eta} - u\gamma^{-1}(1 - e^{-\gamma\eta})Q_G(\nabla\tilde{U}(\theta_k)) + \xi_k^v\right), \\ \theta_{k+1} &= Q_W\left(\theta_k + \gamma^{-1}(1 - e^{-\gamma\eta})v_k + u\gamma^{-2}(\gamma\eta + e^{-\gamma\eta} - 1)Q_G(\nabla\tilde{U}(\theta_k)) + \xi_k^\theta\right). \end{aligned} \tag{5.13}$$

[1]The 2-Wasserstein distance between two probability measures μ and ν is defined as $W_2(\mu,\nu) = \left(\int_{\gamma\Gamma(\mu,\nu)} \|x-y\|^2 d\gamma(x,y)\right)^{1/2}$, where $\Gamma(\mu,\nu)$ denotes the set of joint distributions whose marginals are exactly μ and ν.

Similar to SGLDLP-L, SGHMCLP-L has intrinsic flaws. It directly quantizes the weights after each update, hence a small stepsize update is often quantized to zero update of θ and v, resulting in the sample distribution converging to a Dirac distribution at the initial point. In such cases, ensuring convergence becomes challenging. Wang et al. [1719] investigated the convergence of SGHMCLP-L under 2-Wasserstein distance, exhibiting that the naïve SGHMCLP-L has a much worse convergence upper bound than SGHMCLP-F. To remedy this, we adapt the variance correction quantizer (Algorithm 13) to SGHMCLP-L. Let $\mathbb{V}_v^{hmc} = u(1-e^{-2\gamma\eta})$ and $\mathbb{V}_\theta^{hmc} = u\gamma^{-2}(2\gamma\eta + 4e^{-\gamma\eta} - e^{-2\gamma\eta} - 3)$, which are the desired variances induced from the underdamped Langevin dynamics (5.3). The VC SGHMCLP-L can be done as follows:

$$\begin{aligned} v_{k+1} &= Q^{\text{vc}}\left(v_k e^{-\gamma\eta} - u\gamma^{-1}(1-e^{-\gamma\eta})Q_G(\nabla\tilde{U}(\theta_k)), \mathbb{V}_v^{hmc}, \Delta\right), \\ \theta_{k+1} &= Q^{\text{vc}}\left(\theta_k + \gamma^{-1}(1-e^{-\gamma\eta})v_k + u\gamma^{-2}(\gamma\eta + e^{-\gamma\eta} - 1)Q_G(\nabla\tilde{U}(\theta_k)), \mathbb{V}_\theta^{hmc}, \Delta\right). \end{aligned} \tag{5.14}$$

Next, we present the convergence result for VC SGHMCLP-L under a strongly log-concave target distribution.

Theorem 5.5. *Under assumptions of Theorem 5.1, we assume that the minimum of $U(\theta)$ satisfies $\|\theta^*\|^2 < \mathcal{D}^2$. Given any sufficiently small $\epsilon > 0$, if we set the stepsize to be*

$$\eta = \min\left\{\frac{\epsilon}{479232/5\left(\frac{P}{m} + \mathcal{D}^2\right)\rho}, \frac{\epsilon^2}{90u^2\Delta^2 P\rho + 360u^2\kappa^2\rho}\right\}$$

after K steps starting from the initial point $\theta_0 = v_0 = 0$, the distribution (θ_K, v_K) of the VC SGHMCLP-L (5.14) satisfies

$$W_2(\mu_K, \pi) = \tilde{\mathcal{O}}\left(\epsilon + \sqrt{\Delta}\right), \tag{5.15}$$

for some K satisfied

$$K \leq \frac{\rho}{\eta}\log\left\{\frac{36\left(\frac{P}{m} + \mathcal{D}^2\right)}{\epsilon}\right\} = \tilde{\mathcal{O}}\left(\epsilon^{-2}\log(\epsilon^{-1})(\Delta^2 + 1)\right).$$

Theorem 5.5 shows that the variance corrected quantization function resolves the overdispersion problem for the naïve SGHMCLP-L algorithm. The W_2 distance between the sample distribution and target distribution can be arbitrarily close to $\tilde{\mathcal{O}}(\sqrt{\Delta})$. This convergence rate of VC SGHMCLP-L is the same as VC SGLDLP-L (see the discussion after Theorem 5.3).

5.2.4 Advantage of LP Stochastic Gradient HMC under Non-Convexity

So far, we have introduced the low-precision SGLD and SGHMC algorithms and explored their theoretical convergence behaviour. The theorems in previous sections don't showcase any advantage of low-precision SGHMC over low-precision SGLDLP for strongly log-concave distributions. This doesn't align with the common understanding of SGHMC performance. Cheng et al. [287], Gao et al. [575] showed that SGHMC converges to its target distribution faster than the best-known convergence rate of SGLD in the 2-Wasserstein distance under both strongly log-concave and non-log-concave assumptions. Beyond this, SGHMC is analogous to stochastic gradient methods augmented with momentum, which is shown to have more robust updates w.r.t. gradient estimation noise [1069]. Since the quantization-induced stochastic error in low-precision updates acts as extra gradient noise, we expect that low-precision SGHMC can perform better than low-precision SGLD under certain scenarios beyond strongly convexity of $U(\theta)$.

Wang et al. [1719] explored the convergence of low-precision SGHMC and SGLD under non-log-concave distributions. Instead of assuming $U(\theta) - U(\theta') - \nabla U(\theta')^\intercal(\theta - \theta') \geq (m/2)\|\theta - \theta'\|_2^2$, the *dissipative* condition is considered:

$$\langle\nabla U(\theta), \theta\rangle \geq m\|\theta\|^2 - b, \quad \text{for any } \theta \in \mathbb{R}^P.$$

Such an assumption is commonly used [575, 1362, 1886] in the analysis of sampling from non-log-concave distributions. The theorem below summarizes the theoretical findings of Wang et al. [1719], which emphasize how the convergence rate depends on the quantization gap Δ.

	Test Errors (%)		ECE ↓(%)	
	CIFAR-10	CIFAR-100	CIFAR-10	CIFAR-100
32-bit Floating Point				
SGHMCFP	4.78 ±0.08	22.37 ±0.04	0.72	1.52
8-bit Fixed Point				
VC SGHMCLP-L	6.60 ±0.06	26.43 ±0.19	0.70	2.44
SGHMCLP-F	5.08±0.08	23.54 ±0.10	1.11	1.92
8-bit Floating Point				
VC SGHMCLP-L	5.15 ±0.08	24.45 ±0.16	0.67	5.02
SGHMCLP-F	4.93 ±0.09	22.39 ±0.11	1.12	3.65

Table 5.2: Test Error and ECE on CIFAR dataset with the ResNet-18 model.

Theorem 5.6. *Under assumptions* (5.7) *except for the strongly convexity and dissipative assumption, there exist respective choices of tuning parameter (e.g., stepsize) and the number of iterations K, such that one can achieve*

$$\begin{aligned} W_2(\mu_K, \pi) &= \tilde{\mathcal{O}}\left(\Delta^{e/(1+2e)}\right), \text{ for SGHMCLP-F,} \\ W_2(\mu_K, \pi) &= \tilde{\mathcal{O}}\left(\Delta^{e/(2+2e)}\right), \text{ for SGLDLP-F,} \\ W_2(\mu_K, \pi) &= \tilde{\mathcal{O}}\left(\Delta^{e/(2+4e)}\right), \text{ for VC SGHMCLP-L,} \\ W_2(\mu_K, \pi) &= \tilde{\mathcal{O}}\left(\Delta^{e/(6+6e)}\right), \text{ for VC SGLDLP-L.} \end{aligned} \tag{5.16}$$

where e is Euler's number.

Note that $\Delta < 1$ in general, thus Theorem 5.6 suggests that HMC-based low-precision sampling algorithms have better dependence on the quantization error than SGLD-based algorithms for non-log-concave distributions.

Numerical Experiments Similarly to the experiment settings for low-precision SGLD, we consider image tasks CIFAR-10 and CIFAR-100 on the ResNet-18. The simulation results are summarized in Table 5.2. Together with Table 5.1, it shows that across all evaluation metrics, the low-precision SGHMC has either a better or comparable performance with its SGLD counterpart.

5.3 Post-Training Low-Precision Sampling

The previous section focuses on training low-precision Bayesian DNN models directly by SG-MCMC from scratch. In contrast, it is feasible first to train a full-precision (Bayesian) DNN and then obtain a low-precision Bayesian DNN model via additional processing, such as fine-tuning, over the pre-trained model. More relevant content about model compression with quantization can be found in Chapter 41

5.3.1 Quantized Pre-Trained Bayesian models

To convert a pre-trained Bayesian DNN to its low-precision counterpart, one common approach is to quantize the sampled weights yielded by the full-precision posterior sampling (e.g., from SG-MCMC algorithms). In simple words, let $\theta^{(1)}, \ldots, \theta^{(T)}, \ldots$ be the posterior samples in $\mathbb{R}^P$, then a post-training quantization technique is applied to each sample.

This process seems trivial, but there are multiple factors that need to be considered.

Sampling Thinning Sample thinning refers to the selection of a subset from the original posterior samples. That is, only a proportion of samples will be quantized and used for low-precision Bayesian model average-based inferences, for the sake of computational efficiency. One would also avoid samplers from the burn-in period of posterior sampling to reduce the sampling bias. One common way to include one sample every other f samples (where f denotes the thinning frequency), skipping the first b samples (where b denotes the burn-in period length).

To determine the best thinned sample set, one needs to assess the quality of the samples, say how well the empirical distribution determined by this sample set resembles the true posterior distribution. To this end, we

employ the Kernel Stein Discrepancy [KSD, 652]. KSD is a distance measure between two distributions, where the target distribution is continuously differentiable with an intractable normalization constant. Given a kernel function $k(\cdot,\cdot)$, we define the *Stein Kernel* as

$$\begin{aligned} k_p(\theta,\vartheta) =& \langle \nabla_\theta, \nabla_\vartheta k(\theta,\vartheta)\rangle - \langle \nabla_\theta U(\theta), \nabla_\vartheta k(\theta,\vartheta)\rangle \\ & - \langle \nabla_\vartheta U(\vartheta), \nabla_\theta k(\theta,\vartheta)\rangle + \langle \nabla_\theta U(\theta), \nabla_\vartheta U(\vartheta)\rangle k(\theta,\vartheta). \end{aligned}$$

Then, the KSD between (the empirical distribution of) a sample $\theta^{(1)},\ldots,\theta^{(T)}$ and the target distribution is

$$\mathbb{S}(\theta^{(1)},\ldots,\theta^{(T)}) = \frac{1}{T^2}\sum_{i,j=1}^{T} k_p(\theta^{(i)},\theta^{(j)}).$$

KSD can be used to search for the optimal choices of b and f or the whole subset [1393]. However, it is worth mentioning that the computation of the Stein Kernel scales up quickly with the dimension d, thus it could be very expensive for huge models.

Calibration of Quantization Range and Gap If training low-precision (Bayesian) DNNs from scratch (e.g., Section 5.2), one usually needs to determine the quantization scheme (e.g., the quantization gap Δ in the quantization functions Q^d or Q^s) beforehand. In contrast, when a pre-trained model is available (e.g., a full-precision posterior sample $\theta^{(t)}$), we have direct access to the distribution of its weight values; we can use a representative dataset to approximate the distribution of activations along the forward propagation. This information can help us choose an appropriate quantization scheme to quantize this $\theta^{(t)}$ that fits the computational / memory budgets and has a small impact on its predictive performance. Note that in Section 5.2, we only consider a rounding mechanism that maps a high-precision (e.g., full floating-point value) to its near low-bit fixed-point values. But in general, any mapping from real values to a discrete space can be used. For example, an uniform quantization [827] with W-bit quantization is defined as

$$Q^u(\theta) = \left\lfloor \frac{\text{clip}(\theta,a,b)-a}{s} \right\rceil s + a, \text{ with } s = (b-a)/(2^W-1),$$

where $\lfloor\cdot\rceil$ denotes rounding to the nearest integer. Then a and b, which denote the quantization range, can be set as the minimum and maximum values among entries of the pre-trained model $\theta^{(t)}$.

Parameter Fine-tuning Although a quantized network $Q(\theta^{(t)})$ for some quantization function Q is ready for inference, it may suffer significant performance drops. In such a case, one can first fine-tune the model $\theta^{(t)}$ for a small number of gradient descent steps to compensate for the performance drops caused by quantization, then apply quantization to the fine-tuned model.

Let $f(x;\theta)$ be the neural network model parameterized by the weight vector θ, then the empirical loss is defined as

$$\mathcal{L}(\theta) = -\sum_{(x,y)\in\mathcal{D}} \log p(x,y|\theta) := \sum_{(x,y)\in\mathcal{D}} l(y, f(x;\theta)),$$

for some loss function $l(\cdot,\cdot)$. Instead of running fine-tuning gradient descent on $\mathcal{L}(\theta)$, we design a new neural network model $f^q(\cdot;\theta)$ which approximates $f(\cdot;\theta)$ with quantization applied on weights and activations. For example, Ferianc et al. [515], Jacob et al. [827] insert simulated quantization modules after weights input and every tensor product operation. Therefore, running gradient descent on $\mathcal{L}^q = \sum l(y, f^q(x;\theta))$ initialized from $\theta^{(t)}$, leads to a nearby model whose after-quantization performance is acceptable.

Quantize Pre-trained Variational Bayesian DNNs If a pre-trained variational Bayesian neural network model (refer to Book Part III for details) is given, one may additionally quantize the variational parameters [515, 1571]. Suppose that the variational posterior distribution of one weight entry $\theta_{(i)}$ is $\mathrm{N}(\mu,\sigma^2)$, then given a quantization function Q, the quantized posterior sample of $\theta_{(i)}$ is generated by $Q(Q(\mu)+Q(\sigma)\xi)$ where ξ is a standard Gaussian variable. To accommodate performance degradation induced by quantization, one can also incorporate additional fine-tuning for the variational parameters [515], similarly to the procedure discussed in the above paragraph.

5.3.2 Low-Bit Sub-Distribution

As discussed earlier, a pre-trained full-precision model can provide valuable insights into the distribution of weight values and inform the design of a quantization scheme. The SG-MCMC algorithms in Section 5.2 yield posterior samples whose entries belong to an equally spaced discrete space $\mathcal{S} = \{0, \pm\Delta, \pm 2\Delta, \dots\}$. When a pre-trained model is available, one can train a flexible discrete space $\mathcal{S}$ from which the posterior samples are drawn. The elements in $\mathcal{S}$ can be full floating-point values; thus, model weights of a neural network achieve low-bit representation by sharing the same full precision values in $\mathcal{S}$. This can be viewed as a generalization of sparse DNN, which encourages network weights to share a common value of 0.

Given a pre-trained DNN model θ, a straightforward method is to implement a clustering algorithm over the entries of θ and obtain the cluster centers $\mathcal{S} = \{q_1, \dots, q_M\}$. Then we would like to restrict the posterior samples to $\mathcal{S}$. To accomplish this, we can adapt the mean-field variational Bayes (refer to Chapter 12 for details) and define the discrete variational distribution as

$$q(\theta) = \prod_{i=1}^{P} \sum_{m=1}^{M} p_{im} \mathbf{1}(\theta_{(i)} = q_m), \quad \theta = (\theta_{(1)}, \dots, \theta_{(P)}) \tag{5.17}$$

where variational parameters satisfy $\sum_m p_{im} = 1$. To facilitate the training of variational inference, we can introduce parametric modelling for p_{im} (e.g., p_{im} relates to $|\theta_{(i)} - q_m|$, where $\theta_{(i)}$ is the pre-trained weight). To enhance the model performance, we can also allow $q_1, \dots, q_M$ to update in the variational training procedure [see e.g., 439].

It is worth mentioning that no actual low-precision sampling happens in the variational training process since variational inference boils down to an optimization problem for the variational parameters. Once these parameters are learned, the (approximate) posterior samples are easily drawn from (5.17).

5.4 Conclusions and Discussions

This chapter introduced low-precision sampling algorithms that are useful for modern Bayesian deep learning applications, including SG-MCMC-based direct low-precision samplers and post-training low-precision sampling techniques. These low-precision methods are crucial for deploying large models on resource-constrained hardware and improving Bayesian inference speed.

As discussed in the introduction section, one main reason for pursuing low-precision arithmetics is to reduce the memory and computation costs for DNN training or DNN inference. Beyond this, the low-precision representation is also helpful to reduce communication cost for scenarios where frequent information transfer among several parties is required, but only with limited bandwidth. One such emerging research field is federated learning (FL) [859]. Federated learning is a learning paradigm where multiple parties (e.g., hospitals or edge devices) collaboratively train a global model while keeping the local training data private and decentralized. In a federated SGLD algorithm, each party computes the gradient based on its local data, and all local gradients are shared and aggregated via communication. The aggregated gradient is then used in the SGLD update (5.1) of the global model. Low-precision representations can be used to compress the gradient communications. Vono et al. [1688] utilized a stochastic rounding mechanism and studied how quantization error impacts the convergence of the global Bayesian model.

The theorems and algorithms in this chapter are mostly depicted regarding the theme that quantization of the weight parameter θ suffices. However, neural networks have a multilayer structure, and quantization can be employed on each layer along the forward propagation process (i.e., online conversion of activation values to low-precision representations). In addition, normalization layers, such as Batch Normalization and Layer Normalization, are often sensitive to quantization error and may require different techniques (e.g., BatchNorm folding) to mitigate the impact of quantization. These issues are less explored and discussed in the field of Bayesian low-precision DNNs. Interested readers may refer to Gholami et al. [615], Kim et al. [889].

Chapter 6

Discrete sampling

Many tasks within deep learning involve sampling over discrete spaces. Such sampling tasks are distinct from sampling over continuous spaces: while the primary frameworks of sampling—Markov chain Monte Carlo (MCMC), rejection sampling, and importance sampling—introduced in Chapter 1 can be applied to discrete spaces by adjusting the proposal distribution, this adjustment is often highly non-trivial. In this chapter, we focus on high-dimensional targets whose unnormalised log density can be evaluated but whose state spaces are too large to enumerate. Such targets arise in binary energy-based models, categorical latent-variable models, and neural models over combinatorial structures.

6.1 Introduction

Discrete sampling is notably difficult due to the sparse, discontinuous, and combinatorial nature of discrete spaces. If a state consists of d categorical variables with K possible values each, its state space may contain K^d configurations. In principle, one could evaluate the unnormalised density at every configuration, normalise the resulting weights, and sample from the corresponding categorical distribution. This enumeration-based approach becomes infeasible as the dimension grows. Standard local MCMC methods avoid full enumeration, but target-independent proposals can mix slowly because they do not distinguish promising moves from moves into low-density regions.

As a running example, consider a binary energy-based model with $x \in \{0,1\}^d$ and unnormalised log density $U(x)$. The model assigns a score to each of its 2^d possible configurations, making enumeration infeasible for large d. Nevertheless, $U(x)$, and often the gradient of a differentiable extension of U, can be evaluated at any given configuration. We use this binary model to illustrate the methods below, although the framework also applies to the broader categorical and combinatorial settings described above.

Gradients are a staple of sampling within deep learning, as illustrated by the continuous-space Langevin methods in Section 1.5.1. Although a function defined only on a discrete space has no intrinsic gradient, many deep-learning targets are represented through a differentiable function on a continuous ambient space and evaluated at binary, categorical, or other discrete states. The gradient of this extension can provide a cheap source of local information about promising discrete moves.

We begin with an uninformed local MCMC proposal and show how pointwise-informed and locally balanced proposals incorporate target-specific information. We then use this framework to develop gradient-based discrete samplers, first for single-coordinate moves and then for parallel multi-coordinate moves. Finally, we discuss methods designed for larger-scale exploration.

6.2 Notation

We assume a discrete state space $\mathcal{X}$. We denote a state by $x = (x_1, \ldots, x_d) \in \mathcal{X}$, where each coordinate can take one of K possible values. Throughout the chapter, π denotes the normalised target distribution and U its unnormalised log density. Thus our target is

$$\pi(x) = \frac{1}{Z}\exp(U(x)), \quad Z = \sum_{x' \in \mathcal{X}} \exp U(x') \tag{6.1}$$

The normalising constant or partition function Z in Equation (6.1) generally requires a sum over as many as K^d states. This makes enumeration-based direct sampling computationally infeasible in high dimensions. MCMC

can nevertheless operate on differences such as $U(x') - U(x)$, in which Z cancels.

6.3 Locally Balanced Proposals

A simple Metropolis–Hastings approach proposes a nearby state using a target-independent kernel $K_\sigma(x' \mid x)$. For example, the kernel might choose a coordinate uniformly and replace its value. Such a proposal is inexpensive, but it assigns the same proposal probability to moves towards high- and low-density states. Large moves may explore quickly but often reach low-density states and are rejected; small moves are more likely to be accepted but can produce highly correlated samples and traverse the state space slowly.

For the running binary example, the simplest local kernel selects one of the d bits uniformly and flips it, without using the values of U to prioritise the available moves.

Zanella [1831] introduces a framework that addresses this trade-off by weighting the candidate moves from K_σ using target-density information. The resulting pointwise-informed proposal is

$$P_{g,\sigma}(x' \mid x) = \frac{g\left(\frac{\pi(x')}{\pi(x)}\right) K_\sigma(x' \mid x)}{Z_{g,\sigma}(x)}. \tag{6.2}$$

The pointwise-informed proposal in Equation (6.2) consists of the target-density ratio, a symmetric base kernel or weight K_σ whose scale controls the size of candidate moves, a balancing function g that weights moves using the density ratio, and the state-dependent normaliser $Z_{g,\sigma}(x)$. The base kernel may be specified only up to a state-independent multiplicative constant, because that constant cancels through $Z_{g,\sigma}(x)$. We assume $K_\sigma(x' \mid x) = K_\sigma(x \mid x')$. The limit $\sigma \to 0$ represents a family of kernels that becomes increasingly concentrated at the current state.

The roles of invariance and ergodicity in MCMC are reviewed in Section 1.5, while detailed balance and the Metropolis–Hastings construction are discussed in Section 1.5.1. In the present setting, $P_{g,\sigma}$ is reversible with respect to $\pi_{g,\sigma}$ if it satisfies detailed balance,

$$\pi_{g,\sigma}(x) P_{g,\sigma}(x' \mid x) = \pi_{g,\sigma}(x') P_{g,\sigma}(x \mid x').$$

Detailed balance is sufficient for $\pi_{g,\sigma}$ to be invariant under the transition. Invariance alone does not ensure convergence from an arbitrary initial state. For a discrete chain, irreducibility means that every state with positive target probability can eventually be reached from every admissible initial state, while aperiodicity means that the chain is not constrained to return according to a fixed cycle. Together with invariance and the relevant regularity conditions, these properties yield an ergodic chain. The general role of ergodicity in MCMC is reviewed in Section 1.5.

A pointwise-informed proposal is called *locally balanced* if the transition kernel $P_{g,\sigma}$ is reversible with respect to some distribution $\pi_{g,\sigma}$ such that $\pi_{g,\sigma}$ converges weakly to π as $\sigma \to 0$ [1831]. In plain terms, as the proposal becomes increasingly local, the invariant distribution of the uncorrected informed proposal approaches the desired target. Zanella [1831] demonstrates that a pointwise-informed proposal is locally balanced if and only if the balancing function satisfies $g(t) = tg(1/t)$.

If this property holds for the function g, then the locally balanced proposal will be reversible with respect to the following distribution without any correction:

$$\pi_{g,\sigma}(x) = \frac{\pi(x) Z_{g,\sigma}(x)}{\sum_{x' \in \mathcal{X}} \pi(x') Z_{g,\sigma}(x')}.$$

Here, $Z_{g,\sigma}$ is defined as follows:

$$Z_{g,\sigma}(x) = \sum_{x' \in \mathcal{X}} g\left(\frac{\pi(x')}{\pi(x)}\right) K_\sigma(x' \mid x).$$

For a local kernel satisfying $K_\sigma(x' \mid x) \to \mathbf{1}\{x' = x\}$ and the conventional normalisation $g(1) = 1$, we have $Z_{g,\sigma}(x) \to 1$. Consequently, $\pi_{g,\sigma}$ converges to π as $\sigma \to 0$ [1831].

At a non-zero scale, $\pi_{g,\sigma}$ need not equal π, so a Metropolis–Hastings correction is still required for exact sampling. Local balance makes the uncorrected proposal close to the target in the small-scale regime, which can improve the acceptance rate, while a non-zero σ is required to produce non-degenerate movement across the state space.

Zanella [1831] shows that locally balanced proposals can improve efficiency over uninformed local proposals in the asymptotic regimes considered there. Constructing an exact informed proposal nevertheless requires evaluating

the density ratio for every candidate. Full enumeration can involve K^d states; even a Hamming-one neighbourhood contains $d(K-1)$ candidates. The gradient-based methods below replace these repeated neighbouring-state evaluations with one gradient evaluation at the current state.

6.4 Gradient-Based Discrete Sampling

A function specified only on a discrete state space does not have an intrinsic gradient. Gradient-based discrete samplers therefore assume that the unnormalised log density U is supplied with, or represented through, a differentiable extension to a continuous ambient space. For example, a neural network may accept a continuous vector representation even though the sampler evaluates it only at binary or one-hot encoded states. This extension need not be unique, and its gradient is useful only insofar as it predicts changes between the discrete states of interest.

Given such an extension, the log-density difference for a nearby state can be approximated by

$$U(x') - U(x) \approx \nabla U(x)^\top (x' - x).$$

This approximation is most reliable when the extension is sufficiently smooth and x' is close to x; differentiability alone does not guarantee accuracy if the function changes sharply between neighbouring states. Since $\log \pi(x) = U(x) - \log Z$, their gradients are identical and the unknown normalising constant is not required. One forward and backward pass at x then supplies approximate scores for all local candidate moves, at the expense of Taylor-approximation error.

6.4.1 Gibbs with Gradient

Grathwohl et al. [659] first introduces this notion of gradient-based discrete sampling through the Gibbs-With-Gradient algorithm. This algorithm can be seen as a version of the standard Gibbs algorithm where the updated coordinate is selected based on the gradient.

Returning to the general locally balanced sampler, we can derive the Gibbs-With-Gradient update by choosing an uninformed kernel that uniformly selects a state differing from the current state in exactly one coordinate. Define the Hamming-one neighbourhood by

$$H(x) = \left\{ x' \in \mathcal{X} : \sum_{i=1}^{d} \mathbf{1}\{x'_i \neq x_i\} = 1 \right\}.$$

In the running binary example, $H(x)$ contains exactly the d states obtained by flipping one bit of x.

For this fixed-scale update, the uniform kernel is

$$K_\sigma(x' \mid x) = \frac{\mathbf{1}\{x' \in H(x)\}}{|H(x)|}.$$

When every coordinate has K possible categories, $|H(x)| = d(K-1)$. This factor is constant across candidate moves and is therefore absorbed into the proportionality below.

Next, we choose the balancing function $g(t) = \sqrt{t}$. Before approximation, this yields the exact informed proposal

$$q_{\text{exact}}(x' \mid x) \propto \mathbf{1}\{x' \in H(x)\} \sqrt{\frac{\pi(x')}{\pi(x)}}$$

Using the unnormalised log density U from Equation (6.1), this proposal can be expressed without evaluating Z:

$$q_{\text{exact}}(x' \mid x) \propto \mathbf{1}\{x' \in H(x)\} \exp\left(\frac{1}{2}(U(x') - U(x)) \right).$$

Replacing the exact log-density difference by its first-order Taylor approximation defines the Gibbs-With-Gradient proposal:

$$q_{\text{GWG}}(x' \mid x) \propto \mathbf{1}\{x' \in H(x)\} \exp\left(\frac{1}{2} \nabla U(x)^\top (x' - x) \right).$$

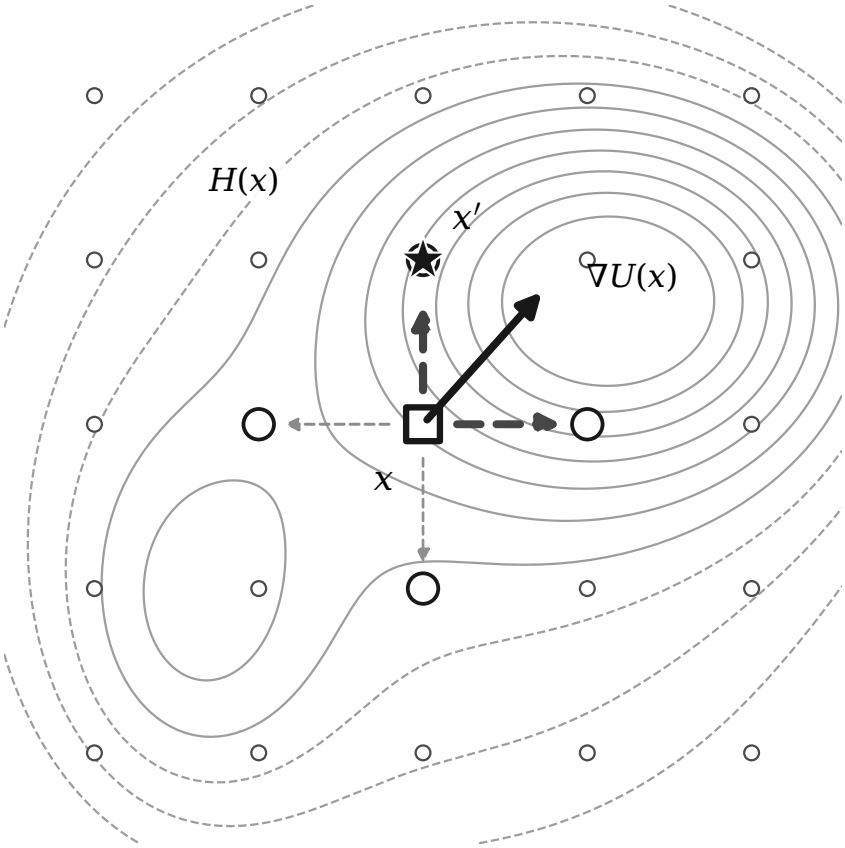


Figure 6.1: Gradient-informed sampling on a discrete state space. The gradient $\nabla U(x)$ of a differentiable extension provides local geometric information at the current state x, which induces different proposal weights over its Hamming-one neighbourhood $H(x)$. The starred state x' represents a draw from this proposal. Adapted from Grathwohl et al. [659].

Figure 6.1 illustrates this Gibbs-With-Gradient construction. The gradient at x assigns different approximate weights to the states in $H(x)$, and x' is then sampled from the resulting proposal over those discrete neighbours.

This can be computed quickly by viewing the update as the selection of a coordinate to modify. In the binary case, this update is always a bit flip. Here, Softmax normalises a vector of real-valued scores into probabilities, and Cat denotes the categorical distribution with those probabilities. The resulting proposal is

$$q_{\mathrm{GWG}}(\cdot \mid x) = \mathrm{Cat}\left(\mathrm{Softmax}\left(-\frac{1}{2}(2x-1)\odot \nabla U(x)\right)\right).$$

The categorical case requires selecting both a coordinate and its proposed category. Let c_i denote the current category of coordinate i. We represent each categorical coordinate by a one-hot vector when evaluating the gradient. For every alternative category $j \neq c_i$, define the approximate log-probability difference

$$\widetilde{d}(x)_{i,j} = \nabla_{x_{i,j}} U(x) - \nabla_{x_{i,c_i}} U(x).$$

The first term measures the local effect of assigning category j to coordinate i, while the second corresponds to its current category.

There are $d(K-1)$ possible coordinate–category updates. We define a categorical proposal over these updates by

$$q_{\mathrm{GWG}}(i, j \mid x) = \mathrm{Softmax}_{i\in[d],\, j\neq c_i}\left(\frac{1}{2}\widetilde{d}(x)_{i,j}\right).$$

After sampling $(i, j) \sim q_{\mathrm{GWG}}(\cdot, \cdot \mid x)$, the proposed state x' is obtained by changing coordinate i from c_i to j and leaving all other coordinates unchanged. The full categorical procedure is given in Algorithm 14.

Algorithm 14: Categorical Gibbs With Gradient Sampler

Inputs : initial categorical state $x = (x_1, \ldots, x_d) \in \mathcal{X}$; unnormalised log-density U
Outputs: sample sequence $\mathcal{S}$
$\mathcal{S} \leftarrow ()$
for *each iteration* **do**
  compute $\widetilde{d}(x)$ and $q_{\mathrm{GWG}}(\cdot, \cdot \mid x)$
  sample $(i, j) \sim q_{\mathrm{GWG}}(\cdot, \cdot \mid x)$
  $c_i \leftarrow$ current category of coordinate i
  $x' \leftarrow \mathrm{replace}(x, i, j)$
  compute the reverse-move probability $q_{\mathrm{GWG}}(i, c_i \mid x')$
  accept x' with probability

$$\min\left\{1, \exp\left(U(x') - U(x)\right)\frac{q_{\mathrm{GWG}}(i, c_i \mid x')}{q_{\mathrm{GWG}}(i, j \mid x)}\right\}$$

  update $x \leftarrow x'$ if accepted
  append x to $\mathcal{S}$
return $\mathcal{S}$

Because the gradient-based proposal uses a first-order approximation and is generally asymmetric, it does not by itself preserve the target distribution. We therefore apply the standard Metropolis–Hastings acceptance correction, described in Section 1.5.1, which ensures that the target distribution is invariant. Under the ergodicity conditions stated above, the resulting Markov chain converges to the target distribution.

6.4.2 Discrete Langevin Proposal

Gibbs-With-Gradient can outperform uninformed single-coordinate proposals by using the gradient to prioritise promising updates. Its Hamming-one restriction also keeps the Taylor approximation local, but only one coordinate can change per iteration, which may produce strongly correlated samples in high dimensions. Zhang et al. [1855] addresses this limitation with the Discrete Langevin Proposal (DLP), which retains gradient-based directionality while adding a distance penalty that permits multiple coordinates to be proposed simultaneously.

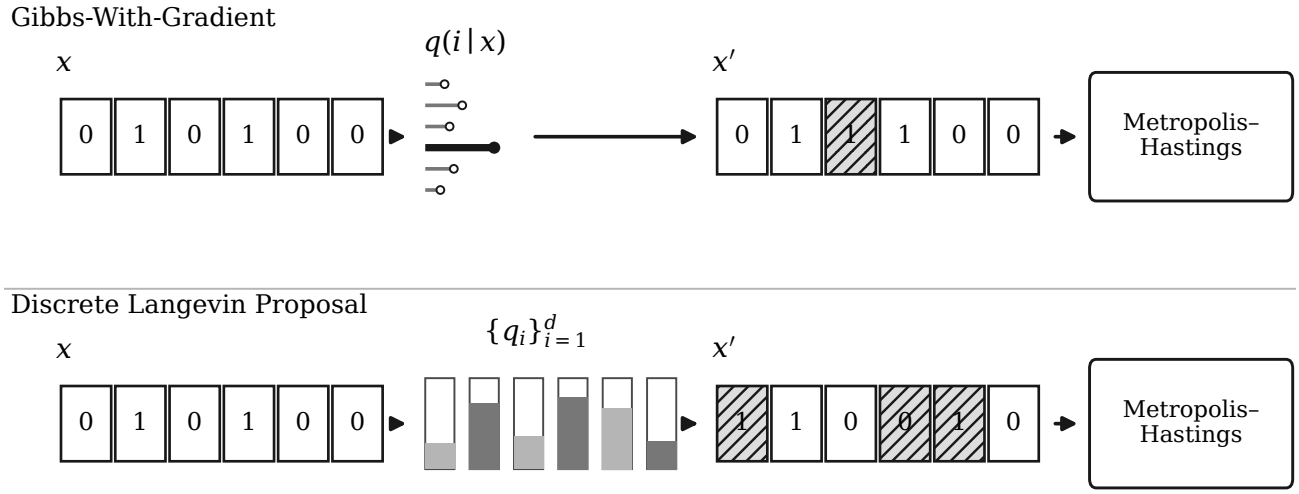


Figure 6.2: Comparison of gradient-based discrete proposals. Gibbs-With-Gradient samples one coordinate from $q_{\mathrm{GWG}}(i \mid x)$ and proposes a state that differs in that coordinate. The Discrete Langevin Proposal constructs coordinate-wise distributions $\{q_i\}_{i=1}^d$ and can propose several changes in parallel. The displayed Metropolis–Hastings correction gives the adjusted variants; omitting it yields DULA in the discrete Langevin case. Based on Grathwohl et al. [659] and Zhang et al. [1855].

In the running binary example, Gibbs-With-Gradient proposes at most one bit flip, whereas DLP constructs a proposal for every bit and may change several bits in the same iteration.

To derive the proposal, consider the discrete analogue of a Gaussian random-walk kernel,

$$K_\sigma(x' \mid x) \propto \exp\left(-\frac{1}{2\sigma}\|x' - x\|_2^2\right).$$

Combining this kernel with a first-order approximation of the locally balanced weight gives the DLP

$$q(x' \mid x) \propto \exp\left(\frac{1}{2}\nabla U(x)^\top (x' - x) - \frac{1}{2\sigma}\|x' - x\|_2^2\right). \tag{6.3}$$

The linear term in Equation (6.3) favours moves predicted to increase U, while the quadratic term penalises moves far from the Taylor expansion point. The step size σ controls the strength of this penalty.

Because the state space factorises as $\mathcal{X} = \prod_{i=1}^d \mathcal{X}_i$ and the exponent in Equation (6.3) is a sum of coordinate-wise terms, the proposal factorises as $q(x' \mid x) = \prod_{i=1}^d q_i(x_i' \mid x)$. Each factor is the categorical distribution

$$q_i(x_i' \mid x) = \mathrm{Cat}\left(\mathrm{Softmax}_{x_i' \in \mathcal{X}_i}\left[\frac{1}{2}\nabla_i U(x)(x_i' - x_i) - \frac{(x_i' - x_i)^2}{2\sigma}\right]\right).$$

Thus, larger values of σ permit more distant proposals, whereas smaller values concentrate the proposal near the current state.

While this may look similar to the proposal formula for Gibbs with Gradient, it is important to note that in DLP, the softmax is computed per coordinate over the potential classes that can be realised. In Gibbs with Gradient, the softmax is over the entire sample to pick only one coordinate. This allows for the parallel update of all coordinates.

The factorisation makes it possible to propose all coordinates in parallel after one gradient evaluation. Although the proposal factors coordinate-wise conditional on the current state, the gradient of the joint log density still guides each coordinate using information from the full current configuration. A Metropolis–Hastings correction can then account for the remaining approximation error and dependence structure. Figure 6.2 contrasts this parallel update with the single-coordinate update used by Gibbs-With-Gradient.

Using DLP yields two samplers: the discrete Metropolis-adjusted Langevin algorithm (DMALA), which includes a Metropolis–Hastings correction, and the discrete unadjusted Langevin algorithm (DULA), which omits it. DULA generally has asymptotic bias. For log-quadratic targets, however, Zhang et al. [1855] show that this

bias vanishes as $\sigma \to 0$; they also bound the bias for targets whose gradients are close to those of a log-quadratic distribution.

In terms of implementation, the cost difference between DULA and DMALA is negligible: while DMALA incorporates a Metropolis–Hastings correction that requires the computation of both $q(\cdot \mid x)$ and $q(\cdot \mid x')$, caching means that only one new proposal distribution is computed per step after initialisation. The cached version of DMALA is given in Algorithm 15.

Algorithm 15: DMALA with Cache

Inputs : initial state $x = (x_1, \ldots, x_d) \in \mathcal{X}$; unnormalised log-density U
Outputs: sample sequence $\mathcal{S}$
$\mathcal{S} \leftarrow ()$
compute $q(\cdot \mid x)$ and cache proposal
for *each iteration* **do**
  sample $x' \sim q(\cdot \mid x)$
  compute $q(\cdot \mid x')$ using DLP proposal
  accept x' with probability

$$\min\left\{1, \exp\left(U(x') - U(x)\right) \frac{q(x \mid x')}{q(x' \mid x)}\right\}$$

  if *accepted* **then**
    update cached proposal to $q(\cdot \mid x')$
    $x \leftarrow x'$
  append x to $\mathcal{S}$
return $\mathcal{S}$

The core logic lies in the fact that the x' of the current iteration is either rejected or becomes the x for the next sampling step. In the case of rejection, the proposal function $q(\cdot|x)$ does not change as x remains the same. In the case of acceptance, the proposal step simply gets updated to match the updated x' by using $q(\cdot|x') \to q(\cdot|x)$, since this proposal must be computed anyway for the accept/reject step. Thus at each step, only one proposal function must be computed.

6.5 Beyond Locally Balanced Proposals

Locally balanced proposals offer a convenient framework for incorporating gradients into discrete samplers, but their defining approximation concerns the small-scale limit $\sigma \to 0$. At finite scales, local proposals may take a long time to move between separated modes, necessitating longer chains or multiple chains.

For example, if the binary target places high probability on two configurations separated by many low-probability intermediate states, a sequence of single-bit proposals may struggle to move from one mode to the other.

This motivates proposals designed for non-local movement. Zanella [1831] already considers $g(t) = t$, calling the resulting proposal the *naively informed* choice because it has the form $P_{g,\sigma}(x' \mid x) \propto K_\sigma(x' \mid x)\pi(x')$. Building on this observation, Sun et al. [1574] formalise the large-scale interpretation: they call a proposal family *globally balanced* if the invariant distribution of its uncorrected informed proposal converges to π as $\sigma \to \infty$. They assume that the base kernel becomes uniform in this limit. Since a state-independent constant cancels in the normalisation of the informed proposal, they write this limit as $K_\sigma(x' \mid x) \to 1$; for a normalised kernel on a finite state space, the corresponding limit is $1/|\mathcal{X}|$. For $g(t) = t$, the limiting proposal is then

$$\lim_{\sigma \to \infty} P_{g,\sigma}(x' \mid x) = \frac{\pi(x')/\pi(x)}{\sum_{z \in \mathcal{X}} \pi(z)/\pi(x)} = \pi(x').$$

Thus the proposal becomes an independent draw from the target itself and is globally balanced in the sense of Sun et al. [1574]. Notice that $g(t) = t$ does not satisfy the locally balanced identity $g(t) = tg(1/t)$; local and global balance refer to different limiting regimes.

6.5.1 Any-scale Balanced Sampler

The any-scale balanced sampler introduced by Sun et al. [1574] makes two distinct modifications for proposals outside the local regime. First, it adjusts the density-ratio weighting to match the proposal scale. The family $g(t) = t^\beta$, with $\beta \in [1/2, 1]$, interpolates between the locally balanced choice $g(t) = \sqrt{t}$ as $\sigma \to 0$ and the globally balanced choice $g(t) = t$ as $\sigma \to \infty$. This scale-dependent relationship is illustrated schematically in Figure 6.3(a).

Adjusting β changes how strongly the proposal favours high-density candidates, but it does not reduce the error in a first-order approximation of $U(x') - U(x)$. The method therefore makes a second modification: it introduces a quadratic approximation intended to remain informative over larger moves. The log-density difference is approximated by

$$U(x') - U(x) \approx (x' - x)^\top \nabla U(x) + \frac{1}{2}(x' - x)^\top W(x' - x), \tag{6.4}$$

where W is a symmetric matrix shared across states. Setting $W = \nabla^2 U(x)$ would recover the usual second-order Taylor approximation locally, but repeatedly computing a Hessian is expensive. Instead, the sampler estimates a global W from pairs (x_n, x'_n) collected during burn-in:

$$\widehat{W} = \operatorname*{arg\,min}_{W = W^\top} \sum_n \left\| W(x'_n - x_n) - \big(\nabla U(x'_n) - \nabla U(x_n)\big) \right\|_2^2 .$$

Thus W is fitted to explain observed changes in the gradient and acts as a global curvature approximation. Scale-dependent balancing and curvature approximation address different problems: β controls the density-ratio weighting, whereas W improves the approximation used to score non-local candidates.

Substituting the quadratic approximation in Equation (6.4) and $g(t) = t^\beta$ into the informed proposal gives

$$q(x' \mid x) \propto \exp\left(\beta \left[(x' - x)^\top \nabla U(x) + \frac{1}{2}(x' - x)^\top W(x' - x) \right] - \frac{1}{2\sigma} \|x' - x\|_2^2 \right). \tag{6.5}$$

The quadratic term captures cross-coordinate interactions, but it also prevents direct coordinate-wise sampling. Sun et al. [1574] therefore introduce an augmented proposal with a Gaussian auxiliary variable. The augmented construction is used directly with the Metropolis–Hastings correction below.

Choose a diagonal positive semi-definite matrix D such that $W + D$ is positive semi-definite, and draw

$$Y \mid x \sim \mathcal{N}\left((W + D)^{1/2} x, I \right).$$

The matrix square root can be computed by an eigendecomposition or Cholesky factorisation. Conditional on Y, define

$$q(x' \mid x, Y) \propto \exp\left(\beta x'^\top \left[\nabla U(x) - Wx + (W + D)^{1/2} Y \right] - \frac{1}{2} x'^\top \left(\beta D + \frac{1}{\sigma} I \right) x' \right),$$

and sample x' from this distribution. Because D is diagonal and the state space factorises, this conditional proposal factorises across coordinates. Equations (6.5) and the augmented proposal express, respectively, the motivating quadratic approximation and the auxiliary-variable transition implemented by the sampler. The Metropolis–Hastings step accounts for the resulting augmented proposal exactly.

The Metropolis–Hastings acceptance probability for the augmented proposal is

$$A(x, x', Y) = \min\left\{ 1, \exp\big(U(x') - U(x)\big) \frac{q(Y \mid x')q(x \mid x', Y)}{q(Y \mid x)q(x' \mid x, Y)} \right\}.$$

The diagonal matrix D controls the conditional proposal variance. Sun et al. [1574] consider two semidefinite programs for choosing it. The AB-max variant minimises the largest eigenvalue of $W + D$:

$$(D^*, \lambda^*) = \operatorname*{arg\,min}_{D, \lambda} \lambda \quad \text{subject to} \quad D \succeq 0, \qquad 0 \preceq W + D \preceq \lambda I.$$

This criterion focuses on the worst spectral direction. The AB-trace variant instead solves

$$D^* = \operatorname*{arg\,min}_{D} \operatorname{tr}(D) \quad \text{subject to} \quad D \succeq 0, \qquad W + D \succeq 0.$$

The motivation for this objective comes from the continuous quadratic surrogate studied by Sun et al. [1574] in the special case $\sigma = \infty$ and $\beta = 1$. In that surrogate, the covariance of the proposal increment is $2D^{-1}$, so minimising

the trace of D encourages dispersion across coordinates. This covariance identity does not hold literally for the general discrete categorical proposal, whose dispersion also depends on β, σ, and the state space. The authors report that AB-trace performs better empirically and use it as their preferred variant.

The proposal parameters β and σ are correlated and target-dependent. Sun et al. [1574] therefore adapt them by alternating coordinate-wise updates that seek to maximise the empirical average jump distance. For one parameter type $a \in \{\sigma, \beta\}$, let ϕ denote its current value and define the feasible sets $\mathcal{I}_\sigma = (0, \infty)$ and $\mathcal{I}_\beta = [1/2, 1]$. With the other parameter held fixed, the adaptation block compares ϕ with feasible multiplicative perturbations of it over three consecutive chain segments, as shown in Algorithm 16.

Algorithm 16: Any-scale Adaptation Block

Inputs : parameter type $a \in \{\sigma, \beta\}$; current value $\phi \in \mathcal{I}_a$; adaptation rate $\gamma \in (0, 1)$; initial state $x^{(0)}$
Outputs: updated parameter ϕ'; samples $x^{(1)}, \ldots, x^{(3N)}$
$\phi_0 \leftarrow \phi$
$\phi_+ \leftarrow \mathrm{proj}_{\mathcal{I}_a}(\phi(1+\gamma))$
$\phi_- \leftarrow \mathrm{proj}_{\mathcal{I}_a}(\phi(1-\gamma))$
use ϕ_0 to compute the proposal and collect $x^{(1)}, x^{(2)}, \ldots, x^{(N)}$
compute $d_0 \leftarrow \sum_{i=1}^{N} \|x^{(i)} - x^{(i-1)}\|_1$
use ϕ_+ to sample $x^{(N+1)}, \ldots, x^{(2N)}$ using the proposal
compute $d_+ \leftarrow \sum_{i=1}^{N} \|x^{(N+i)} - x^{(N+i-1)}\|_1$
use ϕ_- to sample $x^{(2N+1)}, \ldots, x^{(3N)}$ using the proposal
compute $d_- \leftarrow \sum_{i=1}^{N} \|x^{(2N+i)} - x^{(2N+i-1)}\|_1$
if $\max(d_0, d_+, d_-) = d_+$ **then**
  $\phi' \leftarrow \phi_+$
else if $\max(d_0, d_+, d_-) = d_-$ **then**
  $\phi' \leftarrow \phi_-$
else
  $\phi' \leftarrow \phi_0$
return ϕ' *and* $x^{(1)}, \ldots, x^{(3N)}$

The three segments in Algorithm 16 are consecutive: the second begins from $x^{(N)}$ and the third from $x^{(2N)}$. The full adaptation alternates a block for $\phi = \sigma$ and a block for $\phi = \beta$. If neither parameter changes during an outer iteration, the update rate is reduced geometrically, $\gamma \leftarrow \rho\gamma$ for a decay factor $\rho \in (0, 1)$; otherwise the selected parameter values are retained for the next iteration. Because adaptation based on short-run jump distances can be noisy during early mixing, this procedure is intended for burn-in rather than continued adaptation after samples are retained for inference.

Through the combination of a quadratic approximation of the log-probability difference $U(x') - U(x)$ and the adaptive scheme for β and σ, the any-scale sampler interpolates between locally and globally balanced proposals.

6.5.2 Automatic Cyclical Sampling

One common issue with gradient-based samplers for continuous spaces is exploration: as the gradient points in the direction of the closest mode, it gives no information regarding further, more distant modes. This can hinder the accuracy of the sampler in terms of approximating the target distribution.

The same problem applies to gradient-based discrete samplers. Unlike a continuous state space, a discrete space has no arbitrarily small non-trivial perturbations: even the most local move must change at least one coordinate. This granularity can make the trade-off between conservative, high-acceptance moves and larger exploratory moves especially pronounced. Interpolating between locally and globally balanced proposals is therefore a natural way to vary the proposal scale when large movements may be required to escape a local mode.

In the continuous case, many approaches have been proposed for handling the mode-escape problem. One popular approach is cyclical SG-MCMC, discussed in Chapter 2. A cyclical schedule gradually reduces the step-size of a gradient-based sampler from very large magnitudes to small magnitudes and repeats for several cycles. The large steps at the beginning allow for more exploratory movements, which enable mode discovery; and the smaller step-sizes at the end of the cycle allow for exploitation of the mode. The repetition of the cycles allows for the discovery and exploitation of many modes, which is required for effective characterisation of multi-modal distributions.

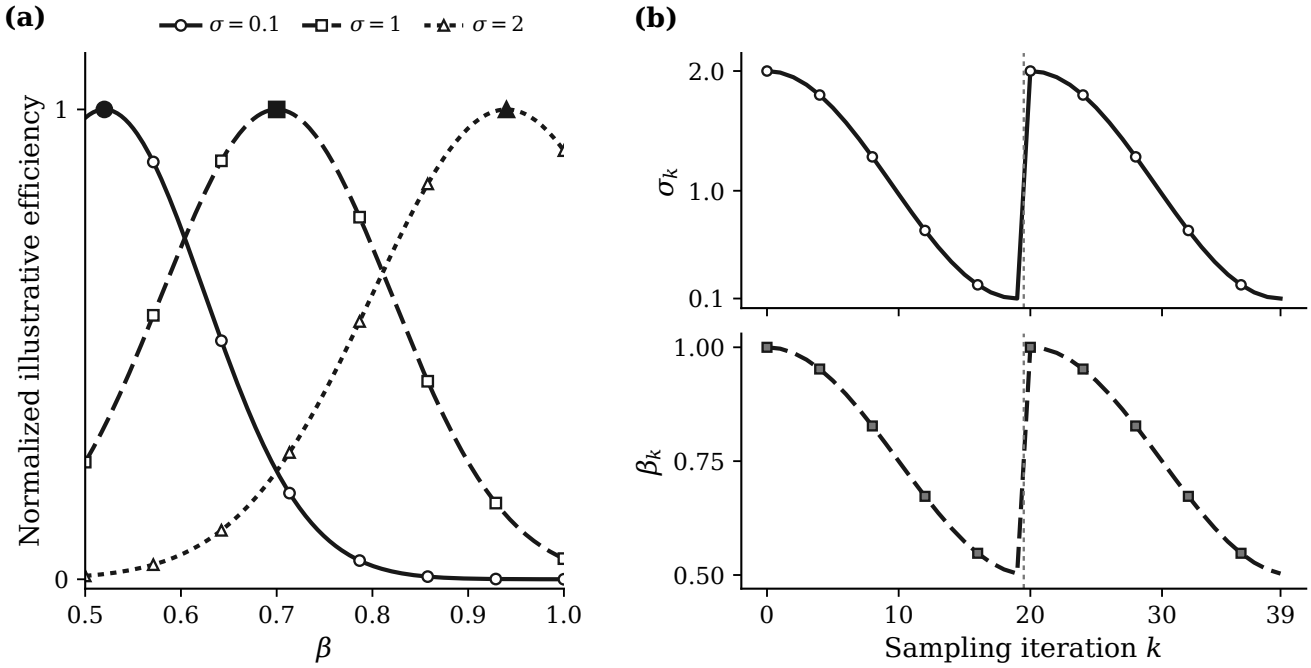


Figure 6.3: Proposal scale, balancing, and cyclical scheduling. Panel (a) schematically illustrates that the preferred balancing exponent increases with the proposal scale: small-scale proposals favour β near $1/2$, whereas larger-scale proposals favour values closer to 1. The curves show normalised illustrative efficiency rather than empirical measurements. Panel (b) shows coupled schedules for σ_k and β_k over two cycles, progressing from large proposals with more globally balanced weighting to small, locally balanced proposals within each cycle. Panel (a) is adapted conceptually from Sun et al. [1574]; panel (b) is adapted from Pynadath et al. [1350].

Building on this intuition, Pynadath et al. [1350] introduce a sampler that cycles jointly through step sizes and balancing exponents. Varying σ alone can produce poorly balanced proposals and low acceptance probabilities, so the method uses the generalised DLP

$$q_i(x_i' \mid x) = \mathrm{Cat}\left(\mathrm{Softmax}_{x_i' \in \mathcal{X}_i}\left[\beta \nabla_i U(x)(x_i' - x_i) - \frac{(x_i' - x_i)^2}{2\sigma}\right]\right). \tag{6.6}$$

Setting $\beta = 1/2$ in Equation (6.6) recovers the standard DLP. Within each cycle, large values of σ are paired with values of β closer to 1 to encourage global exploration; as σ decreases, β moves towards $1/2$ and the proposal becomes more local.

The automatic tuning procedure begins with user-specified endpoint exponents $\beta_{\min}$ and $\beta_{\max}$ and a target acceptance rate. Short burn-in runs determine corresponding endpoint step sizes $\sigma_{\min}$ and $\sigma_{\max}$. These endpoints define the cyclical step-size schedule, following the continuous-space construction of Zhang et al. [1854].

For each step size σ_k, the corresponding β_k is selected to maximise an empirical acceptance-rate objective subject to $\beta_k \in [1/2, \beta_{k-1}]$. This monotonic constraint reflects the transition from large moves with more globally balanced weighting at the beginning of a cycle to small, locally balanced moves at its end, while reducing the cost of searching for the full schedule. Figure 6.3(b) shows the coupled schedules used to move repeatedly from global exploration to local characterisation.

6.6 Conclusion

Gradient information can make sampling in high-dimensional discrete spaces substantially more efficient, even though the state space itself is not differentiable. Locally balanced proposals provide the organising principle: they combine an uninformed local kernel with target-specific information and use a Metropolis–Hastings correction to retain the desired target distribution. Gibbs-With-Gradient uses a first-order approximation to choose an informed single-coordinate update, whereas DLP factorises a Langevin-like proposal so that many coordinates can be proposed in parallel.

The appropriate proposal depends on the scale of movement required. Small, locally balanced moves can characterise a mode accurately but may mix slowly between separated modes. Any-scale balancing adapts the balancing exponent and adds a global curvature approximation for more effective non-local moves. Automatic cyclical sampling instead repeats a schedule from large exploratory proposals to small exploitative proposals, pairing each step size with a suitable balancing exponent.

In the running binary example, this progression moves from informed single-bit proposals, through parallel multi-bit updates, to methods that adapt the scale and weighting of those updates to improve movement between separated modes.

These methods trade additional structure and tuning for improved exploration. Their practical performance depends on the quality of the differentiable extension of U, the accuracy of the local or quadratic approximation, and the cost of evaluating reverse proposals in the Metropolis–Hastings correction. The adjusted variants provide asymptotically exact sampling under the usual irreducibility and aperiodicity conditions; unadjusted variants can be cheaper, but generally introduce stationary bias.

Part II

Laplace approximations

Chapter 7

Introduction

Astute readers of the preceding chapter will have noticed that the methods introduced in Part I come with a concerning amount of computational and memory overhead. Even a perfectly efficient Monte Carlo method, one that draws an i.i.d. sample from the posterior in every single step, still requires us to *store* multiple samples—entire copies of the parameter vector—in memory. At prediction time, we then have to compute the forward pass of the model for each of these samples. The number k of samples enters at least multiplicatively into the computational cost c of whatever it is we are doing with our neural network: We will pay a price of $\Omega(kc)$ (the Landau Ω. Yes, that's a *lower* bound), both in memory and computation, to turn a non-Bayesian computation into a Bayesian one. At a time when prominent non-Bayesian deep models are already maxing out the capacity of the best multi-GPU workstations, this is a nonstarter, and arguably the main reason why Bayesian deep learning has acquired a reputation for unacceptably high computational cost.

This Part, however, will introduce a different, more practically minded way to endow neural networks with Bayesian uncertainty. Its history is just as old as that of sampling methods, and was born out of computational necessity in the early days of deep learning [1105]. In fact, it was originally invented in the early 19th century, long before digital computers [974], to turn intractable integrals into tractable ones, at any cost. As computation is once again a bottleneck in the deep learning age, Laplace approximations have re-emerged as a practical tool [380, 812, 884] with new twists that push the envelope of automatic differentiation [365, 366] and numerical linear algebra [309]. It also offers new perspectives on deep learning from the perspective of differential geometry [931, 1427]. As we will see, it now comes with great power, and grave dangers.

7.1 Laplace approximations are local and differential

As in previous chapters, consider a differential program $f : \mathbb{R}^D \times \mathbb{R}^P \to \mathbb{R}^C$ mapping inputs $x \in \mathbb{R}^D$ to outputs $f(x, \theta) \in \mathbb{R}^C$, parametrised by $\theta \in \mathbb{R}^P$. Consider the regularized empirical risk $\mathcal{L}_\Omega \in \mathbb{R}$ defined as

$$\mathcal{L}_\Omega(\theta) = \sum_{n=1}^{N} \ell(f(x_n, \theta), y_n) + \Omega(\theta). \tag{7.1}$$

with loss function $\ell \colon \mathbb{R}^C \times \mathbb{R}^C \to \mathbb{R}$ and regularizer $\Omega \colon \mathbb{R}^P \to \mathbb{R}$, both strictly convex and twice continuously differentiable.[1]

Also as in previous chapters, we interpret this function as an energy function, the unnormalised negative logarithm of a posterior:

$$\mathcal{L}_\Omega(\theta) = -\log p(Y \mid X, \theta) - \log p(\theta) + \text{const} = -\log p(\theta \mid \mathcal{D}) + \text{const}. \tag{7.2}$$

Definition 7.1. *A Laplace approximation is any algorithm within a general class of methods that approximate $\mathcal{L}_\Omega(\theta)$ with a quadratic function*

$$\mathcal{L}_\Omega(\theta) \approx \frac{1}{2}(\theta - \theta_*)^\top \Psi(\theta_*)(\theta - \theta_*) + \textit{const}, \tag{7.3}$$

*where $\Psi(\theta_*) \in \mathbb{R}^{P \times P}$ is a second-order linear differential operator acting on $\mathcal{L}_\Omega(\theta_*)$ at the point $\theta_* \in \mathbb{R}^P$. A Laplace approximation is thus defined by the tuple (θ_*, Ψ), which we will call a **point estimate** and its **curvature**, respectively.*

[1] Note that convexity here is only assumed for the loss and regularizer. The full loss $\mathcal{L}_\Omega(\theta)$ is not generally convex, since the network f is a nonlinear function of the parameters θ.

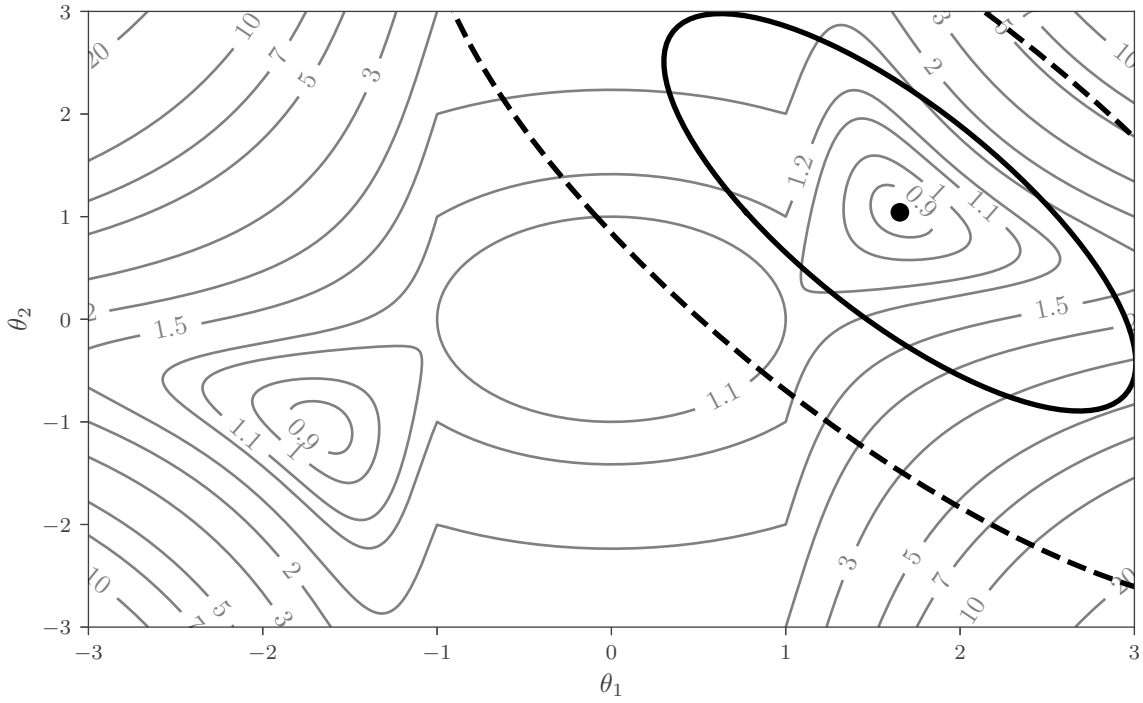


Figure 7.1: A linearized Laplace approximation on the most basic feed-forward perceptron $f(x, \theta) = \theta_2 \cdot \mathrm{ReLU}(\theta_1 x + 1)$ for the dataset $X = (1, -1), Y = (-1, -1)$, with regularizer $\Omega(\theta) = 0.2 \cdot \|\theta\|^2$ and square loss. The grey contour lines denote the energy function. The black dot represents the minimum θ_*; the thick black ellipses denote one (solid) and two (dashed) standard deviations of the Laplace approximation. The Gaussian approximation defined by the Laplace approximation clearly captures the *local* geometry, but not the full *global* shape of the posterior.

Under the assumption of Equation 7.2, any Laplace approximation is directly associated with an approximate **Gaussian measure on the parameter space**,

$$p(\theta \mid \mathcal{D}) = \frac{1}{Z(\mathcal{D})} \exp(-\mathcal{L}_\Omega(\theta)) \tag{7.4a}$$

$$\approx \frac{1}{Z(\mathcal{D})} \exp\left(-\frac{1}{2}(\theta - \theta_*)^\top \Psi(\theta_*)(\theta - \theta_*)\right) \tag{7.4b}$$

$$= \mathcal{N}(\theta; \theta_*, \Psi^{-1}(\theta_*)). \tag{7.4c}$$

The textbook, but impractical, version of the Laplace approximation arises from the Taylor expansion at a local minimizer of the regularized risk:

$$[\Psi(\theta)]_{ij} = \left[\nabla_\theta \nabla_\theta^\top \mathcal{L}_\Omega(\theta)\right]_{ij} = \frac{\partial^2 \mathcal{L}_\Omega(\theta)}{\partial \theta_i \partial \theta_j}, \qquad \theta_* = \operatorname*{arg\,min}_{\theta \in \mathbb{R}^P} \mathcal{L}_\Omega(\theta). \tag{7.5}$$

That is, we pick a point θ_* where the gradient vanishes, and compute the Hessian there (the matrix $\Psi(\theta_*)$ being the matrix representation of the aforementioned operator, evaluated at θ_*).

7.2 What is to like about Laplace?

Central to the appeal of Laplace approximations is that they are an *analytic post-processing* step that can be tagged onto the established way of training a deep model:

How to construct a Laplace approximation:

1. Train your deep network in any way you know and trust, using whatever optimiser you like. Wait for your training loop to terminate and return the point estimate θ_*.
2. After training has finished (!), compute $\Psi(\theta_*)$. Since it is a second-order differential operator, it can be computed with automatic differentiation, using your favourite auto-diff framework.

And that's it! You now have a "Bayesian uncertainty" estimate in parameter space, in the form of the Gaussian from Equation 7.4c. Let us spell out the advantages of this recipe, some more obvious, some less so, explicitly:

- **You get to keep your point-estimate**. People who train neural networks, especially if it is a large model, spend extreme resources to find a good value θ_*. Even if this particular value may not hold up to the theoretical demands of Bayesian analysis, it nevertheless embodies a significant investment, so it is appealing to keep working with this valuable object. For Laplace, this point estimate turns directly into a posterior mean. And to state the obvious: The computation overhead of this part is strictly *zero* over whatever the training cost to begin with.
- **You get to use established tools**. Constructing the Gaussian of Equation 7.4c, requires precisely two computational tools: *Automatic differentiation* to get Ψ, and *numerical linear algebra*, to find its (approximate) inverse. These are arguably *the* most powerful, reliable, optimised functionalities available to the machine learning engineer. They can be made extremely numerically robust and conveniently automated, or tailored for radical computational efficiency. And, using probabilistic numerical linear algebra, the discretisation and iteration error of these internal computations can be tracked for downstream consideration (More about numerical linear algebra tricks in Chapter 8 and Part VIII).
- **Laplace is post hoc**. Since Ψ is only needed after θ_* has been found, its cost is not a function of the complexity of training. In fact, the construction of a Laplace approximation is effectively separated from training entirely, and can be done on a pre-trained model downloaded from the Internet. The only caveat is that, since Ψ is a function of the regularized risk, we need access to the training data $\mathcal{D}$ – or some compressed representation thereof.
- **Laplace is analytical**. Laplace approximations return a Gaussian probability measure, whose parameters are computed using optimization, automatic differentiation, and linear algebra. That is, a deterministic quantity – a probability density function – constructed deterministically. There is no source of randomness involved, apart from the potentially stochastic nature of training to find θ_*. As we will see in Chapter 10, this makes Laplace approximations interesting for downstream applications and decisions that require analytic marginalization over the parameters.
- **Deep theoretical meaning**. We will find in Chapters 8 and 9 that the local differential nature of the curvature at the heart of Laplace approximations gives them profound theoretical connections, and intuitive interpretations. In fact, they open up a whole self-contained theory of deep learning founded on the differential geometry of the energy manifold associated with $\mathcal{L}_\Omega$.
- **Laplace turns every deep network into a Gaussian process**. In Chapter 8, we will see that the Gaussian measure $\mathcal{N}(\theta; \theta_*, \Psi^{-1}(\theta_*))$ constructed by the Laplace approximation can be analytically pushed forward to the output space of the network $f(x, \theta)$, where it manifests itself as a Gaussian process, whose mean-function is exactly the trained network $f(x, \theta_*)$, and whose covariance function (kernel) is given by a structured object, a *tangent kernel* (developed further in Part V). This is a powerful perspective for both practical and conceptual reasons: On the practical side, we can more or less directly transport a lot of notions from the Gaussian process literature to deep learning, recovering advanced functionality sorely missing from non-Bayesian deep learning. And on the conceptual side, the tangent kernel contains very specific terms that capture modular, interpretable statements about the role of the training data, the network architecture, and regularization that make deep learning more transparent and interpretable.

7.3 Practical Considerations

On first encountering Laplace approximations in deep learning, people tend to have one of two, mutually almost contradictory reactions:

7.3.1 Isn't this too simplistic?

A Taylor-expansion? Really? From a fundamentalist Bayesian perspective, a quadratic function is a poor approximation of the posterior, with all its non-convex nonlinear glory (cf. Figure 7.1). But compared to the day-to-day practice of training deep networks, Laplace approximations are a big step forward, from a *zeroth order* approximation (a point estimate) to a *second-order* approximation (a Gaussian probability measure). They offer an easy gateway into the Bayesian world for the deep learning practitioner: Just take your trained model, compute a curvature, and Bob's your uncle.

We should also be careful not to appeal to unrealistic ideals, but compare to realistic alternatives instead. The true posterior may be the theoretical target of all Bayesian analysis, but is completely intractable. All Bayesian methods that are actually accessible are approximations, of different nature. For example, if we draw an ensemble of $S \ll P$ samples $[\theta_s]_{s=1}^S$ from the posterior, and use it to construct a posterior covariance $\hat{\Psi}^{-1} = \frac{1}{S}\sum_{s=1}^S (\theta_s - \theta_*)(\theta_s - \theta_*)^\top$ over the parameters θ, that matrix $\hat{\Psi}$ will be of rank at most S. If we use this object as a proxy for "posterior uncertainty", we may severely over-estimate confidence intervals. A Laplace approximation might overlook nonlocal aspects of the posterior's geometry, but it can construct an approximate covariance matrix of full rank.

7.3.2 Isn't this too expensive?

Hang on, we need a Hessian *over the parameter space? That's a matrix of size $P \times P$, so just storing it will create a memory overhead of $\mathcal{O}(P^2)$. Oh, and then we need to compute its* inverse*!? That will incur a computational cost of $\mathcal{O}(P^3)$, right?* The flaw of such thoughts is that they use worst case statements about objects that are entirely under the user's control.

It is better to think of the curvature Ψ as a *linear operator* [366], a function mapping vectors to vectors. Such a function need not require the explicit instantiation of a matrix in memory. Chapters 8 and 10 will introduce several possible choices for this operator, spanning a range of computational and memory costs. These include the generalised Gauss-Newton approximation (Section 8.3), Kronecker-factored approximate curvature, and diagonal approximations to the Hessian (Section 8.4). In fact, in Section 10.1 we will see that even the trivial choice $\Psi = \alpha I$ for an arbitrary $\alpha \in \mathbb{R}_+$ (i.e. just replacing the point estimate with an arbitrary Gaussian measure) already fixes some tangible, nontrivial pathologies of non-Bayesian deep learning.

Likewise, numerical linear algebra offers a rich toolbox of iterative, sparse, sketched, etc. methods to solve linear systems of equations of the form $\Psi x = b$ for x without the need to even explicitly form an estimate of the matrix inverse Ψ^{-1} (Section 8.4.1). And there are probabilistic versions of these methods that can partially track and estimate the error of their output [e.g. 317, 1740, 1741]. The memory and compute cost of Laplace approximations can thus be completely controlled by the user, opening up a wide spectrum of algorithms of varying quality and cost.

On the flip side, this also means that constructing Laplace approximations, just like training deep networks, is not a turnkey operation, and requires care and expertise. But emerging software tools increasingly encapsulate these choices, and provide abstraction layers that allow constructing Laplace approximations with a few lines of code.

7.4 All approximations are wrong

Despite all the arguments in their favour, Laplace approximations remain just that: approximations. Their local nature – the fact that they only use differential quantities evaluated at a single point in the parameter space – is both their principal strength and their principal weakness. As Figure 7.1 shows, the Gaussian posterior created by a Laplace approximation may not capture the geometry of the posterior – not even within the region of the parameter space delineated by one or two standard deviations of that Gaussian. And when intricate numerical routines are used to operate on the curvature operator Ψ, their own errors can also severely erode the quality of the approximation.

Laplace approximations should thus best be viewed as an important tool of first resort. At least their most basic variants can be constructed on virtually any deep architecture, often with minimal overhead in both implementation and computation. They can be a powerful diagnostic tool, elucidating the interplay between data and architecture; and offer structured, controllable functionality for everyday requirements like uncertainty quantification, model adaptation, continual learning, and more. But whenever the stakes are high, for example in safety critical applications, or when uncertainty is used to draw scientific conclusions, additional safeguards are needed. In such settings, Laplace approximations may be paired with some of the other frameworks described in this book,

to provide a more robust and calibrated uncertainty estimate.

Takeaways:

A Laplace approximation is a quadratic approximation to the log posterior associated with a deep model, interpreted as a Gaussian posterior on the parameter space, or a Gaussian process posterior on the predictive (logit) outputs of the model. Such approximations are constructed by first training a point estimate for the parameters in the "normal" way, then computing the curvature operator (or kernel) post hoc. By this nature, Laplace approximations are *local* and *differential*. On the one hand, this is a weakness, because they fail to capture *global*, *non-linear* structure in the posterior. On the other hand, this is a strength, because it allows the use of powerful automatic differentiation and matrix-free, probabilistic linear algebra, and provides theoretical descriptions of deep learning in the terms of differential geometry. The following chapters will provide further details on these aspects.

Chapter 8

Linearised Laplace approximations

The previous chapter showed a computationally lightweight Gaussian approximation $\mathcal{N}(\theta; \theta_*, \Psi(\theta_*)^{-1})$ to the true posterior $p(\theta \mid \mathcal{D})$ on the parameter space $\mathbb{R}^P$. Our final goal is to use this Gaussian measure to *make decisions* based on the model's output: its predictions feed into the risks and utilities that downstream tasks optimise (Chapter 10). To this end, we first quantify the **predictive uncertainty** of our network on the output space $\mathbb{R}^C$ by turning it into a Gaussian process (GP). Figure 8.1 previews the pipeline, from the maximum a posteriori (MAP) estimate θ_* to the predictive distribution.

8.1 Linearisation

To turn an arbitrary (differentiable) neural network parameterised by $\theta \in \mathbb{R}^P$ into a Gaussian process, we use two ingredients: linearisation in the parameter space and the previously introduced Laplace approximation.

We first introduce the concept of the **linearised network** $f^{\theta_0} \colon \mathbb{R}^D \times \mathbb{R}^P \to \mathbb{R}^C$ as the first-order Taylor approximation of network f around parameters $\theta_0 \in \mathbb{R}^P$:

$$f^{\theta_0}(x, \theta) := f(x, \theta_0) + J_\theta f(x, \theta_0)(\theta - \theta_0), \tag{8.1}$$

where $J_\theta f(x, \theta_0) \in \mathbb{R}^{C \times P}$ is the **network Jacobian** evaluated at (x, θ_0).[1] Analogously to Equation 7.1, we define the regularised empirical risk of the linearised network f^{θ_0} with parameters θ as

$$\mathcal{L}_\Omega^{\theta_0}(\theta) := \sum_{n=1}^{N} \ell\left(f^{\theta_0}(x_n, \theta), y_n\right) + \Omega(\theta). \tag{8.2}$$

Unless explicitly stated otherwise (e.g., in the prior-only discussion in Section 8.2), we fix the linearisation point at a minimiser of the *original* regularised empirical risk $\mathcal{L}_\Omega(\theta)$:

$$\theta_* :\in \arg\min_{\theta \in \mathbb{R}^P} \mathcal{L}_\Omega(\theta), \tag{8.3}$$

that is, we set $\theta_0 \equiv \theta_*$. Linearisation itself only requires differentiability of f at θ_0. The Laplace approximation needs more: θ_* must be a local minimiser of $\mathcal{L}_\Omega$ with zero gradient, and the resulting precision must be positive definite after adding the prior curvature. We note that f^{θ_*} shares its value and its parameter Jacobian with f at θ_*, so $\nabla_\theta \mathcal{L}_\Omega^{\theta_*}(\theta_*) = \nabla_\theta \mathcal{L}_\Omega(\theta_*) = 0$ and θ_* is a stationary point of $\mathcal{L}_\Omega^{\theta_*}$, too. Because f^{θ_*} is affine in θ, convexity of $\ell(\cdot, y)$ in z makes each loss term convex in θ; with a strictly convex Ω, the risk $\mathcal{L}_\Omega^{\theta_*}$ is then strictly convex in θ, hence θ_* is its unique minimiser.

Following Section 7.1, the Laplace approximation using the regularised empirical risk of the linearised network yields the parameter-space Gaussian measure $\mathcal{N}(\theta; \theta_*, \Psi(\theta_*)^{-1})$ with

$$[\Psi(\theta)]_{ij} = \left[\nabla_\theta \nabla_\theta^\top \mathcal{L}_\Omega^{\theta_*}(\theta)\right]_{ij} = \frac{\partial^2 \mathcal{L}_\Omega^{\theta_*}(\theta)}{\partial \theta_i \partial \theta_j}. \tag{8.4}$$

The curvature Ψ of the linearised network's regularised empirical risk differs from the Hessian in Equation 7.5 for networks that are not piecewise affine in their parameters (except at the non-generic parameter values where

[1] Note that this network is only linear in its parameters θ but is still nonlinear in its inputs x.

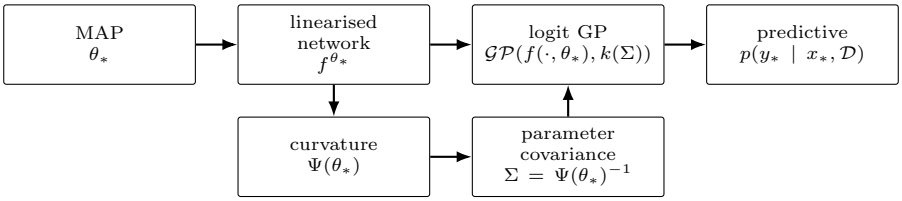


Figure 8.1: The linearised Laplace pipeline. The linearised network f^{θ_*} and parameter covariance $\Sigma = \Psi(\theta_*)^{-1}$ jointly determine the logit Gaussian process, which induces a predictive distribution.

all training residuals vanish). The difference is that $\Psi(\theta_*)$, as defined in Equation 8.4, discards the network's parameter curvature (and higher-order derivatives) and only retains the curvature of the *loss function* over the model outputs. The advantages of this modelling choice are summarised in Section 8.1.1.

An important property of Gaussian measures is their closure under affine transformations, stated formally below.

Proposition 8.1 (Affine Closure of Gaussians)**.** *Let $X \sim \mathcal{N}(\mu, \Sigma)$ with $\mu \in \mathbb{R}^N$ and $\Sigma \succ 0$. For any $A \in \mathbb{R}^{M\times N}$ with full row rank and $b \in \mathbb{R}^M$,*

$$Y := AX + b \;\sim\; \mathcal{N}\left(A\mu + b,\; A\Sigma A^\top\right). \tag{8.5}$$

If $\operatorname{rank}(A) < M$, then $A\Sigma A^\top \succeq 0$ and Y has no Lebesgue density on $\mathbb{R}^M$.

Proof. We first treat an invertible $A \in \mathbb{R}^{N\times N}$. For any Borel set $E \subseteq \mathbb{R}^N$, the event $\{Y \in E\}$ equals $\{X \in A^{-1}(E-b)\}$, so substituting $x = A^{-1}(y-b)$, whose Jacobian has absolute determinant $|\det A^{-1}|$, gives

$$p_Y(y) = p_X\left(A^{-1}(y-b)\right)|\det A^{-1}|. \tag{8.6}$$

Writing $d := y - (A\mu + b)$, we have $A^{-1}(y-b) - \mu = A^{-1}d$, so

$$\left(A^{-1}(y-b)-\mu\right)^\top \Sigma^{-1}\left(A^{-1}(y-b)-\mu\right) = d^\top(A\Sigma A^\top)^{-1}d, \tag{8.7}$$

and $|\Sigma|^{-1/2}|\det A^{-1}| = |A\Sigma A^\top|^{-1/2}$ by $|A\Sigma A^\top| = |A|^2|\Sigma|$. Substituting the Gaussian density of X yields

$$p_Y(y) = (2\pi)^{-N/2}|A\Sigma A^\top|^{-1/2}\exp\left(-\frac{1}{2}d^\top(A\Sigma A^\top)^{-1}d\right), \tag{8.8}$$

so $Y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$.

For the full row rank case $A \in \mathbb{R}^{M\times N}$ with $M \le N$, choose $K \in \mathbb{R}^{(N-M)\times N}$ so that $T := \begin{bmatrix} A \\ K \end{bmatrix} \in \mathbb{R}^{N\times N}$ is invertible, and set $Y := AX + b$, $Z := KX$, and $U := \begin{bmatrix} Y \\ Z \end{bmatrix} = TX + \begin{bmatrix} b \\ 0 \end{bmatrix}$. By the invertible case, U is Gaussian with mean $\begin{bmatrix} A\mu + b \\ K\mu \end{bmatrix}$ and covariance

$$S := T\Sigma T^\top = \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix} = \begin{bmatrix} A\Sigma A^\top & A\Sigma K^\top \\ K\Sigma A^\top & K\Sigma K^\top \end{bmatrix}, \qquad S_{11} = A\Sigma A^\top \succ 0. \tag{8.9}$$

Let $C := S_{22} - S_{21}S_{11}^{-1}S_{12}$ be the (positive definite) Schur complement and $B := S_{21}S_{11}^{-1}$. With $\Delta y := y - (A\mu + b)$ and $\Delta z := z - K\mu$, the block-inverse identity splits the quadratic form as

$$\begin{bmatrix} \Delta y \\ \Delta z \end{bmatrix}^\top S^{-1} \begin{bmatrix} \Delta y \\ \Delta z \end{bmatrix} = \Delta y^\top S_{11}^{-1}\Delta y + (\Delta z - B\Delta y)^\top C^{-1}(\Delta z - B\Delta y), \tag{8.10}$$

and $|S| = |S_{11}||C|$. Marginalising U over z with

$$\int_{\mathbb{R}^{N-M}} \exp\left(-\frac{1}{2}(\Delta z - B\Delta y)^\top C^{-1}(\Delta z - B\Delta y)\right) \mathrm{d}z = (2\pi)^{(N-M)/2}|C|^{1/2} \tag{8.11}$$

leaves

$$p_Y(y) = (2\pi)^{-M/2}|S_{11}|^{-1/2}\exp\left(-\frac{1}{2}\Delta y^\top S_{11}^{-1}\Delta y\right), \tag{8.12}$$

that is, $Y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$. Only $S_{11} = A\Sigma A^\top$ survives, so the result is independent of the choice of K.

Finally, if $\operatorname{rank}(A) < M$, then $\operatorname{rank}(A\Sigma A^\top) = \operatorname{rank}(A) < M$ because $\Sigma \succ 0$, so $A\Sigma A^\top$ is singular and Y is supported on the proper affine subspace $A\mathbb{R}^N + b$, hence has no Lebesgue density on $\mathbb{R}^M$. □

As such, the linearised network f^{θ_*} pushes the parameter posterior forward, via the affine map of Equation 8.1, to a Gaussian in the output space (the logit space in classification). In the network-output variable $z \in \mathbb{R}^C$ (the logits, which the loss $\ell(z, y)$ takes as its first argument), this measure is

$$\mathcal{N}\left(z; f(x_*, \theta_*), J_\theta f(x_*, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x_*, \theta_*)^\top\right). \tag{8.13}$$

This result already equips the network f with a probability measure in its output space (or logit space in classification) for every input x_*. A **Gaussian process** is an even more powerful construction, capable of modelling a Gaussian measure over any *finite set of inputs*, not just a single one. The linearised network f^{θ_*} equipped with its parameter-space Laplace approximation defines a Gaussian process $\mathcal{GP}(f(\cdot, \theta_*), k)$ with the following (Mercer) kernel:

$$\begin{aligned} k(x, x') &= \operatorname{cov}\left(f^{\theta_*}(x, \theta), f^{\theta_*}(x', \theta)\right) && (8.14)\\ &= \operatorname{cov}\left(J_\theta f(x, \theta_*)(\theta - \theta_*), J_\theta f(x', \theta_*)(\theta - \theta_*)\right) && (8.15)\\ &= J_\theta f(x, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x', \theta_*)^\top. && (8.16) \end{aligned}$$

8.1.1 Advantages of Using the Linearised Network

We perform the Laplace approximation w.r.t. $\mathcal{L}_\Omega^{\theta_*}$ (with $\theta_* = \arg\min_{\theta \in \mathbb{R}^P} \mathcal{L}_\Omega(\theta)$) instead of the regularised empirical risk of f for multiple reasons.

First, the Hessian of the regularised empirical risk of f (Equation 7.5) is not guaranteed to be positive definite, preventing us from interpreting it as the inverse covariance matrix of our approximate Gaussian posterior over the parameters θ.[2] In second-order optimisation, the lack of positive definiteness also prohibits using the inverse Hessian as a preconditioner. However, the Hessian of the regularised empirical risk of f^{θ_*}, with a strongly convex regulariser, is *guaranteed* to be positive definite.

Second, the Hessian of the regularised empirical risk of f^{θ_*} is considerably cheaper to use than that of f, allowing Laplace approximations to scale to deep learning architectures (see Section 8.3).

There are thus two uses of the linearised model. First, it defines the curvature Ψ and hence the parameter-space Gaussian posterior approximation. Second, it defines the predictive pushforward in Equation 8.13. Earlier work on Laplace approximations for neural networks [1108, 1402] *sampled* M parameter configurations $\{\theta_s\}_{s=1}^M$ from the parameter-space Gaussian and calculated *predictive* quantities from the nonlinear predictions $\{f(x, \theta_s)\}_{s=1}^M$. This loses the exact output-space Gaussian distribution available from linearisation and yields no posterior GP. Lawrence [984] shows empirically that this approach is prone to underfitting: the predictive becomes *too uncertain* even on in-distribution data points. Immer et al. [813] identify the core reason for performance loss as a mismatch between the model in which we perform approximate Bayesian inference (the linearised network f^{θ_*}) and the model we use for prediction (the nonlinear network f). Figure 8.2 visualises this underfitting and the low-density samples drawn under the true posterior. See also Immer et al. [813, Fig. 3]. By using the linearised network to make predictions and not only to calculate the curvature, the approximation remains internally consistent.

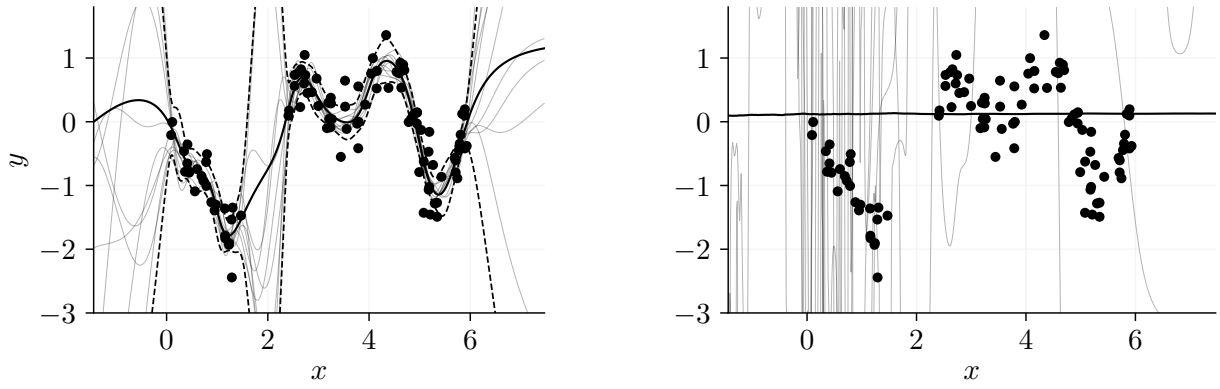
Takeaways

The linearised Laplace approximation turns any differentiable neural network into a *Gaussian process* by (i) linearising the network in parameter space at θ_* and (ii) placing a Gaussian posterior $\mathcal{N}(\theta_*, \Psi(\theta_*)^{-1})$ on parameters. The induced Mercer kernel has the form

$$k(x, x') = J_\theta f(x, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x', \theta_*)^\top,$$

whereby Ψ captures the effect of the dataset on the *parameter-space curvature*, which the Jacobian $J_\theta f(\cdot, \theta_*)$ pushes forward into *function-space uncertainty*.

[2] In particular, the transition from Equation 7.4b to Equation 7.4c is not justified when $\Psi(\theta_*)$ is not positive definite.



(a) (b)

Figure 8.2: The same full Laplace posterior under two predictive rules on a one-dimensional regression example from the code accompanying Immer et al. [813]. Both panels use the same parameter-space approximate posterior and the same ten parameter draws. They differ only in how each parameter draw becomes a function-space sample called a *sample path* in the GP literature. Panel (a) sends each draw through the linearised network f^{θ_*}, the model consistent with the parameter-space Laplace approximation. Panel (b) sends the draws through the original nonlinear network f. Solid curves are predictive means; dashed curves show pointwise 95% credible intervals. The linearised Laplace approximation in Panel (a) yields a GP; therefore, the output-space quantities are available in closed form. The (nonlinear) Laplace approximation in Panel (b) yields an intractable output mean and covariance, which we approximate with two million Monte Carlo samples. The nonlinear scheme is far more uncertain over the training inputs and its mean severely underfits. The linearised Laplace approximation yields consistent function-space uncertainty.

8.2 The Prior

The Gaussian process determined by the *posterior* parameter-space covariance matrix Ψ^{-1} is tied to the dataset $\mathcal{D}$, as Ψ is the Hessian of the linearised model's regularised empirical risk over $\mathcal{D}$. It is worth spending some time on the case when the neural network $f\colon \mathbb{R}^D \times \mathbb{R}^P \to \mathbb{R}^C$ has not been exposed to any dataset $\mathcal{D}$ yet. In this case, the model's behaviour is determined by its architecture and by the prior $p(\theta)$ over the parameters.

For a non-Gaussian parameter prior $p(\theta)$, a Laplace approximation to the prior is centred at a local maximum θ_0 of $p(\theta)$, analogously to Equation 7.4c. Its precision is the negative log-prior Hessian,

$$\Psi = -\nabla_\theta \nabla_\theta \log p(\theta_0). \tag{8.17}$$

For the construction that follows, we specialise to a Gaussian prior centred at the linearisation point,

$$p(\theta) = \mathcal{N}(\theta; \theta_0, I), \tag{8.18}$$

where θ_0 can be any chosen initialisation. The prior's mode is θ_0, so the linearisation and the Laplace approximation of the prior share the same centre, and the precision above evaluates to $\Psi = I$. Pushing this prior through the linearised network gives the kernel $k\colon \mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}^{C\times C}$,

$$k(x, x') = J_\theta f(x, \theta_0) \left(J_\theta f(x', \theta_0)\right)^\top. \tag{8.19}$$

The kernel comes from the same pushforward as the posterior GP kernel in Equation 8.16. Compared with it, the data-dependent pair $(\theta_*, \Psi(\theta_*))$ is replaced by the dataset-independent pair (θ_0, I). This kernel is known as the **Neural Tangent Kernel (NTK)** [829].[3] It differs from the neural network Gaussian process (NNGP) kernel: the NNGP kernel is the covariance of random network outputs $f(x, \theta)$ and $f(x', \theta)$ under a random initialisation distribution, while the NTK is the covariance of the linearised network outputs $f^{\theta_0}(x, \theta)$ and $f^{\theta_0}(x', \theta)$ under the prior $\mathcal{N}(\theta; \theta_0, I)$. As the network width is taken to infinity in a specific way, both kernels are convergent. Where these limits exist, they are different deterministic kernels.

[3]For a finite-width network, this is sometimes called the *empirical* NTK at θ_0, and the NTK is reserved for networks in the infinite-width limit.

The NTK determines the function-space trajectory of f during full-batch gradient flow training on the average empirical loss via the Ordinary Differential Equation (ODE)

$$\frac{\mathrm{d}}{\mathrm{d}t} f_t(x) = -\frac{1}{N} \sum_{n=1}^{N} k_t(x, x_n) \, \nabla_z \ell(z, y_n)|_{z=f_t(x_n)} \,, \tag{8.20}$$

where t is the time variable of the gradient flow and k_t is the NTK of f_t. Jacot et al. [829] show that in the infinite-width limit of fully-connected neural networks with the so-called NTK initialisation, the NTK stays constant during training, and the convergence of optimisation is related to the positive definiteness of the NTK. However, the NTK of regular, finite-width neural networks is not constant and must be recomputed at every evaluation point.

From a modelling perspective, putting an isotropic Gaussian prior over the parameters is seldom well-motivated. However, the framework of Laplace approximations is compatible with interpretable function-space priors. Cinquin et al. [309] define Laplace priors using a prior Gaussian process. In particular, the negative log-prior is, up to an additive constant, equal to the squared Reproducing Kernel Hilbert Space (RKHS) norm of the difference between the linearised network f^{θ_0} and the mean of the prior Gaussian process μ. This negative log-prior is also used as a regulariser during training. As specifying the Gaussian prior in the function space only requires knowledge about the function's behaviour and *not* about the parameter space's structure, the framework of Cinquin et al. [309] allows domain experts to directly encode functional behaviour in the Bayesian inference problem.

Takeaways

Without any data and with a Gaussian prior $\mathcal{N}(\theta; \theta_0, I)$ centred at θ_0, the parameter-space curvature is the identity, $\Psi = I$, and the GP kernel derived from neural networks reduces to the Neural Tangent Kernel (NTK):

$$k(x, x') = J_\theta f(x, \theta_0) J_\theta f(x', \theta_0)^\top.$$

8.3 The Fisher

In Section 8.1, we glossed over the curvature matrix defined in Equation 8.4. This matrix and its components give an information-geometric interpretation to Laplace approximations, warranting its own chapter (Chapter 9).

Consider the regularised empirical risk of the linearised network f^{θ_0} (Equations 8.1 and 8.2). Under the assumption of Equation 7.2 applied to the linearised network, the term

$$\mathcal{L}^{\theta_0}(\theta) = \sum_{n=1}^{N} \ell\left(f^{\theta_0}(x_n, \theta), y_n\right) \tag{8.21}$$

is equal, up to an additive constant, to the negative log-likelihood of the dataset under the model, $-\log p(Y \mid X, \theta)$, whereas the term $\Omega(\theta)$ equals the negative log-prior, $-\log p(\theta)$, again up to an additive constant.

Writing out the Hessian of $\mathcal{L}^{\theta_0}$ evaluated at θ_0, we obtain

$$G(\theta_0) = \nabla_\theta \nabla_\theta^\top \mathcal{L}^{\theta_0}(\theta_0) = \sum_{n=1}^{N} \left(J_\theta f(x_n, \theta_0)\right)^\top \left(\nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f^{\theta_0}(x_n, \theta_0)}\right) J_\theta f(x_n, \theta_0), \tag{8.22}$$

called the **Generalised Gauss-Newton** matrix (GGN). In this section, we show that this matrix is equivalent to the Fisher Information Matrix for a family of loss functions, which is a natural (usually degenerate) Riemannian metric for statistical manifolds parameterised by the model (Section 9.2).

Consider the statistical manifold

$$\mathcal{M} = \left\{p(x, y \mid \theta) : \theta \in \mathbb{R}^P\right\} \tag{8.23}$$

with conditional independence structure $p(x, y \mid \theta) = p(y \mid x, \theta) \cdot p(x)$, whose in-depth study we defer to Chapter 9. The $p(y \mid x, \theta)$ distribution represents the likelihood parameterised by a neural network and $p(x)$ is the ground-truth input distribution. A natural notion of separation between probability distributions on this manifold is the **Kullback-Leibler (KL) divergence**:

$$D_{\mathrm{KL}}\left(p(x, y \mid \theta_1) \parallel p(x, y \mid \theta_2)\right) := \mathbb{E}_{p(x,y|\theta_1)}\left[\log \frac{p(x, y \mid \theta_1)}{p(x, y \mid \theta_2)}\right]. \tag{8.24}$$

The KL divergence is not a valid metric, as it is non-symmetric. However, for parameters θ_1 and θ_2 and $\Delta\theta = \theta_1 - \theta_2$, the KL divergence's second-order Taylor approximation around θ_1 is given by

$$D_{\mathrm{KL}}\left(p(x, y \mid \theta_1) \parallel p(x, y \mid \theta_2)\right) = \frac{1}{2}\Delta\theta^\top F(\theta_1)\Delta\theta + \mathcal{O}\left(\|\Delta\theta\|^3\right). \tag{8.25}$$

Both the zeroth- and first-order terms vanish (the former because $D_{\mathrm{KL}}(p \parallel p) = 0$, the latter because the expected score is zero, Equation 8.27), leaving the quadratic Fisher term as the leading contribution. At each parameter θ, the bilinear form $g_\theta(u, v) := u^\top F(\theta)v$ on tangent vectors u, v is a valid Riemannian metric over manifold $\mathcal{M}$ (up to the identifiability of parameters; see Chapter 9). The matrix

$$F(\theta) = \mathbb{E}_{p(x,y|\theta)}\left[-\nabla_\theta \nabla_\theta^\top \log p(x, y \mid \theta)\right] \tag{8.26}$$

is called the **Fisher Information Matrix (FIM)** of the random variable (X, Y) distributed according to $p(x, y \mid \theta)$.

Differentiating the normalisation $\int p(x, y \mid \theta)\,\mathrm{d}x\,\mathrm{d}y = 1$ under the integral sign and using $\nabla_\theta p = p\nabla_\theta \log p$ gives the zero-mean score,

$$\mathbb{E}_{p(x,y|\theta)}\left[\nabla_\theta \log p(x, y \mid \theta)\right] = 0. \tag{8.27}$$

Differentiating once more yields the two equivalent forms of the Fisher information,

$$F(\theta) = \mathbb{E}_{p(x,y|\theta)}\left[-\nabla_\theta \nabla_\theta^\top \log p(x, y \mid \theta)\right] = \mathbb{E}_{p(x,y|\theta)}\left[\nabla_\theta \log p(x, y \mid \theta)\left(\nabla_\theta \log p(x, y \mid \theta)\right)^\top\right]. \tag{8.28}$$

The FIM of independent random variables is additive. In particular, for N i.i.d. random variables $\{(\mathrm{X}_n, \mathrm{Y}_n)\}_{n=1}^N$, the FIM is N times that of a single random variable (X, Y):

$$\mathbb{E}_{p\left(\{(x_n,y_n)\}_{n=1}^N|\theta\right)}\left[-\nabla_\theta \nabla_\theta^\top \log p\left(\{(x_n, y_n)\}_{n=1}^N \mid \theta\right)\right] \tag{8.29}$$

$$= \sum_{n=1}^{N} \mathbb{E}_{p(x_n,y_n|\theta)}\left[-\nabla_\theta \nabla_\theta^\top \log p(x_n, y_n \mid \theta)\right] \tag{8.30}$$

$$= NF(\theta). \tag{8.31}$$

In the following, we use the notation $F(\theta)$ for the FIM of a single random variable (X, Y) distributed according to $p(x, y \mid \theta)$.

Using Equation 8.25, the utility of the FIM becomes clear in second-order optimisation. In particular, consider the objective

$$\underset{\Delta\theta \in \mathbb{R}^P}{\arg\min}\, \mathcal{L}(\theta + \Delta\theta) \qquad \text{s.t.} \qquad D_{\mathrm{KL}}(p(x, y \mid \theta) \parallel p(x, y \mid \theta + \Delta\theta)) \leq c \tag{8.32}$$

for some constant $c \in \mathbb{R}_{>0}$. The intuition behind this objective is that we want to minimise our model's non-regularised empirical risk while limiting how much the likelihood induced by the model changes, irrespective of the curvature of the loss surface. To find an approximate minimiser of the objective in Equation 8.32, we consider the quadratic approximation to our constraint in Equation 8.25 and linearise the empirical risk around the current parameter θ:

$$\mathcal{L}(\theta + \Delta\theta) = \mathcal{L}(\theta) + \Delta\theta^\top \nabla_\theta \mathcal{L}(\theta) + \mathcal{O}\left(\|\Delta\theta\|^2\right). \tag{8.33}$$

Substituting in the linearised empirical risk and quadratic constraint, our objective becomes

$$\underset{\Delta\theta \in \mathbb{R}^P}{\arg\min}\, \Delta\theta^\top \nabla_\theta \mathcal{L}(\theta) \qquad \text{s.t.} \qquad \frac{1}{2}\Delta\theta^\top F(\theta)\Delta\theta \leq c. \tag{8.34}$$

The above is a convex quadratically constrained linear program. When $F(\theta)$ is positive definite and $\nabla_\theta \mathcal{L}(\theta) \neq 0$, the optimum lies on the boundary of the constraint ellipsoid and can be found in closed form with Lagrange's method, yielding

$$\Delta\theta^* = -\frac{1}{\lambda^*} F(\theta)^{-1} \nabla_\theta \mathcal{L}(\theta), \qquad \lambda^* = \sqrt{\frac{\left(\nabla_\theta \mathcal{L}(\theta)\right)^\top F(\theta)^{-1} \nabla_\theta \mathcal{L}(\theta)}{2c}}. \tag{8.35}$$

In particular, the optimal update direction $\Delta\theta^*$ is given by the so-called **natural gradient** $F(\theta)^{-1}\nabla_\theta \mathcal{L}(\theta)$ with a magnitude that is a function of the constant c. Natural-gradient methods are developed in detail in Chapter 15.

In practice, we replace the unknown distribution of inputs $p(x)$ with the empirical distribution w.r.t. training set $\mathcal{D}$:

$$p(x) \approx p_{\mathcal{D}}(x) = \frac{1}{N} \sum_{n=1}^{N} \delta(x - x_n). \tag{8.36}$$

The corresponding FIM approximation becomes

$$\tilde{F}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{p(y|x_n,\theta)} \left[-\nabla_\theta \nabla_\theta^\top \log p(y \mid x_n, \theta) \right]. \tag{8.37}$$

Using the chain rule w.r.t. the model's output $z = f(x_n, \theta)$, we obtain

$$\tilde{F}(\theta_0) = \frac{1}{N} \sum_{n=1}^{N} \left(J_\theta f(x_n, \theta_0) \right)^\top \mathbb{E}_{p(y|x_n,\theta_0)} \left[-\nabla_z \nabla_z^\top \log p(y \mid z) \big|_{z=f(x_n,\theta_0)} \right] J_\theta f(x_n, \theta_0). \tag{8.38}$$

Here the network's own second-order term vanishes under the expectation by the zero-mean score in z (Equation 8.27 applied to $p(y \mid z)$), which is why the linearised and nonlinear networks share the same $\tilde{F}(\theta_0)$. The expectation over model outputs for each input can be expensive to compute. One approach to mitigate the cost is the Monte Carlo approximation:

$$\mathbb{E}_{p(y|x_n,\theta)} \left[-\nabla_z \nabla_z^\top \log p(y \mid z) \big|_{z=f(x_n,\theta)} \right] \approx -\frac{1}{M} \sum_{s=1}^{M} \nabla_z \nabla_z^\top \log p(y_s \mid z) \big|_{z=f(x_n,\theta)}, \tag{8.39}$$

with $y_s \sim p(y \mid x_n, \theta)$. For small M, we obtain a rather crude but cheap approximation. An alternative approach is to first rewrite $\tilde{F}(\theta)$ as

$$\tilde{F}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{p(y|x_n,\theta)} \left[\nabla_\theta \log p(y \mid x_n, \theta) \left(\nabla_\theta \log p(y \mid x_n, \theta) \right)^\top \right] \tag{8.40}$$

(Equation 8.28, applied to $p(y \mid x_n, \theta)$), then substitute the ground-truth labels instead of approximating the expectation:

$$\hat{F}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \nabla_\theta \log p(y_n \mid x_n, \theta) \left(\nabla_\theta \log p(y_n \mid x_n, \theta) \right)^\top, \tag{8.41}$$

called the **Empirical Fisher Information Matrix (EFIM)**. While this substitution may seem natural, Kunstner et al. [948] show that it has a detrimental effect for optimisation, where the predicted and ground-truth labels can be significantly different, especially at the beginning of training. However, for the Laplace approximation, we use a MAP estimate, which, if it generalises well, approximates the ground-truth label distribution well.

For a wide family of likelihoods, one does *not* have to approximate the Fisher $\tilde{F}(\theta)$ by Monte Carlo. This allows us to connect the FIM to the GGN under some assumptions. Identifying $\ell(z, y_n) = -\log p(y_n \mid z)$ and $f(x_n, \theta_0) = f^{\theta_0}(x_n, \theta_0)$ using Equation 8.1, in the case when the Hessian of the negative log-likelihood loss ℓ does not depend on the target y, the GGN is equal to the FIM (up to a constant scaling factor):

$$G(\theta_0) = N \tilde{F}(\theta_0). \tag{8.42}$$

In fact, the difference in the scaling factor has a natural interpretation. Previously, we saw that $F(\theta)$ is the FIM of the random variable (X, Y) distributed according to $p(x, y \mid \theta)$ and that the FIM is additive for independent random variables. Therefore, under this assumption, the GGN is equal to the FIM w.r.t. N i.i.d. observations $\{(x_n, y_n)\}_{n=1}^N$ distributed as $p(x, y \mid \theta)$ (and *not* as $p(x, y)$).

The requirement of the loss Hessian not depending on the target might seem arbitrary. However, this property is satisfied by a wide family of likelihoods, called **exponential families**.

Definition 8.1. *A probability distribution with probability density function (PDF) of the form*

$$p(y \mid w) = h(y) \exp\left(\phi(y)^\top w - \log Z(w) \right) \tag{8.43}$$

*is called an **exponential family** of probability measures. The function $\phi \colon \mathcal{Y} \to \mathbb{R}^d$ is the vector of **sufficient statistics**. The parameters $w \in \mathbb{R}^d$ are the **natural parameters** of $p(y \mid w)$. The normalisation constant $Z(w) \colon \mathbb{R}^d \to \mathbb{R}$ is the **partition function**. The function $h(y) \colon \mathcal{Y} \to \mathbb{R}_{>0}$ is the **base measure**.*

First, we state some facts about simple parameterisations.

Proposition 8.2. *For a probability distribution $p(y \mid w)$, $-\nabla_w \nabla_w^\top \log p(y \mid w)$ is independent of y if and only if $p(y \mid w)$ is an exponential family with natural parameter w.*

Parameterisations via neural networks are more delicate. The following proposition gives a necessary and sufficient condition for Equation 8.42.

Proposition 8.3. *For an exponential family $p(y \mid x, \theta)$ with natural parameters w given as a function of x and θ, $w(x, \theta) = (g \circ f)(x, \theta)$ for some function $g \colon \mathbb{R}^C \to \mathbb{R}^d$, the Hessian of $-\log p(y \mid x, \theta)$ w.r.t. $f(x, \theta)$ does not depend on y if and only if $\nabla_z \nabla_z^\top g(z)\big|_{z=f(x,\theta)} \cdot \phi(y)$ is constant in y where $\nabla_z \nabla_z^\top g(z)\big|_{z=f(x,\theta)} \in \mathbb{R}^{C \times C \times d}$ is a 3-tensor, and the product $\cdot$ contracts its third (d-dimensional) index against $\phi(y)$, giving a $C \times C$ matrix. A special case of the statement is when g is affine.*

The above statement on neural network parameterisations covers the most popular loss functions in classification and regression. In classification, the cross-entropy loss is the negative log-likelihood of a categorical distribution parameterised by our neural network. The logits are the categorical's natural parameters, so with the identity link $g = \mathrm{Id}$ the cross-entropy loss satisfies Proposition 8.3 in its affine special case. Similarly, the L_2 loss for regression satisfies Proposition 8.3 with $g = \mathrm{Id}$.

A word of caution is in order: the GGN/FIM alone is usually degenerate; thus, it does not serve as a valid precision matrix for the Laplace approximation. Considering a single input-label pair (x_n, y_n), the term

$$(J_\theta f(x_n, \theta_0))^\top \left(\nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f^{\theta_0}(x_n,\theta_0)} \right) J_\theta f(x_n, \theta_0) \tag{8.44}$$

is a positive *semidefinite* (PSD) matrix, as the loss Hessian $\nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f^{\theta_0}(x_n,\theta_0)}$ is positive semidefinite. For softmax cross-entropy, this Hessian is $\mathrm{diag}(p) - pp^\top$, which is rank deficient as the softmax is shift-invariant. Usually, the GGN, that is, the sum of these matrices over the training dataset, is still rank deficient. However, adding the Hessian of the strongly convex regulariser term $\Omega(\theta_0)$ to the GGN results in the *positive definite* matrix

$$\Psi(\theta_0) = \nabla_\theta \nabla_\theta^\top \mathcal{L}_\Omega^{\theta_0}(\theta_0) = \nabla_\theta \nabla_\theta^\top \mathcal{L}^{\theta_0}(\theta_0) + \nabla_\theta \nabla_\theta^\top \Omega(\theta_0), \tag{8.45}$$

which is the Hessian of the regularised empirical risk of the linearised model, $\mathcal{L}_\Omega^{\theta_0}(\theta_0)$. This justifies using this matrix as a parameter-space precision in the Laplace approximation (Equation 7.4c).

With the curvature Ψ in hand, we can place the NTK of Section 8.2 and the posterior GP kernel of Equation 8.16 in a single frame. Both are Jacobian Gram matrices $J_\theta f M^{-1} J_\theta f^\top$ in a parameter-space metric M; they differ in the expansion point (θ_0 for the NTK, θ_* for the posterior kernel) and in the metric M. Preconditioning the gradient flow of Equation 8.20 by M multiplies the parameter velocity by M^{-1}, inserting M^{-1} between the Jacobians and replacing its kernel k_t with $J_\theta f(x, \theta_t) M^{-1} J_\theta f(x_n, \theta_t)^\top$, holding the expansion point fixed and changing only the metric. Vanilla gradient descent uses the Euclidean metric $M = I$ and recovers the NTK, the isotropic-prior pushforward. Regularised natural-gradient descent [26] uses $M = \Psi(\theta_t)$, the GGN/Fisher curvature plus the prior precision (Equation 8.45) at the current parameters. We call the resulting kernel $J_\theta f(\cdot, \theta_t) \Psi(\theta_t)^{-1} J_\theta f(\cdot, \theta_t)^\top$ the **natural-gradient NTK**. Because $\Psi(\theta_t)$ is positive definite at every θ_t, the natural-gradient NTK governs the dynamics all along training, even where the gradient is nonzero and no Laplace posterior exists. It coincides with the posterior GP kernel $J_\theta f \Psi^{-1} J_\theta f^\top$ [813] only at a posterior mode θ_*, as the Laplace approximation requires. The parameter-space metric thus determines both the predictive uncertainty, through the Gaussian we push forward, and the optimisation geometry, through the descent the function follows.

Takeaways

For common losses (exponential-family with an *affine* link; e.g., squared loss, cross-entropy), the curvature of the linearised model satisfies

$$G(\theta_0) = N \tilde{F}(\theta_0),$$

identifying the GGN with the Fisher Information Matrix (FIM) over N i.i.d. samples, and tying the Riemannian and information-geometric views.

The Fisher Information Matrix is the Riemannian metric describing the local behaviour of the KL divergence:

$$D_{\mathrm{KL}}\left(p(x, y \mid \theta_1) \parallel p(x, y \mid \theta_2)\right) = \frac{1}{2} \Delta\theta^\top F(\theta_1) \Delta\theta + \mathcal{O}\left(\|\Delta\theta\|^3\right).$$

Approx.	Storage	Cov.-vec.	Inversion	Correlations	Blind spot
Diag.	$\mathcal{O}(P)$	Elementwise division	Elementwise	None across parameters	Misses coupled directions
Low-rank	$\mathcal{O}(kP)$	Products with $Q_{1:k}$	Woodbury	Top eigenspace of G	Misses broad low-curvature directions
KFAC	$\sum_i \mathcal{O}(d_{i-1}^2 + d_i^2)$	Kronecker identities	Factor eigensystems	Within-layer Kronecker	Drops cross-layer couplings

Table 8.1: Common scalable curvature approximations for linearised Laplace. Here d_{i-1} and d_i are the input and pre-activation dimensions of layer i.

In practice, one uses the empirical input distribution to form the FIM. The matrix G is PSD and typically singular; adding a strongly convex regulariser makes

$$\Psi(\theta_0) = G(\theta_0) + \nabla_\theta \nabla_\theta^\top \Omega(\theta_0) \succ 0,$$

which is the invertible precision used by the Laplace approximation.
The metric M in the form $J_\theta f M^{-1} J_\theta f^\top$ sets which kernel we obtain: $M = I$ gives the NTK (vanilla gradient descent) and $M = \Psi$ gives the natural-gradient NTK (natural-gradient descent), which becomes the posterior GP kernel at a mode.

8.4 Scalable Curvature Approximations

The GGN of the linearised network (cf. Equation 8.22)

$$G(\theta) = \sum_{n=1}^{N} \left(J_\theta f(x_n, \theta)\right)^\top \underbrace{\nabla_z \nabla_z^\top \ell\left(z, y_n\right)\big|_{z=f(x_n,\theta)}}_{=:H_n \succeq 0} J_\theta f(x_n, \theta) \in \mathbb{R}^{P \times P} \tag{8.46}$$

is dense in general and *far* too large to form for modern neural networks. We therefore seek structured surrogates that (i) are cheap to store and invert, yet (ii) retain meaningful parameter correlations. Table 8.1 compares the three curvature proxies we develop: the diagonal, low-rank, and Kronecker-Factored Approximate Curvature (KFAC) approximations. Scalable variants of these curvature approximations are developed further in Chapter 40, and last-layer inference is treated in Chapter 39.

A simple baseline is the diagonal $\operatorname{diag} G(\theta)$, which is trivial to store and invert, though calculating it efficiently requires care. However, it encodes an independence assumption between *all parameters*, destroying the often-crucial within-layer correlations carried by $G(\theta)$. This tends to hurt the utility of the downstream uncertainty estimates [380].

8.4.1 Low-rank GGN Approximations

A scalable yet expressive alternative is a *low-rank* approximation of $G(\theta)$. As $G(\theta)$ is symmetric for any $\theta \in \mathbb{R}^P$, it possesses an eigendecomposition with real eigenvalues of the form

$$G(\theta) = Q(\theta)\Lambda(\theta)Q(\theta)^\top, \tag{8.47}$$

where $Q(\theta) \in \mathbb{R}^{P \times P}$ is an orthogonal matrix (i.e., a matrix with ortho*normal* columns/rows) and $\Lambda(\theta) \in \mathbb{R}^{P \times P}$ is a diagonal matrix.

For $q_{:,i}$, the ith column of $Q(\theta)$, and λ_i, the ith diagonal entry of $\Lambda(\theta)$, it holds that $G(\theta)q_{:,i} = \lambda_i q_{:,i}$. We follow the convention $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_P$. In the following, we drop the dependence of Q and its factors on θ for brevity.

We now seek a *good* low-rank approximation $\tilde{G}_k$ to G, as this reduces the sizes of the objects we need to handle. Since $G \succeq 0$, its eigenvalues are nonnegative, so the k algebraically largest coincide with the k largest in magnitude. The Eckart–Young–Mirsky theorem [484] states that the objective

$$\operatorname*{arg\,min}_{G' \in \mathbb{R}^{P \times P} : \operatorname{rank}(G') \leq k} \|G - G'\| \tag{8.48}$$

is minimised by $\tilde{G}_k := Q_{1:k}\Lambda_{1:k}Q_{1:k}^\top$ where $Q_{1:k} \in \mathbb{R}^{P\times k}$ is the first k columns of Q and $\Lambda_{1:k} \in \mathbb{R}^{k\times k}$ holds the first k eigenvalues of G. This result holds for both the spectral norm $\|\cdot\|_2$ and the Frobenius norm $\|\cdot\|_F$. Therefore, one obtains an *optimal* low-rank approximation $\tilde{G}_k$ to G in the above sense by computing the first k eigenvalues and eigenvectors of G. Storing this low-rank approximation requires storing $(P+1)k$ values instead of P^2.

The most naive way to compute $\tilde{G}_k$ is first calculating the eigendecomposition $G = Q\Lambda Q^\top$ and then truncating it to the first k eigenpairs. However, this strategy does not resolve the infeasibility of storing the full eigendecomposition with $P(P+1)$ values. Instead, one uses efficient *matrix-free* schemes such as the Locally Optimal Block Preconditioned Conjugate Gradient (LOBPCG) method [907] or Lanczos iterations [969] to obtain the low-rank decomposition. Notably, these algorithms only require *matrix-vector products* $v \mapsto Gv$ with the GGN to obtain the top eigenpairs, which does not require representing the entire G matrix in memory.

The low-rank representation is particularly handy, as for simple priors, such as isotropic Gaussians, calculating the approximate covariance matrix is just as efficient as working with the low-rank GGN. Let $\Gamma \succ 0$ be the Hessian of the negative log-prior Ω, that is, the prior-curvature term $\nabla_\theta\nabla_\theta^\top\Omega$ of the precision $\Psi = G + \nabla_\theta\nabla_\theta^\top\Omega$ (Equation 8.45). Then the Woodbury matrix identity yields

$$\left(\Gamma + \tilde{G}_k\right)^{-1} = \Gamma^{-1} - \Gamma^{-1}Q_{1:k}\left(\Lambda_{1:k}^{-1} + Q_{1:k}^\top\Gamma^{-1}Q_{1:k}\right)^{-1}Q_{1:k}^\top\Gamma^{-1}. \tag{8.49}$$

In particular, for isotropic Gaussians, $\Gamma = \tau I$,

$$\left(\tau I + Q_{1:k}\Lambda_{1:k}Q_{1:k}^\top\right)^{-1} = \tau^{-1}I + Q_{1:k}\,\mathrm{diag}\left(-\frac{\lambda_1}{\tau(\lambda_1+\tau)},\dots,-\frac{\lambda_k}{\tau(\lambda_k+\tau)}\right)Q_{1:k}^\top. \tag{8.50}$$

In words: a diagonal-plus-low-rank matrix has a diagonal-plus-low-rank inverse. Multiplying by such a matrix is efficient and does not require forming a $P \times P$ matrix. For a vector $v \in \mathbb{R}^P$, we first calculate $u = Q_{1:k}^\top v \in \mathbb{R}^k$, then elementwise multiply u by the diagonal entries in $\mathrm{diag}\left(-\frac{\lambda_1}{\tau(\lambda_1+\tau)},\dots,-\frac{\lambda_k}{\tau(\lambda_k+\tau)}\right)$, then map the result back to $\mathbb{R}^P$ with $Q_{1:k}$ and add the diagonal term $\tau^{-1}v$.

8.4.2 Kronecker-Factored Approximate Curvature

A complementary path towards scalable approximate curvature representations assumes independence *between layers* only, while retaining rich *within-layer* parameter correlations via a Kronecker factorisation. This idea underlies KFAC, which has recently seen wide use as a scalable curvature proxy [1136]. In this section, we give an introduction to KFAC for dense linear layers and refer to Dangel et al. [367] for a more detailed and general exposition of KFAC.

Consider layer i with weights $W^{(i)} \in \mathbb{R}^{d_i\times d_{i-1}}$, input activation $a_n^{(i-1)} \in \mathbb{R}^{d_{i-1}}$, and pre-activation $s_n^{(i)} = W^{(i)}a_n^{(i-1)} \in \mathbb{R}^{d_i}$.[4] Let $z_n = f(x_n,\theta) \in \mathbb{R}^C$ denote the network output. Let $J_{W^{(i)}}s_n^{(i)} \in \mathbb{R}^{d_i\times(d_id_{i-1})}$ denote the Jacobian of $s_n^{(i)}$ w.r.t. $\mathrm{vec}(W^{(i)})$, where vec flattens its input by stacking its columns. For datum n, define the *output-space* curvature $H_n := \nabla_z\nabla_z^\top\ell(z,y_n)\big|_{z=z_n} \succeq 0$, so $H_n \in \mathbb{R}^{C\times C}$. Writing $H_n = S_nS_n^\top$ with $S_n \in \mathbb{R}^{C\times C}$, the corresponding layer block of the GGN can be expressed as

$$\begin{aligned} G^{(i)} &= \sum_{n=1}^N (J_{W^{(i)}}s_n^{(i)})^\top \underbrace{(J_{s_n^{(i)}}z_n)^\top H_n (J_{s_n^{(i)}}z_n)}_{=:\,\widehat{H}_n^{(i)}\succeq 0} J_{W^{(i)}}s_n^{(i)} \\ &= \sum_{n=1}^N\sum_{c=1}^C (J_{W^{(i)}}s_n^{(i)})^\top g_{n,c}^{(i)}(g_{n,c}^{(i)})^\top J_{W^{(i)}}s_n^{(i)}. \end{aligned} \tag{8.51}$$

with $g_{n,c}^{(i)} := (J_{s_n^{(i)}}z_n)^\top[S_n]_{:,c} \in \mathbb{R}^{d_i}$, where $J_{s_n^{(i)}}z_n$ is the Jacobian of the network output z_n w.r.t. this layer's pre-activation $s_n^{(i)}$.[5] Thus, the sum over c is over the network output curvature factor, while $g_{n,c}^{(i)}$ lives in the layer pre-activation space.

For a linear layer, the output-parameter Jacobian has the form $J_{W^{(i)}}s_n^{(i)} = (a_n^{(i-1)})^\top \otimes I_{d_i}$ where $\otimes$ denotes the Kronecker product. Noting the Kronecker properties $(A\otimes B)^\top = A^\top\otimes B^\top$ and $(A\otimes B)(C\otimes D) = (AC)\otimes(BD)$

[4] Bias terms are omitted for brevity.

[5] For the squared loss, $H_n = I$; for cross-entropy, $H_n = \mathrm{diag}(\mathrm{softmax}(z)) - \mathrm{softmax}(z)\,\mathrm{softmax}(z)^\top$ is the softmax Hessian.

for compatible matrices A, B, C, D, we get

$$G^{(i)} = \sum_{n=1}^{N} \sum_{c=1}^{C} \underbrace{(a_n^{(i-1)}(a_n^{(i-1)})^\top)}_{\text{input factor}} \otimes \underbrace{(g_{n,c}^{(i)}(g_{n,c}^{(i)})^\top)}_{\text{backpropagated factor}} \in \mathbb{R}^{(d_{i-1}d_i)\times(d_{i-1}d_i)}. \tag{8.52}$$

The KFAC approximation then pulls the Kronecker product outside the sum:

$$\begin{aligned} &\sum_{n=1}^{N} \sum_{c=1}^{C} \left(a_n^{(i-1)}(a_n^{(i-1)})^\top\right) \otimes \left(g_{n,c}^{(i)}(g_{n,c}^{(i)})^\top\right) \\ &\approx \underbrace{\left(\sum_{n=1}^{N} a_n^{(i-1)}(a_n^{(i-1)})^\top\right)}_{=:\, A^{(i)}} \otimes \underbrace{\left(\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} g_{n,c}^{(i)}(g_{n,c}^{(i)})^\top\right)}_{=:\, B^{(i)}}. \end{aligned} \tag{8.53}$$

Thus, the layer block is approximated by

$$G^{(i)} \approx A^{(i)} \otimes B^{(i)} =: \tilde{G}^{(i)}, \qquad A^{(i)} \in \mathbb{R}^{d_{i-1}\times d_{i-1}}, B^{(i)} \in \mathbb{R}^{d_i \times d_i}. \tag{8.54}$$

Stacking layers and *neglecting inter-layer couplings* yields a block-diagonal curvature with Kronecker blocks $A^{(i)} \otimes B^{(i)}$.

The Kronecker form is efficient to store. While the dense GGN $G^{(i)}$ for a single linear layer has $(d_{i-1}d_i)^2$ entries, the Kronecker factorisation only requires storing $A^{(i)}$ and $B^{(i)}$, that is, a total of $d_{i-1}^2 + d_i^2$ values. The block-diagonal layer approximation then avoids storing cross-layer blocks. One can also calculate efficient covariance-vector products without forming a dense $P \times P$ matrix by leveraging the cached eigendecompositions of the Kronecker factors and standard Kronecker identities. This property is more involved to show than in the case of low-rank approximations; refer to Dangel et al. [367] for more details.

Takeaways

KFAC assumes independence *across* layers but preserves rich *within-layer* correlations through a Kronecker factorisation of each layer's curvature block: $G^{(i)} \approx A^{(i)} \otimes B^{(i)}$. It is far more expressive than diagonal curvature structures and complementary to low-rank approximations (usually at lower cost). Storing the Kronecker factors is cheap. With an isotropic prior precision, covariance-vector products can be implemented remarkably efficiently. See Dangel et al. [367] for implementation details and extensions.

8.5 The Predictive

Equation 8.13 gives a Gaussian distribution in the output space for input x_*. For regression, this distribution is called the **predictive distribution**: its mean encodes the average prediction under the parameter-space posterior and the linearised model, and its covariance structure quantifies the uncertainty in the prediction, including correlations of the outputs. Figure 8.3 shows a one-dimensional regression example of the linearised Laplace approximation.

In classification, however, we are interested in obtaining a second-order distribution (a distribution over distributions) over the *categorical* distributions of a discrete set of labels. To this end, one has to map the logit-space Gaussian measure in Equation 8.13 through an activation function onto the $(C-1)$-dimensional probability simplex:

$$\mathrm{h}\colon \mathbb{R}^D \xrightarrow{\mathrm{f}} \mathcal{G}\left(\mathbb{R}^C\right) \xrightarrow{\varphi_*} \mathcal{P}\left(\Delta^{C-1}\right) \tag{8.55}$$

where f is the mapping from an input x_* to the logit-space Gaussian distribution given by the Laplace-approximated network, $\mathcal{G}(\mathbb{R}^C)$ is the set of Gaussian measures on the logit space $\mathbb{R}^C$, $\mathcal{P}\left(\Delta^{C-1}\right)$ is the set of probability measures on the simplex, and φ_* pushes forward the Gaussian probability measures through the activation function. The roman f is the network with its parameter posterior pushed through it: the affine pushforward of the Laplace posterior $\mathcal{N}(\theta; \theta_*, \Psi(\theta_*)^{-1})$ along the linearised network, giving $\mathrm{f}(x_*) = \mathcal{N}(f(x_*, \theta_*), J_\theta f(x_*, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x_*, \theta_*)^\top) \in \mathcal{G}(\mathbb{R}^C)$ as in Equation 8.13. The italic $f(x_*, \theta_*) \in \mathbb{R}^C$ plugs in the point estimate θ_* and returns a single logit vector; the roman $\mathrm{f}(x_*)$ returns the full logit-space Gaussian, whose mean is exactly that italic vector. All parameters in the support of the Laplace posterior are taken into account rather than a single one being held fixed, so f takes

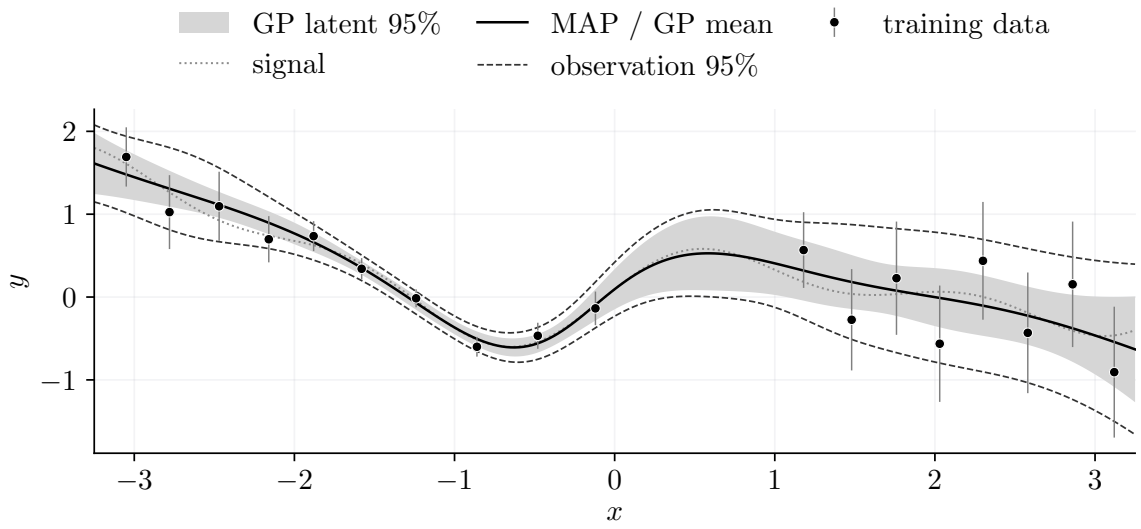


Figure 8.3: A one-dimensional regression example with irregular inputs and input-dependent observation noise. The model is a neural network with a Bayesian last layer, so the linearisation is exact and the linearised GP yields the exact posterior predictive. The shaded region shows the pointwise 95% latent GP interval, while the dashed curves add the input-dependent observation variance.

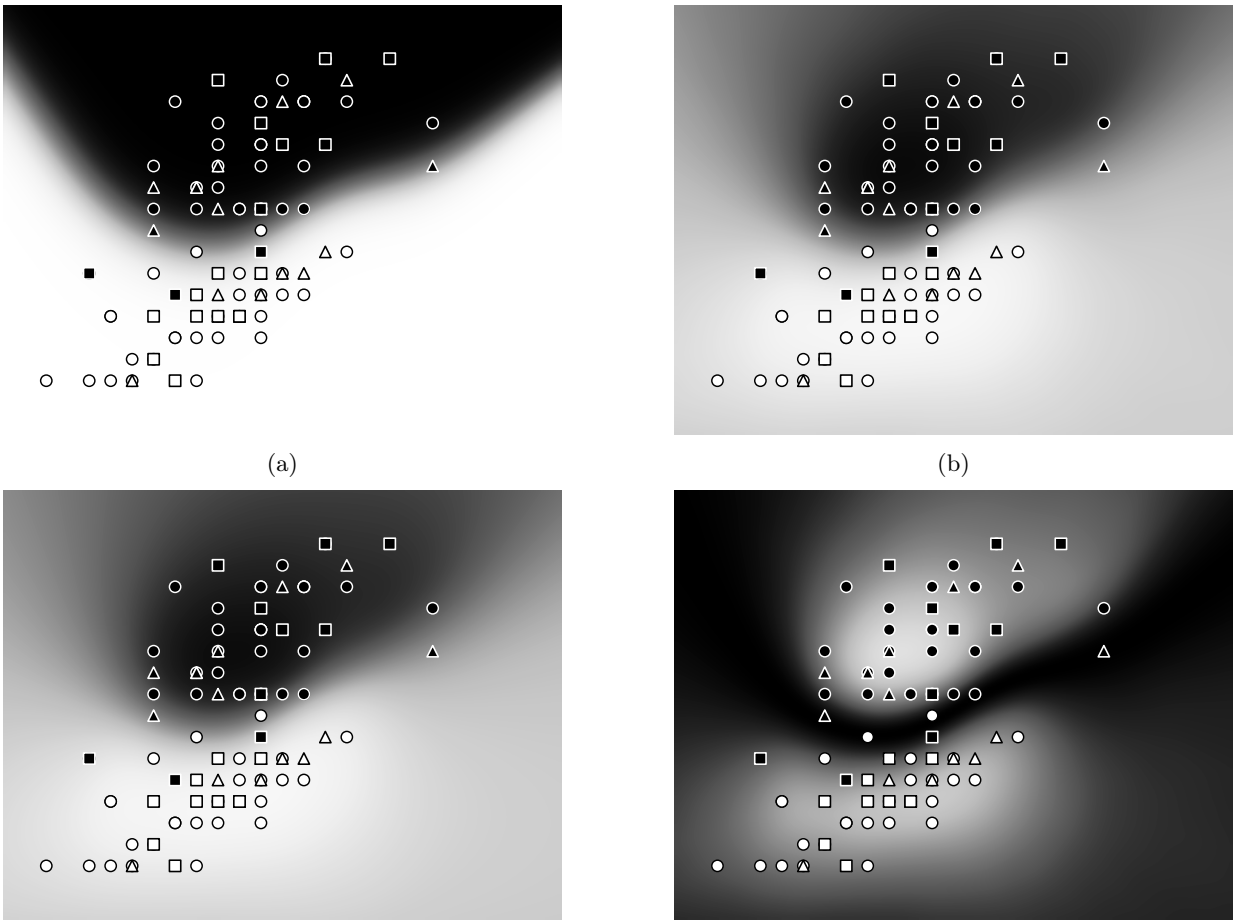
only the input. In words, h is the composition of the Laplace-approximated neural network outputting logit-space Gaussian distributions, f, and the pushforward map through the activation function that gives us a distribution over probability vectors.

In most cases, this activation function is the softmax, defined as $\text{softmax}\colon \mathbb{R}^C \to \Delta^{C-1}$,

$$\text{softmax}_c(z) = \frac{\exp(z_c)}{\sum_{c'=1}^{C} \exp(z_{c'})}. \tag{8.56}$$

In this case, there exists no closed-form expression for $\text{h}(x_*) \in \mathcal{P}\left(\Delta^{C-1}\right)$ given input x_*. Further, $\mathbb{E}_{P\sim\text{h}(x_*)}[P]$, called the posterior predictive, is also intractable. However, sampling from $\text{h}(x_*)$ is efficient, as it only requires sampling from the logit-space Gaussian $\text{f}(x_*)$ and pushing these logit samples forward through the activation function. Thus, in practice, one resorts to Monte Carlo integration, that is, drawing M samples from $\text{h}(x_*)$ and calculating their empirical average. Similarly, one can approximate other derived quantities, such as the expected entropy $\mathbb{E}_{P\sim\text{h}(x_*)}[\text{H}(P)]$, through Monte Carlo sampling. Such derived quantities are often used in the field of uncertainty disentanglement, where different quantities capture different sources of uncertainty [1204]. These uncertainty estimates are applied to selective classification in Chapter 44.

The main issue with the above schemes is the requirement of sampling: the error of Monte Carlo integration decreases as the inverse square root of the number of samples M, meaning that to reduce the error by three orders of magnitude, one needs a million samples. Standard softmax-cross-entropy integration cannot avoid this sampling cost without incurring significant approximation error. However, some output activation functions admit a *closed-form* posterior predictive. For a single output, integrating a normCDF (probit) link against a Gaussian is exact, and the logistic sigmoid admits a classical probit approximation [1106]. Mucsányi et al. [1205] show that the normalised elementwise sigmoid and normCDF functions yield closed-form predictives that have strong approximation guarantees and are naturally compatible with more performant training objectives than the standard softmax-cross-entropy objective. Further, derived quantities such as the expected entropy can also be obtained in closed form by matching the moments of the Gaussian pushforward distributions with Dirichlets. The change in the training objective comes at no cost: models trained this way become better calibrated while providing sample-free posterior predictives and other derived quantities. Therefore, Laplace approximations in classification can be formulated in a purely sample-free fashion, with theoretical guarantees. Figure 8.4 compares the MAP, Monte Carlo softmax, and sample-free probit predictives on a two-dimensional example, together with the predictive entropy.



(a) (b) (c) (d)

Figure 8.4: Two-dimensional binary classification with a Laplace approximation to the last layer of a fixed-feature network on the Iris dataset. The inputs are the measured sepal width and petal width. White denotes Iris versicolor and black denotes Iris virginica. Circles are training points, triangles are validation points, and squares are test points. The prior precision is selected on the validation set by log score. Panels (a)–(c) show the probability of the black class for the MAP predictor, the softmax posterior predictive estimated by Monte Carlo sampling, and the sample-free probit approximation. Panel (d) shows the predictive entropy computed from the Monte Carlo predictive. On the held-out test points, the MAP log score and Brier score are 0.433 and 0.122, while the Monte Carlo GP predictive gives 0.345 and 0.107, and the probit predictive gives 0.347 and 0.108—close to indistinguishable.

Takeaways

For regression, the linearised Laplace output at input x_* is *Gaussian* (Equation 8.13); its mean is $f(x_*, \theta_*)$ with covariance $J_\theta f(x_*, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x_*, \theta_*)^\top$.

In classification with softmax, pushing this logit-space Gaussian through the activation yields a distribution on the simplex with *no closed form* for the posterior predictive $\mathbb{E}_{P\sim \mathrm{h}(x_*)}[P]$, so one typically uses Monte Carlo with $\mathcal{O}(M^{-1/2})$ error.

Replacing softmax by a normalised elementwise sigmoid or normCDF output activation makes the posterior predictive (and derived quantities such as expected entropy via Dirichlet moment matching) *closed-form* with approximation guarantees, usually improving calibration and rendering Laplace classification *sample-free*.

Chapter 9

The differential geometric perspective on Laplace

Linearised Laplace approximations (Section 8.1), and specifically their covariance structure, have interpretations in Riemannian and information geometry. They encode the neural network's parameter space geometry, as informed by the loss.

9.1 The GGN Metric

In the previous chapter, we discussed in detail the curvature matrix defined in Equation 8.4, called the Generalised Gauss-Newton matrix (GGN). It is the precision term from the likelihood and, combined with the precision term from the prior and inverting, it yields the covariance matrix of a linearised Laplace approximation.

Recall that the GGN matrix at the parameters $\theta \in \mathbb{R}^P$ is given by (Equation 8.22)

$$G(\theta) = \nabla_\theta \nabla_\theta^\top \mathcal{L}^\theta(\theta) = \sum_{n=1}^{N} \left(J_\theta f(x_n, \theta)\right)^\top \left(\nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f(x_n,\theta)}\right) J_\theta f(x_n, \theta). \tag{9.1}$$

We will now see how the GGN arises as a metric on the parameter space. Recall our training data $\mathcal{D} = \{(x_n, y_n)\}_{n=1,\dots,N}$. We restrict the neural network $f\colon \mathbb{R}^D \times \mathbb{R}^P \to \mathbb{R}^C$ to the training set, to obtain the function

$$\begin{aligned} f_X\colon \mathbb{R}^P &\to \left(\mathbb{R}^C\right)^N \\ \theta &\mapsto (f(x_n, \theta))_{n=1}^N. \end{aligned} \tag{9.2}$$

For a single input-label pair (x_n, y_n), assuming $\ell(z, y_n)$ is strictly convex (which is the case, for example, for the squared loss in regression), the positive definite loss Hessian

$$H_n(z) := \nabla_z \nabla_z^\top \ell(z, y_n) \tag{9.3}$$

characterises a **Riemannian metric** in the output space $\mathbb{R}^C$. That is, it defines a symmetric positive definite bilinear form $(v_1, v_2) \mapsto v_1^\top H_n(z) v_2$ at each point $z \in \mathbb{R}^C$. Such a metric encodes the *curvature* of the loss given the target y_n. Note that if the likelihood is an exponential family, then H_n is independent of the target y_n (Proposition 8.2).

In fact, we only require $\ell(z, y_n)$ to be (not necessarily strictly) convex, in which case H_n is only positive semidefinite. This is the case for the cross-entropy loss in classification, where the Hessian is rank-deficient. In this case, H_n defines a *degenerate* Riemannian metric on $\mathbb{R}^C$.

We can then equip the image of f_X, $(\mathbb{R}^C)^N$, with the block diagonal (degenerate) Riemannian metric

$$H(z) = \begin{bmatrix} H_1(z_1) & 0 & \cdots & 0 \\ 0 & H_2(z_2) & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & H_N(z_N) \end{bmatrix} \tag{9.4}$$

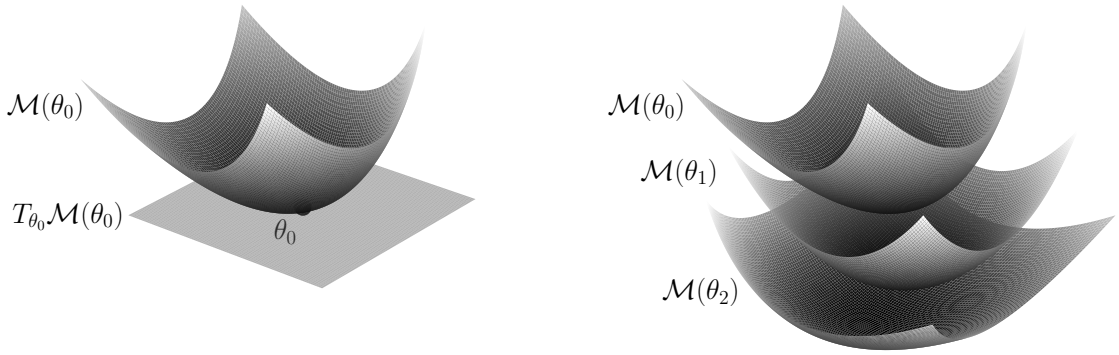


Figure 9.1: *Left:* A fiber manifold $\mathcal{M}(\theta_0)$ in the parameter space, and its tangent space $T_{\theta_0}\mathcal{M}(\theta_0)$ at θ_0. The fiber manifold consists of the parameters yielding the same output values on the training data as θ_0 (Equation 9.7), the tangent space consists of the kernel directions of the GGN at θ_0. *Right:* A collection of three fibers in parameter space. The parameter space is partitioned into a disjoint union of such fibers.

in standard coordinates, where $z = (z_1, \dots, z_N)$. This metric characterises the curvature of the loss on the training set. It can then be pulled back through f_X to the parameter space:

$$\begin{aligned}(f_X^* H)(\theta) &= J_\theta f_X(\theta)^\top H(f_X(\theta)) J_\theta f_X(\theta) \\ &= \sum_{n=1}^{N} J_\theta f(x_n, \theta)^\top H_n(f(x_n, \theta)) J_\theta f(x_n, \theta) \\ &= \sum_{n=1}^{N} \left(J_\theta f(x_n, \theta)\right)^\top \left(\nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f(x_n,\theta)}\right) J_\theta f(x_n, \theta) = G(\theta),\end{aligned} \tag{9.5}$$

i.e., the **pullback** $f_X^* H$ corresponds exactly to the GGN G.

Due to symmetries in the neural network architecture (see Part VII) and overparametrisation, the GGN is rank-deficient and only positive *semi*definite. It therefore defines again a *degenerate* Riemannian metric on the parameter space.[1] Thus, *damping* is needed to get a proper Riemannian metric on the parameter space. The strictly convex regulariser term $\Omega(\theta)$ is such a dampener: adding its Hessian to the GGN results in a positive definite matrix (cf. Equation 8.45)

$$\nabla_\theta \nabla_\theta^\top \mathcal{L}_\Omega^\theta(\theta) = \nabla_\theta \nabla_\theta^\top \mathcal{L}^\theta(\theta) + \nabla_\theta \nabla_\theta^\top \Omega(\theta). \tag{9.6}$$

Using this as a preconditioner in second-order optimisation means we perform Riemannian (stochastic) gradient descent in the parameter space.

But how do we interpret the GGN metric? We base our presentation on Roy et al. [1427]. Fixing a parameter vector θ_0, consider the set

$$\mathcal{M}(\theta_0) = \{\theta \in \mathbb{R}^P : f_X(\theta) = f_X(\theta_0)\} \tag{9.7}$$

i.e., the set of parameters θ for which the neural network defines the same function on the training data as θ_0. This is an equivalence class of parameters, called a **fiber**. By the preimage theorem [1640, Theorem 9.9], it is (locally) a manifold of dimension $P - NC$ if f_X is (locally) continuously differentiable and the Jacobians $J_\theta f_X(\theta)$ are surjective for all (local) $\theta \in \mathcal{M}(\theta_0)$. Intuitively, the parameter space is P-dimensional but we constrain the model's prediction on the training set of N samples, meaning we put NC scalar constraints on the output. This eliminates NC degrees of freedom, yielding a $(P - NC)$-dimensional manifold. The surjectivity condition holds in particular when the eigenvalues of the neural tangent kernel matrix over the entire training set are bounded away from 0, a common assumption for overparametrised models, see Roy et al. [1427, Appendix A] and the references therein.

Further, when the Jacobians $J_\theta f_X(\theta)$ are surjective locally around θ_0, the preimage theorem further tells us that $\ker J_\theta f_X(\theta_0) = T_{\theta_0}\mathcal{M}(\theta_0)$, the tangent space of $\mathcal{M}(\theta_0)$ at θ_0 (see Figure 9.1). If the Hessians $\nabla_z \nabla_z^\top \ell(z, y_n)$ are full rank (for example, when $\ell(z, y_n) = \|z - y_n\|^2$ the squared loss), we get $\ker G(\theta_0) = T_{\theta_0}\mathcal{M}(\theta_0)$. These

[1] Not to be confused with pseudo-Riemannian metrics.

are directions in which the training data gives us no information about the parameters. Thus, in such directions, linearised Laplace approximations solely rely on the prior. The orthogonal complement of this null space consists of the directions in which the training data gives us information.

Now having defined the fibers in parameter space, we can quotient them out to obtain the **effective parameter space**: $\mathbb{R}^P/\sim$ where $\sim$ is the equivalence relation induced by f_X, $\theta_0 \sim \theta_1 \Leftrightarrow f_X(\theta_0) = f_X(\theta_1)$. The effective parameters are parameters for which the neural network f takes *distinct* values on the training data. Formally, the effective parameter space is a *fiber bundle* over the parameter space. It can be thought of as a manifold perpendicular to the fibers $\mathcal{M}(\theta)$; Roy et al. [1427] construct such a manifold explicitly. We note, however, that this is an arbitrary choice, as there is no canonical way to choose representatives for the equivalence classes and hence such a manifold. As a consequence, the non-linear effective parameter space is difficult to leverage computationally. However, the GGN's degenerate metric on the original parameter space $\mathbb{R}^P$ captures the similarity of training outputs between different parameters. One therefore does not have to construct the effective parameter space explicitly, but rather use the GGN's geometry in the parameter space.

Takeaways

The GGN is the **pullback** through the restricted network f_X of the loss-curvature metric on the training outputs,

$$G(\theta) = (f_X^* H)(\theta) = J_\theta f_X(\theta)^\top \, H(f_X(\theta)) \, J_\theta f_X(\theta),$$

and therefore defines a (degenerate) Riemannian metric on the parameter space $\mathbb{R}^P$. Its null space coincides with the tangent space $T_{\theta_0}\mathcal{M}(\theta_0)$ of the *fiber* $\mathcal{M}(\theta_0)$ of parameters producing identical outputs on the training data: these are the directions in which the data is uninformative and a linearised Laplace approximation falls back on the prior. Damping (e.g. a strictly convex regulariser Ω) turns this degenerate metric into a proper one.

9.2 The Fisher Metric

In the previous chapter we saw that the GGN metric can be seen as the pullback of the loss curvature metric on the training outputs. We now ask whether there is a more intrinsic, parametrisation-independent justification for this metric. We offer a such a perspective on the GGN metric via information geometry.

Recall from Chapter 8, Equation 8.26 the **Fisher information metric**

$$\begin{aligned} F(\theta) &= \mathbb{E}_{p(x,y|\theta)} \left[-\nabla_\theta \nabla_\theta^\top \log p(x, y \mid \theta) \right] \\ &= \mathbb{E}_{p(x,y|\theta)} \left[\nabla_\theta \log p(x, y \mid \theta) (\nabla_\theta \log p(x, y \mid \theta))^\top \right] \end{aligned} \tag{9.8}$$

which we now view as a degenerate Riemannian metric on the parameter space $\mathbb{R}^P$. If we take the empirical distribution on the x, $\frac{1}{N}\sum_{n=1}^N \delta_{x_n}$, we get (Equations 8.37 and 8.40)

$$\begin{aligned} \tilde{F}(\theta) &= \frac{1}{N} \sum_{n=1}^N \mathbb{E}_{p(y|x_n,\theta)} \left[-\nabla_\theta \nabla_\theta^\top \log p(y \mid x_n, \theta) \right] \\ &= \frac{1}{N} \sum_{n=1}^N \nabla_\theta \log p(y_n \mid x_n, \theta) \left(\nabla_\theta \log p(y_n \mid x_n, \theta) \right)^\top . \end{aligned} \tag{9.9}$$

We saw in Chapter 8 that $\tilde{F}(\theta) = \frac{1}{N} G(\theta)$, a renormalisation of the GGN metric, under the mild condition that the Hessian of the negative log-likelihood loss does not depend on the target y (see Equation 8.42).

This gives an important connection between the covariance used in linearised approximation and the fundamental object from information geometry, which is the Fisher information metric. The goal of this chapter is to explain the importance of the Fisher information metric, why it can be seen as the natural metric for any probabilistic model, and its interpretation in deep learning.

A **statistical manifold** is a manifold $\mathcal{M}$ consisting of probability measures. Any parametric statistical model defines a parametrisation of a statistical manifold $\mathbb{R}^P \to \mathcal{M}$. What is $\mathcal{M}$ for a neural network? So far, we have viewed a neural network as a map $f\colon \mathbb{R}^D \times \mathbb{R}^P \to \mathbb{R}^C$. However, recall that we assume it induces probability distributions $p(y \mid x, \theta)$, since the loss function $\ell(f(x, \theta), y)$ is assumed to be the negative log-likelihood $-\log p(y \mid x, \theta)$. Therefore we implicitly viewed f as a probabilistic model; for each $x \in \mathbb{R}^D$, and $\theta \in \mathbb{R}^P$, we have a probability

measure $\mathrm{f}(x,\theta) \in \mathcal{P}(\mathcal{Y})$ defined by the neural network, where $\mathcal{Y}$ is the set of labels ($\mathbb{R}^C$ for regression, $\{1,\dots,C\}$ for classification) and $\mathcal{P}(\mathcal{Y})$ is the space of probability measures on the set of labels. f is given by the composition of f with some appropriate $\mathrm{g}\colon \mathbb{R}^C \to \mathcal{P}(\mathcal{Y})$. In regression, we most often take $\mathrm{g}(z) = \mathcal{N}(z, \epsilon^2 I) \in \mathcal{P}(\mathbb{R}^C)$ for some fixed $\epsilon > 0$, while in classification, most often $\mathrm{g}(z) = \mathrm{softmax}(z) \in \Delta^{C-1} \cong \mathcal{P}(\{1,\dots,C\})$. In Chapter 8, we saw that both of these choices guarantee the equivalence between the Fisher and the GGN (Equation 8.42).

So we have a map $\mathrm{f}\colon \mathbb{R}^D \times \mathbb{R}^P \to \mathcal{P}(\mathcal{Y})$. Write $p(y \mid x, \theta)$ for the density of $\mathrm{f}(x,\theta)$, so $p(y \mid x, \theta) = \frac{1}{\sqrt{2\pi}\epsilon} \exp\left(-\frac{1}{2\epsilon^2}\|f(x,\theta) - y\|^2\right)$ in regression, $p(y \mid x, \theta) = \mathrm{softmax}(f(x,\theta))_y$ in classification. Then the **statistical model** $\mathrm{m}\colon \mathbb{R}^P \to \mathcal{P}(\mathbb{R}^D \times \mathcal{Y})$ is defined as the joint distribution over x and y. Namely, if the distribution over the inputs x has a density $p(x)$, the density of $\mathrm{m}(\theta)$ is $p(x, y \mid \theta) := p(x)p(y \mid x, \theta)$. As discussed above, in practice we take the empirical distribution over x. The statistical manifold $\mathcal{M}$ defined by the neural network f is the image of m in $\mathcal{P}(\mathbb{R}^D \times \mathcal{Y})$.

This is where the Fisher information metric comes into play. The Fisher metric is an object *intrinsic* to the statistical manifold $\mathcal{M}$, and is independent of how the manifold is parametrised by $\mathbb{R}^P$. Formally, suppose we have a different parametrisation $\tilde{\mathrm{m}}\colon \mathbb{R}^P \to \mathcal{M}$. Then m and $\tilde{\mathrm{m}}$ differ by some diffeomorphism[2] of the parameter space, i.e. there is a diffeomorphism $\varphi\colon \mathbb{R}^P \to \mathbb{R}^P$ such that $\mathrm{m} = \tilde{\mathrm{m}} \circ \varphi$. Then we see that for $\tilde{\theta} = \varphi(\theta)$, $\mathrm{m}(\theta) = \tilde{\mathrm{m}}(\tilde{\theta})$ or, abusing notation, $p(x, y \mid \theta) = p(x, y \mid \tilde{\theta})$. Therefore, from the definition of the Fisher information metric (Equation 9.8 or Equation 9.9), the Fisher information metric on the two distinct parametrisations coincide $F_{\mathrm{m}}(\theta) = F_{\tilde{\mathrm{m}}}(\tilde{\theta})$.

The fact that the Fisher metric does not depend on the parametrisation implies that it is intrinsic to $\mathcal{M}$. Therefore, there should exist a way to define it directly on $\mathcal{M}$ in an abstract, measure-theoretic way. Doing this rigorously requires some mathematical groundwork, which the keen reader can find in Ay et al. [66]. Briefly, for $\mu \in \mathcal{M}$ and tangent vectors $\nu_1, \nu_2 \in T_\mu\mathcal{M}$, which are signed measures integrating to 0, the intrinsic Fisher information metric is defined by

$$F(\mu)[\nu_1, \nu_2] = \int \frac{\mathrm{d}\nu_1}{\mathrm{d}\mu} \frac{\mathrm{d}\nu_2}{\mathrm{d}\mu} \, \mathrm{d}\mu \tag{9.10}$$

where $\frac{\mathrm{d}\nu_i}{\mathrm{d}\mu}$ is the Radon-Nikodym derivative of ν_i with respect to μ, i.e. the densities of ν_i with respect to μ. Then the degenerate metric on the parameter space $\mathbb{R}^P$ is given by the pullback of this intrinsic metric onto $\mathbb{R}^P$ (see Ay et al. [66]).

There is an infinite number of ways to define an intrinsic Riemannian metric on $\mathcal{M}$. So what makes the Fisher information metric special? It is not special for its invariance under transformations of the *parameter space*, but instead for its invariance under transformations of the *data space*.

Definition 9.1. *Let $\mathcal{D}$ be a measurable space, thought of as a data space. A* statistic *is a measurable map $T\colon \mathcal{D} \to \mathcal{D}'$, where $\mathcal{D}'$ is some other measurable space.*

Let $\mathcal{M}$ be a statistical manifold over $\mathcal{D}$, that is, consisting of probability measures over $\mathcal{D}$. A statistic $T\colon \mathcal{D} \to \mathcal{D}'$ is called sufficient *if the measures in $\mathcal{M}$ have densities $q(T(x), \theta)h(x)$ with respect to some base measure, for $x \in \mathcal{D}$ and θ parametrising $\mathcal{M}$.*

The factorisation condition means that the likelihood depends on θ only through $T(x)$, since the factor $h(x)$ has no θ-dependence. Thus, all information about θ contained in the full data point x is also contained in the statistic $T(x)$, justifying the term 'sufficient'.

In our deep learning setting above, the data space is $\mathcal{D} = \mathbb{R}^D \times \mathcal{Y}$, the product of the input space with the label space.

Note that, for a statistical manifold $\mathcal{M}$ over $\mathcal{D}$, a statistic $T\colon \mathcal{D} \to \mathcal{D}'$ induces a map $T_*\colon \mathcal{M} \to \mathcal{M}'$ acting by pushforwards: $T_*\mu = \mu \circ T^{-1}$. So $\mathcal{M}'$ is a statistical manifold over $\mathcal{D}'$. Then, under mild assumptions on $\mathcal{M}$, we obtain the following theorem:

Theorem 9.1 (Proposition 5.6 in Ay et al. [66])**.** *Suppose $\mathcal{M}$ has a positive regular density function [66, Definition 3.5]. Then the Fisher information metric is invariant under sufficient statistics $T\colon \mathcal{D} \to \mathcal{D}'$, i.e. $T_*\colon \mathcal{M} \to \mathcal{M}'$ is an isometry of Riemannian manifolds, when $\mathcal{M}$ and $\mathcal{M}'$ are equipped with their respective Fisher information metrics.*

Up to a multiplicative constant, the Fisher information metric is the unique metric satisfying this. This is Chentsov's theorem. We leave as an exercise to verify this theorem for the special case of the deep learning setting above.

As explained above, sufficient statistics should be thought of as maps of the data space which do not induce a loss of information on the parameters with respect toa given statistical model. The Fisher information metric,

[2] A diffeomorphism is a smooth bijection with smooth inverse.

being invariant under such transformations, should be thought of as quantifying 'information', hence its name. Note in particular that it is invariant under diffeomorphisms of the data space (which are trivially sufficient statistics owing to their bijective nature), and so does not depend on how the data is 'presented'. This makes it the natural metric for statistical manifold $\mathcal{M}$.

Takeaways

Up to the rescaling $\tilde{F}(\theta) = \frac{1}{N}G(\theta)$, the GGN *is* the empirical Fisher information metric — the intrinsic metric on the statistical manifold $\mathcal{M}$ of distributions defined by the network. Being intrinsic, it is **independent of how $\mathcal{M}$ is parametrised**, and by *Chentsov's theorem* it is the unique metric (up to scale) invariant under sufficient statistics, i.e. under transformations of the *data* space that lose no information about the parameters. This singles out the Fisher as the natural, 'information-theoretic' metric underlying the GGN.

9.3 Geometric Laplace Approximations

Laplace approximations, being Gaussian approximations to the neural network's posterior, are limited in their expressivity. As a consequence, there have been attempts at leveraging the geometry of the parameter space to define more expressive variants of Laplace approximations.

A key observation is that the Laplace approximations are not diffeomorphism-invariant: given a diffeomorphism of the parameter space $\varphi\colon \mathbb{R}^P \to \mathbb{R}^P$, if such diffeomorphism is non-linear, then performing Laplace with respect to the parameters φ, i.e., a **reparametrised Laplace approximation**, yields a distinct posterior approximation than performing Laplace with respect to the parameters θ. In the extreme case, one can show there exists a diffeomorphism φ for which the Laplace approximation with respect to the corresponding parameters is exact [333]. In practice, computing such φ is as difficult as computing the exact posterior itself. However, this result gives hope that one may be able to construct computationally feasible diffeomorphisms φ yielding high-quality Laplace approximations, see for instance MacKay [1109]. An important note is that, while Laplace approximations are not invariant under diffeomorphisms of the parameter space, the predictives of linearised Laplace approximations are, up to changing the prior. We leave this as an exercise to the reader.

Another attempt at leveraging geometry for Laplace is to perform the Laplace approximation in the tangent space of the parameter space and then project it through an exponential map under some Riemannian metric [121, 1823], coined **Riemannian Laplace approximations**. The latter work uses the Fisher as the Riemannian metric. Work remains to be done to make the computation of the exponential map efficient and scalable. See Costa et al. [333] for the relationship between such Riemannian Laplace approximations and the aforementioned reparametrised Laplace approximations.

Finally, **diffusion Laplace approximations** [1427] sample the approximate posterior through random walks with respect to a Riemannian metric in the weight space. Once again, while there are promising preliminary results, further work is needed to scale such approaches computationally. Moreover, the relationship between diffusion Laplace approximations and Riemannian Laplace approximations is yet to be described.

Takeaways

Laplace approximations are **not invariant** under non-linear reparametrisations $\varphi\colon \mathbb{R}^P \to \mathbb{R}^P$ of the parameter space — in the extreme, some φ even makes Laplace exact. This freedom motivates *geometric* variants: **reparametrised**, **Riemannian** (Laplace in the tangent space pushed through an exponential map, often under the Fisher metric), and **diffusion** Laplace approximations. All exploit the parameter-space geometry for more expressive posteriors, but scalable computation — of suitable diffeomorphisms, exponential maps, and random walks — remains an open problem.

Chapter 10

Analytic, low-cost functionality from Laplace

In the previous sections, we have seen how the Laplace approximation endows a neural network with a Gaussian weight-space uncertainty (cf. Chapter 7), and how this uncertainty can be transformed into a closed-form expression for predictive uncertainty (cf. Chapter 8) informed by the training data (cf. Chapter 9). By combining the linearised pushforward with analytic approximations for the softmax integral in classification, we obtain a versatile analytic toolbox for uncertainty-driven decision making.

Classical Bayesian statistical methods – such as probabilistic prediction, out-of-distribution detection, active learning, Bayesian optimisation, continual learning, and model selection – become directly accessible in deep learning under the Gaussian assumption. This setting also enables further approximations for inference at various computational budgets [380, 1730].

In this chapter, we examine two case studies demonstrating how such uncertainty estimates can be leveraged to address open challenges in deep learning. The first focuses on how to mitigate predictive overconfidence in ReLU networks. The second addresses continual learning: we will see that the Laplace approximation provides a natural regularisation term, which alleviates catastrophic forgetting. We then conclude with additional observations on how the Laplace-Gaussian Process connection can be leveraged for optimal data selection under different objectives. For a discussion on computational techniques for scaling these approaches to large networks, such as KFAC and subnetwork approximations, we refer to Section 8.4 and Chapter 40.

Takeaways

Since linearised Laplace turns virtually every deep neural network into a Gaussian process predictive distribution, the powerful practical functionalities of Gaussian process regression – confidence and uncertainty quantification; model adaptation; active learning and design; continual learning; etc. – all become directly available in deep learning. Of course, since this is an approximate framework, these functionalities can only be expected to work to varying degrees of approximation.

10.1 Fixing overconfidence with calibrated uncertainty

10.1.1 Provable overconfidence in ReLU-like neural networks

In supervised classification, deep neural networks are trained to map an input $x \in \mathbb{R}^d$ to a vector of class logits $f_\theta(x) \in \mathbb{R}^C$, which are converted into categorical probabilities via the softmax function (8.56). Ideally, these probabilities should reflect the true uncertainty of the model's predictions. In practice, however, modern deep networks tend to be *overconfident*: they assign very high probabilities to certain classes even when the input is far from the training distribution. This behaviour is particularly problematic in safety-critical settings, such as medical diagnosis or autonomous driving, where unwarranted certainty can have serious consequences [729].

A precise theoretical explanation for this behaviour in ReLU-like networks was given by Hein et al. [729]. Such networks consist of linear layers (fully connected or convolutional) combined with piecewise-linear activations such

as ReLU or LeakyReLU. Consequently, they implement a piecewise affine mapping:

$$f(x) = \mathbf{V}^{(l)}x + \mathbf{a}^{(l)}, \quad x \in \mathcal{Q}_l, \quad l = 1, \ldots, R$$

where the polytopes $\mathcal{Q}_1, \ldots, \mathcal{Q}_R$ partition the input space $\mathbb{R}^d$ into regions of constant activation pattern. Within each such region, the logits grow linearly with $\|x\|$. As the input moves far away from the training data, the behaviour is dominated by a single affine component whose largest logit can grow without bound, while all other logits remain comparatively smaller. Since the softmax function magnifies differences between logits, this causes the predicted probability for a single class to approach one.

Theorem 10.1 (cf. Theorem 3.1, Hein et al. [729])**.** *Suppose that $\mathbf{V}^l$ does not contain identical rows for all $l = 1, \ldots, R$, then for almost any non-zero input x and any $\varepsilon > 0$, there exists a scaling factor $\alpha > 0$ such that for some class k,*

$$\text{softmax}_k(f(\alpha x)) \geq 1 - \varepsilon,$$

and

$$\lim_{\alpha \to \infty} \text{softmax}_k(f(\alpha x)) = 1.$$

In other words, deterministic ReLU networks extrapolate with *increasing certainty* as we move away from training data, regardless of the lack of evidence.

10.1.2 Bayesian perspective as a counter measure

The problem identified in the previous section is an artefact directly caused by the use of point estimates. The natural way to solve it is to assign a probability measure over the weights θ instead of committing to a single value. As we will see, this naturally leads to increasing variance away from the data, which tempers the confidence after the softmax is applied.

The Laplace approximation provides a tractable route to introduce this behaviour into neural networks. To recall, we approximate for a given trained network with parameters θ_* (the maximum a posteriori estimate), the posterior over parameters by a Gaussian:

$$p(\theta|\mathcal{D}) \approx \mathcal{N}(\theta_*, \Psi(\theta_*)^{-1}),$$

where Ψ describes the curvature of the negative log posterior evaluated at θ_*. With linearising the pushforward around θ_*, the logits become Gaussian as well (cf. Section 8.1):

$$z(x) \sim \mathcal{N}(\mu(x), k(x, x))$$

with

$$\mu(x) = f(x, \theta_*), \quad k(x, x) = J_\theta f(x, \theta_*)\Psi(\theta_*)^{-1} J_\theta f(x, \theta_*)^\top. \tag{10.1}$$

For ReLU networks, the components of $J_\theta f(x, \theta_*)$ grow at most linearly with $\|x\|$, so the logit variance $k(x, x)$ grows quadratically with $\|x\|$ (equivalently, the standard deviation grows linearly). This means that, far from the training set, the variance in each logit matches the mean's linear growth. We will now see that this leads to finite asymptotic confidence in each class, bounded away from 1 and 0.

The predictive class probabilities are obtained by marginalising over the Gaussian posterior

$$p(y = k|x, \mathcal{D}) = \mathbb{E}_{z \sim \mathcal{N}(\mu(x), k(x,x))} \left[\text{softmax}_k(z)\right] .$$

While exact evaluation is infeasible, accurate analytic approximations exist (cf. Section 8.5). A well-known approximation in the binary case is due to MacKay [1106]:

$$\mathbb{E}[\sigma(z)] \approx \sigma\left(\frac{\mu}{\sqrt{1 + \frac{\pi}{8}s^2}}\right),$$

where $z \sim \mathcal{N}(\mu, s^2)$ is the Gaussian logit with mean μ and variance $s^2 = k(x, x)$, and $\sigma(t) = 1/(1 + e^{-t})$ is the logistic sigmoid.

This solution was established by Kristiadi et al. [928] in their paper *Being Bayesian, Even Just A Bit, Fixes Overconfidence In ReLU Networks*, where *Even Just A Bit* refers to the observation that the following theorem holds even when only the weights of the last layer are treated probabilistically, and *even* if the covariance $\Psi(\theta_*)^{-1}$ in the Laplace approximation is chosen in an *arbitrary* way (say, equal to the identity $\mathbf{I}$).

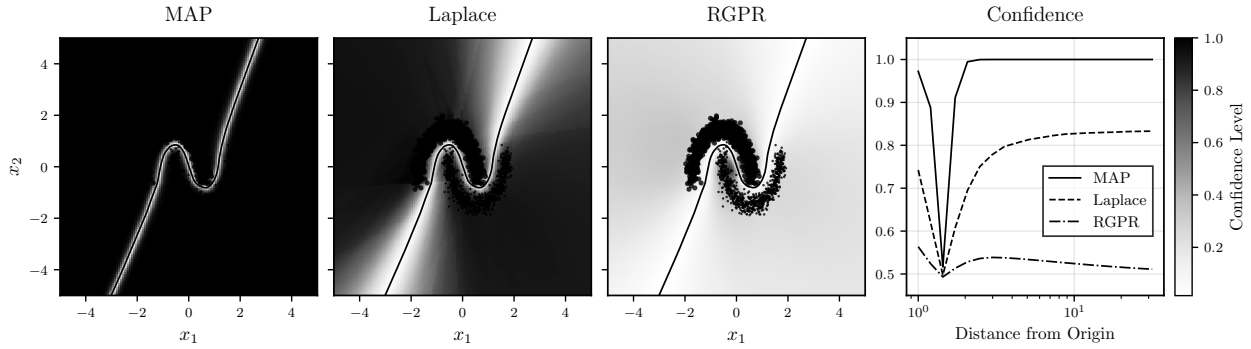


Figure 10.1: **Mitigating overconfidence in a ReLU network with the Laplace approximation.** The figure shows the predictive confidence on a two-moons binary classification task for three different models. Darker background shading indicates higher confidence, while the solid black line marks the decision boundary at $p = 0.5$. **(Left)** The standard MAP-trained ReLU network exhibits severe overconfidence. As described in Theorem 10.1, its predictive confidence incorrectly approaches 1 (maximum certainty) for inputs far away from the training data. **(Middle)** Applying the Laplace approximation introduces uncertainty over the network's weights. The resulting logit variance grows quadratically with distance from the data, which tempers the softmax and reduces predictive confidence in out-of-distribution regions. **(Right)** Adding a ReLU-GP residual term (RGPR) to the Laplace-approximated logits introduces an additional source of variance that grows cubically. This ensures that the predictive confidence correctly converges to the uniform distribution (0.5) for inputs far from the training data, providing better calibration. **(Far Right)** This panel quantitatively confirms the visual results by plotting the maximum confidence of each model as an input point is scaled away from the origin.

Theorem 10.2 (cf. Proposition 2.5, Kristiadi et al. [928])**.** *Consider a binary classification ReLU network f_θ with Laplace posterior $\theta \sim \mathcal{N}(\theta_*, \Psi(\theta_*)^{-1})$. Then, the predictive confidence at input x decreases monotonically as prior variance σ_0^2 increases, satisfying*

$$\lim_{\sigma_0^2 \to \infty} \sigma(|z(x)|) \leq \sigma\left(\frac{|\mu(x)|}{1 + \sqrt{\frac{\pi}{8}\lambda_{max}(H)\,\|J_\theta f(x, \theta_*)\|^2}}\right), \quad \lim_{\sigma_0^2 \to 0} \sigma(|z(x)|) = \sigma(|\mu(x)|).$$

Here, $\lambda_{\max}(H)$ denotes the largest eigenvalue of the negative log-likelihood Hessian evaluated at the posterior mean θ_*, while $\mu(x) = f(x, \theta_*)$ and $J_\theta f(x, \theta_*)$ are the mean logit and network Jacobian at θ_*, exactly as in Equation (10.1). Note that H differs from the posterior precision $\Psi(\theta_*)$ only by the Gaussian-prior term, $\Psi(\theta_*) = H + \sigma_0^{-2}\mathbf{I}$, and this prior contribution vanishes as $\sigma_0^2 \to \infty$, so only the likelihood Hessian survives in the bound.

Of course, this theorem provides only an asymptotic statement. In words: If we assign *any* Gaussian measure to the weights of the network (whether explicitly constructed as a Laplace approximation or simply imposed) then asymptotic overconfidence is "healed" in the weak sense that it is bounded away from 1 and 0. Despite its weak form, this result is a great argument for balancing the cost of Laplace approximations with their goals. If all you want is to fix asymptotic confidence, then you can do so at virtually zero computational cost: Just impose an arbitrary confidence on the parameters. This is still better than using a point estimate instead.

10.1.3 The role of the prior variance

Theorem 10.2 highlights the special role of the prior variance σ_0^2: increasing σ_0^2 broadens the posterior over weights, which in turn accelerates variance growth in the logits and reduces predictive confidence away from the data. On the other hand, letting $\sigma_0^2 \to 0$ recovers the deterministic prediction. Although σ_0^2 is traditionally motivated by the weight decay used during training, in the Laplace setting it can be tuned *post hoc* to improve the calibration.

Hyperparameters such as the prior variance or additional factors entering the curvature can be tuned with techniques often motivated by the literature on Gaussian processes [1382]. This includes downstream objectives (e.g., negative log-likelihood, out-of-distribution entropy) or model fit criteria (e.g. log marginal likelihood). Here, the Gaussian structure of the Laplace approximation allows for closed-form objectives suitable for gradient-based tuning, which is discussed in more detail in addition to other applications in the next chapter (cf. Chapter 11).

For the particular setting of calibrating σ_0^2 such that the resulting uncertainty structure reflects out-of-

distribution well, Kristiadi et al. [928] suggest using

$$\text{argmin}_{\sigma_0^2} -\frac{1}{n}\sum_{i=1}^{n} \log p(y = \hat{t}_i \mid \hat{x}_i, \mathcal{D}) - \gamma\, \mathrm{H}[p(y \mid \tilde{x}_i, \mathcal{D})], \tag{10.2}$$

where the first term is the standard cross-entropy loss over an in-distribution validation set $\{\hat{x}_i, \hat{t}_i\}$, while the second is the predictive entropy over a set of out-of-distribution inputs $\{\tilde{x}_i\}$, which the objective encourages to be large through the leading minus sign. The parameter $\gamma \in [0, 1]$ controls the trade-off between these two terms.

10.1.4 How to mitigate the problem even further

A simplified summary of Theorem 10.2 is that assigning a Gaussian measure to weights bounds asymptotic confidence away from the pathological values 1 and 0. But it will not drive asymptotic confidence down to the ideal uniform (maximum entropy) limit $1/C$ where C is the number of classes. And of course the Theorem says nothing about confidence "close to" or "in-between" the training data. To address either problem, Kristiadi et al. [929, 930] consider two extensions to tune the overall confidence structure: one reshapes the local curvature for better calibration close to the data [930], the other combines the weight-space uncertainty with some additional output-space uncertainty to ensure the ideal uniform confidence limit [929].

The approach *Learnable Uncertainty under Laplace approximations* (LULA) [930] augments a trained network with additional parameters – *uncertainty units* – inserted into the architecture in a way that preserves the MAP prediction $f_{\theta_*}(x)$ but modifies the Hessian H, e.g. for three linear layers without nonlinearities and biases for simplicity, we have,

$$f(x) = \left[W_{\text{MAP}}^{(3)}\right]\left[W_{\text{MAP}}^{(2)}\right]\left[W_{\text{MAP}}^{(1)}\right] x \tag{10.3}$$

$$\Rightarrow f^{(\text{LULA})}(x) = \begin{bmatrix} W_{\text{MAP}}^{(3)} & 0 \end{bmatrix} \begin{bmatrix} W_{\text{MAP}}^{(2)} & 0 \\ \hat{W}_1^{(2)} & \hat{W}_2^{(2)} \end{bmatrix} \begin{bmatrix} W_{\text{MAP}}^{(1)} \\ \hat{W}_1^{(1)} \end{bmatrix} x\,. \tag{10.4}$$

Therefore, the final variance changes while the mean prediction stays the same, which motivates the name *uncertainty units*. These units are then trained using an uncertainty-aware loss that encourages high predictive entropy on out-of-distribution (OOD) inputs and low entropy on in-distribution (ID) inputs, with gradients masked so that the original prediction remains unchanged. It can then be shown that the resulting Laplace approximation with diagonal curvature approximation has higher variance over the output compared to the diagonal Laplace for the network without the uncertainty units (cf. Proposition 2, Kristiadi et al. [930]).

However, residual overconfidence can still be left for inputs extremely far from the training data. This is a consequence of the fact that ReLU networks have only finitely many effective features, whereas a Gaussian process (GP) with an infinite set of ReLU features exhibits a growth of cubic variance, forcing the softmax confidence toward the uniform distribution as $\|x\| \to \infty$. The *ReLU-GP residual* (RGPR) approach [929] adds such a GP as an independent residual term $\hat{f} \sim \mathcal{GP}(0, \hat{k})$ to the Bayesian neural network:

$$\tilde{f}(x) = f(x) + \hat{f}(x)$$

with a specific double-sided cubic spline kernel $\hat{k}$ chosen so that $\hat{k}(x, x) \approx 0$ near the origin but grows cubically with $\|x\|$. The resulting logit distribution is

$$z(x) \sim \mathcal{N}\big(\mu(x), k(x, x) + \hat{k}(x, x)\,\mathbf{I}\big)$$

preserving the mean while adding variance that dominates in the tails. Applying a softmax approximation to this Gaussian results in a predictive distribution that converges to the uniform distribution in the limit (cf. Theorem 4, Kristiadi et al. [929]).

Importantly, this Gaussian process term can be added in such a way that its evaluation has constant and negligible cost, independent of the training data. This is because it is explicitly constructed to not be affected by data within the training region, only contributing prior uncertainty in the OOD regime. It is thus another example of a Laplace approximation adding specific functionality, at extremely minimal cost overhead. The double-sided cubic spline kernel chosen in this work also has a pleasing interpretation as a limit process arising from an infinite number of the ReLU uncertainty units from above, all distributed in a regular fashion around the training domain, pointing "outwards" so that they are not affected by the training data. The implied intuition is that these units "complete" or complement the finitely many ReLUs that make up the deep network that we actually train on the

available (finite) data, and provide a nonparametric sea of ReLUs that cover the domain towards the horizon. To speak even more metaphorically, this is arguably an example of the Bayesian perspective on deep learning providing an intuition – and a concrete, practical solution! – to a pathology of point-estimated deep models. Figure 10.1 compares the predictive confidence of the MAP network, its Laplace approximation, and the RGPR extension on a two-moons task, illustrating how each successive step tempers overconfidence away from the data.

Takeaways

Asymptotic and OOD overconfidence are examples of pathologies arising from the use of point estimates in deep learning. The framework of Laplace approximations (or more generally, assigning a Gaussian measure to the weights and using linearisation to push forward to the output domain) can fix these issues to some tangible, quantifiable degree, while adding, occasionally negligible, or at least very manageable computational overhead. This is arguably because these problems really just arise discretely from a non-Bayesian treatment, and vanish as soon as one assigns *a* probability measure, *any* kind of posterior on the parameters.

10.2 Fixing catastrophic forgetting in continual learning

In *continual learning*, a single neural network is trained on a sequence of tasks

$$t = 1, \dots, T, \qquad \text{with datasets } \mathcal{D}_t, \tag{10.5}$$

with the goal of maintaining good performance on earlier tasks. This setting is increasingly relevant in real-world deployments: modern large-scale models are trained once, but are then confronted with a stream of evolving data and objectives. A key obstacle to adaptation is *catastrophic forgetting*, the loss of previously acquired abilities when learning new tasks.

A naïve approach is to fine-tune the current parameters θ_t by minimising the loss on the next dataset D_{t+1}:

$$\theta^*_{t+1} = \operatorname{argmin}_\theta \ \mathcal{L}(\theta; \mathcal{D}_{t+1}). \tag{10.6}$$

However, this discards all information about earlier tasks: parameters drift toward the new optimum for adapting to the new data $\mathcal{D}_{t+1}$, and end up erasing information by neglecting critical weights for previous performance.

10.2.1 A Bayesian solution

From a Bayesian perspective, training on $\mathcal{D}_{t+1}$ should not start from an uninformative prior. Instead, the posterior after task t should become the prior for task $t+1$:

$$p(\theta_{t+1} \mid D_{1:t+1}) \propto p(\mathcal{D}_{t+1} \mid \theta_{t+1})\, p(\theta_t \mid \mathcal{D}_{1:t}), \tag{10.7}$$

instead of the original uninformative isotropic Gaussian $p(\theta)$ in form of weight decay. Taking the negative log turns this into a regularised optimisation problem, where the regulariser encodes knowledge from earlier tasks.

If $p(\theta_t \mid D_{1:t})$ is approximated by a Gaussian $\mathcal{N}(\theta^*_t, \Psi_t^{-1})$ via the Laplace approximation [1522], the next-task objective becomes:

$$\mathcal{L}^{\text{reg}}_{t+1}(\theta) = \mathcal{L}(\theta; \mathcal{D}_{t+1}) + \tfrac{\lambda}{2} \|\theta - \theta^*_t\|^2_{\Psi_t}, \tag{10.8}$$

where $\|v\|^2_{\Psi_t} := v^\top \Psi_t\, v$ denotes the squared Mahalanobis norm induced by the precision Ψ_t. The precision matrix Ψ_t determines how strongly each direction in parameter space is protected: large precision (high curvature) means a stiff penalty in that direction, since the loss on the previous task would change rapidly, while low precision allows for adaptation. This yields an *anisotropic* weight-space prior that selectively resists change where it would harm past performance. The regularisation strength λ controls the trade-off between retaining performance on past tasks and adapting to new ones by scaling the precision of the prior: higher values express greater confidence in past parameters, lower values signal greater uncertainty and openness to change.

10.2.2 Bayesian filtering in weight space

Starting from the previously motivated interpretation and following Sliwa et al. [1522], we can formalise continual learning as Bayesian state estimation in weight space:

$$\theta_0 \to \theta_1 \to \cdots \to \theta_T \tag{10.9}$$

with a Gaussian *state-transition* model and a likelihood defined by the task loss

$$p(\theta_{t+1} \mid \theta_t) = \mathcal{N}(\theta_t, Q) \qquad \text{(process model)} \tag{10.10}$$

$$p(D_t \mid \theta_t) \propto \exp\big(-\frac{1}{\tau}\mathcal{L}(\theta_t; D_t)\big), \qquad \text{(observation model).} \tag{10.11}$$

Continual learning becomes in this interpretation a problem of tracking a moving target in weight space: the process model predicts where the target might go next, and the observation model corrects this prediction based on new evidence. The additional process noise covariance Q models the expected drift between tasks: large Q corresponds to high *plasticity* (parameters may change substantially), while small Q enforces *stability* (parameters remain close).

The Bayesian filtering recursion consists of:

1. *Predict step*: propagate the posterior through the process model to obtain a prior for θ_t, (potentially) inflating the uncertainty by Q.
2. *Update step*: incorporate the likelihood from $\mathcal{D}_t$, pulling the mean toward the new optimum and shrinking uncertainty according to curvature.

This is analogous to a Kalman filter in parameter space, but with a nonlinear, non-Gaussian observation model arising from the neural network loss. The Laplace approximation replaces this intractable term with a local Gaussian, making the update step analytically tractable within the filtering framework.

10.2.3 From EWC to OSLA to Lo-Fi

The state-space formulation provides a unifying Bayesian view of continual learning. Different methods in the literature can be seen as specific choices for (i) how the posterior covariance is represented and updated, and (ii) whether parameter drift between tasks is allowed. The Laplace approximation supplies the Gaussian form needed for tractable updates, but the quality and scalability of the approach depend on the curvature approximation used.

Elastic Weight Consolidation (EWC). Kirkpatrick et al. [897] proposed *elastic weight consolidation* (EWC), which constrains parameters to remain close to the previous optimum θ_t^* using the diagonal of the Fisher information metric F_t:

$$\mathcal{L} = \mathcal{L}(\theta; \mathcal{D}_{t+1}) + \tfrac{\lambda}{2}(\theta - \theta_t^*)^\top \operatorname{diag}(F_t)(\theta - \theta_t^*). \tag{10.12}$$

This is equivalent to the proposed filtering framework when choosing $Q = 0$ and a diagonal precision. In the sequential setting, a separate quadratic penalty can be added for each of the previous tasks (multi-penalty), or equivalently, the Fisher matrices can be accumulated into a single continuously updated penalty. EWC is computationally efficient but lacks correlation between parameters.

Online Structured Laplace Approximation (OSLA). Ritter et al. [1401] extended EWC by replacing the diagonal Fisher with a *Kronecker-factored* (KFAC) curvature approximation (cf. Section 8.4), capturing intra-layer parameter correlations while remaining scalable. In the Bayesian filtering view, OSLA maintains a single Gaussian posterior over parameters and updates it *recursively* after each task:

$$\Psi_t = \lambda\,\mathrm{KFAC}(F_t) + \Psi_{t-1}, \tag{10.13}$$

where F_t is again the Fisher information matrix computed with the current MAP estimate on task t. The running update replaces the need to store separate penalties for each task, as in multi-penalty EWC, and naturally implements a sequential Bayesian update with $Q = 0$ (no assumed drift). By incorporating inner-layer correlations, OSLA produces a more faithful posterior approximation and better retention over long sequences of tasks.

Low-Rank Laplace-Gaussian Filtering (Lo-Fi). Chang et al. [259] and Sliwa et al. [1522] generalised the Bayesian framework to arbitrary Q, enabling explicit modelling of expected drifts between tasks. They approximate the precision matrix using a low-rank approximation of the GGN (plus diagonal representing the initial prior), which makes both, prediction and update step, scalable while still modelling correlation between different layers.

In this general Bayesian filtering view, the concept of *smoothing* also becomes available. After the forward filtering pass, an optional backward pass refines each task-specific parameter estimate using information from

future tasks without revisiting their original data. The process can be compared to passing a message along a chain of messengers, where each messenger (task) has access only to the message (parameters) they just received and their own local observations; over time, the message can degrade (catastrophic forgetting). Smoothing allows later messengers to send clarifications or corrections back upstream, improving earlier versions without re-reading the original letters (data). In a deep learning context, this means earlier task models can be improved based on knowledge from later tasks, while preserving data privacy by never sharing or replaying the original datasets from other tasks.

Takeaways

The Laplace approximation arising from pre-training a network can be used as a log prior (a regulariser) on subsequent tasks. Doing so encourages the model to use "unused" parts of the parameter space to "store" new training information, and thus mitigates forgetting during continual learning. The underlying algebraic idea is that Bayes' theorem for Gaussians is a closure: Conditioning a Gaussian prior (an ℓ_2 regulariser) on a (linear) Gaussian likelihood (a Laplace approximated training loss) yields a Gaussian posterior (a Laplace approximated trained network). Hence, updating a Gaussian *posterior* with new data is conceptually the same thing as updating a Gaussian *prior*. The only real challenge is to manage the resulting data-structures well, to keep computational cost manageable. But since Laplace maps this process onto Gaussians, these computational aspects are all mapped to numerical linear algebra, for which very powerful, well-studied algorithms are readily available.

10.3 Data-centric objectives

The Laplace approximation provides a tractable way to understand the relationship between a model and its training data, which, in turn, improves properties such as predictive calibration (cf. Section 10.1) and the ability to learn sequentially (cf. Section 10.2). The same model-data relationship is central to the fields studying optimal data selection. Rather than relying on random sampling, methods have been developed to strategically choose the most informative data to label (active learning) and the most efficient points to evaluate (Bayesian optimisation). A fundamentally Bayesian approach underpins these strategies, leveraging a model's current beliefs to guide data acquisition. Also more nuanced data-centric questions exist, such as determining the most effective training order (curriculum learning), that have been studied from a Bayesian perspective with the help of the Laplace approximation [410]. However, here, we focus on active learning and Bayesian optimisation due to their foundational connection to Bayesian principles.

10.3.1 Active learning: selecting what to label next

The goal of **active learning** is to train an optimal model with a minimal data collection effort. For instance, training neural solvers for partial differential equations often requires a vast amount of training data generated by costly numerical simulations. Instead of random generation, active learning aims to strategically select the most informative data points to label, thereby achieving the same model accuracy with significantly less numerical simulations [1216].

From a Bayesian perspective, this selection process can be formalised as an optimisation problem guided by information theory. MacKay [1107] proposed framing active learning as a sequential design task where the goal is to choose the next data point that is expected to provide the maximal information gain about the model's parameters θ. This gain is measured by the expected reduction in the entropy of the parameter posterior after observing a new data point (x_i, y_i):

$$\mathcal{U}(x_i) = \mathrm{H}[p(\theta \mid \mathcal{D})] - \mathbb{E}_{y\sim p(y|x_i,\mathcal{D})}[\mathrm{H}[p(\theta \mid \mathcal{D}, x_i, y)]] \tag{10.14}$$

where $\mathrm{H}[p] = -\int p(u)\log p(u)\,\mathrm{d}u$ denotes the (differential) entropy of a distribution p; for a discrete distribution, such as the class-label predictive $p(y \mid \cdot)$, the integral is replaced by the sum $-\sum_u p(u)\log p(u)$.

In general, computing this expected information gain is intractable, though the linearised Laplace approximation provides again a solution. By approximating the posterior $p(\theta \mid \mathcal{D})$ with a Gaussian, MacKay [1107] showed that the expected information gain from a potential new data point at input x is a monotonic function of the model's current predictive variance at that location $s^2(x) = J_\theta f(x, \theta_*)\Psi(\theta_*)^{-1}J_\theta f(x, \theta_*)^\top$. This provides a principled justification for the intuitive and widely used uncertainty sampling heuristic: *to learn the most, one should query the points where the model is currently most uncertain* [1107].

The objective of maximising parameter information is now widely known as *Bayesian Active Learning by Disagreement* (BALD) [784]. The BALD score is a mathematically equivalent reformulation of the original objective:

$$\mathrm{BALD}(x) = \mathrm{H}[p(y \mid x, \mathcal{D})] - \mathbb{E}_{p(\theta \mid \mathcal{D})}\, \mathrm{H}[p(y \mid x, \theta)]\,. \tag{10.15}$$

This version provides the powerful intuition that one should query points where the model is overall uncertain about its prediction (the first term), but where individual parameters θ from the posterior are confident in their potentially disagreeing predictions (the second term is low).

It has been observed, that BALD's focus on parameter uncertainty can be misaligned with the goal of predictive performance, as it lacks an explicit notion of the relevant input data distribution. This can lead to the selection of obscure or irrelevant data points that offer little benefit to typical test input. To address these shortcomings, the *Expected Predictive Information Gain* (EPIG) [1180] was proposed, which measures the expected information a candidate label y at input x provides about the label y_* of a random target input x_* drawn from a target distribution $p_*(x_*)$. This objective is formalised as the expected reduction in predictive entropy on a random target point, averaged over all possible outcomes of the candidate query:

$$\mathrm{EPIG}(x) = \mathbb{E}_{p_*(x_*)}[\mathrm{H}[p(y_* \mid x_*)] - \mathrm{H}[p(y_* \mid x_*, x, y)]]\,. \tag{10.16}$$

Both the BALD and EPIG objectives are agnostic to the method used for posterior approximation. By shifting the entropy computation to the lower-dimensional prediction space (BALD) or considering only the predictive entropy (EPIG), this information-theoretic approach lessens the reliance on a specific form of weight-space uncertainty, opening the door for various uncertainty quantification tools. The Laplace approximation remains a fast and tractable method for obtaining the predictive uncertainty needed to compute either score. Ultimately, however, the performance of these acquisition functions is strongly coupled to the quality of the resulting uncertainty estimates.

10.3.2 Bayesian optimisation: finding the optimal point

Many problems can be framed as finding the global optimum of an objective function that is very expensive to evaluate. Bayesian optimisation (BO) is a sequential strategy for finding the global optimum of such expensive black-box functions and works by combining a probabilistic surrogate model of the objective with acquisition functions to select the next point for evaluation [584]. Gaussian Processes (GPs) are the standard surrogate model because they are inherently Bayesian, providing a posterior distribution over the function and allowing for many popular acquisition functions to be computed analytically, making the entire process efficient and principled.

The linearised Laplace approximation (LLA) provides a way of using neural networks as surrogates by interpreting them as a Gaussian Process, which allows for reusing existing analytic acquisition functions, such as maximising the *upper confidence bound* given by

$$\alpha_{\mathrm{UCB}}(x) = f(x, \theta_*) + \beta\, s(x) \tag{10.17}$$

where $f(x, \theta_*)$ is the network's mean prediction at the MAP estimate, $s(x)$ is the predictive standard deviation from the LLA posterior and β a hyperparameter balancing exploration and exploitation [932].

The final optimisation process is then iterative. At each step, a neural network is trained on the existing data set and queried using the LLA and an acquisition function for the next data point to evaluate the original black-box function. Then, this new information must be incorporated by retraining or adapting the neural network on the augmented dataset to find a new maximum-a-posteriori (MAP) estimate and then re-computing the Laplace approximation around this new point. This refines the model's beliefs and its weight-space uncertainty, guiding the next decision in the sequence.

Kristiadi et al. [932] demonstrated that the LLA surrogate inherits the strong predictive power and inductive biases of its underlying neural network, allowing it to outperform standard GPs on complex, high-dimensional tasks like searching through image spaces. However, they also note that broken predictive behaviour in extrapolation regions can significantly derail the optimisation process. This becomes clear when considering the Upper Confidence Bound as an example. For a ReLU network, the predictive mean outside the data region becomes a non-constant linear function. This means the predicted value will increase or decrease indefinitely as you move away from the data, while the uncertainty term also tends to grow linearly in these extrapolation regions, following the discussion in Section 10.1.2. Since both terms grow linearly, and the UCB score uses their sum, this utility also increases without bound in the same direction, leading the optimiser to choose a point infinitely far away in an unbounded search space. Note that this behaviour is consistent within the context of UCB: A region of extremely large expected reward *and* extremely large uncertainty is a very appealing target for another experiment. So the

problem here is arguably not the Bayesian treatment, but a badly chosen model class. Namely one that quite literally believes in pots of gold at the end of the rainbow.

Takeaways

Laplace approximations endow deep networks with probabilistic *uncertainty*, at tractable computational cost. A probability measure in itself, though, is just an intermediate mathematical notion. Its principal philosophical purpose, perhaps, is to allow meaningful *decisions*. For example, the Laplace posterior can be used to decide where to collect more training data in order to most efficiently reduce uncertainty and thus improve the model. Or, less directly, to find points where the target function (or a function of it) takes extremal values, provides most return. Interestingly, because these decisions have to be taken within a fixed domain, the Bayesian posterior used for this purpose does not have to be "correct" in an absolute sense, only a relative one. This highlights that when picking an approximate inference scheme for Bayesian deep learning, one should consider the downstream use case.

Chapter 11

Model selection with the marginal likelihood

The main intractability of high-dimensional Bayesian inference, as mentioned earlier in the book, stems from the difficulty of computing the normalising constant of the posterior distribution—the *marginal likelihood* (or *evidence*). While the MCMC methods presented before circumvent this problem by allowing us to sample from an *unnormalised* posterior distribution, the Laplace approximation offers the benefit that we can compute this quantity in closed form due to the Gaussianity. This not only enables inference and predictions (as shown in the previous chapters), but has one additional important use case: *model selection*. In this chapter, we will motivate why the marginal likelihood is a useful model selection criterion, explain how it is computed with the Laplace approximation, and present several practical use cases for model selection in Bayesian deep learning.

11.1 Marginal likelihood for model selection

There are several perspectives to understand why the marginal likelihood can be a useful criterion for model selection. We will review some different ones in this section to hopefully convince you that this pesky normalisation constant is not only a nuisance but a powerful tool when wielded in the right way.

11.1.1 Hierarchical Bayes

Let us look at the familiar Bayes' rule, but with an explicit conditioning on our model of choice, which we will denote by $\mathcal{M}$. This model can encompass both the prior and the likelihood, so in the case of Bayesian deep learning, for instance, the neural network architecture, the likelihood noise level, the prior variance, or its distributional form. In short, the model $\mathcal{M}$ contains everything we have to choose actively before we can start to perform inference. The model-conditioned Bayes' rule looks like this:

$$p(\theta \mid \mathcal{D}, \mathcal{M}) = \frac{p(\theta \mid \mathcal{M})\, p(\mathcal{D} \mid \theta, \mathcal{M})}{p(\mathcal{D} \mid \mathcal{M})} \tag{11.1}$$

If we assume now that we do not actually know how to select the model in advance and would like to be Bayesian about it, we could attempt to treat $\mathcal{M}$ as a random variable in a hierarchical Bayesian model, the posterior of which would take the form

$$p(\mathcal{M} \mid \mathcal{D}) = \frac{p(\mathcal{M})\, p(\mathcal{D} \mid \mathcal{M})}{\int p(\mathcal{M})\, p(\mathcal{D} \mid \mathcal{M})\, \mathrm{d}\mathcal{M}} \tag{11.2}$$

If we further assume that *a priori*, we do not have any preference for any particular model, that is, $p(\mathcal{M})$ is uniform across models (brushing under the rug the fact that this may be an improper prior if the space of models is not integrable), this yields

$$p(\mathcal{M} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathcal{M}) \tag{11.3}$$

We now see that the hierarchical posterior over models is proportional to the marginal likelihood term in Equation 11.1. Therefore, if we want to select the model that maximises the posterior (i.e., a MAP estimate over models), we just need to optimise $\mathcal{M}$ with respect to the marginal likelihood! Since this is essentially just performing a maximum likelihood selection on the second level of this Bayesian hierarchy, it is also often called *maximum likelihood II (ML-II)* model selection.

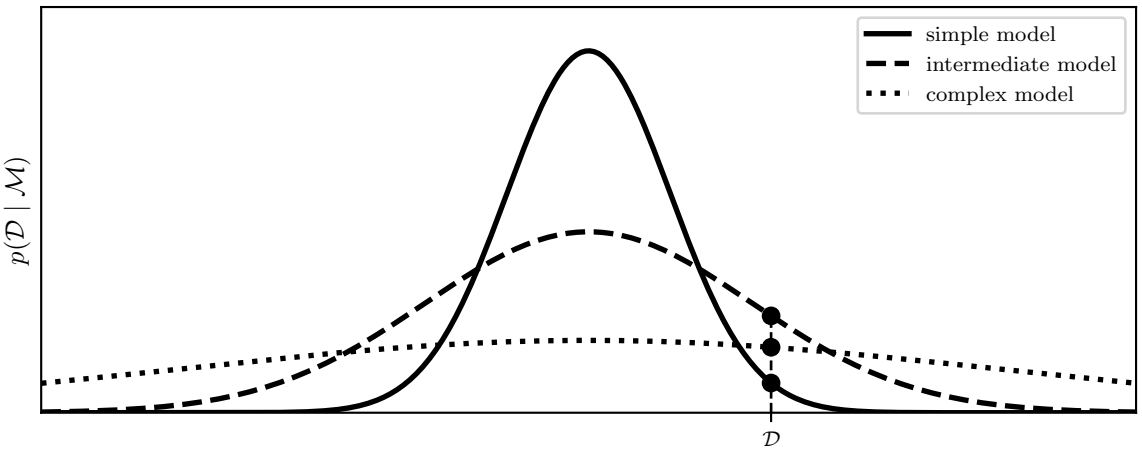


Figure 11.1: A visualisation of the Occam's razor effect of the marginal likelihood. The simplest model can assign the largest likelihoods, but just to a small number of datasets. For the most complex model, it is the other way around. For a given dataset $\mathcal{D}$, the marginal likelihood selects an intermediate model that is just complex enough to model these data.

11.1.2 Occam's razor

Apart from the interpretation as maximum likelihood in a hierarchical Bayesian model, we can also motivate from first principles why the marginal likelihood as a model selection criterion leads to an automatic *Occam's razor* effect. The principle of Occam's razor states that among a set of hypotheses (or models) that explain the same data equally well, one should always prefer the simplest one [1531]. The connection to the marginal likelihood has already been made in David MacKay's popular textbook [1111], based on his thesis [1112], and has been further developed by Rasmussen and Ghahramani [1380].

The argument is relatively simple: since the term $p(\mathcal{D} \mid \mathcal{M})$ has to be a normalised distribution over all possible datasets $\mathcal{D}$ for any given model $\mathcal{M}$, this means that models can either assign a lot of likelihood to a small number of datasets or a small amount of likelihood to many datasets. The former would intuitively be a *simple* model, such as a linear one, whereas the latter would be a *complex* model, such as a large neural network. For a dataset that can be modelled well by a linear function, the linear model will therefore be able to assign a larger likelihood than the neural network, because its likelihood is not distributed over all the non-linear datasets. Generally, this means that the model with the largest marginal likelihood for a given dataset will typically be a model that is just complex enough to model this dataset, but not more complex, thus implementing Occam's razor. This intuition is visualised in Figure 11.1.

11.1.3 Cross-validation

A popular non-Bayesian model selection criterion is cross-validation. The idea is that you split your dataset into $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$, with disjoint training and validation sets $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{val}}$. We can then estimate the generalisation risk (i.e., the expected loss on unseen data) by training a model on $\mathcal{D}_{\text{train}}$ and then evaluating its error on $\mathcal{D}_{\text{val}}$. This fits the intuition that we are ultimately interested in applying our model to unseen test data $\mathcal{D}_{\text{test}}$, so we want to know how well the model predicts on datapoints it has not seen during training.

In their seminal paper, Fong and Holmes [537] show that the log marginal likelihood on $\mathcal{D}$ is equal to the average validation log likelihood over all possible cross-validation splits, that is,

$$\log p(\mathcal{D}) = \mathbb{E}_{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}} \sim \text{split}(\mathcal{D})} \left[\log \mathbb{E}_{\theta \sim p(\theta \mid \mathcal{D}_{\text{train}})} [p(\mathcal{D}_{\text{val}} \mid \theta)] \right] \tag{11.4}$$

where $\text{split}(\mathcal{D})$ is a shorthand for writing a mechanism that first uniformly samples a dataset size split $m < N$ and then samples a uniformly random permutation of assigning $N - m$ points to $\mathcal{D}_{\text{train}}$ and m points to $\mathcal{D}_{\text{val}}$. Note that for notational convenience, we have suppressed the explicit conditioning on $\mathcal{M}$, but you can indeed read all the $p(\cdot)$ here as implicitly being $p(\cdot \mid \mathcal{M})$.

Obviously, actually performing this exhaustive cross-validation procedure would require a computational cost scaling combinatorially with the dataset size N, so nobody would actually compute this in practice. However, we see that with the marginal likelihood, we get the result of this exhaustive procedure by solving just a single integral

over the training dataset. This is another argument for the marginal likelihood as a model selection criterion: if you believe that cross-validation is a good way to evaluate the potential generalisation capabilities of your model, the marginal likelihood will not only compute one cross-validation for you, but *all* of them at the same time!

11.1.4 PAC-Bayes

Another way to study the generalisation behaviour of Bayesian predictors is to use the framework of PAC-Bayes bounds [1154, 1507]. While we will get a more thorough introduction to PAC-Bayes bounds in later chapters (specifically in Chapter 22), for now it suffices to say that they are mathematical statements that bound the expected true risk of our (approximate) posterior—which we cannot know because we do not have access to the true data-generating process—with the expected empirical risk that we can actually measure. These bounds generically look something like this:

Proposition 11.1 (Generic PAC-Bayes bound [24])**.** *For any (approximate) posterior $q(\theta)$, choice of prior $p(\theta)$, and choice of $\delta \in (0, 1)$, the expected true risk is bounded with probability at least $1 - \delta$ over sampling of datasets $\mathcal{D}$ of size N from the data-generating process as*

$$\mathbb{E}_{\theta \sim q(\theta)}[\mathcal{L}(\theta)] \leq \mathbb{E}_{\theta \sim q(\theta)}\left[\hat{\mathcal{L}}(\theta, \mathcal{D})\right] + \mathcal{C}(q(\theta), p(\theta), N, \delta) \tag{11.5}$$

Here, $\mathcal{C}(q(\theta), p(\theta), N, \delta)$ is a term that measures the model complexity. In a simple case, this could, for instance, be [1154, 1314]

$$\mathcal{C}(q(\theta), p(\theta), N, \delta) = \sqrt{\frac{\mathrm{D}_{\mathrm{KL}}(q(\theta) \| p(\theta)) + \log \frac{2\sqrt{N}}{\delta}}{2N}} \tag{11.6}$$

This bound fits several key intuitions: it gets tighter as the size of the dataset grows because the model can overfit less; it gets looser if we reduce δ because then the bound needs to hold with higher probability; and it gets looser as the KL-divergence between posterior and prior grows because then the model can overfit more.

In summary, to get a small bound on the expected risk, you need a posterior that predicts well in terms of empirical risk and at the same time is as simple as possible in terms of the complexity measure $\mathcal{C}$. If this again sounds like Occam's razor to you, you will not be surprised to learn that PAC-Bayes bounds can be formally connected to the marginal likelihood (indeed, this section would probably not be part of this chapter otherwise).

If we take the posterior in the bound to be the true Bayes posterior, Germain et al. [610] derive the following PAC-Bayes bound. Note that the true posterior is often intractable and can only be approximated, in which case the bound will only hold approximately. Note also that it only holds under the technical condition that the log likelihood is sub-Gamma; a sub-Gamma loss is one whose tails are upper-bounded by the tails of a Gamma distribution, essentially ensuring that the loss does not grow unexpectedly fast. This holds, for instance, for Bayesian linear regression with a Gaussian likelihood.

Proposition 11.2 (Marginal likelihood PAC-Bayes bound [610])**.** *For any choice of prior $p(\theta)$, and choice of $\delta \in (0, 1)$, and if the negative log likelihood loss $\ell(\theta, \mathcal{D}) = -\log p(\mathcal{D} \mid \theta)$ is sub-Gamma with variance factor s^2 and scale $c < 1$, the expected true risk under the Bayes posterior $p(\theta \mid \mathcal{D})$ is bounded with probability at least $1 - \delta$ over sampling of datasets $\mathcal{D}$ of size N from the data-generating process as*

$$\mathbb{E}_{\theta \sim p(\theta|\mathcal{D})}[\mathcal{L}(\theta)] \leq \frac{s^2}{2(1-c)} - \frac{1}{N} \log\left(\delta\, p(\mathcal{D})\right) \tag{11.7}$$

We therefore see that maximising the marginal likelihood directly minimises this bound on the generalisation error, providing another strong (frequentist) argument for why it is a reasonable criterion for model selection. Remember, however, as a caveat, that this result only strictly holds for sub-Gamma likelihoods, such as for linearised Laplace in the regression setting, but not in the case of full Laplace inference in BNNs, especially in the classification setting. For those cases, the bound does not provide a proper guarantee, but still an intuitive argument for why the Occam's razor effect of the marginal likelihood can improve generalisation.

11.2 Differentiable marginal likelihood with Laplace

So now that we have convinced ourselves that the marginal likelihood is a great criterion for model selection, we are left with the problem of actually computing (or approximating) it. Indeed, this is one of the main challenges of Bayesian inference, which is why, in practice, the marginal likelihood is not often used for this purpose. However,

as promised above, the Laplace approximation has the lucky property that we can actually compute the marginal likelihood in closed form!

To understand how we can do that, we will need to look under the hood of the Laplace approximation in Equation 7.4b. Let us remind ourselves that we are aiming to approximate the posterior

$$p(\theta \mid \mathcal{D}) = \frac{p(\theta)\, p(\mathcal{D} \mid \theta)}{p(\mathcal{D})} = \frac{1}{p(\mathcal{D})}\, \tilde{p}(\theta) \tag{11.8}$$

where we have now written the unnormalised posterior as $\tilde{p}(\theta)$. In the context of model selection, feel free to mentally condition all of these distributions on $\mathcal{M}$ as in Equation 11.1 above. We will omit this conditioning here to not needlessly clutter the notation. Note also that we can choose the regularised risk from Equation 7.2 to be $\mathcal{L}_\Omega(\theta) = -\log \tilde{p}(\theta)$.

If we approximate $\log \tilde{p}(\theta)$ around a point θ_*, this gives

$$\log \tilde{p}(\theta) \approx \log \tilde{p}(\theta_*) + \nabla_{\theta_*} \log \tilde{p}(\theta_*)^\top (\theta - \theta_*) - \frac{1}{2}(\theta - \theta_*)^\top \Psi(\theta_*)(\theta - \theta_*) + \mathcal{O}(\theta^3) \tag{11.9}$$

where $\Psi(\theta)$ is the same Hessian operator as in Equation 7.5. Note that Ψ is the Hessian of $-\log \tilde{p}(\theta)$, hence the sign change in the Taylor expansion.

If we now choose θ_* to be a maximiser of the (log) posterior, such that the first-order term vanishes because $\nabla_\theta \log \tilde{p}(\theta) = 0$, and ignore the third and higher order terms, we can approximate $\tilde{p}(\theta)$ with a distribution $q(\theta)$, which we define as

$$q(\theta) = \frac{1}{Z_q} \exp\left(\log \tilde{p}(\theta_*) - \frac{1}{2}(\theta - \theta_*)^\top \Psi(\theta_*)(\theta - \theta_*) \right) \tag{11.10}$$

$$= \frac{\tilde{p}(\theta_*)}{Z_q} \exp\left(-\frac{1}{2}(\theta - \theta_*)^\top \Psi(\theta_*)(\theta - \theta_*) \right) \tag{11.11}$$

Now, instead of computing the true marginal likelihood $p(\mathcal{D}) = \int \tilde{p}(\theta) \mathrm{d}\theta$, we can just approximate it with the normaliser of our approximate distribution, that is $p(\mathcal{D}) \approx q(\mathcal{D}) = Z_q$.

Since we see from the quadratic form in the exponential that $q(\theta)$ is multivariate Gaussian with mean θ_* and precision matrix $\Psi(\theta_*)$, we can look up the normalising constant, which is $\sqrt{(2\pi)^P \det \Psi(\theta_*)^{-1}}$, where P is the dimensionality of the parameter space. This means that $\tilde{p}(\theta_*)/Z_q = (2\pi)^{-P/2} \det \Psi(\theta_*)^{1/2}$. Therefore, our approximate log marginal likelihood is

$$\log Z_q = \log q(\mathcal{D}) = \log \tilde{p}(\theta_*) - \frac{1}{2} \log \det \Psi(\theta_*) + \frac{P}{2} \log 2\pi \tag{11.12}$$

$$= \log p(\theta_*) + \log p(\mathcal{D} \mid \theta_*) - \frac{1}{2} \log \det \Psi(\theta_*) + \text{const} \tag{11.13}$$

Note that in the last line, we assumed that the number of parameters P is the same across the different models $\mathcal{M}$ that are being compared. If the model selection is meant to be performed across models with different numbers of parameters, this term naturally is not constant and still needs to be considered in the marginal likelihood.

11.2.1 Computational considerations

Similar to the inversion Ψ^{-1} discussed in previous chapters, the determinant $\det \Psi$ also has a computational cost of $\mathcal{O}(P^3)$ if the matrix is of size $P \times P$. As discussed before, this can get very expensive when the dimensionality of the parameter space is large, that is, for large neural network models. However, luckily, we can use some computational tricks and approximations to reduce the computational cost, such as linearisation and the empirical Fisher information, as well as diagonal or low-rank approximations (see Chapter 8 and Chapter 40 for details). Since none of these considerations are unique to the marginal likelihood computation, we will not discuss them at length in this chapter.

11.2.2 Post-hoc vs. online model selection

When selecting models using the marginal likelihood, the most naïve way is to perform *post-hoc* model selection. For instance, we can train two models, $\mathcal{M}_0$ and $\mathcal{M}_1$, and then compute their respective (log) marginal likelihoods. For illustration purposes, let us say that $\mathcal{M}_0$ is a fully-connected neural network and $\mathcal{M}_1$ is a convolutional neural network. We train them both on some image dataset (e.g., MNIST) and then compute $\log q(\mathcal{D} \mid \mathcal{M}_0) - \log q(\mathcal{D} \mid$



Figure 11.2: Using the Laplace-approximated marginal likelihood for online model selection of the prior variances on a small Bayesian neural network on a 2D classification task. We see that the marginal likelihood converges to a local optimum over the course of training and that the optimised model is better regularised than a naïvely trained (overfitting) model. [Adapted from 812]

$\mathcal{M}_1$). If this quantity is positive, we select $\mathcal{M}_0$, if it is negative, we select $\mathcal{M}_1$. Note that this difference between log marginal likelihoods is equivalent to the likelihood ratio test, which, according to the famous Neyman-Pearson lemma [1233], is the uniformly most powerful hypothesis test.

This selection approach gives us all the benefits of the marginal likelihood described in the previous section, but it still requires that we train each of the different models we want to test. Indeed, for selecting some parts of the model, we can do much better than that! While the neural network architecture is a discrete choice that we have to test post hoc, there are hyperparameters of the neural network with respect to which we can actually compute gradients of the marginal likelihood and then optimise them *online*. These include, for instance, the variance(s) of the prior or the likelihood noise. We will denote as $\mathcal{M}^\partial$ the subset of parameters for which gradients $\nabla_{\mathcal{M}^\partial} \log q(\mathcal{D} \mid \mathcal{M})$ exist. We can then optimise these parameters by performing simple gradient ascent like

$$\mathcal{M}^\partial \leftarrow \mathcal{M}^\partial + \gamma \nabla_{\mathcal{M}^\partial} \log q(\mathcal{D} \mid \mathcal{M}) \tag{11.14}$$

where γ is a step size. This allows us to select all these model hyperparameters within a single training run, without the need to train several models, as would be the case for approaches such as cross-validation.

However, this approach comes with a few caveats: First, recall that the marginal likelihood depends on the Hessian $\Psi(\theta_*)$, so on the second derivatives of the loss. If we want to take gradients of the marginal likelihood, we need to ensure that these are differentiable with respect to the hyperparameters, which is not true for all activation functions. Particularly, it is not true for the ReLU activation function, so we need to ensure to use smoother ones if we want to perform online model selection (e.g., sigmoid, tanh, Swish, GeLU, ELU, etc.). Second, since the marginal likelihood depends on the entire dataset $\mathcal{D}$, computing it requires an entire pass over the dataset, so in deep-learning parlance, one epoch. Since the best parameters $\theta*$ generally depend on the choice of $\mathcal{M}^\partial$, we typically interleave the training, that is, we train θ for one or more epochs and then make one or several gradient steps on the marginal likelihood as written above, each of which corresponds to an additional epoch on the data. Depending on the frequency of these updates, the training will therefore require more epochs; in practice, typically around twice as many. Finally, dropping the first-order term in the Taylor expansion above required θ_* to be a true optimum of the loss, where the gradients vanish, but one might wonder whether standard optimisers actually find such optima in deep learning. Indeed, in practice, the optima we find do not truly have zero gradients everywhere. However, it has been shown empirically that this generally does not impair the model selection with the marginal likelihood [812] and there are even some theoretical arguments for why this might be the case [1041].

11.2.3 Simple example

As a simple illustrative example of online model selection with the marginal likelihood, we will consider a small fully-connected neural network for a synthetic 2D classification task. We place a Gaussian prior over the weights and we assume that the prior variance is shared within each layer. That is, the ith weight in the ℓth layer will have the prior $p(\theta_{\ell i}) = \mathcal{N}(0, \sigma_\ell^2)$. Our hyperparameters to select are now these layerwise variances, so $\mathcal{M}^\partial = \{\sigma_\ell^2\}_{\ell=1}^L$.

We see in Figure 11.2 that our estimate of the marginal likelihood increases monotonically over the course of training and seems to converge to a local optimum. We also see that the prior variances indeed change during training and also seem to converge. The final model achieves a lower training accuracy compared to a naïvely trained model, but a higher test accuracy. This indicates that our selected model is better regularised and overfits less, in line with the Occam's razor effect of the marginal likelihood. The visual smoothness of its decision boundary supports this conclusion.

11.2.4 On the risk of overfitting

Based on the previous example and our arguments above, you may now be convinced that the Laplace marginal likelihood will always select the best model for you, no matter the circumstances. Unfortunately, you would be wrong about that. We do not want to hide the caveat that maximum marginal likelihood (ML-II)—much like standard maximum likelihood (ML-I)—carries the inherent risk of overfitting. In our previous examples, such as choosing between two model architectures or optimising layerwise prior variances, this has not been apparent because the number of models compared (or number of hyperparameters optimised) was small compared to both the number of parameters of the model and the number of datapoints. However, if we were to optimise a number of hyperparameters (or compare a number of models) larger than P and N, we may well overfit the model to the data.

In such cases, just like in standard maximum likelihood, one remedy is to make the hyperprior $p(\mathcal{M})$ in Equation 11.2 more informative and thus regularise the optimisation. We could, for instance, have a separate prior for each weight $p(\theta_{\ell i}) = \mathcal{N}(\mu_{\ell i}, \sigma^2_{\ell i})$ that we optimise with the marginal likelihood, but then have strong hyperpriors $p(\mu)$ and $p(\sigma^2)$ that regularise these two parameters to some reasonable values unless the marginal likelihood really provides strong evidence for different ones. For a more in-depth discussion on the risks of overfitting with the marginal likelihood in model selection and the use of hyperpriors, see Ober et al. [1253].

Another remedy, proposed by Lotfi et al. [1081], is to compute the marginal likelihood not on the whole dataset, but only on a subset, similar to cross-validation. The authors call this the *conditional log marginal likelihood* (CLML). While this can in practice indeed reduce the overfitting effects, it inherits a lot of the computational challenges of standard cross-validation, most importantly the need to choose a validation split or to perform the procedure several times across different splits.

11.2.5 Connection to flat minima

A lot of the arguments we made above in Section 11.1 hold for the true marginal likelihood $p(\mathcal{D})$. Based on some of the arguments in Chapter 7, you might wonder whether our approximation $q(\mathcal{D})$ is not too crude for model selection purposes. If the true BNN posterior has exponentially many modes, each of which has potentially a non-Gaussian geometry, should we really believe that approximating the marginal likelihood with a single Gaussian on one mode is enough to judge the quality of the model? Indeed, the proof of the metaphorical pudding is in the eating, and we will see in the following section that the Laplace marginal likelihood can effectively be used for many model-selection use cases in practice. However, before we get to the empirics, there is one more conceptual argument we can offer to hopefully appease the sceptical reader.

If we have another look at our Laplace marginal likelihood criterion in Equation 11.13, we see that it has three parts: the prior, the likelihood, and the log determinant of the Hessian. That means that models will achieve a large marginal likelihood estimate if they are probable under the prior, fit the data well (in terms of the likelihood), and have a small determinant of the Hessian. The latter of these is clearly the most cryptic. What does a small determinant mean? Recall that we assume that we are at a local optimum, or have at least linearised our network or added a sufficiently strong prior, such that the Hessian is positive definite. That means that all its eigenvalues are positive. Since the determinant is the product of the eigenvalues, a small determinant means that the eigenvalues of the Hessian are small, which in turn means that the loss function (our negative log posterior) has low curvature in the region around θ_*—it is relatively flat.

The idea that such flat minima can lead to better generalisation has been around for a while [759]. In fact, it is quite intuitive, since it essentially means that small changes in θ_* do not lead to large changes in the loss, so even if we are a bit off from the true optimum and even if the loss landscape shifts a bit due to distribution shift in the test data, we will probably still achieve a relatively low test error. This intuition is also sketched in Figure 11.3. If this combination of train error (likelihood) and complexity measure (flatness) sounds like a PAC-Bayes bound to you, you are on the right track: flat minima have indeed been connected to PAC-Bayesian results [699]. Moreover, the deep learning community has embraced flatness-encouraging training criteria—similar to our marginal likelihood—for its purposes, such as the famous sharpness-aware minimisation [SAM, 544] or its Bayesian counterpart [bSAM, 1197].

11.3 Use cases

As we have seen above, the Laplace marginal likelihood can be effectively used to select differentiable hyperparameters online, such as the prior variances or the likelihood noise, and to compare non-differentiable design choices post hoc, such as neural network architectures. These capabilities have been discovered by Immer et al. [812]

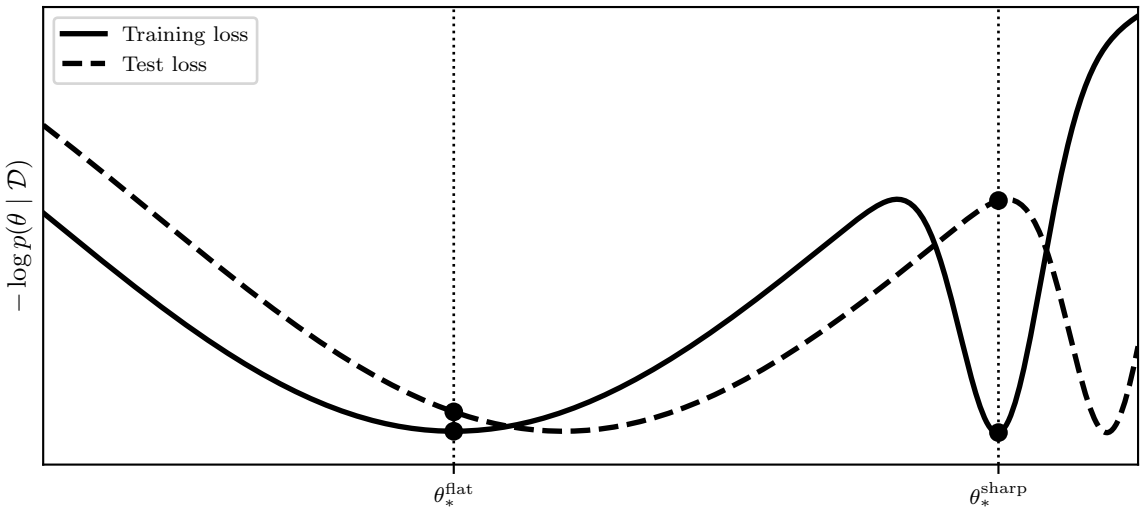


Figure 11.3: Here we show a loss landscape with a flat and a narrow minimum. As the loss landscape shifts from $\mathcal{D}_{\text{train}}$ to $\mathcal{D}_{\text{test}}$, the solution θ_*^{flat} still achieves a good performance, whereas the solution θ_*^{sharp} does not.

for modern deep learning models, and they have shown that this type of model selection is often cheaper and more effective than cross-validation. However, this type of hyperparameter tuning is by far not the only way the marginal likelihood can be used for model selection in deep learning. In recent years, different use cases have been developed, some of which have the potential to unlock entirely new application areas. While we cannot possibly provide a comprehensive overview of all of them, we will introduce a selection of the most interesting and creative ones in this section.

11.3.1 Feature selection

A common use case for model selection in Gaussian process models is feature selection via the automatic relevance determination (ARD) kernel [1382, 1736] (see also Chapter 23). This is a kernel in which each feature dimension has its own associated kernel length scale. These length scales can then be tuned using the marginal likelihood. One can perform a similar kind of feature selection in BNNs with the Laplace approximation, by performing model selection on the input weights. This is naturally more reasonable for data where the different input features have distinct semantic meaning, such as tabular data, than where they do not, such as images. One can even extend this idea to neural additive models (NAMs) [8]—that is, models where each input feature is passed to a separate neural network module—by performing model selection on the entire modules. Bouchiat et al. [197] have shown that the Laplace marginal likelihood provides an effective way to select features in Bayesian NAMs and even further improve their interpretability through its Occam's razor effect.

11.3.2 Neural network pruning

The aforementioned feature selection, in its simplest implementation, amounts to removing weights from the input layer. This suggests the idea that one might also be able to remove weights from deeper layers. Indeed, this is the basic premise of the research field of neural network pruning [286]. There are two major approaches to pruning: unstructured pruning, where weights across the whole weight matrix are removed, and structured pruning, where entire rows or columns of the weight matrix are removed (this corresponds to removing neurons from the layer). Note that while the latter approach is more aggressive, it is the only one that immediately leads to computational gains, since the former approach just makes the matrix more sparse, but not smaller. Dhahri et al. [421] have shown that the Laplace marginal likelihood can be used to improve pruning in both settings, through its Occam's razor effect, which encourages the neural networks to only use just as many weights as are needed for a given task. They have also shown that the Laplace posterior offers a simple yet effective pruning criterion: the log posterior density evaluated at 0.

11.3.3 Invariance learning

While the previous two use cases are still intuitively connected to our original model selection examples, as well as to existing applications in other models such as Gaussian processes, Immer et al. [815] propose to use the marginal likelihood for a more creative task: learning invariances from data. While invariances are commonplace in the context of designing invariant neural network architectures by hand (e.g., convolutional neural networks), there are many applications where invariances may be present in the data but the user does not know them. Following the approach from van der Wilk et al. [1662], Immer et al. [815] suggest using the following parameterisation of a generic invariant function:

$$\hat{f}(x, \theta, \eta) = \mathbb{E}_{x' \sim p(x'|x,\eta)}[f(x', \theta)] \tag{11.15}$$

Here, $f(\cdot)$ is the neural network, $\hat{f}(\cdot)$ is the desired invariant function, $p(x' \mid x, \eta)$ is a data augmentation distribution, with respect to which the function should be invariant, and η are hyperparameters controlling this augmentation distribution. The Laplace marginal likelihood can now be used to select η online during training and therefore enables learning a function that discovers the invariances directly from the data within a single training run. Immer et al. [815] show that this can faithfully recover known invariances and can drastically improve data efficiency on real datasets.

11.3.4 Probing

In addition to the bona fide model selection use cases introduced above, finally, the marginal likelihood can even be used for applications that—on the surface—do not look like model selection, such as probing. Probing is a common interpretability tool in language models, and it tries to answer the question of whether the latent representations of these models encode certain properties of their inputs. For instance, people might wonder "Does BERT encode the concept of verb tense?". To answer this, there is a long line of work trying to take these latent encodings and fitting different simple readout functions to them, such as linear functions, to predict the property of interest, in this case, verb tense [109]. But what if the encodings do not encode the property linearly but quadratically? Then the linear probe might not perform well, but arguably, the property is still encoded. However, a quadratic model is more complex than a linear one, so maybe one should count this property as less well encoded than in a set of encodings where a linear probe would suffice. That again sounds suspiciously like Occam's razor, and indeed Immer et al. [814] have shown that one can treat the problem of *jointly* selecting encodings and probes to predict the desired property as a model selection problem and use the Laplace marginal likelihood for a fair apples-to-apples comparison between different models. This shows that the tools presented in this chapter are much more widely applicable than one might naïvely expect, when one is a bit more creative in the interpretation of what *model selection* might look like.

Part III

Variational inference

Chapter 12

Introduction to variational inference

In probabilistic machine learning, our goal is often not just to find a single "best" set of parameters for a model, but to quantify our uncertainty over all possible parameters. We accomplish this using Bayesian inference.

Let $\Theta \subseteq \mathbb{R}^P$ denote the parameter space, and let $\theta \in \Theta$ be the vector of all parameters in our model (e.g., the weights and biases of a neural network). Let $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ be our observed training data, consisting of N input–target pairs. Bayesian inference updates our prior beliefs about the parameters, $p(\theta)$, after observing the data, yielding the *posterior* distribution $p(\theta \mid \mathcal{D})$.

This update is governed by Bayes' theorem:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})}. \tag{12.1}$$

Here,

- $p(\theta \mid \mathcal{D})$ is the **posterior distribution**, summarising our updated beliefs about θ given the data;
- $p(\mathcal{D} \mid \theta)$ is the **likelihood**, viewed as a function of θ: for a given choice of parameters, it quantifies how likely the observed data are under the model;
- $p(\theta)$ is the **prior distribution**, which encodes our beliefs about θ before observing any data; and
- $p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)p(\theta)\,\mathrm{d}\theta$ is the **marginal likelihood**, also called the **evidence**.

The central computational challenge in most modern Bayesian inference problems is evaluating $p(\mathcal{D})$, which requires integrating the joint distribution $p(\mathcal{D}, \theta)$ over the parameter space Θ. For models such as Bayesian neural networks, this integral is high-dimensional and generally lacks an analytic closed-form solution. Consequently, the posterior density $p(\theta \mid \mathcal{D})$ usually cannot be evaluated in normalised form, motivating approximate inference methods. This chapter focuses on variational inference, also known as variational Bayes, a widely used family of scalable approximate inference methods [170, 851, 1691].

12.1 A Variational Approach: Bayesian Inference as Optimisation

The core idea of variational inference (VI) is to reframe Bayesian inference as an optimisation problem. Instead of representing the generally intractable posterior $p(\theta \mid \mathcal{D})$ exactly, we choose a tractable distribution $q(\theta; \lambda)$ from a variational family $\mathcal{Q}$, where λ denotes its variational parameters. Concretely, $\mathcal{Q}$ could be a family of Gaussian distributions or a family of factorised distributions, as developed in Chapter 13. The goal is to find the member $q(\theta; \lambda^*)$ that best approximates the posterior.

12.1.1 The Kullback–Leibler Divergence

Following Kullback and Leibler [945], the **Kullback–Leibler (KL) divergence** from $q(\theta; \lambda)$ to the posterior $p(\theta \mid \mathcal{D})$ is

$$\begin{aligned} \mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda) \| p(\theta \mid \mathcal{D})) &= \int q(\theta;\lambda) \log \frac{q(\theta;\lambda)}{p(\theta \mid \mathcal{D})}\,\mathrm{d}\theta \\ &= \mathbb{E}_{q(\theta;\lambda)}[\log q(\theta;\lambda) - \log p(\theta \mid \mathcal{D})]. \end{aligned} \tag{12.2}$$

Because the KL divergence is asymmetric, its direction must be stated explicitly. VI seeks variational parameters satisfying

$$\lambda^* \in \arg\min_{\lambda} \mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta \mid \mathcal{D})). \tag{12.3}$$

12.1.2 Rewriting the KL Objective

Expanding the posterior in Equation (12.2) using Bayes' theorem gives

$$\mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta \mid \mathcal{D})) = \mathbb{E}_{q(\theta;\lambda)}[\log q(\theta;\lambda) - \log p(\mathcal{D},\theta)] + \log p(\mathcal{D}). \tag{12.4}$$

Although the KL divergence itself cannot generally be evaluated exactly because $\log p(\mathcal{D})$ is unavailable, the log-evidence term is constant with respect to λ and therefore does not affect the optimiser in Equation (12.3). The remaining expectation does not require evaluating $p(\mathcal{D})$ and can be estimated using samples from $q(\theta;\lambda)$ whenever the joint density $p(\mathcal{D},\theta)$ and variational density $q(\theta;\lambda)$ can be evaluated.

12.1.3 The Evidence Lower Bound (ELBO)

Rearranging Equation (12.4) yields

$$\log p(\mathcal{D}) = \underbrace{\mathbb{E}_{q(\theta;\lambda)}[\log p(\mathcal{D},\theta) - \log q(\theta;\lambda)]}_{\mathcal{L}(\lambda)} + \underbrace{\mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta \mid \mathcal{D}))}_{\geq 0}. \tag{12.5}$$

The first term defines the **evidence lower bound (ELBO)** $\mathcal{L}(\lambda)$. Because the KL divergence is non-negative,

$$\mathcal{L}(\lambda) \leq \log p(\mathcal{D}). \tag{12.6}$$

The gap between the ELBO and the log evidence is exactly $\mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta \mid \mathcal{D}))$; the closer the variational distribution is to the posterior in this divergence, the smaller the gap. Since $\log p(\mathcal{D})$ is constant with respect to λ, Equation (12.5) also gives

$$\arg\max_{\lambda} \mathcal{L}(\lambda) = \arg\min_{\lambda} \mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta \mid \mathcal{D})). \tag{12.7}$$

Thus, VI replaces evaluation of the posterior in normalised form with optimisation of an objective that can be estimated under suitable choices of likelihood, prior, and variational family.

12.1.4 Interpreting the ELBO

Using $p(\mathcal{D},\theta) = p(\mathcal{D} \mid \theta)p(\theta)$, the ELBO can be written as

$$\mathcal{L}(\lambda) = \mathbb{E}_{q(\theta;\lambda)}[\log p(\mathcal{D} \mid \theta)] - \mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda)\|p(\theta)). \tag{12.8}$$

The alternative form in Equation (12.8) reveals a trade-off between two terms:

- **Expected log-likelihood (data fit):** This term rewards variational distributions that place more probability in regions of parameter space where the observed data have high likelihood.
- **KL penalty (regularisation):** This term penalises departures of $q(\theta;\lambda)$ from the prior $p(\theta)$ and can therefore be viewed as a complexity penalty.

VI selects the member of $\mathcal{Q}$ that balances fit to the observed data against departure from the prior.

12.1.5 Other Lower Bounds on the Log-Evidence

The ELBO is one lower bound on $\log p(\mathcal{D})$. More generally, strictly positive unbiased estimators of $p(\mathcal{D})$ can induce lower bounds through Jensen's inequality; this optional subsection states the construction underlying importance-weighted and sequential Monte Carlo objectives.

Assume that $q(\theta;\lambda) > 0$ whenever $p(\mathcal{D},\theta) > 0$. Then

$$\mathbb{E}_{q(\theta;\lambda)}\left[\frac{p(\mathcal{D},\theta)}{q(\theta;\lambda)}\right] = \int p(\mathcal{D},\theta)\,\mathrm{d}\theta = p(\mathcal{D}). \tag{12.9}$$

For a concave function φ and random variable X, Jensen's inequality states that $\varphi(\mathbb{E}[X]) \geq \mathbb{E}[\varphi(X)]$. Taking $\varphi = \log$ in Equation (12.9) gives

$$\begin{aligned} \mathcal{L}(\lambda) &= \mathbb{E}_{q(\theta;\lambda)}\left[\log \frac{p(\mathcal{D},\theta)}{q(\theta;\lambda)}\right] \\ &\leq \log \mathbb{E}_{q(\theta;\lambda)}\left[\frac{p(\mathcal{D},\theta)}{q(\theta;\lambda)}\right] = \log p(\mathcal{D}). \end{aligned} \tag{12.10}$$

More generally, let $r(\mathbf{u};\lambda)$ denote the sampling density of all random variables $\mathbf{u}$ used to construct a strictly positive unbiased estimator $g(\mathcal{D};\mathbf{u},\lambda)$ of $p(\mathcal{D})$, and suppose that $\mathbb{E}_{r(\mathbf{u};\lambda)}[|\log g(\mathcal{D};\mathbf{u},\lambda)|] < \infty$. Then

$$\mathbb{E}_{r(\mathbf{u};\lambda)}[g(\mathcal{D};\mathbf{u},\lambda)] = p(\mathcal{D}). \tag{12.11}$$

Jensen's inequality gives the lower bound

$$\mathcal{L}_g(\lambda) := \mathbb{E}_{r(\mathbf{u};\lambda)}[\log g(\mathcal{D};\mathbf{u},\lambda)] \leq \log p(\mathcal{D}). \tag{12.12}$$

The standard ELBO is recovered by taking $\mathbf{u} = \theta$, $r(\theta;\lambda) = q(\theta;\lambda)$, and $g(\mathcal{D};\theta,\lambda) = p(\mathcal{D},\theta)/q(\theta;\lambda)$. For example, Burda et al. [220] introduce the importance-weighted autoencoder (IWAE) objective, which uses s independent samples $\theta_j \sim q(\cdot;\lambda)$ and the estimator

$$g(\mathcal{D};\mathbf{u},\lambda) = \frac{1}{s}\sum_{j=1}^{s} \frac{p(\mathcal{D},\theta_j)}{q(\theta_j;\lambda)}, \qquad \mathbf{u} = \{\theta_j\}_{j=1}^{s}. \tag{12.13}$$

Related lower bounds can be constructed using sequential Monte Carlo [1114]. Although $\mathcal{L}_g(\lambda)$ may not be available in closed form, it can often be optimised using Monte Carlo gradient estimators. However, a tighter bound alone does not guarantee better optimisation or predictive performance, because the signal-to-noise ratio and other properties of the associated gradient estimator also matter [1368].

In summary, VI approximates $p(\theta \mid \mathcal{D})$ by optimising a distribution from a tractable family, with the ELBO providing an objective that balances data fit against departure from the prior. Chapter 13 develops the mean-field construction in detail, and subsequent chapters consider further variational families and optimisation methods.

Chapter 13

Gaussian mean-field variational inference

In Chapter 12, we introduced the evidence lower bound (ELBO), a tractable objective that reframes Bayesian inference as an optimisation problem. However, optimising

$$\mathcal{L}(\lambda) = \mathbb{E}_{q(\theta;\lambda)}[\log p(\mathcal{D} \mid \theta)] - \mathrm{D}_{\mathrm{KL}}(q(\theta;\lambda) \| p(\theta)) \tag{13.1}$$

still requires evaluating an expectation. Unless the likelihood, prior, and variational family have special conjugate or linear-Gaussian structure, this expectation is rarely available in closed form and is generally unavailable for neural-network likelihoods.

To simplify optimisation, we adopt a strong structural assumption known as the *mean-field* assumption: the variational distribution factorises over the parameters, or over predefined groups of parameters. For $\theta = (\theta_1, \ldots, \theta_P)$, the fully factorised form is

$$q(\theta;\lambda) = \prod_{i=1}^{P} q_i(\theta_i;\lambda_i). \tag{13.2}$$

Thus, the coordinates are independent under the approximation q, even though they are generally dependent under the true posterior $p(\theta \mid \mathcal{D})$. This assumption permits coordinate-wise updates, but it prevents q from representing posterior correlations and often leads to underestimated marginal posterior variances.

13.1 Coordinate Ascent Variational Inference

Under the mean-field assumption, the ELBO can be optimised by cyclically updating one factor q_j while holding the remaining factors fixed. This procedure is known as coordinate ascent variational inference (CAVI). Define

$$q_{-j}(\theta_{-j};\lambda_{-j}) := \prod_{i \neq j} q_i(\theta_i;\lambda_i). \tag{13.3}$$

Substituting the factorised approximation into the ELBO and retaining only terms that depend on q_j gives

$$\log q_j^*(\theta_j) = \mathbb{E}_{q_{-j}(\theta_{-j};\lambda_{-j})}[\log p(\mathcal{D}, \theta)] + \text{const}, \tag{13.4}$$

where the additive constant normalises q_j^*. Equivalently, the optimal factor is proportional to the exponentiated expected log-joint, with the expectation taken over the current approximation to all parameters other than θ_j.

The full procedure is given in Algorithm 17. Each exact coordinate update leaves the ELBO unchanged or increases it; under standard regularity conditions, repeated updates converge to a stationary point, but not necessarily to the global optimum. Closed-form CAVI updates are most readily available in conditionally conjugate models; otherwise, the expectation or the resulting update may itself require approximation.

For a recent hybrid approach combining variational inference and MCMC, see Alexos et al. [21].

We now strengthen the mean-field assumption by taking each factor to be Gaussian:

$$q_i(\theta_i;\lambda_i) = \mathcal{N}(\theta_i \mid \mu_i, \sigma_i^2), \qquad \sigma_i > 0. \tag{13.5}$$

This is the *Gaussian mean-field* assumption. The variational parameters determine the per-coordinate means and positive standard deviations,

$$\lambda = \{(\mu_i, \sigma_i)\}_{i=1}^{P}. \tag{13.6}$$

Algorithm 17: Coordinate ascent variational inference (CAVI).

Result: Converged factors $q_1(\theta_1), \dots, q_P(\theta_P)$
Initialise all factors $q_i(\theta_i; \lambda_i)$ for $i = 1, \dots, P$
while *the ELBO has not converged* **do**
  for $j = 1, \dots, P$ **do**
    Update q_j using the CAVI rule:

$$q_j(\theta_j) \propto \exp\big(\mathbb{E}_{q_{-j}(\theta_{-j};\lambda_{-j})}[\log p(\mathcal{D}, \theta)]\big)$$

  end
end
return converged factors $q_1(\theta_1), \dots, q_P(\theta_P)$

For gradient-based optimisation, each positive standard deviation is typically represented through an unconstrained scale parameter. The variational parameters may be optimised by CAVI when closed-form updates are available or, more generally, by the stochastic-gradient methods introduced next.

13.2 Stochastic Gradient Optimisation

When closed-form CAVI updates are unavailable, the ELBO can instead be maximised by stochastic gradient ascent. Writing

$$g(\theta; \lambda) := \log p(\mathcal{D} \mid \theta) + \log p(\theta) - \log q(\theta; \lambda), \tag{13.7}$$

we require an estimator of

$$\nabla_\lambda \mathcal{L}(\lambda) = \nabla_\lambda \mathbb{E}_{q(\theta;\lambda)}[g(\theta; \lambda)]. \tag{13.8}$$

The dependence of the sampling distribution $q(\theta; \lambda)$ on λ prevents a naive interchange of the gradient and expectation.

13.2.1 The Reparameterisation Gradient Estimator

A standard low-variance approach is the *reparameterisation gradient estimator*, also called the pathwise estimator [892, 1391]. For the Gaussian mean-field family, draw parameter-free noise and transform it according to

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \theta = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}. \tag{13.9}$$

This gives

$$\mathbb{E}_{q(\theta;\lambda)}[g(\theta; \lambda)] = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})}[g(\theta(\lambda, \boldsymbol{\epsilon}); \lambda)]. \tag{13.10}$$

Under the usual regularity conditions, differentiation can now be moved inside the expectation:

$$\nabla_\lambda \mathbb{E}_{q(\theta;\lambda)}[g(\theta; \lambda)] = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})}[\nabla_\lambda g(\theta(\lambda, \boldsymbol{\epsilon}); \lambda)]. \tag{13.11}$$

Here ∇_λ is the total derivative through both $\theta(\lambda, \boldsymbol{\epsilon})$ and the explicit λ-dependence of g. Relative to score-function (REINFORCE) estimators, the reparameterisation estimator is typically much lower variance while remaining unbiased under the stated regularity conditions. With mini-batches, the gradient is doubly stochastic because of both data subsampling and the draw of $\boldsymbol{\epsilon}$; larger mini-batches, multiple independent noise samples, and control variates can reduce variance at additional computational cost, while the resulting noise must be accounted for when selecting the step-size schedule. The pathwise construction also applies to other differentiably reparameterisable variational families.

Algorithm 18 gives a single-sample stochastic-gradient step.

13.3 Examples of Gaussian Mean-Field Variational Inference

We begin with a conjugate linear model, then turn to Bayesian neural networks and variational autoencoders.

Algorithm 18: Reparameterisation-gradient step for Gaussian mean-field variational inference.

Input: Variational parameters λ determining $\boldsymbol{\mu}$ and positive $\boldsymbol{\sigma}$, learning rate ρ, mini-batch size b
Result: Updated variational parameters λ
Sample a mini-batch $\mathcal{B} \subseteq \{1, \dots, N\}$ of size b uniformly without replacement
Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
Compute $\theta = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ `/* Reparameterisation */`
Compute the scaled mini-batch log-likelihood

$$\widehat{\ell}_{\mathcal{B}}(\theta) = \frac{N}{b} \sum_{n \in \mathcal{B}} \log p(y_n \mid x_n, \theta)$$

Compute the gradient estimate

$$\widehat{g} = \nabla_\lambda \left[\widehat{\ell}_{\mathcal{B}}(\theta) + \log p(\theta) - \log q(\theta; \lambda) \right]$$

Update $\lambda \leftarrow \lambda + \rho\, \widehat{g}$
return λ

13.3.1 Example 1: Bayesian Linear Regression

Consider the Bayesian linear regression model

$$y_n = \Phi(x_n)^\top \mathbf{w} + \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, \beta^{-1}), \tag{13.12}$$

where β is the noise precision, $\Phi(\cdot)$ is a feature map, and the model parameters are $\theta = \mathbf{w}$. We use

- **Prior:** $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \alpha^{-1}\mathbf{I})$;
- **Likelihood:** $p(y_n \mid x_n, \mathbf{w}) = \mathcal{N}(y_n \mid \Phi(x_n)^\top \mathbf{w}, \beta^{-1})$.

The exact posterior is a full-covariance Gaussian, which we write as

$$p(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w} \mid \mathbf{m}, \mathbf{S}). \tag{13.13}$$

We approximate it with the Gaussian mean-field family

$$q(\mathbf{w}; \lambda) = \prod_{i=1}^{P} \mathcal{N}(w_i \mid \mu_i, \sigma_i^2). \tag{13.14}$$

In this conjugate setting, the coordinate updates are available in closed form. More precisely, the reverse-KL-optimal diagonal Gaussian has $\mu_i = m_i$ and $\sigma_i^2 = 1/(\mathbf{S}^{-1})_{ii}$. Thus, it discards the off-diagonal covariance terms, and $\sigma_i^2 \le S_{ii}$ shows explicitly how the mean-field approximation can underestimate the exact marginal variances when posterior correlations are present.

13.3.2 Example 2: Bayesian Neural Networks (BNNs)

Consider a Bayesian neural network whose parameters θ collect all weights and biases; the parameter count P may be very large. The model is

- **Prior:** $p(\theta) = \mathcal{N}(\theta \mid \mathbf{0}, \mathbf{I})$ (other prior choices are possible);
- **Likelihood:** $p(\mathcal{D} \mid \theta) = \prod_{n=1}^{N} p(y_n \mid x_n, \theta)$.

With the Gaussian mean-field approximation $q(\theta; \lambda) = \prod_{i=1}^{P} \mathcal{N}(\theta_i \mid \mu_i, \sigma_i^2)$, the ELBO is

$$\mathcal{L}(\lambda) = \sum_{n=1}^{N} \mathbb{E}_{q(\theta;\lambda)}[\log p(y_n \mid x_n, \theta)] - \mathrm{D}_{\mathrm{KL}}(q(\theta; \lambda) \| p(\theta)). \tag{13.15}$$

Because of the network's non-linear transformations, the expected log-likelihood is generally unavailable in closed form, so the analytic CAVI updates above do not apply. The ELBO can instead be optimised using the reparameterisation-gradient procedure in Algorithm 18, yielding the Bayes by Backprop approach [174]. The same stochastic-gradient construction applies to any differentiably reparameterisable variational family, not only to Gaussian mean-field approximations.

13.3.3 Example 3: Variational Autoencoders

Variational inference also applies to generative latent-variable models. In a variational autoencoder (VAE) [892, 1391], VI approximates the posterior of a *local latent variable* $\mathbf{z}_n$ for each data point x_n, rather than a posterior over the decoder's global parameters. The generative model is

1. **Prior:** $\mathbf{z} \sim p(\mathbf{z})$, for example $p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, \mathbf{I})$;
2. **Likelihood:** $p_{\boldsymbol{\psi}}(x \mid \mathbf{z})$, represented by a decoder network with parameters $\boldsymbol{\psi}$.

The goal is to maximise the marginal log-likelihood

$$\log p_{\boldsymbol{\psi}}(x) = \log \int p_{\boldsymbol{\psi}}(x \mid \mathbf{z}) p(\mathbf{z}) \, \mathrm{d}\mathbf{z}, \tag{13.16}$$

which is generally intractable. For any distribution $q(\mathbf{z})$, the variational principle from Section 12.1.3 gives

$$\log p_{\boldsymbol{\psi}}(x) \geq \mathcal{L}(q, \boldsymbol{\psi}; x) := \mathbb{E}_{q(\mathbf{z})}[\log p_{\boldsymbol{\psi}}(x \mid \mathbf{z})] - \mathrm{D}_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z})). \tag{13.17}$$

The bound is tight at the true posterior $q(\mathbf{z}) = p_{\boldsymbol{\psi}}(\mathbf{z} \mid x)$, but this posterior cannot generally be evaluated because its normalising constant $p_{\boldsymbol{\psi}}(x)$ is intractable. We therefore introduce an inference network $q_{\boldsymbol{\phi}}(\mathbf{z} \mid x)$, called the encoder, with parameters $\boldsymbol{\phi}$. It performs *amortised inference* by mapping each input to the parameters of its variational approximation.

We use a diagonal-Gaussian encoder,

$$q_{\boldsymbol{\phi}}(\mathbf{z} \mid x) = \mathcal{N}\big(\mathbf{z} \mid \boldsymbol{\mu}_{\boldsymbol{\phi}}(x), \mathrm{diag}\big(\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(x)\big)\big), \tag{13.18}$$

where the encoder outputs the mean $\boldsymbol{\mu}_{\boldsymbol{\phi}}(x)$ and positive standard deviation $\boldsymbol{\sigma}_{\boldsymbol{\phi}}(x)$. For a single data point, the VAE objective is

$$\mathcal{L}(\boldsymbol{\psi}, \boldsymbol{\phi}; x) = \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|x)}[\log p_{\boldsymbol{\psi}}(x \mid \mathbf{z})]}_{\text{Expected reconstruction log-likelihood}} - \underbrace{\mathrm{D}_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid x) \| p(\mathbf{z}))}_{\text{Latent regulariser}}. \tag{13.19}$$

The total ELBO $\sum_{n=1}^{N} \mathcal{L}(\boldsymbol{\psi}, \boldsymbol{\phi}; x_n)$ is maximised jointly over $\boldsymbol{\psi}$ and $\boldsymbol{\phi}$ by stochastic gradient ascent. The reconstruction term is estimated using reparameterised samples from $q_{\boldsymbol{\phi}}(\mathbf{z} \mid x)$, while the KL term is often available in closed form, as it is for two Gaussian distributions. This joint training of a decoder and an amortised inference network is the defining construction of the variational autoencoder.

13.4 Limitations of Gaussian Mean-Field Variational Inference

Gaussian mean-field variational inference is a fast and scalable approximation that supports complex models such as BNNs and VAEs, but its simplicity rests on strong assumptions:

1. **Mean-field assumption:** the factorised approximation cannot represent posterior correlations between parameters and therefore loses the posterior's covariance structure.
2. **Reverse-KL behaviour:** minimising $\mathrm{D}_{\mathrm{KL}}(q(\theta; \lambda) \| p(\theta \mid \mathcal{D}))$ strongly penalises an approximation that assigns non-negligible probability to regions where the posterior density is low. This mode-seeking behaviour can make q concentrate too narrowly and often contributes to underestimated marginal posterior variances.
3. **Gaussianity assumption:** a single Gaussian cannot represent skewness, heavy tails, or multimodality, and reverse-KL optimisation will often select one mode of a multimodal posterior.

These limitations motivate more expressive variational families, including structured approximations that relax the mean-field factorisation and function-space methods that define the approximation over induced distributions on functions.

Chapter 14

Dropout-based variational inference

So far, we have seen that performing variational inference requires optimising an approximating variational distribution. Interestingly, in the search for a suitable approximation, we can leverage a concept originally introduced for model regularisation: stochastic regularisation. This strategy relies on a simple intuition: injecting stochastic perturbations into the inference process at training time prevents any subset of parameters from co-adapting, forcing the network to distribute the learned representation across redundant pathways. Stochastic regularisation thus effectively acts as an implicit sampling mechanism: each perturbed configuration is nothing but a draw from an underlying distribution over network configurations. In this chapter, we will indeed discover that the intersection of stochastic regularisation techniques and Monte Carlo (MC) sampling can, under certain conditions, lead to an effective approximate variational distribution.

14.1 Monte Carlo dropout

As in all the VI strategies discussed in this book, we need to find a suitable Bayesian approximation to minimise the KL-divergence and deliver all the nice properties discussed in the introduction (Chapter 12). Since we adopt a Bayesian approach within the Deep Learning context (and thus, we deal with neural networks in many different ways), we should seek a strategy to treat the network's parameters probabilistically. But wait, the literature on neural networks already explored randomisation inside the model: can't we just exploit those strategies to lay the foundation for our Bayesian approximation? Of course, we can, and the method we will explore in this chapter is, indeed, based upon well-known pieces of math that have been around for many years: **stochastic regularisation techniques (SRTs)** [565]. As the name may suggest, SRTs serve as a regularisation mechanism for the model, achieved by introducing stochasticity in the predictive process. We can directly use this randomisation approach to derive the approximate distribution we seek throughout this chapter. We now discuss how it is possible to obtain a Bayesian approximation using one of the most renowned SRTs: **dropout** [1552][1].

14.1.1 The dropout objective

Dropout is a powerful and popular SRT that is based on a simple, intuitive principle: randomly shutting neurons down during training. What does it mean, and why should we do such a thing? When we say shutting down, we literally mean *nullifying* the contribution a given neuron delivers to the output computation. For the sake of simplicity, consider a single hidden layer NN, with weight matrix $\mathbf{W}_h \in \mathbb{R}^{\mathrm{D}\times\mathrm{Q}}$ and biases $\mathbf{b}_h \in \mathbb{R}^{\mathrm{Q}}$ (connecting the input to the *hidden* layer) and weight matrix $\mathbf{W}_l \in \mathbb{R}^{\mathrm{Q}\times\mathrm{C}}$ (connecting the hidden layer to the *last*, output layer). Under regular conditions, a given neuron $\mathrm{z}^{(i)}$ on the hidden layer should account for a quantity which equals a non-linearly activated linear combination of the input and the vector $\mathbf{W}_h^{(i)}$ plus a constant $\mathrm{b}_h^{(i)}$, i.e., $\mathrm{z}^{(i)} = \sigma(\mathbf{x}^\top\mathbf{W}_h^{(i)} + \mathrm{b}_h^{(i)})$. If we apply dropout on the hidden layer, some of the layer's neurons $\mathrm{z}^{(i)}$ will preserve their original value $\sigma(\mathbf{x}^\top\mathbf{W}_h^{(i)} + \mathrm{b}_h^{(i)})$, while the remaining ones will be set to zero. Which and how many of these neurons will be zeroed out depends upon a parameter known as the **dropout rate**, which is the probability that

[1] At this point in the chapter, acute readers could ask why one should pick dropout instead of any other SRT, which would be a legitimate question! There is no particular reason for using dropout instead of, for example, dropconnect, and it is indeed possible to derive a Bayesian approximation from other SRTs. For historical reasons, we will only explore dropout, since the original technique, commonly known as "Monte Carlo Dropout", was originally proposed for dropout [566]. We leave to the most curious among the readers the demonstration of a possible Bayesian approximation using different SRTs.

a given neuron will be dropped out. Formally, this is equivalent to updating each hidden neuron by multiplying $\sigma(\mathbf{x}^\top \mathbf{W}_h^{(i)} + \mathrm{b}_h^{(i)}) \cdot \mathrm{o}^{(i)}$ with a random variable $\mathrm{o}^{(i)} \sim \mathcal{B}(1-\phi)$, so that the neuron is zeroed with probability ϕ (the dropout rate mentioned earlier). The same mechanism can be applied to the input layer, dropping individual input components. This procedure will affect the final output $\hat{\mathbf{y}}$ of our predictor, and results in:

$$\hat{\mathbf{y}} = (\sigma(\mathbf{W}_h^\top(\mathbf{x} \circ \mathbf{o}_h) + \mathbf{b}_h) \circ \mathbf{o}_l)^\top \cdot \mathbf{W}_l, \tag{14.1}$$

where $\mathbf{o}_h = [\mathrm{o}_h^{(i)}]_{i=1}^{\mathrm{D}}$ and $\mathbf{o}_l = [\mathrm{o}_l^{(i)}]_{i=1}^{\mathrm{Q}}$ are the vectors containing the realisations $\mathrm{o}_h^{(i)} \sim \mathcal{B}(1-\phi)$ and $\mathrm{o}_l^{(i)} \sim \mathcal{B}(1-\phi)$ for every input $\mathbf{x}^{(i)}$ and hidden neuron $\mathrm{z}^{(i)}$, and $\circ$ represents the Hadamard product. When the hidden neurons pass through the last layer of our example model, the final output no longer contains all the original information of the hidden layer, but only the contribution delivered by the unaltered neurons.

Now that we know how dropout works, the main question is why we should use it: isn't it unfavourable to drop some information out during the forward passes? Not really, if we remember that dropout is a regulariser that works on a neural network at training time. During training, dropping out some neurons would have the effect of forcing the predictor not to rely entirely on a tiny subset of neurons, because they can eventually not always be available. This behaviour avoids sparsity and encourages a dense knowledge distribution across the layer. During the learning process, the loss formulation will not be different from the usual one, with the exception that the output $\hat{\mathbf{y}}$ has been derived using dropout. As an example, in the regression case, we can use the squared error loss:

$$\mathcal{L}_{\mathrm{dr}}(\mathcal{D}, \theta) \coloneqq \frac{1}{2\mathrm{N}} \sum_{n=1}^{\mathrm{N}=|\mathcal{D}|} ||\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}||_2^2, \tag{14.2}$$

where θ denotes the model's parameters (i.e., $\mathbf{W}_h$, $\mathbf{b}_h$ and $\mathbf{W}_l$), used to compute the prediction $\hat{\mathbf{y}}^{(n)}$.

Starting from this dropout objective, we will now show how it is possible to derive its Bayesian approximation[2]. Nevertheless, before diving into the details, we need to add a final ingredient to our objective to balance the accounts, that is, the L_2 regularisation on the network's weights:

$$\mathcal{L}_{\mathrm{dr}}(\mathcal{D}, \theta) \coloneqq \underbrace{\frac{1}{2\mathrm{N}} \sum_{n=1}^{\mathrm{N}=|\mathcal{D}|} ||\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}||_2^2}_{\text{Mean Squared Error}} + \underbrace{\lambda_{\mathbf{W}_h} ||\mathbf{W}_h||_2^2 + \lambda_{\mathbf{W}_l} ||\mathbf{W}_l||_2^2 + \lambda_{\mathbf{b}_h} ||\mathbf{b}_h||_2^2}_{\mathrm{L}_2}, \tag{14.3}$$

where $\lambda_{\mathbf{W}_h}$, $\lambda_{\mathbf{W}_l}$ and $\lambda_{\mathbf{b}_h}$ are the weight decay coefficients.

As we will discuss later in the chapter, including this term is a fundamental step for obtaining a principled approximation; luckily, this does not represent a practical obstacle since introducing this type of regularisation is a common practice in neural network optimisation.

In what follows, dropout is presented as a Bayesian approximation by following the original derivation of Gal and Ghahramani [566], which grounds the argument in Gaussian Processes. This is not the only possible route: an alternative derivation [565] dispenses with GPs altogether and works directly in weight space, casting dropout as variational inference in a Bayesian neural network with a mixture-of-Gaussians approximating distribution, and generalising the argument to the whole family of stochastic regularisation techniques. Although we follow the GP-based derivation, since it makes the equivalence with the dropout objective of Equation 14.2 explicit, the reader should bear in mind that the Bayesian interpretation of dropout does not necessarily depend on the GP machinery.

14.1.2 Gaussian Processes: the cornerstone of the approximation

In order to ascertain whether the dropout objective is a Bayesian approximation, we need to briefly anticipate a useful non-parametric model: the **Gaussian Processes** [1382]. For the sake of simplicity we will only cover the aspects strictly needed for our derivation; for a broader treatment of Gaussian Processes and kernels, see Chapter 23.

Gaussian Processes (GPs) are useful statistical tools that allow us to define distributions over functions. Let $\mathrm{f} : \mathbb{R}^{\mathrm{D}} \to \mathbb{R}$ be a *single-output* regressor;[3] accordingly, we denote by $\mathbf{y} \in \mathbb{R}^{\mathrm{N}}$ the vector of scalar targets and by $\mathbf{f} = [\mathrm{f}(\mathbf{x}^{(n)})]_{n=1}^{\mathrm{N}} \in \mathbb{R}^{\mathrm{N}}$ the vector collecting the evaluations of f at the N inputs. So far, it is sufficient to assume

[2] Again, for the sake of simplicity, we will discuss the Bayesian approximation only for a specific case, i.e., the regression objective. The same properties apply for classification, even though some modifications are needed; we leave a discussion of this case to the reader as an exercise.

[3] The extension to the C outputs of our MLP is immediate, and is discussed at the end of the section.

the existence of a specific prior $p(\mathrm{f})$ over the elements of our hypotheses space, indicating which regressor f is more or less likely to have generated the data. Following from Bayes rule, we can obtain the posterior distribution as:

$$p(\mathrm{f}|\mathbf{X},\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X},\mathrm{f})p(\mathrm{f}|\mathbf{X})}{p(\mathbf{y}|\mathbf{X})} = \frac{p(\mathbf{y}|\mathbf{X},\mathrm{f})p(\mathrm{f}|\mathbf{X})}{\int p(\mathbf{y}|\mathbf{X},\mathrm{f})p(\mathrm{f}|\mathbf{X})\mathrm{df}} \propto p(\mathbf{y}|\mathbf{X},\mathrm{f})p(\mathrm{f}|\mathbf{X}). \tag{14.4}$$

It is precisely now that GPs come into play, allowing us to compute the posterior analytically. For doing so, GPs exploit a clever intuition: similar inputs $\mathbf{x}$'s should result in similar outputs $\mathrm{f}(\mathbf{x})$'s; in a sense, we would like a model that makes the input $\mathbf{x}$'s co-vary. This behaviour can be implemented by employing a covariance function $k(\mathbf{x}_a, \mathbf{x}_b)$, which defines the similarity between two given inputs. The covariance function allows us to place a Gaussian distribution over the function values:

$$\mathbf{f}|\mathbf{X} \sim \mathcal{N}(0, K(\mathbf{X},\mathbf{X})) \tag{14.5}$$

$$\mathbf{y}|\mathbf{f} \sim \mathcal{N}(\mathbf{f}, \tau^{-1}\mathbf{I}_{\mathrm{N}}), \tag{14.6}$$

where $K(\mathbf{X},\mathbf{X})$ is an $\mathrm{N} \times \mathrm{N}$ covariance matrix defined using the covariance function $k(\mathbf{x}_a, \mathbf{x}_b)$ and $\tau^{-1}\mathbf{I}_{\mathrm{N}}$ is the inherent predictive noise of the function, obtained by multiplying the inverse of the precision parameter τ with an $\mathrm{N} \times \mathrm{N}$ identity matrix $\mathbf{I}_{\mathrm{N}}$. Note that the GP prior is a defined over the function f, whereas Equation 14.5 states the distribution it induces over the finite set of evaluations $\mathbf{f}$ at the observed inputs; this is precisely what makes the model tractable.

The extension to the C-dimensional outputs of our MLP is immediate: it is sufficient to model each output component with its own regressor $\mathrm{f}_c : \mathbb{R}^{\mathrm{D}} \to \mathbb{R}$, $c = 1, \ldots, \mathrm{C}$, placing an independent GP on each of them, all sharing the same covariance function. To keep the notation compact, we collect targets and function evaluations into two $\mathrm{N} \times \mathrm{C}$ matrices

$$\mathbf{Y} = \begin{bmatrix}\mathbf{y}_1 & \cdots & \mathbf{y}_{\mathrm{C}}\end{bmatrix} \in \mathbb{R}^{\mathrm{N}\times\mathrm{C}}, \qquad \mathbf{F} = \begin{bmatrix}\mathbf{f}_1 & \cdots & \mathbf{f}_{\mathrm{C}}\end{bmatrix} \in \mathbb{R}^{\mathrm{N}\times\mathrm{C}}, \tag{14.7}$$

whose c-th columns $\mathbf{y}_c, \mathbf{f}_c \in \mathbb{R}^{\mathrm{N}}$ gather, respectively, the c-th target component and the c-th function evaluations $[\mathrm{f}_c(\mathbf{x}^{(n)})]_{n=1}^{\mathrm{N}}$ across the N samples; the n-th rows, which we denote by $\mathbf{y}^{(n)}, \mathbf{f}^{(n)} \in \mathbb{R}^{\mathrm{C}}$, gather instead the C output components of the n-th sample. With this notation, Equation 14.5 holds column-wise,

$$\mathbf{f}_c|\mathbf{X} \sim \mathcal{N}(0, K(\mathbf{X},\mathbf{X})) \tag{14.8}$$

$$\mathbf{y}_c|\mathbf{f}_c \sim \mathcal{N}(\mathbf{f}_c, \tau^{-1}\mathbf{I}_{\mathrm{N}}), \tag{14.9}$$

with the $\mathbf{f}_c$'s mutually independent given the covariance function. In the following we write this set of C independent models compactly as $\mathbf{F}|\mathbf{X} \sim \mathcal{N}(0, K(\mathbf{X},\mathbf{X}))$ and $\mathbf{Y}|\mathbf{F} \sim \mathcal{N}(\mathbf{F}, \tau^{-1}\mathbf{I}_{\mathrm{N}})$, where the Gaussian is understood to act on each column separately. We do not need further details on GPs for our purpose.

At this point, a question remains unanswered: why do we need to introduce GPs at all? The reason is that GPs play a pivotal role in deriving our Bayesian approximation. Recall the ELBO derivation of Chapter 12, maximising the ELBO is equivalent to minimising the KL divergence between the real and the approximated distribution. For the dropout Bayesian approximation, we need to prove that the dropout objective is, in some sense, maximising the evidence lower bound. Starting from a GP on which we can derive the ELBO and using an appropriate covariance function, we can ultimately show that the ELBO objective is, indeed, equivalent to the dropout objective of Equation 14.2.

14.1.3 Dropout as a Bayesian approximation

Now that we have all the ingredients, we can discuss the details of the chapter's core: the use of dropout as a Bayesian approximation. The idea consists of three steps:

1. Modeling the structure of the MLP introduced in Section 14.1.1 using a Gaussian Process with convenient priors and covariance function;

2. Defining a suitable approximate distribution which simulates the effect of the dropout;

3. Finally, showing how maximising the ELBO leads to a natural equivalence with the dropout objective described in Equation 14.2.

Gaussian Process approximation of the MLP

We start by creating a Gaussian Process with a convenient covariance function; in particular, we would like to construct our GP in such a way that its approximation will be equivalent to optimising an MLP with dropout objective. Let's consider the following covariance function:

$$k_{p(\mathbf{w}_h),p(\mathrm{b}_h)}(\mathbf{x}_a,\mathbf{x}_b)=\int\int\sigma(\mathbf{w}_h^\top\mathbf{x}_a+\mathrm{b}_h)\sigma(\mathbf{w}_h^\top\mathbf{x}_b+\mathrm{b}_h)p(\mathbf{w}_h)p(\mathrm{b}_h)\,\mathrm{d}\mathbf{w}_h\,\mathrm{d}\mathrm{b}_h, \tag{14.10}$$

where, for convenience, we use the symbols $\mathbf{w}_h\in\mathbb{R}^{\mathrm{D}}$ and $\mathrm{b}_h\in\mathbb{R}$ to represent a generic weight vector and bias term.[4] Here we are denoting a specific covariance function (thus, a function which takes two inputs $\mathbf{x}_a$ and $\mathbf{x}_b$) but which also depends upon some probability distributions $p(\mathbf{w}_h)$ and $p(\mathrm{b}_h)$. Throughout this chapter we take standard normal priors, $p(\mathbf{w}_h)=\mathcal{N}(0,\mathbf{I}_{\mathrm{D}})$ and $p(\mathrm{b}_h)=\mathcal{N}(0,1)$; this choice will be needed later, when evaluating the KL divergence in closed form. By sampling from these distributions we can consider a MC approximation of the integral, which can shift the dependence upon the distribution toward the dependence upon Q realisations drawn from these distribution. If we extract Q samples $\mathbf{W}_h^{(q)}\sim p(\mathbf{w}_h)$ and $\mathrm{b}_h^{(q)}\sim p(\mathrm{b}_h)$ from the prior probabilities, and denote $\mathbf{W}_h=[\mathbf{W}_h^{(q)}]_{q=1}^{\mathrm{Q}}$ and $\mathbf{b}_h=[\mathrm{b}_h^{(q)}]_{q=1}^{\mathrm{Q}}$, we can approximate Equation 14.10 as:

$$k_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{x}_a,\mathbf{x}_b)=\frac{1}{\mathrm{Q}}\sum_{q=1}^{\mathrm{Q}}\sigma({\mathbf{W}_h^{(q)}}^\top\mathbf{x}_a+\mathrm{b}_h^{(q)})\sigma({\mathbf{W}_h^{(q)}}^\top\mathbf{x}_b+\mathrm{b}_h^{(q)}). \tag{14.11}$$

Then, we use $\mathbf{W}_h$ and $\mathbf{b}_h$ for deriving a graphical model analogously to Equation 14.5:

$$\mathbf{f}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h\sim\mathcal{N}(0,K_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{X},\mathbf{X})) \tag{14.12}$$

$$\mathbf{y}_c|\mathbf{f}_c\sim\mathcal{N}(\mathbf{f}_c,\tau^{-1}\mathbf{I}_{\mathrm{N}}), \tag{14.13}$$

where $K_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{X},\mathbf{X})$, as in Equation 14.5, denotes an $\mathrm{N}\times\mathrm{N}$ covariance matrix derived from applying the function $k_{\mathbf{W}_h,\mathbf{b}_h}$ to each input pair in $(\mathbf{X},\mathbf{X})$. Having placed an independent GP on each output component, conditioning on $\mathbf{W}_h$ and $\mathbf{b}_h$ makes the columns of $\mathbf{Y}$ conditionally independent, so that $p(\mathbf{Y}|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)$ factorises over the components $\mathbf{y}_c$; this gives us the density of the whole matrix $\mathbf{Y}$, from which we can derive $p(\mathbf{Y}|\mathbf{X})$ from marginalising $\mathbf{W}_h$ and $\mathbf{b}_h$:

$$p(\mathbf{Y}|\mathbf{X})=\int\int p(\mathbf{Y}|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)p(\mathbf{W}_h)p(\mathbf{b}_h)\,\mathrm{d}\mathbf{W}_h\,\mathrm{d}\mathbf{b}_h \tag{14.14}$$

$$=\int\int\prod_{c=1}^{\mathrm{C}}p(\mathbf{y}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)p(\mathbf{W}_h)p(\mathbf{b}_h)\,\mathrm{d}\mathbf{W}_h\,\mathrm{d}\mathbf{b}_h. \tag{14.15}$$

Then, the density function $p(\mathbf{y}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)$ can be obtained from marginalising $\mathbf{f}_c$, which allow us to expand $p(\mathbf{Y}|\mathbf{X})$ as:

$$p(\mathbf{Y}|\mathbf{X})=\int\int\left(\int\prod_{c=1}^{\mathrm{C}}p(\mathbf{y}_c|\mathbf{f}_c,\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)p(\mathbf{f}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)\,\mathrm{d}\mathbf{f}_c\right)p(\mathbf{W}_h)p(\mathbf{b}_h)\,\mathrm{d}\mathbf{W}_h\,\mathrm{d}\mathbf{b}_h \tag{14.16}$$

$$=\int\int\left(\prod_{c=1}^{\mathrm{C}}\int p(\mathbf{y}_c|\mathbf{f}_c)p(\mathbf{f}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)\mathrm{d}\mathbf{f}_c\right)p(\mathbf{W}_h)p(\mathbf{b}_h)\,\mathrm{d}\mathbf{W}_h\,\mathrm{d}\mathbf{b}_h. \tag{14.17}$$

We can then compute each term $p(\mathbf{y}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)=\int p(\mathbf{y}_c|\mathbf{f}_c)p(\mathbf{f}_c|\mathbf{X},\mathbf{W}_h,\mathbf{b}_h)\,\mathrm{d}\mathbf{f}_c$ analytically using Equation 14.12, as $\mathcal{N}(0,K_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{X},\mathbf{X})+\tau^{-1}\mathbf{I}_{\mathrm{N}})$, and rewrite $p(\mathbf{Y}|\mathbf{X})$ as:

$$p(\mathbf{Y}|\mathbf{X})=\int\int\prod_{c=1}^{\mathrm{C}}\mathcal{N}(\mathbf{y}_c;0,K_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{X},\mathbf{X})+\tau^{-1}\mathbf{I}_{\mathrm{N}})p(\mathbf{W}_h)p(\mathbf{b}_h)\,\mathrm{d}\mathbf{W}_h\,\mathrm{d}\mathbf{b}_h. \tag{14.18}$$

Our graphical model encodes now the parameters $\mathbf{W}_h$ and $\mathbf{b}_h$; to bring this formulation closer to the one used for a multi-layer perceptron, we need to introduce auxiliary variables. We can factorise the covariance matrix as

[4] Reading this chapter after Chapter 24 may rise the question: is Equation 14.10 a *valid* covariance function? The answer is yes, but for the sake of simplicity, we omit the proof. In this regard, we suggest to the more curious among the readers to delve deeper into the discussion around Marginalised Kernels [1639].

an outer product $\Psi\Psi^\top$, where $\Psi = [\psi(\mathbf{x}^{(n)}, \mathbf{W}_h, \mathbf{b}_h)^\top]_{n=1}^{\mathrm{N}} \in \mathbb{R}^{\mathrm{N}\times\mathrm{Q}}$ is the matrix built from the Q-dimensional vectors $\psi(\mathbf{x}, \mathbf{W}_h, \mathbf{b}_h) = \sqrt{\frac{1}{\mathrm{Q}}}\sigma(\mathbf{W}_h^\top \mathbf{x} + \mathbf{b}_h)$, so that $K_{\mathbf{W}_h,\mathbf{b}_h}(\mathbf{X}, \mathbf{X}) = \Psi\Psi^\top$. Replacing this in Equation 14.18 yields:

$$p(\mathbf{Y}|\mathbf{X}) = \int\int \prod_{c=1}^{\mathrm{C}} \mathcal{N}(\mathbf{y}_c; 0, \Psi\Psi^\top + \tau^{-1}\mathbf{I}_{\mathrm{N}}) p(\mathbf{W}_h) p(\mathbf{b}_h)\, \mathrm{d}\mathbf{W}_h\, \mathrm{d}\mathbf{b}_h. \tag{14.19}$$

For each element on this product, we introduce a $\mathrm{Q}\times 1$ variable $\mathbf{w}_c \sim \mathcal{N}(0, \mathbf{I}_{\mathrm{Q}})$:

$$\mathcal{N}(\mathbf{y}_c; 0, \Psi\Psi^\top + \tau^{-1}\mathbf{I}_{\mathrm{N}}) = \int \mathcal{N}(\mathbf{y}_c; \Psi\mathbf{w}_c, \tau^{-1}\mathbf{I}_{\mathrm{N}}) \mathcal{N}(\mathbf{w}_c; 0, \mathbf{I}_{\mathrm{Q}}) \mathrm{d}\mathbf{w}_c. \tag{14.20}$$

Note that now the mean of the independent $\mathbf{y}_c$'s has the form $\Psi\mathbf{w}_c$, with $\psi(\mathbf{x}^{(n)}, \mathbf{W}_h, \mathbf{b}_h)^\top \mathbf{w}_c$ for the n-th sample; by collecting the auxiliary variables in matrix notation as $\mathbf{W}_l = [\mathbf{w}_c]_{c=1}^{\mathrm{C}}$ we recover the mean prediction $\mathbf{Y} = \Psi\mathbf{W}_l$, which is now more similar to the original MLP formulation. We can then write $p(\mathbf{Y}|\mathbf{X})$ as a marginalisation over $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$:

$$p(\mathbf{Y}|\mathbf{X}) = \int\int\int p(\mathbf{Y}|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) p(\mathbf{W}_h) p(\mathbf{W}_l) p(\mathbf{b}_h)\, \mathrm{d}\mathbf{W}_h\, \mathrm{d}\mathbf{W}_l\, \mathrm{d}\mathbf{b}_h. \tag{14.21}$$

Approximated distribution prior

Once having defined the GP we need to construct the approximated distribution for our model parameters. The idea should be to find a suitable distribution $q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)$ which can minimise the KL divergence with the true distribution $p(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)$. Since we assumed that the parameters $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$ are mutually independent we can derive the approximated distribution $q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)$ as a product of each individual approximated component:

$$q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) = q(\mathbf{W}_h) q(\mathbf{W}_l) q(\mathbf{b}_h). \tag{14.22}$$

Now, remember the main goal of this whole section: we are not seeking a generic approximation, but a *dropout*-based one. In a sense, we would like to model our independent $q(\mathbf{W}_h)$, $q(\mathbf{W}_l)$, and $q(\mathbf{b}_h)$ as the product of a stochastic process involving dropout. But how does the effect of dropout impacts the parameters' probability distribution? Dropping an input neuron $\mathbf{x}_d^{(n)}$ is equivalent to setting to zero all the weights connecting that neuron to the hidden layer. Hence, if we set the same dropout rate ϕ for all the neurons on a given layer, we approximate the individual $\mathbf{W}_h^{(i)}$ and $\mathbf{W}_l^{(i)}$ using a mixture of Gaussians with shared covariance and individual mean vectors, and a simple Gaussian for the biases $\mathrm{b}_h^{(i)}$:

$$q(\mathbf{W}_h^{(i)}) = (1-\phi)\, \mathcal{N}(\mathbf{M}_h^{(i)}, \sigma^2\mathbf{I}_{\mathrm{Q}}) + \phi\, \mathcal{N}(0, \sigma^2\mathbf{I}_{\mathrm{Q}}), \tag{14.23}$$

$$q(\mathbf{W}_l^{(i)}) = (1-\phi)\, \mathcal{N}(\mathbf{M}_l^{(i)}, \sigma^2\mathbf{I}_{\mathrm{C}}) + \phi\, \mathcal{N}(0, \sigma^2\mathbf{I}_{\mathrm{C}}), \tag{14.24}$$

$$q(\mathbf{b}_h) = \mathcal{N}(\mathbf{m}_h, \sigma^2\mathbf{I}_{\mathrm{Q}}). \tag{14.25}$$

The matrices $\mathbf{M}_h = [\mathbf{M}_h^{(i)}]_{i=1}^{\mathrm{D}}$ and $\mathbf{M}_l = [\mathbf{M}_l^{(i)}]_{i=1}^{\mathrm{Q}}$, in a sense, encode the network weights in absence of dropout: the probabilistic weights $\mathbf{W}_h$ and $\mathbf{W}_l$ are thus obtained by drawing from a mixture having $1-\phi$ probability of returning the "actual weight", and ϕ probability of zeroing them out.

Starting from Equation 14.23 we can finally reparametrise the probabilities over $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$ with respect to Bernoulli random variables, precisely as it happens by using dropout. With the same approach of the reparametrisation trick used for AutoEncoders [892], we can reduce the stochasticity to simple events (noise generation and dropout) and compose the probability on the parameters directly with respect to them. First, we need to generate the Bernoulli random variables. We can generate a vector $\boldsymbol{o}_h \in \{0,1\}^{\mathrm{D}}$ where each element $\boldsymbol{o}_h^{(i)} \sim \mathcal{B}(1-\phi)$ follows the Bernoulli distribution with probability $1-\phi$; then, it is sufficient to multiply this random vector by the mean matrix $\mathrm{diag}(\boldsymbol{o}_h)\cdot\mathbf{M}_h$ for zeroing the rows corresponding to the dropped units. An analogous approach can be implemented for modelling dropout on the hidden layer, by introducing a vector $\boldsymbol{o}_l \in \{0,1\}^{\mathrm{Q}}$. Secondly, we generate the noise in Equation 14.23 by introducing the support random variables $\boldsymbol{R}_h \sim \mathcal{N}(0, \mathbf{I}_{\mathrm{D}\times\mathrm{Q}})$, $\boldsymbol{R}_l \sim \mathcal{N}(0, \mathbf{I}_{\mathrm{Q}\times\mathrm{C}})$ and $\mathrm{r}_h \sim \mathcal{N}(0, \mathbf{I}_{\mathrm{Q}})$ for computing the additive noise. By combining these two steps we can derive $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$ as a function of $\boldsymbol{o}_h$, $\boldsymbol{o}_l$, $\boldsymbol{R}_h$, $\boldsymbol{R}_l$ and r_h:

$$\mathbf{W}_h(\boldsymbol{o}_h, \boldsymbol{R}_h) = \mathrm{diag}(\boldsymbol{o}_h)\cdot(\mathbf{M}_h + \sigma\boldsymbol{R}_h) + \mathrm{diag}(1-\boldsymbol{o}_h)\cdot\sigma\boldsymbol{R}_h \tag{14.26}$$

$$\mathbf{W}_l(\boldsymbol{o}_l, \boldsymbol{R}_l) = \mathrm{diag}(\boldsymbol{o}_l)\cdot(\mathbf{M}_l + \sigma\boldsymbol{R}_l) + \mathrm{diag}(1-\boldsymbol{o}_l)\cdot\sigma\boldsymbol{R}_l \tag{14.27}$$

$$\mathbf{b}_h(\mathrm{r}_h) = \mathbf{m}_h + \sigma\mathrm{r}_h. \tag{14.28}$$

Note that the sampled weights still carry the Gaussian perturbation $\sigma \boldsymbol{R}_h$ and $\sigma \boldsymbol{R}_l$: the equivalence with dropout is recovered in the limit $\sigma \to 0$, for which $\mathbf{W}_h(\boldsymbol{o}_h, \boldsymbol{R}_h) \to \text{diag}(\boldsymbol{o}_h) \cdot \mathbf{M}_h$ and $\mathbf{W}_l(\boldsymbol{o}_l, \boldsymbol{R}_l) \to \text{diag}(\boldsymbol{o}_l) \cdot \mathbf{M}_l$, i.e. exactly the masked weights of Section 14.1.1.

We can finally exploit these new reparametrisations for evaluating the evidence lower bound.

Evaluating the ELBO

The notation introduced in Equation 14.26 – thus, separating the random variables from the actual network parameters – is fundamental for facilitating the computation of the ELBO. Recalling again the ELBO derivation of Chapter 12 we obtain:

$$\mathcal{L}_{GP-VI} \coloneqq \int\int\int q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \log p(\mathbf{Y}|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \, \mathrm{d}\mathbf{W}_h \, \mathrm{d}\mathbf{W}_l \, \mathrm{d}\mathbf{b}_h \\ - \text{KL}(q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) || p(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)). \tag{14.29}$$

Let us begin by computing the $\log p(\mathbf{Y}|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)$ term; for doing so, we can exploit the independence between the $\mathbf{y}_c$'s:

$$\log p(\mathbf{Y}|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) = \log \prod_{c=1}^{\text{C}} p(\mathbf{y}_c|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \tag{14.30}$$

$$= \sum_{c=1}^{\text{C}} \log p(\mathbf{y}_c|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \tag{14.31}$$

$$= \sum_{c=1}^{\text{C}} \log \mathcal{N}(\mathbf{y}_c; \Psi \mathbf{W}_l^{(c)}, \tau^{-1} \mathbf{I}_{\text{N}}) \tag{14.32}$$

$$= -\frac{\text{NC}}{2} \log(2\pi) + \frac{\text{NC}}{2} \log(\tau) - \sum_{c=1}^{\text{C}} \frac{\tau}{2} ||\mathbf{y}_c - \Psi \mathbf{W}_l^{(c)}||_2^2. \tag{14.33}$$

Now, let $\hat{\mathbf{Y}} = \Psi \mathbf{W}_l$, with $\hat{\mathbf{y}}^{(n)} = \psi(\mathbf{x}^{(n)}, \mathbf{W}_h, \mathbf{b}_h)^\top \mathbf{W}_l$: by doing so, we obtain a prediction matrix $\hat{\mathbf{Y}} \in \mathbb{R}^{\text{N} \times \text{C}}$, and can thus sum over the first dimension of this matrix:

$$\sum_{c=1}^{\text{C}} ||\mathbf{y}_c - \hat{\mathbf{y}}_c||_2^2 = \sum_{n=1}^{N} ||\mathbf{y}_n - \hat{\mathbf{y}}_n||_2^2. \tag{14.34}$$

We can thus rewrite the first term in the integral of the $\mathcal{L}_{GP-VI}$ objective as a sum over the samples:

$$q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \log p(\mathbf{Y}|\mathbf{X}, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) = \tag{14.35}$$

$$= \sum_{n=1}^{\text{N}} q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \log p(\mathbf{y}_n|\mathbf{x}_n, \mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) \tag{14.36}$$

$$= \sum_{n=1}^{\text{N}} q(\boldsymbol{o}_h, \boldsymbol{R}_h, \boldsymbol{o}_l, \boldsymbol{R}_l, \text{r}_h) \log p(\mathbf{y}_n|\mathbf{x}_n, \mathbf{W}_h(\boldsymbol{o}_h, \boldsymbol{R}_h), \mathbf{W}_l(\boldsymbol{o}_l, \boldsymbol{R}_l), \mathbf{b}_h(\text{r}_h)). \tag{14.37}$$

Now, the integral in Equation 14.29 can be approximated using MC integration. Specifically, instead of integrating over $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$ we now sum over a set of realisations $\tilde{\mathbf{W}}_h$, $\tilde{\mathbf{W}}_l$ and $\tilde{\text{b}}_h$.

$$\mathcal{L}_{GP-MC} \coloneqq \sum_{n=1}^{N} \log p(\mathbf{y}^{(n)}|\mathbf{x}^{(n)}, \tilde{\mathbf{W}}_h^{(n)}, \tilde{\mathbf{W}}_l^{(n)}, \tilde{\text{b}}_h^{(n)}) - \text{KL}(q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) || p(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)). \tag{14.38}$$

Lastly, we need to compute the KL divergence between the real and the approximated distribution. The main obstacle here is represented by the absence of an analytic solution that solves our problem, since we would compare a mixture of Gaussians against a single Gaussian distribution. Since both q and p factorise over $\mathbf{W}_h$, $\mathbf{W}_l$ and $\mathbf{b}_h$, however, the divergence splits *exactly* into the sum of three independent terms:

$$\text{KL}(q(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h) || p(\mathbf{W}_h, \mathbf{W}_l, \mathbf{b}_h)) \\ = \text{KL}(q(\mathbf{W}_h) || p(\mathbf{W}_h)) + \text{KL}(q(\mathbf{W}_l) || p(\mathbf{W}_l)) + \text{KL}(q(\mathbf{b}_h) || p(\mathbf{b}_h)), \tag{14.39}$$

so that the only approximation needed concerns the two mixture terms[5]. For a large enough Q we can approximate the KL on $\mathbf{W}_h$ and $\mathbf{W}_l$ as:

$$\mathrm{KL}(q(\mathbf{W}_h)||p(\mathbf{W}_h)) \approx \frac{\mathrm{DQ}}{2}(\sigma^2 - \log(\sigma^2) - 1) + \frac{1-\phi}{2}\sum_{d=1}^{D} \mathbf{M}_h^{(d)\top}\mathbf{M}_h^{(d)} + \text{const} \tag{14.40}$$

$$\mathrm{KL}(q(\mathbf{W}_l)||p(\mathbf{W}_l)) \approx \frac{\mathrm{QC}}{2}(\sigma^2 - \log(\sigma^2) - 1) + \frac{1-\phi}{2}\sum_{q=1}^{Q} \mathbf{M}_l^{(q)\top}\mathbf{M}_l^{(q)} + \text{const.} \tag{14.41}$$

The bias term can be evaluated analytically as:

$$\mathrm{KL}(q(\mathbf{b}_h)||p(\mathbf{b}_h)) = \frac{1}{2}(\mathbf{m}_h{}^\top\mathbf{m}_h + \mathrm{Q}(\sigma^2 - \log(\sigma^2) - 1)) + \text{const} \tag{14.42}$$

By combining the individual KL terms for the weights and biases terms, ignoring σ (which do not affect the optimisation over $\mathbf{M}_h$, $\mathbf{M}_l$, and $\mathbf{m}_h$) and rescaling by $\frac{1}{\mathrm{N}\tau}$, we finally obtain the MC Bayesian approximation as:

$$\mathcal{L}_{GP-MC} \propto -\frac{\tau}{2}\sum_{n=1}^{\mathrm{N}} ||\mathbf{y}_n - \hat{\mathbf{y}}_n||_2^2 - \frac{1-\phi}{2}(||\mathbf{M}_h||_2^2 + ||\mathbf{M}_l||_2^2) - \frac{1}{2}||\mathbf{m}_h||_2^2 \tag{14.43}$$

$$\propto -\frac{1}{2\mathrm{N}}\sum_{n=1}^{\mathrm{N}} ||\mathbf{y}_n - \hat{\mathbf{y}}_n||_2^2 - \frac{1-\phi}{2\mathrm{N}\tau}(||\mathbf{M}_h||_2^2 + ||\mathbf{M}_l||_2^2) - \frac{1}{2N\tau}||\mathbf{m}_h||_2^2. \tag{14.44}$$

Since maximising $\mathcal{L}_{GP-MC}$ is equivalent to minimising $-\mathcal{L}_{GP-MC}$, the expression above recovers the dropout objective in Equation 14.3 for the choice $\lambda_{\mathbf{W}_h} = \lambda_{\mathbf{W}_l} = \frac{1-\phi}{2\mathrm{N}\tau}$ and $\lambda_{\mathbf{b}_h} = \frac{1}{2\mathrm{N}\tau}$. Note that this correspondence fixes the model precision τ once the weight decay has been chosen. This shows that using a dropout objective along with a proper regularisation is, indeed, a proper Bayesian approximation.

14.2 Concrete Dropout: dropout rate as a variational parameter

The derivation above tells us that dropout *is* a Bayesian approximation, but it says nothing about how the "optimal" value of the dropout rate ϕ should be chosen. A related question, for a neural network with several layers, is whether *different* rates should be used across layers (or even, at the extreme, across individual neurons). An apparently reasonable approach is a grid-search procedure over a given set of candidate dropout rates, to estimate the best value for each layer. This procedure is however an unfeasible solution: it is prohibitively expensive for large models, also taking into account that the number of per-layer dropout configurations grows exponentially with the number of layers (let alone the combinatorial blow-up of a per-neuron configuration!). Moreover, it becomes altogether impractical in settings where the amount of available data changes over time, such as in a reinforcement learning or lifelong learning, since the optimal rate would have to be re-tuned as training progresses.

An alternative and more feasible approach is to *learn* the dropout rate itself, alongside the network weights, using gradient descent, by minimising a suitable objective function. To this aim, the variational interpretation of dropout, which takes the name of **Concrete Dropout** [568], provides an elegant solution: since the dropout rate parametrises the approximating distribution of Equation 14.23, it is a variational parameter on exactly the same footing as $\mathbf{M}_h$, $\mathbf{M}_l$ and $\mathbf{m}_h$, and the ELBO itself is the objective to be optimised with respect to it. Writing it for a generic L-layer network with per-layer rates ϕ_l and mean weight matrices $\mathbf{M}_l$, and normalising by N, we obtain

$$\widehat{\mathcal{L}} = -\frac{1}{\mathrm{N}}\sum_{n=1}^{\mathrm{N}} \log p(\mathbf{y}^{(n)}|\mathbf{x}^{(n)}, \hat{\boldsymbol{\omega}}_n) + \frac{1}{\mathrm{N}}\sum_{l=1}^{L} \mathrm{KL}\left(q_{\phi_l}(\mathbf{W}_l) \,\|\, p(\mathbf{W}_l)\right) \ , \tag{14.45}$$

where $\hat{\boldsymbol{\omega}}_n \sim q$ denotes a single set of weights sampled through the dropout masks, and the per-layer divergence takes the form

$$\mathrm{KL}\left(q_{\phi_l}(\mathbf{W}_l) \,\|\, p(\mathbf{W}_l)\right) \propto \frac{1-\phi_l}{2}\|\mathbf{M}_l\|_2^2 - \mathrm{K}_l\,\mathcal{H}(\phi_l) \ , \tag{14.46}$$

with K_l the number of units of layer l and

$$\mathcal{H}(\phi_l) = -\phi_l \log \phi_l - (1-\phi_l)\log(1-\phi_l) \tag{14.47}$$

[5]At this stage, one could legitimately ask: yet another approximation? Unfortunately, yes. Since going into this level of detail would weigh the argument down, we do not discuss these approximations here. For a more comprehensive treatment, please refer to [566].

the entropy of a Bernoulli random variable of rate ϕ_l. The dropout rate therefore ceases to be a hyperparameter to be searched for, and becomes the outcome of the optimisation itself. This, however, is only possible if the gradient of Equation 14.45 with respect to ϕ_l can actually be computed, which is not the case for the discrete masks used so far.

The key idea for overcoming this issue is to reparametrise the dropout mask itself, so that one can legitimately backpropagate through the "randomised" neurons (that is, through the very units switched off by the dropout layer). Such reparametrisation tricks, which also lay the groundwork for techniques like Variational Autoencoders (VAEs) [892], will be treated in detail in later chapters; here we limit ourselves to the Gumbel-softmax trick [831], which is precisely what allows the dropout rate to be treated as an actual variational parameter during optimisation. Since in the case of dropout we have to optimise the parameters of a *discrete* Bernoulli distribution, the original reparametrisation trick cannot be applied directly: sampling a binary mask is not differentiable with respect to the dropout rate ϕ. A continuous relaxation is therefore required, such as the Concrete distribution [1115], originally proposed for latent random variables in deep generative models. The Concrete distribution replaces the arg max appearing in the Gumbel-max reparametrisation of a discrete variable with a temperature-controlled softmax, thus yielding a continuous, differentiable random variable which concentrates on the vertices of the simplex as the temperature $t \to 0$. This allows a discrete, and thus non-differentiable, random variable $\mathbf{z}$ to be rewritten as $\tilde{\mathbf{z}} = g(\phi, \varepsilon)$ where the whole stochasticity is carried by a random variable ε which does not depend on ϕ, and g is deterministic and differentiable in ϕ. Such a factorisation is precisely what is needed in our case: the dropout masks make the units of the network random variables, and applying the above trick to them allows the gradient to be back-propagated onto the variational parameter ϕ that we are trying to optimise. In the binary case of interest here, the softmax reduces to the sigmoid $\sigma(\cdot)$, and the relaxed mask reads

$$\tilde{\mathbf{z}} = \sigma\left(\frac{1}{t}(\log\phi - \log(1-\phi) + \log u - \log(1-u))\right) , \tag{14.48}$$

where $u \sim \mathcal{U}[0,1]$ is a uniformly distributed variable which does not depend on the parameter ϕ. Gradients therefore flow only through the terms involving ϕ, whereas u acts as an external source of noise, making now $\tilde{\mathbf{z}}$ differentiable with respect to ϕ.

14.3 Dropout injection

The original MC Dropout formulation envisages activating dropout at inference time on a neural network which uses dropout for regularisation during training. If, on the one hand, this technique yields a solution with minimal implementation overhead, on the other hand, its integration within already trained neural networks would require retraining the model. This is often infeasible, either because the original training pipeline or data is no longer available, or because retraining risks degrading the performance of a deployed model that has already undergone a validation process. As a solution to this issue, recent work shows that such a strategy can be applied to arbitrary NNs, regardless of whether or not dropout was used during training; this technique is known as **Dropout Injection** [996]. Such a strategy avoids the costs and pitfalls of retraining altogether, retrofitting uncertainty estimation onto an existing deterministic network without altering its weights or requiring further optimisation. The first solution proposed in the literature stems from a simple idea, i.e., finding a suitable dropout rate ϕ *after training* by minimising the Negative Log-Likelihood (NLL) [1076]. Albeit intuitive in practice, injecting dropout into an already trained neural network hides subtle implications which have not been addressed in the original formulation. Specifically, a noteworthy drawback comes to light when using this strategy for regression problems.

14.3.1 Dropout injection: the regression case

In a regression problem, the predictor $f(\mathbf{x}, \theta)$ with parameters θ takes as input a D-dimensional $\mathbf{x}$ vector and outputs a real number $\mathbf{y}$. Consider now a Monte Carlo Variational Inference (MC-VI) regressor $f_{\text{MC-VI}} : \mathbb{R}^D \to \mathbb{R}^M$ which takes as input a D-dimensional vector and outputs, instead, M MC predictions by sampling from a parameters' distribution $q(\theta|\phi)$ approximated using a dropout rate ϕ. Following Loquercio et al., the main challenge of dropout injection would consist in finding the most suitable dropout rate ϕ which minimises the NLL:

$$\phi = \arg\min_{\phi'} \frac{1}{N}\sum_{n=1}^{N} \frac{1}{2}\frac{(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi'))^2}{\sigma^{2^{(n)}}(\phi')} + \frac{1}{2}\log(\sigma^{2^{(n)}}(\phi')), \tag{14.49}$$

where $\hat{\mathbf{y}}^{(n)}(\phi')$ and $\sigma^{2^{(n)}}(\phi')$ are the empirical mean and variance of the prediction computed using the MC-VI regressor; it is worth noting that both of them depend upon the dropout rate ϕ'. Practically, too high ϕ' values

will, at some point, lead to a severe degradation of the performances, since the Bayesian model average (BMA) $\frac{1}{M}\sum_{m=1}^{M} f(\mathbf{x}, \theta_m)$ would deviate too much from the prediction of the original, deterministic network $f(\mathbf{x}, \theta_{\text{MAP}})$. Hence, the range of suitable ϕ values shrinks to the interval $[0, \phi_{\text{max}}]$, where ϕ_{max} represents the dropout rate for which the performance degradation is tolerable.

In this situation, however, a subtle complication may eventually occur. In principle, one can minimise the NLL either by minimising the squared error or by making the uncertainty measure equal to the squared error for each sample. Hence, the problem arises when for the maximum value ϕ_{max} the variances captured by $\sigma^{2^{(n)}}(\phi)$ are too small to accurately reflect the squared errors $(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi))^2$. In the context of dropout injection, where it is not possible to tune the predictions, the NLL minimisation serves only to identify the value of ϕ which minimises the distance between the squared error and the empirical variance $|\sigma^{2^{(n)}}(\phi) - (\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi))^2|$. But what happens if the range $[0, \phi_{\text{max}}]$ is so small that it will always produce variances significantly lower than the corresponding squared errors? In such a situation, even though the uncertainty measure cannot accurately reflect the errors, we can approach the problem from a different angle: if the difficulties arise from the insufficient magnitude of the uncertainty measure, its rescaling provides a practical solution. By doing so, we can rewrite Equation 14.49 as a two-variable minimisation problem, where we search for the optimal ϕ and scale factor S jointly:

$$\phi, S = \arg\min_{\phi', s} \frac{1}{N}\sum_{n=1}^{N} \frac{1}{2}\frac{(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi'))^2}{s \cdot \sigma^{2^{(n)}}(\phi')} + \frac{1}{2}\log(s \cdot \sigma^{2^{(n)}}(\phi')), \tag{14.50}$$

Notably, one can easily demonstrate that it is possible to compute the optimal scale factor S over a set of data, given ϕ, in closed form. Specifically and originally proposed for calibrating regressors, the solution to this problem takes the name of σ-scaling [981]. The optimal scale factor $s(\phi) = \frac{1}{N}\sum_{n=1}^{N}[(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi))^2 \cdot \sigma_n^{-2}(\phi)]$ can be computed as the expectation of the squared error divided by the variance. The proof is left to the reader as an exercise. By doing so, the problem of Equation 14.50 can be formulated as a simpler optimisation problem:

$$\phi = \arg\min_{\phi'} \frac{1}{N}\sum_{n=1}^{N} \frac{1}{2}\frac{(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)}(\phi'))^2}{\sigma^{2^{(n)}}(\phi') \cdot s(\phi')} + \frac{1}{2}\log(\sigma^{2^{(n)}}(\phi') \cdot s(\phi')). \tag{14.51}$$

Optimising in this way means the search is no longer confined to a fixed one-dimensional curve, but ranges freely across the two-dimensional surface in which that optimal scale curve $s(\phi)$ is embedded, as illustrated in Figure 14.1. In the scale-agnostic case the search is confined to the slice $s = 1$, and the star marks the best (sub-optimal) NLL attainable there; rescaling the uncertainty measure instead grants access to regions of the surface that would otherwise remain unreachable, yielding a different dropout rate with a lower NLL.

14.3.2 Dropout Injection for classification

When discussing Dropout Injection for regression problems, we revealed the existence of an issue related to the scale of the approximated distribution. Interestingly, the same does not hold for classification problems, as it becomes evident when modelling the same conditions for classifiers. For regression problems, the function $f(\mathbf{x}, \theta)$ outputs a scalar $\hat{\mathbf{y}} \in \mathbb{R}$: Here, the output can be modelled by placing a Gaussian on the predictive distribution $f(x, \theta) \sim \mathcal{N}(\mu, \sigma^2)$. For the classification case, instead, the function $f(\mathbf{x}, \theta)$ returns a probability vector $p = \hat{\mathbf{y}} = \text{softmax}(z)$, with $z \in \mathbb{R}^C$: in this case, since $\hat{\mathbf{y}}$ represent a vector on the C-dimensional simplex, the Dirchlet distribution is a convenient choice for modelling the posterior $\hat{\mathbf{y}} \sim \text{Dir}(\alpha)$.

The key architectural difference between classification and regression models is the use of a softmax activation in the final layer for classification. This enables moment matching to be carried out directly on the logits z, which can then be used to estimate the parameters of the Dirichlet distribution. The main challenge in this approach lies in identifying an appropriate distribution over the logits, such that the softmax-transformed outputs follow a Dirichlet distribution. Thankfully, known results on Bayesian statistics tell us that the softmax of independent gamma variables follows a Dirichlet distribution [420]. We can exploit this result by modelling the exponential of each logit using independent Gamma distributions:

$$e^{z_i} \sim \Gamma(\alpha_i, \frac{1}{\lambda_i}),\ \forall i \in [1, C] \implies \text{softmax}(z) \sim \text{Dir}(\alpha). \tag{14.52}$$

We can now perform the moment matching operation directly on the independent Gamma distributions, estimating, for each of them, an empirical mean $\hat{\mu}_i$ and variance $\hat{\sigma}_i^2$ for deriving the shape α_i and scale $\frac{1}{\lambda_i}$ parameters:

$$\begin{bmatrix}\alpha_i\lambda_i^{-1} \\ \alpha_i\lambda_i^{-2}\end{bmatrix} = \begin{bmatrix}\hat{\mu}_i \\ \hat{\sigma}_i^2\end{bmatrix} \implies \alpha_i = \hat{\mu}_i^2\hat{\sigma}_i^{-2},\ \frac{1}{\lambda_i} = \hat{\sigma}_i^2\hat{\mu}_i^{-1} \tag{14.53}$$

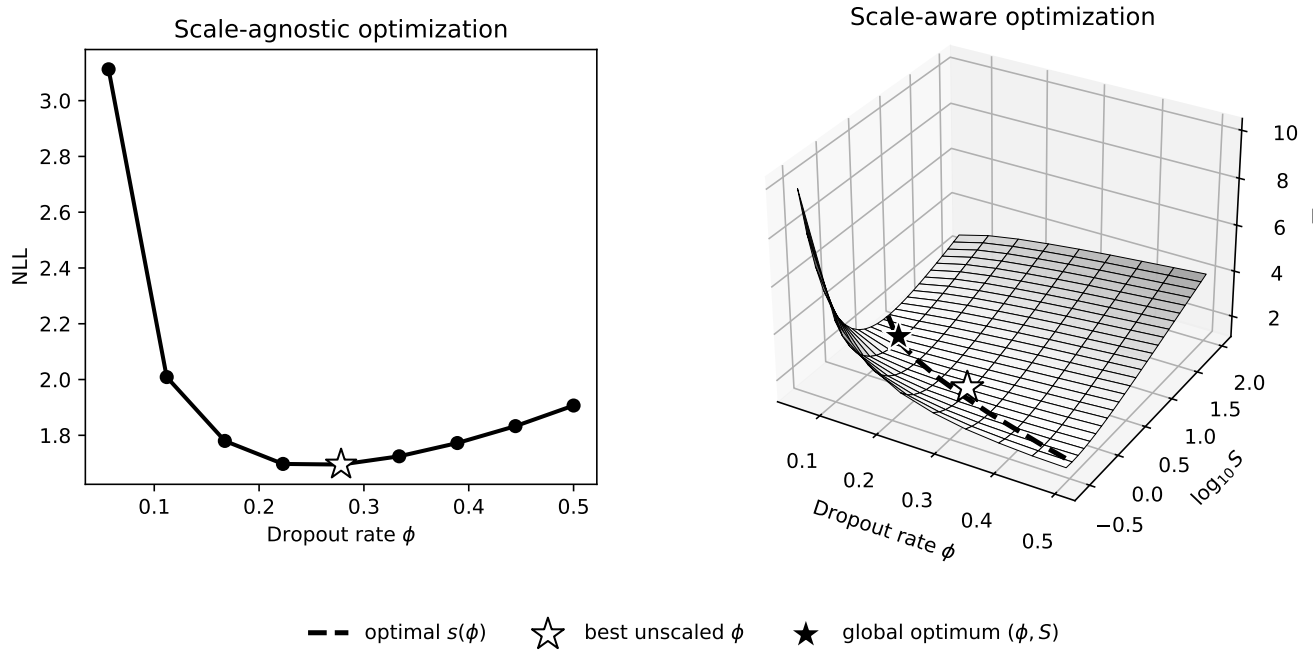


Figure 14.1: Comparison between the scale-agnostic and the scale-aware formulations of the injected-dropout optimisation problem, for an MLP on the *Boston Housing* data set. **Left:** the NLL of Equation 14.49 as a function of the dropout rate ϕ alone. **Right:** the NLL surface of Equation 14.50, considered as a function of both ϕ and the scale factor s; the dashed curve traces the optimal scale factor $s(\phi)$ of Equation 14.51, the white star reproduces the scale-agnostic solution and the black one the global optimum (ϕ, S).

We can then derive the parameters of the resulting Dirichlet on the probability vectors:

$$p \sim \mathrm{Dir}(\hat{\mu}^2 \circ \hat{\sigma}^{-2}). \tag{14.54}$$

The consequence of scaling the empirical variance in this setup is straightforward. The exponential logits would now follow a Gamma distribution with scaled shape and rate parameters, changing the Dirichlet parameters on the output probability vector accordingly:

$$\Gamma(\hat{\mu}_i^2(s^{-1} \cdot \hat{\sigma}_i^{-2}), s \cdot \hat{\sigma}_i^2 \hat{\mu}_i^{-1}) \implies \mathrm{Dir}(\hat{\mu}^2 \circ (s^{-1} \cdot \hat{\sigma}^{-2})). \tag{14.55}$$

Since the Dirichlet mean is preserved when scaling, so does the NLL. Hence, when dealing with classification, scaling the empirical variance on the logits do not affect the NLL.

14.4 Concluding remarks

In this chapter we saw that a technique devised for regularisation can be reinterpreted, under the right assumptions, as approximate Bayesian inference. It is precisely this simplicity that makes the method usable at scale: indeed, dropout-based inference does not require any architectural or optimisation change, and the entire cost of estimating the predictive uncertainty is deferred to the inference phase. On top of this, the possibility of applying the technique to networks that have *already* been trained opens the door to a wide range of practical scenarios, making dropout-based inference a quick solution for experts and practitioners alike. Nevertheless we have to keep in mind that the multiple forward passes required at inference time represent a non-negligible obstacle, particularly as model size increases, where the computation of a single prediction can no longer be taken for granted.

To close the chapter, it is worth drawing attention to a distinctive feature of dropout-based inference: unlike more explicit variational treatments, it introduces no variational parameters beyond those a deterministic network already possesses (with the exception of Concrete Dropout), and attaches no distribution to individual weights or neurons. From the next chapter until the end of this part, we shall see that this is the exception rather than the rule: the methods that follow make the variational object explicit – defined over weights, over functions, or through a set of particles – and consequently require additional quantities to be specified, optimised, or propagated.

Chapter 15

Natural-gradient-based variational inference

Variational inference can make the optimization of large Bayesian neural networks tractable by restricting optimization to a specific class of distributions. Here, we discuss natural-gradient-based methods, where gradients are rescaled according to second-order information in order to adjust for the geometry when optimizing in terms of distributions. Using the frameworks of Generalized Bayes and the Bayesian Learning Rule, we show that standard neural network optimizers can be derived as instances of natural-gradient descent given specific families of distributions. Finally, we derive practical natural-gradient-based optimization algorithms that closely resemble these standard neural network optimizers and can be used to train large neural networks effectively with arbitrary losses.

15.1 Natural-Gradient Descent

As introduced in Chapter 12, variational inference approximates a posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ over neural network parameters $\boldsymbol{\theta} \in \mathbb{R}^P$ given data $\mathcal{D} = \{(x_t, y_t)\}_{t=1..N}$ by minimizing a regularized negative log-likelihood loss restricted to a (sub-)class of distributions $\mathcal{Q}$:

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min}\, \mathbb{E}_q \left[-\log p(\mathcal{D} \mid \boldsymbol{\theta}) \right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0). \tag{15.1}$$

Here, p_0 is a prior belief over neural network weights discussed in detail in Chapter 28, and $\mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0)$ is the Kullback-Leibler divergence. For a well-specified model, and if $\mathcal{Q}$ is the space of all distributions, then the solution $q_*(\boldsymbol{\theta})$ is the Bayesian posterior [904, 1834]:

$$q_*(\boldsymbol{\theta}) \propto p_0(\boldsymbol{\theta}) \prod_{i=1}^{N} p(\mathcal{D}_i \mid \boldsymbol{\theta}), \tag{15.2}$$

here written factorized over individual data examples. This may initially seem surprising, because the objective is very similar to conventional deep learning objectives; the first term corresponds to data fit and the KL-divergence is similar to a regularizer. Later on, we will deepen this connection and show that many existing conventional deep learning optimizers actually solve a version of Equation 15.1 for a specific choice of $\mathcal{Q}$. We will also derive new optimizers of Equation 15.1 and generalize these methods to arbitrary losses beyond likelihood-based objectives. Then, we will use the term variational learning, as opposed to using the term variational inference. These new optimizers use natural-gradient descent but still have a similar form as conventional optimization algorithms.

The difference between natural-gradient and standard gradient descent is how they measure proximity. To illustrate this, we rewrite the objective from Equation 15.1 in terms of variational parameters $\boldsymbol{\lambda}$. Denoting these by $\boldsymbol{\lambda}$ we define the following loss:

$$\mathcal{L}(\boldsymbol{\lambda}) \coloneqq \mathbb{E}_q \left[-\log p(\mathcal{D} \mid \boldsymbol{\theta}) \right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0). \tag{15.3}$$

Intuitively, we might want to optimize this loss using stochastic gradient descent (SGD) on the parameters $\boldsymbol{\lambda}$, since such strategies are well-established and work well for deep learning. This strategy has been used in earlier

variational inference algorithms for neural networks, for example, Bayes-by-Backprop [174] which showed promising results on smaller neural network experiments. SGD then corresponds to using the following rule,

$$\boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t - \rho_t \nabla_{\boldsymbol{\lambda}_t} \mathcal{L}(\boldsymbol{\lambda}_t), \tag{15.4}$$

where the gradient $\nabla_{\boldsymbol{\lambda}_t} \mathcal{L}(\boldsymbol{\lambda}_t)$ may be based on mini-batches and the step-size is $\rho_t > 0$.

Following Khan and Rue [883, Sec. 2], it can be shown that Equation 15.4 solves the following objective:

$$\boldsymbol{\lambda}_{t+1} \leftarrow \arg\min_{\boldsymbol{\lambda}} \langle \nabla_{\boldsymbol{\lambda}} \mathcal{L}(\boldsymbol{\lambda}_t), \boldsymbol{\lambda} \rangle + \frac{1}{2\rho_t} \|\boldsymbol{\lambda} - \boldsymbol{\lambda}_t\|_2^2, \tag{15.5}$$

which reveals a Euclidean penalty on the parameters $\boldsymbol{\lambda}_t$. However, as we show next, this may be a bad choice for comparing distributions, for example, to understand how close a $q(\boldsymbol{\theta})$ is to a previous iterate $q_t(\boldsymbol{\theta})$ or to the "true" posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$.

To illustrate this, we repeat a classical example from the literature [763, 1213] in which two pairs of Gaussian distributions are compared using the squared Euclidean distance. One pair, $\mathcal{N}(\mu_1 = 0, \sigma_1 = 0.01)$ and $\mathcal{N}(\mu_2 = 0.1, \sigma_2 = 0.01)$, is intuitively dissimilar: μ_2 is multiples of σ_1 and σ_2 away from μ_1. Yet, the Euclidean distance of their parameters $\|\boldsymbol{\lambda}_1 - \boldsymbol{\lambda}_2\|_2^2 = (\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2 = 0.01$ is small. The other pair consists of $\mathcal{N}(\mu_1 = 10, \sigma_1 = 1000)$ and $\mathcal{N}(\mu_2 = 0, \sigma_2 = 1000)$. Both have high variance and should intuitively be more similar but the Euclidean distance is 100. This motivates choosing an objective that measures proximity in terms of distributions. The example also shows that the resulting methods should perhaps be tied to second-order information, because the difference between distributions largely depends on their variances. Indeed, we will show in Section 15.3 that natural-gradient descent is tied to second-order methods.

Instead of the Euclidean distance, natural-gradient descent uses the Kullback-Leibler (KL) divergence to compare distributions. Say we are given an approximate posterior $q(\boldsymbol{\theta})$ and a prior $p_0(\boldsymbol{\theta})$. The KL divergence between them is then defined as:

$$\mathbb{D}_{\mathrm{KL}}(q(\boldsymbol{\theta}) \,\|\, p_0(\boldsymbol{\theta})) \coloneqq \mathbb{E}_q \left[\log \left(\frac{q(\boldsymbol{\theta})}{p_0(\boldsymbol{\theta})} \right) \right] = \mathbb{E}_q[\log q(\boldsymbol{\theta})] - \mathbb{E}_q[\log p_0(\boldsymbol{\theta})], \tag{15.6}$$

where the first part is actually the negative of the entropy $\mathcal{H}(q) \coloneqq -\mathbb{E}_q[\log q(\boldsymbol{\theta})]$ of q and the second one is called cross-entropy. Replacing the squared Euclidean distance with the KL finally results in the following [883, Sec. 2.1]:

$$\boldsymbol{\lambda}_{t+1} \leftarrow \arg\min_{\boldsymbol{\lambda}} \langle \nabla_{\boldsymbol{\lambda}} \mathcal{L}(\boldsymbol{\lambda}_t), \boldsymbol{\lambda} \rangle + \frac{1}{\rho_t} \mathbb{D}_{\mathrm{KL}}(q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \,\|\, q_t(\boldsymbol{\theta})), \tag{15.7}$$

By using the following quadratic expansion [1296],

$$\mathbb{D}_{\mathrm{KL}}(q_{t+1}(\boldsymbol{\theta}) \,\|\, q_t(\boldsymbol{\theta})) \approx \frac{1}{2} (\boldsymbol{\lambda}_{t+1} - \boldsymbol{\lambda}_t)^T F(\boldsymbol{\lambda}_t) (\boldsymbol{\lambda}_{t+1} - \boldsymbol{\lambda}_t), \tag{15.8}$$

a new update rule can be derived that uses natural gradients:

$$\boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t - \rho_t F(\boldsymbol{\lambda}_t)^{-1} \nabla_{\boldsymbol{\lambda}_t} \mathcal{L}(\boldsymbol{\lambda}_t). \tag{15.9}$$

This update rule is the Bayesian learning rule [883] when optimizing likelihoods, given that $\boldsymbol{\lambda}$ are the natural parameters of exponential-family distributions, as we will show in Chapter 15.2. The natural gradients are the gradients preconditioned with the Fisher:

$$\widetilde{\nabla}_{\boldsymbol{\lambda}} \mathcal{L}(\boldsymbol{\lambda}_t) \coloneqq F(\boldsymbol{\lambda}_t)^{-1} \nabla_{\boldsymbol{\lambda}_t} \mathcal{L}(\boldsymbol{\lambda}_t). \tag{15.10}$$

The Fisher accounts for an interplay between the different dimensions of $\boldsymbol{\lambda}$ which becomes evident by writing it as the covariance of the score function of q as

$$F(\boldsymbol{\lambda}) = \mathrm{Cov}_{q(\boldsymbol{\theta})}[\nabla_{\boldsymbol{\lambda}} \log q(\boldsymbol{\theta})]. \tag{15.11}$$

As a consequence, natural-gradient-based variational inference can converge faster than regular gradient-descent-based methods when the Fisher is well-conditioned [1045]. Finally, the natural-gradient descent in Equation 15.9 is invariant to the parameterization, which does not always hold when using a Euclidean distance [1044].

Still, the inversion of the Fisher and the fact that it grows quadratically with the dimension of the neural network might seem problematic. In the next Section 15.2, we show that these issues are overcome by using exponential-family distributions. We first introduce the exponential family and then use exponential-family distributions to introduce practical second-order optimizers for natural-gradient descent.

15.1.1 Distinction from Natural Gradient Methods for Maximum Likelihood Estimation

Before that, we note that the natural gradient presented here is distinct from the natural gradient for maximum likelihood estimation of $p(\mathcal{D} \mid \boldsymbol{\theta})$. There, the idea is to use natural gradients to compare the output distributions of a neural network given a point estimate over parameters $\boldsymbol{\theta} \in \mathbb{R}^P$ rather than to compare distributions over parameter configurations. The update is then only written in terms of $\boldsymbol{\theta}$ as:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \rho_t F(\boldsymbol{\theta}_t)^{-1} \nabla_{\boldsymbol{\theta}_t} [-\log p(y_t \mid x_t, \boldsymbol{\theta}_t)]. \tag{15.12}$$

This is often used for second-order optimization [26, 1135]. Then, the Fisher is defined based on data and the corresponding natural gradient is $F(\boldsymbol{\theta}_t)^{-1} \nabla_{\boldsymbol{\theta}_t} [-\log p(y_t \mid x_t, \boldsymbol{\theta}_t)]$. To make the distinction, [1845] also refer to this as *natural gradient for point estimation* and the natural gradient discussed here as *natural gradient for variational inference*.

15.2 Generalized Bayes & The Bayesian Learning Rule

In the previous section we have presented the variational objective for inference in Equation 15.1, discussed that it splits into data fit and regularizer terms, and shown how it can be optimized using natural-gradient-based methods. However, the data fit term in Equation 15.1 is defined assuming that the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ is well-specified in the sense that for some $\boldsymbol{\theta}^* \in \Theta$, we have that $p(\mathcal{D} \mid \boldsymbol{\theta}^*)$ equals the true data-generating distribution. In addition, it is assumed that this is what the practitioner wants to optimize. However, in practice it is often not the case. For example, in machine translation we might instead want to optimize a quality rating assigned to a translation by a human rater or in LLM post-training we might want to optimize some reward assigned to a generation of the LLM by another LLM. These motivate extending variational inference to what is often referred to as Generalized Bayes [904] or also called variational learning, where arbitrary losses can be used. The posterior is then called generalized posterior [164, 246, 1858] that can also be related to Gibbs posteriors [25] with connections to thermodynamics [246].

Let us first rewrite the posterior $q_*(\boldsymbol{\theta})$ from Equation 15.2 by using the loss function $\ell(\mathcal{D}_i, \boldsymbol{\theta}) = -\log p(\mathcal{D}_i \mid \boldsymbol{\theta})$ for a data likelihood $p(\mathcal{D}_i \mid \boldsymbol{\theta})$ as follows [904]:

$$q_*(\boldsymbol{\theta}) \propto p_0(\boldsymbol{\theta}) \prod_{i=1}^{N} \exp\{-\ell(\mathcal{D}_i, \boldsymbol{\theta})\}. \tag{15.13}$$

By using generic $\ell(\mathcal{D}_i, \boldsymbol{\theta})$ we obtain a generalized posterior. Following [164], Equation 15.13 is the solution to the following generalized Bayesian objective if we choose $\mathcal{Q}$ to be the space of all distributions on Θ:

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min}\, \mathbb{E}_q\left[\ell(\mathcal{D}, \boldsymbol{\theta})\right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0). \tag{15.14}$$

Again, $\ell : \mathcal{X} \times \mathcal{Y} \times \Theta \to \mathbb{R}$ can be any suitable loss function and for $\ell(\mathcal{D}, \boldsymbol{\theta}) \propto -\log p(\mathcal{D} \mid \boldsymbol{\theta})$ we recover the Bayesian posterior [1834]. In variational learning we restrict $\mathcal{Q}$ to a specific class of distributions but need to note that the optimal distribution $q_* \in \mathcal{Q}$ might not coincide with the true solution, as this true solution nay not be contained in $\mathcal{Q}$. This is similar to what is referred to as model uncertainty in Hüllermeier and Waegeman [804, inter alia] when choosing a specific hypothesis space in general machine learning.

The objective in Equation 15.14 makes the parallels between standard optimization of neural networks and generalized variational methods more concrete: both optimize a loss term and a regularizer term but in Bayesian methods the loss uses an expectation over possible parameter values to account for uncertainty and a KL divergence to compare distributions, while standard optimization uses a loss based on a point estimate and Euclidean penalties. Intuitively, variational learning might seem to be more general and, indeed, we will next derive a Bayesian learning rule that allows recovering standard optimizers.

15.2.1 Exponential Family Distributions

We introduce the exponential-family distributions which include Gaussian, Poisson, and other distributions. We will show that exponential-family distributions are convenient for optimizing the generalized Bayesian objective in Equation 15.14.

Definition 15.1. *Minimal exponential family*

Given natural parameters $\boldsymbol{\lambda}$, sufficient statistics $\boldsymbol{T}(\boldsymbol{\theta})$, base measure $h(\boldsymbol{\theta})$ and log-partition function $Z(\boldsymbol{\lambda})$, a minimal exponential family distribution is defined as:

$$q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) = h(\boldsymbol{\theta}) \exp[\boldsymbol{\lambda}^T \boldsymbol{T}(\boldsymbol{\theta}) - Z(\boldsymbol{\lambda})]. \tag{15.15}$$

The bijective map $\boldsymbol{\mu}(\boldsymbol{\lambda}) = \mathbb{E}_{q_{\boldsymbol{\lambda}}}[\boldsymbol{T}(\boldsymbol{\theta})]$ is called expectation parameter.

For example, for diagonal Gaussians of the form $\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, \boldsymbol{s})$ the natural parameters are $\boldsymbol{\lambda} = (\boldsymbol{s}\boldsymbol{m}, -\frac{1}{2}\boldsymbol{s})^\top$, where $\boldsymbol{s} \coloneqq (\boldsymbol{\sigma}^2)^{-1}$ is called precision.

15.2.2 Bayesian Learning Rule

We will now describe the Bayesian learning rule (BLR) [883] which is a natural-gradient descent algorithm that optmizes Equation 15.14 using exponential family distributions. First, we rewrite Equation 15.14 by noting that, when choosing a prior $p_0 \propto \exp[-r(\boldsymbol{\theta})]$ proportional to a regularizer $r(\boldsymbol{\theta})$, we can express the KL term using the entropy $\mathcal{H}(q)$ as follows up to an additive constant of q that we drop:

$$\mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0) = \mathbb{E}_q[r(\boldsymbol{\theta})] - \mathcal{H}(q) + \text{const.} \tag{15.16}$$

This leads to the following objective, where we redefine $\bar{\ell}(\mathcal{D}, \boldsymbol{\theta}) \coloneqq \sum_{i=1}^N \ell(\mathcal{D}_i, \boldsymbol{\theta}) + r(\boldsymbol{\theta})$:

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min}\, \mathbb{E}_q\left[\ell(\mathcal{D}, \boldsymbol{\theta})\right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0) \tag{15.17}$$

$$= \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min}\, \mathbb{E}_q\left[\bar{\ell}(\mathcal{D}, \boldsymbol{\theta})\right] - \mathcal{H}(q). \tag{15.18}$$

We can reinterpret the objective as balancing loss and entropy of q.

Using this formulation the BLR updates the natural parameters $\boldsymbol{\lambda}$ of an exponential family distribution as follows, where the natural gradient from Equation 15.9 is now defined in terms of an arbitrary loss with a standard deep learning regularizer:

$$\boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t - \rho_t F(\boldsymbol{\lambda}_t)^{-1} \nabla_{\boldsymbol{\lambda}} \left[\mathbb{E}_{q_t}[\bar{\ell}(\mathcal{D}, \boldsymbol{\theta})] - \mathcal{H}(q)\right] \tag{15.19}$$

$$\Leftrightarrow \boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t - \rho_t \nabla_{\boldsymbol{\mu}(\boldsymbol{\lambda})} \left[\mathbb{E}_{q_t}[\bar{\ell}(\mathcal{D}, \boldsymbol{\theta})] - \mathcal{H}(q)\right]. \tag{15.20}$$

The two updates are equivalent because for exponential-family distributions we have that:

$$F(\boldsymbol{\lambda}_t)^{-1} \nabla_{\boldsymbol{\lambda}} \mathbb{E}_{q_t}[\cdot] = \nabla_{\boldsymbol{\mu}(\boldsymbol{\lambda})} \mathbb{E}_{q_t}[\cdot]. \tag{15.21}$$

This simplifies computation as no inversion of the Fisher is required anymore due to the mapping between natural and expectation parameter [736]. The next important step is then to express the term $\nabla_{\boldsymbol{\mu}(\boldsymbol{\lambda})} \mathbb{E}_{q_t}[\cdot]$ in a way that is convenient for deep learning algorithms.

15.2.3 Stochastic Gradient Descent (SGD) from the Bayesian Learning Rule (BLR)

For Gaussian $q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, \boldsymbol{\Sigma})$ the BLR update simplifies to [883, Eq. 6]:

$$\boldsymbol{\lambda}_{t+1} \leftarrow (1 - \rho_t)\boldsymbol{\lambda}_t - \rho_t \nabla_{\boldsymbol{\mu}(\boldsymbol{\lambda})} \mathbb{E}_q[\bar{\ell}(\mathcal{D}, \boldsymbol{\theta}) + \log h(\boldsymbol{\theta})]. \tag{15.22}$$

To illustrate how the BLR can be used to recover and design optimization algorithms we briefly outline how SGD can be seen as an instance of Equation 15.20 by following the derivations from [883, Sec. 1.3.1]. The derivation starts by substituting an isotropic Gaussian posterior $q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, I)$ in Equation 15.22. Then, we have that $\boldsymbol{\mu}(\boldsymbol{\theta}) = \boldsymbol{m} = \boldsymbol{\lambda}$ and $\mathbb{E}_q[\log h] = -\boldsymbol{m}^\top \boldsymbol{m}/2 + \text{const}$. As a consequence, Equation 15.20 reduces to:

$$\boldsymbol{m}_{t+1} \leftarrow \boldsymbol{m}_t - \rho_t \nabla_{\boldsymbol{m}} \mathbb{E}_{q_t}[\bar{\ell}(\mathcal{D}_t, \boldsymbol{m}_t)] \,|_{\boldsymbol{m}=\boldsymbol{m}_t}, \tag{15.23}$$

where $\boldsymbol{m}_t$ are the model parameters at step t and the variance is kept fixed to identity. This is gradient descent but the gradient is smoothed via the expectation. To arrive at regular gradient descent we use the Delta method as detailed in [883, App. C], where an expectation is approximated by the expectation of its first-order Taylor expansion, i.e

$$\mathbb{E}_q[f(\boldsymbol{\theta})] \approx \mathbb{E}_q[f(\boldsymbol{m}) + (\boldsymbol{\theta} - \boldsymbol{m})^T \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{m}}] = f(\boldsymbol{m}). \tag{15.24}$$

The delta method can be used to map from Bayesian methods to point estimation. Plugging this in Equation 15.23 we immediately get SGD. Next, we turn to a more practical part of the chapter and use similar kinds of derivations to derive NGVI optimizers that can work well even for large deep learning models.

15.3 Variational Online Newton Methods

In this section we discuss practical natural-gradient-based deep learning optimizers that can be derived from the Bayesian learning rule. These optimizers are connected to Newton's method as they use second-order information. The first method is often referred to as Variational Online Newton (VON) and makes use of a Gaussian approximation $q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, \boldsymbol{\Sigma})$. Usually, a mean-field assumption (13) with diagonal Gaussians $q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, \text{diag}(\boldsymbol{\sigma}^2))$ is made but we will be more general at least in the beginning.

We follow the derivations of Khan et al. [881, Sec. 3.1, App. C, App. D] and Khan and Rue [883] and present the main points. A factorized loss $\ell(\mathcal{D}, \boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell(\mathcal{D}_i, \boldsymbol{\theta})$ leads to:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{\theta})}[N \cdot \ell(\mathcal{D}_i, \boldsymbol{\theta})] + \mathbb{D}_{\text{KL}}(q(\boldsymbol{\theta}) \,\|\, p_0(\boldsymbol{\theta})). \tag{15.25}$$

The size of $\mathcal{D}$ balances data fit and regularization which is intuitive as observing more data means that we can rely more on the data fit which we will later exploit for tuning.

Using the chain rule to express $\nabla_{\boldsymbol{\mu}}$ in terms of $\boldsymbol{m}$ and $\boldsymbol{\Sigma}$ we can rewrite Equation 15.22:

$$\boldsymbol{\Sigma}_{t+1}^{-1} = \boldsymbol{\Sigma}_t^{-1} - 2\rho_t[\nabla_{\boldsymbol{\Sigma}}\mathcal{L}(\boldsymbol{\theta})] \tag{15.26}$$

$$\boldsymbol{m}_{t+1} = \boldsymbol{m}_t + \rho_t \boldsymbol{\Sigma}_{t+1}[\nabla_{\boldsymbol{m}}\mathcal{L}(\boldsymbol{\theta})]. \tag{15.27}$$

While this already resembles a practical algorithm, we need to handle the gradient of the expectation that is hidden inside $\mathcal{L}(\boldsymbol{\theta})$. Next, we introduce Bonnet's [187] and Price's theorem [1345], which lets us push the gradient inside the expectations [1264, 1391, inter alia].

Theorem 15.1. *Bonnet's theorem*
Let $f : \Theta \to \mathbb{R}$ once-differentiable, $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$, and $\mathbb{E}_q[\|f(\boldsymbol{\theta}\|] < \infty$. Then, we have:

$$\nabla_{\boldsymbol{m}}\mathbb{E}_q[f(\boldsymbol{\theta})] = \mathbb{E}_q[\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})]. \tag{15.28}$$

Theorem 15.2. *Price's theorem*
Let $f : \Theta \to \mathbb{R}$ twice-differentiable, $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$, and $\mathbb{E}_q[\|f(\boldsymbol{\theta}\|] < \infty$. Then, we have:

$$\nabla_{\boldsymbol{\Sigma}}\mathbb{E}_q[f(\boldsymbol{\theta})] = \frac{1}{2}\mathbb{E}_q[\nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta})]. \tag{15.29}$$

Price's theorem shows that the variance is directly related to the Hessian.

Then we can write the updates as follows using $\delta > 0$[1]:

$$\boldsymbol{\Sigma}_{t+1}^{-1} \leftarrow (1-\rho_t)\boldsymbol{\Sigma}_t^{-1} + \rho_t \left[N \cdot \mathbb{E}_q[\nabla^2_{\boldsymbol{\theta}}\ell(\mathcal{D}_t, \boldsymbol{\theta})] + \delta I\right] \tag{15.30}$$

$$\boldsymbol{m}_{t+1} \leftarrow \boldsymbol{m}_t - \rho_t \boldsymbol{\Sigma}_{t+1}\left[N \cdot \mathbb{E}_q[\nabla_{\boldsymbol{\theta}}\ell(\mathcal{D}_t, \boldsymbol{\theta})] + \delta \boldsymbol{m}_t\right], \tag{15.31}$$

The expectations $\mathbb{E}_q[\cdot]$ can be approximated using Monte-Carlo methods with samples $\boldsymbol{\theta}_t \sim \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_t, \boldsymbol{\Sigma}_t)$ at each iteration. This is similar to standard neural network training but noise is added to the current learned mean $\boldsymbol{m}_t$ which induces parameter exploration.

In addition to the expectations, we need to find tractable approximations of the Hessian $H(\boldsymbol{\theta}) \coloneqq \nabla^2_{\boldsymbol{\theta}}\ell(\mathcal{D}_t, \boldsymbol{\theta}) \in \mathbb{R}^{P\times P}$. While a mean-field approximation using $\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_t, \text{diag}(\boldsymbol{\sigma}^2))$ reduces storage cost to the same as the cost of storing the mean, we still require an efficient computation of $\text{diag}(H(\boldsymbol{\theta}))$. For this, we use Stein's Lemma [1560] for multivariate Gaussians as presented by Lin et al. [1046, Lemma 5] and then apply it for Hessian computation.

Lemma 15.1. *Stein's Lemma for Multivariate Gaussians*
Given $q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}, \boldsymbol{\Sigma})$, $f : \Theta \to \mathbb{R}$, continuous, locally ACL, and $\mathbb{E}_q[\|f(\boldsymbol{\theta})\|] < \infty$. Then, we have the following identity:

$$\mathbb{E}_q[\boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{m})f(\boldsymbol{\theta})] = \mathbb{E}_q[\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})]. \tag{15.32}$$

As a consequence, we can rewrite Price's theorem as follows.

Theorem 15.3. *Price's Theorem - revisited*
Let $f : \Theta \to \mathbb{R}$ once-differentiable, $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$, and $\mathbb{E}_q[\|f(\boldsymbol{\theta}\|] < \infty$. Then, we have:

$$\nabla_{\boldsymbol{\Sigma}}\mathbb{E}_q[f(\boldsymbol{\theta})] = \frac{1}{2}\mathbb{E}_q[\boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{m})\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})]. \tag{15.33}$$

[1] In Khan et al. [881], $\delta = \delta'/N$, where δ' is the prior precision of $p_0 = \mathcal{N}(\boldsymbol{\theta}; 0, \boldsymbol{I}/\delta')$.

Now we only need to require f to be once-differentiable anymore. Similarly, we could also rewrite Bonnet's theorem to obtain a gradient estimator even for non-differentiable functions. By applying Price's theorem to the loss $\ell(\mathcal{D}, \boldsymbol{\theta})$ we get:

$$\mathbb{E}_q[\nabla^2_{\boldsymbol{\theta}} \ell(\mathcal{D}, \boldsymbol{\theta})] = \mathbb{E}_q[\boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{m}) \nabla_{\boldsymbol{\theta}} \ell(\mathcal{D}, \boldsymbol{\theta})]. \tag{15.34}$$

That is, the Hessian of the loss can be calculated by using first-order gradients. A Monte-Carlo estimator of Equation 15.34 is an unbiased estimate of the expected Hessian.

We next present VON with one Monte-Carlo sample and diagonal Gaussians. The variance in VON can be defined as $\boldsymbol{\sigma^2} \coloneqq 1/[N(\boldsymbol{h}+\delta)]$ for a Hessian estimate $\boldsymbol{h}$ and weight-decay δ. The diagonal Hessian $\boldsymbol{h}$ is approximated by using Price's theorem coupled with Stein's Lemma which only requires first-order gradients. These gradients are estimated using an unbiased parameter sample from q_t. VON can then be implemented by the following algorithm for minibatch $\mathcal{B} \subset \mathcal{D}$, where we use the shorthand notation $\ell_i(\boldsymbol{\theta})$ for the loss of the i-th example in a mini-batch:

$$\hat{\boldsymbol{g}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}_t} \ell_i(\boldsymbol{\theta}_t) \quad \text{where } \boldsymbol{\theta}_t \sim q_t(\boldsymbol{\theta}) \tag{15.35}$$

$$\hat{\boldsymbol{h}} \leftarrow (\boldsymbol{\theta}_t - \boldsymbol{m}_t)\hat{\boldsymbol{g}}/\boldsymbol{\sigma}_t^2 \tag{15.36}$$

$$\boldsymbol{g} \leftarrow \beta_1 \boldsymbol{g} + (1-\beta_1)\hat{\boldsymbol{g}} \tag{15.37}$$

$$\boldsymbol{h}_{t+1} \leftarrow \beta_2 \boldsymbol{h}_t + (1-\beta_2)\hat{\boldsymbol{h}} \tag{15.38}$$

$$\boldsymbol{m}_{t+1} \leftarrow \boldsymbol{m}_t - \rho_t \left(\boldsymbol{g} + \delta \boldsymbol{m}_t\right) / (\boldsymbol{h}_{t+1} + \delta) \tag{15.39}$$

$$\boldsymbol{\sigma}^2_{t+1} \leftarrow 1/N(\boldsymbol{h}_{t+1} + \delta) \tag{15.40}$$

The preconditioner in VON uses the Hessian without square root, which is used in Adam-like algorithms, and the Hessian is calculated online. Still, due to its similarity to conventional optimizers VON is easy to implement in standard deep learning frameworks.

However, so far there is no guarantee that the Hessian will stay positive in VON. If the Hessian is not positive, q is not in the family of diagonal Gaussians anymore, as inversion of the covariance is not possible. While the authors of [881] note that backtracking could be used to prevent the Hessian from becoming negative, this strategy is slow. This motivates the next sections on VOGN and IVON, where we will first use a different Hessian estimator that guarantees positivity, and then revisit VON with a positive definite correction that moves q back into the feasible set of Equation 15.18.

15.3.1 VOGN

To optimize the variational objective we need to ensure that the solutions remain in the feasible set consisting of valid distributions $q \in \mathcal{Q}$. In the Gaussian case this means that $\boldsymbol{\Sigma} \in \mathbb{R}^{P \times P}$ has to be invertible. For VON, we have determined the relationship $\boldsymbol{\sigma}^2 = 1/[N(\boldsymbol{h} + \delta)]$ between variance and Hessian but VON does not ensure that $\boldsymbol{h} + \delta$ stays positive. In this section we present the Variational Online Gauss-Newton (VOGN) method based on [881, 1266] which uses a mean-field Gaussian with a positive-definite Hessian approximation.

The approximation is based on the Generalized Gauss-Newton matrix (GGN). In order to define the GGN, let us decompose the Hessian by referring to the neural network output of a network with parameters $\boldsymbol{\theta} \in \mathbb{R}^P$ as $f_{\boldsymbol{\theta}}(x)$. The following result was obtained by [1473] and is also discussed in [1135, inter alia]. It states that the Hessian consists of:

$$\nabla^2_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}) = \underbrace{(\nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(x_i))^T \nabla^2_f \ell_i(\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(x_i)}_{\text{GGN}} + \underbrace{\nabla^2_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(x_i) \nabla_z \ell(y_i, z)_{|z=f(x_i)}}_{\text{residual}}, \tag{15.41}$$

where $\nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(x_i)$ is the Jacobian. By dropping the second term in the decomposition we arrive at the GGN approximation of the Hessian. The GGN retains the curvature induced by the loss with respect to network outputs but drops curvature due to the network's nonlinear parameterization. For many settings it can be shown that the GGN and Fisher coincide. We refer the reader to [1135, Sec. 9.2] for a discussion.

Clearly, the GGN itself again requires the computation of a Hessian and Jacobian. This can be prohibitive in itself, as for a neural network with P parameters and C output classes we have that $\nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(x_i) \in \mathbb{R}^{P \times C}$. For example, for LLMs C is often in the order of tens of thousands as it consists of all possible tokens from the vocabulary. Conveniently, though, there are estimators for the GGN that are easy and comparably fast to compute simply based on first-order gradients. The estimator used in VOGN was presented in Graves [660] and simply

averages the squared gradients of examples in a mini-batch:

$$\hat{\boldsymbol{h}} \approx \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} [\nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta})]^2. \tag{15.42}$$

This estimator is guaranteed to be nonnegative but popular deep learning libraries directly calculate a sum of gradients over the mini-batch and the per-example gradients can only be obtained with additional computation and overhead. Indeed, [1510] report an overhead of almost 1.8x in an image classification setting when compared to the estimator used in VON. Hence, it might seem intuitive to just square the gradients of the minibatch directly, i.e., to move the average inside the square:

$$\hat{\boldsymbol{h}}^{\text{GM}} \approx \left[\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}) \right]^2. \tag{15.43}$$

This is often referred to as gradient magnitude approximation [195] and the authors of [881] use this to derive a variational version of RMSprop called Vprop. Both estimates are compared in Khan et al. [881, Thm. 1] which we restate here next.

Theorem 15.4. *Bias of Gradient Magnitude Approximation*

Given are mini-batches $\mathcal{B}$ of size M drawn uniformly from $p(\mathcal{B})$ over all mini-batches that can be formed based on $\mathcal{D}$ of size N, GGN $\boldsymbol{h}$ calculated on all data, Gradient Magnitude approximation $\hat{\boldsymbol{h}}$, and gradients $\boldsymbol{g}$ on all data. Using $w = \frac{1}{M}(N-M)/(N-1)$ we have:

$$\mathbb{E}_{p(\mathcal{B})}[\hat{\boldsymbol{h}}^{GM}] = w\boldsymbol{h} + (1-w)\boldsymbol{g}^2. \tag{15.44}$$

Only $M = 1$ provides an unbiased estimat and the bias grows with M.

Similar to Osawa et al. [1266, Alg. 1], VOGN can then be implemented by simply replacing the Hessian estimator in VON with:

$$\hat{\boldsymbol{h}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} [\nabla_{\boldsymbol{\theta}_t} \ell_i(\boldsymbol{\theta}_t)]^2. \tag{15.45}$$

Using a stricly positive damping of the Hessian guarantees a strictly positive estimate. The authors of VOGN report promising results, reaching an accuracy that is close to or better than SGD and Adam [894] with improved calibration. However, they report a significant computational overhead of ca. 1.7x due to the per-example gradients required, similar to the overhead reported in Shen et al. [1510]. In many modern applications, where training can take days, weeks, and months, this may be too expensive. Next, we describe the Improved Variational Online Newton (IVON) optimizer [1047, 1510] which is fast because it uses the same estimator as in VON and guarantees that q stays within the feasible set.

15.3.2 shen2024variational

At the beginning of this section, we have introduced the VON algorithm as a practical optimizer for neural networks with a mean-field Gaussian posterior approximation. However, VON does not constrain q to be Gaussian which requires the covariance to be positive-definite (while $\boldsymbol{m}$ is unconstrained). The constraint can be unfulfilled when running VON on a non-convex loss $\ell(\mathcal{D}_i, \boldsymbol{\theta})$, because in Equation 15.30 the Hessian $\mathbb{E}_q[\nabla^2_{\boldsymbol{\theta}} \ell(\mathcal{D}_t, \boldsymbol{\theta})]$ can be indefinite if ℓ is not convex. This is also true for Equation 15.40, where the reparameterization trick is used to estimate the Hessian.

IVON overcomes this by adding a positive-definite correction to the Hessian update, following the work of Lin et al. [1047]. We refer the reader to their work for a detailed derivation and only use their main result in the context of IVON next. In IVON every update step starts with a current approximate posterior $q_t = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_t, \boldsymbol{\sigma}_t^2)$. IVON uses the following update rule for the Hessian, where the positive-definite correction is shown in grey:

$$\hat{\boldsymbol{g}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}_t} \ell_i(\boldsymbol{\theta}_t) \quad \text{where } \boldsymbol{\theta}_t \sim q_t(\boldsymbol{\theta}) \tag{15.46}$$

$$\hat{h} \leftarrow \hat{\boldsymbol{g}} \cdot (\boldsymbol{\theta}_t - \boldsymbol{m}_t)/\boldsymbol{\sigma}_t^2 \tag{15.47}$$

$$\boldsymbol{h}_{t+1} \leftarrow \beta_2 \boldsymbol{h}_t + (1-\beta_2)\hat{\boldsymbol{h}} + \frac{1}{2}(1-\beta_2)^2(\boldsymbol{h}_t - \hat{\boldsymbol{h}})^2/(\boldsymbol{h}_t + \delta). \tag{15.48}$$

Algorithm 19: Improved Variational Online Newton (IVON)

Inputs : learning rates $\{\rho_t\}$; gradient momentum $\beta_1 \in [0, 1)$; Hessian momentum $\beta_2 \in [0, 1)$; weight decay $\delta > 0$; scaling parameter $\lambda > 0$; Hessian initialization $h_0 > 0$

Outputs: mean parameter $\boldsymbol{m}_{t+1}$ and variance $\boldsymbol{\sigma}^2_{t+1}$

initialize $\boldsymbol{m} \leftarrow$ (NN weight init), $\boldsymbol{h} \leftarrow h_0 \cdot \boldsymbol{I}$, $\boldsymbol{\sigma}^2 \leftarrow 1/(\lambda(\boldsymbol{h} + \delta))$, $\boldsymbol{g} \leftarrow 0$, $t \leftarrow 1$, and $\alpha_t \leftarrow (h_0 + \delta)\alpha_t$ for all t

repeat

- $\hat{\boldsymbol{g}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}_t} \ell_i(\boldsymbol{\theta}_t)$, where $\boldsymbol{\theta}_t \sim q_t(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_t, \text{diag}(\boldsymbol{\sigma}_t^2))$
- $\hat{\boldsymbol{h}} \leftarrow \hat{\boldsymbol{g}} \cdot (\boldsymbol{\theta}_t - \boldsymbol{m}_t)/\boldsymbol{\sigma}_t^2$
- $\boldsymbol{g} \leftarrow \beta_1 \boldsymbol{g} + (1 - \beta_1)\hat{\boldsymbol{g}}$
- $\boldsymbol{h} \leftarrow \beta_2 \boldsymbol{h}_t + (1 - \beta_2)\hat{\boldsymbol{h}} + \frac{1}{2}(1 - \beta_2)^2(\boldsymbol{h}_t - \hat{\boldsymbol{h}})^2/(\boldsymbol{h}_t + \delta)$
- $\bar{\boldsymbol{g}} \leftarrow \boldsymbol{g}/(1 - \beta_1^t)$
- $\boldsymbol{m}_{t+1} \leftarrow \boldsymbol{m}_t - \alpha_t(\bar{\boldsymbol{g}} + \delta \boldsymbol{m}_t)/(\boldsymbol{h} + \delta)$
- $\boldsymbol{\sigma}^2_{t+1} \leftarrow 1/(\lambda(\boldsymbol{h} + \delta))$

until *convergence*

return $\boldsymbol{m}_{t+1}, \boldsymbol{\sigma}^2_{t+1}$

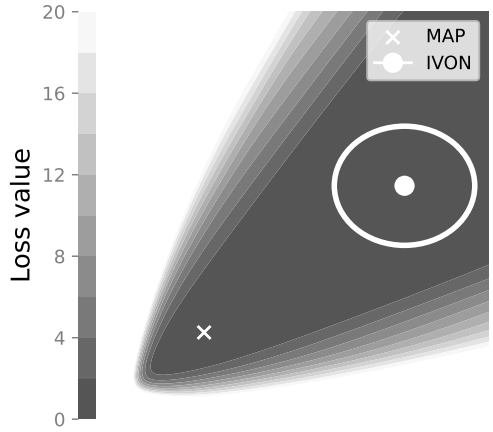


Figure 15.1: The solution found by IVON lands in a flat region away from the MAP. Figure reproduced from the code of Shen et al. [1510].

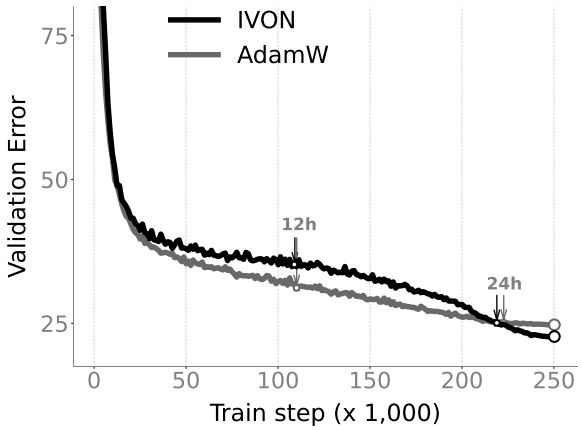


Figure 15.2: IVON vs. AdamW on ImageNet, taken from Shen et al. [1510] with permission.

The correction $\frac{1}{2}(1 - \beta_2)^2(\boldsymbol{h}_t - \hat{\boldsymbol{h}})^2/(\boldsymbol{h}_t + \delta) >^{\text{a.s.}} 0$ ensures that the current approximate posterior $q_{t+1} = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_{t+1}, \text{diag}(\boldsymbol{\sigma}^2_{t+1}))$ is still a valid Gaussian distribution as $\boldsymbol{h}_{t+1}$ is always positive-definite if $\boldsymbol{h}_t$ is positive-definite, following [1047, Thm. 1]. A proof for a generalized setting with full covariance is found in [1047, App. E.1].

In addition to this, IVON uses a slightly different diagonal variance $\boldsymbol{\sigma}^2 = 1/\lambda(\boldsymbol{h} + \delta)$, where $\boldsymbol{h} = h_0 \cdot \boldsymbol{I}$ for $h_0 > 0$ in the first iteration, δ is a weight-decay parameter from using a prior with scalar variance and λ is the effective sample size that can be tuned as a hyperparameter. Tuning λ can be helpful in many situations, for example, when training many epochs on little data it can be set to $\lambda > N$ to simulate a setting where more data is available. Another example is when predicting using the posterior: when sampling from the final $q(\boldsymbol{\theta})$, it is possible to decrease λ to sample more diverse models (that are farther away from $\boldsymbol{m}$) or increase λ to sample less diverse models. We refer the reader to Chapter 43 for a more in-depth discussion of this in the context of large language models. The final updates of IVON are shown in Algorithm 19. IVON can also be used with multiple MC samples to estimate the gradient. In this case, the gradient is just averaged over MC samples and also estimate $\hat{\boldsymbol{h}}$ is averaged over multiple samples.

The noise in IVON can be helpful for regularizing training, because IVON constantly explores new solutions according to the chosen noise level during training. This leads to it "bouncing" away from sharp solutions, as adding even a bit of noise there will heavily increase the loss, towards flatter minima, as is illustrated in Figure 15.1 on a small logistic regression problem, where the maximum-a-posteriori (MAP) problem finds a sharp minimum but IVON finds a flat minimum. Flatter minima have long been connected to improved generalization and better

uncertainty estimates. Furthermore, gradient spikes can be smoothed out when multiple MC samples are used due to the Gaussian smoothing.

Altogether, IVON enables scaling natural-gradient descent with online Newton methods to large problems. Here, we show results on ImageNet for a ResNet-50 in Figure 15.2, which were obtained in Shen et al. [1510], and in Section 43.2 results for pretraining GPT-2 with 773M parameters from scratch are shown. In the ImageNet experiment, IVON obtains better or comparable accuracy as AdamW, improved calibration and uncertainty, and only requiring a computational overhead of 1.0-1.2x (depending on model size) due to the additional parameter sampling step. This shows that variational learning of large models is feasible and due to the techniques shown in this chapter is also easy to implement and use.

15.4 Concluding Remarks

We have presented natural-gradient-descent algorithms to learn generalized posterior using the framework of variational learning, where arbitrary losses can be used. In order to derive these algorithms, we have introduced the Bayesian Learning Rule as well as various convenient techniques to, for example, estimate second-order information. We have introduced various algorithms that can use these techniques to learn distributions over neural network parameters in an efficient way with similar computational requirements as conventional neural network optimizers.

Chapter 16

Particle-Based Variational Inference

16.1 Introduction

Bayesian deep learning (BDL) seeks to estimate the posterior distribution over the parameters of a deep neural network. In this setting, however, traditional methods for performing inference face significant challenges.

Markov chain Monte Carlo (MCMC) methods like Hamiltonian Monte Carlo (HMC) (Chapter 1) are asymptotically exact and can, in principle, generate samples from the true posterior. In practice, however, they are often prohibitively slow and scale poorly to large datasets or high-dimensional neural network models.

Variational inference (VI) offers a more scalable alternative by reframing inference as an optimisation problem (Chapter 12): one posits an approximate distribution $q(\theta)$, and adjusts its parameters to minimise the Kullback–Leibler (KL) divergence $\mathrm{D}_{\mathrm{KL}}(q\|\pi)$ to the target posterior $\pi(\theta)$ [170, 1840]. The most common form of VI used in Bayesian neural networks is mean-field VI (Chapter 13), which assumes all weights are independent (e.g., diagonal Gaussian posteriors). While computationally convenient, this approximation is highly restrictive: it fails to capture complex posterior correlations or multimodality, often underestimates uncertainty, and can lead to miscalibrated models [1108]. More expressive variational families (e.g., full-covariance Gaussians, normalising flows) do exist, but they introduce substantial optimisation challenges and are hard to tune, often requiring model-specific derivations.

If classical MCMC algorithms are asymptotically accurate but too slow for deep learning, whereas mean-field VI is fast but often leads to crude posterior approximations, then is there a way of leveraging the computational efficiency of VI with the nonparametric nature of MCMC? *Particle-Based Variational Inference* (ParVI) methods aim to bridge this gap by combining the strengths of both MCMC and variational inference.

The core idea, first introduced by Liu and Wang [1064], is to maintain a set of n particles $\{\theta^i\}_{i=1}^n$ that collectively represent an approximate posterior. Instead of evolving each particle via random Monte Carlo moves, ParVI (in its prototypical form) deterministically transports the entire set of particles towards regions of high posterior density [1064]. In each iteration, the particles are used to approximate a population transport direction that decreases the KL divergence to the target π, via a form of gradient descent in the space of probability distributions [1064]. By using information from the target's score function (the gradient of $\log \pi(\theta)$) and coupling the particles through a repulsive interaction, ParVI methods encourage particles to spread out and cover the posterior's support rather than collapsing to a single mode (see Figure 16.1 for a visual illustration). In effect, ParVI treats Bayesian inference as an optimisation in function space, finding a distribution (represented by particles) that minimises $\mathrm{D}_{\mathrm{KL}}(q\|\pi)$. This approach retains the flexibility of nonparametric sampling, since the empirical distribution represented by the particles is not constrained to lie within a specific parametric family (e.g., Gaussian), while leveraging gradient-based optimisation for efficiency and scalability. That said, for a finite number of particles the empirical approximation is itself restricted, and its quality depends on the number of particles, the choice of kernel, and the interactions between the particles.

Mathematical Background and Reading Guide. Relative to the preceding chapters, parts of this chapter make use of some additional mathematical background, in particular reproducing kernel Hilbert spaces [e.g., 128], and notions from optimal transport and gradient flows on the space of probability measures [e.g., 27, 292, 1458, 1679]. To help the reader navigate the chapter, sections marked with an asterisk (*) are more advanced, and can safely be skipped on a first reading: a reader interested primarily in the basic methodology can read Sections 16.2, 16.3, 16.6, and 16.7, returning to the starred sections later.

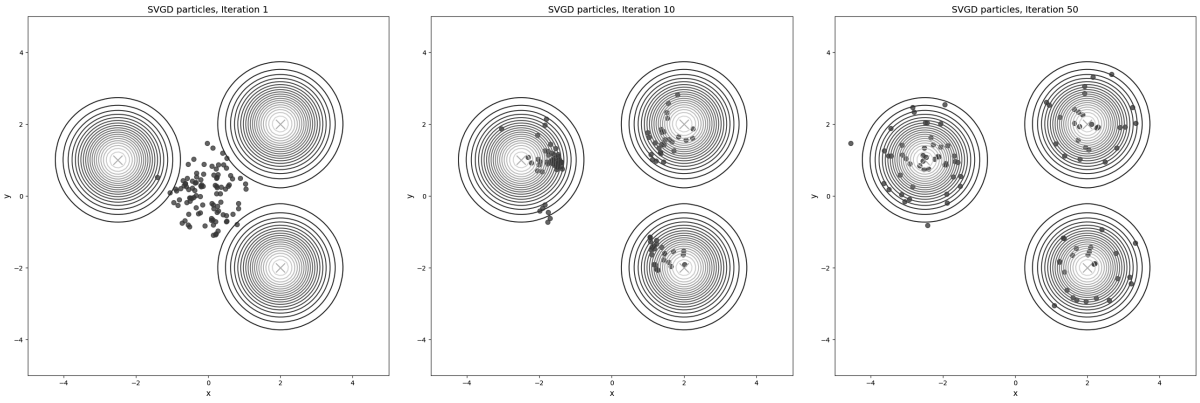


Figure 16.1: An illustration of a particle-based variational inference approach on a mixture of three Gaussians target distribution π. Using the Stein variational gradient descent algorithm (SVGD, Algorithm 20), 100 particles are simulated from $\mathcal{N}(0, 0.5\,I)$ and evolved via the SVGD dynamics; the three panels show the particles after 1, 10, and 50 iterations (left to right). Initially, the particles are attracted to regions of high posterior density, then the repulsive force of the kernel encourages the particles to spread out and produce a nonparametric approximation of the posterior distribution.

16.2 Variational Objectives Revisited

16.2.1 The KL Divergence and the Evidence Lower Bound (ELBO)

VI frames Bayesian posterior estimation as an optimisation problem. In particular, given a target posterior $\pi(\theta) := p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$, VI seeks an approximate distribution $q(\theta)$ that minimises the KL divergence to the target posterior:

$$\mathrm{D}_{\mathrm{KL}}(q\|\pi) = \mathbb{E}_q\left[\log \frac{q(\theta)}{\pi(\theta)}\right] = -\mathbb{E}_q\left[\log \pi(\theta)\right] + \mathbb{E}_q\left[\log q(\theta)\right]. \tag{16.1}$$

Minimising the KL divergence is equivalent to maximising the evidence lower bound (ELBO), since

$$\mathrm{D}_{\mathrm{KL}}(q\|\pi) = -\underbrace{\mathbb{E}_q\left[\log p(\mathcal{D}|\theta) + \log p(\theta) - \log p(\mathcal{D})\right]}_{:=\mathbb{E}_q[\log \pi(\theta)]} + \mathbb{E}_q\left[\log q(\theta)\right] \tag{16.2}$$

$$= -\underbrace{\mathbb{E}_q\left[\log p(\mathcal{D}|\theta) + \log p(\theta) - \log q(\theta)\right]}_{:=\mathrm{ELBO}(q)} + \log p(\mathcal{D}) \tag{16.3}$$

$$= -\mathrm{ELBO}(q) + \text{const.} \tag{16.4}$$

The ELBO provides a tractable objective for fitting the variational approximation. In conventional VI [170], q is constrained to belong to a parametric family, say $\{q_\psi\}_{\psi\in\Psi}$, with variational parameters ψ. For example, one might specify a Gaussian family of distributions, in which case the variational parameters are simply $\psi = [\mu, \Sigma]$ specifying the mean μ and covariance Σ of the variational family. One then maximises the ELBO with respect to the variational parameters (e.g., via stochastic gradient ascent) to obtain the approximation of the posterior.

In ParVI, we still seek a variational distribution $q(\theta)$ that maximises the ELBO. However, the constraint that $q(\theta)$ must belong to a parametric family is now relaxed. Thus, we must now solve an *infinite-dimensional* optimisation problem over the space of probability measures, rather than a *finite-dimensional* optimisation problem over the space of the variational parameters.

16.2.2 Infinitesimal Transport of q by a Vector Field

Let q and π be strictly positive C^1 densities on $\mathbb{R}^d$. Fix a C^1 vector field $\phi : \mathbb{R}^d \to \mathbb{R}^d$ with sufficient decay (or compact support) so that the boundary terms below vanish. For ε small, we consider a transport map obtained as a small perturbation of the identity map, viz

$$T_\varepsilon(\theta) = \theta + \varepsilon\,\phi(\theta), \qquad T_0 = \mathrm{Id}. \tag{16.5}$$

We will consider distributions obtained as the push-forward of q under this transport map, viz

$$q_\varepsilon = (T_\varepsilon)_\# q, \quad \text{i.e.} \quad \int f(\theta)\, q_\varepsilon(\theta)\mathrm{d}\theta = \int f(T_\varepsilon(\theta)) q(\theta)\mathrm{d}\theta \quad \forall f. \tag{16.6}$$

By the change-of-variables formula, the density q_ε is given explicitly by

$$q_\varepsilon(\theta) = q\big(T_\varepsilon^{-1}(\theta)\big)\Big|\det \nabla T_\varepsilon^{-1}(\theta)\Big|, \tag{16.7}$$

where T_ε^{-1} is the inverse of T_ε, and where $\nabla T_\varepsilon^{-1}$ denotes the Jacobian matrix of T_ε^{-1}. A first-order expansion in ε gives the classical *continuity equation*

$$\frac{\mathrm{d}}{\mathrm{d}\varepsilon}\Big|_{\varepsilon=0} q_\varepsilon(\theta) = -\nabla_\theta \cdot \big(q(\theta)\phi(\theta)\big) \quad \Longleftrightarrow \quad \partial_\varepsilon q_\varepsilon + \nabla\cdot(q_\varepsilon\phi) = 0 \quad \text{at } \varepsilon = 0. \tag{16.8}$$

16.2.3 The Derivative of the KL Divergence

We would like to define the transport map T_ε, via the vector field ϕ, in such a way that it decreases the KL divergence: $\mathrm{D}_{\mathrm{KL}}((T_\varepsilon)_\# q\|\pi) \leq \mathrm{D}_{\mathrm{KL}}(q\|\pi)$. In order to identify a suitable vector field, we now differentiate the KL functional w.r.t. the perturbation magnitude ε, viz

$$\mathrm{D}_{\mathrm{KL}}(q_\varepsilon\|\pi) = \int q_\varepsilon(\theta)\log\frac{q_\varepsilon(\theta)}{\pi(\theta)}\mathrm{d}\theta. \tag{16.9}$$

Using the continuity equation in (16.8) and the product rule, we can compute this as follows:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}\varepsilon}\mathrm{D}_{\mathrm{KL}}(q_\varepsilon\|\pi)\Big|_{\varepsilon=0} &= \int \frac{\mathrm{d}}{\mathrm{d}\varepsilon}\Big|_{\varepsilon=0}\Big\{q_\varepsilon(\theta)\log\tfrac{q_\varepsilon(\theta)}{\pi(\theta)}\Big\}\mathrm{d}\theta \\
&= \int \Big[-\nabla\cdot\big(q(\theta)\phi(\theta)\big)\Big]\Big(\log\tfrac{q(\theta)}{\pi(\theta)}+1\Big)\mathrm{d}\theta \\
&= \int q(\theta)\phi(\theta)\cdot\nabla_\theta\Big(\log\tfrac{q(\theta)}{\pi(\theta)}+1\Big)\mathrm{d}\theta \quad \text{(integration by parts)} \\
&= \int q(\theta)\,\phi(\theta)\cdot\Big(\nabla\log q(\theta) - \nabla\log\pi(\theta)\Big)\mathrm{d}\theta \\
&= \mathbb{E}_q\Big[\phi(\theta)^\top\big(\nabla\log q(\theta) - \nabla\log\pi(\theta)\big)\Big] \\
&= -\mathbb{E}_q\Big[\nabla_\theta\cdot\phi(\theta) + \phi(\theta)^\top\nabla\log\pi(\theta)\Big],
\end{aligned}$$

where in the final line we have used the identity $\mathbb{E}_q[\nabla\cdot\phi] = -\mathbb{E}_q[\phi^\top\nabla\log q]$. We have thus shown that

$$\frac{\mathrm{d}}{\mathrm{d}\varepsilon}\mathrm{D}_{\mathrm{KL}}(q_\varepsilon\|\pi)\Big|_{\varepsilon=0} = -\mathbb{E}_q\big[T_\pi\phi(\theta)\big], \qquad T_\pi\phi := \nabla\cdot\phi + \phi^\top\nabla\log\pi. \tag{16.10}$$

16.2.4 The Direction of Steepest Descent

From the previous expression, any direction that *increases* $\mathbb{E}_q[T_\pi\phi]$ yields a *decrease* in $\mathrm{D}_{\mathrm{KL}}(\cdot\|\pi)$. Naturally, we would like to choose the direction ϕ which maximally decreases the KL divergence. That is, we would like to solve

$$\operatorname*{arg\,min}_{\phi\in\mathcal{G}}\Big\{\frac{\mathrm{d}}{\mathrm{d}\varepsilon}\mathrm{D}_{\mathrm{KL}}((T_\varepsilon)_\# q\|\pi)\Big|_{\varepsilon=0} : ||\phi||_{\mathcal{G}} \leq 1\Big\} = \operatorname*{arg\,max}_{\phi\in\mathcal{G}}\Big\{\mathbb{E}_q\left[T_\pi\phi(\theta)\right] : ||\phi||_{\mathcal{G}} \leq 1\Big\}, \tag{16.11}$$

where $\mathcal{G}$ is a normed space of functions. In order to make progress, we will consider the restriction of the vector field ϕ to a ball within a reproducing-kernel Hilbert space (RKHS). This will allow us to obtain a closed-form expression for the vector field which maximises the negative gradient on the RHS of (16.11). We begin by providing some background on the operator T_π.

The Stein Operator and Stein's Identity. The operator T_π is known as the (Langevin) *Stein operator* [1559], due to its connection with *Stein's identity*. Let π be a strictly positive C^1 density on $\mathbb{R}^d$ with score $u_\pi(\theta) := \nabla \log \pi(\theta)$. Then, for any sufficiently regular test function $f : \mathbb{R}^d \to \mathbb{R}^d$ satisfying mild zero boundary conditions, Stein's identity states that

$$\mathbb{E}_\pi[T_\pi f(\theta)] = \mathbb{E}_\pi \left[\nabla_\theta \cdot f(\theta) + f(\theta)^\top u_\pi(\theta)\right] = 0, \tag{16.12}$$

which is easily seen to hold via integration by parts. We emphasise that the (Langevin) Stein operator T_π depends only on the *score* and therefore does not require the normalising constant of π. Indeed, $\nabla_\theta \log \pi = \nabla_\theta \log \tilde{\pi}$ for $\pi = \tilde{\pi}/Z$. This is a particularly useful property in Bayesian deep learning, where the normalising constant is intractable.

The Stein Discrepancy. The identity in (16.12) characterises π in the sense that, under mild conditions on the class of test functions $\mathcal{G}$, $\mathbb{E}_q[T_\pi f] = 0$ for all such $f \in \mathcal{G}$ *if and only if* $q = \pi$ a.e. [e.g., 652, 1559]. More generally, we can use the maximum value of this expectation over all test functions $f \in \mathcal{G}$ to characterise the difference between the distributions q and π. In particular, we define the *Stein discrepancy* as

$$\mathbb{D}(q\|\pi) := \max_{f \in \mathcal{G}} \left\{\mathbb{E}_q\left[T_\pi f\right] : ||f||_{\mathcal{G}} \leq 1\right\}. \tag{16.13}$$

Clearly, the choice of the function class $\mathcal{G}$ here is critical. It must be both sufficiently rich to ensure that $\mathbb{D}(q\|\pi)$ has adequate discriminative power, while not too rich to render $\mathbb{D}(q\|\pi)$ computationally intractable. Following Chwialkowski et al. [307], Gorham and Mackey [652], Liu and Wang [1064], we will take $\mathcal{G}$ to be a vector-valued RKHS. This ensures not only that the linear functional $f \mapsto \mathbb{E}_q[T_\pi f]$ is continuous, but also that the optimisation problem in (16.11), (16.13) has a closed form.

The Kernel Stein Discrepancy. Let $\mathcal{H}$ be a RKHS of scalar-valued functions with positive-definite C^2 kernel $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$. Let $\mathcal{H}^d := \mathcal{H} \times \cdots \times \mathcal{H}$ denote the corresponding $\mathbb{R}^d$-valued RKHS, equipped with inner product $\langle f, g\rangle_{\mathcal{H}^d} = \sum_{\ell=1}^d \langle f_\ell, g_\ell\rangle_{\mathcal{H}}$. Using the reproducing property of the Hilbert space, and integration by parts, it holds for all $f \in \mathcal{H}^d$ that

$$\begin{aligned}\mathbb{E}_q[T_\pi f] &= \mathbb{E}_q\big[\nabla \cdot f(\theta)\big] + \mathbb{E}_q\big[f(\theta)^\top u_\pi(\theta)\big] \\ &= \mathbb{E}_q\big[\mathrm{Tr}(\nabla_\theta f(\theta))\big] + \mathbb{E}_q\big[f(\theta)^\top u_\pi(\theta)\big] \\ &= \big\langle f(\cdot), \mathbb{E}_{\theta\sim q}\big[k(\theta,\cdot)\, u_\pi(\theta) + \nabla_\theta k(\theta,\cdot)\big]\big\rangle_{\mathcal{H}^d} = \langle f, \mu_{q,\pi}\rangle_{\mathcal{H}^d}.\end{aligned} \tag{16.14}$$

where in the final line we have used $\mu_{q,\pi}$ to denote the *Stein witness function*, defined as

$$\mu_{q,\pi}(\cdot) := \mathbb{E}_{\theta\sim q}[k(\theta,\cdot)u_\pi(\theta) + \nabla_\theta k(\theta,\cdot)] \in \mathcal{H}^d. \tag{16.15}$$

Substituting back into (16.13), the *kernel Stein discrepancy* (KSD) between q and π is given by the RKHS dual norm [307, 652, 1064]

$$\mathrm{KSD}(q\|\pi) := \max_{f \in \mathcal{H}^d} \left\{\mathbb{E}_q[T_\pi f] \;: ||f||_{\mathcal{H}^d} \leq 1\right\} = \|\mu_{q,\pi}\|_{\mathcal{H}^d}. \tag{16.16}$$

Using (16.15) and the reproducing property, we can obtain a closed-form expression for the (squared) KSD as

$$\mathrm{KSD}^2(q\|\pi) = \mathbb{E}_{(\theta,\theta')\sim q\otimes q}[\,k_\pi(\theta,\theta')], \tag{16.17}$$

where

$$k_\pi(\theta,\theta') := u_\pi^\top(\theta)k(\theta,\theta')\,u_\pi(\theta') + u_\pi^\top(\theta)\nabla_{\theta'}k(\theta,\theta') + \nabla_\theta^\top k(\theta,\theta')u_\pi(\theta') + \mathrm{Tr}(\nabla_\theta\nabla_{\theta'}k(\theta,\theta'))$$

is the *Stein kernel*. Given a set of particles $\{\theta^i\}_{i=1}^n \sim q$, a (biased) V-statistic estimator for the KSD is then given by

$$\widehat{\mathrm{KSD}^2(q\|\pi)} = \frac{1}{n^2}\sum_{i,j=1}^n k_\pi(\theta^i,\theta^j), \tag{16.18}$$

with $\mathcal{O}(n^2)$ cost. As in the case of the maximum mean discrepancy (MMD) [667], an unbiased U-statistic variant is also available [1066]. Given i.i.d. samples from q, (16.18) is a biased estimator of $\mathrm{KSD}^2(q\|\pi)$; for an interacting particle system, the same expression is more naturally interpreted as the squared KSD of the empirical measure $q^n = \frac{1}{n}\sum_{i=1}^n \delta_{\theta^i}$.

The KSD has a number of special properties which make it particularly useful in the context of Bayesian deep learning:

- **Normalisation-free.** Only $u_\pi = \nabla \log \pi$ is needed from the target π and the normalising constant Z cancels.
- **Convergence-determining.** For suitable kernels (e.g., the inverse multi-quadric (IMQ) kernel with appropriate bandwidth) and mild tail conditions on π, $\mathrm{KSD}(q\|\pi) \to 0$ implies $q \Rightarrow \pi$ [307, 652].
- **Algorithmic role.** By (16.16), KSD quantifies the norm of the steepest-descent direction within $\mathcal{H}^d$. Its Riesz representer $\mu_{q,\pi}$ is exactly the vector field used by SVGD.

The Kernel Stein Discrepancy as the Direction of Steepest Descent. Returning to (16.16), we see that the maximiser of the constrained optimisation problem in (16.11) is given by the normalised witness function

$$\bar{\phi}_{q,\pi} = \arg\max_{\phi \in \mathcal{H}^d} \{\mathbb{E}_q[T_\pi \phi] \ : ||\phi||_{\mathcal{H}^d} \leq 1\} = \frac{\mu_{q,\pi}}{||\mu_{q,\pi}||_{\mathcal{H}^d}}, \tag{16.19}$$

assuming $\mu_{q,\pi} \neq 0$; the corresponding maximum value is precisely $\mathrm{KSD}(q\|\pi) = \|\mu_{q,\pi}\|_{\mathcal{H}^d}$. This characterises the direction of steepest descent of the KL divergence in the vector-valued RKHS $\mathcal{H}^d$. In practice, the SVGD algorithm introduced in the next section uses the *unnormalised* witness function, $\phi^*_{q,\pi} := \mu_{q,\pi}$, which points in the same direction; with this choice, the instantaneous rate of decrease of the KL divergence is $-\|\mu_{q,\pi}\|^2_{\mathcal{H}^d} = -\mathrm{KSD}^2(q\|\pi)$.

16.3 Stein Variational Gradient Descent (SVGD)

16.3.1 Basic Overview

In Section 16.2 we established that, given an infinitesimal transport map $T_\varepsilon(\theta) = \theta + \varepsilon\phi(\theta)$, which we recall satisfies

$$\frac{\mathrm{d}}{\mathrm{d}\varepsilon} \mathrm{D}_{\mathrm{KL}}((T_\varepsilon)_\# q\|\pi)\Big|_{\varepsilon=0} = -\mathbb{E}_q[T_\pi \phi(\theta)], \qquad T_\pi \phi := \nabla \cdot \phi + \phi^\top \nabla \log \pi, \tag{16.20}$$

the direction of *steepest descent* that maximises the negative gradient in (16.20) amongst all perturbations lying in the ball $\{\phi \in \mathcal{H}^d : \|\phi\|_{\mathcal{H}^d} \leq 1\}$ of the vector-valued RKHS $\mathcal{H}^d$, is proportional to the witness function $\mu_{q,\pi}$, cf. (16.15). Following standard practice, we will work with the unnormalised direction

$$\phi^\star_{q,\pi}(\cdot) = \mathbb{E}_{\theta\sim q}\left[k(\theta,\cdot)u_\pi(\theta) + \nabla_\theta k(\theta,\cdot)\right]. \tag{16.21}$$

This result suggests the following iterative procedure for transforming an initial measure q_0 into the target posterior distribution π: for $k = 0, 1, \ldots$

$$q_{k+1} = \left(T^*_{\varepsilon_k}\right)_\# q_k, \quad \text{where } \ T^*_{\varepsilon_k}(\theta) = \theta + \varepsilon_k \phi^*_{q_k,\pi}(\theta). \tag{16.22}$$

We can provide an equivalent description of (16.22) at the level of a single particle. In particular, for all $k = 0, 1, \ldots$, we have $q_k = \mathrm{Law}(\theta_k)$, where $(\theta_k)_{k\in\mathbb{N}}$ are defined according to[1]

$$\theta_{k+1} = \theta_k + \varepsilon_k \mathbb{E}_{\theta\sim q_k}\left[k(\theta,\theta_k)u_\pi(\theta) + \nabla_\theta k(\theta,\theta_k)\right], \quad \theta_0 \sim q_0. \tag{16.23}$$

We identify two terms in the expectation in (16.23), each of which will play a different role in the updates:

$$\underbrace{\mathbb{E}_{\theta\sim q_k}[k(\theta,\theta_k)\nabla_\theta \log \pi(\theta)]}_{\text{attraction to high } \pi} + \underbrace{\mathbb{E}_{\theta\sim q_k}[\nabla_\theta k(\theta,\theta_k)]}_{\text{repulsion \& diversity}}. \tag{16.24}$$

The first term drives particles towards regions of high probability by following a smoothed version of the gradient of the log-density. The second term is a repulsive force which prevents all particles from collapsing to a single point.

[1]Note a slight overload of notation: $k(\cdot,\cdot)$ denotes the kernel, while the subscript k indexes the iteration. The intended meaning will always be clear from context.

Functional Gradient Descent Interpretation. The updates defined by (16.22) can be viewed as a form of functional gradient descent in the vector-valued RKHS $\mathcal{H}^d$ [1064, Theorem 3.3]. Recall that, for any functional $\mathcal{F} : \mathcal{H}^d \to \mathbb{R}$, the *first variation* $D\mathcal{F}(f)(g)$ at $f \in \mathcal{H}^d$ along $g \in \mathcal{H}^d$ can be defined as

$$\mathrm{D}\mathcal{F}(f)(g) := \lim_{\varepsilon \to 0} \frac{1}{\varepsilon} \left(\mathcal{F}(f + \varepsilon g) - \mathcal{F}(f) \right). \tag{16.25}$$

Meanwhile, assuming $\mathcal{F}$ is Fréchet differentiable, its *functional gradient* $\nabla_f \mathcal{F}(f) : \mathbb{R}^d \to \mathbb{R}^d$ is the element in $\mathcal{H}^d$ such that, for all $g \in \mathcal{H}^d$:[2]

$$\mathrm{D}\mathcal{F}(f)(g) = \langle \nabla_f \mathcal{F}(f), g \rangle_{\mathcal{H}^d}. \tag{16.27}$$

Consider the transport map $T_f = \mathrm{id} + f$, where id is the identity map, and $f \in \mathcal{H}^d$. Define the functional

$$\mathcal{F}_{q,\pi}(f) = \mathrm{D}_{\mathrm{KL}}\Big(T_{f\,\#} q \Big\| \pi \Big). \tag{16.28}$$

Then it is possible to show that the functional gradient $\nabla_f \mathcal{F}_{q,\pi}(f)$, evaluated at the zero map $f = 0$, satisfies [1064, Theorem 3.3]

$$\nabla_f \mathcal{F}_{q,\pi}(0) = -\phi^*_{q,\pi}. \tag{16.29}$$

Thus, the transport maps $T^*_{\varepsilon_k}(\theta)$ used to define the sequence of distributions $(q_k)_{k \in \mathbb{N}}$ in (16.22), viz

$$T^*_{\varepsilon_k}(\theta) = \theta + \varepsilon_k \phi^*_{q_k,\pi}(\theta) = \theta - \varepsilon_k \nabla_f \mathcal{F}_{q_k,\pi}(0)(\theta) \tag{16.30}$$

can be thought of as performing a sequence of steps of functional gradient descent in the RKHS.

Continuous-Time Limit. The updates in (16.22) and (16.23) can also be viewed as a time-discretisation of a continuous-time process referred to as the *Stein variational gradient flow*. In particular, as the step-size $\varepsilon_k \to 0$, the evolution of the densities $(q_k)_{k \in \mathbb{N}}$ defined by (16.22) is described by the continuity equation [1063, Section 3.3]

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot (q_t(\theta) \phi^*_{q_t,\pi}(\theta)), \qquad \phi^*_{q_t,\pi}(\cdot) = \mathbb{E}_{\theta \sim q_t}[k(\theta, \cdot) u_\pi(\theta) + \nabla_\theta k(\theta, \cdot)]. \tag{16.31}$$

Equivalently, the evolution of the particle $(\theta_k)_{k \in \mathbb{N}}$ defined by (16.23) is described by the nonlinear ODE

$$\mathrm{d}\theta_t = \phi^*_{q_t,\pi}(\theta_t)\mathrm{d}t, \qquad \phi^*_{q_t,\pi}(\cdot) = \mathbb{E}_{\theta \sim q_t}[k(\theta, \cdot) u_\pi(\theta) + \nabla_\theta k(\theta, \cdot)]. \tag{16.32}$$

One can view the updates in (16.22) or (16.23) as a forward Euler discretisation of the dynamics in (16.31) and (16.32), respectively.

Basic Convergence Results. The idea behind (16.22) or (16.23) is that, for sufficiently small step sizes $(\varepsilon_k)_{k \in \mathbb{N}_0}$, these updates decrease the KL divergence [1063, Theorem 3.3]. Under additional assumptions on the target, kernel, and limiting dynamics, this descent can be used to establish convergence to π; see Section 16.8. Indeed, under suitable assumptions, it is possible to show that [918, Proposition 5]

$$\mathrm{D}_{\mathrm{KL}}(q_{k+1} \| \pi) - \mathrm{D}_{\mathrm{KL}}(q_k \| \pi) \leq -\varepsilon_k \mathrm{KSD}^2(q_k \| \pi) + \mathcal{O}(\varepsilon_k^2). \tag{16.33}$$

Thus, for a sufficiently small step size ε_k, the KL divergence between the current distribution q_k and the target distribution π does indeed decrease at each iteration.

16.3.2 Particle-Based Approximation

In the kth iteration, (16.23) requires computing an expectation with respect to the (unknown) distribution q_k. We will thus need to approximate this term. In order to do so, we can use an empirical approximation. In particular, let $\{\theta_0^i\}_{i=1}^n$ denote a set of particles drawn i.i.d. from the initial distribution q_0. We will then update, for $k = 0, 1, 2, \ldots$, and $i = 1, \ldots, n$,

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \mathbb{E}_{\theta \sim q_k^n} \left[k(\theta, \theta_k^i) u_\pi(\theta) + \nabla_\theta k(\theta, \theta_k^i) \right], \tag{16.34}$$

[2] Equivalently:

$$\mathcal{F}(f + \varepsilon g) = \mathcal{F}(f) + \varepsilon \langle \nabla_f \mathcal{F}(f), g \rangle_{\mathcal{H}^d} + \mathcal{O}(\varepsilon^2). \tag{16.26}$$

Algorithm 20: Stein Variational Gradient Descent (SVGD)

Inputs : target score $u_\pi(\theta) = \nabla \log \pi(\theta)$; kernel k; step sizes $\{\varepsilon_k\}_{k\in\mathbb{N}}$; particles $\{\theta_0^i\}_{i=1}^n$
Outputs: updated particles $\{\theta_K^i\}_{i=1}^n$
for $k = 0$ **to** $K-1$ **do**
  compute scores $g_j \leftarrow u_\pi(\theta_k^j)$ for $j = 1, \dots, n$
  for $i = 1$ **to** n **do**
    $\phi^i \leftarrow \frac{1}{n}\sum_{j=1}^n \left[k(\theta_k^j, \theta_k^i) g_j + \nabla_{\theta_k^j} k(\theta_k^j, \theta_k^i)\right]$
    $\theta_{k+1}^i \leftarrow \theta_k^i + \varepsilon_k \phi^i$
return $\{\theta_K^i\}_{i=1}^n$

where $q_k^n = \frac{1}{n}\sum_{i=1}^n \delta_{\theta_k^i}$ denotes the empirical distribution of the particles $\{\theta_k^i\}_{i=1}^n$ in the kth iteration. That is, for $k = 0, 1, 2, \dots$ and $i = 1, \dots, n$,

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \frac{1}{n}\sum_{j=1}^n \left[k(\theta_k^j, \theta_k^i) u_\pi(\theta_k^j) + \nabla_{\theta_k^j} k(\theta_k^j, \theta_k^i)\right]. \tag{16.35}$$

This algorithm, summarised in Algorithm 20, is known as *Stein variational gradient descent* (SVGD).

16.3.3 Complexity and Practical Implementation

Computational Complexity. For typical choices of the kernel k, each iteration in (16.35) requires the computation of all pairwise interactions. The computational cost thus scales as $\mathcal{O}(n^2)$ w.r.t. the number of kernel (and kernel gradient) evaluations, and as $\mathcal{O}(n)$ w.r.t. the number of score function evaluations. Given a large number of particles, or large datasets, there are several ways to reduce the computational cost:

- **Mini-batch scores.** Estimate the score function $u_\pi(\theta) := \nabla_\theta \log \pi(\theta)$ using mini-batches of the data. This yields a stochastic but unbiased approximation (cf. Section 16.6).
- **Kernel approximations.** Use random Fourier features to reduce the $\mathcal{O}(n^2)$ interaction to $\mathcal{O}(nm)$ or $\mathcal{O}(n \log n)$ [1065]; see Section 16.6.3 for further details.
- **Parallelism**. Use parallel computing, noting that both the kernel matrix and sums are data-parallel and GPU-friendly.

Bandwidth Selection. The choice of the kernel bandwidth critically affects the balance between the attractive and repulsive terms in the update equation, and thus the dispersal of particles. Adaptive schedules (e.g., the median heuristic) are commonly employed.

Step Sizes and Stopping Criteria. As with standard gradient descent, the step sizes $(\varepsilon_k)_{k\in\mathbb{N}}$ must also be chosen with some care: if too large, the discretisation of the underlying gradient flow becomes unstable; if too small, convergence is unnecessarily slow. In practice, a small constant step size, or an adaptive scheme such as AdaGrad, is often used [1064]. Alternatively, learning-rate-free variants of SVGD, based on ideas from parameter-free online optimisation (so-called coin betting), remove the need to select a step-size schedule altogether [1503], and can be extended to constrained domains [1506]. More recently, adaptive step-size schedules derived from the gradient flow perspective (see Section 16.4) have been obtained for SVGD and related sampling algorithms, with convergence guarantees matching those of their optimally tuned counterparts up to logarithmic factors [1504]. Regarding termination, there is no universally agreed stopping criterion: common choices include running for a fixed computational budget, or monitoring the magnitude of the particle updates or an empirical estimate of the KSD, cf. (16.18), and stopping when these fall below a threshold.

16.4 Particle-Based Variational Inference (ParVI) as Gradient Flow*

In Section 16.3, we derived the SVGD algorithm, and showed how it could be viewed as a time discretisation, combined with a particle approximation of the evolving measure, of a continuous-time, mean-field process known

as the Stein variational gradient flow. In this section, we show that this is just one example of a *gradient flow over the space of probability measures*. We provide a general recipe for constructing such gradient flows, and describe several examples, including the Stein variational gradient flow as a special case. We also show how discretising these gradient flows in time, and approximating the evolving measure with particles, yields practical ParVI algorithms.

16.4.1 Basics of Gradient Flows

In this section, we introduce the notion of a gradient flow over the space of probability measures. Our exposition relies on the formal identification of the space of probability measures with an (infinite-dimensional) Riemannian manifold. Thus, we begin by recalling some basic concepts from Riemannian geometry.

Manifold, Tangent Space, Cotangent Space. For convenience, in this section we will consider the manifold of smooth, strictly positive densities over $\mathbb{R}^d$, viz $\mathcal{P} = \left\{q \in C^\infty(\mathbb{R}^d) : \int q \mathrm{d}\theta = 1, q > 0\right\}$. For each $q \in \mathcal{P}$, the *tangent space* of $\mathcal{P}$ is given by $\mathcal{T}_q\mathcal{P} \subseteq \left\{\sigma \in C^\infty(\mathbb{R}^d) : \int \sigma \mathrm{d}\theta = 0\right\}$. We will write $\mathcal{T}_q^\star\mathcal{P}$ for the cotangent space, the topological dual of $T_q\mathcal{P}$.

Metric Tensor, Onsager Operator, Riemannian Metric. Let $M(q) : \mathcal{T}_q\mathcal{P} \to \mathcal{T}_q^\star\mathcal{P}$ denote a *metric tensor*, which maps elements in the tangent space $\mathcal{T}_q\mathcal{P}$ to elements in the cotangent space $\mathcal{T}_q^\star\mathcal{P}$. We can then define a *Riemannian metric* $g_q : \mathcal{T}_q\mathcal{P} \times \mathcal{T}_q\mathcal{P} \to \mathbb{R}$ according to $g_q(\sigma_1, \sigma_2) = \langle M(q)\sigma_1, \sigma_2\rangle$, where $\langle\cdot,\cdot\rangle$ is the *duality pairing* from $\mathcal{T}_q^*\mathcal{P} \times \mathcal{T}_q\mathcal{P} \to \mathbb{R}$. The inverse of the metric tensor $M(q)$, denoted $M^{-1}(q) : \mathcal{T}_q^*\mathcal{P} \to \mathcal{T}_q\mathcal{P}$, is referred to as the *Onsager operator*.

First Variation, Riemannian Gradient. We next recall the definition of the *first variation* of $\mathcal{F}$ w.r.t. q. This is the unique function in $\mathcal{T}_q^*\mathcal{P}$ such that

$$\lim_{\varepsilon \to 0} \frac{1}{\varepsilon} \left[\mathcal{F}(q + \varepsilon\sigma) - \mathcal{F}(q)\right] = \left\langle \frac{\delta \mathcal{F}(q)}{\delta q}, \sigma \right\rangle, \tag{16.36}$$

for all $\sigma \in \mathcal{T}_q\mathcal{P}$. The gradient of $\mathcal{F}$ w.r.t. the Riemannian metric g_q, denoted $\mathrm{grad}\mathcal{F}$, is then defined via the condition

$$g_q\left(\mathrm{grad}\mathcal{F}, \sigma\right) = \left\langle \frac{\delta \mathcal{F}}{\delta q}, \sigma \right\rangle, \tag{16.37}$$

for all $\sigma \in \mathcal{T}_q\mathcal{P}$. Using the characterisation of the Riemannian metric in terms of the Onsager operator (see above), it is possible to write the Riemannian gradient as $\mathrm{grad}\mathcal{F}(q) = M^{-1}(q)\frac{\delta \mathcal{F}}{\delta q}$.

Gradient Flows. We are now ready to define the *gradient flow* of $\mathcal{F}$ over the space $(\mathcal{P}, g)$. It is a curve $t \mapsto q_t$ that satisfies

$$\frac{\partial q_t}{\partial t} = -\mathrm{grad}\mathcal{F}(q_t) := -M^{-1}(q_t) \left.\frac{\delta \mathcal{F}}{\delta q}\right|_{q=q_t}. \tag{16.38}$$

Provided the flow is well posed, and under additional assumptions on the functional $\mathcal{F}$ and the geometry (e.g., suitable convexity, coercivity, or functional inequalities; see below), one can often establish convergence of (16.38) to a minimiser of the functional $\mathcal{F}$; for the KL objective considered below, the target π is this minimiser, i.e., $q_t \to \pi$ as $t \to \infty$. This gradient flow thus provides us with a general method for transporting an initial reference distribution q_0 to a target posterior distribution π. In particular, we obtain different algorithms for sampling from the target posterior distribution according to different choices of:

1. the *objective functional* $\mathcal{F}(q)$ (e.g., KL divergence, other f-divergence, KSD);
2. the *geometry* over $\mathcal{P}$ (e.g., Wasserstein, Stein, Fisher-Rao);
3. the *time-discretisation* (e.g., forward Euler, backward Euler, forward flow);
4. the *space-discretisation* (e.g., finite element, particle-based approximation).

The Objective Functional. In order to simplify our discussion, we will assume throughout this section that the objective functional is the KL divergence: $\mathcal{F}(q) = \mathrm{D}_{\mathrm{KL}}(q\|\pi)$. In this case, we can calculate the first variation of $\mathcal{F}$ with respect to q as [e.g., 27]

$$\frac{\delta \mathcal{F}}{\delta q}(\theta) = \log \frac{q(\theta)}{\pi(\theta)} + 1 = \log q(\theta) - \log \pi(\theta) + 1. \tag{16.39}$$

This choice is well motivated: among all f-divergences with continuously differentiable f defined on the positive reals, the KL divergence is the only one whose first variation is unchanged (up to constants) if we scale π by a positive constant [282, Theorem 2.2].

Other choices of objective functional are, of course, possible, and lead to different ParVI algorithms for sampling from π. These include the kernel Stein discrepancy [919], the mollified interacting energy (MIE) [1030], and the χ^2-divergence [293].

16.4.2 Wasserstein Gradient Flow

The Geometry. We begin by considering the gradient flow induced by the quadratic Wasserstein metric. In order to define the Wasserstein metric tensor, let us first define ψ_σ as the solution of the elliptic PDE

$$-\nabla_\theta \cdot (q \nabla_\theta \psi_\sigma) = \sigma. \tag{16.40}$$

The Wasserstein metric tensor $M(q) : \mathcal{T}_q\mathcal{P} \to \mathcal{T}_q^*\mathcal{P}$ and its inverse $M^{-1}(q) : \mathcal{T}_q^*\mathcal{P} \to \mathcal{T}_q\mathcal{P}$ are then defined according to [1272, 1679]

$$M(q)\sigma = \psi_\sigma, \quad \text{for all } \sigma \in \mathcal{T}_q\mathcal{P} \tag{16.41}$$

$$M^{-1}(q)\psi = -\nabla_\theta \cdot (q \nabla_\theta \psi), \quad \text{for all } \psi \in \mathcal{T}_q^*\mathcal{P}. \tag{16.42}$$

Meanwhile, following some elementary calculations, we identify the Wasserstein-Riemannian metric $g_q : \mathcal{T}_q\mathcal{P} \times \mathcal{T}_q\mathcal{P} \to \mathbb{R}$ as

$$g_q(\sigma_1, \sigma_2) = \int \nabla_\theta \psi_{\sigma_1}(\theta)^\top \nabla_\theta \psi_{\sigma_2}(\theta) q(\theta) \mathrm{d}\theta. \tag{16.43}$$

The Gradient Flow. Substituting (16.42) into (16.38), and rearranging, we see that the Wasserstein gradient flow is given by the continuity equation

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot \big(q_t(\theta)\, v_t(\theta)\big), \quad v_t(\theta) = -\nabla_\theta \left.\frac{\delta \mathcal{F}}{\delta q}\right|_{q=q_t}(\theta). \tag{16.44}$$

In addition, substituting the first-variation of the KL divergence from (16.39) into (16.44) yields the Wasserstein gradient flow of the KL divergence as

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot \left(q_t(\theta) \nabla_\theta \log \frac{\pi(\theta)}{q_t(\theta)}\right) \tag{16.45}$$

$$= -\nabla_\theta \cdot \big(q_t(\theta) \left(\nabla_\theta \log \pi(\theta) - \nabla_\theta \log q_t(\theta)\right)\big). \tag{16.46}$$

Formally, we can also provide a description of the gradient flow in (16.46) at the level of a single particle, the so-called *Lagrangian description*. In particular, for all $t \geq 0$, it holds that $q_t = \mathrm{Law}(\theta_t)$, where $(\theta_t)_{t\geq 0}$ is the solution of

$$\dot{\theta}_t = \nabla_\theta \log \pi(\theta_t) - \nabla_\theta \log q_t(\theta_t), \quad \theta_0 \sim q_0. \tag{16.47}$$

This equation, sometimes referred to as the *deterministic Langevin equation* [675], is *non-linear* in the sense that the time evolution depends not only on the process θ_t, but also on its distribution q_t.

Alternatively, observe that the second term in (16.46) is equal to $\Delta_\theta q_t$, where Δ_θ denotes the Laplacian. We can thus rewrite (16.46) as

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot (q_t(\theta) \nabla_\theta \log \pi(\theta)) + \Delta_\theta q_t(\theta). \tag{16.48}$$

This is precisely the *Fokker–Planck equation* describing the time evolution of the marginal law of the overdamped Langevin dynamics w.r.t. π. In other words, for all $t \geq 0$, it holds that $q_t = \mathrm{Law}(\theta_t)$, where now $(\theta_t)_{t\geq 0}$ is defined as the solution of the *stochastic differential equation*

$$\mathrm{d}\theta_t = \nabla \log \pi(\theta_t)\mathrm{d}t + \sqrt{2}\mathrm{d}w_t, \quad \theta_0 \sim q_0, \tag{16.49}$$

where $w = (w_t)_{t\geq 0}$ is a standard $\mathbb{R}^d$-valued Brownian motion. Thus, one can view the overdamped Langevin dynamics as the gradient flow of the KL divergence with respect to the quadratic Wasserstein metric [852].

Convergence Rate. It is straightforward to obtain the dissipation of the KL divergence along the WGF. In particular, using the chain rule, we have that

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathrm{D}_{\mathrm{KL}}(q_t\|\pi) = -\int q_t(\theta)\|\nabla_\theta \log \tfrac{q_t(\theta)}{\pi(\theta)}\|^2 \mathrm{d}\theta = -\mathcal{I}(q_t\|\pi) \leq 0, \tag{16.50}$$

where $\mathcal{I}(q\|\pi)$ is the *relative Fisher information* of q with respect to π. Thus, $\mathrm{D}_{\mathrm{KL}}(q_t\|\pi)$ dissipates at a rate equal to the relative Fisher information. If, in addition, the target distribution π satisfies a log-Sobolev inequality with constant λ, then this inequality implies exponential convergence to the target distribution, at a rate determined by λ: $\mathrm{D}_{\mathrm{KL}}(q_t\|\pi) \leq e^{-2\lambda t}\,\mathrm{D}_{\mathrm{KL}}(q_0\|\pi)$ [e.g., 292].

Practical Algorithms. In the previous section, we obtained two different differential equations, cf. (16.47) and (16.49), one of them stochastic, both of whose marginal distributions follow the Wasserstein gradient flow of the KL divergence, cf. (16.46). These equations suggest different algorithms for sampling from the target distribution π.

We first consider (16.49), which easily yields a practical sampling algorithm. In particular, using an Euler-Maruyama discretisation, we can define $(\theta_k)_{k=0,1,2,\ldots}$ according to the update equation

$$\theta_{k+1} = \theta_k + \varepsilon_k \nabla_\theta \log \pi(\theta_k) + \sqrt{2\varepsilon_k} z_k, \quad \theta_0 \sim q_0, \tag{16.51}$$

where $(\varepsilon_k)_{k=0,1,\ldots,}$ is a positive, non-increasing sequence of step sizes; and where $(z_k)_{k=0,1,2,\ldots}$ are i.i.d. standard normal random variables. The algorithm in (16.51) is nothing more than the *unadjusted Langevin algorithm* (ULA), also known as *Langevin Monte Carlo* (LMC), a popular MCMC algorithm [472]; its stochastic gradient variants are discussed in detail in Chapter 2, and in the recent monograph of Fearnhead et al. [509]. Despite its connection with the reformulation of sampling as an optimisation problem, this is not typically viewed as a "particle-based variational inference" algorithm, since it does not necessitate the use of an ensemble of particles.

Now consider (16.47). In this case, a forward Euler discretisation defines a sequence $(\theta_k)_{k=0,1,\ldots,}$ according to

$$\theta_{k+1} = \theta_k + \varepsilon_k(\nabla_\theta \log \pi(\theta_k) - \nabla_\theta \log q_k(\theta_k)), \quad \theta_0 \sim q_0. \tag{16.52}$$

One cannot implement this directly, since it depends on the unknown sequence of distributions $(q_k)_{k=0,1,\ldots}$. Instead, one option is to evolve an ensemble of particles $(\theta_k^i)_{k=0,1,\ldots}^{i=1,\ldots,n}$, and update

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \left(\nabla_\theta \log \pi(\theta_k^i) - \widehat{\nabla_\theta \log q_k}(\theta_k^i)\right), \quad \theta_0^i \overset{i.i.d.}{\sim} q_0, \tag{16.53}$$

where $\widehat{\nabla_\theta \log q_k}$ denotes an approximation of $\nabla_\theta \log q_k$, formed using the particles $\{\theta_k^i\}_{i=1}^n$. There are various options for this approximation, including approximating the density q_k directly using a kernel-density estimate or a normalising flow [675], and approximating the score $\nabla \log q_k$ using score matching [179].

16.4.3 Stein Gradient Flow

We now provide a more rigorous interpretation of the mean-field, continuous-time limit of the SVGD algorithm, cf. (16.31), as a gradient flow over the space of probability measures equipped with a suitably chosen Riemannian metric.

The Geometry. Similar to before, cf. (16.40), to define the Stein metric tensor we will first introduce ψ_σ as the solution of the integro-partial differential equation

$$-\nabla_\theta \cdot \left(q(\theta) \int k(\theta', \theta) \nabla_{\theta'} \psi_\sigma(\theta') q(\theta') \mathrm{d}\theta'\right) = \sigma(\theta), \tag{16.54}$$

where $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a symmetric positive-definite kernel. We can then define the *Stein metric tensor* and its inverse as [282, 462]

$$M(q)\sigma = \psi_\sigma, \quad \text{for all } \sigma \in \mathcal{T}_q\mathcal{P} \tag{16.55}$$

$$M^{-1}(q)\psi = -\nabla_\theta \cdot \left(q(\theta) \int k(\theta', \theta) \nabla_{\theta'} \psi(\theta') q(\theta') \mathrm{d}\theta'\right), \quad \text{for all } \psi \in \mathcal{T}_q^*\mathcal{P}. \tag{16.56}$$

It follows that the Stein-Riemannian metric associated with this metric tensor is given by

$$g_q(\sigma_1, \sigma_2) = \int \nabla_\theta \psi_{\sigma_1}^\top(\theta) \left[\int k(\theta', \theta) \nabla_{\theta'} \psi_{\sigma_2}(\theta') q(\theta') \mathrm{d}\theta'\right] q(\theta) \mathrm{d}\theta. \tag{16.57}$$

The Gradient Flow. Similar to before, substituting (16.56) into (16.38), and rearranging, we see that the Stein gradient flow is given by the continuity equation

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot \big(q_t(\theta)\, v_t(\theta)\big), \quad v_t(\theta) = -\int k(\theta', \theta) \nabla_{\theta'} \left.\frac{\delta \mathcal{F}}{\delta q}\right|_{q=q_t} (\theta')\, q_t(\theta') \mathrm{d}\theta'. \tag{16.58}$$

In addition, substituting the first-variation of the KL divergence from (16.39) into (16.58) yields the Stein gradient flow of the KL divergence (i.e., the Stein variational gradient flow) as

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot \left(q_t(\theta) \int k(\theta', \theta) \nabla_{\theta'} \log \frac{\pi(\theta')}{q_t(\theta')} q_t(\theta') \mathrm{d}\theta' \right) \tag{16.59}$$

$$= -\nabla_\theta \cdot \left(q_t(\theta) \int k(\theta', \theta) \left[\nabla_{\theta'} \log \pi(\theta') - \nabla_{\theta'} \log q_t(\theta')\right] q_t(\theta') \mathrm{d}\theta' \right). \tag{16.60}$$

Finally, assuming appropriate boundary conditions on k, and integrating by parts, we arrive back at

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot (q_t(\theta) \phi^*_{q_t,\pi}(\theta)), \qquad \phi^*_{q_t,\pi}(\cdot) = \mathbb{E}_{\theta \sim q_t}[k(\theta, \cdot) \nabla_\theta \log \pi(\theta) + \nabla_\theta k(\theta, \cdot)]. \tag{16.61}$$

Convergence Rate. Using an appropriate version of the chain rule, one can show that the dissipation of the KL divergence along the Stein variational gradient flow is given by [918, Proposition 1]

$$\frac{\mathrm{d}}{\mathrm{d}t} \mathrm{D}_{\mathrm{KL}}(q_t \| \pi) = -\langle \phi_t, \mu_{q_t,\pi} \rangle_{\mathcal{H}^d} = -\|\mu_{q_t,\pi}\|^2_{\mathcal{H}^d} = -\mathcal{I}_{\mathrm{Stein}}(q_t \| \pi) := -\mathrm{KSD}^2(q_t \| \pi) \leq 0. \tag{16.62}$$

Thus, $\mathrm{D}_{\mathrm{KL}}(q_t \| \pi)$ is non-increasing, and dissipates at a rate equal to the *Stein-Fisher information*, i.e., the squared KSD. Similar to before, if π satisfies a *Stein log-Sobolev inequality* with constant λ, then one has exponential convergence: $\mathrm{D}_{\mathrm{KL}}(q_t \| \pi) \leq \mathrm{D}_{\mathrm{KL}}(q_0 \| \pi) e^{-2\lambda t}$.

16.4.4 Fisher-Rao and Wasserstein-Fisher-Rao Gradient Flow

The Wasserstein geometry and the Stein geometry both result in gradient flows which *transport mass*. On the other hand, the Fisher-Rao geometry leads to a gradient flow which changes mass locally through birth-death dynamics rather than spatial transport; informally, mass can be removed in one region and created in another, as if it were *teleported*.

The Geometry. The Fisher-Rao Riemannian metric $g_q : \mathcal{T}_q\mathcal{P} \times \mathcal{T}_q\mathcal{P} \to \mathbb{R}$ is defined according to

$$g_q(\sigma_1, \sigma_2) = \int \frac{\sigma_1 \sigma_2}{q} \mathrm{d}\theta, \tag{16.63}$$

or, writing $\sigma = q\psi_\sigma$,

$$g_q(\sigma_1, \sigma_2) = \int \psi_{\sigma_1} \psi_{\sigma_2} q \mathrm{d}\theta. \tag{16.64}$$

Meanwhile the Fisher-Rao metric tensor $M(q) : \mathcal{T}_q\mathcal{P} \to \mathcal{T}_q^*\mathcal{P}$ associated to g_q, and its inverse $M^{-1}(q) : \mathcal{T}_q^*\mathcal{P} \to \mathcal{T}_q\mathcal{P}$, are given by

$$M(q)\sigma = \psi_\sigma = \frac{\sigma}{q}, \quad \text{for all } \sigma \in \mathcal{T}_q\mathcal{P} \tag{16.65}$$

$$M^{-1}(q)\psi = q\left(\psi - \mathbb{E}_q[\psi]\right), \quad \text{for all } \psi \in \mathcal{T}_q^*\mathcal{P}. \tag{16.66}$$

Strictly speaking, cotangent potentials are identified up to additive constants; equivalently, one may work with the centred representative $\psi - \mathbb{E}_q[\psi]$, in which case $M(q)$ and $M^{-1}(q)$ are strict inverses.

The Gradient Flow. It follows, substituting (16.66) into (16.38), that the Fisher-Rao gradient flow reads

$$\frac{\partial q_t(\theta)}{\partial t} = -q_t(\theta) \left(\left.\frac{\delta \mathcal{F}}{\delta q}\right|_{q=q_t} (\theta) - \mathbb{E}_{\theta \sim q_t} \left[\left.\frac{\delta \mathcal{F}}{\delta q}\right|_{q=q_t} (\theta) \right] \right). \tag{16.67}$$

Thus, substituting also (16.39), the Fisher-Rao gradient flow of the KL divergence is given by

$$\frac{\partial q_t(\theta)}{\partial t} = q_t(\theta)\Big[\log \pi(\theta) - (\log q_t(\theta)) - \mathbb{E}_{\theta\sim q_t}\left[\log \pi(\theta) - \log q_t(\theta)\right]\Big]. \tag{16.68}$$

This gradient flow is an example of a *birth-death process*. Since there is no spatial derivative, it can be viewed as an infinite system of ordinary differential equations, indexed by the parameter. The advantage of such a process is that it allows global moves (i.e., teleportation) of the mass of the probability density (e.g., from one mode to another) according to their relative weights, without the need to transport this mass through low probability regions. The disadvantage is that, due to the absence of diffusion, the support of q_t never changes during the evolution. This means that effective numerical simulation of the Fisher-Rao gradient flow is not straightforward.

One solution is to combine the Wasserstein and Fisher-Rao geometries into a hybrid geometry, which incorporates both mass transport and teleportation. In particular, the gradient flow w.r.t. the *Wasserstein-Fisher-Rao* geometry [1087] reads

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot \big(q_t(\theta)v_t(\theta)\big) + q_t(\theta)\alpha_t(\theta), \tag{16.69}$$

where

$$v_t(\theta) = -\nabla_\theta \frac{\delta\, \mathrm{D}_{\mathrm{KL}}(q_t\|\pi)}{\delta q_t} = \nabla \log \pi(\theta) - \nabla \log q_t(\theta) \tag{16.70}$$

$$\begin{aligned}\alpha_t(\theta) &= -\left(\frac{\delta\, \mathrm{D}_{\mathrm{KL}}(q_t\|\pi)}{\delta q_t} - \mathbb{E}_{q_t}\left[\frac{\delta\, \mathrm{D}_{\mathrm{KL}}(q_t\|\pi)}{\delta q_t}\right]\right) \\ &= (\log \pi(\theta) - \log q_t(\theta)) - \mathbb{E}_{q_t}[\log \pi(\theta) - \log q_t(\theta)].\end{aligned} \tag{16.71}$$

Convergence Rate. Under mild conditions on π and q_0, it is possible to show that the asymptotic convergence rate of the Wasserstein-Fisher-Rao gradient flow is *independent* of the target distribution π. In particular, assuming q_0 and π both have bounded second moments, and that their ratio satisfies a certain growth condition, it holds that $\mathrm{D}_{\mathrm{KL}}(q_t\|\pi) \leq Ce^{-t}$ for all $t \geq t_0$ [283, Theorem 4.1].

16.5 Second-Order Methods and Riemannian ParVI*

In Section 16.3, we saw that SVGD can be viewed as performing a form of functional gradient descent w.r.t. the KL divergence on a vector-valued RKHS. In particular, defining the functional

$$\mathcal{F}_{q,\pi}(f) = \mathrm{D}_{\mathrm{KL}}\Big((\mathrm{id} + f)_{\#}q\Big\|\pi\Big), \quad f \in \mathcal{H}^d, \tag{16.72}$$

then the (infinite-population version) of the SVGD algorithm was defined via the updates

$$q_{k+1} = \big(T^*_{\varepsilon_k}\big)_{\#}\, q_k, \quad \text{where } \; T^*_{\varepsilon_k} = \mathrm{id} - \varepsilon_k \nabla_f \mathcal{F}_{q_k,\pi}(0). \tag{16.73}$$

Based on this perspective of SVGD, there are various ways to accelerate the SVGD algorithm. These include:

- Incorporating second-order information so that steps adapt to local ill-conditioning of the posterior.
- Adapting the *geometry* of the parameter manifold via a Riemannian (natural-gradient) metric.

We outline both of these approaches below.

16.5.1 Stein Variational Newton (SVN)

The Stein variational Newton (SVN) algorithm, introduced by Detommaso et al. [417], incorporates second-order information in the updates to accelerate convergence. In particular, the steepest descent direction in (16.73) is replaced by an approximation of the Newton direction.

Recall that the Newton direction is defined as the minimiser of a local quadratic (rather than linear) approximation of $\mathcal{F}$. In order to define this minimiser, we must first introduce the *second variation* $\mathrm{D}^2\mathcal{F}(0)(g, g')$ of $\mathcal{F}$ at 0, along the pair of directions $g, g' \in \mathcal{H}^d$, defined as

$$\mathrm{D}^2\mathcal{F}(0)(g, g') := \lim_{\varepsilon\to 0} \frac{1}{\varepsilon}\left(\mathrm{D}\mathcal{F}(\varepsilon g')(g) - \mathrm{D}\mathcal{F}(0)(g)\right). \tag{16.74}$$

Meanwhile, assuming $\mathcal{F}$ is twice Fréchet differentiable at 0, its *functional Hessian* $\nabla_f^2\mathcal{F}(0)$ at 0 is the linear operator $\mathcal{H}^d \to \mathcal{H}^d$ such that, for all $g, g' \in \mathcal{H}^d$,

$$\mathrm{D}^2\mathcal{F}(0)(g, g') = \langle \nabla_f^2\mathcal{F}(0)[g], g'\rangle_{\mathcal{H}^d}. \tag{16.75}$$

In the kth iteration, we would like to find the Newton direction $g = (g_1, \ldots, g_d) \in \mathcal{H}^d$ for the functional $\mathcal{F}_{q_k,\pi}(f)$, evaluated at $f = 0$. This is characterised by the first-order stationarity conditions: for all $g' \in \mathcal{H}^d$,

$$\mathrm{D}^2\mathcal{F}_{q_k,\pi}(0)(g, g') = -\mathrm{D}\mathcal{F}_{q_k,\pi}(0)(g') \tag{16.76}$$

or, equivalently, for all $g' \in \mathcal{H}^d$,

$$\langle \nabla_f^2\mathcal{F}_{q_k,\pi}(0)[g], g'\rangle_{\mathcal{H}^d} = -\langle \nabla_f\mathcal{F}_{q_k,\pi}(0), g'\rangle_{\mathcal{H}^d}. \tag{16.77}$$

Due to Liu and Wang [1064, Theorem 3.3] and Detommaso et al. [417, Theorem 1], we have explicit expressions for both the functional gradient $\nabla_f\mathcal{F}_{q_k,\pi}(0)$ and the functional Hessian $\nabla_f^2\mathcal{F}_{q_k,\pi}(0)$, evaluated at $f = 0$. In particular, component-wise,

$$\left[\nabla_f\mathcal{F}_{q_k,\pi}(0)\right]_i(\theta') = -\mathbb{E}_{\theta\sim q_k}\left[k(\theta,\theta')\partial_i \log\pi(\theta) + \partial_i k(\theta,\theta')\right] \tag{16.78}$$

$$\left[\nabla_f^2\mathcal{F}_{q_k,\pi}(0)\right]_{i,j}(\theta',\theta'') = -\mathbb{E}_{\theta\sim q_k}\left[\partial_{ij}^2 \log\pi(\theta)k(\theta,\theta')k(\theta,\theta'') - \partial_i k(\theta,\theta')\partial_j k(\theta,\theta'')\right]. \tag{16.79}$$

In order to solve (16.77), Detommaso et al. [417] propose a Galerkin approximation. In particular, let $(\theta_k^i)_{i=1}^n$ be a collection of particles distributed according to q_k. Define the finite-dimensional linear space $\mathcal{H}_n = \operatorname{span}\left\{k(\theta_k^1,\cdot), \ldots, k(\theta_k^n,\cdot)\right\} \subseteq \mathcal{H}$, and let $\mathcal{H}_n^d = \mathcal{H}_n \times \cdots \times \mathcal{H}_n$ denote the corresponding space of vector-valued functions. We can then look for an approximate solution $g = (g_1, \ldots, g_d) \in \mathcal{H}_n^d$, viz

$$g_j(\theta) = \sum_{i=1}^n \alpha_i^j k(\theta_k^i, \theta), \tag{16.80}$$

for some real-valued coefficients $(\alpha_i^j)_{i=1:n}^{j=1:d}$ to be determined. In this case, (16.77) reduces to a finite-dimensional linear system, given by

$$\sum_{l=1}^n H_{rl}\alpha_l = b_r, \quad \text{for each } r = 1, \ldots, n, \tag{16.81}$$

where $\alpha_l = (\alpha_l^1, \ldots, \alpha_l^d)$ is the vector of unknown coefficients, and where

$$[H_{rl}]_{ij} = \left[\nabla_f^2\mathcal{F}_{q_k,\pi}(0)\right]_{ij}(\theta_k^r, \theta_k^l), \quad b_r = -\nabla_f\mathcal{F}_{q_k,\pi}(0)(\theta_k^r). \tag{16.82}$$

By solving (16.81), and substituting the coefficients back into the ansatz (16.80), we obtain an approximation $g = (g_1, \ldots, g_d) \in \mathcal{H}_n^d \subseteq \mathcal{H}^d$ for the Newton direction. More precisely, we can only evaluate a Monte Carlo approximation of the Hessian H_{rl} and the gradient b_r using the ensemble $(\theta_\cdot^i)_{i=1}^n$, which yields an estimate $\hat{g}^n$ of the Newton direction g. The SVN algorithm then reads

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \hat{g}^n(\theta_k^i). \tag{16.83}$$

In practice, computing the solution of the linear system in (16.81) may be prohibitively expensive in high-dimensional problems or when using a large number of particles, and the solution may not be a valid descent direction. Detommaso et al. [417] propose two solutions. The first is to use the inexact Newton-conjugate gradient (NCG) method, which guarantees a descent direction by terminating the conjugate gradient iteration. This approach is also scalable to high dimensions, as it only requires evaluating the matrix-vector product with each H_{rl}. The second is to use a block-diagonal approximation of the second variation, possibly in combination with a Gauss–Newton approximation of the $\nabla^2 \log\pi$, or inexact NCG.

16.5.2 Riemannian SVGD (RSVGD)

Let (Θ, g) be a Riemannian manifold, i.e., a differentiable manifold Θ equipped with a Riemannian metric g, defined as a collection $g = (g_\theta)_{\theta\in\Theta}$ of inner products $g_\theta(\cdot,\cdot) : \mathcal{T}_\theta\Theta \times \mathcal{T}_\theta\Theta \to \mathbb{R}$ on the tangent spaces $\mathcal{T}_\theta\Theta$, which vary smoothly with θ.

Given a local coordinate chart, the metric g is represented by a $d \times d$ symmetric positive-definite matrix with entries G_{ij}. We can then write the gradient of a function $f : \Theta \to \mathbb{R}$ and the divergence of a vector field V, as

$$\mathrm{grad} f = \sum_{i,j=1}^{d} G^{ij} \frac{\partial f}{\partial x_i} \frac{\partial}{\partial x_j} = G^{-1} \nabla f \tag{16.84}$$

$$\mathrm{div}(V) = |G|^{-\frac{1}{2}} \sum_i \frac{\partial}{\partial x_i} \left(|G|^{\frac{1}{2}} V^i \right) = |G|^{-\frac{1}{2}} \nabla \cdot (|G|^{\frac{1}{2}} V), \tag{16.85}$$

where we use G^{ij} to denote the $(i,j)^{\text{th}}$ entry of the inverse matrix G^{-1}, V^i to denote the ith (contravariant) component of V, and where $|G| = \det(G)$.

Following Liu and Zhu [1058], we can now define the *Riemannian-Stein operator* and the *Riemannian-Stein identity* as

$$T_\pi^{(G)} \phi := \mathrm{div}_G \phi + \langle \phi, \mathrm{grad} \log \pi \rangle_G, \qquad \mathbb{E}_\pi \left[T_\pi^{(G)} \phi \right] = 0, \tag{16.86}$$

where $\langle a, b \rangle_G = a^T G b$, div_G denotes the Riemannian divergence defined above, and π is understood as a density with respect to the Riemannian volume measure. These are the natural generalisations of their (Euclidean) Stein operator and the (Euclidean) Stein identity introduced in Section 16.2.4, cf. (16.12).

Once again, we seek the direction of steepest descent of the KL divergence. Letting $T_\varepsilon(\theta) = \theta + \varepsilon \phi(\theta)$ and $q_\varepsilon = (T_\varepsilon)_\# q$,[3] the derivative of the KL divergence w.r.t. the perturbation magnitude is given by [1058, Theorem 2]

$$\frac{\mathrm{d}}{\mathrm{d}\varepsilon} \mathrm{D}_{\mathrm{KL}}(q_\varepsilon \| \pi) \bigg|_{\varepsilon=0} = -\mathbb{E}_q \left[T_\pi^{(G)} \phi(\theta) \right], \qquad T_\pi^{(G)} \phi := \mathrm{div}_G \phi + \langle \phi, \mathrm{grad} \log \pi \rangle_G. \tag{16.87}$$

Similar to before, the task is now to solve the constrained optimisation problem

$$\phi^* = \arg\min_{\phi \in \mathcal{G}} \left\{ \frac{\mathrm{d}}{\mathrm{d}\varepsilon} \mathrm{D}_{\mathrm{KL}}(q_\varepsilon \| \pi) \bigg|_{\varepsilon=0} : ||\phi||_{\mathcal{G}} \leq 1 \right\} = \arg\max_{\phi \in \mathcal{G}} \left\{ \mathbb{E}_q \left[T_\pi^{(G)} \phi(\theta) \right] : ||\phi||_{\mathcal{G}} \leq 1 \right\}, \tag{16.88}$$

where $\mathcal{G}$ is a normed space of functions. We also require that ϕ^* is a valid vector field, coordinate invariant, and can be expressed in closed form.

The first and second of these requirements rule out the naive Euclidean choice $\mathcal{G} = \mathcal{H}^d$, the vector-valued RKHS used in the derivation of the standard SVGD update, since its elements are not intrinsically defined vector fields on the manifold. Instead, we will now take

$$\mathcal{G} = \{ \mathrm{grad} f : f \in \mathcal{H} \}, \tag{16.89}$$

where $\mathcal{H}$ is the RKHS associated with a positive-definite kernel $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$. This gradient is a well-defined object on Θ; it is guaranteed to be a valid vector field; and it is coordinate invariant [1058, Lemma 3]. Furthermore, it is possible to derive a closed-form solution as [1058, Theorem 4]

$$\phi^* = \mathrm{grad}_{(\cdot)} f^*(\cdot), \quad f^*(\cdot) = \mathbb{E}_{\theta \sim q} \left[\langle \mathrm{grad}_\theta k(\theta, \cdot), \mathrm{grad}_\theta \log \pi(\theta) \rangle_{G_\theta} + \mathrm{div}_\theta \left(\mathrm{grad}_\theta k(\theta, \cdot) \right) \right], \tag{16.90}$$

where all gradient and divergence operators are understood in the Riemannian sense defined above, with the subscript indicating the variable with respect to which they are taken. The Riemannian SVGD update thus reads

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \mathrm{grad}_{\theta_k^i} \left[\frac{1}{n} \sum_{j=1}^{n} \left[\langle \mathrm{grad}_{\theta_k^j} k(\theta_k^j, \theta_k^i), \mathrm{grad}_{\theta_k^j} \log \pi(\theta_k^j) \rangle_{G_{\theta_k^j}} + \mathrm{div}_{\theta_k^j} \left(\mathrm{grad}_{\theta_k^j} k(\theta_k^j, \theta_k^i) \right) \right] \right]. \tag{16.91}$$

In a fully intrinsic implementation, the Euclidean addition in this update should be replaced by a retraction or exponential-map step on Θ. The term $\mathrm{grad} \log \pi = G^{-1} \nabla \log \pi$ is a preconditioned version of the score function. If one chooses the Riemannian metric to approximate the posterior curvature (e.g., the Fisher information), then this has the effect of *whitening* the score. This can result in accelerated convergence in ill-conditioned models [1058].

[3] Here, the transport map is expressed in a local coordinate chart; more intrinsically, one would define T_ε via the exponential map or a retraction.

16.6 Stochastic and Scalable ParVI

In order to scale ParVI algorithms such as SVGD to large datasets and high-dimensional models, several adaptations are required. These include:

- stochastic approximations of the score function (e.g., mini-batches, variance reduction);
- *noisy* dynamics that retain exploration while preserving descent on the KL divergence;
- *systems-level* techniques (random-feature kernels, GPU/distributed implementations).

We develop these in a way consistent with the functional-gradient view established in Sections 16.2–16.3.

16.6.1 Mini-Batch SVGD

Basic Overview. Let the target posterior be $\pi(\theta) \propto p(\mathcal{D} \mid \theta)p(\theta)$ with $\mathcal{D} = \{(X_n, Y_n)\}_{n=1}^N$. The exact score of the target is given by

$$u_\pi(\theta) = \nabla \log p(\theta) + \sum_{n=1}^{N} \nabla \log p(Y_n \mid X_n, \theta). \tag{16.92}$$

Given a mini-batch $B_k \subset \{1, \dots, N\}$ of size b, we can define an *unbiased* estimator of the score function as

$$\widehat{u}_\pi(\theta; B_k) \;=\; \nabla \log p(\theta) \;+\; \frac{N}{b} \sum_{n \in B_k} \nabla \log p(Y_n \mid X_n, \theta), \tag{16.93}$$

with $\mathbb{E}_{B_k}\left[\widehat{u}_\pi(\theta; B_k)\right] = u_\pi(\theta)$. We now replace u_π by $\widehat{u}_\pi$ in the SVGD witness function, cf. (16.21), to obtain

$$\widehat{\phi}^*_{q,\pi}(\cdot; B_k) = \mathbb{E}_{\theta \sim q}\left[k(\theta, \cdot)\widehat{u_\pi}(\theta; B_k) + \nabla_\theta k(\theta, \cdot)\right]. \tag{16.94}$$

Substituting this into the population version of the SVGD algorithm, cf. (16.22) or (16.23), we arrive at

$$q_{k+1} = (\hat{T}^*_{\varepsilon_k})_{\#} q_k, \quad \text{where } \hat{T}^*_{\varepsilon_k}(\theta) = \theta + \varepsilon_k \hat{\phi}^*_{q_k,\pi}(\theta; B_k), \tag{16.95}$$

or equivalently, in terms of a single particle θ_k whose distribution is given by $q_k = \mathrm{Law}(\theta_k)$, the updates

$$\theta_{k+1} = \theta_k + \varepsilon_k \mathbb{E}_{\theta \sim q_k}\left[k(\theta, \theta_k)\hat{u}_\pi(\theta; B_k) + \nabla_\theta k(\theta, \theta_k)\right], \quad \theta_0 \sim q_0. \tag{16.96}$$

In practice, of course, one must approximate the expectation w.r.t. the unknown $(q_k)_{k \in \mathbb{N}}$. Similar to before, given $\{\theta_0^i\}_{i=1}^n \overset{\text{i.i.d.}}{\sim} q_0$, we will update for $i = 1, \dots, n$:

$$\theta^i_{k+1} = \theta^i_k + \varepsilon_k \frac{1}{n} \sum_{j=1}^{n} [k(\theta^j_k, \theta^i_k)\widehat{u}_\pi(\theta^j_k; B_k) + \nabla_{\theta^j_k} k(\theta^j_k, \theta^i_k)]. \tag{16.97}$$

This procedure is summarised in Algorithm 21.

Convergence Results. It is natural to ask whether the stochastic transformations defined in (16.95) will still result in a decrease in the KL divergence. That is, for sufficiently small step sizes ε_k, is it true that $\mathrm{D_{KL}}\left((\hat{T}^*_{\varepsilon_k})_{\#} q \middle\| \pi\right) \leq \mathrm{D_{KL}}(q\|\pi)$?

In (16.33), we reported a descent lemma for SVGD: under suitable conditions, the SVGD algorithm decreases the KL divergence at each iteration. In the stochastic setting, we can establish an analogous result for the *expected descent*. Let $\mu_{q,\pi}$ be the exact witness and $\widehat{\mu}_{q,\pi}$ its mini-batch version, so that $\mathbb{E}[\widehat{\mu}_{q,\pi}] = \mu_{q,\pi}$. In particular, we have

$$\mathbb{E}\left[\mathrm{D_{KL}}(q_{k+1}\|\pi) \mid q_k\right] - \mathrm{D_{KL}}(q_k\|\pi) \leq -\varepsilon_k \mathrm{KSD}^2(q_k\|\pi) + \mathcal{O}(\varepsilon_k^2), \tag{16.98}$$

where the $\mathcal{O}(\varepsilon_k^2)$ term now also depends on the variance of the mini-batch score estimator. Thus, mini-batching preserves the descent of KL in expectation. We note that this statement concerns the population mini-batch dynamics; for the finite-particle algorithm, one must additionally account for the empirical approximation of the witness function. For stronger convergence guarantees (e.g., a.s. convergence), standard Robbins-Monro conditions must be imposed on the step sizes $(\varepsilon_k)_{k \in \mathbb{N}}$.

Algorithm 21: Mini-Batch Stein Variational Gradient Descent (SVGD)

Inputs : particles $\{\theta_0^i\}_{i=1}^n$; kernel k; step sizes $\{\varepsilon_k\}$; batch size b
Outputs: updated particles $\{\theta_K^i\}_{i=1}^n$
for $k = 0$ **to** $K-1$ **do**
  sample mini-batch B_k of size b
  compute $\widehat{u}_\pi(\theta_k^j; B_k)$ for $j = 1, \dots, n$
  for $i = 1$ **to** n **do**
    $\theta_{k+1}^i \leftarrow \theta_k^i + \frac{\varepsilon_k}{n} \sum_{j=1}^n \left[k(\theta_k^j, \theta_k^i) \widehat{u}_\pi(\theta_k^j; B_k) + \nabla_{\theta_k^j} k(\theta_k^j, \theta_k^i) \right]$
return $\{\theta_K^i\}_{i=1}^n$

Extensions. While computationally convenient, using a mini-batch estimate is not without its drawbacks. For example, while unbiased, the stochastic approximation can have a high variance. There are various approaches to reducing the variance, including:

1. **Control variates.** Define an anchor point (e.g., the maximum a posteriori (MAP) estimate) $\tilde{\theta}$, and use a stochastic variance reduced gradient (SVRG) style estimator:
$$\widetilde{u}_\pi(\theta; \tilde{\theta}, B_k) = \widehat{u}_\pi(\theta; B_k) - \widehat{u}_\pi(\tilde{\theta}; B_k) + u_\pi(\tilde{\theta}). \tag{16.99}$$
This remains unbiased for $u_\pi(\theta)$, and has reduced variance in comparison to $\hat{u}_\pi(\theta)$ [77].
2. **Polyak–Ruppert Averaging.** Use an average of the iterates to dampen stochastic oscillations. This results in a reduced (asymptotic) variance [1338, 1440], and may yield a more stable approximation of the target posterior. Some care is required, however: averaging particle locations is not in general equivalent to improving the posterior approximation, and may be inappropriate for multimodal posteriors.

16.6.2 Stochastic Particle Optimisation Sampling (SPOS)

The stochastic particle optimisation sampling (SPOS) algorithm, introduced in Zhang et al. [1847], augments the Stein variational gradient flow, cf. (16.31) or (16.32), with stochastic gradient Langevin dynamics (SGLD) [509]. In the continuous-time, mean-field limit, this algorithm is described by the McKean-Vlasov PDE

$$\partial_t q_t(\theta) = -\nabla \cdot \left[q_t(\theta) \left(\beta^{-1} u_\pi(\theta) + \int_{\mathbb{R}^d} \left[k(\theta', \theta) u_\pi(\theta') + \nabla_{\theta'} k(\theta', \theta) \right] \mathrm{d}q_t(\theta') \right) \right] + \beta^{-1} \Delta q_t(\theta), \tag{16.100}$$

or, writing $q_t = \mathrm{Law}(\theta_t)$, the McKean SDE

$$d\theta_t = \left[\beta^{-1} u_\pi(\theta_t) + \int \left[k(\theta', \theta_t) u_\pi(\theta') + \nabla_{\theta'} k(\theta', \theta_t) \right] \mathrm{d}q_t(\theta') \right] dt + \sqrt{2\beta^{-1}}\, dw_t, \tag{16.101}$$

where $\beta > 0$ is a temperature parameter.

In the limit as the temperature $\beta \to \infty$, the Langevin drift and diffusion vanish, and (16.101) formally reduces to the continuous-time, mean-field SVGD dynamics, cf. (16.32); meanwhile, if the particle interaction term is removed or becomes negligible (e.g., in the limit of vanishing kernel bandwidth, in which the repulsion between distinct particles vanishes), SPOS reduces to a tempered variant of SGLD.

In practice, of course, we will approximate (16.101) using (i) a finite collection of particles and (ii) an appropriate time discretisation (e.g., Euler Maruyama). We can also use a stochastic (e.g., mini-batch) estimate for the score function. This yields the final SPOS algorithm:

$$\theta_{k+1}^i = \theta_k^i + \varepsilon_k \left[\beta^{-1} \hat{u}_\pi(\theta_k^i; B_k) + \frac{1}{n} \sum_{j=1}^n \left(k(\theta_k^j, \theta_k^i)\, \hat{u}_\pi(\theta_k^j; B_k) + \nabla_{\theta_k^j} k(\theta_k^j, \theta_k^i) \right) \right] + \sqrt{2\varepsilon_k \beta^{-1}}\, \xi_k^i, \tag{16.102}$$

where $\xi_k^i \sim \mathcal{N}(0, I)$ are a sequence of i.i.d. standard normal random variables.

Convergence Results. Under appropriate assumptions (e.g., log-concavity, bounded initial moments, constant or decreasing step sizes), it is possible to obtain *non-asymptotic* convergence rates for SPOS in the 1-Wasserstein metric W_1 [1847, Theorem 9, Theorem 10]. In practice, the noise term improves exploration and mitigates spurious fixed points of the deterministic Stein flow.

16.6.3 Random-feature kernels and sub-quadratic interactions

In each iteration, SVGD requires us to compute $\{k(\theta^i, \theta^j)\}_{i,j=1}^n$. Its cost is thus $\mathcal{O}(n^2)$ per iteration. If a very large number of particles is required, there are several options for reducing the computational cost:

- **Random Fourier Features**. For shift-invariant kernels $k(\theta, \theta') = \kappa(\theta - \theta')$, we can use the approximation $k(\theta, \theta') \approx \varphi(\theta)^\top \varphi(\theta')$ with m random Fourier features $\varphi : \mathbb{R}^d \to \mathbb{R}^m$. In this case,

$$\phi_{q,\pi}(\cdot) \approx \underbrace{\mathbb{E}_{\theta\sim q}\big[\varphi(\theta)\, u_\pi(\theta)^\top\big]^\top}_{\in\mathbb{R}^{d\times m}} \varphi(\cdot) + \underbrace{\mathbb{E}_{\theta\sim q}\big[J_\varphi(\theta)\big]^\top}_{\in\mathbb{R}^{d\times m}} \varphi(\cdot), \tag{16.103}$$

  where $J_\varphi(\theta) \in \mathbb{R}^{m\times d}$ denotes the Jacobian of φ at θ; in mini-batch settings, u_π can be replaced by the stochastic estimate $\widehat{u}_\pi$. This reduces the particle-interaction cost from $\mathcal{O}(n^2)$ to $\mathcal{O}(nm)$, up to the cost of evaluating the features and their Jacobians [1064].

- **Structured features**. The use of structured features (e.g., orthogonal random features) can further improve the computational cost.

16.6.4 GPU and distributed implementations

In the single-node GPU setting, both the kernel matrix $\mathbf{K} = [k(\theta^i, \theta^j)]_{ij}$ and score vector $\boldsymbol{u}_\pi = [\hat{u}_\pi(\theta^i)]_i$ are amenable to GPU parallelism. In particular, JIT-compiled implementations (e.g., PyTorch, JAX) fuse the two terms in (16.97) into batched tensor kernels.

For multi-node GPUs, with M devices, each holding roughly n/M particles and a shard of data, a typical iteration would be:

1. compute local scores on local batches;
2. *all-reduce* particle features (or sufficient statistics for (16.103));
3. update local particles.

In the last step, there are several ways to avoid global $\mathcal{O}(n^2)$ communication. These include using a random-feature factorisation (16.103) and *all-reduce* of the m-dimensional summaries; block-sparse kernels (nearest-neighbour graphs); or inducing-particle approximations (analogous to sparse GPs).

16.7 Applications in Bayesian Deep Learning

ParVI methods have been applied to a wide range of problems in BDL, offering scalable alternatives to exact Bayesian inference in high-dimensional, non-convex, and multimodal posterior landscapes.

16.7.1 Bayesian Neural Networks

Bayesian neural networks (BNNs) define a posterior over network parameters θ given data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$:

$$\pi(\theta) \propto p(\mathcal{D} \mid \theta) p(\theta), \tag{16.104}$$

where $p(\mathcal{D} \mid \theta)$ is the likelihood induced by the network outputs $f(\cdot, \theta)$, and $p(\theta)$ is the prior over the network parameters.

Posterior Prediction. Using a BNN, predictions for new inputs x_* are obtained by computing the *posterior predictive distribution*, defined as

$$p(y_* \mid x_*, \mathcal{D}) = \int p(y_* \mid x_*, \theta) \pi(\theta) \mathrm{d}\theta. \tag{16.105}$$

In modern architectures, exact computation of the posterior is infeasible, and the integral in (16.105) must be approximated. ParVI methods such as SVGD approximate π using an ensemble of particles $\{\theta^i\}_{i=1}^n$, and can be readily applied in this context. In particular, these particles can be used to approximate the posterior predictive distribution as

$$p(y_*|x_*, \mathcal{D}) \approx \frac{1}{n} \sum_{i=1}^n p(y_*|x_*, \theta^i). \tag{16.106}$$

SVGD and related Stein ensemble methods have shown competitive empirical performance in BNNs, for both regression and classification tasks. For example, on image classification benchmarks such as CIFAR-10 and MNIST, SVGD with $n \approx 20$ particles has been reported to outperform competing methods (e.g., deep ensembles) in terms of predictive accuracy, functional diversity, and uncertainty quantification [368, 1064, 1704], although such gains are problem- and architecture-dependent.

We highlight, in particular, the work of D'Angelo et al. [368], which unifies several variants of SVGD applicable to BNNs under the title of *Stein variational neural network ensembles*. Their framework includes the original SVGD algorithm – which performs inference directly in the weight space, and uses a kernel defined in the weight space – as well as the standard deep ensemble, which can be viewed as SVGD with a kernel defined by an indicator function.

It also includes several novel variants of SVGD, designed specifically for BNNs, which are derived via careful choices of the kernel function. These include *function-kernel weight SVGD*, in which the kernel is defined in the output space of the neural network:

$$k_f(\theta, \theta') = \exp\left(-\frac{1}{h}\,\mathbb{E}_{x\sim p(x)}\left[\|f(x;\theta) - f(x;\theta')\|^2\right]\right), \tag{16.107}$$

where $h > 0$ is a bandwidth parameter, $p(x)$ denotes the marginal distribution of the inputs, and $f(x;\theta)$ denotes the output of the neural network with parameters θ, given inputs x. In practice, this kernel is approximated using a Monte Carlo estimate over a mini-batch of inputs, typically sampled from the empirical training distribution. This choice of kernel, as shown in D'Angelo et al. [368], encourages functional diversity of the particles and avoids functional posterior collapse. This results in improved calibration, a greater resemblance to the true posterior distribution, and enhanced out-of-distribution (OOD) detection.

Out-of-Distribution Detection. The problem of out-of-distribution detection can be described as follows: given a test point x_*, decide whether x_* is from the same distribution as the training data, or a different distribution. Given a test point x_*, the predictive entropy

$$\mathcal{H}\left[p(y_* \mid x_*, \mathcal{D})\right] = -\sum_{y_*} p(y_*|x_*, \mathcal{D}) \log p(y_*|x_*, \mathcal{D}) \tag{16.108}$$

can be used as an out-of-distribution score. In particular, smaller values of the entropy correspond to confident predictions (i.e., likely to be in distribution); larger values correspond to uncertain predictions (i.e., possible out of distribution). In ParVI methods such as SVGD (and variants), disagreement among particles in predictive space directly increases the entropy for test inputs far from the training data, thereby providing a useful OOD signal; D'Angelo et al. [368] report improved OOD detection relative to methods such as deep ensembles, although relative performance is problem-dependent. See also Chapter 44 for a complementary treatment of predictive-uncertainty-based rejection.

Active Learning. In (Bayesian) active learning, the objective is to select the most informative data points to query in order to improve the performance of the model (e.g., the BNN). In Bayesian models, a popular acquisition function is the decrease in the expected posterior entropy, viz

$$\mathrm{MI}(y_*, \theta \mid x_*, \mathcal{D}) = \mathcal{H}\left[p(y_* \mid x_*, \mathcal{D})\right] - \mathbb{E}_{\theta\sim\pi(\theta)}\left[\mathcal{H}(p(y_* \mid x_*, \theta))\right]. \tag{16.109}$$

This is known as *Bayesian active learning by disagreement* (BALD); see Chapter 53 for a detailed treatment of Bayesian active learning. Using ParVI methods (e.g., SVGD), it is straightforward to approximate this acquisition function, replacing the posterior π by its particle-based approximation.

16.7.2 Large-Scale Models and Bayesian Fine-Tuning

For very large models (e.g., large language models or vision foundation models), a fully Bayesian treatment over all parameters is usually computationally prohibitive, and can be difficult to justify relative to cheaper uncertainty quantification strategies [277, 308]. The large number of parameters renders even approximate posterior inference infeasible, and the vast size of the (pre)training dataset often leads to relatively low posterior uncertainty in many parameters.

On the other hand, during fine-tuning, the number of learnable parameters is much smaller, and we typically have a much smaller dataset, implying significantly more uncertainty. There is, therefore, a stronger justification for the additional expense associated with the use of Bayesian methods.

Let θ denote the parameters of a large pre-trained model, and ϕ denote the parameters (e.g., of a small adapter, of the last layer) that we wish to fine-tune based on downstream data $\mathcal{D}$. We would like to compute the posterior over the fine-tuning parameters, viz

$$\pi(\phi) := p(\phi|\mathcal{D}, \theta) \propto p(\mathcal{D} \mid \theta, \phi)p(\phi), \tag{16.110}$$

where, similar to before, $p(\mathcal{D} \mid \theta, \phi)$ denotes the likelihood induced by the model outputs, and $p(\phi)$ denotes a prior over the fine-tuning parameters.

Since exact inference in $\pi(\phi)$ is generally intractable, we can approximate it using ParVI. In particular, using a ParVI algorithm, one obtains n particles $\{\phi^i\}_{i=1}^n$, each representing the parameters of a slightly different fine-tuned model. This yields an approximate posterior predictive distribution given by

$$p(y_* \mid x_*, \mathcal{D}) \approx \frac{1}{n}\sum_{i=1}^{n} p(y_* \mid x_*, \phi^i, \theta), \tag{16.111}$$

which can quantify (epistemic) uncertainty in low-data regimes [432, 1286]. ParVI-based fine-tuning is compatible with parameter-efficient strategies (e.g., low-rank adaptation (LoRA), bias-only tuning), keeping dimensionality manageable; see Chapters 39 and 43 for detailed treatments of last-layer inference and Bayesian approaches for large language models, respectively. Extensions to continual learning are also possible: as new datasets $\mathcal{D}_1, \mathcal{D}_2, \ldots$ arrive, the particle set can be propagated via sequential ParVI updates (Chapter 18), analogous to particle filtering.

16.8 Theoretical Guarantees and Open Problems*

Despite impressive empirical performance, ParVI algorithms such as SVGD still lack the kind of sharp, dimension-aware theory that is now standard for diffusion-based samplers (e.g., Langevin Monte Carlo, Hamiltonian Monte Carlo). This section summarises existing theoretical results for SVGD, including (i) the continuous-time, infinite-particle dynamics; (ii) the discrete-time, infinite-particle dynamics; (iii) the continuous-time, finite-particle dynamics; and (iv) the discrete-time, finite-particle dynamics. We conclude this section by outlining some open problems and interesting directions for future work.

16.8.1 Continuous Time, Infinite-Particle SVGD

In this section, we review convergence results for the continuous-time, infinite-particle SVGD dynamics, viz

$$\frac{\partial q_t(\theta)}{\partial t} = -\nabla_\theta \cdot (q_t(\theta)\phi^*_{q_t,\pi}(\theta)), \qquad \phi^*_{q_t,\pi}(\cdot) = \mathbb{E}_{\theta\sim q_t}[k(\theta,\cdot)u_\pi(\theta) + \nabla_\theta k(\theta,\cdot)]. \tag{16.112}$$

Well-Posedness. Under smoothness and growth conditions on the target π and the kernel k, this PDE is well posed (i.e., global existence, uniqueness, regularity) [1065]. Although π is a stationary solution of the PDE, establishing uniqueness of the invariant solution, and convergence of q_t to π, generally requires further assumptions: e.g., that a vanishing KSD characterises π (i.e., the KSD is convergence-determining for the chosen kernel), or that the target satisfies an appropriate functional inequality (see below).

Dissipation Identity. Using calculus on the space of probability measures, the dissipation of the KL divergence along the continuous-time, mean-field SVGD dynamics is given by [918, Proposition 1]

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathrm{D}_{\mathrm{KL}}(q_t\|\pi) = -\mathcal{I}_{\mathrm{Stein}}(q_t\|\pi), \tag{16.113}$$

where $\mathcal{I}_{\mathrm{Stein}}(q\|\pi)$ denotes the *Stein-Fisher information* of q relative to π, which coincides with the squared kernel Stein discrepancy (KSD) [462]

$$\mathcal{I}_{\mathrm{Stein}}(q\|\pi) := \mathrm{KSD}^2(q\|\pi). \tag{16.114}$$

Thus, in particular, KL is a Lyapunov functional and the (squared) KSD is the dissipation rate. Under additional assumptions (e.g., bounded first moment, bounded kernel), (16.113) implies that $\mathcal{I}_{\mathrm{Stein}}(q_t\|\pi) \to 0$ as $t \to \infty$ [918, Proposition 2]. Thus, for appropriate choices of the kernel, and given suitable tightness or moment conditions, this further implies that $q_t \Rightarrow \pi$ [652]. Another consequence of (16.113) is that [918, Proposition 3]

$$\min_{0\le s\le t} \mathrm{KSD}^2(q_s\|\pi) \le \frac{1}{t}\int_0^t \mathrm{KSD}^2(q_s\|\pi)\mathrm{d}s \le \frac{\mathrm{D}_{\mathrm{KL}}(q_0\|\pi)}{t}. \tag{16.115}$$

Convergence Rates. Under the assumption that the target π satisfies an appropriate functional inequality, it is possible to quantify how fast the flow converges to π. For example, if the target satisfies a *Stein log-Sobolev inequality* (LSI) with constant λ, viz

$$\mathrm{D}_{\mathrm{KL}}(q\|\pi) \leq \frac{1}{2\lambda}\mathcal{I}_{\mathrm{Stein}}(q\|\pi), \tag{16.116}$$

then the continuous-time, mean-field SVGD dynamics converge exponentially fast to the target distribution [918, Proposition 4]

$$\mathrm{D}_{\mathrm{KL}}(q_t\|\pi) \leq e^{-2\lambda t}\,\mathrm{D}_{\mathrm{KL}}(q_0\|\pi). \tag{16.117}$$

16.8.2 Discrete-Time, Infinite-Particle SVGD

We next consider the forward-Euler discretisation of the continuous-time, infinite-particle dynamics in (16.112), viz

$$q_{k+1} = \left(\mathrm{id} + \varepsilon_k \phi^*_{q_k,\pi}\right)_{\#} q_k, \quad \phi^*_{q_k,\pi}(\cdot) = \mathbb{E}_{\theta\sim q_k}\left[k(\theta,\cdot)u_\pi(\theta) + \nabla_\theta k(\theta,\cdot)\right], \tag{16.118}$$

where $(\varepsilon_k)_{k\in\mathbb{N}}$ is a positive, non-increasing step-size schedule. This corresponds to the *population limit* or *mean-field limit* of the SVGD algorithm.

A First Descent Lemma. The original work of Liu [1063] provided the first theoretical guarantees for SVGD in discrete time at the population level. In particular, assuming boundedness of the KSD and the kernel, Liu [1063, Theorem 3.3] established a *descent lemma*, guaranteeing that for sufficiently small step sizes the KL decreased at each iteration. That is,

$$\mathrm{D}_{\mathrm{KL}}(q_{k+1}\|\pi) - \mathrm{D}_{\mathrm{KL}}(q_k\|\pi) \leq -\varepsilon_k \mathrm{KSD}^2(q_k\|\pi) + \varepsilon_k^2 R \cdot \mathrm{KSD}^2(q_k\|\pi), \tag{16.119}$$

where $0 < R < \infty$ is a positive constant depending on the kernel and the score, namely, $\mathrm{R} = \sup_x \left[\frac{1}{2}||\nabla \log p||_{\mathrm{Lip}} k(x,x) + 2\nabla_{xx'}$

Initial Non-Asymptotic Convergence Rates. Subsequently, Korba et al. [918] obtained a similar descent lemma and established the first non-asymptotic convergence rate for the population version of the SVGD algorithm. To be specific, assuming the kernel and its first derivative are bounded by $0 < B < \infty$; the target is L-smooth, so that the Hessian of the target is bounded by $0 < L < \infty$; and the KSD along the path of the algorithm iterates is bounded by $0 < C < \infty$, it was established in Korba et al. [918, Proposition 5] that

$$\mathrm{D}_{\mathrm{KL}}(q_{k+1}\|\pi) - \mathrm{D}_{\mathrm{KL}}(q_k\|\pi) \leq -\varepsilon_k \mathrm{KSD}^2(q_k\|\pi) + \varepsilon_k^2 \frac{(\alpha^2+L)B^2}{2} \cdot \mathrm{KSD}^2(q_k\|\pi), \tag{16.120}$$

for any $\alpha > 1$, provided the step size $\varepsilon_k := \varepsilon \leq \frac{\alpha-1}{\alpha BC}$. Although this descent lemma is rather similar to the one previously obtained in Liu [1063, Theorem 3.3], it was based on rather different proof techniques, namely, differential calculus on the Wasserstein space.

Following this result, Korba et al. [918] also obtained a $\mathcal{O}(\frac{1}{K})$ convergence rate for the arithmetic mean of the squared KSD between the iterates q_k and π. In particular, writing $c_\varepsilon = \varepsilon(1 - \varepsilon\frac{(\alpha^2+L)B^2}{2})$, and assuming a similar condition on the step size, the authors showed that [918, Corollary 6]

$$\min_{k=0,\ldots,K-1} \mathrm{KSD}^2(q_k\|\pi) \leq \frac{1}{K}\sum_{k=0}^{K-1} \mathrm{KSD}^2(q_k\|\pi) \leq \frac{\mathrm{D}_{\mathrm{KL}}(q_0\|\pi)}{c_\varepsilon K}. \tag{16.121}$$

Complexity Bounds under Talagrand's T_1 Inequality. Following these results, Salim et al. [1451] obtained the first dimension-explicit iteration complexity bounds for SVGD, assuming the target π satisfied Talagrand's T_1 inequality. Notably, unlike in previous works, their results did not require the assumption that the KSD was uniformly bounded along the algorithm iterates, an assumption which cannot be verified prior to running the algorithm.

To be more specific, Salim et al. [1451] first established a descent lemma [1451, Theorem 3.2] and a non-asymptotic convergence rate for the average of the squared KSD [1451, Corollary 4.2]. Their results are essentially identical to the corresponding results in Korba et al. [918], except that now the dependence on the kernel bound is linear rather than quadratic, and the bound on the step size does not require a bound on the KSD. Finally, the authors provided the first dimension-dependent complexity result for SVGD, showing that, provided the number of iterations satisfies [1451, Corollary 4.3]

$$K = \tilde{\Omega}\left(\frac{Ld^{\frac{3}{2}}}{\lambda^{\frac{1}{2}}\varepsilon}\right), \tag{16.122}$$

one obtains $\mathrm{KSD}^2(\bar{q}_K \| \pi) \leq \varepsilon$, where λ denotes the constant in Talagrand's T_1 inequality; and $\bar{q}_K = \frac{1}{K}\sum_{k=0}^{K-1} q_k$ denotes the average of the iterates.

Convergence under Weaker Smoothness Assumptions. Most recently, Sun et al. [1575] established the convergence of SVGD (i.e., a descent lemma, convergence rate, iteration complexity) under weaker assumptions. In particular, their results required that (a) the target satisfies a T_p inequality with constant λ or a generalised (T_p, J) inequality; and (b) the target potential is (L_0, L_1)-smooth, i.e., there exist constants $L_0, L_1 > 0$ such that $\|\nabla^2 V(\theta)\|_{\mathrm{op}} \leq L_0 + L_1 \|\nabla V(\theta)\|$ for all $\theta \in \mathbb{R}^d$. This significantly extended previous results, all of which had assumed L-smoothness of the potential (i.e., global Lipschitz continuity of the score, globally bounded Hessian), thereby excluding a large class of potential functions (e.g., polynomials of degree greater than two).

16.8.3 Finite-Particle SVGD

Finally, we review convergence results for the finite-particle system. We consider both the continuous-time dynamics, viz

$$\dot{\theta}_t^i = \frac{1}{n}\sum_{j=1}^{n}\left[k(\theta_t^j, \theta_t^i) u_\pi(\theta_t^j) + \nabla_{\theta_t^j} k(\theta_t^j, \theta_t^i)\right], \quad i = 1, \ldots, n, \tag{16.123}$$

and its discrete-time counterpart, i.e., the SVGD algorithm itself, viz

$$\theta_{t+1}^i = \theta_t^i + \varepsilon_t \frac{1}{n}\sum_{j=1}^{n}\left[k(\theta_t^j, \theta_t^i) u_\pi(\theta_t^j) + \nabla_{\theta_t^j} k(\theta_t^j, \theta_t^i)\right], \quad i = 1, \ldots, n, \tag{16.124}$$

where, in a slight abuse of notation, we now use t to index both the time in the continuous-time dynamics; and the iteration in the discrete-time algorithm.

General Strategies For Establishing Convergence. Broadly speaking, establishing convergence of the finite-particle system can proceed via two possible routes. Similar to before, let $q_t^n = \frac{1}{n}\sum_{j=1}^{n} \delta_{\theta_t^j}$ denote the empirical distribution of the interacting particle system. Then we can either:

(i) Show that $\lim_{t\to\infty}[\lim_{n\to\infty} q_t^n] = \lim_{t\to\infty} q_t = \pi$. That is:

  (i.a) **Propagation of Chaos.** First establish that the empirical distribution q_t^n converges to the mean-field distribution q_t as $n \to \infty$, for any finite time $t \in [0, \infty)$.

  (i.b) **Infinite-Particle Convergence to Equilibrium.** Then establish that the mean-field distribution q_t converges to the target distribution π as $t \to \infty$. This was the subject of Sections 16.8.1 and 16.8.2.

(ii) Show that $\lim_{n\to\infty}[\lim_{t\to\infty} q_t^n] = \lim_{n\to\infty} \pi^n = \pi$. That is:

  (ii.a) **Finite-Particle Convergence to Equilibrium**. First establish that the empirical distribution q_t^n converges to a (unique) equilibrium, say π^n, as $t \to \infty$.

  (ii.b) **Uniform-In-Time Propagation of Chaos**. Then establish that the empirical equilibrium distribution π^n converges to the target π as $n \to \infty$.

We refer to Banerjee et al. [90], Liu et al. [1067] for a visual illustration of these two approaches. In the subsequent discussion, we will focus on (i), this being the prevailing approach in the literature. Regarding (ii), we mention briefly the recent work of Liu et al. [1067], which analysed this approach for Gaussian SVGD, wherein both the target distribution and the initial distribution are Gaussian, and the kernel is bilinear. In this case, the mean-field distribution remains Gaussian for all times [e.g., 1067, Theorem 3.1] and it is possible to establish quantitative uniform-in-time convergence of the finite particle system to its mean-field limit [e.g., 1067, Theorem 3.7]. Most recently, Balasubramanian et al. [79] have established uniform-in-time propagation-of-chaos bounds for SVGD, obtaining rates with logarithmic dependence on the number of particles in general settings, and parametric $\mathcal{O}(n^{-1/2})$ rates, uniform in time, for Gaussian targets with matrix-valued kernels. Karimi Jaghargh et al. [871], Priser et al. [1346] also study finite-particle asymptotics.

Propagation of Chaos. Several different papers have established the time-dependent convergence of the finite-particle system to its mean-field limit, in both continuous time [241, 1085] and discrete time [653, 918, 1063], under various assumptions on the kernel (e.g., continuous, Lipschitz, smooth) and the potential (e.g., uniformly bounded). For example, regarding the discrete-time algorithm, Korba et al. [918] showed that, for any fixed finite horizon T, and for all iterations t corresponding to times up to T,

$$\mathbb{E}\left[W_2^2(q_t, q_t^n)\right] \leq \frac{1}{2}\left(\frac{1}{\sqrt{n}}\sqrt{\mathrm{Var}(q_0)}e^{LT}\right)\left(e^{2LT}-1\right), \tag{16.125}$$

where L is a constant depending on the kernel k and the target π; see [918, Section 11.4] for details.

Unified Convergence Analyses. By combining convergence of the finite-particle distribution q_t^n to the mean-field distribution q_t as $n \to \infty$ (see above), and convergence of the mean-field distribution q_t to the target distribution π as $t \to \infty$, two groups have now obtained a unified convergence analysis for the SVGD algorithm [90, 1512].

The first such result was established in Shi and Mackey [1512], who combined improved rates for $q_t^n \to q_t$ [1512, Theorem 2], with existing rates for $q_t \to \pi$ [1451, Theorem 3.2], to obtain a bound on $\min_{0\leq s\leq t} \mathrm{KSD}(q_s^n \| \pi)$. However, due to the doubly exponential dependence on time of their bound between the finite-particle distribution and the mean-field distribution (via the sum of the step sizes), the obtained rate is rather slow, namely, of order $1/\sqrt{\log\log n}$.

More recently, Banerjee et al. [90] have obtained significantly improved finite-particle convergence rates for SVGD, in both KSD and Wasserstein-2 metrics. In particular, the authors improve the dependence on the number of particles from the $1/\sqrt{\log\log n}$ rate of Shi and Mackey [1512] to $1/\sqrt{n}$ (a double-exponential improvement), in both continuous and discrete time. Under mild assumptions on the kernel and potential, these bounds also grow polynomially in the dimension d.

16.8.4 Curse-of-Dimensionality

The curse of dimensionality largely affects ParVI algorithms such as SVGD through the geometry of the RKHS.

Even for simple Gaussian targets, SVGD with standard (e.g., radial basis function (RBF), IMQ) kernels can systematically underestimate the posterior variance in high dimensions [e.g., 68, 1882]. This is known as the *variance collapse phenomenon*. Indeed, for RBF kernels, the magnitude of the repulsive term in the Stein update equation, cf. (16.35), can be upper bounded by a quantity that decreases with d [e.g., 1882, Proposition 1]. Thus, the particles tend to exhibit mode collapse.

In Ba et al. [68], the authors undertake a detailed analysis of this phenomenon. They argue that, in high dimensions, the "driving force" (i.e., the kernel-weighted score function) has a higher variance than the repulsion [68, Proposition 2]. This, coupled with the deterministic nature of the SVGD updates (i.e., no noise or resampling), leads to systematic underestimation of the variance [68, Section 4.2]. The authors then consider the limit as $n, d \to \infty$, with $\frac{d}{n} \to \gamma$ for some $\gamma > 1$. For an isotropic Gaussian target, it is possible to explicitly characterise the behaviour of the SVGD variance as a function of γ, for two common choices of the kernel (i.e., RBF, IMQ). In both cases, the learned variance decreases as γ increases, suggesting the number of particles should grow linearly with d in order to reliably estimate the dimension-averaged variance.

The variance collapse phenomenon has motivated various methodological developments. Structural approaches reduce the effective dimensionality, including message-passing SVGD [1882], projected SVGD [270], sliced SVGD [646], Grassmann SVGD [1068], and function-space SVGD [368]. Meanwhile, geometric approaches employ preconditioned or matrix-valued kernels to adaptively whiten the score [1704]. Finally, stochastic variants of SVGD, such as SPOS [1847], inject noise into the dynamics to counteract deterministic collapse.

16.8.5 Open problems

We conclude this section by highlighting a number of open problems.

Kernel Design in High Dimensions. In most applications, practitioners resort to the use of an RBF kernel, with the bandwidth chosen adaptively according to a median heuristic [e.g., 1064]. In very high dimensions, this choice is often ineffective. Open questions include the design of principled, problem-dependent kernels (e.g. function-space, neural tangent kernel (NTK)-like, or Fisher-preconditioned kernels) and adaptive, iteration-dependent kernels; and guarantees that connect kernel choice to *dimension-robust* rates.

Finite-Particle Degeneracy. We currently lack general (e.g., algorithm-independent) conditions that preclude particle collapse for a broad class of targets. Practical heuristics (e.g., an adaptive bandwidth, tempering, occasional noise injection, MCMC rejuvenation) can help, but remain outside of the current theory. Meanwhile, obtaining improved non-asymptotic, finite-particle convergence rates under weaker assumptions remains an ongoing and active topic of research.

Beyond the KSD. Typical convergence rates – including infinite-particle and finite-particle – are stated in terms of the KSD [e.g., 90, 918, 1512, 1575]. In reality, however, practitioners may be interested in other metrics such as the predictive risk, coverage, etc. An open question is when (and at what rate) does decreasing the KSD ensure calibrated predictive distributions? Currently, there exist only partial results assuming additional structure on the target (e.g., Gaussian) or the kernel (e.g., composite, bilinear).

Theory for SVGD with Stochastic Gradients. Typical applications of ParVI in BDL (see Section 16.7.1) necessitate the use of stochastic gradients and/or the injection of additional noise. We currently lack a complete theory for mini-batch SVGD (e.g., with variance reduction) and for noisy variants of SVGD (e.g., SPOS).

Differential Privacy. Differential privacy for ParVI algorithms such as SVGD remains largely unexplored. While results exist for other sampling algorithms such as stochastic gradient MCMC (Chapter 2) [e.g., 1021], the questions of how particle interactions affect privacy loss and how to calibrate ParVI algorithms for differential privacy, without destroying accuracy or causing particle collapse, have not yet been studied in detail.

16.9 Summary

In this chapter, we introduced particle-based variational inference, a family of methods which combine the non-parametric flexibility of MCMC with the optimisation-based efficiency of VI. Starting from SVGD, we showed how ParVI methods can be understood as time discretisations and particle approximations of gradient flows on the space of probability measures, and described a number of second-order, Riemannian, and stochastic variants. We then surveyed applications of these methods in Bayesian deep learning, ranging from Bayesian neural networks to the Bayesian fine-tuning of large-scale models, before reviewing existing theoretical guarantees. ParVI remains a rapidly developing field, and the open problems outlined in the previous section – including kernel design in high dimensions, finite-particle degeneracy, and theory for stochastic variants – are likely to remain active topics of research in the coming years.

Chapter 17

Variational inference in function space

17.1 Introduction

Bayesian neural networks (BNNs) were initially envisioned as a framework that could unify the representational power of deep learning with the principled uncertainty quantification of Bayesian inference. Yet, for particular approximation families and experimental settings, standard weight-space posterior approximations can underperform in terms of predictive accuracy or uncertainty calibration [542, 826]. A key difficulty of the standard weight-space variational formulation lies in the nature of inference in weight space: the map from network parameters to functions is highly nonlinear and many-to-one, leading to complex, multimodal posterior landscapes. This makes it hard both to approximate posteriors in this formulation and to interpret or encode prior knowledge about the data-generating process through a weight-space prior.

These challenges motivate a shift of perspective from parameter-space to function-space variational inference (FSVI). At the theoretical level, rather than performing inference over neural-network weights, one works with probability measures over functions, objects that describe input–output behaviour and more naturally capture inductive biases about the learning problem. In this setting, the prior is defined over functions (for instance, as a Gaussian process), and the likelihood operates on function evaluations. FSVI seeks an approximate posterior measure by minimising its divergence from the posterior measure. Practical methods, however, always must access these measures through finite-dimensional representations either through parameters or through finite-dimensional marginals since it is infeasible to manipulate an infinite-dimensional law in its entirety.

This change in perspective has important implications for both the mathematical and computational structure of variational inference. First, it requires measure-theoretic formulations of Bayes' rule and Kullback–Leibler (KL) divergence in infinite-dimensional spaces, where no analogue of Lebesgue measure exists. Second, it exposes limitations of KL-based objectives: they may be intractable to evaluate directly and can be infinite in many situations, e.g., when a finite-dimensional parameterised neural network represents a variational function law and the prior is a non-degenerate Gaussian process. These difficulties motivate a range of KL approximations and generalised objectives based on alternative discrepancies.

This chapter develops the function-space perspective. We formalise Bayesian inference on a measurable function space, derive the associated variational objectives, and show how finite-dimensional approximations make them usable and motivate new methods. Throughout, we emphasise the interplay between function-space theory and its practical realisation in neural networks and Gaussian processes.

17.2 Function-Space Bayes Theorem

The key difficulty when working with function spaces is that there is no analogue of a translation-invariant, σ-finite Lebesgue measure whenever the corresponding space is infinite-dimensional. Consequently, Bayes' rule cannot be written as "posterior density $\propto$ prior density $\times$ likelihood" relative to a base Lebesgue measure. Instead, the correct formulation is *measure-theoretic*: the posterior is defined via the Radon–Nikodym derivative of the posterior measure with respect to the prior measure. For a broader perspective on the measure-theoretic formulation of variational inference, see [1149] and the references therein.

17.2.1 Setting and Notation

Let $\mathcal{X}$ be a measurable input domain and let $\mathcal{F} \subseteq \mathbb{R}^{\mathcal{X}}$ be a space of real-valued functions equipped with a σ-algebra $\mathfrak{F}$. We use $m \in \mathbb{N}$ for the size of an arbitrary finite evaluation tuple. For $\mathbf{x} = (x_1, \dots, x_m) \in \mathcal{X}^m$, define the **evaluation map**

$$\mathrm{eval}_{\mathbf{x}} : \mathcal{F} \to \mathbb{R}^m, \qquad \mathrm{eval}_{\mathbf{x}}(f) \coloneqq [f(x_1), \dots, f(x_m)]^\top.$$

Two examples are: (i) $\mathcal{F} = \mathbb{R}^{\mathcal{X}}$ with the cylinder, or product, σ-algebra, namely the smallest σ-algebra making every finite evaluation map measurable; and (ii) a separable Banach or Hilbert space of functions with its Borel σ-algebra. Reproducing kernel Hilbert spaces (RKHSs), introduced in Chapter 23, are convenient examples because their point evaluations are continuous linear functionals and hence measurable.

We write $\mathfrak{p}$ and $\mathfrak{q}$ for probability measures on $(\mathcal{F}, \mathfrak{F})$ (e.g., a prior measure $\mathfrak{p}$ and a variational approximation $\mathfrak{q}$). The pushforward finite-dimensional marginal of $\mathfrak{p}$ is $\mathfrak{p}_{\mathbf{x}} \coloneqq (\mathrm{eval}_{\mathbf{x}})_{\#}\mathfrak{p}$.

Let $(\mathcal{Y}, \mathfrak{Y})$ denote the data space, equipped with the corresponding σ-algebra $\mathfrak{Y}$. A **likelihood functional** is a map

$$L(\,\cdot \mid f) : \mathcal{Y} \to [0, \infty], \qquad \text{such that } (f, y) \mapsto L(y \mid f) \text{ is } \mathfrak{F} \otimes \mathfrak{Y}\text{-measurable.}$$

For example, if the data space is $\mathcal{Y} = \mathbb{R}^n$, then $L(y \mid f)$ may be a density $p(y \mid f)$ with respect to Lebesgue measure on $\mathbb{R}^n$, parameterised by $f \in \mathcal{F}$.

In supervised learning, let n denote the number of observations, let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, and write $\mathbf{x}_{\mathrm{tr}} = (x_1, \dots, x_n) \in \mathcal{X}^n$ and $y = (y_1, \dots, y_n) \in \mathcal{Y}$. A common factorised likelihood is

$$L(y \mid f) = \prod_{i=1}^{n} p(y_i \mid f(x_i)) = \widetilde{L}(y \,|\, \mathrm{eval}_{\mathbf{x}_{\mathrm{tr}}}(f)),$$

so the likelihood depends on f only through its values at the training inputs. Other settings are possible. For example, in Cox process models, f parameterises a Poisson-process intensity and the likelihood depends on integrals involving f; see, e.g., [1148].

17.2.2 Function-Space Bayes Theorem

Fix an observation $y \in \mathcal{Y}$ and let $\mathfrak{p}$ denote the prior probability measure. Then the *evidence* (marginal likelihood) is the Lebesgue integral

$$Z(y) \;\coloneqq\; \int_{\mathcal{F}} L(y \mid f)\, d\mathfrak{p} \;\in\; [0, \infty].$$

We will assume throughout this Chapter that $0 < Z(y) < \infty$. Define a set function on $\mathfrak{F}$ by

$$\mathfrak{p}(\,A \mid y\,) \;\coloneqq\; \frac{1}{Z(y)} \int_A L(y \mid f)\, d\mathfrak{p}, \qquad A \in \mathfrak{F}.$$

Then $\mathfrak{p}(\cdot \mid y)$ is a probability measure on $(\mathcal{F}, \mathfrak{F})$, it is absolutely continuous with respect to $\mathfrak{p}$ (denoted $\mathfrak{p}(\cdot \mid y) \ll \mathfrak{p}$), and its Radon–Nikodym derivative is

$$\frac{d\,\mathfrak{p}(\cdot \mid y)}{d\,\mathfrak{p}}(f) \;=\; \frac{L(y \mid f)}{Z(y)} \qquad \mathfrak{p}\text{-a.s.}$$

We call $\mathfrak{p}(\cdot \mid y)$ the *posterior measure* on the function space.

We note here that there is no appeal to "posterior densities" on $\mathcal{F}$. The posterior is always defined *relative to the prior* via the measurable likelihood functional.

17.2.3 Finite-Dimensional (Evaluation) Form

If the likelihood depends only on the training evaluations, i.e.

$$L(y \mid f) = \widetilde{L}(y \,|\, \mathrm{eval}_{\mathbf{x}_{\mathrm{tr}}}(f)),$$

define the finite-dimensional evidence

$$\widetilde{Z}(y) \coloneqq \int_{\mathbb{R}^n} \widetilde{L}(y \mid \mathbf{u})\, d\mathfrak{p}_{\mathbf{x}_{\mathrm{tr}}}(\mathbf{u}).$$

By the definition of a pushforward integral, $\widetilde{Z}(y) = Z(y)$. Thus, in addition to

$$\frac{d\,\mathfrak{p}(\cdot \mid y)}{d\,\mathfrak{p}}(f) \;=\; \frac{\widetilde{L}(y \mid \mathrm{eval}_{\mathbf{x}_{\mathrm{tr}}}(f))}{\widetilde{Z}(y)}, \qquad \mathfrak{p}\text{-a.s.},$$

we also have

$$\frac{d\,\mathfrak{p}_{\mathbf{x}_{\mathrm{tr}}}(\cdot \mid y)}{d\,\mathfrak{p}_{\mathbf{x}_{\mathrm{tr}}}}(\mathbf{u}) = \frac{\widetilde{L}(y \mid \mathbf{u})}{\widetilde{Z}(y)}, \qquad \mathfrak{p}_{\mathbf{x}_{\mathrm{tr}}}\text{-a.s.} \tag{17.1}$$

Thus the posterior marginal at $\mathbf{x}_{\mathrm{tr}}$ is obtained using a familiar finite-dimensional calculation: by reweighting the corresponding prior marginal $\mathfrak{p}_{\mathbf{x}_{\mathrm{tr}}}$ with the likelihood and renormalising. However, although (17.1) gives the correct posterior marginal at the training inputs, that single marginal does not determine the full posterior measure $\mathfrak{p}(\cdot \mid y)$ on $\mathcal{F}$.

17.3 Function-Space KL and Variational Objectives

17.3.1 Measure-theoretic KL divergence

The Kullback–Leibler divergence between two probability measures $\mathfrak{q}$ and $\mathfrak{p}$ on $(\mathcal{F}, \mathfrak{F})$ is

$$\mathrm{KL}(\mathfrak{q}\|\mathfrak{p}) \;=\; \begin{cases} \displaystyle\int_{\mathcal{F}} \log\!\left(\frac{d\mathfrak{q}}{d\mathfrak{p}}\right) d\mathfrak{q}, & \text{if } \mathfrak{q} \ll \mathfrak{p}, \\ +\infty, & \text{otherwise,} \end{cases}$$

where $\frac{d\mathfrak{q}}{d\mathfrak{p}}$ denotes the Radon–Nikodym derivative of $\mathfrak{q}$ with respect to $\mathfrak{p}$. We note that $\mathrm{KL}(\mathfrak{q}\|\mathfrak{p})$ may be infinite even when $\mathfrak{q} \ll \mathfrak{p}$. Absolute continuity guarantees the existence of the Radon–Nikodym derivative $\frac{d\mathfrak{q}}{d\mathfrak{p}}$, but imposes no integrability conditions on $\log\left(\frac{d\mathfrak{q}}{d\mathfrak{p}}\right)$ under $\mathfrak{q}$.

17.3.2 Function-space ELBO

For fixed data y, we henceforth denote the posterior measure $\mathfrak{p}(\cdot \mid y)$ by $\widehat{\mathfrak{p}}$. Suppose $\mathfrak{q} \ll \mathfrak{p}$, $\mathfrak{q} \ll \widehat{\mathfrak{p}}$ and both $\mathrm{KL}\big(\mathfrak{q} \,\|\, \widehat{\mathfrak{p}}\big)$ and $\mathrm{KL}\big(\mathfrak{q} \,\|\, \mathfrak{p}\big)$ are finite. Then, we can apply the chain rule for Radon–Nikodym derivatives:

$$\begin{aligned} \mathrm{KL}\big(\mathfrak{q} \,\|\, \widehat{\mathfrak{p}}\big) &= \int \log\!\left(\frac{d\mathfrak{q}}{d\widehat{\mathfrak{p}}}\right) d\mathfrak{q} = \int \log\!\left(\frac{d\mathfrak{q}}{d\mathfrak{p}}\right) d\mathfrak{q} - \int \log\!\left(\frac{d\widehat{\mathfrak{p}}}{d\mathfrak{p}}\right) d\mathfrak{q} \\ &= \int \log\!\left(\frac{d\mathfrak{q}}{d\mathfrak{p}}\right) d\mathfrak{q} - \int \log\!\left(\frac{L(y \mid f)}{Z(y)}\right) d\mathfrak{q} \\ &= \underbrace{\mathrm{KL}(\mathfrak{q}\|\mathfrak{p})}_{\text{regulariser (KL to prior)}} \underbrace{- \mathbb{E}_{\mathfrak{q}}[\log L(y \mid f)]}_{\text{data fit}} + \log Z(y). \end{aligned}$$

Rearranging yields the evidence lower bound

$$\boxed{\log Z(y) \geq \mathbb{E}_{\mathfrak{q}}[\log L(y \mid f)] - \mathrm{KL}(\mathfrak{q}\|\mathfrak{p}) =: \mathrm{ELBO}(\mathfrak{q})} \tag{17.2}$$

with equality if and only if $\mathfrak{q} = \widehat{\mathfrak{p}}$. This is the *variational objective in function space*. We wish to consider problems of the form

$$\max_{\mathfrak{q} \in \mathcal{Q}} \mathrm{ELBO}(\mathfrak{q}) \tag{17.3}$$

for some variational family $\mathcal{Q}$.

17.3.3 Well-posedness of ELBO maximisation

A simple way to ensure that the ELBO is well defined throughout the variational family $\mathcal{Q}$ is to require that every $\mathfrak{q} \in \mathcal{Q}$ satisfies $\mathfrak{q} \ll \mathfrak{p}$ and that both $\mathrm{KL}(\mathfrak{q}\|\mathfrak{p})$ and $\mathbb{E}_{\mathfrak{q}}[\log L(y \mid f)]$ are finite. This is not always straightforward to ensure. For example, it is possible that the KL to the prior is finite, i.e. $\mathfrak{q} \ll \mathfrak{p}$ and $\mathrm{KL}(\mathfrak{q}\|\mathfrak{p}) < \infty$, while the KL to the posterior is not. Indeed, if $\mathfrak{q}$ puts positive mass where $d\widehat{\mathfrak{p}}/d\mathfrak{p} = 0$, then $\mathrm{KL}\big(\mathfrak{q} \,\|\, \widehat{\mathfrak{p}}\big) = \infty$ and

$\log L(y \mid f) = -\infty$ on a set of positive $\mathfrak{q}$-measure. If, in addition, the positive part of $\log L(y \mid f)$ is integrable, then $\mathbb{E}_{\mathfrak{q}}[\log L(y \mid f)] = -\infty$ and the lower bound (17.2) is vacuous; otherwise the expected log-likelihood may even be undefined. Candidates with an undefined expected log-likelihood must be excluded, whereas a family for which every candidate has ELBO equal to $-\infty$ also yields an ill-posed maximisation problem.

If, in addition to $0 < Z(y) < \infty$, we assume

$$0 < L(y \mid f) < \infty \quad \text{for } \mathfrak{p}\text{-almost every } f,$$

then

$$\frac{d\widehat{\mathfrak{p}}}{d\mathfrak{p}}(f) = \frac{L(y \mid f)}{Z(y)} > 0 \qquad \mathfrak{p}\text{-a.s.},$$

so $\widehat{\mathfrak{p}}$ and $\mathfrak{p}$ are equivalent measures. Consequently, any $\mathfrak{q} \ll \mathfrak{p}$ also satisfies $\mathfrak{q} \ll \widehat{\mathfrak{p}}$, although this equivalence alone does not make the KL divergences, the expected log-likelihood, or the ELBO finite; hence particular care needs to be taken in choosing the variational family. A sufficient condition is that for every variational law under consideration,

$$\mathrm{KL}(\mathfrak{q}\|\mathfrak{p}) < \infty, \qquad \mathbb{E}_{\mathfrak{q}}[|\log L(y \mid f)|] < \infty.$$

Leaving aside the difficulty of whether the ELBO objective is well defined, we still have a challenge of efficiently estimating it and computing its gradients for a given variational family. The next section discusses a supremum characterisation of the function-space KL divergence that motivates practical approaches to ELBO maximisation.

17.4 Supremum characterisation of the ELBO

Let $\mathfrak{p}$ and $\mathfrak{q}$ be arbitrary probability measures on $\mathbb{R}^{\mathcal{X}}$ equipped with the cylinder σ-algebra generated by all finite evaluation maps. For a finite tuple $\mathbf{z} = (z_1, \ldots, z_m)$, denote their pushforward marginals by $\mathfrak{p}_{\mathbf{z}}$ and $\mathfrak{q}_{\mathbf{z}}$. On this measurable space, with no additional topology or sample-path regularity assumptions, [1576] show that

$$\mathrm{KL}(\mathfrak{q}\|\mathfrak{p}) \;=\; \sup_{m \in \mathbb{N},\, \mathbf{z} \in \mathcal{X}^m} \mathrm{KL}\big(\mathfrak{q}_{\mathbf{z}}\|\mathfrak{p}_{\mathbf{z}}\big). \tag{17.4}$$

The identity follows because the cylinder σ-algebra contains precisely the information generated by finite evaluations. Note that one should not transfer the identity without further argument to a larger Borel σ-algebra or to other function spaces.

For the supervised likelihood introduced above, define

$$\mathrm{ELBO}_{\mathbf{z}}(\mathfrak{q}) \coloneqq \mathbb{E}_{\mathfrak{q}}\Big[\log \widetilde{L}(y \,|\, \mathrm{eval}_{\mathbf{x}_{\mathrm{tr}}}(f))\Big] - \mathrm{KL}\big(\mathfrak{q}_{\mathbf{z}}\|\mathfrak{p}_{\mathbf{z}}\big).$$

Substitution in the function-space ELBO gives

$$\mathrm{ELBO}(\mathfrak{q}) = \inf_{m \in \mathbb{N},\, \mathbf{z} \in \mathcal{X}^m} \mathrm{ELBO}_{\mathbf{z}}(\mathfrak{q}).$$

Note that the data-fit term depends on the fixed training tuple $\mathbf{x}_{\mathrm{tr}}$, not on the auxiliary tuple $\mathbf{z}$ over which the infimum is taken. The infimum need not be attained, however, and its value may still be $-\infty$ when the function-space KL is infinite.

17.4.1 Adversarial functional VI

Restricting the tuple $\mathbf{z}$ to a fixed size $m \in \mathbb{N}$ gives the finite-dimensional saddle problem

$$\max_{\mathfrak{q}} \; \min_{\mathbf{z} \in \mathcal{X}^m} \mathrm{ELBO}_{\mathbf{z}}(\mathfrak{q}).$$

Optimisation alternates between minimising over $\mathbf{z}$ and maximising over $\mathfrak{q}$, in analogy with a capacity-limited adversary in a generative adversarial network [650]. This finite-evaluation objective is not generally an ELBO: it retains an evidence-lower-bound interpretation only when $\mathbf{z}$ contains every input whose function value enters the likelihood, here all of $\mathbf{x}_{\mathrm{tr}}$ [1576]. As [1576] point out, the adversarial scheme can generalise poorly. Its selected evaluation locations are likely to remain close to the training data, and the prior enters the objective only through the KL term, so prior information may not be used effectively for extrapolation.

17.4.2 Sampling-based functional VI

To ensure that the KL term constrains behaviour in prediction regions, [1576] augment a random subsample of training inputs with evaluation locations drawn from the prediction domain. Let Π_{tr} be a specified distribution on finite tuples sampled from $\{x_1, \ldots, x_n\}$, and let $\nu^\star$ be a user-chosen probability distribution on a prediction domain. Write $\mathbf{x}_{\mathrm{sub}} \sim \Pi_{\mathrm{tr}}$ for a random training-input tuple, $\mathbf{z}^\star \sim (\nu^\star)^{\otimes m_\star}$ for $m_\star$ independent prediction-domain locations, and $\oplus$ for tuple concatenation. The sampling-based objective of [1576] is

$$\max_{\mathfrak{q}} \mathbb{E}_{\mathbf{x}_{\mathrm{sub}} \sim \Pi_{\mathrm{tr}}} \mathbb{E}_{\mathbf{z}^\star \sim (\nu^\star)^{\otimes m_\star}} \left[\mathrm{ELBO}_{\mathbf{x}_{\mathrm{sub}} \oplus \mathbf{z}^\star}(\mathfrak{q}) \right].$$

The data-fit term is always evaluated at the full training tuple $\mathbf{x}_{\mathrm{tr}}$; only the KL term uses the sampled tuple. The sampling scheme thereby encourages agreement with the prior both on and off the training set. However, we have replaced the infimum with expectation, so the connection to the function-space ELBO is no longer direct. In the special case where the true posterior is a GP and every evaluation tuple contains all the training inputs, the true posterior maximises both the expected finite-measurement objective and the full function-space objective, providing a justification for using expectation; see [1576] for details. The new criterion makes it possible to combine structured GP priors (e.g., periodic plus RBF kernels) with a BNN variational family. In experiments reported by [1576], this combination recovered periodic structure and extrapolated comparably to exact GPs in examples where standard weight-space VI failed to extrapolate sensibly. The study also reports competitive and scalable results on UCI data sets and applies the method to contextual bandits. In those studied settings, the sampling-based scheme yielded better overall performance than the adversarial scheme.

17.4.3 Grid-functional divergence

Sampling-based functional VI replaces the supremum in (17.4) by an expectation of marginal KL divergences but keeps the size of the sampled tuples fixed. [1096] take this idea further and also randomise the tuple size. Let M have a probability mass function π on $\mathbb{N}$ with $\pi(m) > 0$ for every m, and, conditionally on $M = m$, draw $\mathbf{z}_M \sim \nu_m^\star$ on $\mathcal{X}^m$. Define the augmented grid $\mathbf{u}_M = \mathbf{x}_{\mathrm{tr}} \oplus \mathbf{z}_M$ and

$$\mathrm{KL}_{\mathrm{grid}}(\mathfrak{q} \| \mathfrak{p}) \coloneqq \mathbb{E}_{M \sim \pi} \mathbb{E}_{\mathbf{z}_M \sim \nu_M^\star} \left[\mathrm{KL}\big(\mathfrak{q}_{\mathbf{u}_M} \| \mathfrak{p}_{\mathbf{u}_M}\big) \right]. \tag{17.5}$$

In the setting of [1096], where $\mathcal{X}$ is a compact subset of $\mathbb{R}^d$, suppose each $\nu_m^\star$ has full support on $\mathcal{X}^m$. Since $\pi(m) > 0$ for every m, Proposition 1 then gives $\mathrm{KL}_{\mathrm{grid}}(\mathfrak{q} \| \mathfrak{p}) = 0$ if and only if $\mathfrak{q} = \mathfrak{p}$. If $M = m$ is fixed, only the marginals on augmented tuples $\mathbf{x}_{\mathrm{tr}} \oplus \mathbf{z}_m$ are compared; distinct process laws can agree on all such marginals while differing on larger tuples. Depending on the choice of π, the expectation can remain finite even when the supremum KL is infinite. Moreover, because every $\mathbf{u}_M$ contains all training inputs, Proposition 3 of [1096] ensures that we still obtain a lower bound on the evidence,

$$\log Z(y) \geq \mathbb{E}_{\mathfrak{q}} \Big[\log \widetilde{L}(y \,|\, \mathrm{eval}_{\mathbf{x}_{\mathrm{tr}}}(f)) \Big] - \mathrm{KL}_{\mathrm{grid}}(\mathfrak{q} \| \mathfrak{p}). \tag{17.6}$$

The paper combines this objective with stochastic process generators, a flexible non-Gaussian variational family, and analytic and mini-batch estimators. In the experiments reported by [1096], the resulting method reproduced the qualitative posterior structure of piecewise-constant and piecewise-linear implicit priors and performed strongly against the selected baselines on UCI regression, contextual bandits, and image classification with out-of-distribution detection.

17.5 Parameter pushforward measures

Let $\Theta \subset \mathbb{R}^d$ be a finite-dimensional parameter space with Borel σ-algebra $\mathcal{B}(\Theta)$. Let $g : \Theta \to \mathcal{F}$ be a measurable parameter-to-function map, $g(\theta) = f(\cdot; \theta)$. Given a *parameter prior* P_Θ and a *variational parameter law* Q_Θ on $(\Theta, \mathcal{B}(\Theta))$, define the induced pushforward measures

$$\mathfrak{p}^g \;\coloneqq\; g_{\#} P_\Theta, \qquad \mathfrak{q}^g \;\coloneqq\; g_{\#} Q_\Theta,$$

so that $\mathfrak{p}^g(A) = P_\Theta(\{\theta : g(\theta) \in A\})$ and likewise for $\mathfrak{q}^g$.

17.5.1 Data-processing inequality

For any measurable g, the data-processing inequality (cf. e.g. [1339]) gives

$$\boxed{\mathrm{KL}\big(\mathfrak{q}^g \,\|\, \mathfrak{p}^g\big) \;\le\; \mathrm{KL}\big(Q_\Theta \,\|\, P_\Theta\big).} \tag{17.7}$$

Suppose $Q_\Theta \ll P_\Theta$ and the right-hand side is finite, and let $h = dQ_\Theta/dP_\Theta$. Equality holds precisely when h is measurable with respect to the σ-algebra generated by g, equivalently when $h = \varphi \circ g$ P_Θ-a.s. for some measurable φ. Thus, a sufficient condition for equality is P_Θ-almost-sure injectivity of g. In general, however, g is many-to-one and equality typically does not hold.

Thus, if one chooses the *function-space prior* to be the *same* BNN-induced law $\mathfrak{p}^g$ and approximates it with $\mathfrak{q}^g$ induced by a different parameter law, the function-space KL cannot exceed the parameter-space KL. For the same expected log-likelihood, the function-space ELBO is therefore at least as large. This formalises why working directly in function space can give *tighter* objectives than parameter-space VI when g is many-to-one (and, of course, deep neural network models yield many-to-one mappings). This comparison of objective values does not, by itself, guarantee a better predictive approximation, but a tighter function-space objective avoids penalising discrepancies between parameter laws that induce the same function law, thereby aligning the regularisation more closely with the predictive object of interest.

17.5.2 Estimating function-space KL for parameter pushforwards

The function-space KL in (17.7) is finite whenever $\mathrm{KL}(Q_\Theta \| P_\Theta) < \infty$. It is nevertheless a divergence between generally non-standard pushforward laws and still requires approximation.

One tractable approximation is proposed by [1435]. For a finite tuple $\mathbf{z} = (z_1, \dots, z_m) \in \mathcal{X}^m$, write $g_{\mathbf{z}}(\theta) = \mathrm{eval}_{\mathbf{z}}(g(\theta)) \in \mathbb{R}^m$. For $G \in \{P_\Theta, Q_\Theta\}$, define the parameter mean and covariance

$$m_G \coloneqq \mathbb{E}_{\theta\sim G}[\theta], \qquad S_G \coloneqq \mathrm{Cov}_{\theta\sim G}(\theta).$$

Assume that $g_{\mathbf{z}}$ is differentiable at m_G and define its Jacobian by

$$J_{\mathbf{z}}(m_G) \coloneqq \left[\frac{\partial g(z_i,\theta)}{\partial \theta_j}\bigg|_{\theta=m_G} \right]_{\substack{1\le i\le m\\ 1\le j\le d}} \in \mathbb{R}^{m\times d}.$$

Linearise $g_{\mathbf{z}}$ at m_G:

$$g_{\mathbf{z}}(\theta) \approx g_{\mathbf{z}}(m_G) + J_{\mathbf{z}}(m_G)(\theta - m_G).$$

This is the same local parameter-to-function construction used in linearised Laplace approximations; see Chapter 8. It gives the Gaussian marginal approximations

$$(\mathfrak{p}^g)_{\mathbf{z}} \approx \mathcal{N}\big(\mu_P(\mathbf{z}), \Sigma_P(\mathbf{z})\big), \qquad (\mathfrak{q}^g)_{\mathbf{z}} \approx \mathcal{N}\big(\mu_Q(\mathbf{z}), \Sigma_Q(\mathbf{z})\big),$$

where

$$\mu_G(\mathbf{z}) \coloneqq g_{\mathbf{z}}(m_G), \qquad \Sigma_G(\mathbf{z}) \coloneqq J_{\mathbf{z}}(m_G) S_G J_{\mathbf{z}}(m_G)^\top.$$

Thus m_G, S_G are moments in parameter space, whereas μ_G, Σ_G are moments of the linearised function values.

Replacing the supremum in (17.4) by a maximum over user-chosen regularisation tuples $\{\mathbf{z}^{(s)}\}_{s=1}^S$, the surrogate function-space objective becomes

$$\begin{aligned}\widetilde{\mathcal{F}}(Q_\Theta) \coloneqq\; & \mathbb{E}_{\theta\sim Q_\Theta}\Big[\log \widetilde{L}(y \,|\, g_{\mathbf{x}_{\mathrm{tr}}}(\theta))\Big] \\ & - \max_{1\le s\le S} \mathrm{KL}\Big(\mathcal{N}\big(\mu_Q(\mathbf{z}^{(s)}), \Sigma_Q(\mathbf{z}^{(s)})\big) \,\Big\|\, \mathcal{N}\big(\mu_P(\mathbf{z}^{(s)}), \Sigma_P(\mathbf{z}^{(s)})\big)\Big).\end{aligned}$$

The prior moments m_P, S_P are fixed by P_Θ, while the moments of Q_Θ are variational quantities. In practice, samples from Q_Θ are reparameterised and the Jacobians are obtained by automatic differentiation. By construction, (i) the data-processing inequality ensures a well-posed function-space regulariser when the parameter-space KL is finite, and (ii) the linearisation supplies closed-form KL terms at each $\mathbf{z}$, yielding a scalable estimator that preserves the function-space viewpoint while remaining compatible with BNN pushforwards.

17.5.3 Interpretable priors and the singularity issue

Often we do not want the parameter pushforward $\mathfrak{p}^g$ as the prior – it is not clear what prior beliefs on neural network weights represent. Instead, one may specify a non-degenerate GP prior $\mathfrak{p}$ whose covariance kernel expresses structural assumptions about the unknown function, while retaining the BNN variational law $\mathfrak{q}^g$. Function-space priors are discussed further in Chapter 31.

This choice falls outside the common-map setting of (17.7). It also creates a dimension mismatch. Fix $\mathbf{z} = (z_1, \dots, z_m)$ and consider

$$g_{\mathbf{z}} : \theta \longmapsto \big(g(z_1, \theta), \dots, g(z_m, \theta)\big) \in \mathbb{R}^m.$$

Assume that $g_{\mathbf{z}}$ is locally Lipschitz, as is the case for standard finite neural networks with locally Lipschitz activations. If $m > d$, its image is a countable union of sets with Hausdorff dimension at most d, and therefore has m-dimensional Lebesgue measure zero. The marginal $\mathfrak{q}^g_{\mathbf{z}} = (g_{\mathbf{z}})_{\#}Q_\Theta$ assigns probability one to this image. By contrast, if the GP covariance matrix at $\mathbf{z}$ is positive definite, $\mathfrak{p}_{\mathbf{z}}$ has a Lebesgue density on $\mathbb{R}^m$ and assigns probability zero to the image. For geometric intuition, a one-parameter network evaluated at two inputs traces a curve in the plane; a non-degenerate bivariate Gaussian assigns zero probability to that curve.

Hence $\mathfrak{q}^g_{\mathbf{z}}$ and $\mathfrak{p}_{\mathbf{z}}$ are mutually singular for such a tuple, and the cylinder-space identity (17.4) yields

$$\mathrm{KL}\big(\mathfrak{q}^g \,\|\, \mathfrak{p}\big) = \sup_{m,\mathbf{z}} \mathrm{KL}\big(\mathfrak{q}^g_{\mathbf{z}} \,\|\, \mathfrak{p}_{\mathbf{z}}\big) = +\infty.$$

KL-based objectives are hence incompatible with the setting where we wish to posit an interpretable GP prior but to use a flexible neural network parameterisation of the posterior. This motivates considering discrepancies that do not require absolute continuity.

17.6 Generalised Variational Objectives

17.6.1 Motivation

In function-space VI with interpretable priors, the KL-based ELBO may equal $-\infty$ because the variational law and the prior may be mutually singular; in particular, the KL divergences between a parametric pushforward and a GP measure can be infinite in both directions. Even if we instead used a variational measure that is absolutely continuous with respect to the prior, there is generally no easy way to estimate KL in infinite-dimensional spaces. This motivates variational objectives with better analytical structure. Consequently, in this section we seek alternative discrepancies that remain finite and tractable for rich function-space distributions.

17.6.2 Generalised variational objective (GVO)

Let $(\mathcal{F}, \mathfrak{F})$ be a measurable function space, let $\mathfrak{p}$ be a prior, and let $L(y \mid f)$ be a positive likelihood. Following [904], replace the KL regularisation by a discrepancy D and define a generalised variational objective (GVO) as:

$$\mathcal{J}_D(\mathfrak{q}) \coloneqq \mathbb{E}_{\mathfrak{q}}\big[\log L(y \mid f)\big] - D(\mathfrak{q}, \mathfrak{p}).$$

When $D = \mathrm{KL}$, this is the ordinary ELBO. Taking $D = \mathrm{KL}_{\mathrm{grid}}$ as defined in (17.5) yields the approach of [1096]. The linearised Gaussian approximation of [1435] in Section 17.5.2 can likewise be viewed as a generalised objective over parameter laws, with the maximum of its finite-marginal Gaussian KL terms serving as a surrogate discrepancy. For a general D, $\mathcal{J}_D$ need not lower-bound the evidence, and its optimiser need not be the ordinary Bayesian posterior. It instead defines a generalised update whose interpretation depends on D.

17.6.3 Wasserstein-2 GVO for Gaussian measures on function spaces

A particularly attractive choice for D in function space is the squared 2-Wasserstein distance W_2^2, which remains finite for any two Gaussian measures in a separable Hilbert space and has a closed form expression. Moreover, the 2-Wasserstein distance is a genuine metric and measures the geometry of transporting one probability law to another.

Background on Gaussian measures in Hilbert spaces can be found in [352, 943]; Gaussian measures on more general linear spaces such as Banach or Fréchet spaces are treated by [180]. Gaussian measures are closely related to Gaussian processes, but differ in that they are explicitly defined on a specific function space, whereas trajectories of a Gaussian process can be embedded into multiple function spaces.

Assume that $\mathcal{F}$ is a separable Hilbert space, such as $L^2(\mathcal{X}, \rho)$ for a finite measure ρ or a reproducing kernel Hilbert space. An operator A on $\mathcal{F}$ is *Hilbert–Schmidt* if $\sum_i \|Ae_i\|_{\mathcal{F}}^2 < \infty$ for an orthonormal basis $(e_i)_i$. A positive operator C is *trace-class* if $\operatorname{tr}(C) = \sum_i \langle Ce_i, e_i\rangle_{\mathcal{F}} < \infty$. Every positive self-adjoint operator C has a unique positive square root, and C is trace-class if and only if $C^{1/2}$ is Hilbert–Schmidt. Covariance operators of Gaussian measures on $\mathcal{F}$ are always positive, self-adjoint, and trace-class.

Suppose $\mathfrak{p} = \mathcal{N}(m_{\mathfrak{p}}, C_{\mathfrak{p}})$ and $\mathfrak{q} = \mathcal{N}(m_{\mathfrak{q}}, C_{\mathfrak{q}})$ are Gaussian measures on $\mathcal{F}$, with $m_{\mathfrak{p}}, m_{\mathfrak{q}} \in \mathcal{F}$. Then

$$W_2^2(\mathfrak{q}, \mathfrak{p}) = \|m_{\mathfrak{q}} - m_{\mathfrak{p}}\|_{\mathcal{F}}^2 \; + \; \operatorname{tr}\Big(C_{\mathfrak{q}} + C_{\mathfrak{p}} - 2\left(C_{\mathfrak{p}}^{1/2} C_{\mathfrak{q}} C_{\mathfrak{p}}^{1/2}\right)^{1/2}\Big). \tag{17.8}$$

Let $B = C_{\mathfrak{p}}^{1/2} C_{\mathfrak{q}} C_{\mathfrak{p}}^{1/2}$. Since $C_{\mathfrak{p}}^{1/2}$ and $C_{\mathfrak{q}}^{1/2}$ are Hilbert–Schmidt,

$$\operatorname{tr}(B^{1/2}) = \|C_{\mathfrak{q}}^{1/2} C_{\mathfrak{p}}^{1/2}\|_{\mathsf{S}_1} \le \|C_{\mathfrak{q}}^{1/2}\|_{\mathsf{S}_2} \|C_{\mathfrak{p}}^{1/2}\|_{\mathsf{S}_2} < \infty,$$

where S_1 and S_2 denote the trace-class and Hilbert–Schmidt norms. Thus, $W_2(\mathfrak{q}, \mathfrak{p})$ is finite without any absolute-continuity assumptions.

The Feldman–Hájek theorem states that two Gaussian measures on a separable Hilbert space are either equivalent or mutually singular [352, 943]. Equivalence holds only under substantially stricter compatibility conditions. In particular, on the common covariance support, the relative covariance perturbation $S = C_{\mathfrak{p}}^{-1/2} C_{\mathfrak{q}} C_{\mathfrak{p}}^{-1/2} - I$ must admit a Hilbert–Schmidt extension, in addition to conditions on the covariance ranges and the difference of the means. If these conditions fail, the measures are mutually singular and their KL divergence is infinite. This is in contrast to the automatic trace-class property needed for the Wasserstein cross term in (17.8).

Note that we have required both the prior and variational function laws to be Gaussian measures. Non-Gaussian likelihoods can still enter through $\mathbb{E}_{\mathfrak{q}}[\log L]$, but the variational law cannot represent non-Gaussian posteriors. Hence flexibility needs to be introduced through the parametrisation of the mean function and the covariance operator.

17.6.4 Generalised Wasserstein inference with a neural mean (GWI-net)

Having selected the Wasserstein-2 distance, we need to parameterise the variational law $\mathfrak{q}$. One possibility is to approximate a parameter pushforward $\mathfrak{q}^g = g_{\#} Q_\Theta$ by a Gaussian measure on $\mathcal{F}$. Alternatively, the generalised Wasserstein inference network (GWI-net) of [1755] uses a hybrid Gaussian law whose mean $m_{\mathfrak{q}} = f(\cdot; \theta)$ is a deep neural network and whose covariance has a sparse variational GP form. Here θ is a deterministic variational parameter of the mean function, not a random BNN weight.

We focus on $\mathcal{F} = L^2(\mathcal{X}, \rho)$, where ρ is a chosen reference probability measure (typically the input law).

Prior (Gaussian measure). Let $\mathfrak{p} = \mathcal{N}(m_{\mathfrak{p}}, C_{\mathfrak{p}})$, with

$$(C_{\mathfrak{p}}\phi)(x) = \int_{\mathcal{X}} k(x, x')\phi(x')\, d\rho(x'), \qquad \phi \in \mathcal{F},$$

where $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a symmetric positive-definite kernel such that $C_{\mathfrak{p}}$ is trace-class.

Variational law (Gaussian measure). Let $\mathfrak{q} = \mathcal{N}(m_{\mathfrak{q}}, C_{\mathfrak{q}})$, with

$$(C_{\mathfrak{q}}\phi)(x) = \int_{\mathcal{X}} r(x, x')\phi(x')\, d\rho(x'), \qquad \phi \in \mathcal{F},$$

where $r : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a symmetric positive-definite kernel parameterised by additional variational parameters.

The neural mean, covariance parameters, inducing locations, and any likelihood parameters are trained by maximising the expected log-likelihood minus an empirical approximation of the squared Wasserstein distance in (17.8). For Gaussian regression, $y_i \mid F \sim \mathcal{N}(F(x_i), \sigma^2)$, the expected log-likelihood is analytic; for non-Gaussian likelihoods it can be approximated by quadrature or Monte Carlo. The fitted Gaussian law supplies predictive means and covariances, with the observation likelihood providing any additional predictive noise.

To estimate the Wasserstein distance, note that

$$\|m_{\mathfrak{p}} - m_{\mathfrak{q}}\|_{\mathcal{F}}^2 = \int_{\mathcal{X}} (m_{\mathfrak{p}}(x) - m_{\mathfrak{q}}(x))^2\, d\rho(x),$$
$$\operatorname{tr}(C_{\mathfrak{p}}) = \int_{\mathcal{X}} k(x, x)\, d\rho(x), \qquad \operatorname{tr}(C_{\mathfrak{q}}) = \int_{\mathcal{X}} r(x, x)\, d\rho(x),$$

under the usual trace-kernel conditions [208]. Replacing ρ by the empirical design measure $\widehat{\rho}_n = n^{-1}\sum_{i=1}^n \delta_{x_i}$ turns these terms into sample averages. These averages are Monte Carlo-unbiased for fixed integrands when $x_1, \ldots, x_n$ are i.i.d. from ρ; for a fixed design, they instead define a discretisation at the training inputs.

The remaining cross term is estimated from a spectral subsample $\mathbf{x}_S = (x_{s_j})_{j=1}^{n_S}$ of $n_S \ll n$ training inputs. Write

$$r(\mathbf{x}_S, \mathbf{x}_{\mathrm{tr}}) = \big(r(x_{s_j}, x_i)\big)_{j,i}, \qquad k(\mathbf{x}_{\mathrm{tr}}, \mathbf{x}_S) = \big(k(x_i, x_{s_j})\big)_{i,j}.$$

The approximation of [1755] is

$$\begin{aligned}\widehat{W}_2^2 = \frac{1}{n}\sum_{i=1}^n \big(m_{\mathfrak{p}}(x_i) - m_{\mathfrak{q}}(x_i)\big)^2 + \frac{1}{n}\sum_{i=1}^n k(x_i, x_i) \\ + \frac{1}{n}\sum_{i=1}^n r(x_i, x_i) - \frac{2}{\sqrt{n n_S}}\sum_{j=1}^{n_S}\sqrt{\lambda_j(r(\mathbf{x}_S, \mathbf{x}_{\mathrm{tr}})k(\mathbf{x}_{\mathrm{tr}}, \mathbf{x}_S))}.\end{aligned} \tag{17.9}$$

The matrix spectrum is an empirical Nyström approximation to the covariance-operator composition. Increasing n_S improves spectral resolution at greater matrix cost; [1755] gives an empirical study under rapid spectral decay but no unbiasedness or consistency results have been shown. With the empirical design measure, behaviour away from the training inputs is determined indirectly by the kernels and neural mean; additional points drawn from a broader reference measure can instead be used in the discretisation.

Many parameterisations of r are possible. Following [1755], let $\mathbf{Z} = (z_1, \ldots, z_M) \in \mathcal{X}^M$ be inducing locations, and define

$$\mathbf{k}_Z(x) \coloneqq [k(z_1, x), \ldots, k(z_M, x)]^\top, \qquad \mathbf{K}_{ZZ} \coloneqq \big(k(z_i, z_j)\big)_{i,j=1}^M.$$

For a free lower-triangular factor $L \in \mathbb{R}^{M\times M}$ and nonsingular $\mathbf{K}_{ZZ}$, set

$$r(x, x') = k(x, x') - \mathbf{k}_Z(x)^\top \mathbf{K}_{ZZ}^{-1}\mathbf{k}_Z(x') + \mathbf{k}_Z(x)^\top LL^\top \mathbf{k}_Z(x'). \tag{17.10}$$

This is the sparse variational GP covariance construction of [1615]; stochastic mini-batch variants are developed by [737, 738], and Chapter 38 gives further details on inducing-point methods.

In the experiments of [1755], GWI-net was evaluated on synthetic examples, UCI regression, image classification, and out-of-distribution detection. The paper reports the best mean rank in its UCI comparison, strong classification results, and mixed out-of-distribution performance across datasets. The method remains limited by its Gaussian variational family, the spectral-decay approximation, and numerical sensitivity in the inducing-point parameterisation.

17.7 Discussion

Function-space variational inference reframes Bayesian deep learning at the level where inductive bias lives: distributions over functions. This perspective yields a measure-theoretic ELBO, clarifies why weight-space KLs can mislead in infinite dimensions, and motivates workable surrogates—from supremum-over-marginals and sampling-based objectives to grid-functional divergences. It also explains, via data-processing, when parameter pushforwards are overly restrictive and how linearised, finite-marginal Gaussian approximations can recover tractable regularisers. Most broadly, generalised objectives can replace KL with alternative discrepancies, such as grid-based KL or Wasserstein-2, that remain finite and computable in a broader range of settings, enabling hybrid variational laws that pair neural posteriors with interpretable GP-like priors. These approaches pave the way towards a toolkit that preserves Bayesian discipline while delivering the extrapolation, calibration, and scalability modern applications demand. In practice, however, finite discretisations, Gaussian variational restrictions, empirical approximations and numerical sensitivity limit these methods, and the current literature does not establish general guarantees of extrapolation, calibration, or scalability. Looking ahead, the key opportunities are tighter estimators of function-space discrepancies, richer variational family parameterisations, and principled integration of domain priors about the objects of interest—such as invariances, monotonicity or convexity, and conservation constraints—encoded directly in function space to steer inference toward scientifically plausible solutions.

Chapter 18

Sequential variational inference

18.1 Introduction

This chapter is concerned with sequential or online Bayesian inference of the parameters of a deep neural network. That is, we want to recursively compute $p(\theta_t|\mathcal{D}_{1:t})$ as the data streams in, where $\mathcal{D}_t = (x_t, y_t)$ is the tth datapoint, and θ_t are the parameters at time t. This is an online version of the standard Bayesian deep learning (BDL) problem, and is useful for settings where we need to take a decision at each step (e.g., contextual bandits [466]), and/or for settings where the data generating process is non-stationary (e.g., continual learning [1020, 1860]). We approach this problem from the perspective of Bayesian filtering (as opposed to smoothing, which works offline to compute $p(\theta_t|\mathcal{D}_{1:T})$ where $\mathcal{D}_{1:T}$ is the full dataset). If the parameters are assumed to be constant over time — which we shall initially assume, for notational simplicity — we can replace θ_t with θ.

The quantity $p(\theta|\mathcal{D}_{1:t})$ is known as the belief state, since it represents our current beliefs about the unknown parameters given the data so far. (It is also known as the filtering distribution, as mentioned in Section 1.6.) This can be computed using repeated applications of Bayes' rule:

$$p(\theta \mid \mathcal{D}_{1:t}) = \frac{p(y_t \mid x_t, \theta) p(\theta \mid \mathcal{D}_{1:t-1})}{p(y_t \mid x_t, \mathcal{D}_{1:t-1})} \tag{18.1}$$

where the prior $p(\theta|\mathcal{D}_{1:t-1})$ is the previous belief state, and the denominator is given by

$$p(y_t \mid x_t, \mathcal{D}_{1:t-1}) \stackrel{\text{def}}{=} Z_t = \int p(y_t \mid x_t, \theta) p(\theta \mid \mathcal{D}_{1:t-1}) \, \mathrm{d}\theta \tag{18.2}$$

Note that, in the batch (offline) setting, the product of the normalising factors across steps corresponds to the marginal likelihood of the data:

$$Z(\mathcal{D}) = p(y_{1:N} \mid x_{1:N}) = \prod_{n=1}^{N} Z_n \tag{18.3}$$

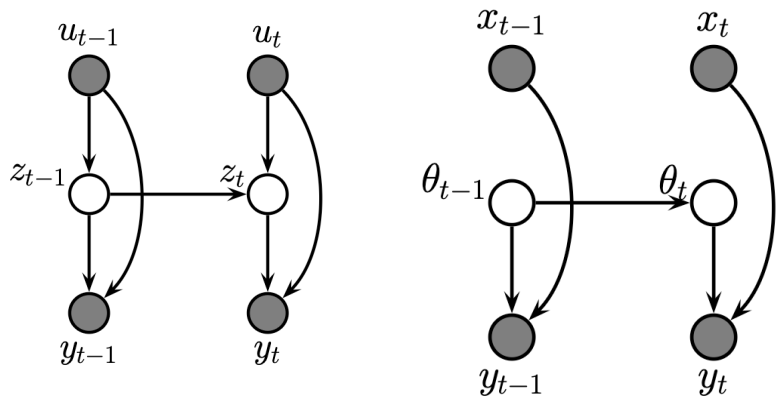
The key challenge we focus on in this chapter is how to efficiently compute the new belief state $p(\theta|\mathcal{D}_{1:t})$ at each step, when the unknown parameters $\theta \in \mathbb{R}^P$ are high dimensional, and where the likelihood is not conjugate to the prior. To make the problem computationally efficient (which is particularly important in online settings), we will focus on variational inference (VI) methods, which were introduced (in a batch setting) in Chapter 12. We will represent our approximate belief state at each step by $q_t = q(\theta|\mathcal{D}_{1:t})$. We will focus on the special case where this variational distribution is a Gaussian, $q_t = \mathcal{N}(\cdot|\mu_t, \Sigma_t)$, where μ_t is the posterior mean and Σ_t is the posterior covariance. (In many cases, it is more convenient to work with the posterior precision matrix, denoted $\Lambda_t = \Sigma_t^{-1}$.) We will discuss a variety of methods for efficiently updating these variational parameters, which will often require imposing additional assumptions on Σ_t (or Λ_t), in order to trade off expressivity with efficiency. We will also make connections between these Bayesian inference methods and optimisation algorithms, such as natural gradient descent (discussed in Chapter 15) and other second-order methods.

18.2 State Space Models

The methods we discuss in this chapter can be applied to many kinds of state estimation problems, not just estimating neural network parameters. We will therefore start with a general presentation before specialising it

to the BDL case. This will make it easier to see connections to the broader literature on Bayesian filtering (e.g., Särkkä and Svensson [1461]).

18.2.1 Modelling assumptions



(a) (b)

Figure 18.1: State Space Model (SSM) as a graphical model (shaded nodes are observed and unshaded are latent). (a) Standard SSM. (b) Specialized version for online parameter inference.

We start by defining a State Space Model (SSM), which is a way to define a joint distribution over a sequence of latent states z_t and observed outputs y_t, given a set of observed inputs or controls u_t. The model has the following form:

$$p(y_{1:T}, z_{1:T}|u_{1:T}) = p(z_1)p(y_1 \mid z_1, u_1) \prod_{t=2}^{T} p(y_t \mid z_t, u_t)p(z_t \mid z_{t-1}, u_t) \tag{18.4}$$

See Figure 18.1(a) for an illustration.

The distribution $p(z_t|z_{t-1}, u_t)$ is called the dynamics (or transition) model and specifies how the latent state changes over time. The distribution $p(y_t|z_t, u_t)$ is called the observation (or measurement) model and specifies the likelihood of the observations given the hidden state (and optionally the controls).

Given the above model, the online inference problem (also called Bayesian filtering) refers to recursively updating the belief state to give $p(z_t|y_{1:t}, u_{1:t})$, taking advantage of the Markov property of the SSM. We discuss algorithms for computing this in Section 18.3. In the offline setting, where the entire dataset is available, we may instead be interested in the Bayesian smoothing problem, which computes $p(z_{1:T}|y_{1:T}, u_{1:T})$ or corresponding marginals, such as $p(z_t|y_{1:T}, u_{1:T})$. However, this is beyond the scope of this chapter. (If you are interested in this case, see e.g., Courts et al. [336], Dowling et al. [448] for methods based on VI and Kamthe et al. [864] for a method based on Expectation Propagation.)

18.2.2 Examples

As a concrete example, consider the problem of tracking a moving object (such as an airplane) given noisy observations of its location. In this case, z_t represents the position and velocity of the object, y_t represent the observation (e.g., from a radar or camera), and u_t represents the input control signals chosen by the pilot which steer the plane. The belief state represents a distribution over the object's state given the noisy observations seen to date.

In the case of neural network parameter estimation, the latent state z_t corresponds to the parameters θ_t of the network, y_t represents the target label, and the input u_t represents the features or covariates x_t which are passed into the network. In this application, the hidden states (network parameters) are assumed to be unaffected by the inputs, so there is no edge from u_t to z_t in the graphical model. The resulting simplified model is shown in Figure 18.1(b). See Section 18.2.5 for more details.

18.2.3 Linear Dynamical Systems

We now consider an important special case of an SSM known as a Linear Dynamical System (LDS). This model is widely used because it supports exact inference, as we discuss in Section 18.3.2. In addition, it will be a building block for the approximate inference algorithms which we will consider later in this chapter.

In an LDS, both the transition model and dynamics model are conditional Gaussian distributions whose means are linear functions of their inputs, and whose variances are fixed. More precisely, we assume

$$p(z_t \mid z_{t-1}, u_t) = \mathcal{N}(z_t \mid A_t z_{t-1} + B_t u_t + a_t, Q_t) \tag{18.5}$$

$$p(y_t \mid z_t, u_t) = \mathcal{N}(y_t \mid C_t z_t + D_t u_t + c_t, R_t) \tag{18.6}$$

Here Q_t is the covariance of the system noise, and R_t is the covariance of the observation noise. If the process is stationary, then the model parameters $(A_t, B_t, C_t, D_t, a_t, c_t, Q_t, R_t)$ will be independent of t.

This formulation is known as the affine form of an LDS (see Särkkä and Svensson [1461, Sec 6.4]). We can remove a_t by appending a value of 1 to the input u_t and appending a_t as a column to B_t. Similarly we can remove c_t by absorbing it into D_t. However, sometimes it is convenient to explicitly include these bias (offset) terms, just like we do in neural networks. Alternatively, since the inputs are fixed, we can absorb $B_t u_t$ into a_t and $D_t u_t$ into c_t.

18.2.4 Nonlinear dynamical systems

The assumption that the SSM has a linear dynamics model and a linear observation model (Equations 18.5-18.6) is quite restrictive. However, it is easy to relax this by defining a Gaussian SSM where the mean is a nonlinear function of its input:

$$p(z_t \mid z_{t-1}, u_t) = \mathcal{N}(z_t \mid d(u_t, z_{t-1}), Q_t) \tag{18.7}$$

$$p(y_t \mid z_t, u_t) = \mathcal{N}(y_t \mid f(u_t, z_t), R_t) \tag{18.8}$$

Note that, since the noise is additive, we can rewrite the above model as follows:

$$z_t = d_t(z_{t-1}) + \epsilon_t \tag{18.9}$$

$$y_t = f_t(z_t) + \epsilon'_t \tag{18.10}$$

where $\epsilon_t \sim \mathcal{N}(0, Q_t)$ is the system noise, $\epsilon'_t \sim \mathcal{N}(0, R_t)$ is the observation noise, and we have defined the shorthands $d_t(z_{t-1}) = d(u_t, z_{t-1})$ and $f_t(z_t) = f(u_t, z_t)$.

18.2.5 SSMs for online Bayesian deep learning

In this section, we discuss how to formulate the problem of online neural network parameter estimation as inference in an SSM, where $z_t = \theta_t$ are the unknown network parameters, $u_t = x_t$ are the inputs to the network, and y_t are the observed outputs.

Observation model

The observation model f corresponds to the neural network that maps inputs to outputs given the hidden parameters, so $f_t(\theta_t) = f(x_t, \theta_t)$. The assumption that the output distribution is Gaussian is often reasonable for real-valued targets (although we may want to use more robust loss functions to handle outliers, as discussed in Duran-Martin et al. [467], Knoblauch et al. [904]). In this case, we often assume $R_t = \sigma_r^2 I$, where $\sigma_r > 0$ is a hyper-parameter.

However, when the targets are discrete labels or integer counts, we should replace the Gaussian observation distribution by a more suitable distribution from the exponential family. In this case, we assume $f_t(\theta_t)$ represents the mean parameter of the output distribution, and $h_t(\theta_t)$ represents the natural parameter.[1] For example, consider neural network classification. In this case, $f(x_t, \theta_t)$ represents the probability of each class label, and $h(x_t, \theta_t)$ represents the logits. We then have

$$p(y_t \mid \theta_t, x_t) = \text{Cat}(y_t \mid f(x_t, \theta_t)) = \text{Cat}(y_t \mid S(h(x_t, \theta_t))) \tag{18.11}$$

where S is the softmax function.

[1] Note that in Jones et al. [849], they use f for the natural parameters and h for the mean parameters, which is the opposite of our convention.

Dynamics model

It is common to assume (e.g., Kurle et al. [951], Titsias et al. [1617]) that the dynamics model is an Ornstein-Uhlenbeck process, i.e.,

$$p(\theta_t \mid \theta_{t-1}) = \mathcal{N}(\theta_t \mid d_t(\theta_{t-1}), Q_t) \tag{18.12}$$

$$d_t(\theta_{t-1}) = \gamma_t \theta_{t-1} + (1-\gamma_t)\mu_0 \tag{18.13}$$

$$Q_t = (1-\gamma_t^2)\Sigma_0 \tag{18.14}$$

where $0 \leq \gamma_t \leq 1$ is the degree of drift and $\mathcal{N}(\theta_0|\mu_0, \Sigma_0)$ is the prior. If $\gamma_t = 1$, then the dynamics is the identity function and there is no noise ($Q_t = 0$), so the latents are constant, i.e., $p(\theta_t|\theta_{t-1}) = \delta(\theta_t - \theta_{t-1})$.

The $(1-\gamma_t^2)$ factor in Equation 18.14 implies a process with stationary marginals $p(\theta_t) = \mathcal{N}(\theta_t|\mu_0, \Sigma_0)$. Another common choice is to use a "variance exploding" Gaussian random walk, where $\gamma_t = 1$ but $Q_t = \sigma_q^2 I$ for $\sigma_q > 0$. This can provide a form of regularisation. In this case, the quantity σ_q^2/σ_r^2, which is the ratio of the measurement precision to the dynamics precision, controls the learning rate of the system. (This can be seen by looking at the form of the Kalman gain matrix in Equation 18.25; see Ollivier [1258] for more discussion.) We discuss how to choose these hyper-parameters in Section 18.3.6.

18.3 Online Inference

We now discuss how to perform sequential (online) inference in SSMs using some common deterministic algorithms. (For a more detailed account, see e.g., Särkkä and Svensson [1461], Triantafyllopoulos [1634].)

18.3.1 Bayesian filtering

If we define $\mathcal{D}_t = (u_t, y_t)$, we can recursively estimate the belief state $p(z_t|\mathcal{D}_{1:t}) = p(z_t|u_t, y_t, \mathcal{D}_{1:t-1})$, by using a slight generalisation of Equation 18.1, as follows:

$$\underbrace{p(z_t \mid \mathcal{D}_{1:t-1}, u_t)}_{\text{posterior predictive}} = \int \underbrace{p(z_t \mid z_{t-1}, u_t)}_{\text{dynamics}} \underbrace{p(z_{t-1} \mid \mathcal{D}_{1:t-1})}_{\text{previous posterior}} \mathrm{d}z_{t-1} \tag{18.15}$$

$$\underbrace{p(z_t \mid \mathcal{D}_{1:t})}_{\text{new posterior}} = \frac{1}{Z_t} \underbrace{p(y_t \mid z_t, u_t)}_{\text{likelihood}} \underbrace{p(z_t \mid \mathcal{D}_{1:t-1}, u_t)}_{\text{prior}} \tag{18.16}$$

$$Z_t = \underbrace{p(y_t|\mathcal{D}_{1:t-1}, u_t)}_{\text{observation predictive}} \tag{18.17}$$

Equation 18.15 is called the predict step. This uses the Chapman-Kolmogorov equation to compute the one-step-ahead predictive distribution for the latent state, before seeing the outcome y_t; this becomes the prior for step t. Equation 18.16 is called the update step. This uses Bayes' rule to compute the posterior after seeing the outcome y_t. Finally, Equation 18.17 is the one-step-ahead predictive distribution for observations evaluated at y_t.[2]

18.3.2 Kalman filtering

In this section, we present the celebrated Kalman filter, which applies the Bayesian filtering equations from Section 18.3 to the LDS model defined in Section 18.2.3. The result is a recursive procedure for estimating the belief state $p(z_t|y_{1:t}, u_{1:t}) = \mathcal{N}(z_t|\mu_t, \Sigma_t)$. The derivation is well-known and can be found in many textbooks (see, e.g., Särkkä and Svensson [1461, Sec 6.3] or Murphy [1213, Sec 8.2.2]). For brevity, we drop the conditioning on the inputs u_t and absorb them into the bias terms.

The predict step is as follows:

$$p(z_t \mid y_{1:t-1}) = \mathcal{N}(z_t \mid \mu_{t|t-1}, \Sigma_{t|t-1}) \tag{18.18}$$

$$\mu_{t|t-1} = A_t \mu_{t-1} + a_t \tag{18.19}$$

$$\Sigma_{t|t-1} = A_t \Sigma_{t-1} A_t^\top + Q_t \tag{18.20}$$

[2] We call $p(y_t|u_t, \mathcal{D}_{1:t-1})$ the observation predictive, to distinguish it from $p(z_t|u_t, \mathcal{D}_{1:t-1})$, which is the posterior predictive. The former can be computed without the latter, as emphasized by the martingale posterior methodology of Holmes and Walker [773].

The predict step takes $O(P^3)$ time (recall that P is the dimension of the latent state).

The update step is as follows:

$$p(z_t \mid y_{1:t}) = \mathcal{N}(z_t \mid \mu_t, \Sigma_t) \tag{18.21}$$
$$\hat{y}_t = C_t \mu_{t|t-1} + c_t \tag{18.22}$$
$$e_t = y_t - \hat{y}_t \tag{18.23}$$
$$S_t = C_t \Sigma_{t|t-1} C_t^\top + R_t \tag{18.24}$$
$$K_t = \Sigma_{t|t-1} C_t^\top S_t^{-1} \tag{18.25}$$
$$\mu_t = \mu_{t|t-1} + K_t e_t \tag{18.26}$$
$$\Sigma_t = \Sigma_{t|t-1} - K_t S_t K_t^\top = \Sigma_{t|t-1} - K_t C_t \Sigma_{t|t-1} \tag{18.27}$$

Here e_t is the innovation or residual error, S_t is the innovation covariance, and K_t is the Kalman gain matrix. The update step takes $O(P^2C + PC^2 + C^3)$ time (recall that C is the dimension of the observation).

We see that the posterior mean is the prior predicted mean plus the gain matrix times the residual, and the posterior covariance is the prior covariance minus a positive definite term, indicating that the uncertainty is usually reduced after conditioning. This becomes clearer when we use the information form, where we work with precision matrices:

$$\Sigma_t^{-1} = \Sigma_{t|t-1}^{-1} + C_t^\top R_t^{-1} C_t \tag{18.28}$$

We see that the posterior precision is the prior precision plus a term which depends on the precision of the observations.

Finally, we note that the observation predictive is given by

$$p(y_t \mid y_{1:t-1}) = \mathcal{N}(y_t \mid \hat{y}_t, S_t) \tag{18.29}$$

from which we can compute Z_t.

18.3.3 Extended Kalman filtering

We now extend the Kalman filtering equations to the case of a nonlinear (but still Gaussian) SSM. We do this by linearising both the dynamics and measurement models around the current prior mean, and then performing a step of Kalman filtering. This is known as the Extended Kalman Filter (EKF), and was first developed at NASA [1524].

To explain the method in more detail, consider a nonlinear function $g(z)$. We can compute a linear approximation of this around some point μ using a first-order Taylor series expansion:

$$\hat{g}(z) = g(\mu) + J(z - \mu) \tag{18.30}$$

where $J = \text{Jac}(g)|_\mu$ is the Jacobian of g evaluated at μ (see also Section 8.1). If $g : \mathbb{R}^D \to \mathbb{R}^C$, then computing J costs $O(C|g|)$ using reverse mode auto-differentiation, or $O(D|g|)$ using forward mode, where $|g|$ is the cost of evaluating g. We can now approximate the original conditional Gaussian distribution, with nonlinear mean, by a conditional Gaussian whose mean is an affine function of its input:

$$\mathcal{N}(g(z), \Sigma) \approx \mathcal{N}(Az + b, \hat{\Sigma}) \tag{18.31}$$
$$A = \text{Jac}(g)|_\mu \tag{18.32}$$
$$b = g(\mu) - A\mu \tag{18.33}$$
$$\hat{\Sigma} = \Sigma \tag{18.34}$$

Applying this to our SSM, we get the following:

$$\hat{d}_t(z_{t-1}) = \mu_{t|t-1} + A_t(z_{t-1} - \mu_{t-1}) \tag{18.35}$$
$$\mu_{t|t-1} = d_t(\mu_{t-1}) \tag{18.36}$$
$$A_t = \text{Jac}(d_t)(\mu_{t-1}) \tag{18.37}$$
$$\hat{f}_t(z_t) = \hat{y}_t + C_t(z_t - \mu_{t|t-1}) \tag{18.38}$$
$$\hat{y}_t = f_t(\mu_{t|t-1}) \tag{18.39}$$
$$C_t = \text{Jac}(f_t)(\mu_{t|t-1}) \tag{18.40}$$

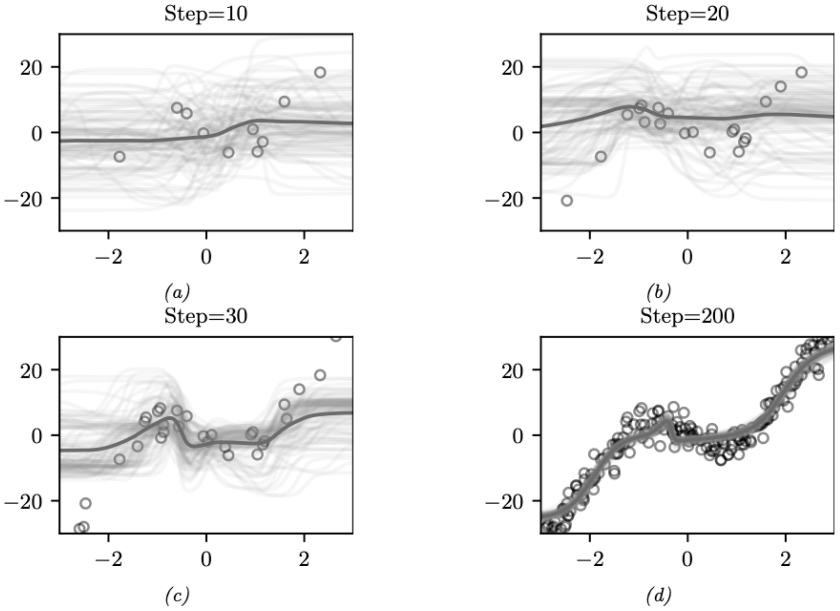


Figure 18.2: EKF applied to an MLP for a nonlinear 1d regression problem. We show samples from the posterior predictive distribution over the output space after seeing the first 10, 20, 30 and 200 observations. (For a video of this, see `https://github.com/probml/probml-data/blob/main/data/ekf_mlp_demo.mp4`.) The code to reconstruct this plot (written by Gerardo Duran-Martin) is at `https://github.com/probml/JSL/blob/main/jsl/demos/ekf_mlp.py`.

We then apply one step of Kalman filtering.

The EKF can be iterated, by relinearising the model around the updated posterior mean, instead of the prior mean. This is equivalent to a Gauss-Newton optimisation [111] and can improve the accuracy of the approximation.

The EKF can be applied to estimate the parameters of a neural network regression model as discussed in Puskorius and Feldkamp [1348, 1349], Singhal and Wu [1518]. See Figure 18.2 for an example. However, this takes $O(P^3)$ time per step, in general, which is intractable if the number of parameters is large. In addition, the method is restricted to Gaussian observations. In Section 18.4, we will present faster and more general methods.

18.3.4 Statistical Linear Regression

We now present an alternative way to linearise a nonlinear Gaussian system, known as Statistical Linear Regression (SLR) [1461], which can give higher accuracy but at increased computational cost. The basic idea is to compute the parameters of the linear Gaussian approximation by minimising an expected regression loss:

$$\mathcal{N}(g(z), \Sigma) \approx \mathcal{N}(Az + b, \hat{\Sigma}) \tag{18.41}$$

$$A, b = \arg\min \mathbb{E}_{y,z}[||y - Az - b||_2^2] = \arg\min \mathbb{E}_z[||g(z) - Az - b||_2^2] \tag{18.42}$$

$$A = \text{Cov}[y, z]\mathbb{V}[z]^{-1} \tag{18.43}$$

$$b = \mathbb{E}[y] - A\mathbb{E}[z] \tag{18.44}$$

$$\hat{\Sigma} = \mathbb{V}[y] - A\mathbb{V}[z]A^\top \tag{18.45}$$

(A similar method can be used for the dynamics model, substituting z_{t-1} and z_t for z and y.) The integrals needed to evaluate the relevant expectations and variances can be approximated using Gaussian quadrature. This gives rise to a variety of popular algorithms, including the Unscented KF [857] (also called sigma-point filtering), the Gauss-Hermite KF [42], and the cubature KF [41]. We can also iterate these methods, giving rise to the posterior linearisation filter [577].

These techniques have been applied to neural network parameter estimation (e.g., de Lima et al. [387]), but they are even less scalable than EKF-based methods. We therefore now turn to VI methods, which are much faster.

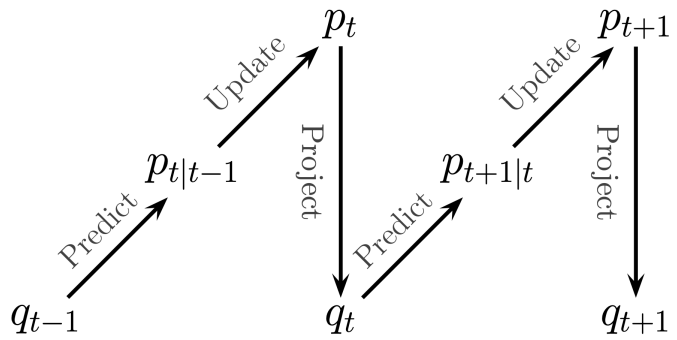


Figure 18.3: Illustration of the predict-update-project cycle for the measurement step, where $p_t = p_{t|t}$. We omit the corresponding predict-project step for the dynamics, which computes $q_{t|t-1} \approx p_{t|t-1}$, since this is often easy to compute.

18.3.5 Variational Inference

The methods discussed in Sections 18.3.3 and 18.3.4 for computing an approximate Gaussian posterior do not give any guarantees that the result is the optimal Gaussian approximation. We therefore turn to variational inference methods, where we explicitly solve for the optimal Gaussian approximation.

Predict step

In the predict step, we can approximate the integral in the Chapman-Kolmogorov equation 18.15 by minimising the forwards KL:

$$q_{t|t-1}(z_t) = \arg\min_{q \in \mathcal{Q}} \mathrm{D}_{\mathrm{KL}}\big(p_{t|t-1}(z_t) \big\| q(z_t)\big) \tag{18.46}$$

This can be solved by moment matching, as is well known, giving rise to

$$q_{t|t-1}(z_t) = \mathcal{N}(z_t|\mu_{t|t-1}, \Sigma_{t|t-1}) \tag{18.47}$$

$$\mu_{t|t-1} = \mathbb{E}_{p_{t|t-1}}[z_t] \tag{18.48}$$

$$\Sigma_{t|t-1} = \mathrm{Cov}_{p_{t|t-1}}[z_t] \tag{18.49}$$

In the case of nonlinear dynamics, we can use linearisation (as in the EKF discussed in Section 18.3.3) or Gaussian quadrature methods (as in the UKF discussed in Section 18.3.4) to approximate these moments. In the case of BDL with static parameters, where the dynamics is the identity function and the system noise is $Q_t = \sigma_q^2 I$, the predict step is much simpler, and reduces to $\mu_{t|t-1} = \mu_{t-1}$ and $\Sigma_{t|t-1} = \Sigma_{t-1} + \sigma_q^2 I$

Update step

In the update step, a natural approach is again to minimize the forwards KL:

$$q_t(z_t) = \arg\min_{q \in \mathcal{Q}} \mathrm{D}_{\mathrm{KL}}\big(p_{t|t}(z_t) \big\| q(z_t)\big) \tag{18.50}$$

$$p_{t|t}(z_t) = \frac{1}{Z_t} p(y_t|z_t) q_{t|t-1}(z_t) \tag{18.51}$$

This can be interpreted as taking the prior distribution, $q_{t|t-1}$, updating it using Bayes' rule, and then projecting the resulting posterior $p_{t|t}$ back to the space of tractable distributions $\mathcal{Q}$, as illustrated in Figure 18.3. This is known as Assumed Density Filtering [1152] or Minimum Divergence Filtering [372]. In the offline (smoothing) setting, this corresponds to the Expectation Propagation algorithm [1181] (see also Chapter 25). As in the predict step, minimising $\mathrm{D}_{\mathrm{KL}}(p\|q)$ amounts to matching the moments of the two distributions.[3]

[3]There is an interesting connection between ADF and the Gaussian Filtering (GF) methods discussed in Section 18.3.3 and Section 18.3.4. As noted in Wüthrich et al. [1783], we can interpret GF as first computing an approximate jointly Gaussian distribution $q(z_t, \tilde{y}_t)$ by minimising $\mathrm{D}_{\mathrm{KL}}(p(z_t, \tilde{y}_t|y_{1:t-1})\|q)$ using moment matching, and then conditioning this joint on the event $\tilde{y}_t = y_t$, which corresponds to the standard Gaussian updating procedure, to get an approximation $q_t(z_t|y_{1:t})$. By contrast, ADF first computes the (locally) exact posterior $p_t(z_t|y_{1:t})$ and then approximates it (by moment matching) to compute $q_t(z_t|y_{1:t})$. Thus ADF is more accurate but also much slower.

The ADF methodology has been applied to online neural network training, for example the expectation backpropagation algorithm of Soudry et al. [1545], the probabilistic backpropagation algorithm of Hernandez-Lobato and Adams [743] and the work of Ghosh et al. [621]. However, in high-dimensional spaces, ADF is intractable, since it first requires computing the exact one-step posterior $p_{t|t}$ before extracting its moments (which in turn requires computing high dimensional integrals).

We therefore turn to the more common VI method, in which we minimize the reverse KL, corresponding to information projection (resulting in mode seeking behaviour, rather than mode covering). This is more tractable than ADF and forwards KL since the expectations are wrt the chosen distribution q rather than wrt the true posterior p. More precisely, we compute

$$q_t(z_t) = \arg\min_q \mathrm{D}_{\mathrm{KL}}\big(q(z_t)\big\|p_{t|t}(z_t)\big) \tag{18.52}$$

$$= \arg\min_q \mathbb{E}_q\left[\log \frac{q(z_t)}{p(y_t|z_t)q_{t|t-1}(z_t)/Z_t}\right] \tag{18.53}$$

$$= \arg\min_q \underbrace{\mathrm{D}_{\mathrm{KL}}\big(q(z_t)\big\|q_{t|t-1}(z_t)\big) - \mathbb{E}_q[\log p(y_t|z_t)]}_{\mathcal{L}_{t|t}(\mu_t,\Sigma_t)} \tag{18.54}$$

where we dropped the constant Z_t from the last line since it does not depend on q and hence does not affect the arg min. The resulting objective takes the familiar form of an expected negative log likelihood (NLL) loss plus a KL regulariser to the prior, and corresponds to an online version of the negative ELBO (evidence lower bound).

It can be shown (e.g., Cao et al. [231], Jones et al. [849], Lambert et al. [965], Zeno et al. [1835]) that the fixed points of the objective $\mathcal{L}_{t|t}$ satisfy the following conditions:

$$\mu_t = \mu_{t|t-1} - \Sigma_{t|t-1}\mathbb{E}_{\mathcal{N}(z_t|\mu_t,\Sigma_t)}[\nabla_{z_t}\ell(z_t,y_t)] \tag{18.55}$$

$$\Sigma_t^{-1} = \Sigma_{t|t-1}^{-1} + \mathbb{E}_{\mathcal{N}(z_t|\mu_t,\Sigma_t)}[\nabla^2_{z_t}\ell(z_t,y_t)] \tag{18.56}$$

where $\ell(z_t, y_t) = -\log p(y_t|z_t)$ is the NLL loss.

Note that the unknown parameters μ_t, Σ_t occur on both the left and right hand sides of the above equations, forming an implicit update. Therefore, in general, we must iterate these equations to find a solution. We discuss suitable algorithms of this kind in Section 18.4. However, in the case of linear Gaussian systems, one can show that the expectations on the RHS have analytic solutions, so we can perform the update in one step, using the Kalman filtering equations discussed in Section 18.3.2.

18.3.6 State estimation vs parameter estimation

So far, we have assumed the parameters representing the dynamics model and the observation model are known. More precisely, we have been computing $p(z_t|\mathcal{D}_{1:t}, \phi_t)$, where ϕ_t are the parameters of the dynamics function d_t and the observation function f_t. This problem is called "state estimation". If the parameters are unknown, we can add them to the hidden state space and perform joint state and parameter estimation, by computing $p(z_t, \phi_t|\mathcal{D}_{1:t})$. This is known as "system identification" and is a much more complicated problem (see e.g., Courts et al. [335], Dowling et al. [448]).

Fortunately, in the case of BDL, the dynamics and observation models typically have very few free parameters (e.g., $\phi_t = (\gamma_t, \sigma_q)$ for OU dynamics in Section 18.2.5), since the unknown network parameters θ_t can be treated as hidden states z_t. We can therefore treat ϕ as hyper-parameters, which can be estimated using empirical Bayes (i.e., by optimising them wrt the marginal likelihood). For example, Titsias et al. [1617] optimise the drift term γ_t of the OU dynamics by maximizing the one-step-ahead log predictive likelihood, $p(y_t|y_{1:t-1}, \gamma_t)$, which can be computed from the normalization constant Z_t of the approximate posterior distribution. By optimising the drift term γ_t online, they can adapt to non-stationarity in the data.

18.4 Bayesian Online Natural Gradient

In this section, we present the Bayesian Online Natural Gradient (BONG) algorithm of Jones et al. [849], which minimizes the VI objective in Equation 18.54 using natural gradient descent. BONG is a version of the Bayesian learning rule (BLR) of Khan and Rue [883] (see Chapter 15) adapted to the online case. The BLR update also works by natural gradient descent on a negative ELBO, but it uses a batch objective rather than the recursive one in Equation 18.54. Thus BONG differs from BLR by incorporating the online inference tools presented in

Sections 18.2-18.3, namely dynamics models, iterative priors, and filtering-style updates with a sequential predict-update structure.[4] In addition, we discuss how to compute the various expectations required by these updates in an efficient manner. The main advantages of BONG are that it defines a computationally efficient update for Bayesian neural networks during online learning, and it provides a unified framework encompassing many existing methods.

18.4.1 Background

We first introduce some background material. We assume q_t is in the exponential family, where ψ_t are the natural parameters, and ρ_t are the dual (moment) parameters. For a Gaussian, the natural and moment parameters are given by

$$\psi_t = (\Sigma_t^{-1}\mu_t, -\tfrac{1}{2}\Sigma_t^{-1}) \tag{18.57}$$

$$\rho_t = (\mu_t, \mu_t\mu_t^\top + \Sigma_t) \tag{18.58}$$

It can be shown that the Jacobian of the mapping from dual parameters to natural parameters for any exponential-family distribution is equal to the Fisher information matrix:

$$F_{\psi_t} = \frac{\partial \rho}{\partial \psi}|_{\psi_t} \tag{18.59}$$

Consequently, the natural gradient wrt the natural parameters is equal to the regular gradient wrt the dual parameters, i.e.

$$F_{\psi_t}^{-1}\nabla_{\psi_t}\ell(\psi) = \nabla_{\rho_t}\ell(\rho) \tag{18.60}$$

Converting from the natural to the dual parameters has the same computational cost as computing the Fisher information matrix.

We continue to write the latent state as z_t and the input or controls as u_t, to reflect that our treatment applies to general state estimation problems. As a reminder, for online neural network parameter estimation we take $z_t = \theta_t$ as the unknown network parameters and $u_t = x_t$ as the inputs to the network.

18.4.2 Update step

We can minimize Equation 18.54, or maximize its negative, by performing iterative updates following the natural gradient direction. At iteration i, the update becomes

$$\psi_t^i = \psi_t^{i-1} + \alpha_t F_{\psi_t^{i-1}}^{-1}\nabla_{\psi_t^{i-1}}\left(\mathbb{E}_{\psi_t^{i-1}}\left[\log p(y_t|f_t(z_t)) - \mathrm{D}_{\mathrm{KL}}\left(q_{\psi_t^{i-1}}\middle\|q_{\psi_{t|t-1}}\right)\right]\right) \tag{18.61}$$

$$= \psi_t^{i-1} + \alpha_t \nabla_{\rho_t^{i-1}}\left(\mathbb{E}_{\psi_t^{i-1}}\left[\log p(y_t|f_t(z_t)) - \mathrm{D}_{\mathrm{KL}}\left(q_{\psi_t^{i-1}}\middle\|q_{\psi_{t|t-1}}\right)\right]\right) \tag{18.62}$$

where $\alpha_t > 0$ is a learning rate. (The Bayes By Backprop (BBB) method of Blundell et al. [174] uses a similar update, but uses standard gradient for ψ_t, instead of natural gradient. This requires more iterations due to the lack of preconditioning.)

If we initialise with $\psi_t^0 = \psi_{t|t-1}$, and just do a single iteration, the gradient of the KL regulariser vanishes, and we just perform a gradient update on the expected log likelihood, as follows:

$$\psi_t = \psi_{t|t-1} + \nabla_{\rho_{t|t-1}} \underbrace{\mathbb{E}_{\psi_{t|t-1}}[\log p(y_t|f_t(z_t)]}_{\overline{L}_{t|t-1}} \tag{18.63}$$

This is the approach used in the BONG method of Jones et al. [849]. Even though we have dropped the KL term, we are using an implicit (algorithmic) form of regularisation by initialising q_t at the prior and not moving far from it (since we just take one step). Furthermore, it can be shown that a single gradient step with unit learning rate ($\alpha_t = 1$) yields the exact Bayesian update in the conjugate case.

[4] When using a full covariance posterior, we will show that BONG takes $O(P^2)$ time per step whereas BLR takes $O(P^3)$. Both methods can be reduced to $O(P)$ time per step in the case of diagonal covariance, but BONG is still a constant factor faster, and is fully deterministic.

In Jones et al. [849], they show that the result of this explicit update is given by the following:

$$\Sigma_t^{-1} = \Sigma_{t|t-1}^{-1} - \underbrace{\mathbb{E}_{\mathcal{N}(z_t|\mu_{t|t-1},\Sigma_{t|t-1})}[\nabla_{z_t}^2 \log p(y_t|f_t(z_t))]}_{G_t} \tag{18.64}$$

$$\mu_t = \mu_{t|t-1} + \Sigma_t \underbrace{\mathbb{E}_{\mathcal{N}(z_t|\mu_{t|t-1},\Sigma_{t|t-1})}[\nabla_{z_t} \log p(y_t|f_t(z_t))]}_{g_t} \tag{18.65}$$

Algorithm 22: BONG algorithm, assuming static parameters

for $t = 1$ **to** ∞ **do**
- `// Predict`
- $\mu_{t|t-1} \leftarrow \mu_{t-1}$, $\Sigma_{t|t-1} \leftarrow \Sigma_{t-1}$
- `// Update`
- define loss $\ell_t(z_t) \leftarrow -\log p(y_t \mid u_t, z_t)$
- compute g_t and G_t using methods in Section 18.4.3 applied to ℓ_t
- compute Σ_t^{-1} using Equation 18.64
- compute μ_t using Equation 18.65

See Algorithm 22 for the resulting pseudocode. In Section 18.4.3, we explain how to compute the g_t and G_t terms, and in Section 18.4.4, we discuss the computational complexity of the algorithm. The bottom line is that, if we use a full covariance Gaussian, the method takes either $O(P^2)$ or $O(P^3)$ time, depending on algorithmic choices, which is intractable for most applications. Therefore, in Section 18.4.5 we consider a diagonal approximation, which only takes $O(P)$ time. Finally, in Section 18.4.6, we consider using a low rank representation, which is more expressive than diagonal but still scales linearly in P.

18.4.3 Computing the expectations

In Cao et al. [231] they approximate the relevant expectations using Gaussian quadrature, which they combine with natural gradient descent to get a method they call "NANO" (short for natural gradient Gaussian approximation filter). However, quadrature does not scale to high dimensions.

Another approach (used e.g., in the RVGA method of Lambert et al. [965]) is to use a Monte Carlo approximation, where we sample $z_t^m \sim \mathcal{N}(\mu_{t|t-1}, \Sigma_{t|t-1})$ and then compute

$$g_t^{\mathrm{MC}} = \frac{1}{M}\sum_{m=1}^{M} g_t^m, \;\; g_t^m = \nabla_{z_t^m} \log p(y_t|f_t(z_t)) \tag{18.66}$$

$$G_t^{\mathrm{MC}} = \frac{1}{M}\sum_{m=1}^{M} G_t^m, \;\; G_t^m = \nabla_{z_t^m}^2 \log p(y_t|f_t(z_t)) \tag{18.67}$$

Since computing the Hessian is intractable in high dimensions, we can replace G_t^{MC} with an empirical Fisher (EF) approximation (e.g., Martens [1135]), defined as follows:

$$G_t^{\mathrm{MCEF}} = -\frac{1}{M} G_t^{1:M} (G_t^{1:M})^\top, \;\; G_t^{1:M} = [g_t^1, \ldots, g_t^M] \tag{18.68}$$

Another approach is to linearise the observation model, as in the EKF (Section 18.3.3). In addition, we approximate the observation distribution with its first two (conditional) moments, following Ollivier [1258], Tronarp et al. [1637]. That is, we use the following approximate likelihood:

$$p_t^{\mathrm{LG}}(y_t|z_t) = \mathcal{N}(y_t|\hat{f}_t(z_t), R_t) \tag{18.69}$$

$$\hat{f}_t(z_t) = \hat{y}_t + C_t(z_t - \mu_{t|t-1}) \tag{18.70}$$

$$\hat{y}_t = f_t(\mu_{t|t-1}) \tag{18.71}$$

$$C_t = \mathrm{Jac}(f_t)|_{\mu_{t|t-1}} \tag{18.72}$$

$$R_t = \mathbb{V}\big[y_t|z_t = \mu_{t|t-1}\big] \tag{18.73}$$

This yields

$$g_t^{\text{Lin}} = C_t^\top R_t^{-1}(y_t - \hat{y}_t),\ G_t^{\text{LinHess}} = -C_t^\top R_t^{-1} C_t \tag{18.74}$$

To avoid having to compute the Jacobian (which takes $O(C|f|)$ time, and hence can be slow for high-dimensional targets), we can use an empirical Fisher style approximation:

$$g_t^{\text{LinEF}} = g_t^{\text{Lin}},\ G_t^{\text{LinEF}} = -g_t^{\text{Lin}}(g_t^{\text{Lin}})^\top \tag{18.75}$$

Finally, we can approximate the expectations by using a plugin or delta approximation, where we replace $\mathcal{N}(z_t|\mu_{t|t-1}, \Sigma_{t|t-1})$ with $\delta(z_t - \mu_{t|t-1})$. This gives rise to

$$g_t^\delta = \nabla_{z_t} \log p(y_t|f_t(z_t))|_{\mu_{t|t-1}},\ G_t^\delta = \nabla_{z_t}^2 \log p(y_t|f_t(z_t))|_{\mu_{t|t-1}}, \tag{18.76}$$

We can also combine the plugin approximation with the linearisation or empirical fisher approximation to get cheaper approximations to G_t:[5]

$$G_t^{\delta\text{Lin}} = G_t^{\text{LinHess}},\ G_t^{\delta\text{EF}} = -g_t^\delta (g_t^\delta)^\top \tag{18.77}$$

Thus in summary, we have identified 4 ways to approximate g_t: quadrature, sampling, linearisation, and plugin. Similarly, we have identified 7 ways to approximate G_t: quadrature, sampling (with or without EF), linearisation (with or without EF), and plugin (with or without EF). The different methods trade off speed vs accuracy, as we discuss in Section 18.4.4.

18.4.4 Computational complexity

In this section, we briefly discuss the computational complexity (running time) of the above algorithms. The time depends on three factors: (1) the form of the variational posterior q, (2) the method chosen for estimating g_t and G_t, and (3) whether we do a one step update on the implicitly regularised expected log likelihood (ELL) objective $\overline{L}_{t|t-1}$ defined in Equation 18.63, or do multiple update steps on the explicitly regularised ELBO objective $\mathcal{L}_{t|t}$ defined in Equation 18.54. The details can be found in Jones et al. [849], but we briefly summarize some key results here.

Suppose we use a full covariance q, we compute g_t and G_t by linearising the model, as in Equation 18.74, and we do a one-step update on the ELL objective. One can show that the resulting update is equivalent to the information form of the Kalman filter shown in Equation 18.28:

$$\Sigma_t^{-1} = \Sigma_{t|t-1}^{-1} - G_t = \Sigma_{t|t-1}^{-1} + C_t^\top R_t^{-1} C_t \tag{18.78}$$

$$\mu_t = \mu_{t|t-1} + \Sigma_t g_t = \mu_{t|t-1} + \Sigma_t C_t^\top R_t^{-1}(y_t - \hat{y}_t) \tag{18.79}$$

To avoid having to convert from Σ_t^{-1} to Σ_t, we can use the matrix inversion lemma, which gives the standard form of the KF, which we repeat here for convenience:

$$K_t = \Sigma_{t|t-1} C_t^\top (R_t + C_t \Sigma_{t|t-1} C_t^\top)^{-1} \tag{18.80}$$

$$\mu_t = \mu_{t|t-1} + K_t(y_t - \hat{y}_t) \tag{18.81}$$

$$\Sigma_t = \Sigma_{t|t-1} - K_t C_t \Sigma_{t|t-1} \tag{18.82}$$

This method, which is equivalent to the CM-EKF method of Ollivier [1258], Tronarp et al. [1637], takes $O(CP^2)$ time.

If we approximate G_t using the Monte Carlo EF approximation in Equation 18.68, as is done in the RVGA method of Lambert et al. [965], the time is $O(MP^2)$. If we approximate G_t using the linearised EF approximation in Equation 18.75, we can reduce the time to $O(P^2)$. All other variants (including the original BLR method of Khan and Rue [883]) take $O(P^3)$ time. Therefore, in the sections below, we discuss more restricted parameterisations for q, so we can reduce the complexity to linear in P.

[5] To define $G_t^{\delta\text{Lin}}$, we linearise the natural parameters of the observation distribution $h_t(z_t)$ instead of the mean parameters $f_t(z_t)$ as in Equation 18.69. See Jones et al. [849] for details and the proof that $G_t^{\delta\text{Lin}} = G_t^{\text{LinHess}}$.

18.4.5 Diagonal case

The easiest way to speed up the algorithm is to assume that the covariance matrix of the variational posterior is diagonal, i.e., we use $\psi_t = (\sigma_t^{-2}\mu_t, -\frac{1}{2}\sigma_t^{-2})$, or equivalently $\rho_t = (\mu_t, \text{diag}(\mu_t^2 + \sigma_t^2))$, where the product and exponents are taken elementwise. The relevant update equations can be found in Jones et al. [849, Sec E.3-E.4], but all can be done in $O(P)$ time.

When combined with the linearised Hessian approximation of Equation 18.74, and applied to the one-step ELL objective, we recover the variational diagonal EKF method of Chang et al. [258]. When combined with the MC Hessian approximation of Equation 18.67, and applied to the multi-step ELBO objective, we recover an online version of the VON algorithm from Khan et al. [881]. If instead we use the MC EF approximation of Equation 18.68, we recover an online version of the VOGN algorithm from Khan et al. [881]. (See also the recently improved VON method in Shen et al. [1510].) If we use standard gradient descent (instead of NGD), and optimise the mean parameters ρ_t instead of ψ_t, we recover an online version of the BBB method of Blundell et al. [174].

18.4.6 Low-rank case

Finally, we consider the case where the posterior precision matrix is diagonal plus low rank (DLR), i.e., $\Sigma_t^{-1} = D_t + W_t W_t^\top$, where D_t is a diagonal matrix of size $P \times P$, and W_t is a tall and skinny matrix of size $P \times R$, where $R \ll P$ is the rank of the approximation. This representation captures correlation among the latent parameters, unlike the diagonal representation, yet still scales linearly in the dimension, as we will see. (It is also possible to use the DLR approximation for the posterior covariance, Σ_t, as explained in Tomczak et al. [1622].)

Directly applying NGD to the ELL objective of Equation 18.63 or the ELBO objective of Equation 18.54 is challenging, because the Fisher information matrix for the DLR parameterisation cannot be easily inverted. We therefore first derive the update for the full covariance natural parameters, leveraging the fact that the prior $\Sigma_{t|t-1}^{-1}$ is DLR to make this efficient, and then use SVD to project the resulting posterior precision matrix back to low-rank form. This can be done in $O(R^2 P)$ time; for the details, see Jones et al. [849, Sec E.5].

If we combine this method with the linearised Hessian approximation for G_t given in Equation 18.74, and apply to the one-step ELL objective, we recover the LOFI (low-rank filtering) method of Chang et al. [259]. If instead we use the MC-EF approximation for G_t given in Equation 18.68[6] and apply to the iterated ELBO objective, we recover an online variant of the SLANG method of Mishkin et al. [1186]. (SLANG uses a randomized SVD algorithm from Halko et al. [702], whereas LOFI uses a deterministic SVD method.)

An alternative to the above methods is to use standard gradient descent to optimise the DLR parameterisation. This avoids the need to compute the Fisher information matrix. This simple scheme (which is called "BBB-DLR" in Jones et al. [849]) has not been experimentally evaluated, as far as we know.

We can also drop the diagonal term from the DLR representation, giving rise to a degenerate low-rank matrix. This is the basic idea behind the low-rank Kalman filter (LRKF) and variants described in Duran-Martin et al. [468]. In more detail, LRKF uses the approximation $\Sigma_t = W_t W_t^\top$. Note that the resulting belief state, $p(z_t|y_{1:t})$ is not a proper Gaussian distribution, since the covariance matrix is singular, but it is still a valid probability measure. Also the corresponding predictive distribution over observations, $p(y_{t+1}|y_{1:t})$, is generally a proper Gaussian. Although the lack of a diagonal term means we need to use a larger rank to capture a desired statistical performance, empirically the algorithm still offers a better compute-accuracy tradeoff than LOFI, which relies on the DLR representation.

18.4.7 Connections to second-order optimisation

In this section, we show an equivalence between the BONG algorithm — when combined with certain approximations and applied to the case of inference for static parameters — and standard second-order optimisation algorithms. In particular, consider the BONG update for Σ_t given in Equation 18.64 and μ_t given in Equation 18.65. For static parameters, we have $\mu_{t|t-1} = \mu_{t-1}$ and $\Sigma_{t|t-1} = \Sigma_{t-1}$, since there is no added noise. If we combine this with the delta-function (plugin) approximation to the expectations, we get g_t^δ from Equation 18.76 for the gradient, and $G_t^{\delta\text{EF}}$ from Equation 18.77 for the Hessian (assuming we also use the EF approximation).

[6] Note that, to implement the MC-EF method, we have to be able to sample from the DLR prior, without inverting the full matrix. An efficient algorithm for this, based on EM, is given in the LRVGA paper of Lambert et al. [966]. This takes $O(R(R+M)P)$ time to draw M samples.

The resulting update becomes

$$\Sigma_t^{-1} = \Sigma_{t-1}^{-1} + g_t g_t^\top \tag{18.83}$$

$$\mu_t = \mu_{t-1} + \Sigma_t g_t \tag{18.84}$$

The mean update is equivalent to a standard preconditioned gradient ascent update. Furthermore, the precision update is a cumulative sum of approximate Hessians, as in most second-order optimisers. In fact, if we use $\Sigma_t^{\frac{1}{2}}$ as the preconditioner in Equation 18.84, and add a learning rate, we exactly recover full-matrix Adagrad [458]. Most adaptive gradient methods (including Adagrad [458], Shampoo [691], and Adam [894]) use a square root preconditioner, but performance can be comparable or improved without it [1048]. There are also very close connections between this (delta function) approximation to BONG and mirror descent methods used in online learning (e.g., Chérief-Abdellatif et al. [289], Lyu and Tsang [1095], van der Hoeven et al. [1656]). More connections between VI and popular gradient based learning methods used in the deep learning community are discussed in Khan and Rue [883] (see Chapter 15).

18.5 Conclusions

This chapter has shown how we can learn the parameters of a deep neural network online using tools from Bayesian filtering. Incorporating an SSM for parameter dynamics allows the model to maintain plasticity and is especially well-suited for nonstationary or continual learning tasks. The resulting learning rules have a two-step structure: a predict step to advance the dynamics and an update step to assimilate the next observation into the belief state. linearising the network (and dynamics) makes these steps analytically tractable and allows us to use classic Kalman filter methods. More generally we can use variational inference for the update step to maintain a belief state within a given parametric family. In the Gaussian case, optimising the online variational objective is efficiently approximated by one step of natural gradient descent, formalised in the BONG update. Making the update computationally efficient leads to a family of methods indexed by how the relevant integrals and Hessians are approximated (including Monte Carlo, quadrature, linearisation, plugin, empirical Fisher) and how the posterior covariance is represented (diagonal, low rank, DLR, or full covariance). Open questions for future research include how to determine the optimal tradeoff between computational efficiency and model expressiveness or fidelity for a given problem; richer dynamic models or learning the dynamics for specific domains; a deeper understanding of the connection to second-order optimisers; and whether variational families other than Gaussians yield effective methods for certain problems.

Part IV

Ensemble methods

Chapter 19

Introduction and connections with approximate Bayesian inference

Deep ensembles [962] train the same neural-network architecture several times from different random initialisations and combine the resulting predictive distributions. Their appeal lies in their simplicity, straightforward parallelisation, competitive predictive performance, and broad applicability.

Deep ensembles were initially presented as a non-Bayesian alternative to approximate Bayesian inference methods such as variational inference (VI; Chapter 12) [962, 1274]. Subsequent conceptual and empirical work has shown that, in several studied settings and under realistic computational budgets, their predictive distributions can be closer to a high-fidelity Bayesian reference than those of standard single-basin approximations [826, 1765]. This does not make a standard deep ensemble an exact posterior approximation: its members are independently optimised solutions with equal weights, rather than posterior draws or basins weighted by posterior mass.

This chapter introduces the deep-ensemble procedure (Section 19.1), examines its connection with Bayesian model averaging (Section 19.2), discusses ensemble-based developments (Section 19.3), and considers the method under fixed computational budgets (Section 19.4).

19.1 Deep ensembling procedure

Let w denote the network parameters, let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denote the training data, and let J denote the number of ensemble members. Deep ensembling proceeds as follows:

1. For each $j = 1, \dots, J$, initialise the network independently and use stochastic gradient descent (SGD), or another optimiser, to obtain a local solution w_j. When the training objective is, up to an additive or multiplicative constant, the negative log-posterior $-\log p(w \mid \mathcal{D})$, these local solutions can be interpreted as local maximum a posteriori (MAP) estimates.

2. Form the equally weighted ensemble predictive distribution

$$\widehat{p}_{\mathrm{DE}}(y \mid x, \mathcal{D}) := \frac{1}{J} \sum_{j=1}^{J} p(y \mid x, w_j). \tag{19.1}$$

The original formulation trains probabilistic neural networks using proper scoring rules and optionally uses adversarially perturbed examples to encourage locally smoother predictive distributions [962]. For Gaussian-likelihood regression, member j commonly predicts

$$p(y \mid x, w_j) = \mathcal{N}\big(y \mid \mu_j(x), \sigma_j^2(x)\big)\,. \tag{19.2}$$

Although the resulting equal mixture is generally not itself Gaussian, its first two moments are

$$\begin{aligned} \bar{\mu}(x) &= \frac{1}{J}\sum_{j=1}^{J}\mu_j(x), \\ v_{\mathrm{DE}}(x) &= \frac{1}{J}\sum_{j=1}^{J}\sigma_j^2(x) + \frac{1}{J}\sum_{j=1}^{J}\big(\mu_j(x)-\bar{\mu}(x)\big)^2 \\ &= \frac{1}{J}\sum_{j=1}^{J}\big(\sigma_j^2(x)+\mu_j^2(x)\big) - \bar{\mu}(x)^2. \end{aligned} \tag{19.3}$$

The first contribution to $v_{\mathrm{DE}}(x)$ averages the per-model aleatoric variance, while the second measures between-model disagreement.

The procedure is simple but computationally expensive: training, storing, and evaluating J full models generally costs approximately J times as much as using one model. Because the training runs are independent, they can be distributed across devices, which reduces wall-clock time when sufficient parallel hardware is available but does not remove the total computation or storage cost.

19.2 Deep ensembles as approximate Bayesian inference

The Bayesian posterior predictive distribution, or Bayesian model average (BMA), is

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, w)\, p(w \mid \mathcal{D})\, \mathrm{d}w. \tag{19.4}$$

There are two useful perspectives on the relationship between deep ensembles and approximate Bayesian inference. First, an average of the form in Equation (19.1) is an ordinary Monte Carlo estimator of Equation (19.4) only when the w_j are samples from $p(w \mid \mathcal{D})$. Deep-ensemble members are instead local solutions obtained by optimisation from different initialisations, so their empirical distribution need not match the posterior in its locations, relative mass assigned to different basins, or within-basin spread [692].

The second perspective asks how a small number of evaluations of $p(y \mid x, w)$ should be allocated under a fixed computational budget. If predictive distributions vary less within a basin than across functionally distinct basins, representing several basins may capture more of the predictive variation relevant to Equation (19.4) than allocating all evaluations to one basin [1765]. This is a heuristic for finite-budget approximation, not a claim that independently optimised modes are superior to well-mixed exact posterior samples. Figures 19.1 and 19.2 illustrate this distinction.

This comparison also appears in image classification. Using full-batch Hamiltonian Monte Carlo (HMC) as a high-fidelity reference, Izmailov et al. [826] found that deep ensembles were substantially closer to the HMC predictive distribution than mean-field VI and standard SGD, and were competitive with several stochastic-gradient MCMC methods (Figure 19.3).

These results support a qualified revision of the usual description of deep ensembles. Rather than being dismissed as wholly unrelated to Bayesian inference, they can provide a useful finite-budget approximation to Bayesian predictive marginalisation and, in some settings, a closer approximation to a high-fidelity Bayesian reference than several traditional single-basin methods [826, 1765]. Their empirical success also motivates methods that combine coverage of multiple basins with explicit within-basin marginalisation, as discussed in Section 19.3.

The Bayesian interpretation should not be read literally, however. Standard deep ensembles assign equal weight to independently trained members rather than weighting basins by posterior probability or evidence. They do not sample from $p(w \mid \mathcal{D})$ and ordinarily represent between-member variation without explicitly representing uncertainty within each basin. It is therefore more accurate to describe Equation (19.1) as an equally weighted finite predictive mixture that may approximate the BMA than as a posterior-weighted model average.

It is also useful to distinguish Bayesian model averaging from model combination. As discussed by Minka [1182], BMA represents uncertainty among hypotheses in a specified probabilistic model using posterior weights, whereas model combination may be used to construct a richer predictive family. The members of a deep ensemble use the same neural-network architecture, but their predictive mixture may nevertheless represent distributions that no single member represents. This distinction is another reason not to identify a standard deep ensemble directly with the exact BMA. The same caution applies to MC Dropout [566]: its stochastic mask distribution defines an approximation rather than posterior sampling, so properties of the exact posterior cannot be assumed automatically.

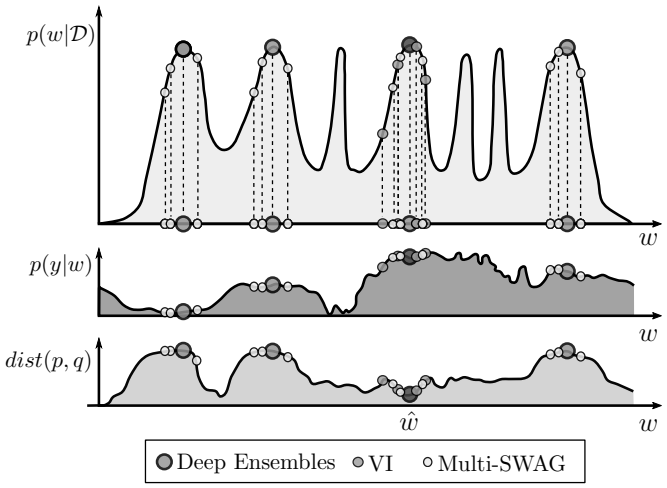


Figure 19.1: Conceptual illustration of finite approximations to the BMA $p(y \mid x, \mathcal{D}) = \int p(y \mid x, w)p(w \mid \mathcal{D})\,\mathrm{d}w$. Top: a multimodal parameter posterior together with three forms of approximation. The large circular markers represent deep-ensemble point estimates; the compact group of small markers around one mode represents a unimodal VI approximation; and the small markers distributed around several modes represent MultiSWAG, a mixture of local Gaussian approximations. Dashed vertical lines indicate the corresponding parameter-space evaluation locations. Middle: the predictive distribution $p(y \mid x, w)$ as a function of w for one test input x; in this illustration, predictions change less within a mode than between modes. Bottom: the discrepancy between the BMA and a finite approximation as a function of the location of one additional evaluation, given that one mode is already represented. Under the assumptions illustrated here, representing a previously unexplored basin reduces the discrepancy more than adding another evaluation within an already represented basin. Figure adapted from Wilson and Izmailov [1765].

Several works analyse different aspects of deep ensembles as approximate Bayesian inference [310, 368, 453, 718, 1306, 1765].

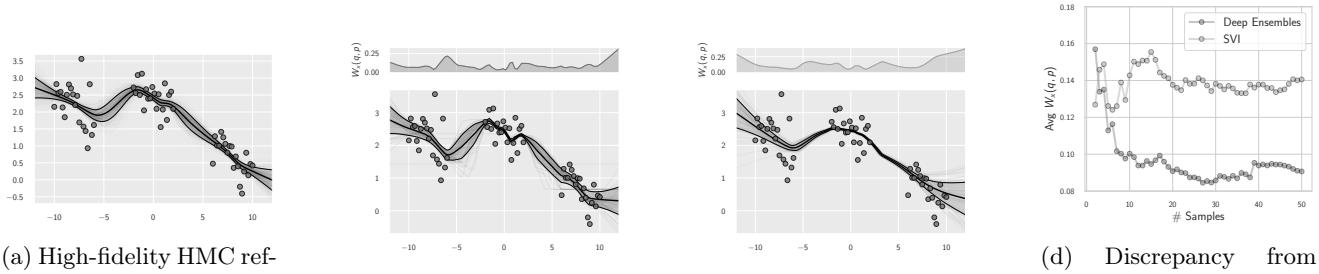
A central lesson is that practical Bayesian inference in deep learning is almost always approximate, so comparisons should state both the target distribution and the computational budget. Exact posterior draws, when available and representative of the posterior, are valid Monte Carlo inputs to the BMA in Equation (19.4). The practical question is how efficiently a method obtains evaluations that represent the predictive variation relevant to that integral. Deep ensembles can outperform less expensive single-basin approximations at fixed computational budgets, but this empirical observation should not be interpreted as a claim that deep ensembles are superior to well-mixed exact posterior sampling.

19.3 Developments of deep ensembles

A single Laplace or mean-field variational approximation (Chapter 13) usually represents uncertainty within one local basin. Deep ensembles instead represent several basins using one optimised point in each. These complementary properties motivate mixture approximations that perform explicit marginalisation within several basins.

Stochastic Weight Averaging–Gaussian (SWAG) [1116] fits a local low-rank-plus-diagonal Gaussian approximation using an SGD trajectory. MultiSWAG [1765] repeats this construction in several independently discovered basins and combines the resulting Gaussian components into a mixture approximation.

The approximate-inference competition reported by Wilson et al. [1770] received 337 submissions from 12 teams, and the highest-ranked methods all used multimodal ensemble constructions. The winning team used a uniformly weighted mixture of diagonal-Gaussian approximations, each fitted with improved Variational Online-Newton (iVON) [1047]. One of the shared second-place teams replaced the standard optimisation trajectories in MultiSWAG with stochastic-gradient Langevin dynamics (SGLD), while another used a computationally efficient sequential anchored-ensemble construction guided by MCMC [1306].



(a) High-fidelity HMC reference (b) Deep ensemble (c) Factorised VI (d) Discrepancy from HMC

Figure 19.2: Approximating a high-fidelity HMC reference predictive distribution on a one-dimensional regression task. **(a)** Reference predictive distribution obtained by combining 10 long HMC chains. **(b)** Deep-ensemble predictive distribution using 50 independently trained networks. **(c)** Predictive distribution from factorised variational inference. In panels **(b)** and **(c)**, the narrow upper strip reports the input-wise Wasserstein discrepancy from the HMC reference, while the lower plot shows the corresponding predictive distribution. **(d)** Average Wasserstein distance from the HMC reference as a function of the number of predictive components or samples. On this example, the multi-basin deep ensemble approaches the HMC reference more closely than the single-basin VI approximation, which is overconfident between data clusters. Figure adapted from Wilson and Izmailov [1765].

19.4 Are deep ensembles necessary?

Deep ensembles perform well in an absolute sense, but training, storing, and evaluating J full models is expensive. SWAG [1116] generally uses one main training trajectory followed by a posterior-collection phase, although drawing J predictive samples still requires J model evaluations. Deep kernel learning (Chapter 24) [1769] requires one forward pass through its feature extractor for each test input, followed by GP prediction in the learned feature space. A standard deep ensemble instead requires approximately J independent training runs and J forward passes at prediction time.

Several alternatives reduce the cost of obtaining diverse predictions, including BatchEnsemble [1739], snapshot ensembles [795], fast geometric ensembling [581], rank-1 Bayesian neural networks [474], and anchored ensembles [1306]. These methods make different trade-offs between diversity, memory, training cost, and fidelity to a Bayesian predictive distribution.

Any fair comparison should control for computational budget and for the property being evaluated. Retraining and averaging J copies of one network may outperform one training run of that network, but a single larger model may use the same computation more effectively when predictive accuracy is the only objective [3].

Predictive accuracy is not the only relevant objective, however. Deep-ensemble disagreement provides an operational measure of epistemic uncertainty that can support tasks such as exploration in Bayesian optimisation [78]. Simply increasing the size of a deterministic model does not, by itself, provide such uncertainty. An explicit uncertainty mechanism is required, such as an ensemble, a variational posterior (Chapter 13), a GP head (Chapter 24), or a Laplace approximation. Comparisons between ensembles and single models should therefore distinguish predictive accuracy from the quality and computational cost of uncertainty estimation.

19.5 Summary

Deep ensembles form an equally weighted predictive mixture from independently trained neural networks. They are not posterior samples or posterior-mass-weighted basin averages, but they can provide an effective finite-budget approximation to Bayesian predictive marginalisation when their members represent functionally distinct solutions. Their combination of simplicity, parallelisability, predictive performance, and useful disagreement estimates explains their practical success, while their linear training, storage, and prediction costs motivate more efficient and more explicitly Bayesian multi-basin approximations.

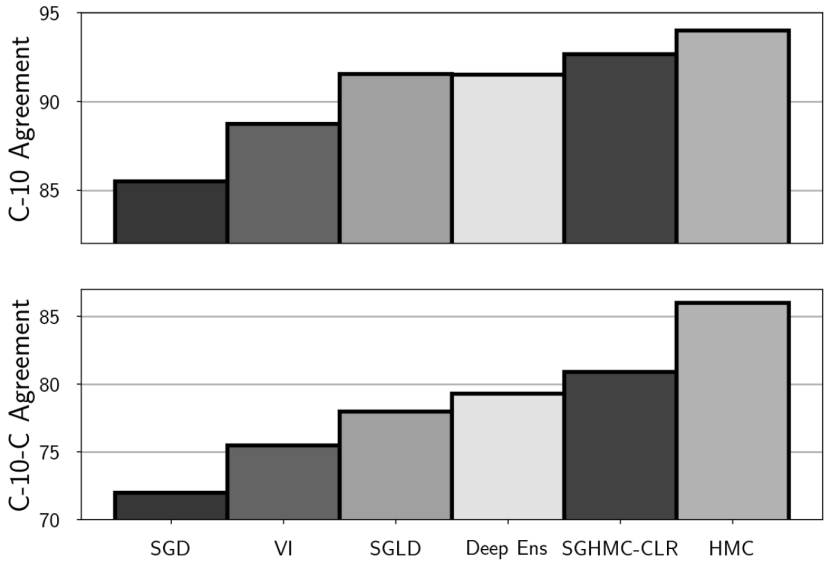


Figure 19.3: Agreement with a high-fidelity HMC reference on image classification. The upper panel reports agreement on clean CIFAR-10, while the lower panel reports agreement on CIFAR-10-C averaged across corruption types and intensities. The bars compare standard SGD, mean-field VI (labelled "VI" in the plot), stochastic-gradient Langevin dynamics (SGLD), deep ensembles, stochastic-gradient HMC with a cyclical learning-rate schedule (SGHMC-CLR), and an HMC reference. For the HMC reference, one HMC chain is compared with the ensemble of two other chains. Higher agreement indicates greater similarity of the top-one predictions. Deep ensembles are substantially closer to HMC than VI and are competitive with SGLD, while SGHMC-CLR is closer to HMC in both panels. All methods use ResNet-20-FRN trained on CIFAR-10. Figure adapted from Izmailov et al. [826].

Chapter 20

Hybrid ensemble sampling methods

This chapter highlights that the effectiveness of navigating high-dimensional, non-convex, and multimodal posterior landscapes typical in Bayesian deep learning can be significantly enhanced by hybrid approaches that combine sampling with optimization-based techniques, or ensembles thereof.

20.1 On the Efficacy of Optimization-Informed Initialization

The posterior landscapes of interest are typically non-convex and characterized by complex geometries, often featuring multiple, disjoint modal regions of significant probability mass. The successful navigation of this terrain under finite computational constraints necessitates a strategy for balancing global exploration against local exploitation.

To formalize the problem of posterior approximation via sampling, it is instructive to decompose the total approximation error. Following the framework presented in Sommer et al. [1535], this error may be attributed to three distinct sources:

1. **Initialization Error:** An error component induced by commencing the sampling trajectory at a point of low probability, distant from the posterior's *typical set*—the high-probability volume where the bulk of the posterior mass is concentrated [144].
2. **Discretization Error:** An error component arising from the numerical approximation of a continuous-time stochastic differential equation (e.g., the Langevin diffusion) by a discrete-time iterative algorithm.
3. **Monte Carlo (MC) Error:** A statistical error component due to the use of a finite ensemble of samples to estimate expectations under the true posterior. This error is inversely related to the number of effective samples drawn.

In this chapter, we are primarily concerned with the mitigation of initialization and Monte Carlo errors, as their interplay motivates the adoption of hybrid sampling strategies. Adjusting the discretization error can be done independently of this interplay.

Most MCMC samplers are designed to be ergodic processes in order to explore the full support of the target distribution. This exploratory capacity distinguishes them from optimization algorithms. Consider, for example, the relationship between Stochastic Gradient Descent and Stochastic Gradient Langevin Dynamics (SGLD; see Chapter 2). With potential $U(\theta) := -\log p(\theta \mid \mathcal{D})$, SGD performs gradient descent $\theta_{t+1} = \theta_t - \eta_t \nabla U(\theta_t)$. The unadjusted Langevin algorithm and its stochastic variant (SGLD) add isotropic Gaussian perturbations,

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{2}\nabla U(\theta_t) + \sqrt{\eta_t}\boldsymbol{\xi}_t, \qquad \boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{20.1}$$

Equivalently,

$$\theta_{t+1} = \theta_t + \frac{\eta_t}{2}\nabla \log p(\theta_t \mid \mathcal{D}) + \boldsymbol{\zeta}_t, \qquad \boldsymbol{\zeta}_t \sim \mathcal{N}(\mathbf{0}, \eta_t \mathbf{I}). \tag{20.2}$$

The injected noise facilitates transitions between modes, yet when far from the typical set it can slow the approach to high-density regions [1097, 1737].

Optimization procedures such as SGD [809], while typically not suited to sample from the posterior, are highly adept at rapidly identifying points of high posterior density, such as Maximum A Posteriori (MAP) estimates [1108]. This observation motivates a synergistic, two-stage methodology that integrates optimization and sampling [1533]. This hybrid approach may be conceptualized as follows:

1. **Optimization-based Initialization:** An optimization algorithm is first employed to find a parameter vector θ^* that resides within a high-density region of the posterior, effectively minimizing an objective such as the unnormalized negative log-posterior or the negative log-likelihood. This stage is designed to substantially mitigate the initialization error by positioning the sampler's starting point within or proximal to the typical set. For most relevant prior choices (having an unbounded support like an isotropic Gaussian), the obtained θ^* lies in a region of reasonably high prior density, making it a valid initialization for the MCMC procedure.
2. **Localized Posterior Sampling:** Subsequent to optimization, an MCMC sampler is initialized at θ^*. Relieved of the protracted and often inefficient burn-in phase (induced by the potentially very complex neural network posterior landscape) required when starting from a random point in the parameter space, the sampler can allocate its computational budget to the more focused task of characterizing the local posterior geometry, thereby more efficiently reducing the Monte Carlo error.

In other words, the practical imperative for such a hybrid strategy is to overcome the inefficiency and instability of samplers initialized from the prior, $p_0(\theta)$, particularly in high dimensions [1534]. As shown in Table 20.1, prior-initialized chains frequently fail to discover parameter configurations that yield meaningful predictive performance. This phenomenon is also demonstrated in Figure 20.1 for a simple overparameterized linear network $f(x) = w_1 w_2 x$ with known multimodal posterior, and maximum posterior density at $w_1 = w_2 = 1$ and $w_1 = w_2 = -1$. A sampler with an optimized initialization rapidly converges to and explores one of the true posterior modes, whereas a prior-initialized counterpart fails to explore higher density regions as well as the initialized one within the same budget of sampling steps. This gain in initial sampling efficacy is particularly important for high-dimensional models. Moreover, the computational overhead of the preliminary optimization phase is often negligible relative to the total cost of sampling, yet yields substantial improvements in the overall efficiency of posterior exploration and characterization [1534–1536]. Even in the very small-scale example provided in Figure 20.1, the first optimization stage only took 5% of the total runtime.

20.2 Parallelized Exploration for Mode Discovery

The hybrid two-stage strategy effectively solves the problem of finding an initial point close to the typical set. However, a significant challenge remains in posteriors characterized by multiple, well-separated modes (see Chapter 33 for related geometric and topological considerations). In high-dimensional spaces, the regions of low probability density that lie between these modes can practically form energy barriers for an MCMC sampler. Consequently, a single MCMC chain, even when run for a very long time, will likely remain confined to the basin of attraction of its initial mode.

This phenomenon is illustrated in Figure 20.1. The long-run simulation (bottom centre plot) is initialized in the posterior mode in the upper-right quadrant but never traverses the low-probability region around the origin to explore the second, symmetric mode in the lower-left quadrant. Relying on the formal ergodicity of a single chain to explore a multimodal landscape is, therefore, not a practically feasible strategy given that computation is usually constrained.

A simple and powerful solution is to leverage parallel computation by running multiple independent chains [600, 1421], a strategy shown in the lower right plot of Figure 20.1. The core idea is to initialize M different chains from distinct starting points $\{\theta_1^*, \ldots, \theta_M^*\}$. If these starting points end up in different basins of attraction, the combined collection of samples from all M chains will form a much more faithful representation of the global posterior. This approach not only provides a mechanism for exploring functionally distinct modes but also represents a more efficient path to reducing the overall Monte Carlo error compared to running a single, exceptionally long chain. This approach is particularly appealing if parallel computing resources are available.

20.3 Variations of Hybrid Sampling

The conceptual framework of initializing sampling with optimization can be realized through several distinct methodologies, ranging from classical techniques to modern ensemble-based approaches. These methods differ in their approaches to managing the transition from optimization to sampling and in how they address the fundamental challenge of multimodality.

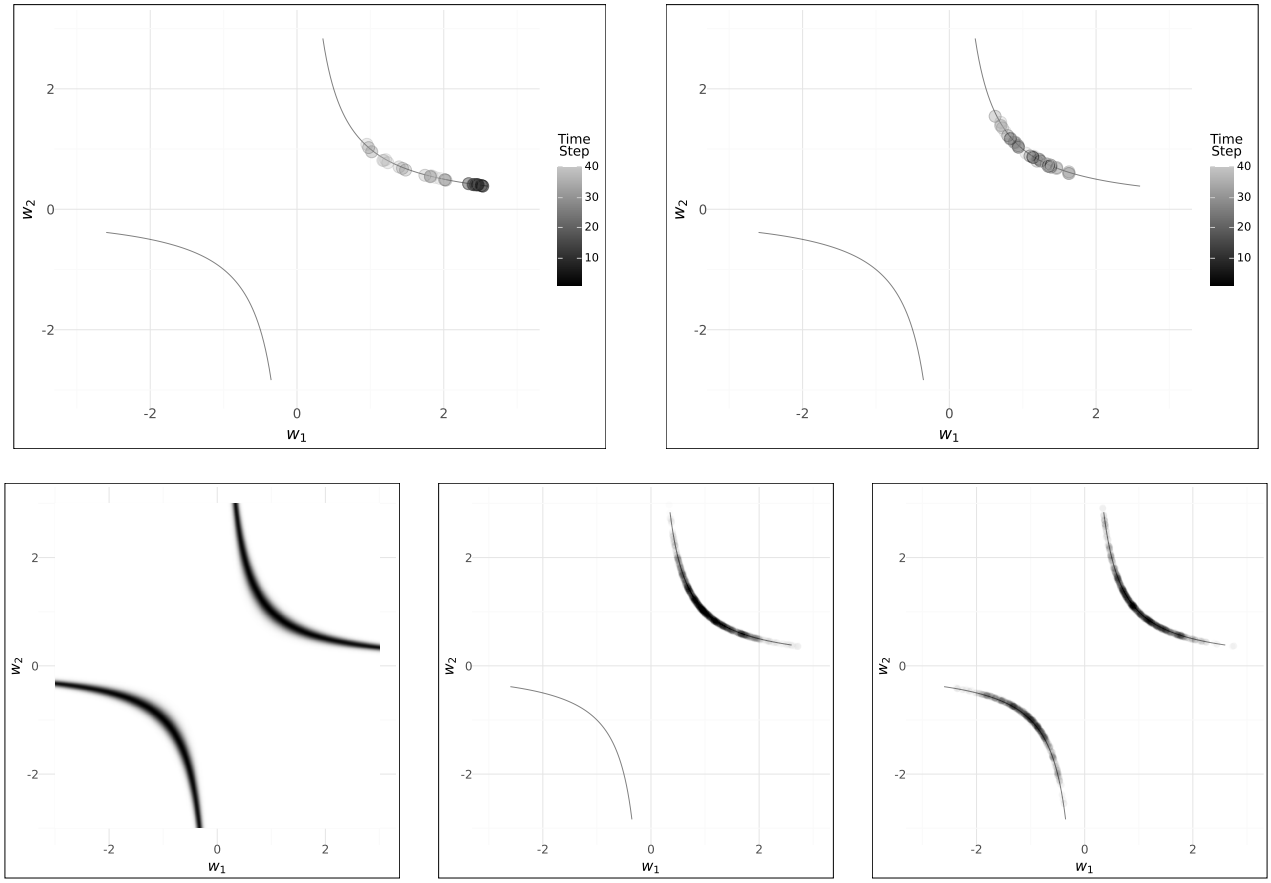


Figure 20.1: Upper left: Prior initialized sampling using the No U-Turn Sampler [NUTS; 762]. Upper right: Optimization initialized sampling using NUTS. Bottom left: Ground truth posterior obtained via numerical integration. Bottom centre: Long run sampling of a single NUTS chain from an optimized initialization. Bottom right: Samples obtained from multi-chain NUTS sampling with optimized initializations. For reference, the symmetry manifold $w_1 w_2 = 1$ is highlighted as a solid black line.

Table 20.1: The proportion of chains resulting in better root-mean-square error (RMSE) performance than a linear model for two UCI benchmark datasets [1638, 1813] (the higher the better). NUTS used 5,000 warmup samples. The Bayesian neural network used is a single hidden-layer Multi-Layer Perceptron with 50 hidden neurons. Values are averaged across 12 chains and 3 replications, and SD denotes standard deviation. Adapted from Table 5 of Sommer et al. [1534].

Activation	Data Set	Prop. RMSE < Linear Model	SD
ReLU	concrete	0.14	0.13
	energy	0.31	0.13
SiLU	concrete	0.14	0.10
	energy	0.28	0.05
tanh	concrete	0.58	0.25
	energy	0.67	0.08

20.3.1 MAP Initialization and Tempered Transitions

A common practice in the broader Bayesian literature is to initialize a single MCMC chain from a previously computed MAP estimate [1671]. This represents the most direct instantiation of the two-stage hybrid approach: a "hard" transition from a pure optimization objective to a pure sampling objective.

A more gradual transition can be achieved by sampling from a **tempered target distribution**, a technique that smoothly interpolates between optimization and sampling. Following Chen et al. [273], Duffield et al. [461], Welling and Teh [1737], we can define a tempered posterior indexed by a temperature $T \in (0, \infty)$ as

$$p_T(\theta|\mathcal{D}) \propto [p(\theta|\mathcal{D})]^{1/T}. \tag{20.3}$$

The temperature T controls the "flatness" of the distribution. In the low-temperature limit $T \to 0$, the distribution p_T concentrates on the global maximizers of the original posterior density, while the corresponding Langevin dynamics approach gradient-based optimization. Conversely, setting $T = 1$ recovers the original, untempered posterior distribution.

This provides a formal mechanism for annealing. As an example, we can modify the SGLD update rule by keeping the posterior drift fixed and scaling the injected Langevin noise by the temperature. For $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, this gives

$$\theta_{t+1} = \theta_t + \frac{\eta_t}{2}\left(\nabla \log p_0(\theta_t) + \frac{N}{N_{\text{batch}}} \sum_{i \in \mathcal{B}_t} \nabla \log p(y_i \mid x_i, \theta_t)\right) + \boldsymbol{\zeta}_t, \qquad \boldsymbol{\zeta}_t \sim \mathcal{N}(\mathbf{0}, \eta_t T \mathbf{I}). \tag{20.4}$$

Here, N is the dataset size, $\mathcal{B}_t \subset \{1, \ldots, N\}$ is a minibatch index set of size N_{batch}, and the minibatch likelihood term is an unbiased estimator of $\nabla \log p(\mathcal{D} \mid \theta_t)$. By starting with a small temperature $T \ll 1$ and gradually increasing it to $T = 1$ over a series of iterations, the algorithm transitions smoothly from an optimization-dominated phase that quickly finds a mode to a sampling phase that explores its local geometry. This perspective also helps to formalize the close relationship between optimization and sampling. For instance, the exploration phase of cyclical SGLD [1854], which aims to find new posterior modes, operates close to the zero-temperature limit during exploration phases, hence approaching optimization before resetting to $T = 1$ to sample.

20.3.2 Ensemble-Based Initialization for Multi-Modality

While a single optimized starting point can effectively initialize sampling within one posterior mode, it cannot resolve the broader challenge of exploring a multimodal landscape. As discussed previously, traversing the low-probability regions between modes is often infeasible under computational constraints. To approximate a posterior with functionally distinct solutions, one can leverage **Deep Ensembles** [DE; 962], as introduced in Chapter 19.

The independent training runs of a DE typically converge to different local minima, yielding a set of diverse parameter vectors $\{\theta_1^*, \ldots, \theta_M^*\}$. This collection of solutions provides an ideal set of starting points for multi-chain MCMC. The hybrid process then becomes the following:

1. Train a DE to obtain M distinct, high-performing models. For this step, one can also leverage previous empirical findings in the deep learning community and use readily available pre-trained weight vectors.

2. Initialize M parallel MCMC chains, with each chain m starting from its corresponding ensemble member's parameters, θ_m^*.

This DE-initialized sampling approach turns posterior exploration into a *post hoc* procedure that can be applied to already-trained models to enhance their uncertainty quantification. This method has been shown to effectively capture diverse posterior structures in various task settings [461, 1534–1536] and is also referred to as *Bayesian Deep Ensembles (BDEs)*.

20.3.3 Related and Emerging Techniques

The core idea of finding diverse, high-quality starting points has inspired several related methods, such as the following three:

- **SWA-based Initialization:** Stochastic Weight Averaging (SWA) is an optimization technique that averages weights from later epochs of an SGD trajectory, often finding wider, more generalizable minima [824, 1128]. Instead of averaging the points along the SGD trajectory as in Izmailov et al. [824], the late-epoch iterates can serve as a set of correlated but diverse initializations for MCMC chains, offering a cheaper alternative to training a full Deep Ensemble [1303].
- **Diffusion-based Localization:** Stochastic localization methods build on score-based learning by perturbing data samples, which in the setting of sampling-based inference would be the model weights, with noise instead of optimizing them and then progressively learning a denoiser [755, 1199, 1541]. For example one such recent work introduces Stochastic Localization via Iterative Posterior Sampling, the so called SLIPS sampler, which uses MCMC-estimated denoisers to approximate samples from complex target distributions, offering a promising new class of initialization techniques [665]. Currently, however, these diffusion-based localization techniques struggle to cope with the very high dimensional posteriors present in modern neural network architectures (see Chapter 50 for details on diffusion and score-based methods).
- **Symmetry-Informed Initialization:** We can also leverage the inherent symmetries in neural network architectures for initialization (see Chapters 32 and 33). For many networks, such as Multi-Layer Perceptrons, one can permute the neurons within a hidden layer without changing the function computed by the network. As shown in Kobialka et al. [908], one can take a *single* pre-trained model and apply a set of M random, function-preserving permutations to its weights. This generates a "pseudo-ensemble" of M parameter vectors that are numerically distinct but functionally identical. Using these permuted vectors to initialize MCMC chains still seems to provide enough diversity to generate a well-working BDE while being highly efficient, requiring only one trained model.

20.4 Limitations

While the ensemble-based hybrid methods discussed offer a powerful, practical solution to exploring multimodal posterior landscapes, they are not without their own set of challenges [1533]. These limitations define an active frontier of research in Bayesian deep learning.

20.4.1 Weighting across Modes

One of the most significant theoretical challenges is how to combine the samples from different chains exploring distinct posterior modes. When running M independent chains, let S_m denote the number of retained samples from chain m, and let $\theta_{m,j}$ denote the j-th retained parameter sample from this chain. A simple and common approach is to form the final predictive distribution by treating each chain as an equally weighted component in a mixture,

$$p(y^*|x^*, \mathcal{D}) \approx \frac{1}{M} \sum_{m=1}^{M} \left(\frac{1}{S_m} \sum_{j=1}^{S_m} p(y^*|x^*, \theta_{m,j}) \right). \tag{20.5}$$

However, this implicitly assumes that each of the M discovered modes contributes equally to the total posterior probability mass. This is rarely the case, as the true "weight" of a mode is its marginal likelihood, which is notoriously difficult to approximate in high dimensions, and its intractability is the reason for using MCMC in the first place. This problem of weighting components in a Bayesian model average is a classic challenge. Proposed solutions include, for example, stacking [1811, 1812], but in practice, most researchers and practitioners still resort to an equal weighting.

20.4.2 Computational and Resource Costs

Despite their relative efficiency compared to naïve sampling, hybrid methods remain computationally intensive. The initial step of training a DE requires the resources to train M separate neural networks. Subsequently, the MCMC sampling phase for each of the M chains adds further computational overhead. This combined cost, while often justifiable by the superior quality of the resulting uncertainty estimates, can be prohibitive for very large models or resource-constrained environments. Furthermore, storing the parameters for M full models and their corresponding posterior samples demands considerable memory and increases inference costs.

20.4.3 Fidelity of Mode Discovery

The entire framework rests on the assumption that the initial optimization phase successfully identifies diverse and meaningful posterior modes. However, this is not guaranteed under finite budget constraints. While unlikely, the members of a DE might converge to the same basin of attraction or to functionally almost equivalent solutions, limiting the exploratory advantage of the multi-chain approach [1534]. Moreover, there is a subtle mismatch between the objectives of the hybrid procedure: the optimizer seeks points that maximize the likelihood (or posterior density), while the Bayesian objective is to integrate over the entire typical set [144]. This can lead to the optimizer overlooking broad, flat modes that contribute significantly to the total posterior mass in favour of sharper, narrower peaks in the loss landscape [544].

20.4.4 Challenges of Local Exploration

Even after successfully initializing a chain in the close vicinity of a major posterior mode, fully exploring the local region can be difficult. The local geometry itself can be pathological, containing complex ridges, narrow funnels, or areas of high curvature that challenge even sophisticated samplers like Hamiltonian Monte Carlo [HMC, see Chapter 1; 456, 1229]. This can result in the sampler exploring only a small sub-region of the mode, leading to an underestimation of the local posterior variance. To avoid this, one could tune sampler hyperparameters such as its step size or mass matrix for each of the M chains. This, however, further complicates the practical application of these methods and stresses the need for high-performance and auto-tuned samplers.

Chapter 21

Practical applications of deep ensembles

Deep ensembles are related to Bayesian model averaging: under certain conditions an ensemble of independently trained networks can approximate the posterior predictive distribution and thus capture epistemic uncertainty, while also demonstrating robust performance across classification, regression, and calibration benchmarks. By training multiple neural networks independently and aggregating their predictions, deep ensembles approximate Bayesian model averaging — a long-standing principle for capturing epistemic uncertainty and improving generalization. Unlike many explicitly Bayesian neural network approaches, they achieve these benefits without requiring complex posterior inference 2 or specialized training procedures 15, making them both conceptually simple and computationally tractable. In this chapter, we examine how these properties translate to applied settings, highlighting concrete examples where deep ensembles enhance model reliability, interpretability, and decision-making under uncertainty. We also discuss how their strengths and limitations inform the design of scalable, uncertainty-aware systems.

21.1 Introduction

Deep ensembles have emerged as one of the most widely used techniques for uncertainty quantification in modern deep learning practice. The core idea is deceptively simple: independently train multiple neural networks with different random initializations and stochastic training paths, then aggregate their predictions at test time, typically via averaging. This approach is strongly motivated by the Bayesian paradigm, which advocates for posterior distributions instead of point estimates.

This simplicity, combined with strong empirical performance, has made deep ensembles an appealing off-the-shelf tool for practitioners. They routinely deliver improved predictive accuracy and well-calibrated uncertainty estimates across a variety of domains, including computer vision, natural language processing, and forecasting tasks. Several Bayesian deep learning libraries readily provide support for deep ensembles 47.

Deep ensembles have been used in practical applications in a variety of settings. In this section we will explore a non exhaustive list of practical applications of deep ensembles. Specifically we will cover applications in LLMs, weather forecasting, healthcare and reinforcement learning. Some of these applications (like in LLMs) can be surprising to the reader. Others, such as healthcare seem more intuitive given the high stakes sometimes involved in medical decisions. Given the high computational cost of deep ensembles, this must be offset by significant performance gains to justify their usage. In general, we can discern several common themes making deep ensembles useful in practice:

- Scarcity of data, here deep ensembles can be used to provide meaningful uncertainty estimates that can be used to obtain informative data in an online fashion. Ensembles can also be used to learn well-performing models that avoid overfitting.
- Large data diversity, here ensembles can be used to create specialized deep neural networks for subdivisions of the data. At test time the ensemble members can be recombined in various ways, often by first identifying the data point subdivision and applying the corresponding specialized ensemble member.
- Very high cost of predictive mistakes. Ensembles can provide well calibrated uncertainty estimates that can be used to avoid costly mistakes made by deep learning algorithms. Ensembles also tend to perform well for the task of selective classification, where the model can abstain from making a prediction when it is not confident enough.

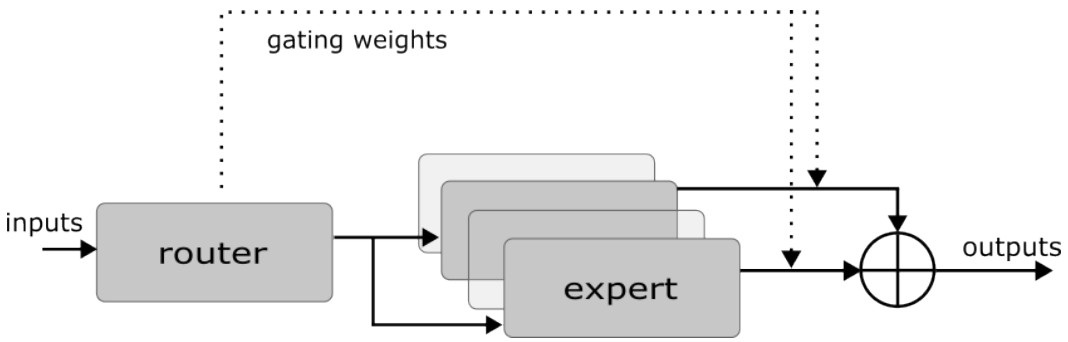


Figure 21.1: Mixture of Experts Layer. Each input vector is assigned to 2 of the 8 experts by a router. The layer's output is the weighted sum of the outputs of the two selected experts. In Mixtral, an expert is a standard feedforward block as in a vanilla transformer architecture. Image and description from Jiang et al. [843].

Ensembles have also been proposed and evaluated without explicit reference to Bayesian inference [962, 1274]. This relaxes the Bayesian formalism and as we will see allows for practical methods, related to but separate from strict theory. Also, practical applications often introduce strict computational or other requirements. Thus, researchers might sometimes benefit from taking the above themes into account, when developing new ensembling methods.

21.2 Applications to Large Language Models

We start our discussion with practical applications of deep ensembles for Large Language Models (LLMs). At the time of writing of this text Large Language Models are the fastest growing field of deep neural networks. Ensembles are used in variety of sometimes overlooked ways in modern Large Language Models. They underpin several crucial parts of models in production providing significant efficiency gains in inference, easing generalization to new tasks and making finetuning more stable. In the next sections we will explore these different applications in detail.

21.2.1 Mixture-of-Experts

A popular variant of deep ensembles widely used in modern LLMs are Mixture of Experts models. In vanilla Deep Ensembles inference is performed on all ensemble members and the results are then averaged. Mixture of Experts models use a learnable router which selects a subset of ensemble members to be activated for every input. Conceptually, MoE implements a form of conditional or dynamic ensembling: the router composes a subset of experts per input, producing routing diversity that differs from static ensemble averaging. The result is a highly expressive model (given the high number of total parameters) that is also fast at inference time (due to the low number of activated parameters). Well know examples of Mixture of Experts models include Mixtral-of-Experts [843] and DeepSeekR1 [391–394].

We first reproduce a brief overview of the Mixtral Mixture of Experts layer (Figure 1). Given an input x, a router $G(x)$ and expert layers $E_i(x)$ with $i \in \{0, n-1\}$, the output of the MoE module for a given input x is determined by the weighted sum of the outputs of the expert networks, where the weights are given by the gating network's output:

$$\sum_{i=0}^{n-1} G(x)_i \cdot E_i(x). \tag{21.1}$$

The crucial observation in this formulation is that, if the gating vector is sparse, one can avoid computing the outputs of experts whose gates are zero. This results in a model that is very flexible while being efficient at test time. The authors implemented the gating module by taking the softmax over the Top-K logits of a linear layer [1508]. Notably, more complicated methods have been explored in the literature [311, 717, 1875]. Concretely, the authors implement

$$G(x) := \text{Softmax}(\text{Top}_K(x \cdot W_g)) \tag{21.2}$$

where $(\text{Top}_K(\ell))_i := \ell_i$ if ℓ_i is among the Top_K coordinates of logits $\ell \in \mathbb{R}^n$ and $(\text{Top}_K(\ell))_i := -\infty$ otherwise. One can then modulate the amount of compute used to process each token, by appropriately choosing the value of K the number of experts used per token. In practice, the total number of experts n is kept much larger than the number of active experts K. For Mixtral the authors used the SwiGLU [843] architecture as the expert function

Model	Active Params	MMLU	HellaS
LLaMA 2 7B	7B	44.4%	77.1%
LLaMA 2 13B	13B	55.6%	80.7%
LLaMA 1 33B	33B	56.8%	83.7%
LLaMA 2 70B	70B	69.9%	85.4%
Mistral 7B	7B	62.5%	81.0%
Mixtral 8x7B	13B	70.6%	84.4%

Table 21.1: Comparison of Mixtral and LLaMA models on MMLU and HellaSwag benchmarks Jiang et al. [843].

$E_i(x)$. The number of total experts is $n = 8$ while the number of active experts is $k = 2$. Taking this all together, the output y for an input token x is computed as:

$$y = \sum_{i=0}^{n-1} \text{Softmax}(\text{Top}_2(x \cdot W_g))_i \cdot \text{SwiGLU}_i(x). \tag{21.3}$$

At training time different experts specialize on different parts of the data distribution resulting in a network that can model complicated data distributions. At test time, only experts relevant to the test sample are activated significantly reducing the computational load. In the context of the Mixtral model (and more generally LLM models), since the experts are applied independently on different input tokens, the Mixture of Experts module is used to replace fully connected layers of the architecture. We reproduce the following table of results

We observe that the Mixtral model is competitive with the LLaMA 2 70B model while using 5× fewer active parameters during inference.

We now describe an improved version of the mixture of experts architecture implemnented in the DeepSeekMoE model [393]. The crucial innovation is that the DeepSeekMoE uses finer-grained experts and isolates some experts as shared ones. Let u_t denote the FFN input of the t_{th} token. The authors compute the FFN output h'_t as follows:

$$h'_t = u_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} g_{i,t}\text{FFN}_i^{(r)}(u_t) \tag{21.4}$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{i,t}} \tag{21.5}$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Top}_k(\{s_{j,t} | 1 \le j \le N_r\}, K_r) \\ 0, & \text{otherwise} \end{cases} \tag{21.6}$$

$$s_{i,t} = \text{Sigmoid}(u_t^T e_i) \tag{21.7}$$

where N_s and N_r denote the numbers of shared experts and routed experts, respectively; $\text{FFN}_i^{(s)}(\cdot)$ and $\text{FFN}_i^{(r)}(\cdot)$ denote the i-th shared expert and the i-th routed expert, respectively; K_r denotes the number of activated routed experts; $g_{i,t}$ is the gating value for the i-th expert; $s_{i,t}$ is the token-to-expert affinity; e_i is the centroid vector of the i-th routed expert; and $\text{Top}_k(\cdot, K)$ denotes the set comprising K highest scores among the affinity scores calculated for the t-th token and all routed experts.

Importantly, routing diversity should be distinguished from Bayesian (posterior) or model uncertainty. Routing diversity arises from conditional composition — which experts are activated for a given input — and increases representational flexibility and conditional behavior. Posterior uncertainty, by contrast, refers to uncertainty over model parameters and predictions (epistemic uncertainty) and is typically estimated via Bayesian model averaging or independent deep ensembles. Router activations therefore do not by themselves provide calibrated epistemic uncertainty; when principled uncertainty estimates are required, routing-based models are best combined with ensemble or Bayesian techniques.

Another important innovation is the "Auxiliary-Loss-Free Load Balancing". This is designed to avoid routing collapse [1508] while retaining model performance [511, 940, 1592]. Specifically the authors introduce a bias term b_i [1706] for each expert and add it to the corresponding affinity scores $s_{i,t}$ to determine the top-K routing:

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Top}_k(\{s_{j,t} + b_j | 1 \le j \le N_r\}, K_r) \\ 0, & \text{otherwise.} \end{cases} \tag{21.8}$$

Benchmark (Metric)		DeepSeek-V2	Qwen2.5	LLaMA-3.1	DeepSeek-V3
# Shots		72B	236B	72B	671B
# Activated Params		21B	72B	405B	37B
# Total Params		236B	72B	405B	671B
Pile-test (BPB)		0.606	0.638	0.542	0.548
BBH (EM)	3-shot	78.8	79.8	82.9	87.5
MMLU (EM)	5-shot	78.4	85.0	84.4	87.1
MMLU-Redux (EM)	5-shot	75.6	83.2	81.3	86.2
MMLU-Pro (EM)	5-shot	51.4	58.3	52.8	64.4
DROP (F1)	3-shot	80.4	80.6	86.0	89.0
ARC-Easy (EM)	25-shot	97.6	98.4	98.4	98.9
ARC-Challenge (EM)	25-shot	92.2	94.5	95.3	95.3
HellaSwag (EM)	10-shot	87.1	84.8	89.2	88.9
PIQA (EM)	0-shot	83.9	82.6	85.9	84.7
WinoGrande (EM)	5-shot	86.3	82.3	85.2	84.9
RACE-Middle (EM)	5-shot	73.1	68.1	74.2	67.1
RACE-High (EM)	5-shot	52.6	50.3	56.8	51.3
TriviaQA (EM)	5-shot	80.0	71.9	82.7	82.9
NaturalQuestions (EM)	5-shot	38.6	33.2	41.5	40.0
AGIEval (EM)	0-shot	57.5	75.8	60.6	79.6

Table 21.2: Comparison of English benchmark performance for DeepSeek-V2, Qwen2.5, LLaMA-3.1, and DeepSeek-V3, reproduced from [393].

Note that the bias term is only used for routing. The gating value, which will be multiplied with the FFN output, is still derived from the original affinity score $s_{i,t}$. During training, the bias is tuned dynamically.

The authors further propose a “Sequence-Wise Auxiliary Loss". This loss is aimed at preventing extreme imbalance within any single sequence. Concretely the authors implement the following sequence-wise loss:

$$\ell_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \tag{21.9}$$

where the balance factor α is a hyper-parameter, which is assigned a small value, and where

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(\{s_{i,t} \in \text{Top}_k\left(\{s_{j,t} \mid 1 \leq j \leq N_r\}, K_r\right)\}) \tag{21.10}$$

and

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}, \quad \text{with} \quad s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}. \tag{21.11}$$

We observe in Table 21.2 that DeepSeek-V3 matches the performance of, or outperforms, the benchmark models while using significantly less activated parameters.

21.3 Applications to reinforcement learning

21.3.1 Reinforcement Learning from Human Feedback

The most known component of RL algorithms that is implemented with ensembles is Reinforcement Learning from Human Feedback (RLHF) [303]. RLHF is motivated by the difficulty in creating training signals for RL tasks such as open-ended text generation. To understand the difficulty we can compare with the case of image classification. For the task of image classification we can create a labeling of the dataset by first assigning an element of the set of natural numbers $\mathbb{N}$ to each class of the dataset, and train the network to predict this number for each labeled instance. For open-ended text it is much more difficult to assign an absolute number to the text quality as text quality has multiple degrees of freedom and includes subjective components. On the opposite, it is much easier to

	walker	hopper	swimmer	cheetah	ant	reacher	d-pendulum	pendulum
Ensemble	5000	5000	175	4500	−200	−3	7500	1250
No ensemble	3800	3900	60	3100	−300	−4	2500	1250

Table 21.3: Final reward of agents trained with RLHF on MuJoCo tasks before and after removing ensembling. We observe that ensembling results in significant improvements in final reward. Christiano et al. [303].

compare two texts and rank one as better than the other. RLHF utilizes such a ranking by human evaluators to learn a function (the reward function) that assigns consistent rankings to open-ended text. This function is then used to improve a policy given a state.

Let $\tau^A = (s, a^A)$ and $\tau^B = (s, a^B)$ be two (state, action) pairs and a reward model parametrized by ϕ. Human labelers provide pairwise preferences, such as:

$$\tau^A \succ \tau^B \quad \text{or} \quad \tau^B \succ \tau^A. \tag{21.12}$$

The learned reward function $\hat{r}_\phi$ produces a scalar reward for pair:

$$R(\tau) = \hat{r}_\phi(s, a). \tag{21.13}$$

The reward model assigns a probability that the question-answer pair τ^A is preferred over τ^B using a softmax:

$$P[\tau^A \succ \tau^B] = \frac{\exp(R(\tau^A))}{\exp(R(\tau^A)) + \exp(R(\tau^B))}. \tag{21.14}$$

Given a dataset of human preferences, the reward model is trained to minimize the following cross-entropy loss:

$$\ell_\phi = -\sum_{(\tau^A, \tau^B, \mu)} \left[\mu \log P[\tau^A \succ \tau^B] + (1-\mu) \log P[\tau^B \succ \tau^A]\right], \tag{21.15}$$

where $\mu = 1$ (resp. $\mu = 0$) if the human preferred τ^A (resp. τ^B), and $\mu = 0.5$ if the preference was a tie. The learned reward model is then used in standard reinforcement learning (e.g., PPO) to train a policy π_θ:

$$\max_\theta \mathbb{E}_{q\sim\mathcal{D}, q\sim\pi_\theta(q)} \left[\hat{r}_\phi(q, a)\right] \tag{21.16}$$

where $\mathcal{D}$ is the query distribution.

Labelling huge amounts of data is a significant and expensive bottleneck for training RL models. It is thus crucial to choose labeling examples judiciously. For this reason instead of fitting a single reward model to different preference triplets, the original RLHF paper proposed fitting an ensemble of reward models. These can then be used to estimate a measure of uncertainty about triplets, and choose for labelling only the ones with high uncertainty. Concretely, the algorithm is as follows [303]:

- Fit an ensemble of predictors, each trained on $|D|$ triples sampled from D with replacement. The estimate r is defined by independently normalizing each of these predictors and then averaging the results.
- A fraction of $1/e$ of the data is held out to be used as a validation set for each predictor. Use ℓ_2 regularization and adjust the regularization coefficient to keep the validation loss between 1.1 and 1.5 times the training loss. In some domains we also apply dropout for regularization.
- Instead of applying a softmax directly, assume there is a 10% chance that the human responds uniformly at random. This adjustment is made because human raters have a constant probability of making an error, which doesn't decay to 0 as the difference in reward difference becomes extreme.
- Decide how to query preferences based on an approximation to the uncertainty in the reward function estimator: sample a large number of pairs of trajectory segments of length k, use each reward predictor in the ensemble to predict which segment will be preferred from each pair, and then select those trajectories for which the predictions have the highest variance across ensemble members.

We close by noting that RLHF is a crucial component of finetuning modern LLMs to make them more safe and reliable [76, 1273].

21.3.2 Model Ensemble in Model-Based RL

In the literature there exist various other applications of deep ensembles in RL [31, 216, 832, 952, 1193, 1825], these introduce ensembles both for model-based and model-free RL methods. We elaborate more on Kurutach et al. [952], Yu et al. [1825] which are model-based RL methods and use ensembles to capture the world model uncertainty.

We present simplified versions of the algorithms below. Yu et al. [1825] uses multiple Gaussians with learned means and covariances to model the relationship between future states and rewards. It implements the Model-based Offline Policy Optimization (MOPO) algorithm 23 for model-based RL. Similarly Kurutach et al. [952] uses multiple world models to generate synthetic samples for learning the policy. It implements the Model Ensemble Trust Region Policy Optimization algorithm (ME-TRPO) algorithm 24.

In more detail, MOPO [1825] operates in the offline RL regime. The approach begins by training an ensemble of dynamics models — each model, for example, learns a Gaussian distribution over next states (mean and covariance) given current state and action. The ensemble variance thus provides a quantitative measure of model uncertainty. When using the learned models to simulate transitions for policy learning, MOPO subtracts a penalty proportional to this uncertainty from the simulated rewards. This reward-penalised simulated MDP is shown to induce a lower bound on performance in the true environment, thereby guarding against model exploitation and distributional shift. Empirically, MOPO surpasses prior offline RL methods on standard benchmarks. Meanwhile, ME-TRPO [952] targets the standard online model-based RL setup, and likewise employs an ensemble of learned world models to account for areas of high model error. ME-TRPO generates synthetic rollouts from the ensemble, updates the policy via a trust-region method (ensuring stable updates), and uses the ensemble disagreement to filter or penalise untrustworthy model trajectories. By doing so, ME-TRPO reduces real-environment interactions while maintaining policy robustness. Both methods highlight how leveraging ensembles of world-models — and using their predictive variance or disagreement — enables uncertainty-aware policy optimisation in model-based RL.

Algorithm 23: MOPO instantiation with regularized probabilistic dynamics and ensemble uncertainty

Input: Reward penalty coefficient λ, rollout horizon h, rollout batch size b
Output: Trained policy π
`// Train ensemble dynamics model`
Train on batch data $\mathcal{D}_{\text{env}}$ an ensemble of N probabilistic dynamics $\{T_i^b(s', r \mid s, a) = \mathcal{N}(\mu_i(s,a), \Sigma_i(s,a))\}_{i=1}^N$
Initialize policy π and empty replay buffer $\mathcal{D}_{\text{model}} \leftarrow \emptyset$
for *epoch* $= 1, 2, \ldots$ **do**
  `// Outer iteration of MBPO`
  for $= 1, 2, \ldots, b$ **do**
    Sample state s_1 from $\mathcal{D}_{\text{env}}$ for rollout initialization
    for $j = 1, 2, \ldots, h$ **do**
      Sample action $a_j \sim \pi(s_j)$
      Randomly pick dynamics T^b from $\{T_i^b\}_{i=1}^N$ and sample $(s_{j+1}, r_j) \sim T^b(s_j, a_j)$
      Compute $\tilde{r}_j = r_j - \lambda \max_{i=1,\ldots,N} \|\Sigma_i(s_j, a_j)\|_F$
      Add $(s_j, a_j, \tilde{r}_j, s_{j+1})$ to $\mathcal{D}_{\text{model}}$
  Draw samples from $\mathcal{D}_{\text{env}} \cup \mathcal{D}_{\text{model}}$ and update π

While the way in which ensembles are constructed differs significanty, we see that in both cases ensembles of models are used to maintain the world-model uncertainty and regularize the learning process. This in turn significantly improves the sample complexity and final rewards. We also reproduce here the reward results with and without ensembles for MOPO and MBPO for various Mujoco [1619] environments. We observe that ensembles improve both MOPO as well as the MBPO baseline.

21.4 Applications to weather forecasting

Another critical application of deep ensembles is in weather models. Ensemble methods provide important uncertainty estimates for weather models which can help in mitigating the effects of extreme weather conditions. Example works include Azizzadenesheli et al. [67], Chen et al. [269], Pathak et al. [1302], Weyn et al. [1744]. It has

Algorithm 24: Model Ensemble Trust Region Policy Optimization (ME-TRPO)

Input: Number of world models K
Output: Optimized policy π_θ
Initialize policy π_θ and world models $\{\hat{f}_{\phi_i}\}_{i=1}^K$; set dataset $\mathcal{D} \leftarrow \emptyset$
repeat
  Collect samples from real system f using π_θ and add them to $\mathcal{D}$
  Train all models on $\mathcal{D}$
  repeat
    Collect fictitious samples from $\{\hat{f}_{\phi_i}\}_{i=1}^K$ using π_θ
    Update π_θ with TRPO on fictitious samples
    Estimate performances $\hat{\eta}(\theta; \phi_i)$ for $i = 1, \ldots, K$
  until *performances stop improving*
until π_θ *performs well in real environment* f

Method	halfcheetah-mixed	walker2d-mixed	halfcheetah-jump	ant-angle
MOPO w/ ens.	6448.7 ± 115	1923.6 ± 752	3577.3 ± 461	2256.0 ± 288
MOPO w/o ens.	6405.8 ± 35	1916.4 ± 611	4016.6 ± 144	2530.9 ± 137
MBPO w/ ens.	5598.4 ± 1285	1021.8 ± 586	2971.4 ± 1262	13.6 ± 65
MBPO w/o ens.	2247.2 ± 581	500.3 ± 34	-68.7 ± 1936	-720.1 ± 728

Table 21.4: Performance comparison of different methods on various tasks MOPO [1825] and MBPO [832] reproduced from the original papers. We observe that ensembles improve both MOPO as well as the MBPO baseline.

been long established that ensembles of traditional numerical methods, using different initial conditions, result in improved weather estimates. The key innovation of most deep learning papers is in the speed of computation for each ensemble member. Deep neural networks can significantly speed up the solution of the Partial Differential Equations needed to predict weather patterns. This in turn allows to use significantly larger ensembles than what would have otherwise been possible. For example the speed of FourCastNet [1302] enables the creation of rapid and inexpensive large-ensemble forecasts with thousands of ensemble-members for improving probabilistic forecasting, and is orders of magnitude faster than traditional approaches.

21.5 Applications to medical data

There is a large number of works that utilize ensembles for medical applications [32, 775, 841, 1718, 1746]. These range from using ensembles for reliable predictions in clinical time series [1746] to reliable detection of Alzheimer's disease [841]. We elaborate more on Wickstrøm et al. [1746]. Since this work operates on time-series data, it serves to highlight the diverse settings in which deep ensembles improve test metrics. Here the authors explore the class activation mapping (CAM) method [1870], for explainable deep learning prediction. CAM was originally developed for image data, but has also been shown to be applicable for temporal data such as clinical time series [58, 1717]. By utilizing DNNs with a particular processing structure, CAM can assign a score to each time step in an input that indicates how important that time step is for a given prediction. Example cases of time-series data are mortality prediction, detection of infections, and patient treatment trajectory prediction.

Let $w_{c,k}$ denote the weight connecting the kth filter in the last convolutional block with the neuron corresponding to class c in the output layer, and $z_{k,t}$ denote the activation at time step t produced by the kth filter in the last convolutional block of the FCN. Then, the input to the final softmax function in the output layer can be expressed as:

$$g_c = \sum_{k=1}^{K} w_{c,k} \sum_{t=1}^{T} z_{k,t} = \sum_{t=1}^{T} r_{c,t}, \tag{21.17}$$

where K is the number of filters in the last convolutional block, T is the length of the time series, and $r_{c,t}$ denotes the relevance score of time step t for class c:

$$r_{c,t} = \sum_{k=1}^{K} w_{c,k} z_{k,t}. \tag{21.18}$$

Removing all negative relevance scores can be achieved by modifying Equation (21.18) as follows:

$$r_{c,t} = \max\left(0, \sum_{k=1}^{K} w_{c,k} z_{k,t}\right). \tag{21.19}$$

To calculate the ensemble mean and standard deviation of the relevance scores, let each CNN be parametrized by a set of parameters $\{\theta_1, \ldots, \theta_M\}$, where M is the number of models in the ensemble. Then, when considering the positive relevance scores from Equation (21.19), the mean relevance across the ensemble is defined as:

$$\mu_r = \frac{1}{M} \sum_{m=1}^{M} r_{\theta_m}, \tag{21.20}$$

and the standard deviation across the ensemble is defined as:

$$\sigma_r = \sqrt{\frac{1}{M-1} \sum_{m=1}^{M} \left(r_{\theta_m} - \mu_r\right)^2}. \tag{21.21}$$

In all experiments the authors scale the relevance scores between 0 and 1 using the following equation:

$$r_c \leftarrow \frac{r_c - \min(r_c)}{\max(r_c) - \min(r_c)}. \tag{21.22}$$

If the standard deviation is above some threshold, the relevance scores are set to zero. This can be formulated as:

$$\tilde{\mu}_{r_t} = \begin{cases} \mu_{r_t}, & \text{if } \sigma_{r_t} < \varepsilon, \\ 0, & \text{else,} \end{cases} \tag{21.23}$$

where the threshold ε is chosen to fit the particular data.

This approach results in significant improvement in explainability over the single network. To calculate how accurate a model is at locating relevant time steps, the authors compare the k most relevant time steps for a prediction with the known most relevant time steps. For a given sample i, let

$$Y_i = \{y_1^{(i)}, \ldots, y_{N_r}^{(i)}\}, \tag{21.24}$$

denote the set of relevant points, where N_r is the number of known relevant time steps in the time series, and let $R_i(k)$ denote the k most relevant time steps for the prediction of the model.

Perfect relevance accuracy is achieved when all elements of Y_i are contained in $R_i(k)$. For a given sample i, relevance accuracy can be calculated as:

$$\text{RelevanceAccuracy}_i(k) = \frac{|R_i(k) \cap Y_i|}{|Y_i|}. \tag{21.25}$$

For a given sample i, computing the relevance consistency across M models can then be achieved by:

$$\text{RelevanceConsistency}_i(k) = \frac{1}{M} \sum_{m,n=1}^{M} \frac{\left|R_i^{(m)}(k) \cap R_i^{(n)}(k)\right|}{k}. \tag{21.26}$$

We reproduce here some results on the EGG200 dataset [374] which consists of EGG time-series that trace the electrical activity recorded during a single heartbeat. The task is to discriminate between normal heartbeats and those exhibiting signs of myocardial infarction (heart attack). Table 21.5 shows the relevance consistency of 10 single and 10 ensemble models on both datasets.

We observe that ensembles significantly improve the relevance consistency of the relevance scores.

21.6 Applications to autonomous driving

Ensemble learning has become an important strategy for improving the reliability, robustness, and uncertainty estimation of autonomous driving systems, particularly for trajectory forecasting in safety-critical environments. By combining the predictions of multiple models, ensembles reduce overconfidence, improve generalization, and

Top k	Single	Ensemble
$k = 5$	0.57 ± 0.15	0.82 ± 0.07
$k = 7$	0.60 ± 0.15	0.84 ± 0.07
$k = 10$	0.63 ± 0.14	0.85 ± 0.06
$k = 15$	0.67 ± 0.13	0.88 ± 0.05

Table 21.5: Performance comparison of Single vs. Ensemble models on ECG200 dataset. The numbers indicate the statistical significance at a significance level of 0.01.

provide more accurate representations of the multimodal nature of future vehicle and pedestrian behaviors. Traditional deep ensembles, however, incur substantial computational and memory costs, limiting their deployment in real-time autonomous driving applications. Recent research [957, 958] addresses this challenge through efficient ensemble architectures, such as lightweight transformer ensembles and packed or grouped subnetworks, which retain the predictive diversity and uncertainty calibration of conventional ensembles while significantly reducing computational overhead. These approaches enable autonomous vehicles to generate more reliable long-term trajectory predictions, better quantify uncertainty, and make safer planning and decision-making under complex and ambiguous traffic scenarios.

21.7 Patents on deep ensembles

Patents on deep ensembles demonstrate the growing relevance of ensemble-based deep learning techniques across diverse real-world applications. For example, [685] illustrates how ensemble-based approaches can improve decision-making in autonomous vehicles by incorporating uncertainty estimates alongside object detection outputs, enabling safer and more reliable predictions. Similarly, [222] demonstrates the use of ensemble deep learning models for enhancing digital experiences by combining multiple machine learning estimators with neural networks to generate improved recommendations. Furthermore, [1484] highlights how ensemble machine learning techniques can improve future event detection by combining models with different prediction characteristics to balance accuracy and timeliness. These patents collectively demonstrate the practical relevance of deep ensembles in improving robustness, uncertainty estimation, adaptability, and predictive performance across complex real-world applications.

Chapter 22

PAC-Bayesian deep neural network ensembles

PAC-Bayesian analysis is a frequentist technique for deriving generalisation bounds for randomised classifiers. As a special case, PAC-Bayesian analysis can be used to provide generalisation bounds for Bayesian classifiers. We provide a basic introduction to PAC-Bayesian analysis and discuss similarities and distinctions from Bayesian learning. We then discuss ensemble classifiers and weighted majority votes and provide first and second order oracle bounds on the expected loss of weighted majority votes. The oracle bounds relate the expected loss of a weighted ensemble to the expected loss of a randomised classifier. Since the expected loss of a randomised classifier is an unknown (oracle) quantity, the bounds are called oracle bounds. We apply PAC-Bayesian analysis to bound the expected loss of randomised classifiers in terms of their empirical loss, thereby converting oracle bounds into empirical ones. The empirical bounds can be used to provide generalisation guarantees, as well as optimise weighting of weighted majority votes.

22.1 A basic introduction to PAC-Bayesian analysis

PAC-Bayesian analysis was invented by Shawe-Taylor and Williamson [1507] and McAllester [1154]. Their motivation was to introduce a possibility to incorporate prior knowledge within the frequentist learning framework. The name *PAC-Bayesian* is derived from *PAC*, standing for *Probably Approximately Correct* learning framework proposed by Valiant [1652], and *Bayesian*, which refers to the method's ability to incorporate prior knowledge in the analysis. PAC-Bayesian analysis is a technique for deriving frequentist generalisation bounds for randomised classifiers. A *randomised classifier* defined by a distribution ρ on a set of parameters Θ acts by sampling a new parameter vector $\theta \in \Theta$ according to $\rho(\theta)$ each time it classifies a new data point.

Definition 22.1 (Randomised Classifier)**.** *Let ρ be a distribution over Θ. A* randomised classifier *associated with ρ (and named ρ) is defined by the following protocol. At each prediction round it:*

1. *Samples $\theta \in \Theta$ according to $\rho(\theta)$*
2. *Observes x*
3. *Returns $f(x, \theta)$*

We use $\ell(f(x,\theta), y)$ to denote a loss function for predicting $f(x,\theta)$ when the true label is y. In contrast to most other chapters in the book, where ℓ is the negative log-likelihood, most results in this chapter assume that $\ell(\cdot,\cdot)$ is bounded in the $[0,1]$ interval. For example, $\ell(\cdot,\cdot)$ could be the zero-one loss function. We use $\mathcal{L}(\theta) = \mathbb{E}[\ell(f(x,\theta),y)]$ to denote the expected loss of θ and $\hat{\mathcal{L}}(\theta,\mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}\ell(f(x_i,\theta),y_i)$ the empirical loss of θ. Then the expected loss of ρ is $\mathbb{E}_{\theta\sim\rho}[\mathcal{L}(\theta)]$ and the empirical loss of ρ is $\mathbb{E}_{\theta\sim\rho}\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$. Whenever it causes no confusion, we will shorten the notation to $\mathbb{E}_{\rho}[\mathcal{L}(\theta)]$ and $\mathbb{E}_{\rho}\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$.

There exist numerous forms of PAC-Bayesian bounds. We start with one due to McAllester [1155], which is convenient for interpreting the nature of PAC-Bayes. It is a relaxation of a tighter bound of Seeger [1483] that will be shown later in Theorem 22.3, together with a proof of both results.

Theorem 22.1 (Interpretation-friendly PAC-Bayesian inequality)**.** *Assume that the loss function ℓ is bounded in the $[0,1]$ interval. For any "prior" distribution π over Θ that is independent of $\mathcal{D}$ and any $\delta \in (0,1]$*

$$\mathbb{P}\left(\forall \rho : \mathbb{E}_\rho[\mathcal{L}(\theta)] \leq \mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right] + \sqrt{\frac{2\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]\left(\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}\right)}{N}} + \frac{2\left(\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}\right)}{N}\right) \geq 1-\delta. \quad (22.1)$$

We emphasise that Theorem 22.1 is a frequentist generalisation bound and the random variable inside the probability statement is $\mathcal{D}$ (and thus $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$), but not θ (and thus not $\mathbb{E}_\rho[\mathcal{L}(\theta)]$).

We can restate the result of the theorem verbally by saying that with probability at least $1-\delta$, for all "posterior" distributions ρ

$$\mathbb{E}_\rho[\mathcal{L}(\theta)] \leq \mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right] + \sqrt{\frac{2\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]\left(\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}\right)}{N}} + \frac{2\left(\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}\right)}{N}. \quad (22.2)$$

We wrote "prior" and "posterior" in quotation marks, because the semantic meaning of PAC-Bayesian priors and posteriors differs from Bayesian priors and posteriors. In PAC-Bayes a prior distribution can be any distribution that does not depend on the data $\mathcal{D}$ used in the computation of the bound. It can be used to represent a prior belief on the performance $\mathcal{L}(\theta)$ of different parameters θ, but the bound holds irrespective of the source of π. And the bound holds for all posterior distributions ρ. In particular, it allows for the selection of ρ based on the data $\mathcal{D}$. The bound also holds for the Bayes posterior, but is not restricted to it.

Since Theorem 22.1 holds for all posterior distributions ρ, it can be used to find a distribution ρ that yields the best guarantee on the expected loss $\mathbb{E}_\rho[\mathcal{L}(\theta)]$ according to the bound in equation (22.2). A distribution ρ that minimises the bound should balance between placing a high mass on the parameters θ with a low empirical risk $\hat{\mathcal{L}}(\theta,\mathcal{D})$ in order to minimise the $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$ term, and staying close to the prior distribution π in the D_{KL} sense to keep $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ under control. The $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ term measures the complexity of a distribution ρ. If $\rho = \pi$, then $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) = 0$, meaning that if the posterior takes no information from the sample and remains the same as the prior, then the complexity is zero. While the bound holds for the Bayes posterior, it is typically *not* the Bayes posterior that minimises the right hand side of equation (22.2).

The prior distribution π is a tool to steer learning before revealing the data. If the prior allocates high probability mass to a subset of the parameter space that happens to work well on the data (happens to achieve low $\hat{\mathcal{L}}(\theta,\mathcal{D})$ after the data is revealed), the posterior ρ minimising the right hand side of equation (22.2) will not need to deviate much from π, leading to a small $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ term and a tight bound. Otherwise the bound might be loose, but it always remains a valid bound.

The bound in Theorem 22.1 provides a "fast convergence rate". Specifically, if for some ρ the empirical risk $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$ happens to be of order $O\left(\frac{1}{N}\right)$, the last two terms in equation (22.2) scale with $\frac{1}{N}$, which is known as "fast rate", as opposed to scaling of the middle term with $\frac{1}{\sqrt{N}}$ for large $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$, which is known as "slow rate". In other words, when $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$ is small, with high probability it is even closer to $\mathbb{E}_\rho[\mathcal{L}(\theta)]$ than when it is large. In the context of deep learning, where reaching zero empirical risk is often possible, it provides an extra benefit for allocating higher posterior mass to well-performing parameters and bringing $\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]$ close to zero.

Theorem 22.1 is convenient for explaining the nature of PAC-Bayes, but it is not the most convenient for optimisation, because the bound is not convex in ρ. The following result by Thiemann et al. [1599] provides an optimisation-friendly version of the bound. Similar to Theorem 22.1, it is a relaxation of Theorem 22.3 that we discuss later, and it is at least as tight as Theorem 22.1 (see Tolstikhin and Seldin [1620]).

Theorem 22.2 (Optimisation-friendly PAC-Bayesian inequality)**.** *Assume that the loss function ℓ is bounded in the $[0,1]$ interval. For any prior distribution π over Θ that is independent of $\mathcal{D}$ and any $\delta \in (0,1]$*

$$\mathbb{P}\left(\forall \rho : \mathbb{E}_\rho[\mathcal{L}(\theta)] \leq \min_{\lambda\in(0,2)}\left(\frac{\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]}{1-\frac{\lambda}{2}} + \frac{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}}{\lambda\left(1-\frac{\lambda}{2}\right)N}\right)\right) \geq 1-\delta. \quad (22.3)$$

In other words, with probability at least $1-\delta$ over the draw of a sample $\mathcal{D}$, for all distributions ρ over Θ and all $\lambda \in (0,2)$

$$\mathbb{E}_\rho[\mathcal{L}(\theta)] \leq \frac{\mathbb{E}_\rho\Big[\hat{\mathcal{L}}(\theta,\mathcal{D})\Big]}{1-\frac{\lambda}{2}} + \frac{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}}{\lambda\big(1-\frac{\lambda}{2}\big)N}. \tag{22.4}$$

For a fixed λ the right hand side of the inequality (22.4) is convex in ρ (because $\mathbb{E}_\rho\Big[\hat{\mathcal{L}}(\theta,\mathcal{D})\Big]$ is linear in ρ and $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ is convex in ρ), and for a fixed ρ it is convex in λ (see Tolstikhin and Seldin [1620]). Therefore, the bound in Theorem 22.2 is amenable to alternating minimisation. For a fixed λ the bound is minimised by

$$\rho^*_\lambda(\theta) = \frac{\pi(\theta)e^{-\lambda N\hat{\mathcal{L}}(\theta,\mathcal{D})}}{\mathbb{E}_\pi\Big[e^{-\lambda N\hat{\mathcal{L}}(\theta',\mathcal{D})}\Big]} \tag{22.5}$$

(we leave the derivation as an exercise) and for a fixed ρ the bound is minimised by

$$\lambda^*_\rho = \frac{2}{\sqrt{\frac{2N\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]}{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{N}}{\delta}}+1}+1} \tag{22.6}$$

(see Tolstikhin and Seldin [1620]). Alternating application of the update rules (22.5) and (22.6) leads to a local minimum of the bound (22.4). We note that the bound (22.4) is not jointly convex in ρ and λ, which would have been a sufficient, but not a necessary condition for convergence to the global minimum. Thiemann et al. [1599] provide sufficient conditions under which the bound (22.4) is strongly quasi-convex in ρ and λ, which guarantees convergence of alternating minimisation to the global minimum, as well as examples of cases where the strong quasi-convexity conditions are violated and the procedure only converges to a local minimum.

22.1.1 Similarities and differences between PAC-Bayesian analysis and Bayesian learning

Theorems 22.1 and 22.2 provide sufficient background to compare PAC-Bayesian analysis to Bayesian learning. We start with the similarities and then discuss the differences.

Similarities to Bayesian learning

1. Similar to Bayesian learning, PAC-Bayesian analysis provides an explicit way to incorporate prior knowledge via a prior distribution π.

2. Similar to Bayesian learning, it also allows sequential updates of the prior knowledge π as more data become available. We refer to the work of Wu et al. [1781] on Recursive PAC-Bayes, which is a procedure for sequential prior knowledge update without loss of confidence information.

Differences with Bayesian learning

1. PAC-Bayesian analysis is a frequentist learning framework. The random variables in PAC-Bayesian analysis are the observations (the data $\mathcal{D}$), but not the parameters θ. The prior π in PAC-Bayesian analysis is just an auxiliary construction for deriving the bound (as will be formally shown in the next section). It is not a prior belief in the Bayesian sense on the distribution from which the parameters of a "true" model generating the data were sampled. PAC-Bayesian bounds are valid for any choice of the prior. The choice of a prior impacts the tightness of the bound, but never compromises its validity, as long as the prior is independent of the data on which the bound is computed.

2. PAC-Bayesian posteriors are averaging distributions $\rho(\theta)$ and the bounds hold for all possible posterior distributions $\rho(\theta)$. Bayesian posteriors are conditional distributions $\rho(\theta|\mathcal{D})$. Given a Bayesian prior $\pi(\theta)$ and a dataset $\mathcal{D}$, the Bayesian posterior is uniquely determined. Thus, the semantics of the PAC-Bayesian and Bayesian posteriors are different.

3. The loss function ℓ is a central element in PAC-Bayesian analysis, because the PAC-Bayesian objective is to bound the expected risk $\mathbb{E}_\rho[\mathcal{L}(\theta)]$. To work with a PAC-Bayesian bound, one has to define a loss function, and a change of the loss function changes the minimiser ρ of the bound. In contrast, the definition of the Bayes rule, $\rho(\theta|\mathcal{D}) = \pi(\theta)\mathbb{P}(\mathcal{D}|\theta)/\mathbb{P}(\mathcal{D})$, is not based on a loss function.

4. To draw further parallels and distinctions between Bayesian and PAC-Bayesian learning, assume that $f(x, \theta)$ returns a probability distribution on $\mathcal{Y}$ and that the loss function ℓ is the negative log-likelihood. Then the conditional data probability $\mathbb{P}(\mathcal{D}|\theta)$ can be written as $\mathbb{P}(\mathcal{D}|\theta) = e^{-N\hat{\mathcal{L}}(\theta,\mathcal{D})}$ and the Bayesian posterior can be written as $\rho(\theta) = \frac{\pi(\theta)e^{-N\hat{\mathcal{L}}(\theta,\mathcal{D})}}{\mathbb{E}_\pi\left[e^{-N\hat{\mathcal{L}}(\theta',\mathcal{D})}\right]}$, which corresponds to taking $\lambda = 1$ in equation (22.5). This connection is useful to underline two important differences.

   (a) PAC-Bayesian analysis relies on convergence of empirical estimates of the risk $\hat{\mathcal{L}}(\theta, \mathcal{D})$ to their expectations $\mathcal{L}(\theta)$. The convergence holds for bounded loss functions and, therefore, the assumption that the loss function is bounded in the $[0, 1]$ interval is common in the PAC-Bayesian literature. Unbounded loss functions, including the negative log-likelihood, do not guarantee such convergence by default and require extra care. For example, smoothing of the posterior distribution, as done by Seldin and Tishby [1486], or introduction of assumptions on the data generating distribution that would guarantee convergence of empirical means to their expectations. Therefore, in case of unbounded loss functions application of PAC-Bayesian bounds to provide generalisation guarantees for Bayesian posteriors may demand extra analysis, assumptions on the data generating distribution, and/or posterior adjustments. Since Bayesian inference is not directly concerned with concentration of empirical estimates of the losses (and loss functions in general), unboundedness of the loss function is not a direct concern for it.

   (b) The second distinction is that the optimal value of λ in PAC-Bayesian bounds is typically smaller than 1, meaning that they weigh prior belief $\pi(\theta)$ and evidence differently from Bayesian learning, and if the loss is not the negative log-likelihood, then the shape of the optimal posterior might be altogether completely different.

5. The Bayes rule requires access to the prior $\pi(\theta)$ to compute the posterior $\rho(\theta|\mathcal{D})$. In PAC-Bayesian bounds π appears only in $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ and, therefore, it is sufficient to have an upper bound on $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ to work with the bounds. For example, Lever et al. [1015, 1016] define $\pi(\theta)$ in terms of the true data generating distribution. Such $\pi(\theta)$ is valid, because it does not depend on the data, but it is not accessible to the learner. Nevertheless, Lever et al. [1015, 1016] show that if both the prior and the posterior are Gibbs distributions, it is possible to bound $\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ using a self-bounding technique and work with the bound without direct access to $\pi(\theta)$.

22.1.2 PAC-Bayes-kl inequality

Now we present the PAC-Bayes-kl inequality due to Seeger [1483] (with a slight improvement by Maurer [1151]). The inequality is at the basis of Theorems 22.1 and 22.2 we have discussed earlier, and will be followed by a proof. In order to proceed, we need to define the binary kl-divergence, which is a shorthand for the standard KL divergence (see Chapter 12.1) between two Bernoulli distributions.

Definition 22.2 (Binary kl-divergence)**.** *Let p and q be biases of two Bernoulli random variables. The* binary D_{kl} divergence *is defined as:*

$$\mathrm{D}_{\mathrm{kl}}(p\|q) = \mathrm{D}_{\mathrm{KL}}([1-p, p]\|[1-q, q]) = p\ln\frac{p}{q} + (1-p)\ln\frac{1-p}{1-q}. \tag{22.7}$$

Pay attention that the parameters of $\mathrm{D}_{\mathrm{KL}}(\cdot\|\cdot)$ are two probability distributions, whereas the parameters of $\mathrm{D}_{\mathrm{kl}}(\cdot\|\cdot)$ are two numbers in the $[0, 1]$ interval. PAC-Bayes-kl inequality presented next involves binary kl divergence between the empirical and the expected loss of a randomised classifier ρ (which are two numbers in $[0, 1]$) and the standard KL divergence between the posterior and the prior (which are two distributions).

Theorem 22.3 (PAC-Bayes-kl inequality)**.** *Assume that the loss function ℓ is bounded in the $[0, 1]$ interval. For any prior distribution π over Θ that is independent of $\mathcal{D}$ and any $\delta \in (0, 1]$:*

$$\mathbb{P}\left(\exists\rho : \mathrm{D}_{\mathrm{kl}}\left(\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta,\mathcal{D})\right]\middle\|\mathbb{E}_\rho[\mathcal{L}(\theta)]\right) \geq \frac{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}}{N}\right) \leq \delta. \tag{22.8}$$

Theorem 22.1 follows from Theorem 22.3 by the refined Pinsker's inequality [1140, 1141, 1455], [196, Lemma 8.4]. We leave the proof of this statement as an exercise.

Lemma 22.1 (Refined Pinsker's inequality)**.** *If $\mathrm{D}_{\mathrm{kl}}(p\|q) \leq \varepsilon$ then*

$$q \leq p + \sqrt{2p\varepsilon} + 2\varepsilon. \tag{22.9}$$

Theorem 22.2 also follows from Theorem 22.3 by another form of the refined Pinsker's inequality.

Lemma 22.2 (Variational form of the refined Pinsker's inequality)**.** *If* $\mathrm{D}_{\mathrm{kl}}(p\|q) \leq \varepsilon$ *then*

$$q \leq \min_{\lambda \in (0,2)} \left(\frac{p}{1-\frac{\lambda}{2}} + \frac{\varepsilon}{\lambda\left(1-\frac{\lambda}{2}\right)} \right). \tag{22.10}$$

The proof of PAC-Bayes-kl inequality is based on change of measure inequality, which acts as a replacement of the union bound in continuous spaces. The change of measure inequality uses Jensen's inequality, which we present first.

Theorem 22.4 (Jensen's inequality)**.** *If* g *is a convex function and* Z *a random variable, then*

$$\mathbb{E}[g(Z)] \geq g(\mathbb{E}[Z]). \tag{22.11}$$

We refer the reader to Cover and Thomas [337] for a proof. Next, we present the change of measure inequality.

Theorem 22.5 (Change of measure inequality)**.** *For any measurable function* $g(\theta)$ *on* Θ *and any distributions* ρ *and* π*:*

$$\mathbb{E}_{\theta\sim\rho}[g(\theta)] \leq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln \mathbb{E}_{\theta\sim\pi}\left[e^{g(\theta)}\right]. \tag{22.12}$$

Proof.

$$\begin{aligned}
\mathbb{E}_{\theta\sim\rho}[g(\theta)] &= \mathbb{E}_{\theta\sim\rho}\left[\ln\left(\frac{\rho(\theta)}{\pi(\theta)} \times e^{g(\theta)} \times \frac{\pi(\theta)}{\rho(\theta)}\right)\right] && (22.13)\\
&= \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \mathbb{E}_{\theta\sim\rho}\left[\ln\left(e^{g(\theta)} \times \frac{\pi(\theta)}{\rho(\theta)}\right)\right] && (22.14)\\
&\leq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln \mathbb{E}_{\theta\sim\rho}\left[e^{g(\theta)} \times \frac{\pi(\theta)}{\rho(\theta)}\right] && (22.15)\\
&= \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln \mathbb{E}_{\theta\sim\pi}\left[e^{g(\theta)}\right], && (22.16)
\end{aligned}$$

where the inequality in the third line is by Jensen's inequality. Note that there is nothing probabilistic in the statement of the theorem – it is a deterministic result. □

The change of measure inequality can also be obtained from the variational formula by Donsker and Varadhan [441], which states that

$$\ln \mathbb{E}_{\theta\sim\pi}\left[e^{g(\theta)}\right] = \sup_{\rho \ll \pi} (\mathbb{E}_{\theta\sim\rho}[g(\theta)] - \mathrm{D}_{\mathrm{KL}}(\rho\|\pi)), \tag{22.17}$$

where $\rho \ll \pi$ is used to denote absolute continuity of ρ with respect to π, and $\ln \mathbb{E}_{\theta\sim\pi}\left[e^{g(\theta)}\right]$ is assumed to be finite. Note that taking $g(\theta) = \ln p(\mathcal{D}|\theta)$ in the change of measure inequality recovers the ELBO (Evidence Lower BOund) discussed in the earlier chapters.

The change of measure inequality is used in the PAC-Bayes lemma, which is the foundation of most PAC-Bayesian bounds. The PAC-Bayes lemma also uses Markov's inequality, so we start with it.

Theorem 22.6 (Markov's inequality)**.** *For any non-negative random variable* Z *and* $\varepsilon > 0$

$$\mathbb{P}(Z \geq \varepsilon) \leq \frac{\mathbb{E}[Z]}{\varepsilon}. \tag{22.18}$$

Equivalently (taking $\delta = \frac{\mathbb{E}[Z]}{\varepsilon}$*), for any* $\delta \in (0, 1]$*:*

$$\mathbb{P}\left(Z \geq \frac{\mathbb{E}[Z]}{\delta}\right) \leq \delta. \tag{22.19}$$

We refer the reader to Seldin [1485] for a proof and move on to the PAC-Bayes lemma.

Lemma 22.3 (PAC-Bayes lemma)**.** *For any measurable function* $g : \Theta \times (\mathcal{X} \times \mathcal{Y})^N \to \mathbb{R}$ *and any distribution* π *on* Θ *that is independent of the sample* $\mathcal{D}$

$$\mathbb{P}\left(\exists \rho : \mathbb{E}_{\theta\sim\rho}[g(\theta,\mathcal{D})] \geq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln \frac{\mathbb{E}_{\theta\sim\pi}\left[\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]\right]}{\delta}\right) \leq \delta, \tag{22.20}$$

where the probability is with respect to the draw of the sample $\mathcal{D}$ *and* $\mathbb{E}_{\mathcal{D}}[\cdot]$ *is the expectation with respect to the draw of* $\mathcal{D}$*.*

Proof.

$$\mathbb{P}\left(\exists \rho : \mathbb{E}_\rho[g(\theta,\mathcal{D})] \geq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln \frac{\mathbb{E}_\pi\left[\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]\right]}{\delta}\right) \tag{22.21}$$

$$\leq \mathbb{P}\left(\mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right] \geq \frac{\mathbb{E}_\pi\left[\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]\right]}{\delta}\right) \tag{22.22}$$

$$= \mathbb{P}\left(\mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right] \geq \frac{\mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right]\right]}{\delta}\right) \tag{22.23}$$

$$\leq \delta, \tag{22.24}$$

where equation (22.22) follows by the change of measure inequality (for a fixed π, we have $\mathbb{E}_\rho[g(h,S)] - \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) \leq \ln \mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right]$ for all ρ); in equation (22.23) we can exchange the order of expectations, because π is independent of $\mathcal{D}$; and in the last step we apply Markov's inequality to the random variable $Z = \mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right]$.

An important point is that the change-of-measure inequality bounds $\mathbb{E}_\rho[g(h,S)] - \mathrm{D}_{\mathrm{KL}}(\rho\|\pi)$ for all posterior distributions ρ by a single quantity $\ln \mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right]$ in a deterministic way, and the probabilistic argument is applied to a single random variable $\mathbb{E}_\pi\left[e^{g(\theta,\mathcal{D})}\right]$, avoiding the need to take a union bound. In this way, the change of measure inequality acts as a replacement of the union bound. □

Various PAC-Bayesian inequalities are obtained from the PAC-Bayes lemma by making suitable choices of the function $g(\theta,\mathcal{D})$. A key consideration in the choice of $g(\theta,\mathcal{D})$ is the possibility to bound $\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]$. For example, Seldin et al. [1487] have used Hoeffding's lemma and Bernstein's lemma to derive PAC-Bayes-Hoeffding and PAC-Bayes-Bernstein inequalities, respectively. In PAC-Bayes-kl the choice is $g(\theta,\mathcal{D}) = N\,\mathrm{D}_{\mathrm{kl}}\Big(\hat{\mathcal{L}}(\theta,\mathcal{D})\Big\|\mathcal{L}(\theta)\Big)$. The following lemma is used to control the expectation of the exponent of g.

Lemma 22.4 (kl Lemma)**.** *Let $Z_1,\ldots,Z_N$ be independent identically distributed random variables with $Z_1 \in [0,1]$, $\mathbb{E}[Z_1] = p$, and $\hat{p} = \frac{1}{N}\sum_{i=1}^N Z_i$. Then*

$$\mathbb{E}\left[e^{N\,\mathrm{D}_{\mathrm{kl}}(\hat{p}\|p)}\right] \leq 2\sqrt{N}. \tag{22.25}$$

We refer the reader to Maurer [1151], Seldin [1485] for a proof and to Seldin [1485] for examples of other choices of g.

Taking $Z_i = \ell(f(x_i,\theta),y_i)$, assuming that the loss function ℓ is bounded in $[0,1]$, and applying Lemma 22.4, to the above choice of g, we obtain $\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right] \leq 2\sqrt{N}$, which is the last bit that we need to present a proof of Theorem 22.3.

Proof of Theorem 22.3. We take $g(\theta,\mathcal{D}) = N\,\mathrm{D}_{\mathrm{kl}}\Big(\hat{\mathcal{L}}(\theta,\mathcal{D})\Big\|\mathcal{L}(\theta)\Big)$. Then:

$$\mathbb{P}\left(\exists \rho : \mathrm{D}_{\mathrm{kl}}\Big(\mathbb{E}_\rho\Big[\hat{\mathcal{L}}(\theta,\mathcal{D})\Big]\Big\|\mathbb{E}_\rho[\mathcal{L}(\theta)]\Big) \geq \frac{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}}{N}\right)$$

$$\leq \mathbb{P}\left(\exists \rho : \mathbb{E}_\rho\Big[\mathrm{D}_{\mathrm{kl}}\Big(\hat{\mathcal{L}}(\theta,\mathcal{D})\Big\|\mathcal{L}(\theta)\Big)\Big] \geq \frac{\mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}}{N}\right) \tag{22.26}$$

$$= \mathbb{P}\left(\exists \rho : \mathbb{E}_\rho\Big[N\,\mathrm{D}_{\mathrm{kl}}\Big(\hat{\mathcal{L}}(\theta,\mathcal{D})\Big\|\mathcal{L}(\theta)\Big)\Big] \geq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{2\sqrt{N}}{\delta}\right) \tag{22.27}$$

$$\leq \mathbb{P}\left(\exists \rho : \mathbb{E}_\rho[g(\theta,\mathcal{D})] \geq \mathrm{D}_{\mathrm{KL}}(\rho\|\pi) + \ln\frac{\mathbb{E}_\pi\left[\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]\right]}{\delta}\right) \tag{22.28}$$

$$\leq \delta. \tag{22.29}$$

The first line of the derivation exploits convexity of D_{kl} and Jensen's inequality, which allows us to take the expectation outside of D_{kl} on the left side of the inequality inside the probability clause. The second line multiplies both sides of the inequality by N. The third line applies the kl lemma (22.4), by which $\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right] \leq 2\sqrt{N}$, and so $\mathbb{E}_\pi\left[\mathbb{E}_{\mathcal{D}}\left[e^{g(\theta,\mathcal{D})}\right]\right] \leq 2\sqrt{N}$. The last line is by the PAC-Bayes lemma (22.3).

We note that the requirement of boundedness of ℓ in the $[0,1]$ interval is required in Lemma 22.4 and the requirement on independence of π and $\mathcal{D}$ is required in Lemma 22.3. □

22.2 Ensemble classifiers and weighted majority vote

An alternative to randomised classification from Definition 22.1 is to take a ρ-weighted majority vote of the predictions given by parameters $\theta \in \Theta$. This approach has two advantages. One is that it transforms a randomised prediction into a deterministic one. (Imagine hearing that a doctor has sampled parameters of a model to predict your diagnosis. While it may be a statistically sound approach, psychologically it may be unsettling to some people.) The second, more important one, is the *cancellation of errors effect*. Informally, if in a set of classification models the individual models err with probability smaller than $\frac{1}{2}$ and their errors are independent, the errors tend to cancel out. For example, consider binary classification and imagine that we have a finite set $\Theta = (\theta_1, \ldots, \theta_M)$ of size M, and that for every $\theta_i \in \Theta$ the probability of making an error on a random sample (x, y) is $\frac{1}{2} - \varepsilon$ for a small $\varepsilon > 0$, which implies that $\mathcal{L}(\theta) = \frac{1}{2} - \varepsilon$. Also, assume that for $i \neq j$ the predictions $f(x, \theta_i)$ and $f(x, \theta_j)$ are independent. Then a uniformly weighted majority vote of $f(x, \theta_1), \ldots, f(x, \theta_M)$ will err only if at least half of the predictions are incorrect. By Hoeffding's inequality [761] the probability of this happening is $\mathbb{P}\Big(\sum_{i=1}^{M} \mathbb{1}(f(x, \theta_i) \neq y) \geq \frac{1}{2}M\Big) \leq e^{-2M\varepsilon^2}$, meaning that even for a small M the error rate of a uniformly weighted majority vote will be much smaller than the $\frac{1}{2} - \varepsilon$ error rate of the individual classifiers.

In practice, the true error rates are unknown and are not guaranteed to be below $\frac{1}{2}$, and the errors are not necessarily independent. If the error rates of individual models are above $\frac{1}{2}$, taking a majority vote may deteriorate the prediction accuracy. It is possible to use PAC-Bayesian analysis to provide generalisation bounds for weighted majority votes, as well as to optimise the weighting of classifiers ρ, as we demonstrate next.

From this point on we restrict ourselves to multiclass classification, i.e., $\mathcal{Y}$ is finite. We also assume that $f(x, \theta) \in \mathcal{Y}$. Under these assumptions, prediction of ρ-weighted majority vote $\mathrm{MV}_\rho(x)$ is defined as

$$\mathrm{MV}_\rho(x) = \arg\max_{y \in \mathcal{Y}} \mathbb{E}_{\theta \sim \rho}[\mathbb{1}(f(x, \theta) = y)], \tag{22.30}$$

where ties are resolved arbitrarily. In words, $\mathrm{MV}_\rho(x)$ predicts the label that received the most ρ-weighted votes from the models $\theta \in \Theta$. In binary classification with $\mathcal{Y} = \{\pm 1\}$ and $f(x, \theta) \in \{\pm 1\}$, equation (22.30) can be simplified to

$$\mathrm{MV}_\rho(x) = \mathrm{sign}(\mathbb{E}_\rho[f(x, \theta)]). \tag{22.31}$$

22.3 Oracle bounds for the weighted majority vote

A key observation is that if a ρ-weighted majority vote made an error, we know that at least a ρ-weighted half of the predictors θ have erred. For the zero-one loss it implies

$$\ell(\mathrm{MV}_\rho(x), y) \leq \mathbb{1}(\mathbb{E}_\rho[\mathbb{1}(f(x, \theta) \neq y)] \geq 0.5). \tag{22.32}$$

The observation leads to the following basic bound on the expected loss of the weighted majority vote.

Lemma 22.5 (A basic bound on the expected loss of a weighted majority vote)**.**

$$\mathcal{L}(\mathrm{MV}_\rho) \leq \mathbb{P}(\mathbb{E}_\rho[\mathbb{1}(f(x, \theta) \neq y)] \geq 0.5). \tag{22.33}$$

Proof. We have

$$\begin{aligned} \mathcal{L}(\mathrm{MV}_\rho) &= \mathbb{E}_{(x,y)\sim p(x,y)}[\ell(\mathrm{MV}_\rho(x), y)] && (22.34)\\ &\leq \mathbb{E}_{(x,y)\sim p(x,y)}[\mathbb{1}(\mathbb{E}_\rho[\mathbb{1}(f(x, \theta) \neq y)] \geq 0.5)] && (22.35)\\ &= \mathbb{P}(\mathbb{E}_\rho[\mathbb{1}(f(x, \theta) \neq y)] \geq 0.5), && (22.36) \end{aligned}$$

where the first line is by the definition of $\mathcal{L}(\cdot)$, the second line is by equation (22.32), and the last holds since $\mathbb{1}(\cdot)$ is a Bernoulli random variable. □

The basic bound leads to a variety of oracle bounds, which are all based on taking a random variable $Z = \mathbb{E}_\rho[\mathbb{1}(f(x, \theta) \neq y)]$ and bounding the probability $\mathbb{P}(Z \geq 0.5)$ on the right hand side of equation (22.33). At large, there is a "first order oracle bound", which is based on the first moment of Z, and several variations of "second order oracle bounds", which are based on the second moment of Z. It is possible, but uncommon to use higher moments of Z due to the statistical and computational difficulty of empirical estimation of higher moments.

First order oracle bound

The first bound is so basic that it was called "a folk theorem" [973].

Theorem 22.7 (First order oracle bound)**.**

$$\mathcal{L}(\mathrm{MV}_\rho) \leq 2\mathbb{E}_\rho[\mathcal{L}(\theta)]. \tag{22.37}$$

Proof. By Lemma 22.5 and Markov's inequality (22.6)

$$\begin{aligned}
\mathcal{L}(\mathrm{MV}_\rho) &\leq \mathbb{P}(\mathbb{E}_\rho[\mathbb{1}(f(x,\theta) \neq y)] \geq 0.5) && (22.38)\\
&\leq 2\mathbb{E}_{(x,y)\sim p(x,y)}[\mathbb{E}_\rho[\mathbb{1}(f(x,\theta) \neq y)]] && (22.39)\\
&= 2\mathbb{E}_\rho\big[\mathbb{E}_{(x,y)\sim p(x,y)}[\mathbb{1}(f(x,\theta) \neq y)]\big] && (22.40)\\
&= 2\mathbb{E}_\rho[\mathcal{L}(\theta)]. && (22.41)
\end{aligned}$$

□

The bound is called an *oracle* bound, because it relates the expected loss of the majority vote to the expected losses of individual classifiers, $\mathcal{L}(\theta)$, which are unknown quantities. In the following section, we use PAC-Bayes to convert oracle bounds into empirical bounds. However, a shortcoming of the first order bound is evident already at the oracle level. The first order oracle bound takes no interactions between predictions of different classifiers into account, whereas cancellation of errors is the major source of power of the majority vote. The minimum of the first order oracle bound is achieved by placing all the mass $\rho(\theta)$ on the best performing classifier $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$, which "kills" the majority vote. Even though in practice the first order empirical bound presented in the next section is often tighter than higher order bounds, it has been shown that optimisation of the weighting ρ through minimisation of the first order empirical bound typically degrades the expected loss $\mathcal{L}(\mathrm{MV}_\rho)$ of the majority vote [1142].

Second order oracle bound

A line of works has sought to overcome the shortcoming of the first order oracle bound by applying the Chebyshev-Cantelli inequality instead of Markov's inequality to bound $\mathbb{P}(Z \geq 0.5)$ [609, 954, 982, 1428], but they have suffered from statistical and computational inefficiency due to oracle quantities appearing in the denominator of the Chebyshev-Cantelli inequality. A breakthrough was achieved by Masegosa et al. [1142], who used a second order Markov's inequality.

Theorem 22.8 (Second order Markov's inequality)**.** *For a random variable Z and $\varepsilon > 0$*

$$\mathbb{P}(Z \geq \varepsilon) \leq \frac{\mathbb{E}\big[Z^2\big]}{\varepsilon^2}. \tag{22.42}$$

Proof.

$$\mathbb{P}(Z \geq \varepsilon) \leq \mathbb{P}\big(Z^2 \geq \varepsilon^2\big) \leq \frac{\mathbb{E}\big[Z^2\big]}{\varepsilon^2}, \tag{22.43}$$

where the last step is by Markov's inequality. □

The second order oracle bound is defined in terms of tandem loss, which we define next.

Definition 22.3 (Tandem loss)**.** *The* tandem loss *of θ and θ' on (x, y) is defined by*

$$\ell(f(x,\theta), f(x,\theta'), y) = \mathbb{1}(f(x,\theta) \neq y)\mathbb{1}(f(x,\theta') \neq y). \tag{22.44}$$

The tandem loss counts an error if both θ and θ' erred on (x, y). The expected tandem loss *is defined by*

$$\mathcal{L}(\theta,\theta') = \mathbb{E}_{(x,y)\sim p(x,y)}[\ell(f(x,\theta), f(x,\theta'), y)]. \tag{22.45}$$

And the empirical tandem loss *is defined by*

$$\hat{\mathcal{L}}(\theta,\theta',\mathcal{D}) = \frac{1}{N}\sum_{n=1}^{N} \ell(f(x_n,\theta), f(x_n,\theta'), y_n). \tag{22.46}$$

The tandem loss has the following property.

Lemma 22.6.

$$\mathbb{E}_{(x,y)\sim p(x,y)}\left[\left(\mathbb{E}_{\theta\sim\rho}[\mathbb{1}(f(x,\theta)\neq y)]\right)^2\right] = \mathbb{E}_{(\theta,\theta')\sim\rho^2}[\mathcal{L}(\theta,\theta')], \tag{22.47}$$

where ρ^2 is a product distribution on $\Theta\times\Theta$.

The property relates the tandem loss to $\mathbb{E}_\rho[\mathbb{1}(f(x,\theta)\neq y)]$, which appears in the basic bound on the expected loss of weighted majority vote in Lemma 22.5 and will be used shortly. But before using it we provide a proof of the lemma.

Proof.

$$\begin{aligned}
\mathbb{E}_{p(x,y)}\left[\mathbb{E}_\rho[\mathbb{1}(f(x,\theta)\neq y)]^2\right] &= \mathbb{E}_{p(x,y)}[\mathbb{E}_\rho[\mathbb{1}(f(x,\theta)\neq y)]\mathbb{E}_\rho[\mathbb{1}(f(x,\theta)\neq y)]] && (22.48)\\
&= \mathbb{E}_{p(x,y)}\left[\mathbb{E}_{\rho^2}[\mathbb{1}(f(x,\theta)\neq y)\mathbb{1}(f(x,\theta')\neq y)]\right] && (22.49)\\
&= \mathbb{E}_{\rho^2}\left[\mathbb{E}_{p(x,y)}[\mathbb{1}(f(x,\theta)\neq y)\mathbb{1}(f(x,\theta')\neq y)]\right] && (22.50)\\
&= \mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')]. && (22.51)
\end{aligned}$$

□

Combining Lemma 22.5, Theorem 22.8, and Lemma 22.6 leads to the following result.

Theorem 22.9 (Second order oracle bound)**.**

$$\mathcal{L}(\mathrm{MV}_\rho) \leq 4\mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')] \tag{22.52}$$

Proof. Take a random variable $Z = \mathbb{E}_\rho[\mathbb{1}(f(x,\theta)\neq y)]$ in Lemma 22.5, plug it into Theorem 22.8, and apply Lemma 22.6 to $\mathbb{E}\left[Z^2\right]$. □

A major advantage of the second order oracle bound over the first order oracle bound is that it takes correlations of errors into account through the tandem loss. Therefore, the weighting distribution ρ obtained by minimisation of the corresponding second order empirical bound typically does not deteriorate the expected loss of the resulting majority vote $\mathcal{L}(\mathrm{MV}_\rho)$ [1142]. A disadvantage is that in many cases the factor 4 in the bound is typically loose (typically $\mathrm{MV}_\rho(x)$ performs better than the corresponding tandem classifier rather than within factor 4 worse) and as a bound (rather than as an optimisation objective) it is typically weaker than the first order bound.

Wu et al. [1780] proposed further refinement of the bound by replacing second order Markov's inequality with variational Chebyshev-Cantelli inequality.

Theorem 22.10 (Variational Chebyshev-Cantelli oracle bound)**.** *For all ρ:*

$$\mathcal{L}(\mathrm{MV}_\rho) \leq \min_{\mu<0.5}\left(\frac{\mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')] - 2\mu\mathbb{E}_\rho[\mathcal{L}(\theta)] + \mu^2}{(0.5-\mu)^2}\right). \tag{22.53}$$

We refer the reader to their paper for further details and an additional alternative form of the bound based on tandem losses with an offset. Additional improvements were proposed by Wu and Seldin [1779] and Wu et al. [1781].

For binary classification Germain et al. [609] and Masegosa et al. [1142] provide an alternative way of writing $\mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')]$, as a difference between $\mathbb{E}_\rho[\mathcal{L}(\theta)]$ and average disagreements between predictions. Since in binary classification disagreements can be estimated using unlabelled data only, this approach can benefit from large amounts of unlabelled data. However, for a noticeable advantage, the amount of unlabelled data should be more than square the amount of labelled data. We refer to Masegosa et al. [1142] for more details.

Comparison of the oracle bounds

We provide a brief comparison of the first and second order oracle bounds.

A positively correlated errors case. Since $\mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')] \leq \mathbb{E}_\rho[\mathcal{L}(\theta)]$, the second order oracle bound is at most twice worse than the first order oracle bound. The worst case occurs, for example, if all parameters θ in Θ give identical predictions. Then $\mathbb{E}_{\rho^2}[\mathcal{L}(\theta,\theta')] = \mathbb{E}_\rho[\mathcal{L}(\theta)] = \mathcal{L}(\mathrm{MV}_\rho)$ for all ρ and the first order bound has a factor 2 slack, whereas the second order bound has a factor 4 slack.

A negatively correlated errors case. Imagine that $|\Theta| = M$ is finite of size $M \geq 3$, such that each set of parameters $\theta \in \Theta$ errs on $1/M$ of the sample space (according to the distribution $p(x, y)$) and that the error regions are disjoint. Then $\mathcal{L}(\theta) = \mathcal{L}(\theta, \theta) = 1/M$ for all θ and $\mathcal{L}(\theta, \theta') = 0$ for all $\theta \neq \theta'$. For a uniform distribution ρ on Θ the first order bound is $2\mathbb{E}_\rho[\mathcal{L}(\theta)] = 2/M$ and the second order bound is $4\mathbb{E}_{\rho^2}[\mathcal{L}(\theta, \theta')] = 4/M^2$ and $\mathcal{L}(\mathrm{MV}_\rho) = 0$. In this case the second order bound is an order of magnitude tighter than the first order.

An uncorrelated errors case. Assume that all parameter sets $\theta \in \Theta$ produce independent errors and have the same error rate, $\mathcal{L}(\theta) = \mathcal{L}(\theta')$ for all θ and θ'. Then for $\theta \neq \theta'$ we have $\mathcal{L}(\theta, \theta') = \mathbb{E}_{p(x,y)}[\mathbb{1}(f(x, \theta) \neq y)\mathbb{1}(f(x, \theta') \neq y)] = \mathbb{E}_{p(x,y)}[\mathbb{1}(f(x, \theta) \neq y)]\mathbb{E}_{p(x,y)}[\mathbb{1}(f(x, \theta') \neq y)] = \mathcal{L}(\theta)^2$ and $\mathcal{L}(\theta, \theta) = \mathcal{L}(\theta)$. For a uniform distribution ρ the second order bound is $4\mathbb{E}_{\rho^2}[\mathcal{L}(\theta, \theta')] = 4\big(\mathcal{L}(\theta)^2 + \frac{1}{M}\mathcal{L}(\theta)(1 - \mathcal{L}(\theta))\big)$ and the first order bound is $2\mathbb{E}_\rho[\mathcal{L}(\theta)] = 2\mathcal{L}(\theta)$. Assuming that M is large, so that we can ignore the second term in the second order bound, we obtain that it is tighter for $\mathbb{E}_\rho[\mathcal{L}(\theta)] < 1/2$ and looser otherwise. The former is the interesting regime.

22.4 Empirical bounds for the weighted majority vote

Empirical bounds for the weighted majority vote are obtained from the oracle bounds in a straightforward manner by applying any applicable PAC-Bayes bound to $\mathbb{E}_\rho[\mathcal{L}(\theta)]$ or $\mathbb{E}_{\rho^2}[\mathcal{L}(\theta, \theta')]$, depending on the order. Below we provide an example based on the optimisation-friendly form of the PAC-Bayesian bound in Theorem 22.2.

Theorem 22.11 (First order empirical bound)**.** *Assume that ℓ is the zero-one loss function. For any prior distribution π over Θ that is independent of $\mathcal{D}$ and any $\delta \in (0, 1]$*

$$\mathbb{P}\left(\forall \rho : \mathcal{L}(\mathrm{MV}_\rho) \leq 2 \min_{\lambda \in (0,2)} \left(\frac{\mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta, \mathcal{D})\right]}{1 - \frac{\lambda}{2}} + \frac{\mathrm{D}_{\mathrm{KL}}(\rho \| \pi) + \ln \frac{2\sqrt{N}}{\delta}}{\lambda\big(1 - \frac{\lambda}{2}\big)N} \right)\right) \geq 1 - \delta. \tag{22.54}$$

Theorem 22.12 (Second order empirical bound)**.** *Assume that ℓ is the zero-one loss function. For any prior distribution π over Θ that is independent of $\mathcal{D}$ and any $\delta \in (0, 1]$*

$$\mathbb{P}\left(\forall \rho : \mathcal{L}(\mathrm{MV}_\rho) \leq 4 \min_{\lambda \in (0,2)} \left(\frac{\mathbb{E}_{\rho^2}\left[\hat{\mathcal{L}}(\theta, \theta', \mathcal{D})\right]}{1 - \frac{\lambda}{2}} + \frac{2\,\mathrm{D}_{\mathrm{KL}}(\rho \| \pi) + \ln \frac{2\sqrt{N}}{\delta}}{\lambda\big(1 - \frac{\lambda}{2}\big)N} \right)\right) \geq 1 - \delta. \tag{22.55}$$

Theorem 22.11 is obtained from Theorem 22.7 by applying Theorem 22.2 to $\mathbb{E}_\rho[\mathcal{L}(\theta)]$. Theorem 22.12 is obtained from Theorem 22.9 by applying Theorem 22.2 to $\mathbb{E}_{\rho^2}[\mathcal{L}(\theta, \theta')]$ and using the fact that $\mathrm{D}_{\mathrm{KL}}\big(\rho^2 \big\| \pi^2\big) = 2\,\mathrm{D}_{\mathrm{KL}}(\rho \| \pi)$ (see Germain et al. [609, Page 814]).

We note that it is also possible to use Theorem 22.3 instead of Theorem 22.2 to obtain tighter bounds, but, as already mentioned, D_{kl} is less convenient for optimisation. A common practice is to use Theorem 22.11 or 22.12 to optimize ρ, and then to plug the optimised ρ into a D_{kl}-based bound to obtain a tighter generalisation certificate (which is a valid procedure, because Theorem 22.3 holds for all ρ, including ρ obtained by optimisation of any other bound).

Comparison of the empirical bounds

On top of the differences of the underlying oracle bounds discussed earlier, empirical bounds add some practical considerations. In general, $\mathbb{E}_{\rho^2}\left[\hat{\mathcal{L}}(\theta, \theta', \mathcal{D})\right] \leq \mathbb{E}_\rho\left[\hat{\mathcal{L}}(\theta, \mathcal{D})\right]$, thus the second order bound benefits from lower empirical risk and faster convergence rates (refer to the discussion of "fast convergence rates" in Theorem 22.1, where we mentioned that low empirical risk reduces the impact of the square root term). However, the second order bound has $2\,\mathrm{D}_{\mathrm{KL}}(\rho \| \pi)$ in the complexity term compared to $\mathrm{D}_{\mathrm{KL}}(\rho \| \pi)$ in the first order bound. Additionally, in cross-validation and bagging-style applications it is data-efficient to use out-of-bag samples for the computation of the bounds. But since the tandem losses must be estimated on overlaps of the corresponding out-of-bag samples, there are less data for their estimation compared to out-of-bag samples available for estimation of plain losses. Therefore, the effective sample size $\mathcal{D}$ in Theorem 22.12 is smaller than in Theorem 22.11. We discuss this way of ensemble and bound construction in more detail in the next section. Masegosa et al. [1142] show that empirically as an objective for optimising ρ the second order bound most often produces majority votes with lower expected risk $\mathcal{L}(\mathrm{MV}_\rho)$ than the first order bound. However, the first order bound often produces tighter generalisation guarantees. A practical possibility is to use the second order bound to find a good ρ and then plug it into the first order bound to get a generalisation guarantee, because the first order bound holds for all posteriors, including those produced by other methods.

22.5 PAC-Bayesian deep ensembles

There are numerous ways to construct prediction ensembles and various options for the choice of ensemble members, such as decision stumps, decision trees, or even ensembles of heterogeneous classifiers. In this book, we focus on ensembles of deep neural networks, see Chapter 19 for an introduction.

Combining deep neural networks into an ensemble model is a common and old approach to improving generalisation performance [161, 649, 705]. The most basic way to create an ensemble of M neural networks is to repeat the training process M times using all training data and to combine the resulting networks uniformly [705, 962]. This procedure can be viewed as a simple ensembling, where random weight initialisations and stochastic training (randomly composed mini-batches, randomised augmentations) are assumed to generate sufficiently diverse ensemble members. Although this approach is often explicitly referred to as non-Bayesian (see, e.g., Lakshminarayanan et al. [962], Ovadia et al. [1274], Wenzel et al. [1742]), the resulting ensemble may also be interpreted as a Bayesian model average from an approximate posterior [453, 692, 1765], see Chapter 19.

Uniform weighting is an easy, but not necessarily the best way to combine predictions. Optimisation of the weights has the potential to yield higher accuracy, as already pointed out in early works on ensembles [936]. The weights can be optimised using prior information and empirical performance of ensemble members.

The idea of using PAC-Bayesian bounds as optimisation objectives for optimising the weighting ρ has been floating around for a long time, however, as discussed earlier and demonstrated empirically in Masegosa et al. [1142], the first order bounds are unsuitable for this purpose, because they completely ignore (un)correlation of predictions, which is the main source of power for the majority vote. Therefore, meaningful optimisation of ρ is only possible based on second or higher-order bounds.

22.5.1 Construction of an ensemble

While the bounds presented above apply equally to finite and infinite sets Θ, computationally it is easier to work with finite sets. Infinite sets require approximations or highly restricted parametric models allowing closed-form calculation of the relevant integrals. We opt for a finite set $\Theta = (\theta_1, \dots, \theta_M)$ of prediction models, and then optimise weighting of the models in an ensemble by minimising a PAC-Bayes bound. Thiemann et al. [1599] have noted as long as the empirical risk estimates are unbiased, $\mathbb{E}\Big[\hat{\mathcal{L}}(\theta, \mathcal{D})\Big] = \mathcal{L}(\theta)$, it is possible to use different (sub)sets of the data to compute the empirical risk of different models θ_i in PAC-Bayesian bounds. For example, in cross-validation or bagging [205] a data point $(x, y) \in \mathcal{D}$ can be used in training some models in Θ and computing the empirical risk of the models that have not used it in training. (Seldin [1485] provides several exercises playing around with this approach.) To formalise this idea we let $\mathcal{D}_i$ denote the set of data that were *not* used for training model θ_i. Then $\mathbb{E}\Big[\hat{\mathcal{L}}(\theta_i, \mathcal{D}_i)\Big] = \mathcal{L}(\theta_i)$. We further use $\mathcal{D}_{i,j} = \mathcal{D}_i \cap \mathcal{D}_j$ to denote the set of data that were *not* used for training of neither θ_i nor θ_j. Then $\mathbb{E}\Big[\hat{\mathcal{L}}(\theta_i, \theta_j, \mathcal{D}_{i,j})\Big] = \mathcal{L}(\theta_i, \theta_j)$. We let $K = \min_{i,j} |\mathcal{D}_i \cap \mathcal{D}_j|$ denote the minimal overlap size and $\hat{\mathcal{L}} \in \mathbb{R}^{M \times M}$ denote the matrix of empirical tandem losses with entries $\hat{\mathcal{L}}_{i,j} = \hat{\mathcal{L}}(\theta_i, \theta_j, \mathcal{D}_{i,j})$. Theorem 22.12 undergoes a slight adjustment to accommodate the use of individualised sets of data for computation of empirical risks.

Theorem 22.13 (Second order empirical bound with individualised empirical risk estimators)**.** *Assume that ℓ is the zero-one loss function. Let $\pi = (\pi_1, \dots, \pi_M)^\top$ and $\rho = (\rho_1, \dots, \rho_M)^\top$ be prior and posterior distributions on $\{1, \dots, M\}$, where π is independent of $\{\mathcal{D}_1, \dots, \mathcal{D}_M\}$. Assume that θ_i is independent of $\mathcal{D}_i$. Then for any $\delta \in (0, 1]$*

$$\mathbb{P}\left(\forall \rho : \mathcal{L}(\mathrm{MV}_\rho) \leq 4 \min_{\lambda \in (0,2)} \left(\frac{\rho^\top \hat{\mathcal{L}} \rho}{1 - \frac{\lambda}{2}} + \frac{2\,\mathrm{D}_{\mathrm{KL}}(\rho \| \pi) + \ln \frac{2\sqrt{K}}{\delta}}{\lambda\big(1 - \frac{\lambda}{2}\big) K} \right) \right) \geq 1 - \delta, \tag{22.56}$$

where K and $\hat{\mathcal{L}}$ are defined above.

In other words, with probability at least $1 - \delta$, for all ρ and all $\lambda \in (0, 2)$

$$\mathcal{L}(\mathrm{MV}_\rho) \leq 4 \left(\frac{\rho^\top \hat{\mathcal{L}} \rho}{1 - \frac{\lambda}{2}} + \frac{2\,\mathrm{D}_{\mathrm{KL}}(\rho \| \pi) + \ln \frac{2\sqrt{K}}{\delta}}{\lambda\big(1 - \frac{\lambda}{2}\big) K} \right). \tag{22.57}$$

22.5.2 Optimising ensemble weighting through minimisation of a second-order PAC-Bayesian bound

Similar to Theorem 22.2, the bound in Theorem 22.13 (the right hand side of equation (22.57)) is convex in λ for a fixed ρ. If the $\mathcal{D}_{i,j}$ are identical for all (i,j) pairs, then the bound is also convex in ρ for a fixed λ, however, if the $\mathcal{D}_{i,j}$ differ, convexity in ρ is not guaranteed (see Masegosa et al. [1142]). We optimise weighting ρ for the majority vote by alternating minimisation of the right hand side of the bound in equation (22.57) with respect to λ and with respect to ρ. For λ we use the exact minimisation step given earlier in equation (22.6). Namely, for a fixed ρ we take

$$\lambda_\rho = \frac{2}{\sqrt{\frac{2K\rho^\top \hat{\mathcal{L}}\rho}{2\,\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{K}}{\delta}}+1}+1} \ . \tag{22.58}$$

For a fixed λ, ρ is updated by taking a gradient descent step, where the gradient ∇ of the right hand side of equation (22.57) is given by

$$\nabla = \frac{4}{1-\frac{\lambda}{2}}\left(2\hat{\mathcal{L}}\rho + \frac{2}{\lambda K}\left(1+\ln\frac{\rho}{\pi}\right)\right) \ , \tag{22.59}$$

where $\ln\frac{\rho}{\pi} = \left(\ln\frac{\rho_1}{\pi_1},\ldots,\ln\frac{\rho_M}{\pi_M}\right)^\top$.

In practice, we represent ρ by unconstrained parameters ρ' using a differentiable mapping from $\mathbb{R}^M$ to the probability simplex Δ_M and consider the derivative of the objective w.r.t. ρ'. This allows us to apply unconstrained optimisation. For example, in the experiments below, we use the softmax function and an Rprop variant for gradient-based unconstrained optimisation [810].

Algorithm 25 summarises the minimisation procedure. Since the optimisation problem is low-dimensional ($\rho \in \mathbb{R}^M$), the algorithm quickly converges to a local minimum. Theorem 22.13 provides a generalisation guarantee for the ensemble produced by Algorithm 25.

Algorithm 25: Alternating minimisation of the bound in Theorem 22.13

Input: Matrix of empirical tandem losses $\hat{\mathcal{L}}$, prior π, size K of the smallest set $\mathcal{D}_{i,j}$, stopping criterion parameter T_ρ
Output: optimised ρ and λ

$\rho \leftarrow \frac{1}{M}\mathbf{1}_M$ `// 1_M is the M-dimensional vector of all ones`
`// Update λ using current ρ`
$\lambda \leftarrow 2\left(\sqrt{\frac{2K\rho^\top \hat{\mathcal{L}}\rho}{2\,\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{K}}{\delta}}+1}+1\right)^{-1}$
while *the objective* $\frac{\rho^\top \hat{\mathcal{L}}\rho}{1-\frac{\lambda}{2}} + \frac{2\,\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{K}}{\delta}}{\lambda\left(1-\frac{\lambda}{2}\right)K}$ *decreases* **do**
  `// Update ρ using current λ`
  repeat
    $\nabla = \frac{4}{1-\frac{\lambda}{2}}\left(2\hat{\mathcal{L}}\rho + \frac{2}{\lambda K}\left(1+\ln\frac{\rho}{\pi}\right)\right)$
    Update ρ by an iteration of gradient descent using ∇
  until *the objective* $\frac{\rho^\top \hat{\mathcal{L}}\rho}{1-\frac{\lambda}{2}} + \frac{2\,\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{K}}{\delta}}{\lambda\left(1-\frac{\lambda}{2}\right)K}$ *does not improve for* T_ρ *steps*
  `// Update λ using current ρ`
  $\lambda \leftarrow 2\left(\sqrt{\frac{2K\rho^\top \hat{\mathcal{L}}\rho}{2\,\mathrm{D}_{\mathrm{KL}}(\rho\|\pi)+\ln\frac{2\sqrt{K}}{\delta}}+1}+1\right)^{-1}$

22.5.3 Empirical example

To illustrate optimisation of ensemble weighting via minimisation of the bound in Theorem 22.13, the approach is applied to stacking CNN LSTMs (models combining Convolutional Neural Networks and Long Short-Term Memory) [1816] to solve the IMDb binary classification benchmark [1100].

The IMDb data are commonly used to evaluate deep ensembles. We adopted an experimental setup similar to Wenzel et al. [1742]. The standard training protocol for the IMDb dataset includes the use of a validation

dataset to mitigate overfitting. Neural networks are trained on the training dataset, and the model parameters that yield the lowest error on the validation dataset are selected as the final model. We used 20,000 training samples and 5,000 random validation samples. The test set included all 25,000 samples from the original test set.

We compared three ensembling strategies. For that, we trained N_{trials} networks for N_{epochs} epochs and computed the validation error after each epoch using the model parameters at that point of the training process, also referred to as a model checkpoint. In the first method, referred to as $f^{\text{val}}_{\text{uniform}}$, we took the model checkpoint from the epoch with the lowest validation error for each of the N_{trials} training trials, ending up with an ensemble of $M = N_{\text{trials}}$ networks. These were uniformly combined into a deep ensemble. In the second method, $f^{\text{all}}_{\text{uniform}}$, we took all the model checkpoints from all trials, resulting in $M = N_{\text{trials}} \times N_{\text{epochs}}$ models in the ensemble. We combined them uniformly, disregarding the validation set. In the third method, f^{all}_{ρ}, the weighting of the ensemble from the previous method was optimised using Algorithm 25 and the validation set. The results, averaged over five repetitions, are shown in Figure 22.1. The uniform ensemble of the best checkpoints according to the validation error ($f^{\text{val}}_{\text{uniform}}$) performed better than taking all checkpoints from each trial ($f^{\text{all}}_{\text{uniform}}$), as expected when utilising validation data for model selection. However, after optimising the weights of the uniform ensemble using Algorithm 25 (f^{all}_{ρ}), the resulting ensemble classifier showed the lowest test error, even lower than the uniform ensemble of the best checkpoints ($f^{\text{val}}_{\text{uniform}}$).

The example shows that using validation data for optimising a second-order PAC-Bayesian bound does not only provide a performance guarantee, but can also lead to a better performing ensemble.

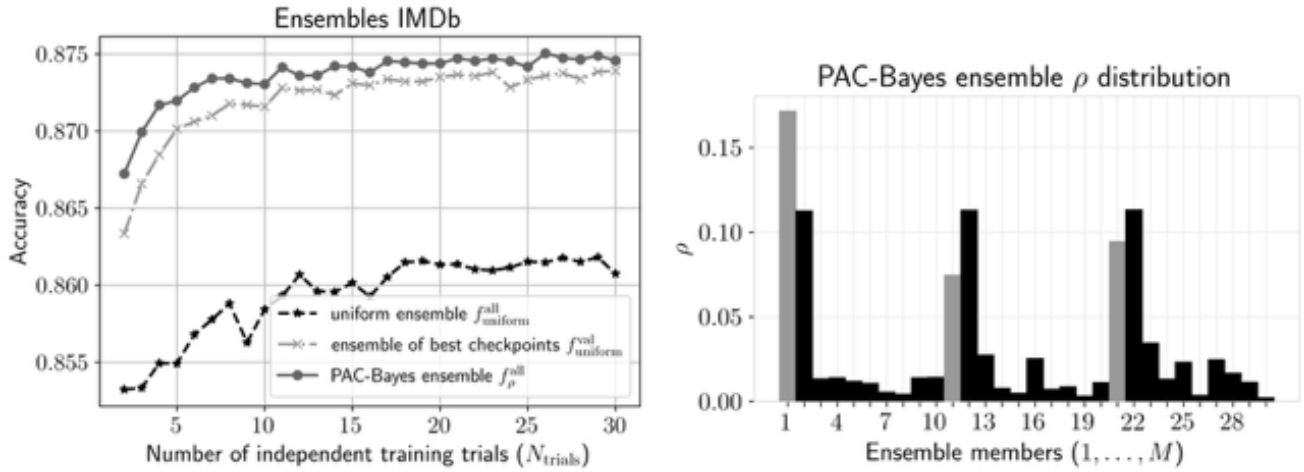


Figure 22.1: Left: Results for three different ensembling strategies for CNN LSTMs [1816] applied to the IMDb binary classification benchmark, averaged over five repetitions. We compared uniformly combining the best network from each trial as determined by the validation data set ($f^{\text{val}}_{\text{uniform}}$), combining uniformly the networks from all epochs and all trials ($f^{\text{all}}_{\text{uniform}}$), and a weighted combination of the networks from all epochs and all trials (f^{all}_{ρ}), where the weights were determined using Algorithm 25. Right: Distribution ρ over $N_{\text{trials}} = 3$ models with $N_{\text{epochs}} = 10$ checkpoints each (i.e., $M = 30$), the i-th snapshot from the j-th trial is assigned ensemble member number $i + N_{\text{epochs}} \cdot (j - 1)$. The figure shows that most weight is placed on the first two checkpoints of each trial, indicating overfitting after the early epochs. The lighter colour highlights the three checkpoints chosen by $f^{\text{val}}_{\text{uniform}}$, which are the initial checkpoints.

22.5.4 Ensemble pruning

The PAC-Bayesian bounds can further be used for ensemble pruning. After optimisation of ρ using a second order bound, pruning can be easily done by zeroing the lowest values of the optimised $\rho(\theta)$ and renormalising ρ. Since PAC-Bayesian bounds hold for all posterior distributions ρ, including those obtained by pruning, they provide a rigorous data-dependent trade-off between sparsity and accuracy of the pruned majority votes.

22.6 Summary

The chapter gave a basic introduction to PAC-Bayesian analysis, which is a frequentist way of incorporating prior information into learning. We contrasted PAC-Bayesian and Bayesian learning frameworks in Section 22.1.1.

One application of PAC-Bayesian analysis that we focused on is analysis and optimisation of weighted majority votes. It includes providing generalisation guarantees for weighted majority votes, optimising the weighting, and pruning.

Part V

Kernel methods

Chapter 23

Introduction to kernel methods

Kernel methods form one of the earliest and most elegant realizations of Bayesian learning in function space. Before deep networks, they provided a way to express flexible, non-parametric priors while preserving exact Bayesian inference, building on early work connecting Bayesian ideas to neural networks [412, 1226, 1612]. Today they play a dual role: (i) as stand-alone probabilistic models such as Gaussian Processes (GPs) and (ii) as analytic limits or building blocks of deep architectures—indeed, in the infinite-width limit, neural networks converge to GPs whose kernel is determined by the network architecture. From the perspective of **Bayesian deep learning**, kernels are the original tools for placing priors over functions rather than parameters.

While Parts I–IV developed inference schemes (sampling, variational, ensemble), Part V explores how the *function priors themselves* can be made expressive, deep, and learnable. The chapter supplies the fundamentals used throughout Part V and motivates the deep extensions that follow.

We begin with kernel functions and how they encode inductive biases (23.2), then introduce the *kernel trick*, the RKHS view, and the Representer theorem (23.3). Building on these foundations, we present Gaussian processes as distributions over functions and derive standard regression formulas (23.4). We then discuss the key limitations of standard kernels and GPs, including representation rigidity and scalability, and motivate deep, adaptive alternatives (23.5). This sets up the next chapters on Deep Gaussian Processes (25), Deep Kernel Learning (24), and Meta-Learning function priors (27).

23.1 Kernel as Feature Map

At the heart of kernel methods lies a powerful geometric intuition: complex patterns in raw data can often be understood more easily after transforming the data into a different representation space. A *kernel function* formalizes this idea by implicitly defining a transformation from an input space to a feature space where data relationships become more tractable.

23.1.1 From Input Space to Feature Space

Consider a dataset of objects $\{x_1, \ldots, x_n\}$ residing in some *input space* $\mathcal{X}$. The input space represents the original, observed form of the data, for example, raw pixel values of an image, coordinates in Euclidean space, or even non-numeric structures like sequences or graphs.

A *feature map* $\phi : \mathcal{X} \to \mathcal{F}$ is a function that transforms each input x into a vector $\phi(x)$ in a *feature space* $\mathcal{F}$. The feature space is typically a vector space (often a Hilbert space) equipped with an inner product $\langle \cdot, \cdot \rangle_{\mathcal{F}}$, which allows us to measure angles, distances, and perform geometric operations.

The choice of feature map ϕ determines what aspects of the data are emphasized. By mapping data into a carefully chosen feature space, we can:

- Make nonlinear patterns in $\mathcal{X}$ appear linear in $\mathcal{F}$
- Encode domain knowledge about relevant data characteristics
- Change the dimensionality to enable different representations (increasing it for richer structure, or reducing it via feature selection)
- Define meaningful notions of similarity that are difficult to express in the original space

Algorithm 26: Kernel nearest neighbour regression

Input: kernel k on $\mathcal{X}$, minimum number of neighbours $n \in \mathbb{N}^+$, data $\{(x_i, y_i)\}_{i=1}^P \subset \mathcal{X} \times \mathbb{R}$, new input $x_* \in \mathcal{X}$
Output: predicted label y of x_*
$S = \{(x_i, y_i)\}_{i=1}^P$
$S_n = \emptyset$
while $|S_n| < n$ **do**
  $S' \leftarrow \left\{\text{argmin}_{(x,y)\in S} \sqrt{k(x_*, x_*) - 2\,k(x_*, x) + k(x, x)}\right\}$
  $S_n \leftarrow S_n \cup S'$
  $S \leftarrow S \setminus S'$
return $y = \frac{1}{|S_n|} \sum_{(x_i,y_i)\in S_n} y_i$

23.1.2 Kernels as Inner Products in Feature Space

A *kernel function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ computes the inner product between feature representations without explicitly constructing those representations. Specifically, for any two inputs $x, x' \in \mathcal{X}$:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{F}} \tag{23.1}$$

That is, every kernel is a dot product in a (possibly infinite-dimensional) transformed space. This fact also holds in reverse: every feature map determines a kernel. This equation also reveals the kernel's dual nature: it is both a similarity measure in the input space and an inner product in the feature space. The kernel value $k(x, x')$ quantifies how aligned the feature representations $\phi(x)$ and $\phi(x')$ are in $\mathcal{F}$.

Geometric Interpretation. In the feature space $\mathcal{F}$, the inner product $\langle \phi(x), \phi(x') \rangle_{\mathcal{F}}$ captures:

- **Magnitude and direction:** Large positive values indicate that $\phi(x)$ and $\phi(x')$ point in similar directions and have large norms
- **Similarity:** The inner product is maximized when the two vectors are parallel
- **Orthogonality:** When $k(x, x') = 0$, the feature representations are orthogonal, indicating complete dissimilarity in the feature space

Note that the inner product conflates magnitude and direction: a large value of $k(x, x')$ may reflect either strong alignment or large norms $\|\phi(x)\|$, $\|\phi(x')\|$. To interpret the inner product purely as directional similarity, one can normalize by the norms, yielding the cosine similarity $k(x, x')/\sqrt{k(x, x)\,k(x', x')}$. Alternatively, the kernel induces a metric in feature space:

$$d_k(x, x') = \|\phi(x) - \phi(x')\|_{\mathcal{F}} = \sqrt{k(x, x) + k(x', x') - 2\,k(x, x')}, \tag{23.2}$$

which provides a proper distance that separates magnitude from direction. The feature map can capture complex, nonlinear relationships, and this distance need not correspond to simple distance in the input space.

This metric immediately gives rise to simple kernelized algorithms. As an example, Algorithm 26 shows kernel nearest neighbour regression, which finds the closest training points to a query using only kernel evaluations and averages their labels.

23.1.3 Example: Quadratic Feature Map

To make these ideas concrete, consider a simple nonlinear transformation in two dimensions.

Example 23.1 (Quadratic Feature Map)**.** *Let $\mathcal{X} = \mathbb{R}^2$ be the two-dimensional input space. Define the quadratic feature map $\phi : \mathbb{R}^2 \to \mathbb{R}^3$ by:*

$$\phi\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \end{pmatrix} \tag{23.3}$$

This mapping transforms each point $x = (x_1, x_2)^\top$ into a three-dimensional vector whose components are quadratic monomials of the input coordinates. The feature space $\mathcal{F} = \mathbb{R}^3$ is equipped with the standard Euclidean inner product.

Computing similarity via inner products. *Consider two input points:*

$$x = \begin{pmatrix} 5.9 \\ 3.0 \end{pmatrix}, \qquad y = \begin{pmatrix} 6.9 \\ 3.1 \end{pmatrix}$$

Their feature representations are:

$$\phi(x) = \begin{pmatrix} 5.9^2 \\ 3.0^2 \\ \sqrt{2} \cdot 5.9 \cdot 3.0 \end{pmatrix} = \begin{pmatrix} 34.81 \\ 9.00 \\ 25.03 \end{pmatrix}$$

$$\phi(y) = \begin{pmatrix} 6.9^2 \\ 3.1^2 \\ \sqrt{2} \cdot 6.9 \cdot 3.1 \end{pmatrix} = \begin{pmatrix} 47.61 \\ 9.61 \\ 30.25 \end{pmatrix}$$

The inner product in feature space is:

$$\begin{aligned} \langle \phi(x), \phi(y) \rangle_{\mathcal{F}} &= 34.81 \times 47.61 + 9.00 \times 9.61 + 25.03 \times 30.25 \\ &= 1657.30 + 86.49 + 757.16 \\ &= 2500.95 \end{aligned}$$

This value quantifies the similarity between x and y in the quadratic feature space. Large positive values indicate strong similarity under this particular nonlinear transformation.

Why use feature maps? The quadratic feature map enables algorithms that operate via inner products (such as linear classifiers or regression) to capture quadratic decision boundaries in the input space. What appears as a nonlinear pattern in $\mathbb{R}^2$ becomes a linear pattern in $\mathbb{R}^3$.

23.1.4 The Computational Challenge

While feature maps provide a principled way to define similarity, they introduce a computational problem: the feature space $\mathcal{F}$ may be very high-dimensional, or even infinite-dimensional. Computing $\phi(x)$ explicitly can be expensive or impossible.

For instance, a polynomial feature map of degree q in d dimensions has feature space dimensionality:

$$\dim(\mathcal{F}) = \binom{d+q}{q}$$

which grows rapidly with both d and q. For $d = 100$ and $q = 3$, this yields over 176,000 dimensions. Explicitly computing and storing $\phi(x)$ for each data point becomes prohibitively expensive. Even more dramatically, some kernels, such as the Gaussian RBF kernel, correspond to infinite-dimensional feature spaces, making explicit feature computation impossible in principle.

This observation motivates the central insight of kernel methods, which we explore in Section 23.3: ***if our algorithm can be expressed entirely in terms of inner products*** $\langle \phi(x), \phi(x') \rangle_{\mathcal{F}}$***, we need never compute*** $\phi(x)$ ***explicitly. Instead, we compute the kernel function*** $k(x, x')$ ***directly in the input space, bypassing the feature space entirely while achieving the same result.***

Remark 23.1. *The kernel function $k(x, x')$ serves as an implicit representation of a potentially very complex feature map ϕ. This implicit representation is what makes kernel methods both computationally practical and theoretically elegant: we can work in rich, high-dimensional spaces without ever constructing them explicitly.*

23.2 Kernel Functions

A *kernel function* provides a principled way to measure alignment between data points in a feature space, which can informally be thought of as the similarity between data points. Formally, a kernel is a symmetric function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ that satisfies certain mathematical properties, i.e. symmetry and positive semi-definiteness of the corresponding kernel (or Gram) matrices. Given any $N \in \mathbb{N}$ and points $\{x_i\}_{i=1}^N$, the Gram matrix $K \in \mathbb{R}^{N \times N}$ with $K_{ij} = k(x_i, x_j)$ must be positive semidefinite.

23.2.1 Positive semi-definite kernels

Definition 23.1 (Positive semi-definite kernel)**.** *A function $k(\cdot,\cdot) : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a* positive semi-definite *(psd) kernel if it is symmetric, i.e., $k(x, x') = k(x', x)$ for all $x, x' \in \mathcal{X}$, and for any finite set of points $\{x_1, \ldots, x_N\} \subset \mathcal{X}$ and any $\boldsymbol{\alpha} \in \mathbb{R}^N$,*

$$\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i \alpha_j k(x_i, x_j) = \boldsymbol{\alpha}^\top K \boldsymbol{\alpha} \geq 0, \tag{23.4}$$

where $K \in \mathbb{R}^{N\times N}$ is the Gram matrix with entries $K_{ij} = k(x_i, x_j)$. A kernel is called strictly positive definite *if the inequality is strict for all $\boldsymbol{\alpha} \neq \mathbf{0}$ whenever the points $x_1, \ldots, x_N$ are distinct.*

In the literature, psd kernels are often simply called "positive definite kernels." The positive semi-definiteness of K guarantees that the kernel corresponds to an inner product in some (possibly infinite-dimensional) feature space, enabling geometric reasoning about similarity, a connection we explore in detail in 23.3. Most commonly used kernels (e.g., the squared exponential) are in fact strictly positive definite, but the psd condition suffices for the theory developed here.

Common Examples

- **Linear kernel:** $k(x, x') = x^\top x'$; when used in GP regression, this is equivalent to Bayesian linear regression with a Gaussian prior.
- **Polynomial kernel:** $k(x, x') = (x^\top x' + c)^d$ with $c \geq 0$, $d \in \mathbb{N}$.
- **Squared Exponential (Gaussian/RBF):**

  $$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right),$$

  where the length-scale parameter $\ell > 0$ controls smoothness (larger ℓ means smoother functions).
- **Matérn kernel:** A family of kernels parameterized by smoothness $\nu > 0$:

  $$k_\nu(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu}\frac{r}{\ell}\right)^\nu K_\nu \left(\sqrt{2\nu}\frac{r}{\ell}\right)$$

  where $r = \|x - x'\|$, $\ell > 0$ is the length-scale, and K_ν is the modified Bessel function of the second kind. Common special cases include Matérn-1/2, Matérn-3/2, and Matérn-5/2.
- **Laplacian Kernel:**

  $$k(x, x') = \exp(-\gamma\|x - x'\|_1),$$

  where $\|x - x'\|_1$ is the Manhattan distance (or L1 norm) between the vectors (the L2 norm can also be used) and γ is a tuning parameter that controls the width or "spread" of the kernel. A small γ gives a broad, smooth kernel, while a large γ creates a sharper, more localized kernel.
- **Indicator Kernel:**

  $$k(x, x') = \mathbf{1}(x = x'),$$

  where $\mathbf{1}(\cdot)$ is the indicator function.
- **Arc-cos kernel:** Used in neural network GP limits (26):

  $$k(x, x') = \sigma_b^2 + \frac{\sigma_w^2}{\pi}\sqrt{k_{11}k_{22}}(\sin\theta + (\pi - \theta)\cos\theta)$$

  where $\theta = \arccos(k_{12}/\sqrt{k_{11}k_{22}})$ with $k_{ij} = x_i^\top x_j / N_0$.
- **Periodic kernel:** $k(x, x') = \exp[-2\sin^2(\pi\|x - x'\|/p)/\ell^2]$.

23.2.2 Kernel as a Similarity Measure

Having defined several kernel families above, we now make explicit the role that the kernel value plays as a notion of similarity. A kernel value $k(x, x')$ measures similarity between two inputs x and x'. In later sections where a function f is modelled with this kernel, $k(x, x')$ will play the role of the expected covariance between the function evaluations $f(x)$ and $f(x')$. Large values imply that function values at x and x' are likely similar, while small values indicate independence or dissimilarity. Hence, the kernel implicitly defines a notion of distance or correlation in the input space: the smaller the effective distance between x and x', the stronger their correlation.

Different kernels correspond to different inductive biases about how similarity should behave:

- The RBF (squared exponential) kernel corresponds to very smooth, globally correlated functions that vary slowly with input distance.
- The Matérn family provides explicit control of smoothness and local variability through the parameter ν, enabling rougher or smoother function priors.
- Periodic kernels impose repeating structure, suitable for cyclic or seasonal patterns.

The practitioner's choice of kernel thus amounts to specifying a *prior* over function behaviour and determining what kinds of variation are expected in unseen regions of the input space.

23.2.3 Kernel Compositions and Operations

A powerful feature of kernels is that they can be combined to create more expressive similarity measures while preserving positive definiteness. Sums, products, and positive scalings of kernels preserve positive definiteness.

Proposition 23.1 (Kernel Operations)**.** *If k_1 and k_2 are positive definite kernels, then so are:*

1. *Sum:* $k(x, x') = k_1(x, x') + k_2(x, x')$
2. *Product:* $k(x, x') = k_1(x, x') \cdot k_2(x, x')$
3. *Scaling:* $k(x, x') = c \cdot k_1(x, x')$ *for* $c > 0$

These composition rules enable practitioners to build complex kernels from simple building blocks, encoding structured prior knowledge. For example, a sum of periodic and SE kernels can model functions with both trend and seasonal components.

23.2.4 Spectral Perspective

This subsection can be skipped on a first reading.

For stationary kernels, i.e. those that depend only on the difference $\tau = x - x'$, it is often useful to describe them in terms of their frequency content rather than their shape in input space. *Bochner's theorem* [176] provides this connection: a continuous, shift-invariant kernel $k(\tau)$ on $\mathbb{R}^d$ is positive definite if and only if it is the Fourier transform of a nonnegative measure $S(\omega)$:

$$k(\tau) = \int_{\mathbb{R}^d} e^{i\omega^\top \tau} S(\omega)\, \mathrm{d}\omega. \tag{23.5}$$

The function $S(\omega)$ is called the *spectral density* of the kernel and describes how much each frequency ω contributes to the covariance between inputs.

This spectral view provides an intuitive link between a kernel's smoothness and its frequency content:

- Smooth kernels (e.g., the RBF) have rapidly decaying $S(\omega)$, concentrating power at low frequencies and producing slowly varying functions.
- Rougher kernels (e.g., Matérn) have heavier-tailed $S(\omega)$, reflecting higher-frequency variability and less smoothness.

This spectral viewpoint motivates approximations like *random Fourier features*, which represent kernels through sampled frequencies $\omega \sim S(\omega)$. It also clarifies how kernel hyperparameters (e.g., the length-scale) determine the effective frequency range of the modelled functions.

Finally, some kernels are called *universal* or *characteristic*, meaning they can approximate any continuous function on compact domains or uniquely represent probability distributions through their mean embeddings. These properties make them powerful tools for both regression and statistical testing.

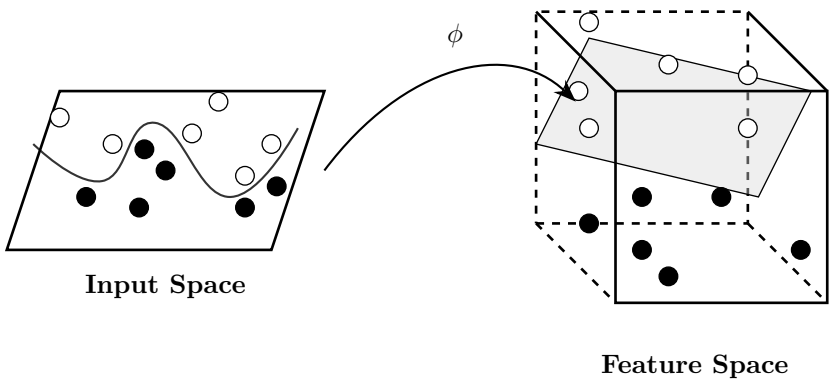


Figure 23.1: **The kernel trick.** A nonlinear mapping $\phi(x)$ transforms data from the input space (left) to a high-dimensional feature space (right) where the classes become linearly separable. Although the feature space may be high- or infinite-dimensional, the kernel trick allows computation using only pairwise kernel evaluations $k(x, x') = \langle \phi(x), \phi(x') \rangle$, without explicitly constructing $\phi(x)$.

23.3 The Kernel Trick

The kernel trick is a computational technique that enables efficient operations in high-dimensional (even infinite-dimensional) feature spaces without explicitly computing the features themselves. This elegant idea underpins much of kernel methods' success.

23.3.1 Feature Maps and Implicit Representations

Every positive definite kernel $k(\cdot, \cdot)$ corresponds to a feature map $\phi : \mathcal{X} \to \mathcal{H}$ into some Hilbert space $\mathcal{H}$ such that:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}} \tag{23.6}$$

The key insight is that we can perform all computations using only kernel evaluations, without ever explicitly computing $\phi(x)$. In other words, Equation (23.6) shows that every positive definite kernel implicitly defines a mapping $\phi : \mathcal{X} \to \mathcal{H}$ such that the kernel value is simply the inner product between features in $\mathcal{H}$. The feature space $\mathcal{H}$ can be extremely high-dimensional or even infinite, but all computations depend only on the pairwise kernel evaluations $k(x, x')$.

This observation means that any algorithm that can be expressed in terms of dot products can be *kernelized* by substituting $\langle x, x' \rangle \to k(x, x')$. This single substitution turns linear methods into nonlinear ones while preserving the same algebraic structure.

As illustrated by the quadratic feature map in Example 23.1, computing $\phi(x)$ explicitly for N_0-dimensional input requires $O(N_0^2)$ operations, but evaluating the kernel directly takes only $O(N_0)$ operations. The kernel trick thus achieves the same geometric effect of projecting into a different feature space at a fraction of the computational cost. For some kernels like the squared exponential (SE), the corresponding feature space is infinite-dimensional, making explicit feature computation impossible. Yet the kernel can be evaluated efficiently in $O(N_0)$ time.

23.3.2 Reproducing Kernel Hilbert Spaces

The mathematical foundation underlying the kernel trick is the concept of a *Reproducing Kernel Hilbert Space* (RKHS). An RKHS provides the link between kernel functions and inner product spaces of functions, ensuring that evaluation at any point can be represented as an inner product.

Definition 23.2 (Reproducing Kernel Hilbert Space)**.** *Let $\mathcal{H}$ be a Hilbert space of functions $f : \mathcal{X} \to \mathbb{R}$. We say that $\mathcal{H}$ is a* Reproducing Kernel Hilbert Space *if there exists a kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ satisfying:*

1. *For every $x \in \mathcal{X}$, the function $k(x, \cdot)$ belongs to $\mathcal{H}$.*
2. *(**Reproducing property**) For all $f \in \mathcal{H}$ and all $x \in \mathcal{X}$,*

$$f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}}. \tag{23.7}$$

The reproducing property ensures that pointwise evaluation is a continuous linear functional in $\mathcal{H}$, a feature that distinguishes RKHSs from generic Hilbert spaces of functions. Each positive definite kernel $k(\cdot,\cdot)$ uniquely defines an RKHS $\mathcal{H}_k$, and conversely, every RKHS admits a unique reproducing kernel. This result is known as the **Moore-Aronszajn Theorem**, cf. e.g. [128].

This duality establishes a one-to-one correspondence between kernels and function spaces: the choice of kernel implicitly determines the class of admissible functions and their smoothness. For instance, the RKHS associated with the SE kernel consists of infinitely differentiable functions, while that of the Matérn kernel contains functions with finite smoothness controlled by the parameter ν.

23.3.3 Computational Advantage and the Representer Theorem

The computational power of the kernel trick becomes evident in regularized learning problems defined over a Reproducing Kernel Hilbert Space $\mathcal{H}$. Given a training set $\{(x_i, y_i)\}_{i=1}^P \subset \mathcal{X} \times \mathbb{R}$, consider the objective

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{P} \ell(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2, \tag{23.8}$$

where $\ell(\cdot,\cdot)$ is a pointwise loss function (e.g., squared, hinge, or logistic loss) and $\lambda > 0$ is a regularization parameter controlling model complexity.

The **Representer Theorem** [891, 1472] states that any optimal solution to (23.8) can be expressed as a finite linear combination of kernel evaluations at the training points:

$$f^*(x) = \sum_{i=1}^{P} \alpha_i k(x_i, x), \tag{23.9}$$

for some coefficients $\alpha = [\alpha_1, \ldots, \alpha_P]^\top \in \mathbb{R}^P$. This result implies that even though $\mathcal{H}$ may be infinite-dimensional, the optimization problem reduces to one over the coefficients α, involving only the kernel (Gram) matrix K with entries $K_{ij} = k(x_i, x_j)$. Predictions thus require only kernel evaluations $k(x_i, x)$ rather than explicit feature computations, providing the foundation for the efficiency of kernel methods.

23.4 Gaussian Processes

Having introduced kernels as measures of similarity and as inner products in feature spaces, we are now ready to use them as *covariance functions* that define distributions over functions. A Gaussian process (GP) can be thought of as the probabilistic analogue of a kernel machine: where a kernel specifies how inputs are related, a GP interprets this as a statement about how function values co-vary across the input space. In this sense, GPs turn the geometry induced by a kernel into a fully Bayesian model of function learning. Frequentist kernel methods and GPs are closely related and share common mathematical foundations, as detailed in [865].

GP extends the notion of a Gaussian distribution from finite-dimensional vectors to functions (thinking of functions as infinitely long vectors containing the value of the function at every input).

Definition 23.3 (Gaussian Process [1382])**.** *Let $f : \mathcal{X} \to \mathbb{R}$ be a real-valued function defined on an input space $\mathcal{X}$. We say that f is a Gaussian Process, denoted*

$$f \sim \mathcal{GP}(m(\cdot), k(\cdot,\cdot)),$$

if for any finite collection of inputs $\mathbf{x} = [x_1, \ldots, x_P]^\top$, the random vector of function evaluations

$$\mathbf{f} = [f(x_1), \ldots, f(x_P)]^\top$$

follows a multivariate normal distribution

$$\mathbf{f} \sim \mathcal{N}\big(\boldsymbol{\mu}, K_{\mathbf{xx}}\big),$$

with mean $\boldsymbol{\mu}_i = m(x_i)$ and covariance $[K_{\mathbf{xx}}]_{ij} = k(x_i, x_j)$.

The mean function $m(x) = \mathbb{E}[f(x)]$ describes the expected value of the process at x, while the kernel or covariance function $k(x, x') = \mathrm{Cov}[f(x), f(x')]$ encodes how function values co-vary across inputs. In practice, one often assumes $m(x) = 0$. This is reasonable when the data have been centred or when no specific prior knowledge about the function's mean behaviour is available; the mean function can always be absorbed into the model by subtracting a prior estimate of the trend from the observations before fitting the GP.

Kernel Matrix. For inputs $\mathbf{x} = [x_1, \ldots, x_P]$, the kernel matrix is

$$K_{\mathbf{xx}} = \begin{bmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_P) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_P) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_P, x_1) & k(x_P, x_2) & \cdots & k(x_P, x_P) \end{bmatrix}.$$

As established in Definition 23.1, the kernel must be symmetric and positive semi-definite, which guarantees that $K_{\mathbf{xx}}$ defines a valid covariance matrix.

Remark 23.2. *The kernel function in a GP both defines the covariance structure of the process and determines the similarity measure for interpolation. This connection between the probabilistic and geometric views is fundamental to GP reasoning.*

23.4.1 Posterior Inference

Given training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^P$, we assume that observations are noisy evaluations of an underlying latent function f:

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where σ^2 denotes the noise variance.

Let $\mathbf{y} = [y_1, \ldots, y_P]^\top$ be the observed targets, and consider a new input x_* with latent value $f_* = f(x_*)$. The joint prior over training and test outputs is

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mathbf{0} \\ 0 \end{bmatrix}, \begin{bmatrix} K_{\mathbf{xx}} + \sigma^2 I & K_{\mathbf{x}x_*} \\ K_{x_*\mathbf{x}} & K_{x_*x_*} \end{bmatrix} \right),$$

where $K_{\mathbf{x}x_*} = [k(x_1, x_*), \ldots, k(x_P, x_*)]^\top$. The noise term $\sigma^2 I$ is added only to $K_{\mathbf{xx}}$ because the training targets $\mathbf{y}$ are noisy observations, whereas f_* denotes the latent (noise-free) function value at x_*.

Conditioning on the observed data gives the posterior distribution:

$$p(f_* | \mathbf{x}, \mathbf{y}, x_*) = \mathcal{N}\big(\mu_*,\, \sigma_*^2\big),$$

with

$$\mu_* = K_{x_*\mathbf{x}} \left[K_{\mathbf{xx}} + \sigma^2 I\right]^{-1} \mathbf{y}, \qquad \sigma_*^2 = k(x_*, x_*) - K_{x_*\mathbf{x}} \left[K_{\mathbf{xx}} + \sigma^2 I\right]^{-1} K_{\mathbf{x}x_*}.$$

Interpretation. The posterior mean μ_* provides the predictive expectation of $f(x_*)$, while the variance σ_*^2 quantifies uncertainty. Near observed data points, σ_*^2 tends to be small, while it increases in regions far from the training data. This property makes GPs particularly useful for uncertainty-aware regression, Bayesian optimization, and active learning (see Chapter 53).

23.5 Limitations

Despite their elegance, standard GPs face several fundamental challenges that motivate the deep extensions explored in Chapters 25–27. These limitations concern both their *representational capacity* and their *computational scalability.*

23.5.1 Fixed Representations and Expressiveness

Standard kernel methods operate directly on raw input features, defining similarity through fixed metrics such as the Euclidean distance. However, this assumption often fails in modern high-dimensional or structured data domains.

Uninformative raw features. In domains such as images, text, or graphs, the raw input coordinates lack semantic content and do not encode the relational or compositional structure necessary for generalization. Kernels defined over such features therefore fail to capture the meaningful invariances or dependencies that underlie the data.

High-dimensional inputs. As the dimensionality of the input space grows, pairwise distances tend to concentrate, making commonly used stationary kernels, such as the RBF, nearly constant. This "distance concentration" effect diminishes the discriminative power of kernel similarity measures, rendering them ineffective in capturing local geometry or fine-grained variation within the data manifold.

Limited adaptability. Once the functional form and hyperparameters of a kernel are fixed, its inductive bias becomes static. Consequently, standard kernels cannot learn task- or data-dependent notions of similarity, preventing them from adapting to complex, structured patterns. This lack of flexibility fundamentally restricts their expressiveness in modern representation learning settings, where adaptive, data-driven feature extraction is essential.

23.5.2 Computational Scalability

For most practical applications, exact Gaussian process inference is computationally expensive: evaluating and inverting the $P \times P$ kernel matrix scales as $O(P^3)$, which limits standard GPs to datasets of a few thousand points.

To make GPs tractable for large datasets and complex likelihoods, approximate inference schemes are employed:

- **Variational inference:** introduces an approximate posterior distribution that can be optimized efficiently and supports non-Gaussian likelihoods (e.g., classification or count models) [1615].
- **Sparse and inducing-point methods:** summarize the dataset using a small set of representative inputs, reducing complexity to $O(P_i^3 + P_i^2 P)$ with $P_i \ll P$ [1764].

These ideas are central to the deep kernel models discussed later. Each layer or GP component uses such approximations to remain scalable. Scalable GP methods are treated in depth in Chapter 38, and detailed derivations of deep kernel models appear in Chapters 24 and 27.

23.5.3 The Path Forward: Deep and Adaptive Gaussian Processes

The next chapters explore how to overcome these limitations by making *the kernel itself learnable, hierarchical, and data-adaptive*. This evolution parallels the transition from linear models to deep networks in standard machine learning.

- **Chapter 25 — Deep Gaussian Processes:** Introduces hierarchical compositions of GPs, where the outputs of one GP feed into another. These deep GPs capture multiple levels of abstraction and non-stationarity, addressing the expressiveness gap of shallow kernels.
- **Chapter 24 — Deep Kernel Learning:** Combines the representation power of deep neural networks with the uncertainty quantification of GPs. Here, the kernel is learned from data via a neural network feature extractor, providing scalable, end-to-end trainable models.
- **Chapter 27 — Meta-Learning and Function Priors:** Generalizes the concept further by learning priors over functions across tasks, enabling few-shot adaptation and transfer. These models tie together the Bayesian principles of GPs with the flexibility of meta-learning.

Together, these developments trace the progression from classical kernel methods to modern *Bayesian deep learning in function space*. Each subsequent chapter builds directly on the probabilistic foundations established here, transforming GPs from elegant theoretical constructs into scalable, representation-learning engines.

BDL Connection

How kernel methods connect to Bayesian deep learning.
Kernel methods are among the earliest approaches to Bayesian learning in function space [1382]. A *kernel* defines a covariance structure over functions, specifying how values at different inputs are expected to covary. In Gaussian processes, this covariance is combined with a Gaussian prior to produce full predictive distributions, capturing both predictions and their uncertainty.
From the perspective of deep learning, kernel methods provide the *Bayesian limit of neural networks*: when a network's width tends to infinity, its outputs converge to a GP whose kernel is determined by the network architecture (e.g. the Neural Network Gaussian Process [1000, 1226, 1759] or Neural Tangent Kernel [829, 1794]). Thus, every deep network implicitly defines a kernel and training it corresponds to learning that kernel's geometry.
Modern *Bayesian deep learning* builds on this connection:

- **Representation learning:** Deep kernel learning replaces the fixed input kernel by one computed from a learned deep representation.

- **Hierarchical function priors:** Deep Gaussian processes stack multiple GPs to model complex, non-stationary mappings.
- **Uncertainty calibration:** The probabilistic nature of GPs provides well-founded uncertainty estimates, a property extended in BDL models.

Understanding kernels therefore clarifies the foundations on which deep Bayesian models are built: they describe *how we believe functions should behave* before data are seen, and how these beliefs evolve as models grow deeper.

Chapter 24

Deep kernel learning

Historically, neural networks and kernel methods have often been treated as competing approaches. Deep kernel learning (DKL) [1769] combines a deterministic neural-network feature extractor with a kernel method, most commonly a Gaussian process (GP). At prediction time, standard DKL evaluates the feature extractor once for each test input and then performs GP prediction in the learned feature space. This avoids the repeated stochastic neural-network passes required by some Bayesian neural-network approximations, but GP prediction still requires kernel computations against stored training or inducing-point features.

Deep kernel learning transforms the inputs of a base kernel, such as a radial basis function (RBF) kernel, through a neural network. The resulting *deep kernel* has hyperparameters from both the base kernel and the neural-network feature map. For Gaussian-likelihood regression, the GP predictive distribution and marginal likelihood are available in closed form for any fixed values of these hyperparameters. Learning the hyperparameters still requires numerical optimisation, and classification or other non-Gaussian likelihoods require approximate inference. Moreover, standard DKL is an empirical-Bayes (type-II) construction: it integrates over GP function values but point-estimates the neural-network and kernel parameters rather than placing a posterior over them.

The remainder of the chapter reviews Gaussian processes (Section 24.1), introduces the deep-kernel construction (Section 24.2), describes stochastic variational deep kernel learning (Section 24.3), discusses estimation choices (Section 24.4), and closes with practical resources (Section 24.5).

Applications and surveys involving DKL or closely related GP–neural-network hybrids span biological sequence design [1557], physics-informed machine learning [872], materials science [299], computational chemistry [877], autonomous vehicles [18], semi-supervised learning [838], few-shot learning [1298], healthcare [268, 1034], and astrobiology [1880]. Related hybrid methods and DKL extensions include [202, 226, 1062, 1714]. DKL has also been used in Bayesian optimisation [e.g., 1036, 1557], where predictive uncertainty informs the exploration–exploitation trade-off; what counts as "far from the data", however, is determined by the learned representation.

24.1 Review of Gaussian Processes

Although deep kernels can be used with other kernel methods, this chapter focuses on Gaussian processes because they provide both predictive distributions and a marginal-likelihood objective for hyperparameter estimation.

For a full treatment, see Rasmussen and Williams [1382]. A *Gaussian process* is a distribution over functions whose every finite-dimensional marginal is Gaussian. We write $f \sim \mathcal{GP}(m, k)$ when, for every finite input set $X = \{x_1, \ldots, x_n\}$,

$$\big(f(x_1), \ldots, f(x_n)\big)^\top \sim \mathcal{N}(m_X, K_{XX}), \tag{24.1}$$

where $(m_X)_i = m(x_i)$ and $(K_{XX})_{ij} = k(x_i, x_j)$.

For simplicity, we assume $m(x) = 0$ unless stated otherwise. The kernel then plays the central role in determining the covariance and generalisation properties of the process. A standard choice is the RBF kernel

$$k_{\mathrm{RBF}}(x_i, x_j) = a^2 \exp\left(-\frac{\|x_i - x_j\|^2}{2\ell^2}\right), \tag{24.2}$$

where the amplitude a^2 controls the signal variance and the length-scale ℓ controls how rapidly correlations decay with distance. Larger length-scales produce more slowly varying functions, whereas shorter length-scales permit more rapidly varying functions. In forecasting, for example, ℓ can be interpreted as the time range over which past observations remain substantially correlated with a forecast.

For Gaussian-likelihood regression, assume

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2),$$

with the noise independent of f. Given training inputs $X = \{x_i\}_{i=1}^N$ and observations $\boldsymbol{y} = (y_1, \dots, y_N)^\top$, the latent-function predictive distribution at x_* is

$$\begin{aligned} p(f_* \mid \boldsymbol{y}, x_*, X) &= \mathcal{N}(f_* \mid \mu_*, v_*), \\ \mu_* &= k_{*X}\big(K_{XX} + \sigma^2 \mathbf{I}_N\big)^{-1}\boldsymbol{y}, \\ v_* &= k_{**} - k_{*X}\big(K_{XX} + \sigma^2 \mathbf{I}_N\big)^{-1}k_{X*}. \end{aligned} \tag{24.3}$$

The predictive distribution for a noisy observation y_* has the same mean and variance $v_* + \sigma^2$.

Writing $\boldsymbol{f} = (f(x_1), \dots, f(x_N))^\top$, the marginal likelihood is

$$p(\boldsymbol{y} \mid \theta, \sigma^2, X) = \int p(\boldsymbol{y} \mid \boldsymbol{f}, \sigma^2) p(\boldsymbol{f} \mid \theta, X)\, \mathrm{d}\boldsymbol{f}. \tag{24.4}$$

For Gaussian-likelihood regression,

$$\log p(\boldsymbol{y} \mid \theta, \sigma^2, X) = -\frac{1}{2}\boldsymbol{y}^\top\big(K_\theta + \sigma^2 \mathbf{I}_N\big)^{-1}\boldsymbol{y} - \frac{1}{2}\log\big|K_\theta + \sigma^2 \mathbf{I}_N\big| + \text{const.} \tag{24.5}$$

The first term measures data fit, while the log-determinant term acts as a complexity penalty and connects to minimum description length [1111]. The marginal likelihood can therefore be used to estimate kernel hyperparameters and the observation-noise variance.

24.2 Deep Kernel Learning

Let k_θ be a base kernel with hyperparameters θ, and let g_w be a neural-network feature map with weights w. The resulting deep kernel is

$$k_\gamma(x, x') := k_\theta(g_w(x), g_w(x')), \qquad \gamma = (\theta, w). \tag{24.6}$$

Composition with a deterministic feature map preserves positive semidefiniteness, so k_γ is a valid kernel whenever k_θ is. Structurally, this places a GP on the features produced by the neural network.

The base kernel supplies a covariance model, while the neural network learns a task-adapted representation in which that covariance model is applied. For Gaussian-likelihood regression and fixed γ, prediction requires one forward pass through g_w for each test input, followed by GP prediction using cross-covariances with stored training or inducing-point features. When γ is learnt by marginal-likelihood maximisation, the usual DKL construction is empirical Bayes: w and θ are point-estimated rather than integrated out.

Once constructed, k_γ can be used like any other kernel. The principal estimation strategies are discussed in Section 24.4.

Dense GP regression requires $\mathcal{O}(N^3)$ time and $\mathcal{O}(N^2)$ memory. Structured kernel interpolation (SKI, also known as KISS-GP) [1764] introduces a structured inducing grid and sparse local interpolation. In favourable regimes, this yields fast matrix–vector products and iterative inference whose cost can be close to linear in N, with additional dependence on the number and structure of the inducing points. SKI is an approximation rather than exact GP inference, and it is most effective in low-dimensional spaces; a neural feature map can supply such a space.

Other scalable GP approaches can also be combined with DKL. GPU-accelerated conjugate-gradient solves and stochastic log-determinant estimates are effective in practice [580], but finite solve tolerances and stochastic estimators remain numerical approximations, while inducing-point and variational methods introduce further statistical approximations.

A stationary RBF kernel measures similarity through Euclidean distance in its input space. That distance may be appropriate in low dimensions but can be poorly aligned with the prediction task in high-dimensional inputs. For example, two face images can be far apart in pixel space while having similar orientations. A learned feature map can make Euclidean distance more task-relevant, so DKL learns a task-adapted kernel rather than the "right" kernel in any globally optimal sense. Figure 24.1 gives a stylised one-dimensional illustration of both the benefit and the caveat: feature warping changes the fit and the predictive uncertainty, and distance from the training data is measured in the learned feature space rather than the original input space.

The RBF kernel is a common base kernel, but it is not the only option. A spectral-mixture kernel [1763], for example, can encode quasi-periodic structure, leaving the neural feature map to capture complementary patterns.

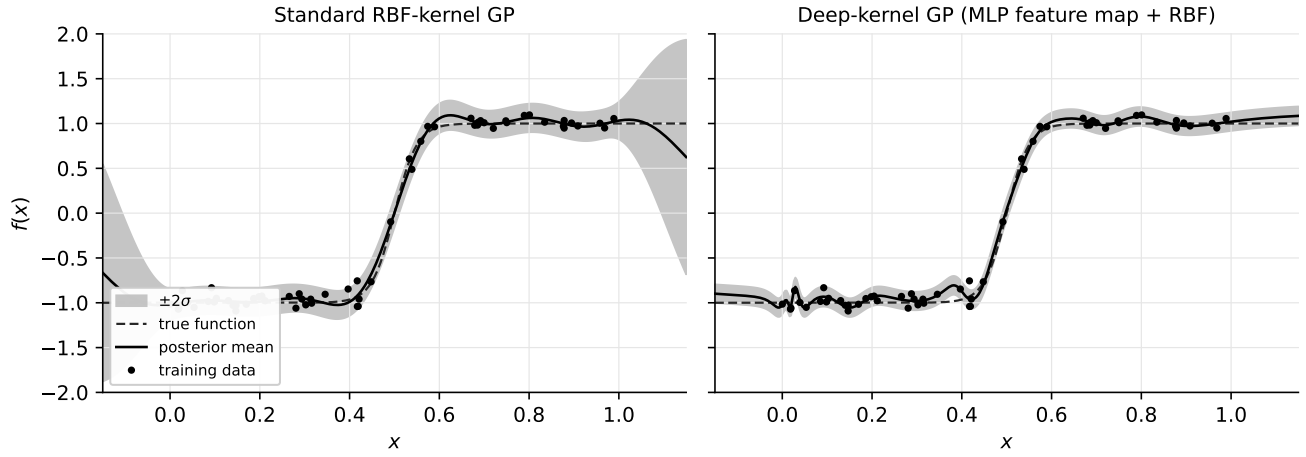


Figure 24.1: Illustrative deep-kernel regression for a target with a sharp transition, $f(x) = \tanh(20(x - 0.5))$, observed with independent Gaussian noise. The dashed line is the target, dots are the same 60 training observations in both panels, the solid line is the predictive mean, and the shaded region is the mean plus or minus two predictive standard deviations for a future noisy observation. Left: a GP with a stationary RBF kernel in the original input space. Right: a GP with the same RBF base kernel applied after a small multilayer-perceptron feature map, with all parameters learnt by marginal-likelihood maximisation. In this illustrative optimisation run, the deep kernel produces much narrower intervals, including outside the training range. The figure illustrates representation-dependent predictive behaviour, not superior calibration: narrower intervals are not automatically better calibrated and may reflect overconfidence under extrapolation.

24.3 Stochastic Variational Deep Kernel Learning

So far, we have considered DKL for Gaussian-likelihood regression, where the GP predictive distribution and marginal likelihood are available in closed form for fixed deep-kernel parameters. For classification and other non-Gaussian likelihoods, exact prediction and evaluation of the marginal likelihood are analytically intractable. Multi-class problems also require a construction for correlated outputs.

Wilson et al. [1766] address these challenges with *stochastic variational deep kernel learning* (SVDKL), building on sparse variational GP methods [737, 738, 1615]. SVDKL optimises a variational lower bound rather than the exact log marginal likelihood, supports mini-batch stochastic gradients, and accommodates non-Gaussian likelihoods and multiple outputs. For these reasons, it is also used for large-scale regression. Alternative formulations include Bradshaw et al. [202].

24.3.1 Details of SVDKL

We describe the construction of Wilson et al. [1766]; Section 24.5 discusses implementations that need not follow every detail of this formulation. Let

$$\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}, \qquad \boldsymbol{y}_i \in \{0,1\}^C,$$

where C is the number of classes and $\boldsymbol{y}_i$ is a one-hot label. The neural feature extractor produces $\boldsymbol{z}_i = g_w(\boldsymbol{x}_i) \in \mathbb{R}^Q$. For each $j = 1, \dots, J$, let $\boldsymbol{z}_i^{(j)}$ denote the subset or projection of $\boldsymbol{z}_i$ supplied to latent GP f_j, and define

$$\boldsymbol{f}_i := \big(f_1(\boldsymbol{z}_i^{(1)}), \dots, f_J(\boldsymbol{z}_i^{(J)})\big)^\top \in \mathbb{R}^J. \tag{24.7}$$

The collection of latent GPs forms an additive GP layer over subsets of the learnt features [e.g., 479]. Their outputs are mixed through $A \in \mathbb{R}^{C \times J}$, so the class logits are $A\boldsymbol{f}_i$. This mixing induces correlations between the outputs even when the latent GPs are independent. For a softmax observation model,

$$p(\boldsymbol{y}_i \mid \boldsymbol{f}_i, A) = \prod_{c=1}^{C} \left[\frac{\exp((A\boldsymbol{f}_i)_c)}{\sum_{c'=1}^{C} \exp((A\boldsymbol{f}_i)_{c'})} \right]^{y_{ic}}. \tag{24.8}$$

For GP j, let $H_j = \{\boldsymbol{z}_i^{(j)}\}_{i=1}^N$ denote its transformed training inputs, let Z_j contain m inducing inputs, let $\boldsymbol{f}_j$ collect its latent values on H_j, and let $\boldsymbol{u}_j$ collect its latent values on Z_j. The standard sparse-GP conditional [e.g.,

1354, 1615] is

$$p(\boldsymbol{f}_j \mid \boldsymbol{u}_j) = \mathcal{N}\Big(\boldsymbol{f}_j \mid K^{(j)}_{H_jZ_j}(K^{(j)}_{Z_jZ_j})^{-1}\boldsymbol{u}_j, \widetilde{K}_j\Big), \\ \widetilde{K}_j = K^{(j)}_{H_jH_j} - K^{(j)}_{H_jZ_j}(K^{(j)}_{Z_jZ_j})^{-1}K^{(j)}_{Z_jH_j}. \tag{24.9}$$

Under the local kernel-interpolation approximation of Wilson and Nickisch [1764],

$$K^{(j)}_{H_jZ_j} \approx M_j K^{(j)}_{Z_jZ_j}, \qquad K^{(j)}_{H_jH_j} \approx M_j K^{(j)}_{Z_jZ_j} M_j^\top, \tag{24.10}$$

where $M_j \in \mathbb{R}^{N\times m}$ is a sparse local-interpolation matrix. Substituting these approximations into Equation (24.9) sets the residual covariance to zero in the interpolated model and gives $\boldsymbol{f}_j \approx M_j\boldsymbol{u}_j$. This deterministic relation is an approximation introduced by interpolation, not an exact consequence of the original GP conditional.

Let $\boldsymbol{u} = (\boldsymbol{u}_1^\top, \dots, \boldsymbol{u}_J^\top)^\top$ and let $\boldsymbol{f}$ collect all latent GP values. SVDKL uses a factorised Gaussian variational distribution

$$q(\boldsymbol{u}) = \prod_{j=1}^{J} \mathcal{N}(\boldsymbol{u}_j \mid \boldsymbol{\mu}_j, \boldsymbol{S}_j). \tag{24.11}$$

Writing $\boldsymbol{y} = (\boldsymbol{y}_1^\top, \dots, \boldsymbol{y}_N^\top)^\top$, Jensen's inequality gives the evidence lower bound

$$\mathcal{L}(q;\gamma,A) := \mathbb{E}_{q(\boldsymbol{u})p(\boldsymbol{f}\mid\boldsymbol{u})}[\log p(\boldsymbol{y} \mid \boldsymbol{f}, A)] - \mathrm{D}_{\mathrm{KL}}(q(\boldsymbol{u})\|p(\boldsymbol{u})) \le \log p(\boldsymbol{y} \mid \gamma, A). \tag{24.12}$$

For fixed (γ, A), maximising Equation (24.12) over q within the chosen variational family is equivalent to minimising the KL divergence from $q(\boldsymbol{u})$ to the exact posterior over the inducing variables. When (γ, A) are optimised jointly with the variational parameters, the same bound is used both for posterior approximation and for point estimation of the model parameters.

Because the likelihood factorises over observations, a uniformly sampled mini-batch $\mathcal{B} \subseteq \{1, \dots, N\}$ of size b gives the unbiased ELBO estimator

$$\widehat{\mathcal{L}}_{\mathcal{B}} = \frac{N}{b}\sum_{i\in\mathcal{B}} \mathbb{E}_{q(\boldsymbol{f}_i)}[\log p(\boldsymbol{y}_i \mid \boldsymbol{f}_i, A)] - \mathrm{D}_{\mathrm{KL}}(q(\boldsymbol{u})\|p(\boldsymbol{u})). \tag{24.13}$$

Here $q(\boldsymbol{f}_i)$ is the marginal induced by $q(\boldsymbol{u})p(\boldsymbol{f} \mid \boldsymbol{u})$. Stochastic gradients of this estimator update the variational parameters $\{(\boldsymbol{\mu}_j, \boldsymbol{S}_j)\}_{j=1}^J$ together with w, the base-kernel hyperparameters, and A. The expected log-likelihood is generally estimated by Monte Carlo for non-Gaussian observation models. Thus, SVDKL performs variational lower-bound optimisation, not exact marginal-likelihood training.

24.4 Kernel Estimation

Compared with more deeply nested probabilistic constructions such as deep Gaussian processes [361], standard DKL usually presents a simpler estimation problem. Nevertheless, practical performance depends strongly on how its many point-estimated parameters are trained.

Three strategies are common:

1. pre-train the neural network to learn w, freeze it, and fit only the base-kernel hyperparameters θ;
2. use pre-training as a warm start and then fine-tune w and θ jointly;
3. learn $\gamma = (\theta, w)$ end-to-end from scratch.

The strategies differ in computational cost, susceptibility to overfitting, and reliance on a pre-training pipeline; none is uniformly best. In all three cases, standard DKL point-estimates w and θ, so its tractability should not be confused with a fully Bayesian treatment of uncertainty in the feature extractor.

A second choice is the training objective. For Gaussian-likelihood regression one can optimise the exact GP log marginal likelihood when dense computation is feasible, or an approximation to it when using scalable numerical methods. For non-Gaussian likelihoods and mini-batch training, SVDKL instead optimises the ELBO in Equation (24.12). The latter enables stochastic optimisation but introduces a variational approximation. Lotfi et al. [1081] propose the *conditional marginal likelihood*, which forms a posterior using one subset of the data and evaluates the marginal likelihood of the remainder; they report that this can mitigate marginal-likelihood underfitting in model selection.

Overfitting remains a legitimate concern. Ober et al. [1253] identify overfitting pathologies in marginal-likelihood-trained DKL and attribute them to excessive flexibility in the learned kernel. Wilson et al. [1771] dispute part of the accompanying explanation of the marginal-likelihood complexity term, arguing that it omits an additional data-fit dependence. These positions do not eliminate the practical need to check predictive performance and uncertainty on held-out data. A fully Bayesian treatment of the neural-network parameters can represent additional uncertainty but is substantially more expensive; standard regularisation such as weight decay is a more common practical safeguard.

When specialised base kernels such as the spectral-mixture kernel are used, tailored initialisation may be important [1763]. A sensible workflow is therefore to begin with a pre-trained and frozen feature extractor, fit the base kernel, and adopt joint fine-tuning, conditional marginal likelihood, or end-to-end training only when validation results justify the additional complexity.

24.5 DKL in Practice, Implementations, and Resources

Several mature frameworks support DKL. *GPyTorch* [580] integrates PyTorch neural-network modules with exact and approximate GP inference, GPU-accelerated linear algebra, and scalable variational methods. It is therefore well suited to prototyping DKL architectures and SVDKL variants, but the approximation and numerical assumptions of the selected GP method still apply.

A convenient starting point is the GPyTorch neural-network integration tutorial at `https://docs.gpytorch.ai/en/stable/examples/06_PyTorch_NN_Integration_DKL/index.html`. Before adding SKI or a variational approximation, it is often useful to test whether the default exact or iterative solvers are adequate for the dataset at hand. SVDKL provides a separate route to scale through inducing variables and mini-batch stochastic optimisation.

GPJax provides another deep-kernel tutorial at `https://docs.jaxgaussianprocesses.com/_examples/deep_kernels/`. For Bayesian optimisation, BoTorch [78] integrates with GPyTorch and supports DKL-based optimisation pipelines.

Chapter 25

Deep Gaussian processes

Bayesian deep learning strives to combine the expressive power of deep models with Bayesian uncertainty quantification. By placing distributions over the unknowns, we can propagate the uncertainty to predictions and embody Occam's Razor. We saw in Chapter 23 that Gaussian process (GP) models also offer a Bayesian treatment over the unknowns and their predictions naturally incorporate both data fit and epistemic uncertainty. Unlike deep neural networks, GPs operate directly in *function space*. By specifying distributions over functions rather than over parameters they provide flexible, non-parametric, probabilistic approaches to function estimation in an analytically tractable manner [1382].

Traditional Gaussian process models have been extended to more expressive variants, for example by considering sophisticated covariance functions [473, 645] or by embedding GPs in more complex probabilistic structures [480, 1763] able to learn more powerful representations of the data. Going further, the research community has sought approaches which combine the advantages of GPs with the ability of deep learning architectures to achieve hierarchical feature learning, such as deep kernel learning which was discussed in the previous chapter.

In this chapter, we discuss an alternative approach to combining GPs with deep feature learning: Deep Gaussian processes (deep GPs). Deep GPs are deep architectures which compose functions drawn from GP priors. In particular, here we will consider constructions where the output of one layer is the input to the next as follows:

$$\begin{aligned} \mathbf{f}^{1:L}(\mathbf{x}) &= \mathbf{f}^{L}(\mathbf{f}^{L-1}(\dots \mathbf{f}^{1}(\mathbf{x}))\dots), \\ \text{where: } & f^{\ell} \sim \mathcal{GP}\left(0, k_j^{\ell}(\cdot,\cdot)\right). \end{aligned} \tag{25.1}$$

Here, $\mathbf{f}$ denotes a vector-valued function and L is the total number of layers in the deep GP construction. Crucially, the overall process is *no longer Gaussian*, due to the nonlinear warping induced by each layer, and the model can learn much more complex interactions between data.

25.1 Composing functions and processes

25.1.1 Function composition for deep learning

Activation functions employed in neural networks are vector valued functions $\boldsymbol{\phi}(\cdot)$ of an adjustable basis, which are controlled by a parameter matrix $\mathbf{U}$. In a similar way to generalised linear models, the final activation $\mathbf{g}(\mathbf{x})$ for a single layer neural network and input $\mathbf{x}$ is obtained by linearly weighting the bases using a matrix $\mathbf{V}$:

$$\mathbf{g}(\mathbf{x}) = \mathbf{V}^T\boldsymbol{\phi}(\mathbf{U}\mathbf{x}). \tag{25.2}$$

Deep neural networks then take the form of a functional composition of the basis functions,

$$\mathbf{g}(\mathbf{x}) = (\mathbf{V}^L)^T\boldsymbol{\phi}^L(\mathbf{U}^L\mathbf{g}^{L-1}(\dots \mathbf{U}^2\mathbf{g}^1(\mathbf{x}))), \tag{25.3}$$

where L denotes the total number of layers and we represent the overall composition with the unindexed notation $\mathbf{g}(\cdot)$. We can alternatively re-write the above expression by using equation (25.2) to replace all $\mathbf{g}^{\ell}(\cdot)$ terms:

$$\mathbf{g}(\mathbf{x}) = (\mathbf{V}^L)^T\boldsymbol{\phi}^L(\mathbf{W}^L\boldsymbol{\phi}^{L-1}(\dots \mathbf{W}^2\boldsymbol{\phi}^1(\mathbf{U}^1\mathbf{x}))), \tag{25.4}$$

where $\mathbf{W}^{\ell}$ is the matrix mapping between each set of basis functions and is typically treated as a parameter to be learned. A serious challenge for deep networks, when trained in a feed-forward manner, is overfitting. As the

number of layers and the number of basis functions per layer increases, a very powerful but also highly parametrised representation is formed. The matrix $\mathbf{W}^\ell$ has size $d^\ell \times d^{\ell-1}$, where d^ℓ is the number of basis functions in the ℓth layer.

25.1.2 Weight matrix factorisation

One approach to dealing with the many parameters contained in the matrices $\mathbf{W}^\ell$, is to replace them with a lower rank form. This draws inspiration from the first function composition formulation shown in equation (25.3), but now the low rank decomposition is made explicit and clear. We select $r^\ell < d^\ell, d^{\ell-1}$ so as to obtain:

$$\mathbf{W}^\ell = \mathbf{U}^\ell (\mathbf{V}^{\ell-1})^\top,$$

where $\mathbf{U}^\ell \in \mathbb{R}^{d^\ell \times r^\ell}$ and $\mathbf{V}^{\ell-1} \in \mathbb{R}^{d^{\ell-1} \times r^\ell}$.

Denil et al. [411] have empirically shown that trained neural networks might be well approximated by low rank matrices as evidenced by the ability to predict one part of the weight matrix given another. The approach of "dropout" [1552] is also widely applied to control the complexity of the model implying the models are over parameterised. Substituting the low rank form into the compositional structure for the deep network we have

$$\mathbf{g}(\mathbf{x}) = (\mathbf{V}^L)^\top \boldsymbol{\phi}^L(\mathbf{U}^L(\mathbf{V}^{L-1})^\top \boldsymbol{\phi}^{L-1}(\dots \mathbf{U}^2(\mathbf{V}^1)^T \boldsymbol{\phi}^1(\mathbf{U}^1\mathbf{x}))).$$

We can now identify the following form inside the functional decomposition,

$$\mathbf{g}^\ell(\mathbf{z}) =^\top \mathbf{V}^\ell \boldsymbol{\phi}^\ell(\mathbf{U}^\ell \mathbf{z}) \tag{25.5}$$

and once again obtain a functional composition,

$$\mathbf{g}(\mathbf{x}) = \mathbf{f}^L(\mathbf{f}^{L-1}(\dots \mathbf{f}^1(\mathbf{x}))). \tag{25.6}$$

The standard deep network is recovered when we set $r^\ell = \min(d^\ell, d^{\ell-1})$.

25.1.3 Process composition with latent variables

To obtain a deep Gaussian process we consider a nested structure as shown above with equation (25.6). The deep Gaussian process is recovered by keeping r^ℓ finite and allowing $d^\ell \to \infty$ for all layers. Of course, the mappings in the Gaussian process are treated probabilistically and integrated out[1], rather than optimised, so despite the increase in layer size the parameters in the resulting model are many times fewer than those in a standard deep neural network. Further, Duvenaud et al. [481] additionally analyse the effect of dropout in the GP limit, showing it produces spike-and-slab lengthscale priors.

The latent variable construction above is not the only route to a deep GP. Alternative formulations include viewing the composition as a stochastic differential equation whose layers become continuous [728], representing the deep GP as a state-space model amenable to sequential filtering [1867], or describing it through composed covariance operators [463]. We discuss these perspectives further below.

The compositional idea also places deep GPs in a broader landscape of models. Warped GPs [1529] constitute a single-layer precursor, in which the output of a GP is transformed through a parametric monotonic function (which can also admit a GP prior [363, 986]), while normalising flows [1290, 1390] likewise construct flexible distributions by composing nonlinear transformations, although through deterministic, invertible parametric maps rather than draws from nonparametric function priors.

To generalise a Gaussian process to the deep domain, rather than assuming that a data observation, $\mathbf{y}$, is a draw from a noise corrupted Gaussian process prior, we instead make use of a functional composition,

$$\begin{aligned} \mathbf{y} &= \mathbf{f}^{1:L} + \boldsymbol{\epsilon} \\ &= \mathbf{f}^L(\mathbf{f}^{L-1}(\dots \mathbf{f}^1(\mathbf{x}))) + \boldsymbol{\epsilon}. \end{aligned} \tag{25.7}$$

This is the same form as for standard neural networks, shown in equation (25.6), but now each function in the composition, $\mathbf{f}^\ell(\cdot)$, is a draw from a Gaussian process. However, the overall prior $\mathbf{f}^{1:L}$ is no longer a Gaussian process. In the above equation, $\boldsymbol{\epsilon}$ represents the noise, and we can introduce Gaussian noise in every layer. In this

[1] If the mappings are instead fixed, then we obtain a standard (shallow) GP with a hierarchical covariance function, as noted and investigated by Duvenaud et al. [481].

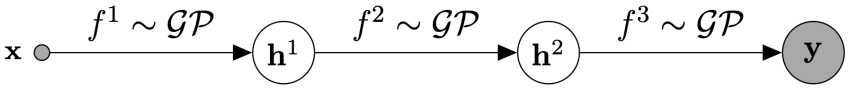


Figure 25.1: A deep Gaussian process with two hidden layers.

case, we can collect the noise corrupted function instantiations (corresponding to n data points) into a vector for each layer, so that we get the following recursive definition:

$$\mathbf{h}^\ell = \mathbf{f}^\ell(\mathbf{h}^{\ell-1}) + \boldsymbol{\epsilon}^\ell. \tag{25.8}$$

From this recursive definition, we obtain the whole hierarchical construction if we denote $\mathbf{x} \triangleq \mathbf{h}^0$ and consider $\mathbf{y}$ as the last layer. The corresponding graphical model for a deep Gaussian process is illustrated in Figure 25.1.

As can be seen, the generative procedure defined by the function $\mathbf{f}^{1:L}$ can be represented in a graphical model as a deep architecture where each layer corresponds to a generative procedure associated with a function $\mathbf{f}^\ell$.

Given equation (25.7), it is clear that this chapter develops a full probabilistic model through *process composition*. Process composition has the appealing property of retaining the theoretical qualities of the underlying stochastic process (such as Kolmogorov consistency) whilst providing a richer class of process priors. For example, for deep Gaussian processes, it has been shown that for particular assumptions of covariance function parameters, the derivatives of functions sampled from the process have a marginal distribution that is heavier tailed than a Gaussian [481]. In contrast, it is known that in a standard Gaussian process the derivatives of functions sampled from the process are jointly Gaussian with the original function. In the next section we compare the properties of deep Gaussian processes with the properties of traditional deep learning methods.

25.1.4 Inference challenges and expressiveness

We have so far focused on a deep GP construction with a hierarchy of intermediate latent variables. To highlight the inference challenges and expressiveness of the model, it is instructive to compare this construction to the generative deep learning approach that was dominant when deep GPs were first introduced: restricted Boltzmann machines (RBMs), stacked to construct deep belief networks. Deep belief networks have since been superseded as the workhorses of deep generative modelling, first by variational autoencoders [892, 1391] and more recently by diffusion models (Chapter 50), but the comparison remains instructive: like the deep GP, a deep belief network is a hierarchy of latent variables whose training requires marginalising the intermediate layers, and the contrast cleanly exposes what changes when each layer is a nonparametric function rather than a parametric mapping.

Various approximate inference techniques (such as contrastive divergence) are used for estimating model parameters in RBMs. A significant amount of work has then to be done with annealed importance sampling if even the *likelihood*[2] of a data set under the RBM model is to be estimated [1450]. When deeper hierarchies are considered, the estimate is only of a lower bound on the data likelihood. By considering sigmoidal activation functions, $\sigma(z) = (1 + \exp(-z))^{-1}$, we obtain the conditional probability of a single hidden unit in a deep belief network given its parents as

$$p(y|\mathbf{x}) = \sigma(\mathbf{U}\mathbf{x})^y(1 - \sigma(\mathbf{U}\mathbf{x}))^{(1-y)}.$$

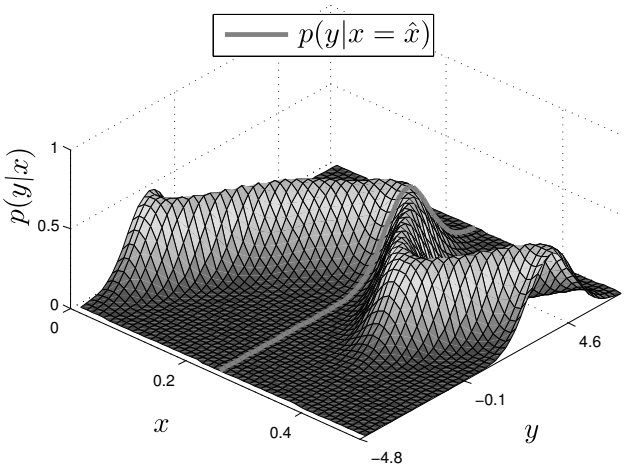
The conditional density of the output y depends only on a linear weighted sum of the inputs $\mathbf{x}$. The representational power of a Gaussian process in the same role is significantly greater than that of an RBM. For the GP the corresponding likelihood is over a continuous variable, but it is a nonlinear function of the inputs,

$$p(y|\mathbf{x}) = \mathcal{N}(y|f(\mathbf{x}), \beta^{-1}).$$

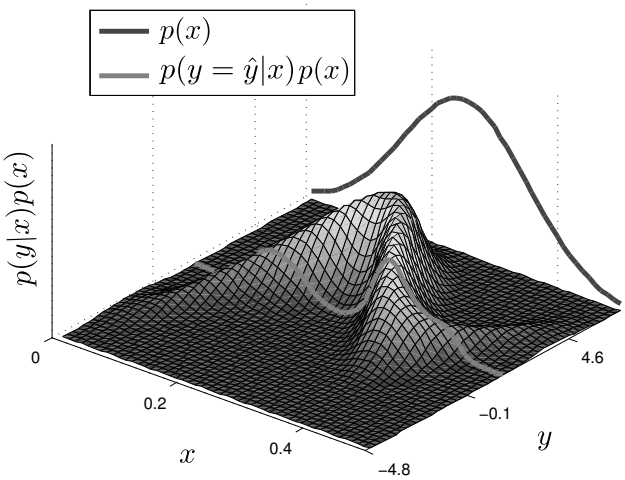
In this case the likelihood is dependent on a mapping function, $f(\cdot)$, rather than a set of intermediate parameters, $\mathbf{U}$. By placing a GP prior over f, we can analytically integrate out the mappings. In the RBM, instead, the model likelihood is estimated and maximised with respect to the parameters, $\mathbf{U}$. For the RBM, marginalising $\mathbf{U}$ is not analytically tractable. Note that the two approaches can be mixed if

$$p(y|\mathbf{x}) = \sigma(f(\mathbf{x}))^y(1 - \sigma(f(\mathbf{x})))^{(1-y)},$$

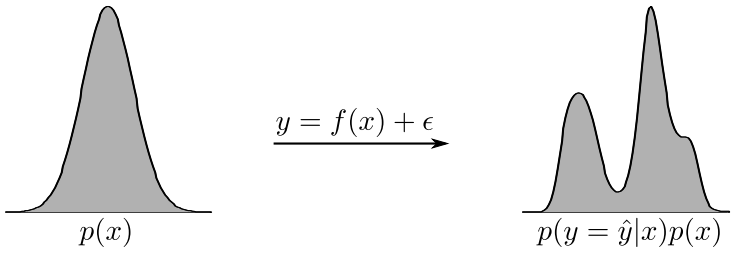
[2]Emphasis is used to clarify we are referring to the model likelihood, not the marginal likelihood required in Bayesian model selection.



(a) The likelihood plotted as a function of y and x. y is obtained through a non-linear function plus noise. The faint line corresponds to fixing x to some value $\hat{x}$.



(b) Unnormalised posterior plotted as a function of x and y. The faint line corresponds to a fixed y and the darker line shows $p(x)$.



(c) Propagating the Gaussian density $p(x)$ through a non-linear mapping.

Figure 25.2: A Gaussian distribution propagated through a non-linear mapping. Figures a and b show the likelihood and unnormalised posterior respectively. The posterior density (plotted for given data using a faint line in figure b and on the right in figure c) is multimodal and difficult to normalise.

which recovers a GP classification model. Analytic integration is no longer possible though, and a common approach to approximate inference is the expectation propagation algorithm [see e.g. 1382]. Expectation propagation has also been applied to deep GPs [217]; we discuss this and other inference approaches in Section 25.4. Here we focus on the variational approximation.

Inference in deep models requires marginalisation of the stacked input spaces, denoted as $\{\mathbf{h}^\ell\}_{\ell=1}^L$ in equation (25.8). These *intermediate* spaces are typically treated as *latent* variables[3], which in the case of the RBM are binary variables. The number of the terms in the sum scales exponentially with the input dimension rendering it intractable for anything but the smallest models. In practice, sampling and, in particular, the contrastive divergence algorithm, are used for training. Similarly, marginalising the intermediate, latent spaces in the deep GP is analytically intractable: despite the simplicity of the Gaussian density its propagation through non-linear mappings is challenging. This is illustrated in Figure 25.2.

For this reason, the hierarchical GP-LVM [985], which constituted the first approach to stacking GPs, was defined within a maximum a posteriori (MAP) framework where the latent variables were maximised rather than marginalised out. For this MAP approach to work, however, a strong prior is required on the top level of the hierarchy. Further, MAP learning prohibits model selection because no estimate of the marginal likelihood is available.

Most deep algorithms require a large amount of data to perform learning. Fitting such models to smaller data sets and using Bayesian approaches to deal with the complexity seems completely futile when faced with the associated computational intractabilities. However, we know that humans are able to perform inductive reasoning (equivalent to concept generalisation) with only a few examples [1594]. This provokes the question as to whether deep structures and the learning of abstract structure can be undertaken in *smaller* data sets. For smaller data

[3]Some of the variables can also be treated as observed, e.g. in the upper most layer of the hierarchy where we might include the data label or inputs for regression.

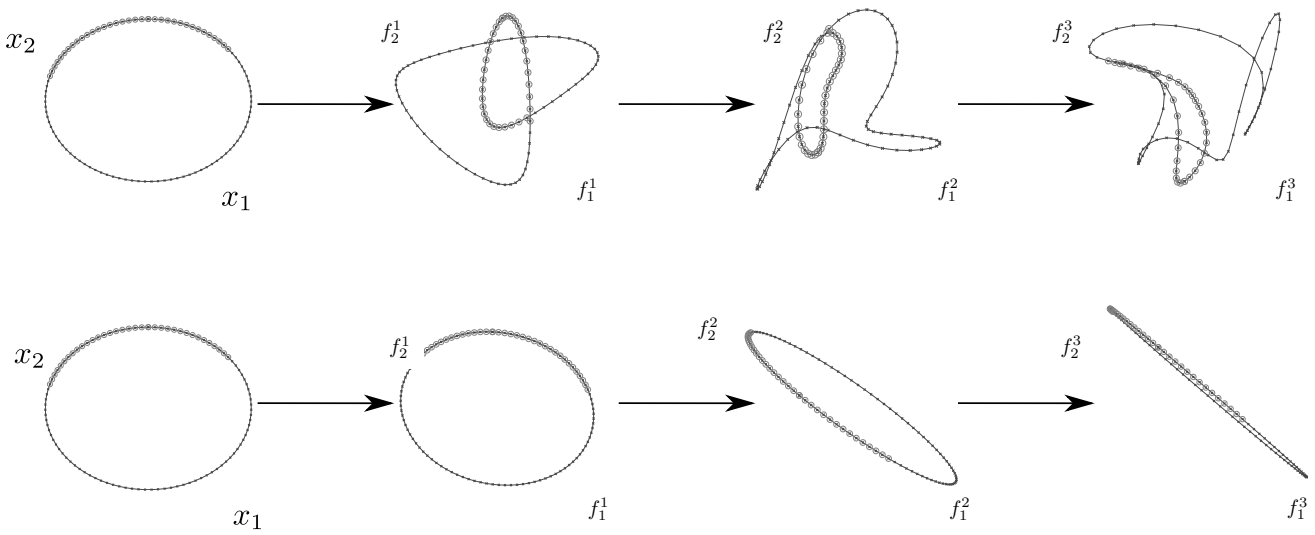


Figure 25.3: Samples from a deep GP showing the generation of features. The upper plot shows the successive non-linear warping of a two-dimensional input space. The circles correspond to specific locations in the input space for which a feature (a "loop") is created in layer 1. As can be seen, as we traverse the hierarchy towards the right, this feature is maintained in the next layers and is potentially further transformed. The bottom plot is similar as the upper plot, but a linear covariance function is used (equivalent to performing stacked principal component analysis (PCA) for training given observed data). As can be seen, linear transformations seem nonsensical in this setting, since multiplying an input by two matrices (two linear transformations) is equivalent to multiplying the input with a single combined matrix (one linear transformation).

sets, questions of generalisation arise: to demonstrate such structures are justified it is useful to have an objective measure of the model's applicability. This chapter demonstrates the ability of deep Gaussian processes to learn even from small training sets. This property can be attributed to their fully Bayesian nature.

25.1.5 Characteristics of deep GPs as priors

Deep GPs compose nonlinear mappings to yield much richer prior function classes compared to single layer GPs. To start with, a single-layer GP defines a distribution over functions which is Gaussian: any finite collection of function values has a joint multivariate Gaussian distribution. This Gaussianity imposes certain limitations on expressiveness. For instance, function derivatives in a GP are also Gaussian distributed which means that very sharp changes or large jumps in the function space are a priori unlikely under a GP. On the other hand, the output of a deep GP is not Gaussian distributed in general; even if each layer is a GP, the marginal distribution $p(y|x)$ as modelled by $y = \mathcal{DGP}(x)$, where $\mathcal{DGP}$ denotes the composite deep GP mapping, is an integral of Gaussian distributions over random inputs, which can produce heavy tails, skewness or multimodality. More generally, as the number of layers increases, the distribution of outputs given an input can develop increasingly non-trivial shapes, as discussed by Duvenaud et al. [481] who also elaborate on caveats related to this characteristic.

Furthermore, as a function prior, a deep GP can induce non-stationarity and input warping. By composing multiple GPs, the effective covariance function of the input-output mapping becomes input-dependent. For instance, a single GP with a stationary kernel (such as the exponentiated quadratic) is limited in that it has a single global lengthscale of correlation. A deep GP can *learn* location-varying lengthscales: the first GP layer might warp the input x into a new space h^1, stretching or contracting regions differently. The second layer then operates on h^1 and so on. The result is that the final function can exhibit non-stationary behaviour, even if each layer used stationary kernels. For instance, one part of the input space might effectively be magnified (leading to fine-scale, rapidly varying output), while another part is shrunk (output varies more slowly), because the latent h^1 learned that certain inputs require more detail. This is illustrated in Figures 25.3, 25.4 and 25.7.

Another view is to consider what covariance function a deep GP implies for the direct input-output relationship. In a few cases, for example simple linear compositions or analogies to infinite-width neural networks, one can derive an equivalent covariance function for the composed function analytically. Classic results by Neal [1227] and Williams [1759] showed that a single-layer Bayesian neural network with an infinite number of hidden units converges to a GP with a closed-form covariance function for certain activation functions. However, for a general

deep GP with nonlinear covariance at each layer, there is generally no closed-form kernel for the overall composed mapping.

An important line of work has extended the single-layer GP limit to deep networks. Dutordoir et al. [477] showed that a fully-connected Bayesian neural network of fixed depth, with i.i.d. weight priors converges to a single GP as layer widths are taken to infinity sequentially. The limiting kernel $\mathbf{K}^\ell$ is obtained by a deterministic recursion: each layer applies a map to the previous layer's kernel, determined by the activation function and the weight and bias variances. de G. Matthews et al. [386] made this result rigorous for simultaneous width growth across all layers, resolving the key technical obstacle that the standard central limit theorem no longer applies in deep architectures (since the required independence assumptions fail) by reformulating the layerwise sums as exchangeable sequences and invoking a central limit theorem for exchangeable arrays. Crucially, in both cases the limit is a single GP, not a deep GP: depth enters only through the kernel recursion, and the hierarchy collapses into a fixed, deterministic kernel. This limiting object is sometimes called the neural network GP (NNGP) and is best understood as a *deep kernel GP* rather than a deep GP in the sense of Damianou and Lawrence [361]. The relationship to deep kernel learning, where a neural network is used to parameterise the kernel of a (single-layer) GP, is discussed in Chapter 24.

Agrawal et al. [9] resolve the apparent disconnect between these GP limits and deep GPs. They unify the NNGP and deep GP perspectives by showing that when some hidden layers are kept at finite width (bottlenecks) while others tend to infinity, the network converges to a composition of Gaussian processes, i.e. a deep Gaussian process. These bottleneck layers introduce shared latent representations that induce dependence between outputs and preserve discriminative structure through depth. In contrast, when all layers are taken to infinite width, the model reduces to a single Gaussian process. Conversely, Pleiss and Cunningham [1330] demonstrate that width and depth have opposite effects in hierarchical models: while depth accentuates non-Gaussianity, increasing the output dimensionality of the intermediate layers causes a deep GP to converge toward a standard GP. Because deep GPs are already nonparametric, this collapse cannot be attributed to gaining capacity; it is a structural consequence of width itself, reinforcing the importance of narrow bottleneck layers for preserving hierarchical structure. Since Bayesian neural networks constitute a subclass of deep GPs with degenerate, finite-capacity layers, these structural findings extend to standard neural network architectures as well.

Connecting in the reverse direction, Dutordoir et al. [477] construct deep sparse GP models whose layerwise mean propagation matches the forward pass of a standard neural network through interdomain inducing-feature constructions. This provides a practical bridge: neural networks can serve as point estimates for deep GP inference, and a trained neural network can be used to initialise a deep GP.

The covariance-function perspective connects to a broader class of constructions of deep Gaussian processes, including formulations based on covariance operators and convolution [463]. Dunlop et al. [463] place these approaches within a unified framework in which the sequence of functions across layers forms a Markov chain, and analyse its ergodicity properties to characterise an effective depth: in ergodic settings, the process admits a stationary distribution and additional layers yield little further benefit beyond the mixing time, whereas in non-ergodic settings the behaviour can remain non-trivial or unstable as depth increases. From a statistical perspective, Finocchio and Schmidt-Hieber [519] complement this view by showing that deep Gaussian process priors can achieve (near-)minimax posterior contraction rates for regression functions with compositional structure, demonstrating that depth can be advantageous in such settings.

A practical concern related to depth is the pathology identified by Duvenaud et al. [481], whereby the prior's expressive degrees of freedom can collapse as the number of layers grows. In the context of variational inference, this can manifest as layer collapse: mean-field approximations may lead to intermediate layers being effectively 'turned off', with their contribution diminishing as the signal-to-noise ratio vanishes [1453]. This pathology is not intrinsic, however: as we discuss in the following section, architectural modifications at the bottleneck layers can alleviate it.

25.1.6 Modelling bottleneck layers

The preceding section established that bottleneck layers (narrow intermediate layers) are the load-bearing structural element of a deep GP. We now turn to the complementary question how these bottleneck layers can be modelled. We first discuss architectural choices aimed at preserving the contribution of intermediate layers, directly addressing the pathology just mentioned, and then turn to a deeper modelling axis: whether the bottleneck is represented as an explicit latent variable in the generative model, or left implicit in the composition.

To mitigate the pathology noted by Duvenaud et al. [481] and discussed in the previous section, the authors propose architectures that connect the input directly to each layer, and Salimbeni and Deisenroth [1453] introduce a closely related modification in the deep GP setting by equipping each layer with a linear mean function $m(\mathbf{x}) = \mathbf{W}\mathbf{x}$.

This acts as a residual connection: even if a layer's stochastic component contributes little, the input can still propagate through the linear mean. This modification has been shown to alleviate the degeneracies identified by Duvenaud et al. [481] and is consistent with the analysis of deep GP priors in Salimbeni and Deisenroth [1453]. More recently, de Souza et al. [388] propose *thin and deep* architectures that combine low-dimensional latent representations with interpretable kernel parametrisations at each layer, addressing these pathologies through architectural design while preserving the benefits of depth.

Beyond architectural modifications, the bottleneck layer itself admits two fundamentally different modelling treatments, distinguished by whether it is represented as an explicit latent variable in the generative model or only implicitly via the composition. In the explicit-latent view, followed by Damianou and Lawrence [361], each bottleneck is a random variable $\mathbf{h}^\ell = f^\ell(\mathbf{h}^{\ell-1}) + \boldsymbol{\epsilon}^\ell$, distinct from the GP function values, with Gaussian noise $\boldsymbol{\epsilon}^\ell$ injected between every pair of layers. Each layer then has the structural form of a GP-LVM, and inference includes estimating $p(\{\mathbf{h}^\ell\}_{\ell=1}^L|\text{Data})$, the posterior over the intermediate bottlenecks. In the implicit-latent view, followed by Salimbeni and Deisenroth [1453], the inter-layer noise is absorbed into the covariance function and there is no separately named bottleneck variable: the state at layer ℓ is the GP function value $\mathbf{f}^\ell$ itself, and the key interest is the induced marginal process $y(x) = \mathcal{DGP}(x)$, with intermediate $\mathbf{f}^\ell$ regarded as instrumental rather than as objects of inference.

This distinction looks small, as it just amounts to whether noise is injected between layers, but its consequences are substantial. The explicit-latent view makes each layer a GP-LVM and so it places the model squarely in the lineage of representation learning: the inferred intermediate bottlenecks are analogues of the embeddings obtained from intermediate layers of deep networks, and the generative interpretation readily accommodates supervised, unsupervised, and semi-supervised [362] learning, as well as the modelling of high-dimensional data. The implicit-latent view instead emphasises the induced function $y(x) = \mathcal{DGP}(x)$ and its properties rather than the learned inner representations, making function learning the natural framing; this reorientation has also motivated different, more scalable inference schemes for the supervised learning scenario. Finally, the implicit-latent view is sometimes taken to include nested GP models, where a GP prior is placed on the parameters of a GP as in Zhao et al. [1867].

25.1.7 Outline for the rest of this chapter

After establishing these alternative modelling choices for deep GP bottleneck layers, we proceed in this chapter by diving deeper into two families of approaches which are particularly instructive. Firstly, we discuss the explicit-latent view; this has been the initial introduction of deep GPs with variational Bayesian inference by Damianou and Lawrence [361] and constitutes an intuitive description of the generative model and its relation to BDL. This view also has been amenable to a variety of inference schemes, including variational inference which has been discussed in this book. Secondly, we discuss approaches such as Salimbeni and Deisenroth [1453] where estimation of the distribution of intermediate latents is not the key focus, further motivating a relaxation for the requirement that inner expectations be evaluated analytically. In turn, this gives rise to a Monte-Carlo approximation of the predictive (approximate) posterior which unlocks both scalability and stability.

Overall, by elaborating on the above two approaches, their different points of focus and how their associated variational inference schemes deeply relate to each other, we hope that the reader will get a rounded understanding of deep GPs, also facilitated by the previous discussion of different deep GP constructions.

We will give a detailed explanation of the VI-based inference schemes used in these approaches to expose the readers to the precise approximations made; this is important since the central technical challenge of deep GPs is inference. Indeed, we have established deep GPs are BDL models operating in function space directly, giving us the combined benefit of: (a) function space inference (b) fully Bayesian treatment and (c) non-linear mappings; however exactly this induced requirement of propagating densities through non-linearities means that we can no longer apply the analytically tractable methodologies of shallow GPs, nor can we employ directly deep learning optimisation techniques which work in the parameter space (they are not required to propagate densities). The intra-layer as well as inter-layer intractabilities mean that standard variational inference also fails. Consequently, practical trade-offs and challenges for deep GP constructions and usage depend critically on inference. We will complement the thorough discussion of the VI schemes by additionally discussing other inference schemes besides variational inference. Finally, we will close this chapter with an illustrative example.

25.2 Deep Gaussian processes with latent variables

This section describes the deep GP framework in more detail, starting with the probabilistic definition and then developing the variational inference methodology used for training.

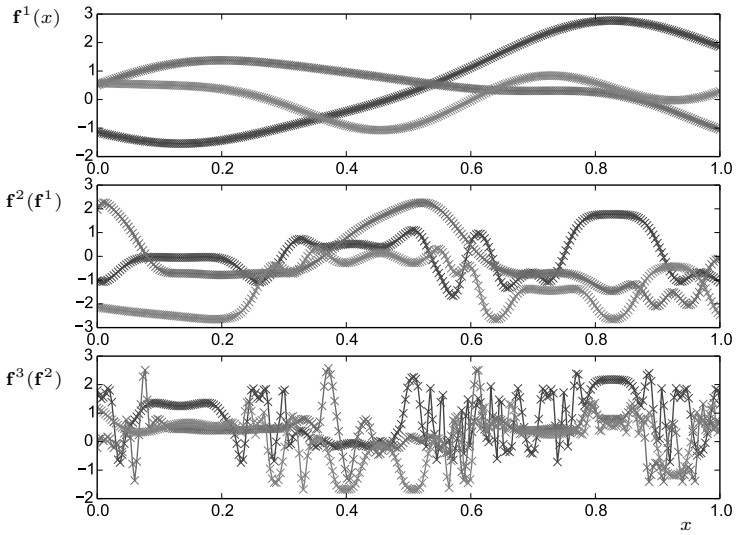


Figure 25.4: Samples from a deep Gaussian process with three-dimensional hidden and output nodes. A linear input x (not depicted) was propagated through two hidden layers (top and middle plot) to the output layer (bottom plot).

25.2.1 Probabilistic definition

The deep Gaussian process class of probabilistic models is now formally described. To do so, single-dimensional variables will again be considered, although it is underlined that this is only for simplicity and without loss of generality for models with conditionally independent output dimensions. Further, the analysis is made by firstly considering the unsupervised learning scenario, where a hierarchy of latent spaces is assumed to generate observed outputs.

The deep GP consists of a cascade of L hidden layers of latent variables, $\{\mathbf{h}^\ell\}_{\ell=1}^L$ with the observed outputs being placed in the leaves of the hierarchy. An intermediate node $\mathbf{h}^\ell \in \mathbb{R}^n$ associated with layer ℓ constitutes the output of that layer and acts as an input for the generative procedure of the subsequent layer, $\ell + 1$. Gaussian processes govern the mappings between the layers. Each mapping corresponds to a separate GP, having covariance function k^ℓ and hyperparameters $\boldsymbol{\theta}^\ell$. Further, every set of latent variables $\mathbf{h}^\ell$ is obtained from the functional output of the previous layer with the addition of Gaussian noise $\boldsymbol{\epsilon}^\ell \sim \mathcal{N}(0, (\beta^\ell)^{-1}\mathbf{I})$. Therefore, a single layer of the deep GP is effectively a GP-LVM, just as a single layer of a deep belief network is an RBM. The joint distribution of a deep GP model with L hidden layers can be written as:

$$p(\mathbf{y}, \{\mathbf{h}^\ell\}_{\ell=1}^L) = p(\mathbf{y}|\mathbf{h}^L)p(\mathbf{h}^L|\mathbf{h}^{L-1}) \cdots p(\mathbf{h}^2|\mathbf{h}^1)p(\mathbf{h}^1), \tag{25.9}$$

where $p(\mathbf{h}^1) = \mathcal{N}(\mathbf{h}^1|\mathbf{0}, \mathbf{I})$. As for the conditional probabilities, they can be expanded as:

$$p(\mathbf{h}^\ell|\mathbf{h}^{\ell-1}) = \int p(\mathbf{h}^\ell|\mathbf{f}^\ell)p(\mathbf{f}^\ell|\mathbf{h}^{\ell-1})\mathrm{d}\mathbf{f}^\ell, \tag{25.10}$$

where inside the integral we have the likelihood $p(\mathbf{h}^\ell|\mathbf{f}^\ell) \sim \mathcal{N}(\mathbf{f}^\ell, (\beta^\ell)^{-1}\mathbf{I})$ and the conditional $p(\mathbf{f}^\ell|\mathbf{h}^{\ell-1}) \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_{f^{\ell-1}f^{\ell-1}})$. Notice that the covariance matrix for each layer is evaluated with the corresponding covariance function evaluated at the inputs of the previous layer, that is, $\mathbf{K}_{f^{\ell-1}f^{\ell-1}} = k^\ell(\mathbf{h}^{\ell-1}, \mathbf{h}^{\ell-1})$. The observations are included in this recursive formulation by considering the special notation $\mathbf{h}^{L+1} \triangleq \mathbf{y}$.

Figure 25.4 shows examples of samples drawn from the deep GP architecture, where the top layer was chosen deterministically. This figure makes evident that deep GPs constitute much more powerful priors compared to traditional GPs (observe, for example, the long range correlations in the samples). However, although sampling from a deep GP is easy, training it is challenging. MAP optimisation for $\{\mathbf{h}^\ell\}_{\ell=1}^L$ turns all hidden nodes into extra model parameters, creating a severe regularisation problem [985]. Here we instead focus on marginalising out all hidden layers in a Bayesian paradigm, which drastically reduces the number of parameters (each layer mainly contributes variational rather than model parameters) and enables automatic structure discovery through the model evidence and automatic relevance determination (ARD) [364].

However, the nonlinearities introduced by the GP covariance functions make the Bayesian treatment of this model challenging. To demonstrate this, consider a deep GP with two hidden layers. Then, the marginal likelihood

can be written as:

$$p(\mathbf{y}) = \int p(\mathbf{y}|\mathbf{h}^2) \left(\int p(\mathbf{h}^2|\mathbf{h}^1) p(\mathbf{h}^1) \mathrm{d}\mathbf{h}^1 \right) \mathrm{d}\mathbf{h}^2. \tag{25.11}$$

The nested integral can be written according to equation (25.10) as:

$$p(\mathbf{h}^2) = \int p(\mathbf{h}^2|\mathbf{f}^2) p(\mathbf{f}^2|\mathbf{h}^1) p(\mathbf{h}^1) \mathrm{d}\mathbf{h}^1 \mathrm{d}\mathbf{f}^2 = \left\langle p(\mathbf{h}^2|\mathbf{h}^1) \right\rangle_{p(\mathbf{h}^1)} \tag{25.12}$$

where $\langle \cdot \rangle_p$ denotes expectation with respect to p; the above integral is intractable, because the density $p(\mathbf{h}^1)$ cannot be propagated through the function $p(\mathbf{f}^2|\mathbf{h}^1)$ which is constructed by a nonlinear covariance function $k^2(\mathbf{h}^1, \mathbf{h}^1)$. A similar situation arises in every layer when deeper architectures are employed. That is, if there was an additional layer we would have to compute the expectation $\left\langle p(\mathbf{h}^3|\mathbf{h}^2) \right\rangle_{\tilde{p}(\mathbf{h}^2)}$, where $\tilde{p}(\mathbf{h}^2) = \left\langle p(\mathbf{h}^2|\mathbf{h}^1) \right\rangle_{p(\mathbf{h}^1)}$ is coming from equation (25.12). We can, thus, formalise the problem by writing the recursive definition

$$\tilde{p}(\mathbf{h}^\ell) = \left\langle p(\mathbf{h}^\ell|\mathbf{h}^{\ell-1}) \right\rangle_{\tilde{p}(\mathbf{h}^{\ell-1})}$$

of the densities required for the calculation of the marginal likelihood and observing that all required expectations are intractable, as demonstrated for the two hidden layer architecture above. In the next section we develop variational methodologies which solve this intractability.

25.2.2 Variational inference

Inducing variables and sparse GP conditionals. Here we will demonstrate the variational framework employed by Damianou and Lawrence [361] to introduce tractable deep GPs. We also point the reader's attention to Chapter 12 and the chapters that follow it for more information on variational inference. First, following the variational sparse GP framework [1615], we augment the probability space of every layer with m inducing function values $\mathbf{u}^\ell$ evaluated at pseudo-inputs $\mathbf{x}_u^\ell$ [1354] (Figure 25.5), treated in depth in Chapter 38. The inducing inputs will be omitted from our expressions, since they always appear in the conditioning set of any probability involving $\mathbf{u}$. For layers with latent inputs ($\ell \geq 2$ in the unsupervised formulation), the inducing inputs $\mathbf{x_u}^\ell$ must be placed in the same latent space as $\mathbf{h}^{\ell-1}$, whose geometry is itself being learned; both are optimised jointly, and poor coordination between them can degrade the sparse approximation.

Given these inducing variables, the standard sparse GP decomposition [1354] gives the conditional of the latent function values at layer ℓ:

$$p(\mathbf{f}^\ell|\mathbf{u}^\ell, \mathbf{h}^{\ell-1}) = \mathcal{N}(\mathbf{f}^\ell|\mathbf{a}^\ell, \widetilde{\mathbf{K}}^\ell), \tag{25.13}$$

where:

$$\begin{aligned} \mathbf{a}^\ell &= \mathbf{K}_{f^{\ell-1}u^{\ell-1}} \mathbf{K}^{-1}_{u^{\ell-1}u^{\ell-1}} \mathbf{u}^\ell \\ \widetilde{\mathbf{K}}^\ell &= \mathbf{K}_{f^{\ell-1}f^{\ell-1}} - \mathbf{K}_{f^{\ell-1}u^{\ell-1}} \mathbf{K}^{-1}_{u^{\ell-1}u^{\ell-1}} \mathbf{K}_{u^{\ell-1}f^{\ell-1}}. \end{aligned} \tag{25.14}$$

Variational compression within each layer. Even with the inducing point expansion, we cannot propagate a density of $\mathbf{h}^{\ell-1}$ through the nonlinear $p(\mathbf{f}^\ell|\mathbf{u}^\ell, \mathbf{h}^{\ell-1})$. Following Titsias [1615], Titsias and Lawrence [1616], we set $q(\mathbf{f}^\ell|\mathbf{u}^\ell, \mathbf{h}^{\ell-1}) = p(\mathbf{f}^\ell|\mathbf{u}^\ell, \mathbf{h}^{\ell-1})$, which yields a tractable bound; this bound becomes tight in the limit where u^ℓ is a sufficient statistic for f^ℓ [360, 361]. Applying Jensen's inequality within each layer then yields a per-layer contribution to the bound:

$$\mathcal{L}^\ell = \log \mathcal{N}(\mathbf{h}^\ell|\mathbf{a}^\ell, (\beta^\ell)^{-1}\mathbf{I}) - \frac{\beta^\ell}{2} \mathrm{tr} \widetilde{\mathbf{K}}^\ell. \tag{25.15}$$

Equation (25.15) lower bounds $\log p(\mathbf{h}^\ell|\mathbf{u}^\ell, \mathbf{h}^{\ell-1})$: the Gaussian uses only the noise covariance $(\beta^\ell)^{-1}\mathbf{I}$ in place of the full $\widetilde{\mathbf{K}}^\ell + (\beta^\ell)^{-1}\mathbf{I}$, while the trace term penalises the residual GP variance not captured by the inducing variables [1615]. Conditioned on $\mathbf{u}$, this per-layer bound factorises across data points.

Integrating out hidden variables and inducing outputs. It remains to marginalise out the hidden variables $\{\mathbf{h}^\ell\}$ *across layers* and the inducing function values $\{\mathbf{u}^\ell\}$. To do so, we introduce a variational distribution

$$\mathcal{Q} = \prod_{\ell=1}^{L} q(\mathbf{u}^{\ell+1}) q(\mathbf{h}^\ell). \tag{25.16}$$

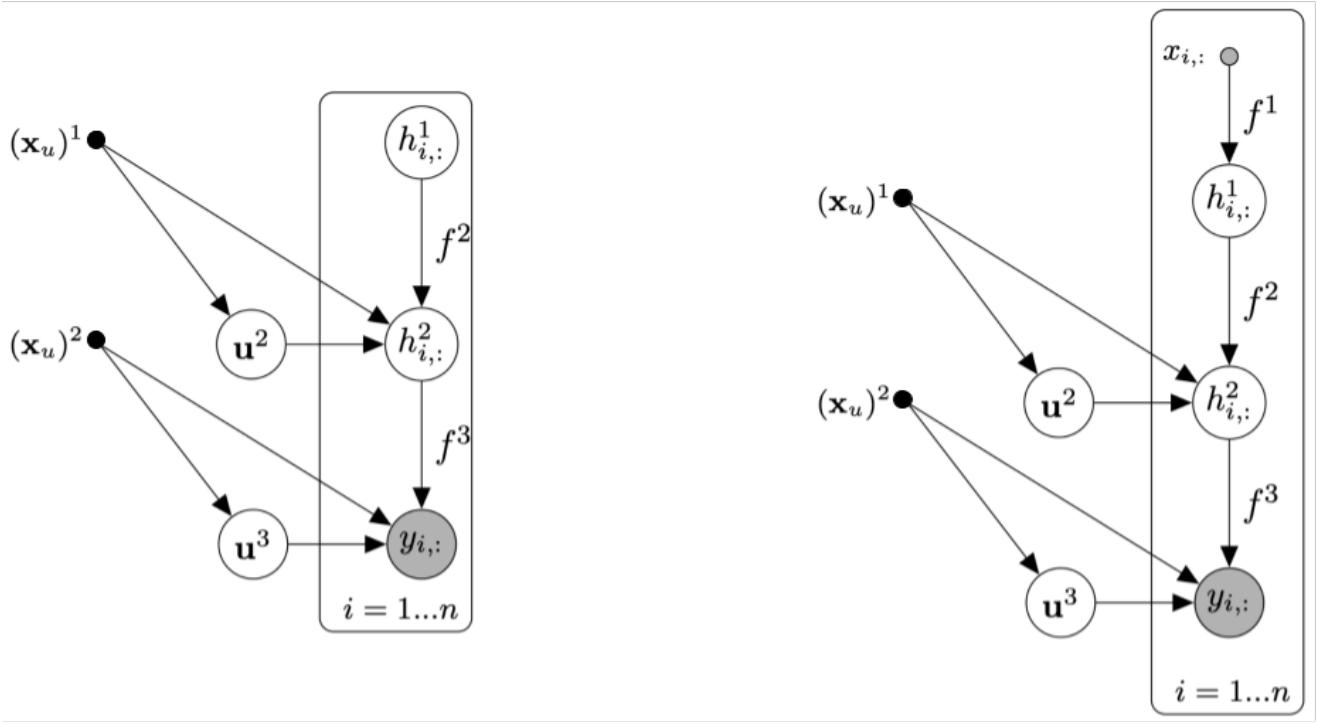


Figure 25.5: A graphical model representation of: the unsupervised (left) and the supervised (right) deep GP structure with inducing variables and $L = 2$ hidden layers. Nodes arranged horizontally form part of the same layer.

where both $q(\mathbf{h}^\ell)$ and $q(\mathbf{u}^\ell)$ are chosen to be Gaussian, with $q(\mathbf{h}^\ell)$ factorised across data points (diagonal covariance per data point) and $q(\mathbf{u}^\ell)$ having full covariance. Notice that factorising $q(\{\mathbf{h}^\ell\})$ across layers is a (sub-optimal) mean-field approximation.

Applying Jensen's inequality with this $\mathcal{Q}$ and substituting the per-layer bounds from equation (25.15), we obtain the variational lower bound $\mathcal{F} \leq \log p(\mathbf{y})$ (see Damianou [360], Damianou and Lawrence [361] for the full derivation):

$$\begin{aligned}\mathcal{F}(q\{\mathbf{H}^\ell\}, q\{\mathbf{U}^\ell\}) = \sum_{\ell=2}^{L+1} \left(\left\langle \log \mathcal{N}(\mathbf{h}^\ell | \mathbf{a}^\ell, (\beta^\ell)^{-1}\mathbf{I}) \right\rangle_{q(\mathbf{h}^{\ell-1})q(\mathbf{h}^\ell)q(\mathbf{u}^\ell)} - \frac{\beta^\ell}{2} \left\langle \mathrm{tr}\widetilde{\mathbf{K}}^\ell \right\rangle_{q(\mathbf{h}^{\ell-1})} \right) \\ - \mathrm{D}_{\mathrm{KL}}\big(q(\mathbf{h}^1) \| p(\mathbf{h}^1)\big) - \sum_{\ell=2}^{L+1} \mathrm{D}_{\mathrm{KL}}\big(q(\mathbf{u}^\ell) \| p(\mathbf{u}^\ell)\big) + \sum_{\ell=2}^{L} \mathcal{H}q(\mathbf{h}^\ell),\end{aligned} \tag{25.17}$$

where $\mathbf{h}^{L+1} \triangleq \mathbf{y}$ and $\mathcal{H}\cdot$ denotes the entropy. We note that within the ELBO derived using $\mathcal{Q}' = \prod_\ell p(\mathbf{f}^\ell | \mathbf{u}^\ell, \mathbf{h}^{\ell-1})\, q(\mathbf{u}^\ell)\, q(\mathbf{h}^\ell)$, the per-layer term $\mathcal{L}^\ell$ arises as an exact computation — the $p(\mathbf{f}^\ell | \mathbf{u}^\ell, \mathbf{h}^{\ell-1})$ terms cancel between model and variational distribution — and the looseness relative to $\log p(\mathbf{h}^\ell | \mathbf{u}^\ell, \mathbf{h}^{\ell-1})$ is a consequence of using the prior conditional $p(\mathbf{f}^\ell | \mathbf{u}^\ell, \mathbf{h}^{\ell-1})$ rather than the posterior $p(\mathbf{f}^\ell | \mathbf{u}^\ell, \mathbf{h}^{\ell-1}, \mathbf{h}^\ell)$, a choice made for tractability.

Hence, all terms are tractable: the KL and entropy terms are standard Gaussian quantities, while the expectations involving covariance matrices require that the chosen kernel can be feasibly convolved with Gaussian densities, restricting the choice to kernels such as the exponentiated quadratic and the linear kernel. The bound factorises across data points, and is maximised with respect to model parameters $\{\beta^\ell, \boldsymbol{\theta}_f^\ell\}$ and variational parameters $\{\mathbf{x}_u^\ell, \mathbf{m}^\ell, \mathrm{diag}(\mathbf{S}_i^\ell), \boldsymbol{\mu}^{\ell+1}, \boldsymbol{\Sigma}^{\ell+1}\}$. Notice that the variational parameters for $q(\mathbf{h}^\ell)$ scale as $\mathcal{O}(n)$ per layer.

Collapsed bound. Alternatively, one can determine the optimal $q(\mathbf{u}^\ell)$ analytically and substitute it back, yielding a *collapsed* bound that depends only on $q(\{\mathbf{H}^\ell\})$ [360]. This gives a tighter approximation, but the marginalisation of $q(\mathbf{u}^\ell)$ re-introduces coupling across data points: the collapsed objective no longer factorises in n, preventing minibatch training. For smaller datasets (where processing all data points jointly is feasible)

the collapsed bound may be preferred for its tightness; for larger datasets, the uncollapsed, factorised bound is necessary, and stochastic variational inference methods become essential (Section 25.3).

25.2.3 Supervised learning with noisy bottleneck layers

So far, we demonstrated the variational framework for the general scenario where all inputs are associated with a density, such as in an unsupervised learning scenario. However, in typical supervised learning applications, the top layer's inputs are observed and deterministic. Therefore, inference for that layer is simpler, as there is no intractable density propagation issue; we explicitly model correlations in the input space through the approximate posterior $q(\mathbf{h}^1)$ which, in this case, constitutes a variational distribution q which is a Gaussian with *full* covariance matrix (rather than a diagonal one). Writing the general multi-dimensional form, we let $\mathbf{h}_j^\ell$ denote the j-th output dimension across all n datapoints (a column of $\mathbf{H}^\ell$) and $\mathbf{h}_{i,:}^\ell$ the i-th datapoint across all dimensions (a row of $\mathbf{H}^\ell$). The latent space distribution would then factorise as:

$$q(\{\mathbf{H}^\ell\}) = \prod_{j=1}^{d^1} \mathcal{N}(\mathbf{h}_j^1 | \mathbf{m}_j^1, \mathbf{S}_j^1) \prod_{\ell=2}^{L} \prod_{i=1}^{n} \mathcal{N}(\mathbf{h}_{i,:}^\ell | \mathbf{m}_{i,:}^\ell, \mathbf{S}_i^\ell), \tag{25.18}$$

where $\mathbf{S}_j^1$ is a *full* $n \times n$ matrix, as in Damianou et al. [364].

25.3 Scalable deep GPs

In section 25.2.2 we discussed two versions of the variational lower bound: collapsed and uncollapsed. For smaller training sets the collapsing approach might be preferred, because of the tighter bound and the reduced size of the parameters. For larger training sets, however, this approach becomes infeasible since all datapoints need to be considered together during the optimisation of the parameters. To tackle this issue, Dai et al. [357], Gal et al. [567] have devised parallel training regimes, while Dai et al. [358] takes this one step further by also amortising the variational distributions $q(\mathbf{h})$ such that their variational parameters do not scale per-datapoint but, instead, they are determined by a fixed, user-selected set of neural network variables; essentially, a multilayer perceptron (MLP) is trained to output these variables (see also Meng and Zhang [1165] for a recent treatment).

To scale to real-world big datasets, the researchers have further developed methodologies for stochastic variational inference (SVI) which allow the variational lower bound to handle training data as batches. These methodologies adopt the factorised, non-collapsed version of the variational bound, following the paradigm developed by Hensman et al. [737] for single-layer GPs. Indeed, since the likelihood factorises across data points, the bound involves the sparse GP conditional only through its per-datapoint marginals $p(f_n|\mathbf{u})$, with $f_n = f(x_n)$; the bound therefore decomposes into a sum over data points in which all dependence between data points is mediated by the global variables $\mathbf{u}$. Note that it is the structure of the bound, not of the model, that decouples the data points: the exact conditional $p(\mathbf{f}|\mathbf{u})$ itself remains correlated across data points through the residual covariance $\widetilde{\mathbf{K}}$ of equation (25.14). Taking this one step further, Hensman and Lawrence [735] eliminate the variational distribution for the bottleneck layers entirely by marginalising via layer-wise compression. This allows for mini-batch optimisation, reduced variational parameter space as well as the incorporation of "messages" (in the form of Gaussian moments resulting from expectations of the marginalised $q(\mathbf{h})$) passed across layers.

Salimbeni and Deisenroth [1453] proposed *doubly stochastic variational inference* (DSVI), an approach in which no explicit $q(\mathbf{h}^\ell)$ distributions appear. Rather than propagating Gaussian messages across layers, DSVI retains the exact model conditional inside the approximate posterior and handles the resulting intractabilities through Monte Carlo sampling. A key departure from the framework of Sections 25.2.2–25.2.2 is in how layers are connected. Previously, each layer introduced a noise-corrupted bottleneck $\mathbf{h}^\ell = \mathbf{f}^\ell(\mathbf{h}^{\ell-1}) + \boldsymbol{\epsilon}^\ell$, and inference required estimating distributions over these noisy representations. In DSVI, the inter-layer noise is absorbed into the covariance function, so the GP function values $\mathbf{f}^\ell$ serve directly as the output of layer ℓ and input to layer $\ell+1$. Concretely, this means the GP kernel at each layer includes a noise component $\sigma_\ell^2 \delta_{ij}$, but no separate latent variable $\mathbf{h}^\ell$ is introduced. Therefore, the distinction between function values $\mathbf{f}^\ell$ and noisy latent function instantiations $\mathbf{h}^\ell$ dissolves, and there is no separate $q(\mathbf{h}^\ell)$ to estimate.

Consider a deep GP with L layers, inputs $\mathbf{f}^0 = \mathbf{x}$, and each layer augmented with inducing variables $(\mathbf{u}^\ell, (\mathbf{x}_u)^\ell)$. The approximate posterior is:

$$\mathcal{Q}_{\text{DSVI}} = \prod_{\ell=1}^{L} p(\mathbf{f}^\ell | \mathbf{u}^\ell;\, \mathbf{f}^{\ell-1})\, q(\mathbf{u}^\ell), \tag{25.19}$$

where $q(\mathbf{u}^\ell) = \mathcal{N}(\mathbf{u}^\ell|\boldsymbol{\mu}^\ell, \boldsymbol{\Sigma}^\ell)$ and $p(\mathbf{f}^\ell|\mathbf{u}^\ell; \mathbf{f}^{\ell-1})$ is the exact sparse GP conditional from the model. Since this conditional depends on $\mathbf{f}^{\ell-1}$ (the actual output of the previous layer, not a variational approximation to it), inter-layer dependencies are preserved. Since both factors are Gaussian, $\mathbf{u}^\ell$ can be marginalised analytically to yield:

$$q(\mathbf{f}^\ell \mid \mathbf{f}^{\ell-1}) = \int p(\mathbf{f}^\ell|\mathbf{u}^\ell; \mathbf{f}^{\ell-1})\, q(\mathbf{u}^\ell)\, \mathrm{d}\mathbf{u}^\ell = \mathcal{N}(\mathbf{f}^\ell|\tilde{\boldsymbol{\mu}}^\ell, \tilde{\mathbf{K}}_q^\ell), \tag{25.20}$$

where $\tilde{\boldsymbol{\mu}}^\ell$ and $\tilde{\mathbf{K}}_q^\ell$ take the same structural form as $\mathbf{a}^\ell$ and $\widetilde{\mathbf{K}}^\ell$ in equation (25.14) but with $\mathbf{u}^\ell$ replaced by the variational mean $\boldsymbol{\mu}^\ell$ and the covariance adjusted to incorporate $\boldsymbol{\Sigma}^\ell$. The i-th marginal of this Gaussian depends only on $f_{i,:}^{\ell-1}$, since both its mean and its variance involve kernel evaluations of the layer inputs only at that point (together with the inducing inputs); as the bound below requires only these marginals, each data point can be propagated through the hierarchy independently.

In the ELBO, the model conditionals cancel between numerator and denominator, giving:

$$\mathcal{F}_{\mathrm{DSVI}} = \sum_{i=1}^{n} \mathbb{E}_{q(f_{i,:}^L)}\big[\log p(y_{i,:}|f_{i,:}^L)\big] - \sum_{\ell=1}^{L} \mathrm{D}_{\mathrm{KL}}\big(q(\mathbf{u}^\ell)\big\|p(\mathbf{u}^\ell)\big). \tag{25.21}$$

The KL terms are analytic. The expected log-likelihood involves propagating each input $x_{i,:}$ through L layers of the stochastic conditionals (25.20), a composition of nonlinear transformations that is intractable in closed form. This is handled through two sources of stochasticity (hence the name): first, Monte Carlo sampling through the layers using the reparameterisation trick; second, minibatch subsampling over data points. Together, these yield unbiased gradient estimates of $\mathcal{F}_{\mathrm{DSVI}}$.

Comparing with the bound of equation (25.17), the DSVI formulation differs in three main respects. First, the only variational parameters are the inducing distributions $q(\mathbf{u}^\ell)$, scaling as $\mathcal{O}(m^2)$ per layer regardless of n; the $\mathcal{O}(n)$ parameters of $q(\mathbf{h}^\ell)$ are absent. The reduction in parameter count comes at the cost of not maintaining an explicit latent variable posterior over the intermediate representations, which the bound of equation (25.17) does provide. Second, no trace penalty $\frac{\beta^\ell}{2}\langle\mathrm{tr}(\widetilde{\mathbf{K}}^\ell)\rangle$ appears. In contrast to earlier approaches that rely on analytic expectations of kernels under Gaussian inputs, this formulation avoids such expectations and therefore does not impose corresponding restrictions on the choice of covariance function. Third, inter-layer correlations are retained: each layer's function values are conditioned on the sampled outputs of the previous layer rather than decoupled by a mean-field factorisation across layers. This reduces the variance underestimation typical of mean-field approximations, at the cost of the analytic tractability and closed-form per-layer bound that the factorisation affords. The first two differences reflect both the model structure (no separate noise variables) and the inference method (Monte Carlo rather than analytic expectations); the third reflects the inference method alone.

However, a mean-field assumption remains at the level of the inducing distributions $q(\mathbf{u}^\ell)$, which are treated as independent across layers; this factorisation can force intermediate layers to collapse to near-deterministic transformations, losing compositional uncertainty [1251, 1649]. In supervised regression and classification benchmarks, Salimbeni and Deisenroth [1453] demonstrated that deep GPs trained with DSVI consistently match or exceed single-layer GP performance, with the largest gains on large datasets — scaled up to a demonstration on over a billion points.

25.4 Further inference approaches

Sections 25.2.2 and 25.3 developed two variational inference frameworks in detail: the mean-field approach of Damianou and Lawrence [361] and the doubly stochastic formulation of Salimbeni and Deisenroth [1453]. These are not the only routes to approximate inference in deep GPs. In this section we survey three complementary approaches, each addressing different limitations of the variational schemes above. Together, they illustrate the trade-offs between approximation quality, computational cost, and modelling assumptions that shape practical choices.

25.4.1 Expectation propagation

Bui et al. [217] proposed an alternative to variational inference for deep GPs based on approximate expectation propagation (EP). Where variational methods optimise a lower bound on the marginal likelihood, EP approximates the posterior by iteratively matching the moments of local factors. Combined with inducing point representations and *stochastic* EP (which ties the local factors across data points for scalability), it propagates Gaussian approximations through successive layers, rather than maintaining an explicit variational distribution

over the hidden variables. On eleven regression benchmarks, Bui et al. [217] report that their method consistently outperforms single-layer GP regression and is competitive with Bayesian neural network baselines, while providing well-calibrated uncertainty estimates. A limitation is that EP requires moment computations under the approximate posterior that restrict the choice of kernel, similarly to the psi-statistic requirements of the [360, 1616] framework. See also Hernández-Muñoz et al. [744] for a more recent extension combining EP with Monte Carlo sampling.

25.4.2 Random feature approximations

A different strategy is to approximate each GP layer with a finite-dimensional feature expansion, turning the deep GP into a form of Bayesian neural network. Cutajar et al. [350] realise this at each layer using random feature expansions [1363], so that the deep GP composition takes the layer-wise form:

$$\mathbf{F}^{\ell+1} = \mathbf{\Phi}^{\ell}\mathbf{W}^{\ell}, \qquad \mathbf{\Phi}^{\ell} = \gamma^{\ell}(\mathbf{F}^{\ell}\mathbf{\Omega}^{\ell}), \tag{25.22}$$

up to the standard random-feature scaling, which we omit for clarity; in the above, $\mathbf{F}^{\ell}$ is the matrix of layer-ℓ outputs, $\mathbf{W}^{\ell}$ are Bayesian linear weights, the columns of $\mathbf{\Omega}^{\ell}$ are random frequencies sampled from a kernel-specific measure, and γ^{ℓ} is a kernel-specific nonlinearity.

Replacing each GP layer with this approximation transforms the deep GP into a deep network with random fixed features and Bayesian linear weights, on which stochastic variational inference can be performed in the standard weight-space manner. This approach is presented as substantially more scalable than inducing-point-based approaches and is demonstrated on datasets with over 8 million observations [350]. The trade-off is that the GP posterior at each layer is no longer represented explicitly in function space but rather through the weight distribution, and the feature expansion is restricted to kernels that admit a tractable random feature representation; Cutajar et al. [350] derive expansions for the squared exponential (via Bochner's theorem, yielding trigonometric features) and for the arc-cosine kernel (via the integral representation of Cho and Saul [294], yielding ReLU features).

There is an appealing conceptual symmetry here with the constructions of Section 25.1: the chapter began by deriving deep GPs from deep neural networks via the infinite-width limit; random feature methods reverse this by approximating each GP layer with a finite-width neural network layer, returning to weight space for tractability.

25.4.3 Sampling-based and structured variational inference

The variational methods discussed so far, whether based on inducing points or random features, employ Gaussian approximate posteriors. Havasi et al. [714] argue that this can be a "potentially poor unimodal approximation of the generally multimodal posterior" and propose to bypass the Gaussian assumption entirely by applying stochastic gradient Hamiltonian Monte Carlo (SGHMC) (see Chapter 2) to sample from the posterior over the inducing outputs. Working within the inducing-point framework of Section 25.3, they interleave these SGHMC draws with hyperparameter optimisation via a novel moving window Monte Carlo EM algorithm. The resulting method achieves significantly better predictive performance at lower computational cost than variational inference on several regression benchmarks, and importantly, provides empirical evidence for the non-Gaussian nature of deep GP posteriors (see also Rossi et al. [1425], who extend this approach to jointly sample inducing inputs and hyperparameters). From a different angle, Dunlop et al. [463] propose a Metropolis-within-Gibbs sampler across the levels of the hierarchy. The sampler is demonstrated on function-space Bayesian inverse problems using a covariance-operator deep GP construction (distinct from the composition deep GPs that are the focus of the chapter). While neither sampler has been demonstrated at the very largest scales reached by the variational methods above, they establish quality benchmarks that expose fundamental limitations of Gaussian variational approximations.

Beyond switching inference paradigm, several works have sought to improve the fidelity of the variational family itself. Ustyuzhaninov et al. [1649] show that the standard mean-field approximation causes deep GP layers to collapse toward near-deterministic transformations, severely limiting uncertainty quantification; they propose a structured variational distribution that preserves inter-layer correlations. More recently, Xu et al. [1787] replace the Gaussian variational family with diffusion-based posteriors learned through a denoising process (cf. Chapter 50), shifting the paradigm from choosing a fixed-form parametric family to learning a flexible one.

In practice, DSVI [1453] remains a widely adopted inference method, with implementations available in frameworks such as GPflux [478] and GPyTorch [580]. The alternatives surveyed here offer complementary strengths: EP swaps variational ELBO optimisation for an EP energy function optimisation via moment matching, random features enable weight-space inference without matrix decompositions, and sampling methods and richer variational families relax the Gaussian posterior assumption itself.

25.5 Illustration

To further illustrate the properties and behaviour of deep GPs we consider a simple example, that is, attempting to fit the simulated step function; this follows the experimental setting of Calandra et al. [226], Hensman and Lawrence [735]. $n = 40$ equidistant training inputs $\{x_{i,:}\}_{i=1}^{n}$ were generated in the interval $[0, 1]$. As for the outputs, we set $y_{i,:} = 1$ if $x_{i,:} < 0.5$ and 0 otherwise, plus the addition of Gaussian noise with standard deviation 0.02. Therefore, the output space is very "flat" but broken by a big discontinuity (step), thus making the application of a standard GP very challenging. In contrast, a deep GP as a generative model is able to capture this extreme non-stationarity and bimodality by making use of the available intermediate layers that gradually "push" the points in the two extrema. The results presented below confirm this, and they are very similar to those of Hensman and Lawrence [735] who used a deep GP variant with nested variational compression.

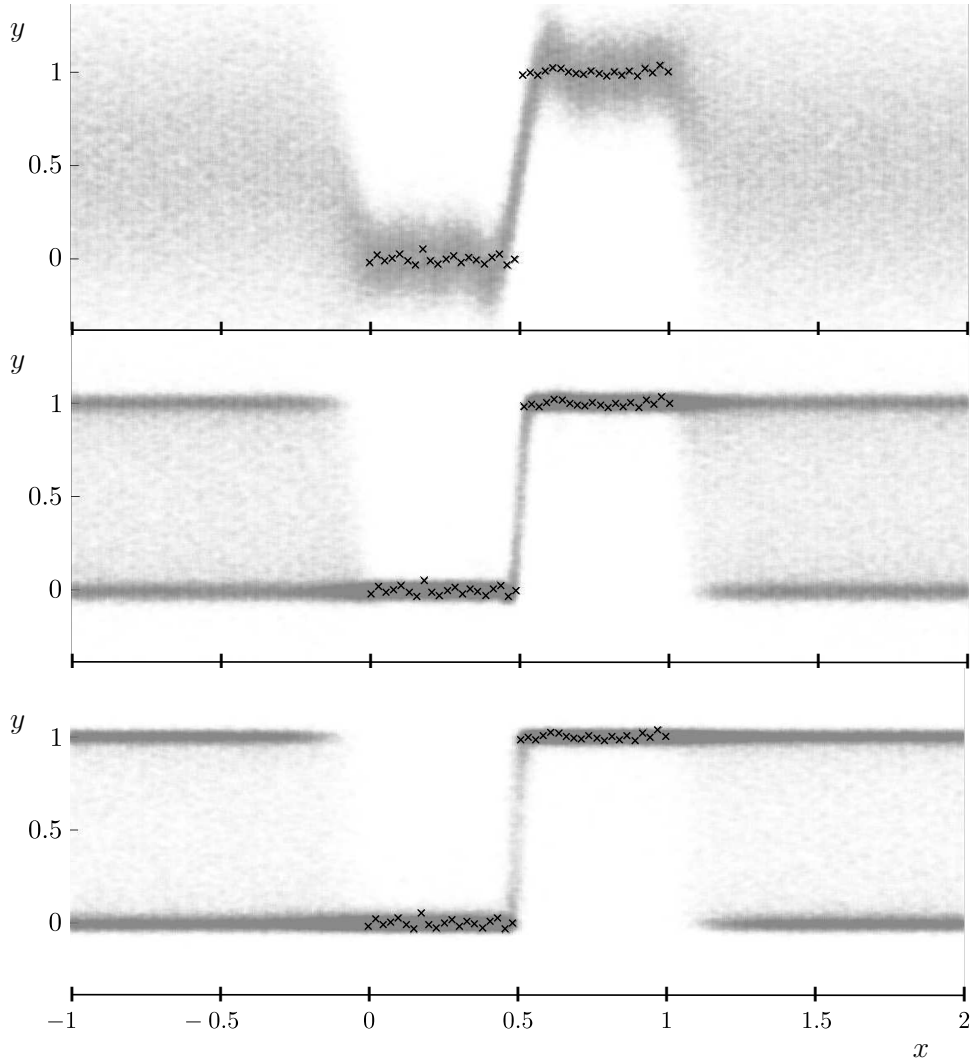
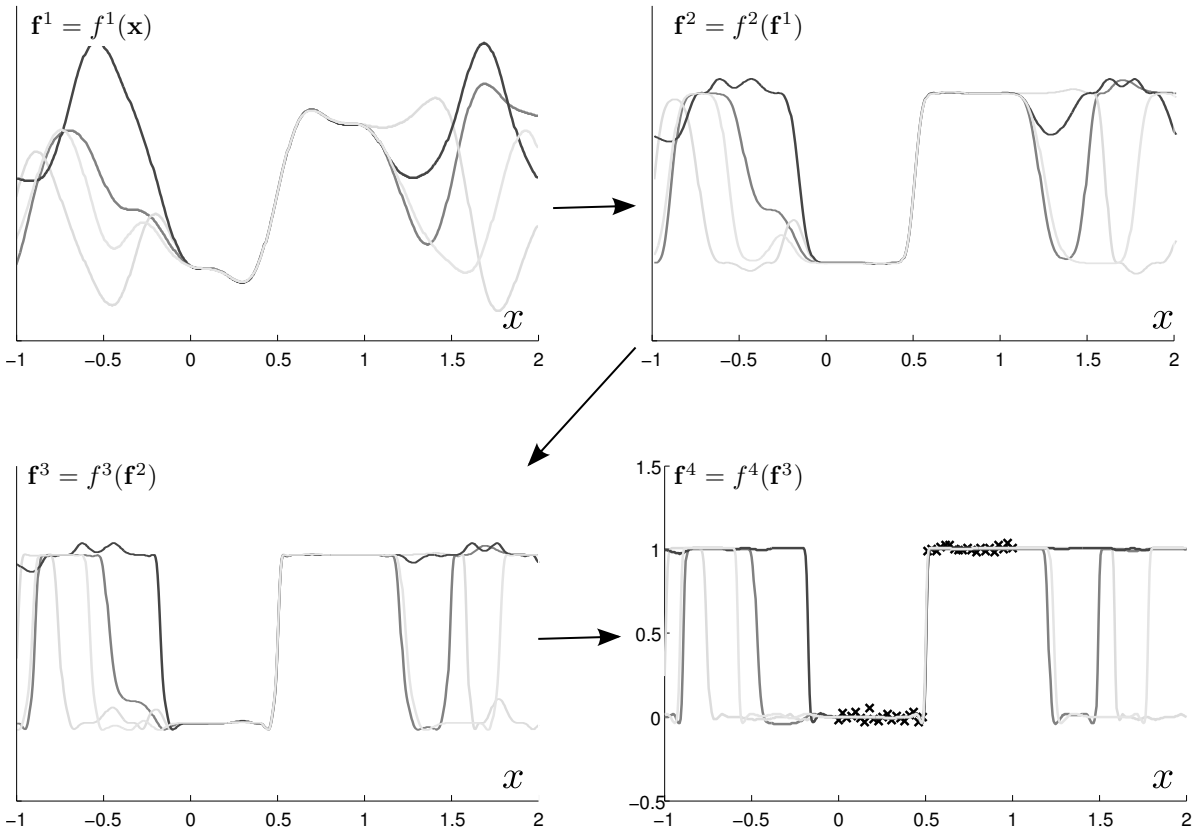
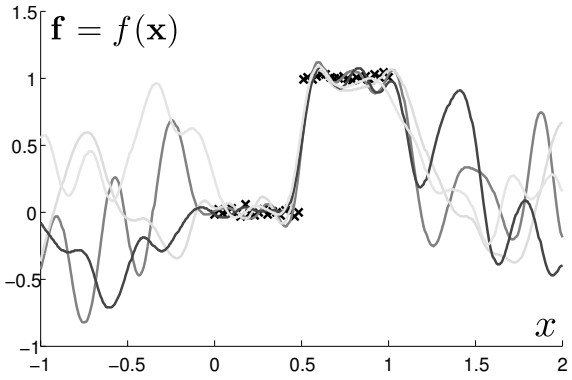


Figure 25.6: Samples drawn from the posterior of a standard GP (top), a deep GP with 1 hidden layer (middle) and a deep GP with three hidden layers (bottom). As in Hensman and Lawrence [735], for better visualisation the sampled points are drawn as small, faint semi-transparent squares, whereas the training points are plotted as dark opaque x's.

Indeed, Figure 25.6 shows 300 sample paths from the posterior obtained by a standard GP with exponentiated quadratic covariance function (top panel) and a deep GP with one (middle panel) and three (bottom panel) hidden layers. Each of the paths corresponds to the discretisation of the interval $[-1, 2]$ in 500 points. The figure demonstrates how even a 1 hidden layer deep GP is able to easily operate in a regime which seems bimodal.



(a) Deep GP with three hidden plus one input-transformation layer



(b) Standard GP

Figure 25.7: (a) Successive warping of sample paths drawn from a deep GP, gradually "pushing" them towards a bimodal regime. (b) Sample paths drawn from a standard GP. In this figure, each sample is drawn versus the initial input $\mathbf{x}$. Instead, figure 25.8 shows a plot of each layer's output signal versus its input signal.

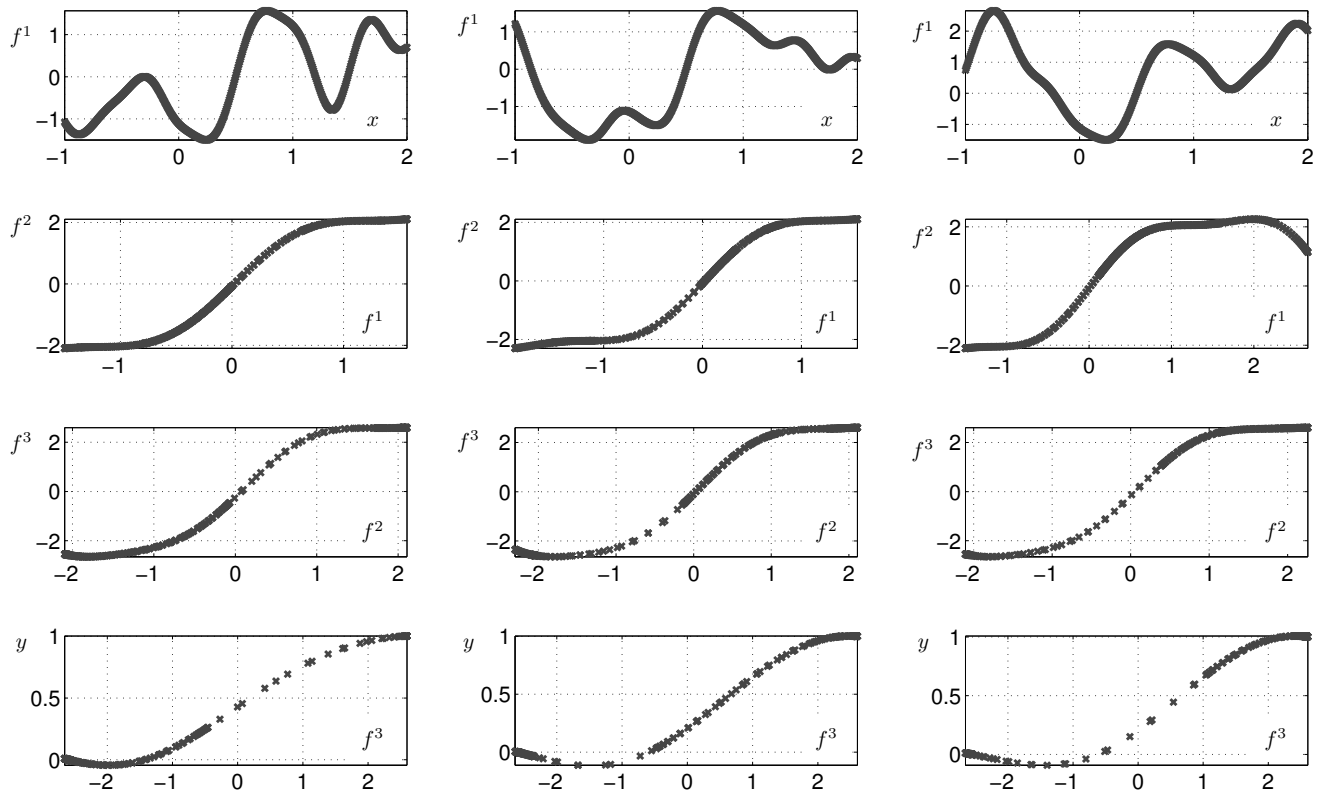


Figure 25.8: Plotting the input versus the output sample signals of every layer in the deep GP. This reveals sigmoidal warping functions which successively "push" the top layer's samples into the bi-modal regime. In the axis labels, the superscript denotes the layer number; each column corresponds to a single sample propagated through the hierarchy.

A deeper architecture (bottom panel) fits in this regime even better, with fewer samples falling between the two extrema. It is interesting to examine the locations of these samples: they seem to cover uniformly and very sparsely the area between the "modes". Further, such samples are only obtained away from the interval $[0, 1]$, signifying that a "jump" is expected to happen (although with extremely low probability) in regions away from the data. On the other hand, away from the data the standard GP produces many samples corresponding to y values that are too far from the training points. Furthermore, the GP is even forced to underfit the data; there are no samples falling in the region close to the "jump" even if two training points (one up and one down) exist there while, at the same time, the variance around the training data is too large.

To confirm the above intuitions, as well as to draw further conclusions, in Figure 25.7 we plot whole sample paths from the deep and the standard GP. For the deep GP, Figure 25.7(a) shows the samples' successive warping through the deep hierarchy, which gradually "pushes" them towards the bimodal regime. As can be seen, the samples (especially in the middle layers) are smooth, and are probabilistically assigned to one of the two modes. Jumps can occur but are rare, perhaps because only one jump exists in the training data. The interpolation suggests that the deep GP interprets the differences of the points *within* each cluster mostly as noise, which matches the actual toy data generation. Finally, Figure 25.8 also depicts sample paths, but there each plot shows the sample output signal in every layer versus its corresponding *input signal* – in contrast to Figure 25.7 where the $x-$axis is the input $\mathbf{x}$. As can be seen in Figure 25.8, each layer's warping function is learned to have a sigmoidal shape, so as to successively "push" the top layer's samples into the bi-modal regime.

Chapter 26

Deep kernel processes and deep kernel machines

26.1 Introduction

Neural networks have become the default choice for complex tasks such as computer vision and natural language processing due to their huge performance advantage over other methods [74, 213, 445, 721, 748, 935, 1361, 1782]. Due to their black-box nature, neural network design is almost entirely based on trial-and-error, and once deployed, it is very difficult to determine what factors contribute to a neural network's decisions; it is often found that neural networks are using unexpected and extraneous features for their predictions [444, 1395]. To exacerbate the issue, our understanding of deep learning theory currently lags far behind practice [e.g. 110, 1407, 1795, 1842]. This leads to models that, although powerful, are time-consuming and finicky to design (e.g. compare He et al. [721] and He et al. [722], or consider the fact that neural architecture search is its own extensive research area [491]), often more computationally expensive than necessary [411, 749, 1588], and potentially unsuitable for use in safety-critical applications like medicine and autonomous systems [30, 499, 1758].

Kernel methods are a class of models that offer strong interpretability [492, 798, 1341, 1863], since their operation explicitly computes similarities between datapoints. Furthermore, many theoretical models of neural networks reduce to kernel methods, allowing a cleaner analysis [386, 829, 1000, 1792]. Unfortunately, kernel methods largely fall short of neural networks in predictive performance on many important machine learning tasks.

Recent developments in kernel method are capable of fixing this shortfall [16, 1792]. These are Deep Kernel Processes (DKPs), and Deep Kernel Machines (DKMs).

26.2 Background

In this chapter we provide the necessary background to understand DKPs and DKMs. We first recap Gaussian processes, before introducing the approximate sparse (inducing point) Gaussian processes that underlie DKPs/DKMs, as well as Deep Gaussian Processes (DGPs). We then introduce deep kernel processes, which are a "no-feature" view DGPs , and deep kernel machines, which are the infinite-width version of a DKPs. Note that the notation of this chapter differs from that of other chapters.

26.2.1 Gaussian Processes

Whilst neural networks have shown impressive abilities to learn features and generalise on complex tasks [935], the precise reasons for why they work so well in these settings is poorly understood, and in many scenarios, such as those with small amounts of data, neural networks can struggle compared to more traditional methods. Hence in this section, we recap Gaussian processes (or GPs) from Bayesian statistics, which are a type of Bayesian kernel method, and in later sections will explore the relationship between neural networks and Gaussian processes. Gaussian processes were previously introduced in Chapter 23.

A Gaussian process [1382] is a stochastic process (a collection of random variables) where any finite subset is multivariate Gaussian distributed. A Gaussian process is completely specified by its mean function $m(\cdot) \in \mathbb{R}^d \to \mathbb{R}$

and positive-definite covariance / kernel function $k(\cdot,\cdot) \in \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$, for which we write,

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot,\cdot)). \tag{26.1}$$

Gaussian processes are used to represent prior distributions over functions. Concretely, if $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^{N_0}$ are two datapoints of dimensionality N_0, then for a function $f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot,\cdot))$, we have,

$$\mathbb{E}\left[f(\mathbf{x})\right] = m(\mathbf{x}) \tag{26.2}$$
$$\mathbb{E}\left[f(\mathbf{x}')\right] = m(\mathbf{x}') \tag{26.3}$$
$$\text{cov}(f(\mathbf{x}), f(\mathbf{x}')) = k(\mathbf{x}, \mathbf{x}'). \tag{26.4}$$

In particular, if we have P datapoints $\{\mathbf{x}_1, \ldots, \mathbf{x}_P\}$, we have that the joint distribution of the function values evaluated at those points is multivariate Gaussian:

$$\begin{pmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_P) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_P) \end{pmatrix}, \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_P) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_P, \mathbf{x}_1) & \cdots & k(\mathbf{x}_P, \mathbf{x}_P) \end{pmatrix}\right). \tag{26.5}$$

Hence we can see that the mean function $m(\cdot)$ tells us the average / expected value of the function at a particular input point, and the kernel function $k(\cdot,\cdot)$ tells us how the function value at a particular point is influenced by the function value at nearby points. Probably the most intuitive class of kernels are the isotropic kernels, which can be computed as a function of the distance between two points, i.e. $k(||\mathbf{x} - \mathbf{x}'||)$. A very common example is the squared exponential kernel,

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{||\mathbf{x} - \mathbf{x}'||^2}{2l^2}\right), \tag{26.6}$$

where l is a hyperparameter known as the "lengthscale", which controls how quickly covariances decay with distance (a bigger lengthscale represents smoother functions that capture broad trends, whilst a smaller lengthscale represents rapidly varying functions that capture local information). There are many other types of kernel function, such as periodic kernels whose covariance oscillates with distance, giving periodic functions, which we will not discuss here. In many cases, we don't have any prior information about the mean of the function, so the mean function $\mathbf{m}(\cdot)$ will often be set to the zero function $\mathbf{m}(\mathbf{x}) = \mathbf{0}$ for all $\mathbf{x} \in \mathbb{R}^{N_0}$.

Assuming we have targets $\{y_1, \ldots, y_P\}$ and a likelihood function $p(y_i|f(x_i))$, with a Gaussian process prior, we can perform Bayesian inference for **non-parametric** (or "function-space") function approximation. This is in contrast to **parametric** / weight-space methods, which assume a parametric form for a function like $f(x_i) = w_1 x_i + w_2$ in Bayesian linear regression, and compute a posterior over the weights $p(w_1, w_2|\{x_1, \ldots, x_P\}, \{y_1, \ldots, y_P\})$ (which itself could then be used to compute a predictive distribution).

Specifically, given a new set of test points $\{\mathbf{x}_1^*, \ldots, \mathbf{x}_{P_*}^*\}$, we wish to compute the posterior $p(f(\mathbf{x}_1^*), \ldots, f(\mathbf{x}_{P_*}^*)|\{x_1, \ldots, x_{P_*}\}$, over the value of the function at the new test point. This is made possible in certain cases due to the closed-form "partitioned Gaussian" formulae. Suppose I have a multivariate Gaussian random vector which is partitioned into two shorter vectors $\mathbf{a}$ and $\mathbf{b}$ in the following way:

$$\begin{pmatrix} \mathbf{a} \\ \mathbf{b} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}, \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}\right). \tag{26.7}$$

Then the conditional distribution $p(\mathbf{b}|\mathbf{a})$ is also multivariate Gaussian, and has the form,

$$\mathbf{b}|\mathbf{a} \sim \mathcal{N}\left(\boldsymbol{\mu}_b + \boldsymbol{\Sigma}_{ba}\boldsymbol{\Sigma}_{aa}^{-1}(\mathbf{a} - \boldsymbol{\mu}_a), \boldsymbol{\Sigma}_{bb} - \boldsymbol{\Sigma}_{ba}\boldsymbol{\Sigma}_{aa}^{-1}\boldsymbol{\Sigma}_{ab}\right). \tag{26.8}$$

Assuming noiseless observations $y_i = f(x_i)$ (i.e. we directly observe the latents $\mathbf{f}$), we can form a partitioned multivariate Gaussian prior using our train points $\{f(\mathbf{x}_1), \ldots, f(\mathbf{x}_P)\}$ and our test points $\{f(\mathbf{x}_1^*), \ldots, f(\mathbf{x}_{P_*}^*)\}$ as follows:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mathbf{m}(\mathbf{X}) \\ \mathbf{m}(\mathbf{X}_*) \end{pmatrix}, \begin{pmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{pmatrix}\right) \tag{26.9}$$

where $\mathbf{X} \in \mathbb{R}^{P \times N_0}, \mathbf{X}_* \in \mathbb{R}^{P_* \times N_0}$ are the vertically stacked inputs and $\mathbf{f} \in \mathbb{R}^P, \mathbf{f}_* \in \mathbb{R}^{P_*}$ are the vertically stacked function values evaluated at those inputs, $\mathbf{m}(\mathbf{X}), \mathbf{m}(\mathbf{X}_*)$ are the vertically stacked mean values for those inputs, and $\mathbf{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{P \times P}, \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \in \mathbb{R}^{P \times P_*}, \mathbf{K}(\mathbf{X}_*, \mathbf{X}) \in \mathbb{R}^{P_* \times P}, \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \in \mathbb{R}^{P_* \times P_*}$ are the "kernel matrices", formed by applying the kernel function $k(\cdot,\cdot)$ to all pairs of train vs. train, train vs. test, test vs. train, and

test vs. test points, respectively. Note that since $k(\cdot,\cdot)$ is a symmetric positive-definite function, $\mathbf{K}(\mathbf{X},\mathbf{X})$ and $\mathbf{K}(\mathbf{X}_*,\mathbf{X}_*)$ are symmetric positive-definite matrices, and $K(\mathbf{X},\mathbf{X}_*) = K(\mathbf{X}_*,\mathbf{X})^T$.

Applying the partitioned Gaussian formula allows us to compute our posterior predictive distribution as,

$$\mathbf{f}_*|\mathbf{f} \sim \mathcal{N}\left(\mathbf{m}_* + \mathbf{K}_*^T\mathbf{K}^{-1}(\mathbf{f}-\mathbf{m}), \mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_*\right), \tag{26.10}$$

where we used the abbreviations $\mathbf{m} = \mathbf{m}(\mathbf{X}), \mathbf{m}_* = \mathbf{m}(\mathbf{X}_*), \mathbf{K} = \mathbf{K}(\mathbf{X},\mathbf{X}), \mathbf{K}_* = \mathbf{K}(\mathbf{X},\mathbf{X}_*), \mathbf{K}_{**} = \mathbf{K}(\mathbf{X}_*,\mathbf{X}_*)$. This is common in the Gaussian process literature. Hence we have a closed-form solution for the distribution of our test predictions $\mathbf{f}_*$. Note it is common to drop the conditioning on the inputs $\mathbf{X}$.

We also get a closed-form solution when we assume Gaussian noise on our data, i.e. $y_i = f(\mathbf{x}_i) + \epsilon$ with $\epsilon \sim \mathcal{N}(0,\sigma^2), \sigma^2 \in \mathbb{R}^+$. Since the sum of two Gaussians is itself Gaussian, the joint distribution over training labels $\mathbf{y} = \mathbf{f} + \boldsymbol{\epsilon}$ and test labels $\mathbf{y}_* = \mathbf{f}_* + \boldsymbol{\epsilon}$ (where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ becomes

$$\begin{pmatrix}\mathbf{y}\\ \mathbf{y}_*\end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix}\mathbf{m}(\mathbf{X})\\ \mathbf{m}(\mathbf{X}_*)\end{pmatrix}, \begin{pmatrix}\mathbf{K}(\mathbf{X},\mathbf{X}) + \sigma^2\mathbf{I} & \mathbf{K}(\mathbf{X},\mathbf{X}_*)\\ \mathbf{K}(\mathbf{X}_*,\mathbf{X}) & \mathbf{K}(\mathbf{X}_*,\mathbf{X}_*) + \sigma^2\mathbf{I}\end{pmatrix}\right). \tag{26.11}$$

We can form the posterior predictive distribution

$$\mathbf{y}_*|\mathbf{y} \sim \mathcal{N}\left(\mathbf{m}_* + \mathbf{K}_*^T(\mathbf{K}+\sigma^2\mathbf{I})^{-1}(\mathbf{y}-\mathbf{m}), \mathbf{K}_{**} - \mathbf{K}_*^T(\mathbf{K}+\sigma^2\mathbf{I})^{-1}\mathbf{K}_* + \sigma^2\mathbf{I}\right). \tag{26.12}$$

In the simple case of Gaussian noise regression, it is not uncommon to see texts present "prediction" as predicting the underlying latent $\mathbf{f}_*$, so we actually form the joint distribution between $\mathbf{y}$ and $\mathbf{f}_*$:

$$\begin{pmatrix}\mathbf{y}\\ \mathbf{f}_*\end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix}\mathbf{m}(\mathbf{X})\\ \mathbf{m}(\mathbf{X}_*)\end{pmatrix}, \begin{pmatrix}\mathbf{K}(\mathbf{X},\mathbf{X}) + \sigma^2\mathbf{I} & \mathbf{K}(\mathbf{X},\mathbf{X}_*)\\ \mathbf{K}(\mathbf{X}_*,\mathbf{X}) & \mathbf{K}(\mathbf{X}_*,\mathbf{X}_*)\end{pmatrix}\right), \tag{26.13}$$

and hence we can compute the posterior over $\mathbf{f}_*$ as

$$\mathbf{f}_*|\mathbf{y} \sim \mathcal{N}\left(\mathbf{m}_* + \mathbf{K}_*^T(\mathbf{K}+\sigma^2\mathbf{I})^{-1}(\mathbf{y}-\mathbf{m}), \mathbf{K}_{**} - \mathbf{K}_*^T(\mathbf{K}+\sigma^2\mathbf{I})^{-1}\mathbf{K}_*\right). \tag{26.14}$$

In summary, we assumed a multivariate Gaussian prior $p(\mathbf{f})$ over our latents, and a Gaussian likelihood $p(\mathbf{y}|\mathbf{f}) = \mathcal{N}(y_i; \mathbf{f}, \sigma^2\mathbf{I})$.

For completeness and to intuitively motivate GPs, we can think of a GP as an infinite-dimensional multivariate Gaussian, where the dimensions represent the function value at all possible points $\mathbf{x}$ in the input space. Specifically, it retains the property that all marginal and conditional distributions over subsets of variables are also Gaussian. Thus, the prior at a finite set of input locations $\mathbf{X}$ is a finite-dimensional multivariate Gaussian $p(\mathbf{f})$, and the "prior conditional" distribution at a set of finite set of input locations $\mathbf{X}_*$ given the known function values at a different set of inputs $\mathbf{X}$ is a multivariate Gaussian $p(\mathbf{f}_*|\mathbf{f})$, given by the conditional Gaussian formulae in Eq. 26.8. In the case of a Gaussian likelihood function, **the posterior process is also a Gaussian process**, i.e. the posterior distribution over the entire function $f(\cdot)$ has the form

$$f(\cdot)|X_{\text{data}}, y_{\text{data}} \sim \mathcal{GP}(m_{\text{post}}(\cdot), k_{\text{post}}(\cdot,\cdot)) \tag{26.15}$$

where

$$m_{\text{post}}(x) = m(x) + k(x, X_{\text{data}})[K(X_{\text{data}}, X_{\text{data}}) + \sigma_n^2 I]^{-1}(y_{\text{data}} - m(X_{\text{data}})) \tag{26.16}$$

$$k_{\text{post}}(x,x') = k(x,x') - k(x, X_{\text{data}})[K(X_{\text{data}}, X_{\text{data}}) + \sigma_n^2 I]^{-1}k(X_{\text{data}}, x'). \tag{26.17}$$

That is, $m(\cdot)$, $k(\cdot,\cdot)$ are our prior mean and kernel function, and $m_{\text{post}}(\cdot), k_{\text{post}}(\cdot,\cdot)$ are our posterior mean and kernel function. This is what makes the Gaussian process 'non-parametric'. The posterior is really over the infinite-dimensional Gaussian process, but in practice, we can only manipulate expressions involving finite sets of locations $\mathbf{X}$ and $\mathbf{X}_*$. As we will see in the next section, the posterior process is only a Gaussian process in the case of a Gaussian likelihood. When using a non-Gaussian likelihood (such as for classification), we must resort to approximations.

26.2.2 Variational Inference for Gaussian Process Classification

Variational inference is discussed in depth in Chapter 12, but we provide a self-contained recap here. Suppose we have data $\mathbf{x}_i \in \mathbb{R}^{N_0}$, with binary labels $y_i \in \{0,1\}$. A Gaussian likelihood function is not appropriate here, as

our labels take discrete values, whereas a Gaussian distribution has continuous and unbounded support. We work around this by using a logistic / sigmoid function to form a Bernoulli likelihood by transforming the latents,

$$p(y_i = 1|f_i) = \sigma(f_i) = \frac{1}{1 + \exp(-f_i)}. \tag{26.18}$$

More generally, if $y_i \in \{0, \dots, K\}$ are labels in a K-class classification task, we can use a Categorical likelihood with softmax probabilities computed from a vector latent $\mathbf{f}(\cdot) \in \mathbb{R}^K$,

$$p(y_i = k|\mathbf{f}_i) = \text{softmax}(\mathbf{f}_i)_k = \frac{\exp(f_{ik})}{\sum_{k'} \exp(f_{ik'})}. \tag{26.19}$$

We immediately encounter a problem: given a multivariate Gaussian prior $p(\mathbf{f})$ over the latents, and a Bernoulli likelihood $p(\mathbf{y}|\mathbf{f})$, the posterior over the latents $p(\mathbf{f})$ is not closed-form. Even if it were closed-form, but not Gaussian, then the integral required to compute the predictive distribution $p(\mathbf{y}_*|\mathbf{y}) = \int p(\mathbf{y}_*|\mathbf{f}_*)p(\mathbf{f}_*|\mathbf{y})d\mathbf{f}_*$ almost certainly does not have a closed form solution.

Hence approximation is necessary. There are many ways we could go about this, but here we consider using variational inference to obtain an approximate posterior $q(\mathbf{f}_*)$, and then use samples $\mathbf{f}_{*,(m)}$ from this approximate posterior to compute the Monte-Carlo average

$$p(\mathbf{y}_*|\mathbf{y}) = \int p(\mathbf{y}_*|f_*)p(\mathbf{f}_*|\mathbf{y})d\mathbf{f}_* \approx \frac{1}{M}\sum_m^M \sigma\left(\mathbf{f}_{*,(m)}\right) \qquad \mathbf{f}_{*,(m)} \in \mathbb{R}^{P_*} \sim q(\mathbf{f}_*). \tag{26.20}$$

In variational inference [170, 851, 1691], we seek to approximate an intractable posterior $p(\mathbf{f}|\mathbf{y})$ with an approximate posterior $q(\mathbf{f})$ which is easier to manipulate and sample from. A very common choice, is a multivariate Gaussian approximate posterior $q(\mathbf{f}) = \mathcal{N}(\mathbf{f}; \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ parametrised by learnable mean and covariance matrix $\boldsymbol{\mu}_q \in \mathbb{R}^P, \boldsymbol{\Sigma}_q \in \mathbb{R}^{P\times P}$. In VI, q is called a 'variational approximate posterior', and $\boldsymbol{\mu}_q$ and $\boldsymbol{\Sigma}_q$ are 'variational parameters'. The main benefit of this approximation is that it allows closed-form expressions for the approximate predictive posterior $q(\mathbf{f}_*)$.

If $q(\mathbf{f}) = \mathcal{N}(\mathbf{f}; \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ is our approximate posterior over the training latents, then a sensible choice of the approximate posterior over the test latents $\mathbf{f}_*$ is given by the integral,

$$q(\mathbf{f}_*) = \int p(\mathbf{f}_*|\mathbf{f})q(\mathbf{f})d\mathbf{f}. \tag{26.21}$$

This is sensible because it closely resembles the true posterior,

$$p(\mathbf{f}_*) = \int p(\mathbf{f}_*, \mathbf{f})d\mathbf{f} = \int p(\mathbf{f}_*|\mathbf{f})p(\mathbf{f})d\mathbf{f}. \tag{26.22}$$

Since $p(\mathbf{f}_*|\mathbf{f}) = \mathcal{N}\left(\mathbf{f}_*; \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{f}, \mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_*\right)$ and $q(\mathbf{f}) = \mathcal{N}(\mathbf{f}; \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ are both Gaussian, it is a standard result that their integral will also be Gaussian, and we can compute the mean and variance using the laws of total expectation and total variance:

$$\begin{aligned}
\mathbb{E}_{q(\mathbf{f})}[\mathbf{f}_*] &= \mathbb{E}_{q(\mathbf{f})}\left[\mathbb{E}[\mathbf{f}_*|\mathbf{f}]\right] && (26.23)\\
&= \mathbb{E}_{q(\mathbf{f})}[\mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{f}] && (26.24)\\
&= \mathbf{K}_*^T\mathbf{K}^{-1}\boldsymbol{\mu}_q && (26.25)\\
\text{Var}_{q(\mathbf{f})}[\mathbf{f}_*] &= \mathbb{E}_{q(\mathbf{f})}\left[\text{Var}[\mathbf{f}_*|\mathbf{f}]\right] + \text{Var}_{q(\mathbf{f})}\left[\mathbb{E}[\mathbf{f}_*|\mathbf{f}]\right] && (26.26)\\
&= \mathbb{E}_{q(\mathbf{f})}\left[\mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_*\right] + \text{Var}_{q(\mathbf{f})}\left[\mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{f}\right] && (26.27)\\
&= \mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_* + \mathbf{K}_*^T\mathbf{K}^{-1}\text{Var}_{q(\mathbf{f})}\left[\mathbf{f}\right]\mathbf{K}^{-1}\mathbf{K}_* && (26.28)\\
&= \mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_* + \mathbf{K}_*^T\mathbf{K}^{-1}\boldsymbol{\Sigma}_q\mathbf{K}^{-1}\mathbf{K}_*. && (26.29)
\end{aligned}$$

Note that for clarity we have used subscript $q(\mathbf{f})$ in the expectations and variances which integrate over the approximate posterior, and omitted the subscripts in expressions that only involve the prior $p(\mathbf{f}* \mid \mathbf{f})$. Hence, assuming we have values for $\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q$, we can generate samples from the approximate predictive distribution

$$q(\mathbf{f}_*) = \mathcal{N}\left(\mathbf{f}_*; \mathbf{K}_*^T\mathbf{K}^{-1}\boldsymbol{\mu}_q, \mathbf{K}_{**} - \mathbf{K}_*^T\mathbf{K}^{-1}\mathbf{K}_* + \mathbf{K}_*^T\mathbf{K}^{-1}\boldsymbol{\Sigma}_q\mathbf{K}^{-1}\mathbf{K}_*\right) \tag{26.30}$$

and then compute Monte-Carlo averaged predictions for $p(\mathbf{y}_*|\mathbf{y})$ as in Eq. (26.20).

All that remains is to determine the values for $\boldsymbol{\mu}_q \in \mathbb{R}^P, \boldsymbol{\Sigma}_q \in \mathbb{R}^{P\times P}$, that make our approximate posterior $q(\mathbf{f})$ as close to the true posterior $p(\mathbf{f}|\mathbf{y})$ as possible. In variational inference, the desired measure of "closeness" is the KL divergence:

$$\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q = \arg\min_{\boldsymbol{\mu},\boldsymbol{\Sigma}} D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f}|\mathbf{y})\right) = \arg\min_{\boldsymbol{\mu},\boldsymbol{\Sigma}} \int q(\mathbf{f}) \log \frac{q(\mathbf{f})}{p(\mathbf{f}|\mathbf{y})} d\mathbf{f}. \tag{26.31}$$

At first glance, this seems difficult because the true posterior $p(\mathbf{f}|\mathbf{y})$ is unknown. However, we can use a fairly common trick of rewriting the divergence in terms of the ELBO (Evidence Lower Bound Objective),

$$\begin{aligned}
&D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f}|\mathbf{y})\right) && \text{(26.32a)}\\
&= \int q(\mathbf{f}) \log \frac{q(\mathbf{f})}{p(\mathbf{f}|\mathbf{y})} d\mathbf{f} && \text{(26.32b)}\\
&= \int q(\mathbf{f}) \log q(\mathbf{f}) d\mathbf{f} - \int q(\mathbf{f}) \log p(\mathbf{f}|\mathbf{y}) d\mathbf{f} && \text{(26.32c)}\\
&= \int q(\mathbf{f}) \log q(\mathbf{f}) d\mathbf{f} - \int q(\mathbf{f}) \log \frac{p(\mathbf{y}|\mathbf{f})p(\mathbf{f})}{p(\mathbf{y})} d\mathbf{f} && \text{(26.32d)}\\
&= \int q(\mathbf{f}) \left[\log q(\mathbf{f}) - \log p(\mathbf{f})\right] d\mathbf{f} - \int q(\mathbf{f}) \log p(\mathbf{y}|\mathbf{f}) d\mathbf{f} + \int q(\mathbf{f}) \log p(\mathbf{y}) d\mathbf{f} && \text{(26.32e)}\\
&= \int q(\mathbf{f}) \log \frac{q(\mathbf{f})}{p(\mathbf{f})} d\mathbf{f} - \int q(\mathbf{f}) \log p(\mathbf{y}|\mathbf{f}) d\mathbf{f} + \log p(\mathbf{y}) \int q(\mathbf{f}) d\mathbf{f} && \text{(26.32f)}\\
&= \underbrace{D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f})\right) - \mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}|\mathbf{f})\right]}_{\text{Negative ELBO}} + \underbrace{\log p(\mathbf{y})}_{\text{Evidence}}. && \text{(26.32g)}
\end{aligned}$$

Rearranging, in terms of the evidence,

$$\log p(\mathbf{y}) = D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f}|\mathbf{y})\right) + \underbrace{\mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}|\mathbf{f})\right] - D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f})\right)}_{\text{ELBO}}. \tag{26.33}$$

The divergence between the approximate and true posteriors, $D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f}|\mathbf{y})\right)$, is always positive, so the ELBO acts as a lower bound to the model evidence. When the data and model are fixed, the evidence is constant, and so by maximising the ELBO, we simultaneously minimise the KL divergence between the approximate posterior and the true posterior, which is our overall goal. To see whether we can optimise the ELBO,

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}|\mathbf{f})\right]}_{\text{Expected Log Likelihood (under } q)} - \underbrace{D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f})\right)}_{\text{KL between approx. post. and prior}}. \tag{26.34}$$

Observe that the first term is the expected log-likelihood under the approximate posterior $q(\mathbf{f})$. As suggested earlier, we can estimate this term by taking samples from $q(\mathbf{f})$. Since we wish to differentiate the ELBO in order to maximise it with gradient-ascent, we will use the reparametrisation trick [892] to detach the random sampling process from the computation graph. That is, our Gaussian random sample $\mathbf{f}$ will be written as a transformation of a standard Gaussian sample, so that all learnable parameters are in the deterministic transformation, rather than random variables,

$$\mathbf{f} = \boldsymbol{\mu}_q + \mathbf{L}\mathbf{z} \qquad \text{where } \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ and } \boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^T. \tag{26.35}$$

The second term is the KL divergence between the approximate posterior and the prior over $\mathbf{f}$. Since these are both Gaussian ($p(\mathbf{f}) = \mathcal{N}(\mathbf{0}, \mathbf{K})$ and $q(\mathbf{f}) = \mathcal{N}(\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$), this can be computed in closed form:

$$D_{\mathrm{KL}}\left(q(\mathbf{f})||p(\mathbf{f})\right) = \frac{1}{2}\left[\log \frac{|\mathbf{K}|}{|\boldsymbol{\Sigma}_q|} - P + \boldsymbol{\mu}_q \mathbf{K}^{-1} \boldsymbol{\mu}_q^T + \mathrm{tr}\left(\mathbf{K}^{-1}\boldsymbol{\Sigma}_q\right)\right] \tag{26.36}$$

where $|\mathbf{A}|$ denotes the determinant of a matrix $\mathbf{A}$, and $\mathrm{tr}(\mathbf{A})$ the trace, and as a reminder, P is the number of training datapoints, dictating the size of $\mathbf{K} \in \mathbb{R}^{P\times P}$.

As a final point, we note that since the ELBO is a lower bound to the model evidence, it is common to optimise model hyperparameters (i.e. hyperparameters of the prior $p(\mathbf{f})$) by maximising the ELBO as a proxy to the model evidence.

Now that we've seen how we can use variational inference to do GP classification, let's discuss how practical (or, rather, impractical) it is. The key problem is the dependence on the dataset size P. The kernel matrix $\mathbf{K} \in \mathbb{R}^{P\times P}$ and the approximate posterior covariance $\boldsymbol{\Sigma}_q \in \mathbb{R}^{P\times P}$ both have P^2 elements, where P is the number

of training datapoints. To store $\mathbf{\Sigma}_q$ in memory for moderately sized dataset with $P = 10^6$ datapoints using 32-bit floating point numbers would requires $(10^6)^2 \times 4$ bytes, which equates to roughly $3,725$GB of memory. Supposing that memory were not a concern, we must still compute expensive operations on the kernel matrix, such as its inverse $\mathbf{K}^{-1}$ and its determinant $|\mathbf{K}|$. Both of these operations are very expensive and are impractical for large P.

In the next section, we introduce "sparse" variational Gaussian processes, which replace the P training points with P_i "inducing points" which seek to summarise the datapoint, but with $P_i \ll P$ so that we get practical $\mathcal{O}(P_i^3)$ operations.

26.2.3 Sparse Variational Gaussian Processes

Sparse Gaussian processes [737, 738, 1354, 1528, 1615] seek to alleviate the $\mathcal{O}(P^2)$ space complexity and $\mathcal{O}(P^3)$ time complexity of Gaussian processes (where P is the number of training points) by replacing the training points with a small set of $P_i \ll P$ "inducing points" that somehow summarise the dataset. These inducing points may simply be a subset of the original training data, or they may be learnable parameters, optimised in some way. Here, we follow the variational inducing point approach as introduced by Titsias [1615] and refined by Hensman et al. [737], where inducing points are inserted via the approximate posterior.

Concretely, let $\mathbf{X}_\mathrm{i} \in \mathbb{R}^{P_\mathrm{i} \times N_0}$ be a set of "inducing inputs", that is, a set of P_i input datapoints, and let $\mathbf{f}_\mathrm{i} \in \mathbb{R}^{P_\mathrm{i}}$ be the "inducing values", i.e. function values evaluated at those inducing inputs. Furthermore, for absolute clarity, we introduce new notation $\mathbf{f}_\mathrm{t} \in \mathbb{R}^{P_\mathrm{t}}$ for our set of P_t training points.

As always, the joint distribution between function evaluations is multivariate Gaussian due to our Gaussian process prior. This means the joint prior over our inducing points $\mathbf{f}_\mathrm{i}$ and our train points $\mathbf{f}_\mathrm{t}$ is multivariate Gaussian:

$$\mathbf{f} = \begin{pmatrix} \mathbf{f}_\mathrm{i} \\ \mathbf{f}_\mathrm{t} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mathbf{0} \\ \mathbf{0} \end{pmatrix}, \begin{pmatrix} \mathbf{K}_\mathrm{ii} & \mathbf{K}_\mathrm{it} \\ \mathbf{K}_\mathrm{ti} & \mathbf{K}_\mathrm{tt} \end{pmatrix} \right). \tag{26.37}$$

Here, $\mathbf{f} \in \mathbb{R}^{P_\mathrm{i}+P_\mathrm{t}}$ denotes the concatenation of our inducing and train points, $\mathbf{K}_\mathrm{ii} = \mathbf{K}(\mathbf{X}_\mathrm{i}, \mathbf{X}_\mathrm{i}) \in \mathbb{R}^{P_\mathrm{i} \times P_\mathrm{i}}$ is the kernel matrix formed by applying the kernel to all pairs of inducing points, $\mathbf{K}_\mathrm{ti} = \mathbf{K}_\mathrm{it}^T = \mathbf{K}(\mathbf{X}_\mathrm{t}, \mathbf{X}_\mathrm{i}) \in \mathbb{R}^{P_\mathrm{t} \times P_\mathrm{i}}$ is the kernel matrix formed by applying the kernel to all pairs of train vs. inducing points, and $\mathbf{K}_\mathrm{tt} = \mathbf{K}(\mathbf{X}_\mathrm{t}, \mathbf{X}_\mathrm{t}) \in \mathbb{R}^{P_\mathrm{t} \times P_\mathrm{t}}$ is the kernel matrix formed by applying the kernel to all pairs of train points.

We assume our approximate posterior factorises as follows [1615],

$$q(\mathbf{f}) = q(\mathbf{f}_\mathrm{i}, \mathbf{f}_\mathrm{t}) = p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i}) q(\mathbf{f}_\mathrm{i}), \qquad q(\mathbf{f}_\mathrm{i}) = \mathcal{N}(\boldsymbol{\mu}_\mathrm{i}, \mathbf{\Sigma}_\mathrm{ii}). \tag{26.38}$$

In other words, we only model an approximate posterior $q(\mathbf{f}_\mathrm{i})$ on the inducing points, and use the prior conditional $p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})$ to give the implied distribution over train points. Note that, unlike in the previous chapter, our approximate posterior is no longer over the train latents alone, but also the inducing latents $\mathbf{f}_\mathrm{i}$. The implied approximate posterior over just the train latents $\mathbf{f}_\mathrm{t}$ is obtained by marginalising out the inducing points,

$$q(\mathbf{f}_\mathrm{t}) = \int p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i}) q(\mathbf{f}_\mathrm{i}) d\mathbf{f}_\mathrm{i}. \tag{26.39}$$

Since this is a Gaussian integral, it has a closed form solution, which is essentially identical to Eq. (26.30), i.e.

$$q(\mathbf{f}_\mathrm{t}) = \mathcal{N}\left(\mathbf{f}_\mathrm{t}; \mathbf{K}_\mathrm{ti}^T \mathbf{K}_\mathrm{ii}^{-1} \boldsymbol{\mu}_\mathrm{i}, \mathbf{K}_\mathrm{tt} - \mathbf{K}_\mathrm{ti}^T \mathbf{K}_\mathrm{ii}^{-1} \mathbf{K}_\mathrm{it} + \mathbf{K}_\mathrm{ti}^T \mathbf{K}_\mathrm{ii}^{-1} \mathbf{\Sigma}_\mathrm{ii} \mathbf{K}_\mathrm{ii}^{-1} \mathbf{K}_\mathrm{it}\right) \tag{26.40}$$

and so the ELBO given by performing variational inference over just the training latents, as in the previous section, is

$$\mathrm{ELBO} = \mathbb{E}_{q(\mathbf{f}_\mathrm{t})}\left[\log p(\mathbf{y}_\mathrm{t}|\mathbf{f}_\mathrm{t})\right] - D_\mathrm{KL}\left(q(\mathbf{f}_\mathrm{t}) || p(\mathbf{f}_\mathrm{t})\right) \tag{26.41}$$

with $p(\mathbf{f}_\mathrm{t}) = \mathcal{N}(\mathbf{f}_\mathrm{t}; \mathbf{0}, \mathbf{K}_\mathrm{tt})$ the prior over the training latents.

However, there is a problem with this ELBO. The KL divergence involves inverting the matrix $\mathbf{K}_\mathrm{tt} \in \mathbb{R}^{P_\mathrm{t} \times P_\mathrm{t}}$, which is very expensive; we introduced the inducing point scheme to avoid such operations!

Titsias [1615] proposed an alternative ELBO, by directly performing variational inference over the joint distributions $q(\mathbf{f}_\mathrm{i}, \mathbf{f}_\mathrm{t})$ and $p(\mathbf{f}_\mathrm{i}, \mathbf{f}_\mathrm{t})$. This results in,

$$\begin{aligned} \mathrm{ELBO}_\mathrm{Titsias} &= \mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}_\mathrm{t}|\mathbf{f}_\mathrm{t})\right] - D_\mathrm{KL}\left(q(\mathbf{f}_\mathrm{i}, \mathbf{f}_\mathrm{t}) || p(\mathbf{f}_\mathrm{i}, \mathbf{f}_\mathrm{t})\right) & (26.42) \\ &= \mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}_\mathrm{t}|\mathbf{f}_\mathrm{t})\right] - D_\mathrm{KL}\left(q(\mathbf{f}_\mathrm{i}) || p(\mathbf{f}_\mathrm{i})\right) & (26.43) \end{aligned}$$

where the second equality holds since

$$D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})\right) = D_{\mathrm{KL}}\left(p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i})||p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})p(\mathbf{f}_\mathrm{i})\right) \tag{26.44}$$

$$= \mathbb{E}_{q(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})}\left[\log \frac{p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i})}{p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})p(\mathbf{f}_\mathrm{i})}\right] \tag{26.45}$$

$$= \mathbb{E}_{q(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})}\left[\log \frac{q(\mathbf{f}_\mathrm{i})}{p(\mathbf{f}_\mathrm{i})}\right] \tag{26.46}$$

$$= \int\int p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i}) \log \frac{q(\mathbf{f}_\mathrm{i})}{p(\mathbf{f}_\mathrm{i})} d\mathbf{f}_\mathrm{t} d\mathbf{f}_\mathrm{i} \tag{26.47}$$

$$= \int q(\mathbf{f}_\mathrm{i}) \log \frac{q(\mathbf{f}_\mathrm{i})}{p(\mathbf{f}_\mathrm{i})} \left(\int p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i}) d\mathbf{f}_\mathrm{t}\right) d\mathbf{f}_\mathrm{i} \tag{26.48}$$

$$= \int q(\mathbf{f}_\mathrm{i}) \log \frac{q(\mathbf{f}_\mathrm{i})}{p(\mathbf{f}_\mathrm{i})} d\mathbf{f}_\mathrm{i} \tag{26.49}$$

$$= D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i})||p(\mathbf{f}_\mathrm{i})\right). \tag{26.50}$$

Critically, when we perform variational inference over the joint process, our KL divergence simplifies to only include the inducing point distributions, which have covariance matrices of size P_i, resulting in $\mathcal{O}(P_\mathrm{i}^3)$ computations which are much more tractable than $\mathcal{O}(P_\mathrm{t}^3)$.

Furthermore, as shown in Hensman et al. [737], an i.i.d.likelihood function means the expected log-likelihood term decomposes into a sum over training datapoints,

$$\mathbb{E}_{q(\mathbf{f})}\left[\log p(\mathbf{y}_\mathrm{t}|\mathbf{f}_\mathrm{t})\right] = \sum_{p=1}^{P_\mathrm{t}} \mathbb{E}_{q(\mathbf{f})}\left[\log p(y_p^\mathrm{t}|f_p^\mathrm{t})\right] \tag{26.51}$$

$$= \sum_{p=1}^{P_\mathrm{t}} \mathbb{E}_{p(f_p^\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i})}\left[\log p(y_p^\mathrm{t}|f_p^\mathrm{t})\right], \tag{26.52}$$

where f_p^t denotes the pth training latent. In the second equality we point out that each expectation term does not rely on the joint $q(\mathbf{f}) = p(\mathbf{f}_\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i})$ over all training points but only the marginal $p(f_p^\mathrm{t}|\mathbf{f}_\mathrm{i})q(\mathbf{f}_\mathrm{i})$ over a single training point (which can be obtained using a similar marginalisation process to Eq. (26.39)). In other words, when using inducing points, the training latents are conditionally independent given the inducing latents, unlike in the non-sparse approximate posterior in Section 26.2.2, where the training latents are conditional on all the other training latents. Crucially, this means that each term in the sum can be estimated independently of all other datapoints, and hence we can estimate the expected log-likelihood with minibatches of data. This is similar to minibatching in neural networks, which allow training on very large datasets without exceeding device memory or requiring the entire dataset for a single step of gradient descent.

In summary, even when using the same form of approximate posterior (Eq. 26.38), we have a choice on which objective function (i.e. which ELBO) to optimise it with. Marginalising out the inducing points first and minimising the KL divergence $D_{\mathrm{KL}}(q(\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{t}|\mathbf{y}_\mathrm{t}))$ between the approximate posterior and the true posterior over the training latents can be rewritten as an ELBO where we needed to compute the KL divergence $D_{\mathrm{KL}}(q(\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{t}))$ between the marginalised approximate posterior and the prior over training latents, which is an $\mathcal{O}(P_\mathrm{t}^3)$ operation. The approach introduced by Titsias [1615] instead suggests to minimise the KL divergence $D_{\mathrm{KL}}(q(\mathbf{f}_\mathrm{t},\mathbf{f}_\mathrm{i})||p(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t}|\mathbf{y}_\mathrm{t}))$ between the approximate posterior and the true posterior over the joint inducing and training latents (and marginalise out the inducing points later), which results in a more tractable ELBO because the KL divergence between the approximate posterior and prior joint simplifies as $D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})\right) = D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i})||p(\mathbf{f}_\mathrm{i})\right)$ which is a much cheaper $\mathcal{O}(P_\mathrm{i}^3)$ operation. The expected log-likelihood term is the same in both approaches, and can be estimated using minibatching. Hence the only difference is the KL divergence used in the ELBO. For completeness, we note that, by the data processing inequality, $D_{\mathrm{KL}}(q(\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{t})) \leq D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})||p(\mathbf{f}_\mathrm{i},\mathbf{f}_\mathrm{t})\right) = D_{\mathrm{KL}}\left(q(\mathbf{f}_\mathrm{i})||p(\mathbf{f}_\mathrm{i})\right)$, and so the bound introduced in Eq. 26.42 [1615] is potentially a looser bound on the model evidence / marginal likelihood than the bound in Eq. 26.41. However, the literature shows this bound works well in practice [737, 738, 1615].

26.2.4 Deep Gaussian Processes

Deep Gaussian processes [DGPs; 361] were introduced in detail in Chapter 25; we recap the essentials here. The Deep Kernel Machines (DKMs) and Deep Kernel Processes (DKPs) we introduce later will be based on these. A DGP maps inputs $\mathbf{X} \in \mathbb{R}^{P\times N_0}$ to outputs $\mathbf{Y} \in \mathbb{R}^{P\times N_{L+1}}$ using L intermediate layers of features $\mathbf{F}^\ell \in \mathbb{R}^{P\times N_\ell}$.

Here P is the number of input points, and N_ℓ is the number of features at layer ℓ, so that N_0 is the number of input features, and N_{L+1} is the number of output features. Writing $\mathbf{F}^\ell$ and $\mathbf{Y}$ as stacks of vectors,

$$\mathbf{F}^\ell = (\mathbf{f}_1^\ell \quad \mathbf{f}_2^\ell \quad \cdots \quad \mathbf{f}_{N_\ell}^\ell) \in \mathbb{R}^{P \times N_\ell} \tag{26.53a}$$

$$\mathbf{Y} = (\mathbf{y}_1 \quad \mathbf{y}_2 \quad \cdots \quad \mathbf{y}_{N_{L+1}}) \in \mathbb{R}^{P \times N_{L+1}}, \tag{26.53b}$$

where $\mathbf{f}_\lambda^\ell \in \mathbb{R}^P$ gives the values of a particular feature λ at layer ℓ for all P input points and $\mathbf{y}_\lambda \in \mathbb{R}^P$ gives the value of a particular output λ for all P input points. The features $\mathbf{F}^1, \ldots, \mathbf{F}^L$ have i.i.d.Gaussian process priors whose covariance depends on the previous layer's features,

$$p(\mathbf{F}^\ell \mid \mathbf{F}^{\ell-1}) = \prod_{\lambda=1}^{N_\ell} \mathcal{N}(\mathbf{f}_\lambda^\ell; \mathbf{0}, \mathbf{K}(\mathbf{F}^{\ell-1})). \tag{26.54}$$

For regression the outputs are also sampled from a Gaussian process,

$$p(\mathbf{Y} \mid \mathbf{F}^L) = \prod_{\lambda=1}^{N_{L+1}} \mathcal{N}(\mathbf{y}_\lambda; \mathbf{0}, \mathbf{K}(\mathbf{F}^L) + \sigma^2 \mathbf{I}). \tag{26.55}$$

Other likelihoods could be used. For example, Milsom et al. [1179] uses a categorical likelihood with softmax probabilities, as in Eq. (26.19).

26.3 Deep Kernel Processes

The DGP defined successively alternates between (a) constructing a kernel using the features from the previous layer, and (b) sampling new features from that kernel. Remarkably, it turns out that for certain kernel functions, $\mathbf{K}(\cdot)$, we are able to reframe the DGP as a process over *kernel matrices* (a.k.a. gram matrices). We call this a Deep Kernel Process (DKP) [16, 1254]. In the intermediate layers of a DKP, there are no features — only the covariances between data points matter.

Since DKPs are equivalent to DGPs, you may be wondering why we would worry at all about replacing the features with kernel/Gram matrices. One big motivation is the large number of symmetries involved when modelling a posterior over features. The main culprit is the large number of permutation symmetries — if we swap the order of the features in a NN (or a DGP), then the model's output distribution is identical, and thus the posterior over features ought to account for this by assigning equal probability density to both feature permutations. The DKP completely sidesteps this issue by modelling the posterior over Gram matrices, which are invariant to feature permutations. To see why, let $\mathbf{G}$ be the Gram matrix formed from the features $\mathbf{F}$, then

$$\mathbf{G} = \mathbf{F}\mathbf{F}^T. \tag{26.56}$$

We can permute, or even rotate, and reflect the features, and the Gram matrix remains unchanged. This is because these operations are equivalent to multiplying the features by an orthogonal matrix $\mathbf{O}$, i.e. $\mathbf{F}' = \mathbf{F}\mathbf{O}$, where $\mathbf{O}^T\mathbf{O} = \mathbf{O}\mathbf{O}^T = \mathbf{I}$,

$$\mathbf{F}'(\mathbf{F}')^T = \mathbf{F}\mathbf{O}\mathbf{O}^T\mathbf{F}^T = \mathbf{F}\mathbf{F}^T = \mathbf{G}. \tag{26.57}$$

Full Bayesian inference is still intractable for the DKP (just like it is intractable for the DGP), but any approximate posterior we choose will by default account for a plethora of symmetries in our model. For context, even if we only think about removing the permutation symmetries, the kernel reframing removes $N!$ symmetries from our approximate posterior, where N is the number of features. Even for modestly sized networks, this is *enormously* large. A more detailed discussion on symmetries in Bayesian machine learning can be found in Chapter 32.

26.3.1 Constructing a Deep Kernel Process

Recall the DGP above, where at every layer ℓ, feature vectors $\mathbf{f}_\lambda^\ell \in \mathbb{R}^P$ are sampled i.i.d.from the kernel $\mathbf{K}(\mathbf{F}^{\ell-1})$,

$$p(\mathbf{F}^\ell | \mathbf{F}^{\ell-1}) = \prod_{\lambda=1}^{N_\ell} \mathcal{N}(\mathbf{f}_\lambda^\ell; \mathbf{0}, \mathbf{K}(\mathbf{F}^{\ell-1})). \tag{26.58}$$

We will concretely demonstrate that we can write the DGP as a kernel process. Define the rescaled Gram matrix for each layer $\mathbf{G}^\ell = \nu_\ell^{-1}\mathbf{F}^\ell(\mathbf{F}^\ell)^T$. The rescaling ensures $\mathbb{E}[\mathbf{G}^\ell] = \mathbf{K}(\mathbf{F}^{\ell-1})$. Suppose we set the kernel to be the RBF kernel, i.e. $\mathbf{K}(\mathbf{F})_{ij} = \exp(-(2\sigma^2)^{-1}\|\mathbf{f}_i - \mathbf{f}_j\|_2^2)$. Remarkably, we are able to compute the kernel at each layer in the DGP using only the Gram matrix $\mathbf{G}^\ell$, since we can write the distance between the datapoints i and j in terms of $\mathbf{G}^\ell$,

$$R_{ij}^\ell := \nu_\ell^{-1}\|\mathbf{f}_i^\ell - \mathbf{f}_j^\ell\|_2^2 = \nu_\ell^{-1}\sum_{\lambda=1}^{\nu_\ell}\left(F_{i\lambda}^\ell - F_{j\lambda}^\ell\right)^2 = \nu_\ell^{-1}\sum_{\lambda=1}^{\nu_\ell}\left((F_{i\lambda}^\ell)^2 + (F_{j\lambda}^\ell)^2 - 2F_{i\lambda}^\ell F_{j\lambda}^\ell\right) \tag{26.59}$$

$$= G_{ii}^\ell + G_{jj}^\ell - 2G_{ij}^\ell. \tag{26.60}$$

The DGP is therefore equivalent to,

$$p(\mathbf{F}^\ell|\mathbf{G}^{\ell-1}) = \prod_{\lambda=1}^{\nu_\ell}\mathcal{N}(\mathbf{f}_\lambda^\ell; \mathbf{0}, \mathbf{K}_\mathbf{G}(\mathbf{G}^{\ell-1})). \tag{26.61}$$

where,

$$\mathbf{K}_\mathbf{G}(\mathbf{G}^{\ell-1})_{ij} = \exp\left(-\frac{1}{2\sigma^2}\left(G_{ii}^{\ell-1} + G_{jj}^{\ell-1} - 2G_{ij}^{\ell-1}\right)\right). \tag{26.62}$$

Note that the property we used above was *isotropy* of the RBF kernel, and the same idea can be applied to other isotropic kernels, such as Matern (and even the ReLU kernel when we have a large number of features). Also see the subscript in $\mathbf{K}_\mathbf{G}$ used to denote that this is a kernel formed from a Gram matrix rather than features; however it is clear from context whether we are forming the kernel using a Gram matrix or with features, so we omit the subscript henceforth.

The final step is to observe that we can sample the Gram matrix $\mathbf{G}^\ell$ directly without computing the features $\mathbf{F}^\ell$ by observing that the Gram matrix is Wishart distributed [1551, 1646]. This is true by definition, since we say a matrix $\mathbf{S} \sim \mathcal{W}(\mathbf{V}, N)$ is Wishart distributed with positive-definite scale matrix $\mathbf{V}$ and degrees of freedom N if,

$$\mathbf{x}_1, \ldots, \mathbf{x}_N \sim_{\text{i.i.d.}} \mathcal{N}(\mathbf{0}, \mathbf{V}), \tag{26.63}$$

$$\sum_{i=1}^N \mathbf{x}_i\mathbf{x}_i^T = \mathbf{S} \sim \mathcal{W}(\mathbf{V}, N). \tag{26.64}$$

Care must be taken when writing the Wishart probability density function. In the case that $N < \text{rank}(\mathbf{V})$, the matrix $\mathbf{S}$ is low-rank, and one must consult the *singular* Wishart distribution [1646].

Our Gram matrix is given by $\mathbf{G}^\ell = \nu_\ell^{-1}\sum_{i=1}^{\nu_\ell}\mathbf{f}_i^\ell(\mathbf{f}_i^\ell)^T$, where $\mathbf{f}_i^\ell \sim \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}^{\ell-1}))$. Thus, the DGP prior for intermediate layers is equivalent to,

$$p(\mathbf{G}^\ell|\mathbf{G}^{\ell-1}) \sim \mathcal{W}(\mathbf{G}^\ell; \mathbf{K}(\mathbf{G}^{\ell-1})/\nu_\ell, \nu_\ell). \tag{26.65}$$

Since the distribution over the Gram matrices is Wishart, we call Eq. (26.65) a deep Wishart process [DWPs 16]. While the Wishart distribution is very natural, due to the connection with DGPs, we can imagine placing other distributions over the Gram matrices to obtain different kernel processes. For example, using the inverse Wishart distribution, gives a 'deep inverse Wishart process' (DIWP). The constraint is that the distribution must be over positive semi-definite Gram matrices.

For the top-layer, we need to map intermediate Gram matrices back to our data. For example, if data is Gaussian, then the following is a possible prior:

$$p(\mathbf{Y}|\mathbf{G}^L) = \prod_{\lambda=1}^{\nu_{L+1}}\mathcal{N}(\mathbf{y}_\lambda; \mathbf{0}, \mathbf{K}(\mathbf{G}^L) + \sigma^2\mathbf{I}). \tag{26.66}$$

Note that this is analogous to the DGP top-layer prior, Eq. (26.55), but in terms of Gram matrices rather than features.

26.3.2 Bayesian Inference in a Deep Wishart Process

Similarly to how one might use Variational Inference (VI) in a DGP, Ober and Aitchison [1250] introduced a variational inference scheme for a deep Wishart process. In the literature, approximate posteriors have been

Table 26.1: (From [1250]) ELBOs and log-likelihoods for UCI datasets [566] for a five-layer network. Significantly better results (within ELBO or LL) are bolded.

Dataset	DWP ELBO	DGP ELBO	DWP LL	DGP LL
boston	$\mathbf{-0.38 \pm 0.01}$	-0.45 ± 0.01	-2.39 ± 0.04	-2.44 ± 0.04
concrete	$\mathbf{-0.49 \pm 0.00}$	-0.50 ± 0.00	-3.13 ± 0.01	-3.14 ± 0.02
energy	$\mathbf{1.41 \pm 0.00}$	1.39 ± 0.00	-0.70 ± 0.03	-0.70 ± 0.03
kin8nm	-0.14 ± 0.00	$\mathbf{-0.13 \pm 0.00}$	$\mathbf{1.40 \pm 0.00}$	1.38 ± 0.01
naval	3.65 ± 0.07	$\mathbf{3.91 \pm 0.10}$	8.21 ± 0.05	8.31 ± 0.07
power	$\mathbf{0.03 \pm 0.00}$	0.02 ± 0.00	-2.77 ± 0.01	-2.78 ± 0.01
protein	-1.01 ± 0.00	$\mathbf{-1.00 \pm 0.00}$	-2.73 ± 0.00	-2.73 ± 0.00
wine	-1.19 ± 0.00	-1.19 ± 0.00	-0.96 ± 0.01	-0.96 ± 0.01
yacht	$\mathbf{1.65 \pm 0.01}$	1.37 ± 0.03	$\mathbf{-0.43 \pm 0.10}$	-0.88 ± 0.08

proposed for both DWPs and DIWPs [16, 1250, 1254], but in this chapter we focus on the DWP, and a generalised Wishart approximate posterior.

The goal is to perform Bayesian inference over the Gram matrices $\mathbf{G}^\ell$ in a DWP,

$$p(\mathbf{G}^\ell | \mathbf{G}^{\ell-1}) \sim \mathcal{W}(\mathbf{G}^\ell; \mathbf{K}_\mathbf{G}(\mathbf{G}^{\ell-1})/\nu_\ell, \nu_\ell). \tag{26.67}$$

Thus, we desire a flexible distribution over positive semi-definite matrices $\mathbf{G}^\ell$ to serve as our approximate posterior. This is highly unusual, and a significant departure from more usual Gaussian approximate posterior distributions we see over features. Multiple possible approximate posteriors have been proposed for the Gram matrices $\mathbf{G}$ [1250, 1250, 1254], though here we introduce the *Generalised Singular Wishart* [1254]. The idea is to parameterise $\mathbf{G}$ as,

$$\mathbf{G} = \mathbf{LAA}^T\mathbf{L}^T. \tag{26.68}$$

Here, $\mathbf{L}$ is a Cholesky decomposition of a positive definite scale matrix $\mathbf{V} = \mathbf{LL}^T$, and $\mathbf{A}$ is a matrix of independent Gamma random variables,

$$\mathbf{A} = \begin{pmatrix} A_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ A_{\nu 1} & \cdots & A_{\nu\nu} \\ \vdots & \vdots & \vdots \\ A_{P1} & \cdots & A_{P\nu} \end{pmatrix}, \tag{26.69a}$$

$$p\left(A_{jj}^2\right) = \text{Gamma}\left(A_{jj}^2; \alpha_{jj}, \beta_{jj}\right), p(A_{i>j}) = \mathcal{N}(A_{ij}; \mu_{ij}, \sigma_{ij}^2). \tag{26.69b}$$

When $\mathbf{A}$ is a tall and skinny matrix, we have that $\mathbf{AA}^T$ is singular, and therefore the distribution supports positive semi-definite, but not positive definite matrices — hence 'Generalised *Singular* Wishart'.

To actually use the Generalised Singular Wishart for VI, we need to be able to (a) sample from it and (b) evaluate its density. Sampling is straightforward; first sample $\mathbf{A}$ using Eqs. (26.69), and then use Eq. (26.68) to calculate $\mathbf{G}$. Calculating the density is very non-obvious, but [1250] showed that $\mathbf{G}$ has the following density,

$$\mathcal{GSW}(\boldsymbol{G} \mid \boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\mu}, \boldsymbol{\sigma}) = \left(\prod_{j=1}^{P} \frac{1}{L_{jj}^{\min(j,\nu)}}\right) \prod_{j=1}^{\min(P,\nu)} \frac{\text{Gamma}\left(A_{jj}^2; \alpha_{jj}, \beta_{jj}\right)}{A_{jj}^{P-j} L_{jj}^{P-j+1}} \prod_{i=j+1}^{P} \mathcal{N}(A_{ij}; \mu_{ij}, \sigma_{ij}^2). \tag{26.70}$$

By using an inducing-point version of this approximate posterior, [1250] were able to demonstrate performance improvements over similar DGP models (see Table 26.1)

26.4 Deep Kernel Machines

In this section, we introduce deep kernel machines [DKMs; 1792], a class of supervised learning models that have multiple layers of learned kernel matrices. To construct a DKM, we start with the same Wishart prior that we used for the DKP. Instead of proceeding with VI immediately however, which can become complex due to requiring

an approximate posterior over the latent Gram matrices, we first take a particular infinite width limit called the Bayesian Representation Learning Limit (BRLL). The BRLL turns out to be desirable because the posterior over Gram matrices becomes a Dirac-delta distribution, which means we can model the optimal Gram matrices as constant, and we end up learning them via gradient descent.

Before introducing the BRLL and DKMs, we first discuss standard infinite-width neural networks, NNGPs.

26.4.1 Infinite-Width Neural Network Limits

Neural networks with infinitely many features, under Gaussian priors on the weights, are equivalent to Gaussian processes with particular kernel functions [294, 1000, 1226, 1759]. In this section, we will review the so-called "Neural Network Gaussian Process" (NNGP) limit [1000] of neural networks to aid understanding of the BRLL. The infinite-width / NNGP correspondence is also briefly discussed in Chapter 31.

Consider an L-layer neural network with $N_\ell = N \cdot \nu_\ell$ hidden features at layer ℓ, and weights $\mathbf{W}^\ell \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$. We omit biases for brevity. The preactivations and postactivations are given by,

$$F^\ell_{i\lambda} = \sum_{\mu=1}^{N_\ell - 1} W^\ell_{\lambda\mu} H^{\ell-1}_{i\mu}, \qquad H^\ell_{i\lambda} = \phi\left(F^\ell_{i\lambda}\right), \qquad H^0_{i\lambda} = X_{i\lambda}. \tag{26.71}$$

We assume zero-mean random i.i.d.initialisation of the weights, with variance σ_w^2/N_ℓ. Starting at the first layer, the central limit theorem tells us that, in the limit of infinitely many features $N_1 \to \infty$, the preactivations $\mathbf{F}^1$ will be Gaussian distributed. Lee et al. [1000] formalise this argument using induction to extend it to all layers in the network, whilst de G. Matthews et al. [386] provides an alternative argument where all layers are taken to the infinite-width limit simultaneously, rather than sequentially. We omit the details here, but the result is that the preactivations $\mathbf{f}^\ell_\lambda$ have multivariate Gaussian distribution $\mathbf{f}^\ell_\lambda \sim \mathcal{N}(\mathbf{0}, \mathbf{K}^\ell)$, where the covariance matrix $\mathbf{K}^\ell \in \mathbb{R}^{P\times P}$ is given by the recursion,

$$K^\ell_{ij} = \sigma_w^2 \mathbb{E}_{\mathbf{f}^{\ell-1}_\lambda \sim \mathcal{N}(0, K^{\ell-1})} \left[\phi(F^{\ell-1}_{i\lambda})\phi(F^{\ell-1}_{j\lambda})\right]. \tag{26.72}$$

which depends on the choice of nonlinearity ϕ and the initialisation variance of the weights σ_w^2/N^ℓ. When ϕ is the ReLU nonlinearity, the recursion has a closed-form expression, the arccos kernel [294, 1000]:

$$K^\ell_{ij} = \frac{\sigma_w^2}{2\pi}\sqrt{K^{\ell-1}_{ii}K^{\ell-1}_{jj}}\left(\sin\theta^{\ell-1}_{ij} + (\pi - \theta^{\ell-1}_{ij})\cos\theta^{\ell-1}_{ij}\right) \tag{26.73}$$

$$\theta^{\ell-1}_{ij} = \arccos\left(\frac{K^{\ell-1}_{ij}}{\sqrt{K^{\ell-1}_{ii}K^{\ell-1}_{jj}}}\right). \tag{26.74}$$

Regardless of the choice of nonlinearity, the base case is for datapoints $\mathbf{x}_i \in \mathbb{R}^{N_0}$ at the beginning of the network, with weight variance σ_w^2/N_0, which gives the covariance matrix,

$$K^0_{ij} = \sigma_w^2\left(\frac{\mathbf{x}_i \cdot \mathbf{x}_j}{N_0}\right). \tag{26.75}$$

Hence, we can perform Bayesian inference with an infinite-width neural network prior by recursively computing covariances $\mathbf{K}^\ell$, and using a Gaussian process prior with covariance matrix $\mathbf{K}^L$ to perform inference on our datapoints. This is known as the "Neural Network Gaussian Process" (NNGP) limit [1000], with the kernels referred to as NNGP kernels.

NNGPs are useful because they frame neural networks as equivalent to kernel methods, for which many theoretical properties are understood (e.g. convergence properties, generalisation bounds, etc.). However, as we will see later, the fact that they simplify to fixed kernel functions means that they do not perform representation learning, which is a key strength of deep learning, and so theoretical conclusions based on NNGPs may not transfer to practical networks. Hence, in the next section, we will modify the NNGP limit so that the resulting models (deep kernel machines) can learn representations.

26.4.2 Standard infinite-width limit

We now consider the DGP analogue of the standard NNGP infinite-width limit, that is, we will take the intermediate layers to have infinite width / infinitely-many features $\mathbf{F}^\ell \in \mathbb{R}^{P\times N_\ell}$:

$$N_\ell = N\nu_\ell \quad \text{for} \quad \ell \in \{1, \ldots, L\} \quad \text{with} \quad N \to \infty. \tag{26.76}$$

The log-posterior (i.e. the log-joint plus a normalising constant) for the DGP written in terms of Gram matrices is,

$$\log p\left(\mathbf{G}^1, \ldots, \mathbf{G}^L | \mathbf{X}, \mathbf{Y}\right) = \log p(\mathbf{Y}|\mathbf{G}^L) + \sum_{\ell=1}^{L} \log p\left(\mathbf{G}^\ell | \mathbf{G}^{\ell-1}\right) + \text{const}, \tag{26.77}$$

where we emphasise that $\mathbf{G}^1, \ldots, \mathbf{G}^L$ are $P \times P$ matrix arguments chosen by us to specify where we wish to evaluate the log-posterior, and so are not affected by any infinite-width limits we take. That said, the actual value of $\log p(\mathbf{G}^\ell \mid \mathbf{G}^{\ell-1})$ is affected by the limit. In particular, in the infinite limit specified in Eq. (26.76), the log-prior term $\log p(\mathbf{G}^\ell \mid \mathbf{G}^{\ell-1})$ (Eq. 26.65) scales with N_ℓ and therefore N, so we must divide by N to obtain a finite limit:

$$\begin{aligned}\lim_{N\to\infty} \tfrac{1}{N} \log p(\mathbf{G}^\ell|\mathbf{G}^{\ell-1}) &= \tfrac{\nu_\ell}{2} \log\left|\mathbf{K}(\mathbf{G}^{\ell-1})^{-1}\mathbf{G}^\ell\right| - \tfrac{\nu_\ell}{2}\mathrm{Tr}\left(\mathbf{K}(\mathbf{G}^{\ell-1})^{-1}\mathbf{G}^\ell\right) + \lim_{N\to\infty} \tfrac{\alpha_\ell}{N} \\ &= -\nu_\ell \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\mathbf{0}, \mathbf{G}^\ell) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}^{\ell-1}))) + \text{const}. \end{aligned} \tag{26.78}$$

where $\lim_{N\to\infty} \alpha_\ell/N$ is shown to converge to a constant in Yang et al. [1792]. Note that this limit has been written as a KL-divergence between two Gaussians. By contrast, the log-likelihood term $\log p(\mathbf{Y}|\mathbf{G}^L)$ is constant w.r.t. N, so will vanish if we divide by N in the limit. Putting this all together, the limiting log-posterior (divided by N to remain finite) is

$$\lim_{N\to\infty} \tfrac{1}{N} \log p(\mathbf{G}^1, \ldots, \mathbf{G}^L|\mathbf{X}, \mathbf{Y}) = -\sum_{\ell=1}^{L} \nu_\ell \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\mathbf{0}, \mathbf{G}^\ell) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}^{\ell-1}))) + \text{const}. \tag{26.79}$$

Now, as the log-posterior scales with N, the posterior itself converges to a point distribution at its global maximum $(\mathbf{G}^1)^*, \ldots, (\mathbf{G}^L)^*$ (see Yang et al. [1792] for a more rigorous discussion of weak convergence),

$$\lim_{N\to\infty} p(\mathbf{G}^1, \ldots, \mathbf{G}^L \mid \mathbf{X}, \mathbf{Y}) = \prod_{\ell=1}^{L} \delta(\mathbf{G}^\ell - (\mathbf{G}^\ell)^*). \tag{26.80}$$

From Eq. (26.79), it is clear that the unique global maximum is obtained when $(\mathbf{G}^\ell)^* = \mathbf{K}((\mathbf{G}^{\ell-1})^*)$, since this minimises the KL divergence. In other words, we can compute the MAP (maximum a posteriori) Gram matrices recursively by applying the kernel repeatedly:

$$\mathbf{G}^\ell = \mathbf{K}(\mathbf{G}^{\ell-1}) = \underbrace{\mathbf{K}(\mathbf{K}(\cdots \mathbf{K}(\mathbf{G}^0)))}_{\ell \text{ times}}, \qquad \mathbf{G}^0 = \frac{1}{N_0}\mathbf{X}\mathbf{X}^T. \tag{26.81}$$

Importantly, this means that the limiting posterior has no dependence on $\mathbf{Y}$, our training targets, and so it is not possible for representation learning to occur. This is a big problem, since the success of modern deep learning hinges on its ability to flexibly learn good representations from data.

26.4.3 The Bayesian representation learning limit and DKMs

Representation learning was lost because the likelihood term in the normalised posterior vanished in the infinite limit, whilst the prior term did not. The Bayesian representation learning limit [1792] retains representation learning by sending the number of output features to infinity, in addition to the intermediate features:

$$N_\ell = N\nu_\ell \quad \text{for} \quad \ell \in \{1, \ldots, L+1\} \quad \text{with} \quad N \to \infty. \tag{26.82}$$

Note that the only difference between this limit and Eq. (26.76) is that we include layer $L+1$ (the output layer) in our limit. To account for an infinitely wide output layer, we use the following prior (a similar one can be used for classification),

$$p(\tilde{\mathbf{Y}}|\mathbf{G}^L) = \prod_{\lambda=1}^{N_{L+1}} \mathcal{N}(\tilde{\mathbf{y}_\lambda}; \mathbf{0}, \mathbf{K}(\mathbf{G}^L) + \sigma^2\mathbf{I}). \tag{26.83}$$

where, $\tilde{\mathbf{Y}} \in \mathbb{R}^{P \times N_{L+1}}$. Since we are taking the limit $N_{L+1} \to \infty$, one might wonder where to find data with infinitely many features. To apply the DKM to finite-width data $\mathbf{Y} \in \mathbb{R}^{P \times \nu_{L+1}}$, we let $\tilde{\mathbf{Y}}$ be formed from N copies of $\mathbf{Y}$ concatenated together, i.e.

$$\tilde{\mathbf{Y}} = \begin{pmatrix} \mathbf{Y} & \cdots & \mathbf{Y} \end{pmatrix}. \tag{26.84}$$

Note that since we assume each output is i.i.d., but apply copies of our data labels to them, we have a case of Bayesian model mismatch, and so the uncertainty quantification of the deep kernel machine may not be optimal. However, for the purposes of investigating representation learning in deep models, this approximation is good enough. Since we assumed the output channels to be i.i.d.(Eq. 26.83), the log-likelihood now scales with N:

$$\log p(\tilde{\mathbf{Y}}|\mathbf{G}^L) = N \log p(\mathbf{Y}|\mathbf{G}^L). \tag{26.85}$$

Computing the log-posterior as before, we therefore find that the likelihood no longer vanishes in the infinite limit, thus allowing representation learning:

$$\begin{aligned}\mathcal{L}(\mathbf{G}^1, \ldots, \mathbf{G}^L) &= \lim_{N\to\infty} \tfrac{1}{N} \log p(\mathbf{G}^1, \ldots, \mathbf{G}^L|\mathbf{X}, \tilde{\mathbf{Y}}) + \text{const}, \\ &= \log p(\mathbf{Y}|\mathbf{G}^L) - \sum_{\ell=1}^{L} \nu_\ell \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\mathbf{0}, \mathbf{G}^\ell) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}^{\ell-1}))).\end{aligned} \tag{26.86}$$

The limiting normalised log-posterior, $\mathcal{L}(\mathbf{G}^1, \ldots, \mathbf{G}^L)$, forms the "DKM objective", so named because it is used to optimise the Gram matrices $\mathbf{G}^\ell$ in a deep kernel machine. The first term encourages good predictions by maximising the log-likelihood of the training data $\mathbf{Y}$, whilst the other KL-divergence terms form a regulariser that encourages $\mathbf{G}^\ell$ to be close to $\mathbf{K}(\mathbf{G}^{\ell-1})$. The model reduces to the standard infinite-width limit (Eq. 26.81) when no data is observed (equivalently, when $\nu_\ell = \infty$ for all ℓ), since the KL-divergence is minimised by setting $\mathbf{G}^\ell = \mathbf{K}(\mathbf{G}^{\ell-1})$. Assuming the global maximum of this expression is unique, then the posterior is a point distribution around that maximum (Eq. 26.80). However, unlike in the NNGP case, we cannot simply compute the maximum by recursively applying the kernel function to our data, since the maximum now also depends on the likelihood term. Instead, we must optimise the Gram matrices using e.g. gradient ascent. Remarkably, given a set of optimal Gram matrices $(\mathbf{G}^1)^*, \ldots, (\mathbf{G}^L)^*$, Yang et al. [1792] showed that the posterior distribution over features is zero-mean multivariate Gaussian (assuming the global maximum for the Gram matrices is unique),

$$p(\mathbf{f}_\lambda^\ell|\mathbf{X}, \mathbf{Y}) = \mathcal{N}\left(\mathbf{f}_\lambda^\ell; \mathbf{0}, (\mathbf{G}^\ell)^*\right). \tag{26.87}$$

This fact allows *sparse* DKMs with variational inference, since a Gaussian approximate posterior can capture the true posterior. We discuss sparse DKMs in the next section, which extend the inducing-point framework from the GP literature to DKMs. When applying the BRLL to neural networks instead of DGPs, the posterior is not Gaussian, and Yang et al. [1792] needed to use normalising flows to accurately approximate the posterior, which adds significant complexity, and it is also not obvious how to integrate inducing points. As such, we stick to DKMs derived from DGPs.

Once the Gram matrices have been optimised according to the DKM objective using the training data, one can perform inference on test points using a method similar to that in DGPs. We detail the full prediction procedure for the sparse DKM in the next section, and omit the details for the full-rank DKM, which are very similar to the sparse DKM case.

26.4.4 Sparse DKMs

As with all naive kernel methods, directly computing the DKM objective for all data is $\mathcal{O}(P_\mathrm{t}^3)$, where P_t is the number of datapoints, which is typically infeasible. As a DKM is ultimately an infinite-width DGP parameterised by Gram matrices rather than features, Yang et al. [1792] developed an inducing point scheme inspired by the doubly stochastic inducing point framework for DGPs [361, 1453] to reduce the computational complexity of the DKM objective. We will first give a high level overview of the scheme, before detailing the full derivation of the sparse DKM objective, closely following the original derivation in Yang et al. [1792].

In DGPs, each layer is a GP, so we learn one set of inducing points for each layer. When using inducing points, the features at each layer are partitioned into inducing points $\mathbf{F}_\mathrm{i}^\ell$ and training points $\mathbf{F}_\mathrm{t}^\ell$. Likewise, the kernel matrix is partitioned into blocks corresponding to inducing and training points. In a DKM, we partition $\mathbf{G}^\ell$ similarly,

$$\mathbf{G}^\ell = \begin{pmatrix} \mathbf{G}_{\mathrm{ii}}^\ell & \mathbf{G}_{\mathrm{it}}^\ell \\ \mathbf{G}_{\mathrm{ti}}^\ell & \mathbf{G}_{\mathrm{tt}}^\ell \end{pmatrix} \qquad \mathbf{K}(\mathbf{G}^\ell) = \begin{pmatrix} \mathbf{K}_{\mathrm{ii}}(\mathbf{G}_{\mathrm{ii}}^\ell) & \mathbf{K}_{\mathrm{it}}(\mathbf{G}^\ell) \\ \mathbf{K}_{\mathrm{ti}}(\mathbf{G}^\ell) & \mathbf{K}_{\mathrm{tt}}(\mathbf{G}_{\mathrm{tt}}^\ell) \end{pmatrix}, \tag{26.88}$$

with $\mathbf{G}_{\mathrm{ii}}^\ell \in \mathbb{R}^{P_\mathrm{i} \times P_\mathrm{i}}$, $\mathbf{G}_{\mathrm{ti}}^\ell \in \mathbb{R}^{P_\mathrm{t} \times P_\mathrm{i}}$, and $\mathbf{G}_{\mathrm{tt}}^\ell \in \mathbb{R}^{P_\mathrm{t} \times P_\mathrm{t}}$. At each layer ℓ, $\mathbf{G}_{\mathrm{ii}}^\ell$ is learnt, whilst $\mathbf{G}_{\mathrm{ti}}^\ell$ and $\mathbf{G}_{\mathrm{tt}}^\ell$ are predicted using $\mathbf{G}_{\mathrm{ii}}^\ell$, $\mathbf{K}(\mathbf{G}^{\ell-1})$, and properties of the assumed Gaussian approximate posteriors. This results in the sparse

DKM objective,

$$\mathcal{L}_{\text{ind}}(\mathbf{G}_{\text{ii}}^1,\ldots,\mathbf{G}_{\text{ii}}^L)=\text{GP-ELBO}(\mathbf{Y}_{\text{t}};\mathbf{G}_{\text{ii}}^L,\mathbf{G}_{\text{ti}}^L,\mathbf{G}_{\text{tt}}^L)-\sum_{\ell=1}^{L}\nu_\ell \text{D}_{\text{KL}}(\mathcal{N}(\mathbf{0},\mathbf{G}_{\text{ii}}^\ell)\,||\,\mathcal{N}(\mathbf{0},\mathbf{K}(\mathbf{G}_{\text{ii}}^{\ell-1}))) \tag{26.89}$$

where GP-ELBO $\left(\mathbf{Y}_{\text{t}};\mathbf{G}_{\text{ii}}^L,\mathbf{G}_{\text{ti}}^L,\mathbf{G}_{\text{tt}}^L\right)$ is the ELBO for a sparse GP (i.e. Eq. 26.42) with data $\mathbf{Y}_{\text{t}}$ and kernel $\mathbf{K}(\mathbf{G}^L)$, including a likelihood term which measures the performance of the model on training data. Hence the sparse DKM objective looks very similar in structure to the full-rank DKM objective (Eq. 26.86). This scheme is efficient, with time complexity $\mathcal{O}(L(P_{\text{i}}^3+P_{\text{i}}^2P_{\text{t}}))$, because the KL terms in the sparse DKM objective depend only on $\mathbf{G}_{\text{ii}}\in\mathbb{R}^{P_{\text{i}}\times P_{\text{i}}}$ (not $\mathbf{G}_{\text{tt}}\in\mathbb{R}^{P_{\text{t}}\times P_{\text{t}}}$). Furthermore, obtaining predictions and computing the expected log-likelihood term in the sparse DKM objective requires only the diagonal of $\mathbf{G}_{\text{tt}}^\ell$. Since we are using an i.i.d.likelihood, all covariances $\mathbf{K}_{\text{tt}}$ and $\mathbf{G}_{\text{tt}}$ between train / test points in the batch do not affect the final prediction (covariances between datapoints are instead implicitly encoded via the inducing points $\mathbf{G}_{\text{ii}}$), and so we only ever need to compute the diagonal in the train / test blocks.

We now present the full derivation of the sparse DKM objective, which was originally based on the "Doubly Stochastic Variational Inference for Deep Gaussian Processes" scheme from Salimbeni and Deisenroth [1453]. The majority of this derivation is a modified reproduction of Appendix M (Inducing Point DKMs) in Yang et al. [1792], the primary modification of note being that we explicitly model an approximate posterior over the top-layer features $\tilde{\mathbf{F}}_{L+1}$, which were just treated as deterministic variational parameters in Yang et al. [1792].

To do inducing-point variational inference, we explicitly introduce top-layer features $\mathbf{F}^{L+1}\in\mathbb{R}^{P\times\nu_{L+1}}$ and compute the likelihood from those features. Formally, each feature, $\mathbf{f}_1^{L+1},\ldots,\mathbf{f}_{\nu_{L+1}}^{L+1}$ is i.i.d., conditioned on $\mathbf{F}^L$,

$$p(\mathbf{F}^{L+1}|\mathbf{F}^L)=\prod_{\lambda=1}^{\nu_{L+1}}\mathcal{N}(\mathbf{f}_\lambda^{L+1};\mathbf{0},\mathbf{K}(\mathbf{G}(\mathbf{F}^L))), \tag{26.90a}$$

$$p(\mathbf{Y}|\mathbf{F}^{L+1})=\prod_{\lambda=1}^{\nu_{L+1}}\mathcal{N}(\mathbf{y}_\lambda;\mathbf{f}_\lambda^{L+1},\sigma^2\mathbf{I}), \tag{26.90b}$$

where for simplicity we give the likelihood for regression, but other likelihoods (e.g. for classification) are possible. Since we are working with the DKM which uses infinite-width outputs, we also work with the features replicated N times, $\tilde{\mathbf{F}}^{L+1}\in\mathbb{R}^{P\times N_{L+1}}$, which look exactly like the features at the previous layers, but with $N_{L+1}=N\nu_{L+1}$ features, where N is the number of copies of the data we are using to approximate the infinite-width limit:

$$p(\tilde{\mathbf{F}}^{L+1}\mid\mathbf{F}_L)=\prod_{\lambda=1}^{N_{L+1}}\mathcal{N}(\tilde{\mathbf{f}}_\lambda^{L+1};\mathbf{0},\mathbf{K}(\mathbf{G}(\mathbf{F}^L))), \tag{26.91a}$$

$$p(\tilde{\mathbf{Y}}|\tilde{\mathbf{F}}^{L+1})=\prod_{\lambda=1}^{N_{L+1}}\mathcal{N}(\tilde{\mathbf{y}}_\lambda;\tilde{\mathbf{f}}_\lambda^{L+1},\sigma^2\mathbf{I}), \tag{26.91b}$$

Further, we take the total number of points, P, to be made up of P_{i} inducing points and P_{t} test/train points, so that $P=P_{\text{i}}+P_{\text{t}}$. Thus, we can separate all features, $\mathbf{F}^\ell\in\mathbb{R}^{P\times N_\ell}$, into the inducing features, $\mathbf{F}_{\text{i}}^\ell\in\mathbb{R}^{P_{\text{i}}\times N_\ell}$, and the test/train features, $\mathbf{F}_{\text{t}}^\ell\in\mathbb{R}^{P_{\text{t}}\times N_\ell}$. Likewise, we separate the inputs, $\mathbf{X}$, and outputs, $\mathbf{Y}$, into (potentially trained) inducing inputs, $\mathbf{X}_{\text{i}}$, and trained inducing outputs, $\mathbf{Y}_{\text{i}}$, and the real test/training inputs, $\mathbf{X}_{\text{t}}$, and outputs, $\mathbf{Y}_{\text{t}}$,

$$\mathbf{F}^\ell=\begin{pmatrix}\mathbf{F}_{\text{i}}^\ell\\ \mathbf{F}_{\text{t}}^\ell\end{pmatrix}\qquad\tilde{\mathbf{F}}^{L+1}=\begin{pmatrix}\tilde{\mathbf{F}}_{\text{i}}^{L+1}\\ \tilde{\mathbf{F}}_{\text{t}}^{L+1}\end{pmatrix}\qquad\mathbf{X}=\begin{pmatrix}\mathbf{X}_{\text{i}}\\ \mathbf{X}_{\text{t}}\end{pmatrix}\qquad\mathbf{Y}=\begin{pmatrix}\mathbf{Y}_{\text{i}}\\ \mathbf{Y}_{\text{t}}\end{pmatrix}\qquad\tilde{\mathbf{Y}}=\begin{pmatrix}\tilde{\mathbf{Y}}_{\text{i}}\\ \tilde{\mathbf{Y}}_{\text{t}}\end{pmatrix} \tag{26.92}$$

We follow the usual doubly stochastic inducing point approach for DGPs [1453]. In particular, we treat all the features at intermediate layers, $\mathbf{F}^1,\ldots,\mathbf{F}^L$, and the top-layer features, $\tilde{\mathbf{F}}^{L+1}$ as latent variables. The prior and approximate posterior over $\mathbf{F}^1,\ldots,\mathbf{F}^L,\tilde{\mathbf{F}}^{L+1}$ are given by,

$$q\left(\mathbf{F}^1,\ldots,\mathbf{F}^L,\tilde{\mathbf{F}}^{L+1}|\mathbf{X}\right)=q\left(\tilde{\mathbf{F}}^{L+1}|\mathbf{F}^L\right)\prod_{\ell=1}^{L}q\left(\mathbf{F}^\ell|\mathbf{F}_{\ell-1}\right), \tag{26.93a}$$

$$p(\mathbf{F}^1,\ldots,\mathbf{F}^L,\tilde{\mathbf{F}}^{L+1}|\mathbf{X})=p(\tilde{\mathbf{F}}^{L+1}|\mathbf{F}^L)\prod_{\ell=1}^{L}p(\mathbf{F}^\ell|\mathbf{F}_{\ell-1}), \tag{26.93b}$$

and $\mathbf{F}^0 = \mathbf{X}$. The prior and approximate posterior at each layer factorises into a distribution over the inducing points and a distribution over the test/train points,

$$q\left(\mathbf{F}^\ell|\mathbf{F}^{\ell-1}\right) = p(\mathbf{F}_\mathrm{t}^\ell|\mathbf{F}_\mathrm{i}^\ell, \mathbf{F}^{\ell-1}) q\left(\mathbf{F}_\mathrm{i}^\ell\right), \tag{26.94a}$$

$$p(\mathbf{F}^\ell|\mathbf{F}^{\ell-1}) = p(\mathbf{F}_\mathrm{t}^\ell|\mathbf{F}_\mathrm{i}^\ell, \mathbf{F}^{\ell-1}) p(\mathbf{F}_\mathrm{i}^\ell|\mathbf{F}_\mathrm{i}^{\ell-1}). \tag{26.94b}$$

Similarly, for the top-layer features,

$$q\left(\tilde{\mathbf{F}}^{L+1}|\mathbf{F}^L\right) = p(\tilde{\mathbf{F}}_\mathrm{t}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^{L+1}, \mathbf{F}^L) q\left(\tilde{\mathbf{F}}_\mathrm{i}^{L+1}\right), \tag{26.95a}$$

$$p(\tilde{\mathbf{F}}^{L+1}|\mathbf{F}^L) = p(\tilde{\mathbf{F}}_\mathrm{t}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^{L+1}, \mathbf{F}^L) p(\tilde{\mathbf{F}}_\mathrm{i}^{L+1}|\mathbf{F}_\mathrm{i}^L) \tag{26.95b}$$

Critically, the approximate posterior for the test/train points is the conditional prior $p(\mathbf{F}_\mathrm{t}^\ell|\mathbf{F}_\mathrm{i}^\ell, \mathbf{F}^{\ell-1})$, which is going to lead to cancellation when we compute the ELBO. Likewise, the approximate posterior over $\tilde{\mathbf{F}}_\mathrm{t}^{L+1}$ is the conditional prior,

$$q\left(\tilde{\mathbf{F}}_\mathrm{t}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^{L+1}, \mathbf{F}^L\right) = p(\tilde{\mathbf{F}}_\mathrm{t}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^{L+1}, \mathbf{F}^L). \tag{26.96}$$

Concretely, the approximate posteriors and priors over inducing points for intermediate layers are given by,

$$q\left(\mathbf{F}_\mathrm{i}^\ell\right) = \prod_\lambda^{N_\ell} \mathcal{N}(\mathbf{f}_{\mathrm{i};\lambda}^\ell; \mathbf{0}, \mathbf{G}_\mathrm{ii}^\ell), \tag{26.97a}$$

$$p(\mathbf{F}_\mathrm{i}^\ell|\mathbf{F}_\mathrm{i}^{\ell-1}) = \prod_\lambda^{N_\ell} \mathcal{N}(\mathbf{f}_{\mathrm{i};\lambda}^\ell; \mathbf{0}, \mathbf{K}(\mathbf{G}(\mathbf{F}_\mathrm{i}^{\ell-1}))) \tag{26.97b}$$

and for the top-layer, we let the approximate posterior over $\mathbf{F}_\mathrm{i}^{L+1} \in \mathbb{R}^{P_i \times \nu_{L+1}}$ learn a separate mean for each channel (we usually use the same covariance for all channels to save memory), and define the approximate posterior over the infinite-copies $\tilde{\mathbf{F}}_\mathrm{i}^{L+1} \in \mathbb{R}^{P_i \times N_{L+1}}$ by "copying" the approximate posterior for each concrete channel N times, whilst the prior assumes that the infinite copies are i.i.d.(again, this is a slightly strange thing to do, but it allows us to use the DKM to learn representations from finite-width data, as discussed in Section 26.4.3):

$$q\left(\mathbf{F}_\mathrm{i}^{L+1}\right) = \prod_\lambda^{\nu_{L+1}} \mathcal{N}(\mathbf{f}_{\mathrm{i};\lambda}^{L+1}; \boldsymbol{\mu}_\lambda, \mathbf{A}) \tag{26.98a}$$

$$q\left(\tilde{\mathbf{F}}_\mathrm{i}^{L+1}\right) = q\left(\mathbf{F}_\mathrm{i}^{L+1}\right)^N \tag{26.98b}$$

$$p(\tilde{\mathbf{F}}_\mathrm{i}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^L) = \prod_{\lambda=1}^{N_{L+1}} \mathcal{N}(\tilde{\mathbf{f}}_{\mathrm{i};\lambda}^{L+1}; \mathbf{0}, \mathbf{K}(\mathbf{G}(\tilde{\mathbf{F}}_\mathrm{i}^L))). \tag{26.98c}$$

Now we compute the ELBO (in the form of expected log-likelihood minus the KL between approximate posterior and true prior, as given in Eq. 26.42) with respect to our variational parameters

$$\mathrm{ELBO}(\mathbf{G}_\mathrm{ii}^1, \ldots, \mathbf{G}_\mathrm{ii}^L, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_{N_{L+1}}, \mathbf{A}) = \mathbb{E}_q \left[\log p(\tilde{\mathbf{Y}}_\mathrm{t}|\tilde{\mathbf{F}}_\mathrm{t}^{L+1}) + \log \frac{p(\mathbf{F}^1, \ldots \mathbf{F}^L, \tilde{\mathbf{F}}^{L+1}|\mathbf{X})}{q(\mathbf{F}^1, \ldots \mathbf{F}^L, \tilde{\mathbf{F}}^{L+1}|\mathbf{X})}\right] \tag{26.99}$$

Substituting Eq. (26.93–26.96) and cancelling the $p(\mathbf{F}_\mathrm{t}^\ell|\mathbf{F}_\mathrm{i}^\ell, \mathbf{F}^{\ell-1})$ and $p(\tilde{\mathbf{F}}_\mathrm{t}^{L+1}|\tilde{\mathbf{F}}_\mathrm{i}^{L+1}, \mathbf{F}^L)$ terms in the fraction, we obtain

$$\mathrm{ELBO}(\ldots) = \mathbb{E}_q \left[\log p(\tilde{\mathbf{Y}}_\mathrm{t}|\tilde{\mathbf{F}}_\mathrm{t}^{L+1}) + \log \frac{p(\tilde{\mathbf{F}}_\mathrm{i}^{L+1}|\mathbf{F}_\mathrm{i}^L)}{q(\tilde{\mathbf{F}}_\mathrm{i}^{L+1})} + \sum_{\ell=1}^L \log \frac{p(\mathbf{F}_\mathrm{i}^\ell|\mathbf{F}_\mathrm{i}^{\ell-1})}{q(\mathbf{F}_\mathrm{i}^\ell)}.\right] \tag{26.100}$$

In a DGP, $\mathbf{F}_\mathrm{i}^\ell$ only depends on $\mathbf{F}_\mathrm{i}^{\ell-1}$ through $\mathbf{G}_\mathrm{ii}^{\ell-1}$, and the prior and approximate posterior factorise across features λ (Eq. 26.97). Thus, in the infinite limit, individual terms in the ELBO can be written,

$$\lim_{N\to\infty} \mathbb{E}_q \left[\log \frac{p(\mathbf{F}_\mathrm{i}^\ell|\mathbf{F}_\mathrm{i}^{\ell-1})}{q(\mathbf{F}_\mathrm{i}^\ell)}\right] = N\nu_\ell \mathbb{E}_q \left[\log \frac{p(\mathbf{f}_{\mathrm{i};\lambda}^\ell|\mathbf{G}_\mathrm{ii}^{\ell-1})}{q(\mathbf{f}_{\mathrm{i};\lambda}^\ell)}\right] \tag{26.101}$$

$$= -N\nu_\ell \mathrm{D}_\mathrm{KL}(\mathcal{N}(\mathbf{0}, \mathbf{G}_\mathrm{ii}^\ell) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}_\mathrm{ii}^{\ell-1}))), \tag{26.102}$$

where the final equality arises when we notice that the expectation can be written as a negative KL-divergence. Similarly, for the top-layer features, recalling Eq. 26.98, we have

$$\lim_{N\to\infty} \mathbb{E}_q \left[\log \frac{p(\tilde{\mathbf{F}}_{\mathrm{i}}^{L+1} | \mathbf{F}_{\mathrm{i}}^{L})}{q(\tilde{\mathbf{F}}_{\mathrm{i}}^{L+1})} \right] = N \sum_{\lambda=1}^{\nu_{L+1}} \mathbb{E}_q \left[\log \frac{p(\mathbf{f}_{\mathrm{i};\lambda}^{L+1} | \mathbf{G}_{\mathrm{ii}}^{L})}{q(\mathbf{f}_{\mathrm{i};\lambda}^{L+1})} \right] \tag{26.103}$$

$$= -N \sum_{\lambda=1}^{\nu_{L+1}} \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\boldsymbol{\mu}_\lambda, \mathbf{A}) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}_{\mathrm{ii}}^{L}))). \tag{26.104}$$

Note that for the top layer we have a sum of ν_ℓ terms rather than just a factor of ν_ℓ as in the intermediate layer terms, since we have a separate μ_λ in the approximate posterior for each feature λ. Finally, we note that since our data labels $\tilde{\mathbf{Y}}_{\mathrm{t}}$ are replicated N times, and the top-layer features $\tilde{\mathbf{F}}_{\mathrm{t}}^{L+1}$ are also replicated N times in the same way, the expected log-likelihood term can be written as

$$\mathbb{E}_q \left[\log p(\tilde{\mathbf{Y}}_{\mathrm{t}} | \tilde{\mathbf{F}}_{\mathrm{t}}^{L+1}) \right] = N \mathbb{E}_q \left[\log p(\mathbf{Y}_{\mathrm{t}} | \mathbf{F}_{\mathrm{t}}^{L+1}) \right] \tag{26.105}$$

The inducing DKM objective, $\mathcal{L}_{\mathrm{ind}}$, which is the ELBO divided by N to ensure that it remains finite in the infinite limit, can therefore be written as

$$\begin{aligned}
&\mathcal{L}_{\mathrm{ind}}(\mathbf{F}_{\mathrm{i}}^{L+1}, \mathbf{G}_{\mathrm{ii}}^{1}, \ldots, \mathbf{G}_{\mathrm{ii}}^{L}) \\
&= \lim_{N\to\infty} \tfrac{1}{N} \mathrm{ELBO}(\mathbf{F}_{\mathrm{i}}^{L+1}, \mathbf{G}_{\mathrm{ii}}^{1}, \ldots, \mathbf{G}_{\mathrm{ii}}^{L}) \\
&= \underbrace{\mathbb{E}_q \left[\log p(\mathbf{Y}_{\mathrm{t}} | \mathbf{F}_{\mathrm{t}}^{L+1}) \right] - \sum_{\lambda=1}^{N_{L+1}} \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\boldsymbol{\mu}_\lambda, \mathbf{A}_\lambda) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}_{\mathrm{ii}}^{L})))}_{\text{Sparse GP ELBO}} \\
&\quad \underbrace{- \sum_{\ell=1}^{L} \nu_\ell \mathrm{D}_{\mathrm{KL}}(\mathcal{N}(\mathbf{0}, \mathbf{G}_{\mathrm{ii}}^{\ell}) \,||\, \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{G}_{\mathrm{ii}}^{\ell-1})))}_{\text{DKM Regularisation Terms}} .
\end{aligned} \tag{26.106}$$

Note that this has almost exactly the same form as the standard DKM objective for DGPs (Eq. 26.89). In particular, the first term is a "performance" term that here depends on the quality of the predictions given the inducing points, and the second and third terms are a chain of KL-divergences, with the only difference that these KL-divergences apply only to the inducing points, and we have to deal with the top-layer separately due to the approximate posterior.

Now that we have a simple form for the ELBO, we need to compute the expected likelihood, $\mathbb{E}_q \left[\log p(\mathbf{Y}_{\mathrm{t}} | \mathbf{F}_{\mathrm{t}}^{L+1}) \right]$. Since the top layer is a sparse GP, computing the expected log-likelihood term in the sparse DKM objective is similar to Section 26.2.3. For Gaussian likelihoods, we can compute the expected log-likelihood without a Monte-Carlo estimate, along with the concrete formula used for the Categorical likelihood. This relies on us having the last full Gram matrix $\mathbf{G}^L$, which depends on previous layers, and therefore requires us to compute the full Gram matrices, including test/train point blocks, conditioned on the optimised inducing Gram matrices $\mathbf{G}_{\mathrm{ii}}^{\ell}$ at every layer. Once we have the last full Gram matrix $\mathbf{G}^L$, we can compute the expected log-likelihood as in a sparse GP.

We start by recapping the definition of the full Gram matrix,

$$\mathbf{G}^{\ell} = \begin{pmatrix} \mathbf{G}_{\mathrm{ii}}^{\ell} & \mathbf{G}_{\mathrm{it}}^{\ell} \\ \mathbf{G}_{\mathrm{ti}}^{\ell} & \mathbf{G}_{\mathrm{tt}}^{\ell} \end{pmatrix} \tag{26.107}$$

for both inducing points (labelled 'i') and test/training points (labelled 't'). For clarity, we have $\mathbf{G}^{\ell} \in \mathbb{R}^{P\times P}$, $\mathbf{G}_{\mathrm{ii}}^{\ell} \in \mathbb{R}^{P_{\mathrm{i}}\times P_{\mathrm{i}}}$, $\mathbf{G}_{\mathrm{ti}}^{\ell} \in \mathbb{R}^{P_{\mathrm{t}}\times P_{\mathrm{i}}}$, $\mathbf{G}_{\mathrm{tt}}^{\ell} \in \mathbb{R}^{P_{\mathrm{t}}\times P_{\mathrm{t}}}$, where P_{i} is the number of inducing points, P_{t} is the number of train/test points and $P = P_{\mathrm{i}} + P_{\mathrm{t}}$ is the total number of inducing and train/test points.

The conditional distribution over $\mathbf{F}_{\mathrm{t}}^{\ell}$ given $\mathbf{F}_{\mathrm{i}}^{\ell}$ is,

$$p(\mathbf{F}_{\mathrm{t}}^{\ell} \mid \mathbf{F}_{\mathrm{i}}^{\ell}, \mathbf{G}_{\ell-1}) = \prod_{\lambda=1}^{\nu_\ell} \mathcal{N}(\mathbf{f}_{\mathrm{t};\lambda}^{\ell}; \mathbf{K}_{\mathrm{ti}} \mathbf{K}_{\mathrm{ii}}^{-1} \mathbf{f}_{\mathrm{i};\lambda}^{\ell}, \mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}}) \tag{26.108}$$

where $\mathbf{f}_{\mathrm{t};\lambda}^{\ell}$ is the λth feature for all train/test points, $\mathbf{f}_{\mathrm{i};\lambda}^{\ell}$ is the λth feature for all inducing points, and

$$\begin{pmatrix} \mathbf{K}_{\mathrm{ii}} & \mathbf{K}_{\mathrm{ti}}^T \\ \mathbf{K}_{\mathrm{ti}} & \mathbf{K}_{\mathrm{tt}} \end{pmatrix} = \mathbf{K}(\tfrac{1}{N_{\ell-1}} \mathbf{F}^{\ell-1} (\mathbf{F}^{\ell-1})^T) = \mathbf{K}(\mathbf{G}_{\ell-1}) \tag{26.109}$$

$$\mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}} = \mathbf{K}_{\mathrm{tt}} - \mathbf{K}_{\mathrm{ti}} \mathbf{K}_{\mathrm{ii}}^{-1} \mathbf{K}_{\mathrm{ti}}^T. \tag{26.110}$$

Algorithm 27: DKM prediction

Inputs : parameters $\{\nu_\ell\}_{\ell=1}^L$; optimised Gram matrices $\{\mathbf{G}_{\mathrm{ii}}^\ell\}_{\ell=1}^L$; inducing and train/test inputs $\mathbf{X}_\mathrm{i} \in \mathbb{R}^{P_\mathrm{i} \times \nu_0}, \mathbf{X}_\mathrm{t} \in \mathbb{R}^{P_\mathrm{t} \times \nu_0}$; inducing output means $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_{N_{L+1}} \in \mathbb{R}^{P_\mathrm{i}}$ and covariance $\mathbf{A} \in \mathbb{R}^{P_\mathrm{i} \times P_\mathrm{i}}$

Outputs: predicted outputs $\mathbf{F}_\mathrm{t}^{L+1}$

`// Initialise full Gram matrix`

$$\begin{pmatrix} \mathbf{G}_{\mathrm{ii}}^0 & \mathbf{G}_{\mathrm{ti}}^{0;T} \\ \mathbf{G}_{\mathrm{ti}}^0 & \mathbf{G}_{\mathrm{tt}}^0 \end{pmatrix} = \tfrac{1}{\nu_0} \begin{pmatrix} \mathbf{X}_\mathrm{i}\mathbf{X}_\mathrm{i}^T & \mathbf{X}_\mathrm{i}\mathbf{X}_\mathrm{t}^T \\ \mathbf{X}_\mathrm{t}\mathbf{X}_\mathrm{i}^T & \mathbf{X}_\mathrm{t}\mathbf{X}_\mathrm{t}^T \end{pmatrix}$$

`// Propagate full Gram matrix`

for $\ell = 1$ **to** L **do**

$$\begin{pmatrix} \mathbf{K}_{\mathrm{ii}} & \mathbf{K}_{\mathrm{ti}}^T \\ \mathbf{K}_{\mathrm{ti}} & \mathbf{K}_{\mathrm{tt}} \end{pmatrix} = \mathbf{K}\left(\begin{pmatrix} \mathbf{G}_{\mathrm{ii}}^{\ell-1} & (\mathbf{G}_{\mathrm{ti}}^{\ell-1})^T \\ \mathbf{G}_{\mathrm{ti}}^{\ell-1} & \mathbf{G}_{\mathrm{tt}}^{\ell-1} \end{pmatrix}\right)$$

$\mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}} \leftarrow \mathbf{K}_{\mathrm{tt}} - \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{ti}}^T$

$\mathbf{G}_{\mathrm{ti}}^\ell \leftarrow \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{G}_{\mathrm{ii}}^\ell$

$\mathbf{G}_{\mathrm{tt}}^\ell \leftarrow \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{G}_{\mathrm{ii}}^\ell\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{ti}}^T + \mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}}$

`// Final prediction using Gaussian process expressions`

$$\begin{pmatrix} \mathbf{K}_{\mathrm{ii}} & \mathbf{K}_{\mathrm{ti}}^T \\ \mathbf{K}_{\mathrm{ti}} & \mathbf{K}_{\mathrm{tt}} \end{pmatrix} = \mathbf{K}\left(\begin{pmatrix} \mathbf{G}_{\mathrm{ii}}^L & (\mathbf{G}_{\mathrm{ti}}^L)^T \\ \mathbf{G}_{\mathrm{ti}}^L & \mathbf{G}_{\mathrm{tt}}^L \end{pmatrix}\right)$$

$$\mathbf{f}_{\mathrm{t};\lambda}^{L+1} \sim \mathcal{N}(\mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\boldsymbol{\mu}_\lambda, \mathbf{K}_{\mathrm{tt}} - \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{it}} + \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{A}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{it}})$$

In the infinite limit, the intermediate layer Gram matrices become deterministic via the law of large numbers:

$$\lim_{N\to\infty} \mathbf{G}^\ell = \lim_{N\to\infty} \tfrac{1}{N_\ell}\mathbf{F}^\ell(\mathbf{F}^\ell)^T = \lim_{N\to\infty} \tfrac{1}{N_\ell}\sum_{\lambda=1}^{N_\ell} \mathbf{f}_\lambda^\ell(\mathbf{f}_\lambda^\ell)^T = \mathbb{E}_q\left[\mathbf{f}_\lambda^\ell(\mathbf{f}_\lambda^\ell)^T\right] \tag{26.111}$$

(where the third equality follows since we are averaging an infinite number of i.i.d.terms, and q is defined as in Eq. 26.93 and Eq. 26.97) and as such $\mathbf{G}_{\mathrm{it}}$ and $\mathbf{G}_{\mathrm{tt}}$ become deterministic and equal to their expected values. Using Eq. (26.108), we can write,

$$\mathbf{F}_\mathrm{t}^\ell = \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{F}_\mathrm{i}^\ell + \mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}}^{1/2}\mathbf{Z}_\mathrm{i}. \tag{26.112}$$

where $\mathbf{Z}_\mathrm{i}$ is a matrix with i.i.d.standard Gaussian elements. Hence, $\mathbf{G}_{\mathrm{it}}$ and $\mathbf{G}_{\mathrm{tt}}$ can be written as

$$\begin{aligned} \mathbf{G}_{\mathrm{ti}}^\ell &= \tfrac{1}{N_\ell}\mathbb{E}\left[\mathbf{F}_\mathrm{t}^\ell(\mathbf{F}_\mathrm{i}^\ell)^T\right] && (26.113)\\ &= \tfrac{1}{N_\ell}\mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbb{E}\left[\mathbf{F}_\mathrm{i}^\ell(\mathbf{F}_\mathrm{i}^\ell)^T\right] && (26.114)\\ &= \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{G}_{\mathrm{ii}}^\ell && (26.115) \end{aligned}$$

and

$$\begin{aligned} \mathbf{G}_{\mathrm{tt}}^\ell &= \tfrac{1}{N_\ell}\mathbb{E}\left[\mathbf{F}_\mathrm{t}^\ell(\mathbf{F}_\mathrm{t}^\ell)^T\right] && (26.116)\\ &= \tfrac{1}{N_\ell}\mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbb{E}\left[\mathbf{F}_\mathrm{i}^\ell(\mathbf{F}_\mathrm{i}^\ell)^T\right]\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{ti}}^T + \tfrac{1}{N_\ell}\mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}}^{1/2}\mathbb{E}\left[\mathbf{Z}_\mathrm{i}\mathbf{Z}_\mathrm{i}^T\right]\mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}}^{1/2} && (26.117)\\ &= \mathbf{K}_{\mathrm{ti}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{G}_{\mathrm{ii}}\mathbf{K}_{\mathrm{ii}}^{-1}\mathbf{K}_{\mathrm{ti}}^T + \mathbf{K}_{\mathrm{tt}\cdot\mathrm{i}} && (26.118) \end{aligned}$$

For the full prediction algorithm, see Alg. 27.

Chapter 27

Meta-learning deep kernel Gaussian processes

Although the deep kernel learning method introduced in Chapter 24 regularises deep kernels with type-II maximum likelihood estimation, it still tends to overfit to small datasets [1253]. This chapter introduces *Adaptive Deep Kernel Fitting with Implicit Function Theorem* (ADKF-IFT) [275], a general framework for learning deep kernel Gaussian processes on low-data tasks by interpolating between meta-learning and conventional deep kernel learning. ADKF-IFT employs a bilevel optimisation objective where generally useful feature representations are meta-learned across tasks, such that task-specific Gaussian process models estimated on top of such features achieve the lowest possible predictive loss on average. The resulting nested optimisation problem is solved using the implicit function theorem. The ADKF-IFT framework provably contains Deep Kernel Learning and Deep Kernel Transfer as special cases. Although ADKF-IFT is a completely general method, it is especially well-suited for drug discovery problems due to their low-data nature, significantly outperforming previous state-of-the-art methods on a variety of real-world few-shot molecular property prediction tasks and out-of-domain molecular property prediction and optimisation tasks.

27.1 Introduction

Many real-world applications require machine learning algorithms to make robust predictions with well-calibrated uncertainty given very limited training data. One important example is drug discovery, where practitioners not only want models to accurately predict biochemical and physicochemical properties of molecules, but also want to use models to guide the search for novel molecules with desirable properties, leveraging techniques such as Bayesian optimisation (BO) which heavily rely on accurate uncertainty estimates [557]. Despite the meteoric rise of neural networks over the past decade, their notoriously overconfident and unreliable uncertainty estimates [1582] make them generally ineffective surrogate models for BO. Instead, most contemporary BO implementations use Gaussian processes (GPs) [1382] as surrogate models due to their analytically tractable and generally reliable uncertainty estimates, even on small datasets.

Traditionally, GPs are fit on hand-engineered features (e.g., molecular fingerprints), which can limit their predictive performance on complex, structured, high-dimensional data such as molecules where designing informative features is challenging. Naturally, a number of works have proposed to improve performance by instead fitting GPs on features learned by a deep neural network, a family of models generally called Deep Kernel GPs as introduced in Chapter 24. However, there is no clear consensus about how to train these models on low-data tasks: maximising the GP marginal likelihood [750, 1769] has been shown to overfit on small datasets [1253], while meta-learning [1298] and fully-Bayesian approaches [1253] avoid this at the cost of making strong, often unrealistic assumptions.

This chapter presents a general framework called *Adaptive Deep Kernel Fitting with Implicit Function Theorem* (ADKF-IFT) [275] for training deep kernel GPs, which is especially well-suited to small datasets. ADKF-IFT essentially trains a subset of the model parameters with a meta-learning loss, and separately adapts the remaining parameters on each task using maximum marginal likelihood estimation. In contrast to previous methods which use a single loss for all parameters, ADKF-IFT is able to utilise the implicit regularisation of meta-learning to prevent overfitting while avoiding the strong assumptions of a pure meta-learning approach which may lead to underfitting.

The structure of this chapter is as follows. Section 27.3.1 presents the general ADKF-IFT framework and its

natural formulation as a bilevel optimisation problem. Section 27.3.2 explains how the implicit function theorem (IFT) can be used to efficiently solve the bilevel optimisation problem with gradient-based methods in a few-shot learning setting. Section 27.3.3 shows how ADKF-IFT can be viewed as a *generalisation* and *unification* of previous approaches based purely on single-task learning [1769] or purely on meta-learning [1298] for training deep kernel GPs. This chapter will focus on a specific practical instantiation of ADKF-IFT wherein all feature extractor parameters are meta-learned, which has a clear interpretation and obviates the need for any Hessian approximations. Section 27.3.4 argues why this particular instantiation is well-suited to retain the best properties of previously proposed methods. Motivated by the general demand for better GP models in chemistry, Section 27.5 presents an extensive empirical evaluation of ADKF-IFT on several chemical tasks, finding that it significantly improves upon previous state-of-the-art methods.

27.2 Preliminaries

27.2.1 Meta-learning and few-shot learning

Few-shot learning [1681] refers to learning on many related tasks when each task has few labelled examples [961, 1178]. It requires learning algorithms to be able to adapt to unseen tasks given few labelled examples. Few-shot learning algorithms often benefit from knowledge transfer among related low-data tasks, whereas single-task learning algorithms can easily overfit or underfit in this setting. In the few-shot learning setup, one is given a set of training tasks $\mathcal{D} = \{\mathcal{T}_t\}_{t=1}^T$ (a *meta-dataset*) and some unseen test tasks $\mathcal{D}_* = \{\mathcal{T}_*\}$. Each task $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{N_\mathcal{T}}$ is a set of points in the domain $\mathcal{X}$ (e.g., space of molecules) with corresponding labels (continuous, categorical, etc.), and is partitioned into a *support set* $\mathcal{S}_\mathcal{T} \subseteq \mathcal{T}$ for meta-training and a *query set* $\mathcal{Q}_\mathcal{T} = \mathcal{T} \setminus \mathcal{S}_\mathcal{T}$ for meta-testing. Typically, the total number of training tasks $|\mathcal{D}|$ is large, while the size of each support set $|\mathcal{S}_\mathcal{T}|$ is small. Models for few-shot learning are typically trained to accurately predict unseen examples in the query set $\mathcal{Q}_\mathcal{T}$ given labelled examples in the support set $\mathcal{S}_\mathcal{T}$ for all tasks $\mathcal{T} \in \mathcal{D}$ during a *meta-training* phase, then evaluated by their prediction error on $\mathcal{Q}_{\mathcal{T}_*}$ given $\mathcal{S}_{\mathcal{T}_*}$ for unseen test tasks $\mathcal{T}_* \in \mathcal{D}_*$ during a *meta-testing* phase.

27.2.2 Deep kernel Gaussian processes

As introduced in Chapter 24, *Deep kernel* Gaussian processes are GPs whose covariance function is constructed by first using a neural network *feature extractor* f_ϕ with parameters $\phi \in \Phi$ to create feature representations $h = f_\phi(x), h' = f_\phi(x')$ of the input points x, x', then feeding these feature representations into a standard *base kernel* $c_\theta(h, h')$ (e.g., an RBF kernel) [202, 226, 750, 1766, 1769]. The complete covariance function is therefore $k_\psi(x, x') \coloneqq c_\theta(f_\phi(x), f_\phi(x'))$ with learnable parameters $\psi \coloneqq (\theta, \phi)$. Deep kernel GPs can be viewed as performing Bayesian inference for the last linear layer of a neural network in the function space [709, 988, 1252, 1729].

The *Deep Kernel Learning* (DKL) [1769] method introduced in Chapter 24 is a single-task method for fitting a deep kernel to a dataset. DKL jointly fits both the feature extractor parameters ϕ and base kernel parameters θ by maximising the GP marginal likelihood on a single dataset (i.e., DKL essentially fits a neural network with a GP "head" to a dataset). It is well known that neural networks will easily overfit to small datasets [1462]. This overfitting also happens in DKL, despite the fact that it fits the neural network parameters using a type-II maximum likelihood approach: although early DKL papers suggested that the "model complexity" term (as measured by the log determinant of the kernel matrix) in the GP marginal likelihood objective would prevent this overfitting from happening [1769], recent follow-up work showed that this is not the case [1253] as a deep-kernel GP can simultaneously overfit to the training data and appear to have a low "model complexity".

Deep Kernel Transfer (DKT) [1298] is a meta-learning method for fitting a deep kernel to a distribution of datasets. DKT jointly fits both the feature extractor parameters ϕ and base kernel parameters θ by maximising the expected GP marginal likelihood over a distribution of datasets (i.e., a meta-dataset). To mitigate the overfitting issue of DKL using meta-learning, DKT makes a very strong assumption that different tasks in the task distribution are drawn from an identical GP prior over functions. Explicitly, this means that the data generating process is assumed to have the same noise level, same amplitude, and same "characteristic lengthscale" for every task in the meta-dataset, which is a very restrictive assumption violated by most real-world problems. Taking drug discovery as an example, the drug properties in different datasets in a meta-dataset may have (1) highly varying noise levels, so modelling all tasks with the same amount of observation noise will not be realistic; (2) different output ranges and units for regression: for example, one task might have data in the range 1-20 μM, while another might have 0-100% inhibition, meaning that a single signal variance (kernel amplitude) will not model the data well; (3) different "characteristic lengthscales": for some tasks, structurally similar molecules have very strongly correlated output labels, while for other tasks it is much weaker (i.e., there is much more variation in the labels of very

similar molecules), suggesting that the "characteristic lengthscale" will be different. Inevitably, trying to fit such a mis-specified model will result in a set of compromised base kernel parameters θ which fit all datasets fine on average but do not fit each individual dataset very well. This is the underfitting issue of DKT.

27.3 Adaptive Deep Kernel Fitting with Implicit Function Theorem

Let Θ and Φ respectively be the sets of base kernel and feature extractor parameters for a deep kernel GP. Denote the set of all parameters by $\Psi \equiv \Theta \times \Phi$.

27.3.1 The general ADKF-IFT framework with bilevel optimisation

The key idea of the general ADKF-IFT framework is to partition the parameters into two disjoint subsets $\Psi = \Psi_{\text{adapt}} \times \Psi_{\text{meta}}$, where only a subset of the parameters $\psi_{\text{adapt}} \in \Psi_{\text{adapt}}$ will be adapted to each individual task by minimising a train loss $\mathcal{L}_T$, with the remaining set of parameters $\psi_{\text{meta}} \in \Psi_{\text{meta}}$ meta-learned during a *meta-training* phase to yield the best possible validation loss $\mathcal{L}_V$ *on average* over many related training tasks (*after* ψ_{meta} is separately adapted to each of these tasks). This can be naturally formalised as the following *bilevel optimisation* problem:

$$\psi^*_{\text{meta}} = \arg\min_{\psi_{\text{meta}}} \mathbb{E}_{p(\mathcal{T})}[\mathcal{L}_V(\psi_{\text{meta}}, \psi^*_{\text{adapt}}(\psi_{\text{meta}}, \mathcal{S}_\mathcal{T}), \mathcal{T})], \tag{27.1}$$

$$\text{such that} \quad \psi^*_{\text{adapt}}(\psi_{\text{meta}}, \mathcal{S}_\mathcal{T}) = \arg\min_{\psi_{\text{adapt}}} \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_\mathcal{T}). \tag{27.2}$$

Equations 27.1 and 27.2 are most easily understood by separately considering the meta-learned parameters ψ_{meta} and the task-specific parameters ψ_{adapt}. For a given task $\mathcal{T}$ and an arbitrary value for the meta-learned parameters ψ_{meta}, in Equation 27.2 the task-specific parameters ψ_{adapt} are chosen to minimise the *train loss* $\mathcal{L}_T$ evaluated on the task's support set $\mathcal{S}_\mathcal{T}$. That is, ψ_{adapt} is *adapted* to the support set $\mathcal{S}_\mathcal{T}$ of the task $\mathcal{T}$, with the aim of producing the best possible model on $\mathcal{S}_\mathcal{T}$ for the given value of ψ_{meta}. The result is a model with optimal task-specific parameters $\psi^*_{\text{adapt}}(\psi_{\text{meta}}, \mathcal{S}_\mathcal{T})$ for the given meta-learned parameters ψ_{meta} and task $\mathcal{T}$. The remaining question is how to choose a value for the meta-learned parameters ψ_{meta}, knowing that ψ_{adapt} will be adapted separately to each task. In Equation 27.1, the optimal meta-learned parameters ψ^*_{meta} is obtained by minimising the expected *validation loss* $\mathcal{L}_V$ over a distribution of training tasks $p(\mathcal{T})$. There are two reasons for this choice as follows:

1. On any given task $\mathcal{T}$, the validation loss usually reflects the performance metric of interest on the query set $\mathcal{Q}_\mathcal{T}$ of $\mathcal{T}$ (e.g., the prediction error).
2. Since the same value of ψ_{meta} will be used for all tasks, it makes sense to choose a value whose *expected performance* is good across many tasks drawn from $p(\mathcal{T})$. That is, ψ_{meta} is chosen such that a GP achieves the lowest possible average validation loss on the query set $\mathcal{Q}_\mathcal{T}$ of a random training task $\mathcal{T} \sim p(\mathcal{T})$ after ψ_{adapt} is adapted to the task's support set $\mathcal{S}_\mathcal{T}$.

In practice, ψ_{meta} would be optimised during a *meta-training* phase using a set of training tasks $\mathcal{D}$ to approximate Equation 27.1. After meta-training (i.e., at meta-test time), the joint GP posterior predictive distribution is used to make predictions for each unseen test task $\mathcal{T}_*$ with optimal parameters ψ^*_{meta} and $\psi^*_{\text{adapt}}(\psi^*_{\text{meta}}, \mathcal{S}_{\mathcal{T}_*})$:

$$p(\mathcal{Q}^y_{\mathcal{T}_*} | \mathcal{Q}^x_{\mathcal{T}_*}, \mathcal{S}_{\mathcal{T}_*}, \psi^*_{\text{meta}}, \psi^*_{\text{adapt}}(\psi^*_{\text{meta}}, \mathcal{S}_{\mathcal{T}_*})). \tag{27.3}$$

Note that the description above does not specify a particular choice of $\Psi_{\text{meta}}, \Psi_{\text{adapt}}, \mathcal{L}_T, \mathcal{L}_V$. This is intentional, as there are many reasonable choices for these quantities. Because of this, ADKF-IFT should be considered a *general framework*, with a particular choice for these being an *instantiation* of the ADKF-IFT framework. Examples of this are provided in Sections 27.3.3 and 27.3.4.

27.3.2 Efficient meta-training algorithm for ADKF-IFT

In general, optimising bilevel optimisation objectives such as Equation 27.1 is computationally complex, mainly because each evaluation of the objective requires solving a separate inner optimisation problem (Equation 27.2).

Algorithm 28: Exact hypergradient computation in ADKF-IFT

Input : training task $\mathcal{T}'$; current meta-learned parameters ψ'_{meta}
Output: hypergradient $\frac{\mathrm{d}\mathcal{L}_V}{\mathrm{d}\psi_{\text{meta}}}$ for $\mathcal{T}'$ evaluated at ψ'_{meta}
solve Equation 27.2 to obtain $\psi'_{\text{adapt}} = \psi^*_{\text{adapt}}(\psi'_{\text{meta}}, \mathcal{S}_{\mathcal{T}'})$
compute $g_1 = \frac{\partial \mathcal{L}_V(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{T}')}{\partial \psi_{\text{meta}}}\Big|_{\psi'_{\text{meta}}, \psi'_{\text{adapt}}}$ and $g_2 = \frac{\partial \mathcal{L}_V(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{T}')}{\partial \psi_{\text{adapt}}}\Big|_{\psi'_{\text{meta}}, \psi'_{\text{adapt}}}$
compute Hessian $H = \frac{\partial^2 \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_{\mathcal{T}'})}{\partial \psi_{\text{adapt}} \partial \psi_{\text{adapt}}^\top}\Big|_{\psi'_{\text{meta}}, \psi'_{\text{adapt}}}$
solve the linear system $vH = g_2$ for v
compute mixed partial derivatives $P = \frac{\partial^2 \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_{\mathcal{T}'})}{\partial \psi_{\text{adapt}} \partial \psi_{\text{meta}}^\top}\Big|_{\psi'_{\text{meta}}, \psi'_{\text{adapt}}}$
compute hypergradient $\frac{\mathrm{d}\mathcal{L}_V}{\mathrm{d}\psi_{\text{meta}}} = g_1 - vP$

Although calculating the *hypergradient* (i.e., total derivative) of the validation loss $\mathcal{L}_V$ with respect to the meta-learned parameters ψ_{meta} would allow Equation 27.1 to be solved with gradient-based optimisation:

$$\frac{\mathrm{d}\mathcal{L}_V}{\mathrm{d}\psi_{\text{meta}}} = \frac{\partial \mathcal{L}_V}{\partial \psi_{\text{meta}}} + \frac{\partial \mathcal{L}_V}{\partial \psi^*_{\text{adapt}}} \frac{\partial \psi^*_{\text{adapt}}}{\partial \psi_{\text{meta}}}, \tag{27.4}$$

Equation 27.4 reveals that this requires calculating $\frac{\partial \psi^*_{\text{adapt}}}{\partial \psi_{\text{meta}}}$, i.e., how the optimal task-specific parameters $\psi^*_{\text{adapt}}(\psi_{\text{meta}}, \mathcal{S}_{\mathcal{T}})$ change with respect to the meta-learned parameters ψ_{meta}. Calculating this naively with automatic differentiation platforms would require tracking the gradient through many iterations of the inner optimisation (Equation 27.2), which in practice requires too much memory to be feasible. Fortunately, because ψ^*_{adapt} is an optimum of the train loss $\mathcal{L}_T$, Cauchy's *Implicit Function Theorem* (IFT) provides a formula for calculating $\frac{\partial \psi^*_{\text{adapt}}}{\partial \psi_{\text{meta}}}$ for an arbitrary value of the meta-learned parameters ψ'_{meta} and a given task $\mathcal{T}'$:

$$\frac{\partial \psi^*_{\text{adapt}}}{\partial \psi_{\text{meta}}}\Big|_{\psi'_{\text{meta}}} = -\left(\frac{\partial^2 \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_{\mathcal{T}'})}{\partial \psi_{\text{adapt}} \partial \psi_{\text{adapt}}^\top}\right)^{-1} \frac{\partial^2 \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_{\mathcal{T}'})}{\partial \psi_{\text{adapt}} \partial \psi_{\text{meta}}^\top}\Big|_{\psi'_{\text{meta}}, \psi'_{\text{adapt}}}, \tag{27.5}$$

where $\psi'_{\text{adapt}} = \psi^*_{\text{adapt}}(\psi'_{\text{meta}}, \mathcal{S}_{\mathcal{T}'})$. The only potential problem with Equation 27.5 is the computation and inversion of the Hessian matrix $\frac{\partial^2 \mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_{\mathcal{T}})}{\partial \psi_{\text{adapt}} \partial \psi_{\text{adapt}}^\top}$. This computation can be done exactly if $|\Psi_{\text{adapt}}|$ is small, which is the case considered in this chapter (as will be discussed in Section 27.3.4). Otherwise, an approximation to the inverse Hessian (e.g., Neumann approximation [313, 1080]) could be used, which reduces both the memory and computational complexities to $\mathcal{O}(|\Psi|)$. Combining Equations 27.4 and 27.5, Algorithm 28 summarises a recipe for computing the hypergradient $\frac{\mathrm{d}\mathcal{L}_V}{\mathrm{d}\psi_{\text{meta}}}$ exactly for a single task. The meta-learned parameters ψ_{meta} can then be updated with the expected hypergradient over $p(\mathcal{T})$.

27.3.3 ADKF-IFT as a unification framework

The most common method used to train deep kernel GPs is to minimise the negative log marginal likelihood (NLML) on a single dataset (optionally with extra regularisation terms as will be discussed in Section 27.4.1). This is commonly referred to as *Deep Kernel Learning* (DKL) [1769] and introduced in Chapter 24:

$$\psi^* = \arg\min_{\psi} \mathrm{NLML}\left(\psi, \mathcal{S}_{\mathcal{T}}\right). \tag{27.6}$$

The most notable departure from DKL is *Deep Kernel Transfer* (DKT) [1298], which instead proposes to train deep kernel GPs entirely using meta-learning, minimising the *expected* NLML over a distribution of training tasks:

$$\psi^* = \arg\min_{\psi} \mathbb{E}_{p(\mathcal{T})}[\mathrm{NLML}(\psi, \mathcal{T})]. \tag{27.7}$$

Interestingly, both DKL and DKT can be viewed as *special cases* of the general ADKF-IFT framework. It is easy to see that choosing the partition to be $\Psi_{\text{meta}} = \varnothing$, $\Psi_{\text{adapt}} = \Psi$ and the train loss $\mathcal{L}_T$ to be the NLML in Equations 27.1 and 27.2 yields Equation 27.6: DKL is just ADKF-IFT if no parameters are meta-learned. Similarly, choosing the partition to be $\Psi_{\text{meta}} = \Psi$, $\Psi_{\text{adapt}} = \varnothing$ and the validation loss $\mathcal{L}_V$ to be the NLML in Equations 27.1 and 27.2 yields Equation 27.7: DKT is just ADKF-IFT if all parameters are meta-learned. This makes ADKF-IFT *strictly more general* than these two previous methods.

27.3.4 Highlighted instantiation of ADKF-IFT

Among the many possible variations of ADKF-IFT, the following instantiation is highlighted.

- $\Psi_{\text{meta}} = \Phi$, i.e., all feature extractor parameters ϕ are meta-learned across tasks.
- $\Psi_{\text{adapt}} = \Theta$, i.e., all base kernel parameters θ (e.g., noise variance, lengthscales, amplitude) are adapted to each task.
- The train loss $\mathcal{L}_T$ is chosen to be the negative log GP marginal likelihood evaluated on the support set $\mathcal{S}_\mathcal{T}$, as is common practice for choosing GP base kernel parameters:

$$\begin{aligned}\mathcal{L}_T(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{S}_\mathcal{T}) &= -\log p(\mathcal{S}_\mathcal{T}^y | \mathcal{S}_\mathcal{T}^x, \psi_{\text{meta}}, \psi_{\text{adapt}}) \\ &= \frac{1}{2}(\mathcal{S}_\mathcal{T}^y)^\top K_{\mathcal{S}_\mathcal{T}}^{-1} \mathcal{S}_\mathcal{T}^y + \frac{1}{2}\log\det(K_{\mathcal{S}_\mathcal{T}}) + \frac{N_{\mathcal{S}_\mathcal{T}}}{2}\log(2\pi),\end{aligned} \tag{27.8}$$

  where $K_{\mathcal{S}_\mathcal{T}} \coloneqq k_{\psi_{\text{meta}},\psi_{\text{adapt}}}(\mathcal{S}_\mathcal{T}^x, \mathcal{S}_\mathcal{T}^x) + \sigma^2 I_{N_{\mathcal{S}_\mathcal{T}}}$.
- The validation loss $\mathcal{L}_V$ is chosen to be the negative log joint GP predictive posterior evaluated on the query set $\mathcal{Q}_\mathcal{T}$ given the support set $\mathcal{S}_\mathcal{T}$, also due to its common usage for making predictions with GPs:

$$\begin{aligned}\mathcal{L}_V(\psi_{\text{meta}}, \psi_{\text{adapt}}, \mathcal{T}) &= -\log p(\mathcal{Q}_\mathcal{T}^y | \mathcal{Q}_\mathcal{T}^x, \mathcal{S}_\mathcal{T}, \psi_{\text{meta}}, \psi_{\text{adapt}}) \\ &= -\log \mathcal{N}(\mathcal{Q}_\mathcal{T}^y | K_{\mathcal{Q}_\mathcal{T}\mathcal{S}_\mathcal{T}} K_{\mathcal{S}_\mathcal{T}}^{-1} \mathcal{S}_\mathcal{T}^y, K_{\mathcal{Q}_\mathcal{T}} - K_{\mathcal{S}_\mathcal{T}\mathcal{Q}_\mathcal{T}}^\top K_{\mathcal{S}_\mathcal{T}}^{-1} K_{\mathcal{S}_\mathcal{T}\mathcal{Q}_\mathcal{T}}),\end{aligned} \tag{27.9}$$

  where $K_{\mathcal{Q}_\mathcal{T}} \coloneqq k_{\psi_{\text{meta}},\psi_{\text{adapt}}}(\mathcal{Q}_\mathcal{T}^x, \mathcal{Q}_\mathcal{T}^x) + \sigma^2 I_{N_{\mathcal{Q}_\mathcal{T}}}$ and $K_{\mathcal{S}_\mathcal{T}\mathcal{Q}_\mathcal{T}} \coloneqq k_{\psi_{\text{meta}},\psi_{\text{adapt}}}(\mathcal{S}_\mathcal{T}^x, \mathcal{Q}_\mathcal{T}^x)$.

There are several benefits to this choice as detailed below.

1. This particular choice of loss functions has the advantage that the prediction procedure during meta-testing as defined in Equation 27.3 exactly matches the meta-training procedure, thereby closely following the principle of *learning to learn*.
2. The partition of parameters can be intuitively understood as meta-learning a generally useful feature extractor f_ϕ such that it is possible on average to fit a low-loss GP to the feature representations extracted by f_ϕ for each individual task. This is very similar to previous transfer learning approaches.
3. Since most GP base kernels have only a handful of parameters, the Hessian in Equation 27.5 can be computed and inverted exactly during meta-training using Algorithm 28; this removes any need for Hessian approximations.
4. The inner optimisation (Equation 27.2) for ψ_{adapt} is computationally efficient, as it does not require back-propagating through the feature extractor f_ϕ.

More generally, ADKF-IFT combines DKL and DKT in a way that can potentially inherit the strengths of both methods and the weaknesses of neither. By adapting the base kernel parameters θ specifically to each task, it prevents underfitting due to varying ranges, lengthscales, or noise levels between datasets. By meta-learning the feature extractor on many datasets, it prevents overfitting as observed by Ober et al. [1253]. This advantage is both *theoretically principled* (by solving a bilevel optimisation objective using the implicit function theorem) and *empirically observable* (as will be shown in Section 27.5).

The relationship between these methods is visualised in Figure 27.1. Panel (c) shows DKL, which trains a separate deep kernel GP for each task. It is not hard to imagine that this can lead to severe overfitting for small datasets, which has been observed empirically by Ober et al. [1253]. Panel (b) shows DKT, which prevents overfitting by fitting one deep kernel GP for all tasks. However, this implicitly makes a strong assumption that all tasks come from an identical distribution over functions, including the same noise level, same amplitude, and same characteristic lengthscales, which is unlikely to hold in practice. Panel (a) shows ADKF-IFT, which allows these important parameters to be adapted, while still regularising the feature extractor with meta-learning. The choice of this specific instantiation is based on the assumption that adapting the base kernel parameters is more appropriate given the expected differences between tasks: two related tasks are more likely to have different noise levels or characteristic lengthscales than to require substantially different feature representations (as discussed in Section 27.2.2).

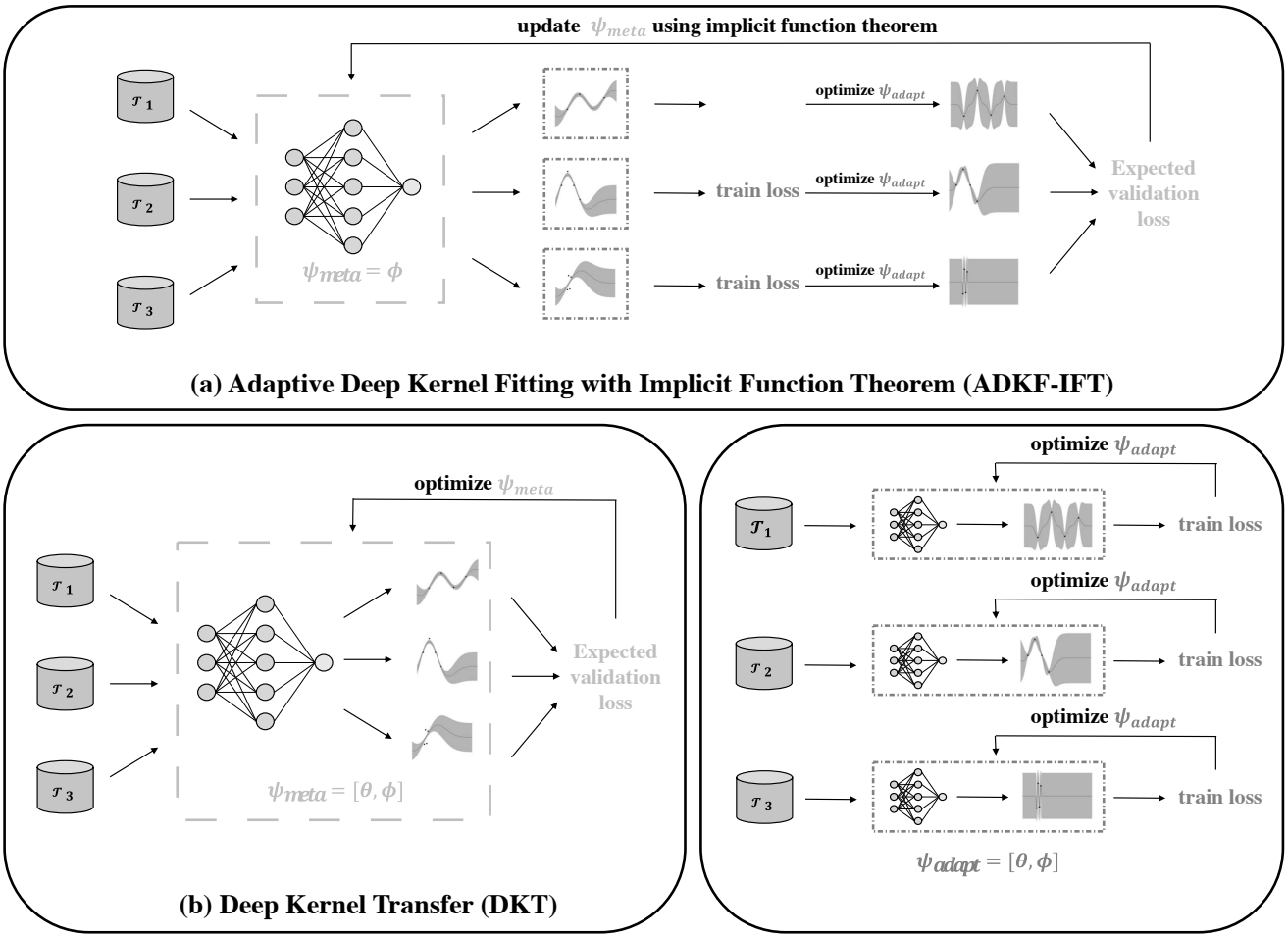


Figure 27.1: A diagram contrasting the training procedures of ADKF-IFT, DKT and DKL. (a) ADKF-IFT meta-learns the feature extractor parameters ϕ across all tasks and adapts the base kernel parameters θ to each task under a bilevel optimisation framework. (b) DKT meta-learns all parameters across all tasks. (c) DKL adapts all parameters to each task.

27.4 Connection to other methods

27.4.1 Deep kernel Gaussian processes

ADKF-IFT is part of a growing body of literature of techniques to train deep kernel GPs. As discussed in Section 27.3.3, ADKF-IFT *generalises* DKL [1769] and DKT [1298], which exclusively use single-task learning and meta-learning, respectively. Liu et al. [1061] and van Amersfoort et al. [1654] propose adding regularisation terms to the loss of DKL in order to mitigate overfitting. These works are better viewed as *complementary* to ADKF-IFT rather than *alternatives*: their respective regularisation terms could easily be added to $\mathcal{L}_T$ in Equation 27.2 to improve performance. However, the regularisation strategies in both of these papers are designed for continuous inputs only, limiting their applicability to structured data like molecules.

The preprint of Tossou et al. [1624] proposes an alternative way to meta-train adaptive deep kernel GPs called ADKL-GP, where adaptation is performed by conditioning the feature extractor on an embedding of the entire support set rather than adjusting a subset of the kernel parameters as in ADKF-IFT. In general, this method is very prone to overfitting. This is because the training objective for ADKL-GP is equivalent to the objective for DKT with an added contrastive loss, weighted by a sensitive hyperparameter γ, which can be interpreted as balancing the degree of regularisation between two extremes. For this method to be useful it would appear that γ would need to be carefully tuned to balance between these extremes. However, empirical results suggest that there is no consistent trend besides $\gamma > 0$ being slightly helpful, although the differences in performance were small. This suggests that the method may be difficult to use in practice. ADKF-IFT however has no such tunable hyperparameters, which is a significant strength. Instead, the balance between DKL and DKT is controlled by

selecting which parameters are adapted and meta-learned, which is much more interpretable and makes it easier to use in practice.

27.4.2 Meta-learning

ADKF-IFT can also be viewed as a meta-learning algorithm comparable to many previously-proposed methods [267, 284, 582, 961, 1059, 1294, 1299, 1606, 1633, 1681, 1775]. One distinguishing feature of ADKF-IFT is that it is specially designed for deep kernel GPs, whereas most methods from computer vision are designed exclusively for neural network models, which as previously stated are unsuitable when reliable uncertainty estimates are required. Furthermore, many of these algorithms such as ProtoNet [1527] are designed principally or exclusively for classification, while ADKF-IFT is suited to both regression and classification. Compared to model-agnostic frameworks like MAML [518, 1372] which meta-learn a common initialisation for adaptation of all parameters, ADKF-IFT meta-learns a subset of parameters across tasks and adapts the rest of parameters to each task.

27.4.3 Multi-Task Gaussian processes

ADKF-IFT can be considered as a method to learn multi-task GPs. The dominant approach to this problem in prior works is to learn a shared kernel for all data points across all tasks, transmitting information by explicitly modelling the covariance between data from different tasks [186, 545, 880, 1334, 1580]. The main difference between methods in this family is the exact form of the kernel [1610], which is typically assumed to have a particular structure (e.g., Kronecker product or weighted sum of simpler kernels). ADKF-IFT cannot be naturally viewed in this way because the covariance between data points from separate tasks is always zero; information is instead transmitted across tasks via a shared set of model parameters. Therefore, ADKF-IFT is a significant departure from the dominant paradigm in GP transfer learning.

27.4.4 Implicit function theorem in machine learning

The implicit function theorem employed in ADKF-IFT has been used in many previous machine learning papers in various contexts, e.g., neural architecture search [1852], hyperparameter-tuning [114, 313, 1080, 1093, 1308], and meta-learning [284, 1005, 1372].

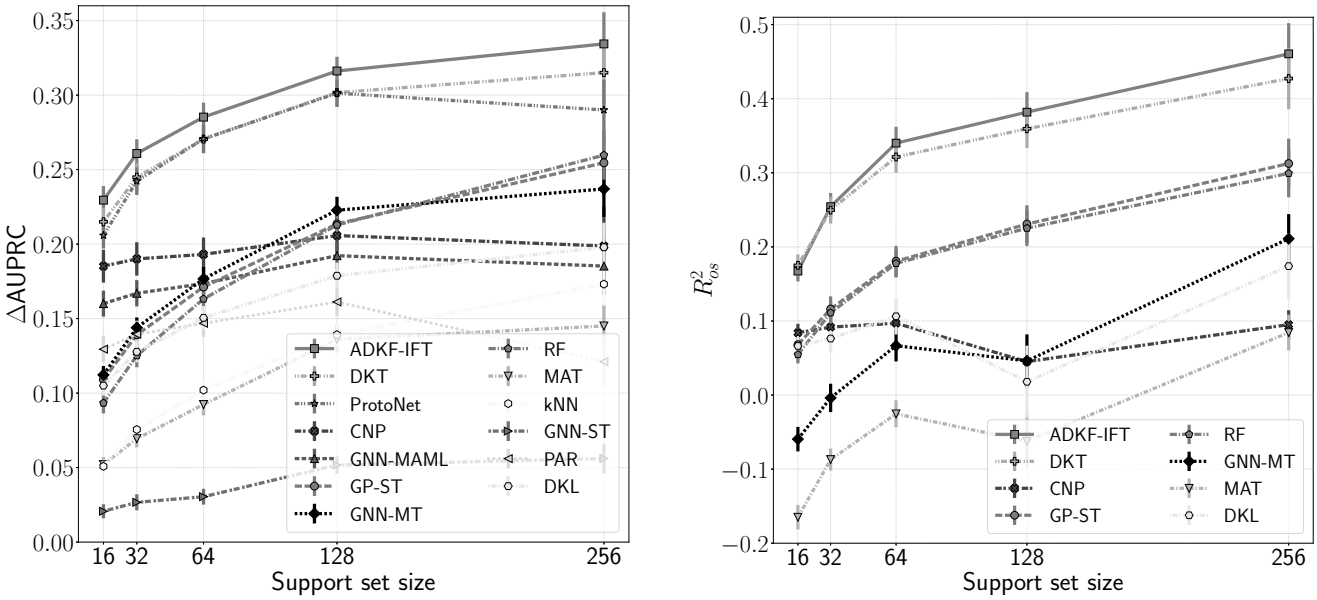
One challenge of applying implicit function theorem is the computation of the inverse Hessian. While earlier work compute this explicitly [114, 977] as in ADKF-IFT, recent work employ various approximation techniques such as Neumann approximation [313, 1080], conjugate gradient [284, 1308, 1372, 1713], and the identity matrix [1093] to retain computational tractability.

27.5 Empirical evaluation

This section presents empirical evaluation results for the specific instantiation of ADKF-IFT from Section 27.3.4. The experiments are chosen to focus exclusively on molecular property prediction and optimisation tasks, because this application would benefit greatly from better GP models: firstly because many existing methods struggle on small datasets of size $\sim 10^2$ which are ubiquitous in chemistry, and secondly because many tasks in chemistry require high-quality uncertainty estimates. Specifically, ADKF-IFT is evaluated on a larger-scale molecular property prediction benchmark named FS-Mol [1555]. Empirical results confirm that ADKF-IFT is the best-performing method (Section 27.5.1), supporting the hypothesis from Section 27.3.4 that ADKF-IFT achieves a better balance between overfitting and underfitting than DKL and DKT. Moreover, additional Bayesian optimisation results demonstrate that the ADKF-IFT feature representation is transferable to out-of-domain (OOD) molecular property prediction and optimisation tasks (Section 27.5.2).

27.5.1 Few-shot molecular property prediction on FS-Mol

The empirical evaluation is conducted on the FS-Mol benchmark [1555], which contains a carefully constructed set of few-shot learning tasks for molecular property prediction. FS-Mol contains over 5,000 tasks with 233,786 unique compounds from ChEMBL27 [1164], split into training (4,938 tasks), validation (40 tasks), and test (157 tasks) sets. Each task is associated with a protein target. The original benchmark only considers binary classification of active/inactive compounds. Nevertheless, the regression task for the actual numeric activity target IC50 or EC50 is included in this evaluation, as it is a desired and more preferred task to do in real-world drug discovery projects.



(a) Classification (157 tasks).

(b) Regression (111 tasks).

Figure 27.2: Mean performance with standard error of each compared method on all FS-Mol test tasks.

ADKF-IFT is compared with four categories of baselines: (1) *Single-task learning methods*: random forest (**RF**), k-nearest neighbours (**kNN**), single-task GP with Tanimoto kernel (**GP-ST**) [1373], single-task GNN (**GNN-ST**) [632], deep kernel learning (**DKL**) [1769]; (2) *Multi-task pretraining method*: multi-task GNN (**GNN-MT**) [331, 632]; (3) *Self-supervised pretraining method*: molecule attention transformer (**MAT**) [1153]; (4) *Meta-learning methods*: property-aware relation networks (**PAR**) [1712], prototypical network with Mahalanobis distance (**ProtoNet**) [1527], model-agnostic meta-learning (**GNN-MAML**) [518], conditional neural process (**CNP**) [582], deep kernel transfer (**DKT**) [1298].

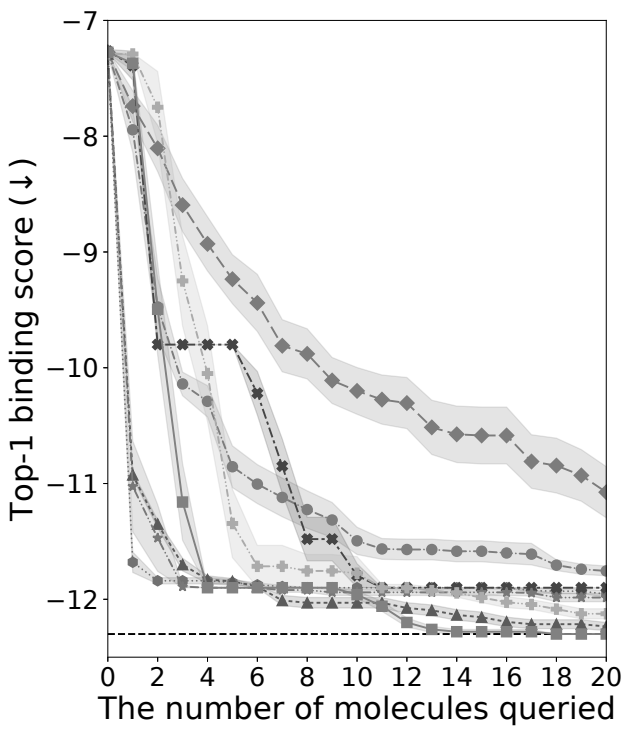
The GNN feature extractor architecture f_ϕ used for DKL, PAR, CNP, DKT, and ADKF-IFT is the same as that used for ProtoNet, GNN-ST, GNN-MT, and GNN-MAML in Stanley et al. [1555]. All multi-task and meta-learning methods are trained from scratch on FS-Mol training tasks. MAT is pretrained on 2 million molecules sampled from the ZINC15 dataset [1563]. The classification results for RF, kNN, GNN-ST, GNN-MT, MAT, ProtoNet, and GNN-MAML are reproduced according to Stanley et al. [1555].

The task-level metrics for binary classification and regression are ΔAUPRC (change in area under the precision-recall curve) and R^2_{os} (predictive/out-of-sample coefficient of determination), respectively. The evaluation procedure is the same as that in Stanley et al. [1555], where the averaged performance over ten different stratified support/query random splits of every test task is reported for each compared method. This evaluation process is performed for five different support set sizes 16, 32, 64, 128, and 256. Note that unlike MoleculeNet, the support sets are generally unbalanced for the classification task in FS-Mol, which is natural as the majority of the candidate molecules are inactive in drug discovery.

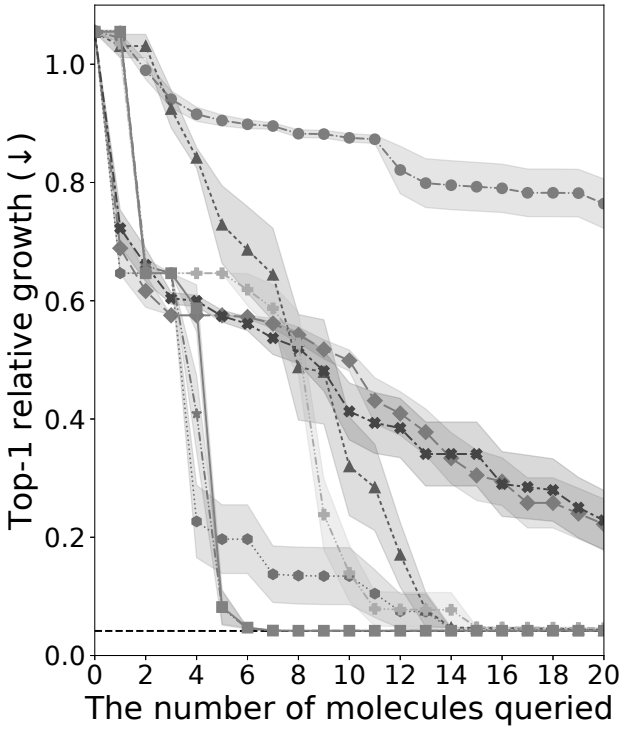
Figure 27.2 shows the overall test performance of all compared methods aggregated over all tasks. Note that RF is a strong baseline method, as it is widely used in real-world drug discovery projects and has comparable performance to many pretraining methods. The results indicate that ADKF-IFT outperforms all the other compared methods at all considered support set sizes for the classification task. For the regression task, the performance gains of ADKF-IFT over the second best method, namely DKT, get larger as the support set size increases.

27.5.2 Out-of-domain molecular property prediction and optimisation

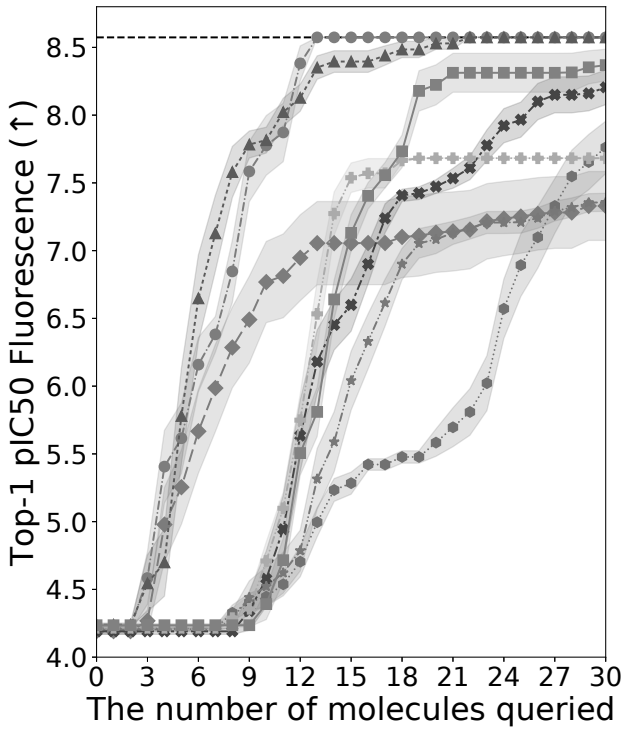
This section further demonstrates that the feature representation learned by ADKF-IFT is useful not only for in-domain molecular property prediction tasks but also for out-of-domain (OOD) molecular property prediction and optimisation tasks. For this, experiments are performed to find molecules with best desired target properties within given OOD datasets using Bayesian optimisation (BO) with a GP surrogate model operating on top of compared feature representations. These representations are evaluated on four representative molecular design tasks [579, 695, 1567, 1595] outside of FS-Mol, as summarised in Table 27.1. For the GP surrogate model,



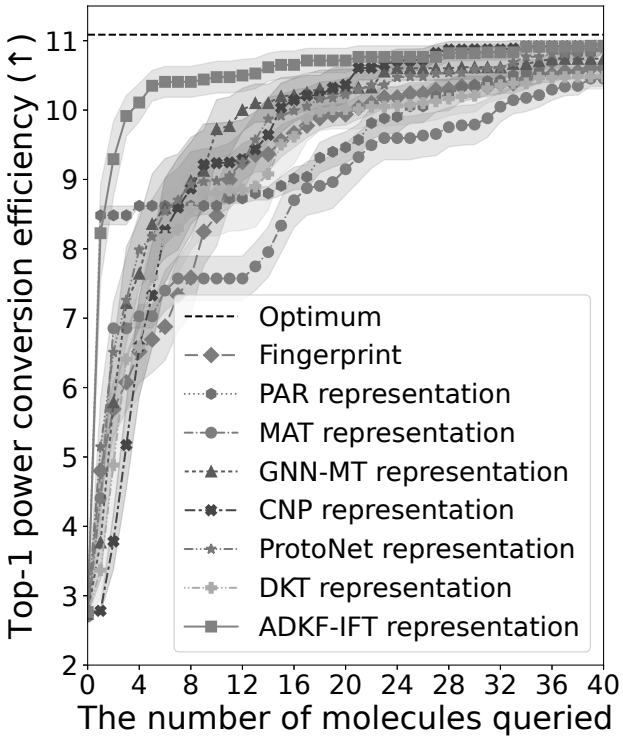
(a) Molecular docking.



(b) Antibiotic discovery.



(c) Antiviral drug design.



(d) Material design.

Figure 27.3: Mean top-1 target value with standard error as a function of the number of molecules queried for each compared feature representation on four out-of-domain molecular optimisation tasks.

Table 27.1: Descriptions of four out-of-domain molecular design tasks. The datasets for the molecular docking and material design tasks are sub-sampled from the much larger datasets provided in DockString [579] and Harvard Clean Energy Project [695], respectively. The datasets for the antibiotic discovery and antiviral drug design tasks are taken from the antibiotic training set and the COVID Moonshot dataset provided in Stokes et al. [1567] and The COVID Moonshot Consortium [1595], respectively.

Molecular design task	Data source	#compounds	Target
Docking (ESR2)	DockString training set	2,312	binding score
Antibiotic (E. coli BW25113)	Antibiotic training set	2,335	relative growth
Antiviral (SARS-CoV-2)	COVID Moonshot	1,926	pIC50 Fluorescence
Material (OPV)	Harvard Clean Energy Project	2,012	power conversion efficiency

Table 27.2: Mean predictive performance (test NLL) with standard error of a GP operating on top of each feature representations on the four out-of-domain molecular design tasks.

Feature	Out-of-domain molecular design task			
	Molecular docking	Antibiotic discovery	Antiviral drug design	Material design
Fingerprint	$\underline{1.138 \pm 0.014}$	1.669 ± 0.075	$\mathbf{4.601 \pm 0.086}$	1.091 ± 0.011
PAR	1.270 ± 0.019	2.185 ± 0.115	4.840 ± 0.086	1.283 ± 0.017
MAT	1.528 ± 0.028	2.390 ± 0.104	4.797 ± 0.088	2.198 ± 0.063
GNN-MT	1.994 ± 0.050	3.692 ± 0.225	6.399 ± 0.181	7.254 ± 0.217
CNP	1.493 ± 0.028	2.537 ± 0.162	5.005 ± 0.086	1.741 ± 0.043
ProtoNet	1.147 ± 0.013	1.615 ± 0.094	5.060 ± 0.086	1.032 ± 0.009
DKT	1.167 ± 0.012	$\underline{1.602 \pm 0.073}$	4.975 ± 0.092	$\underline{1.026 \pm 0.009}$
ADKF-IFT	$\mathbf{1.137 \pm 0.011}$	$\mathbf{1.496 \pm 0.043}$	$\underline{4.781 \pm 0.087}$	$\mathbf{0.996 \pm 0.007}$

the Tanimoto kernel is used for the fingerprint representation (with radius 2 and 2,048 bits based on count simulation). All the other compared feature representations employ Matérn-5/2 kernel without automatic relevance determination (ARD) but with a log-normal prior over the lengthscale centred at the median heuristic initialisation [585]. For BO, the expected improvement (EI) [848] acquisition function is employed with a query-batch size 1. The base kernel parameters are re-fit at the beginning of each BO iteration using all available data points. All compared feature representations are extracted using the models trained on the FS-Mol dataset from scratch in Section 27.5.1, except for the pretrained MAT representation and the fixed fingerprint representation. Each BO experiment is repeated 20 times, each time starting from 16 randomly sampled molecules from the worst ~ 700 molecules within the dataset. Figure 27.3 shows that the ADKF-IFT representation enables fastest discovery of top performing molecules for the molecular docking, antibiotic discovery, and material design tasks. For the antiviral drug design task, although the ADKF-IFT representation underperforms the MAT and GNN-MT representations, it still achieves competitive performance compared to other baselines.

Table 27.2 explicitly reports the regression predictive performance of a GP operating on top of each compared feature representation for these four out-of-domain molecular design tasks. The configuration of the GP is the same as that in the BO experiments. The test negative log likelihood (NLL) averaged over 200 support/query random splits (100 for each of the support set sizes 32 and 64) is reported. The results show that the ADKF-IFT representation has the best test NLL on the molecular docking, antibiotic discovery, and material design tasks, and ranks second on the antiviral drug design task.

27.6 Summary

This chapter presented Adaptive Deep Kernel Fitting with Implicit Function Theorem (ADKF-IFT), a general framework for fitting deep kernels that interpolates between meta-learning and conventional deep kernel learning. ADKF-IFT meta-learns a feature extractor across tasks such that the task-specific GP models estimated on top of the extracted feature representations can achieve the lowest possible prediction error on average. ADKF-IFT is implemented by solving a bilevel optimisation objective via implicit differentiation, which results in a unifying framework containing DKL and DKT as special cases. The empirical results demonstrated that ADKF-IFT learns generally useful feature representations, achieving state-of-the-art performance on a variety of real-world few-shot molecular property prediction tasks and on out-of-domain molecular property prediction and optimisation tasks, outperforming a wide range of single-task, multi-task, pre-training, and meta-learning methods. ADKF-IFT

could potentially be an important method to produce well-calibrated models for fully-automated high-throughput experimentation in the future.

Having said that, one potential factor that could limit the use case of ADKF-IFT is that it requires a meta-dataset of many related tasks for meta-training the feature extractor, which is not always available in practice. Furthermore, the inner optimisation procedure is a major bottleneck in ADKF-IFT since it needs to be performed until convergence in every training and inference step.

Part VI

Priors

Chapter 28

Introduction to priors

In classical Bayesian statistics, a *prior* $p(\boldsymbol{\theta})$ represents our initial beliefs about the parameters $\boldsymbol{\theta} \in \mathbb{R}^P$ of a given statistical model before observing data. Prior knowledge is a cornerstone of Bayesian analysis, as it allows us to integrate existing knowledge or expert insights into our models. This is typically motivated by subjective Bayesian probability, which represents a degree of belief, but is also contrasted with objective Bayesian approaches that seek to minimise the prior's influence when no prior knowledge is available.

To illustrate classical prior elicitation, consider a highly informative prior expressing strong historical beliefs about the average heights of students. For first-grade students, we use a prior suggesting a normal distribution with mean 1.2 metres and variance 0.003. For third-grade students, the prior is a normal distribution with mean 1.4 metres and variance 0.002. Suppose we observe heights for 10 first-graders: {1.18, 1.22, 1.25, 1.20, 1.21, 1.19, 1.21, 1.24, 1.17, 1.26} metres (sample mean 1.213), and 5 third-graders: {1.37, 1.39, 1.42, 1.36, 1.40} metres (sample mean 1.388). We model the mean height θ_k for each group k, assuming heights $y_{k,i} \sim \mathcal{N}(\theta_k, 0.001)$. The

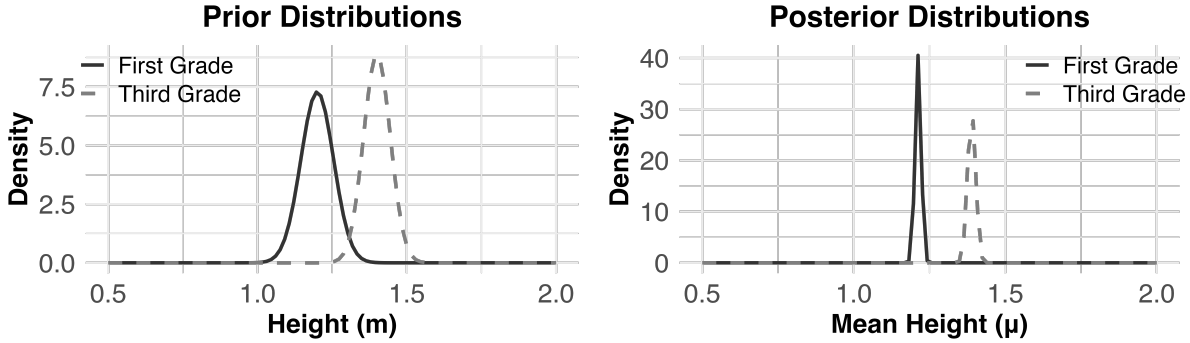


Figure 28.1: Informative priors for students' average height (solid line for first-grade: $\mathcal{N}(1.2, 0.003)$, dashed line for third-grade: $\mathcal{N}(1.4, 0.002)$) and their posteriors (right panel), showing updated beliefs after observing the data.

posterior distribution is $p(\theta_k \mid \mathbf{y}_k) \propto p(\theta_k)p(\mathbf{y}_k \mid \theta_k)$. The prior is $\mathcal{N}(\theta_k; \mu_{0,k}, \sigma^2_{0,k})$, and the likelihood for n_k observations is:

$$p(\mathbf{y}_k \mid \theta_k) \propto \exp\left(-\frac{n_k}{2 \times 0.001}(\bar{y}_k - \theta_k)^2\right).$$

Since the prior and likelihood are both Gaussian, this is a conjugate prior-likelihood pair. The posterior is also Gaussian, $\mathcal{N}(\theta_k; \mu_{n,k}, \sigma^2_{n,k})$, with parameters given analytically by:

$$\mu_{n,k} = \frac{\frac{\mu_{0,k}}{\sigma^2_{0,k}} + \frac{n_k \bar{y}_k}{0.001}}{\frac{1}{\sigma^2_{0,k}} + \frac{n_k}{0.001}}, \quad \sigma^2_{n,k} = \frac{1}{\frac{1}{\sigma^2_{0,k}} + \frac{n_k}{0.001}}.$$

Thus, for first-graders, the posterior is $\mathcal{N}(1.212581, 0.00009677)$; while for third-graders, it is $\mathcal{N}(1.389091, 0.0001818)$. We have significantly reduced the uncertainty after observing the data and updated our prior expectation, as shown in Figure 28.1. This solution is conjugate and analytical.

In classical statistics, conjugate priors are highly valued because they yield closed-form posterior distributions. For example, a Beta prior combined with a Binomial likelihood yields a Beta posterior, and a Gamma prior with

a Poisson likelihood yields a Gamma posterior. In BDL, however, conjugate updates are generally unavailable due to the highly non-linear nature of neural networks, where weights are passed through multiple layers of non-linear activations. An exception is last-layer BNN inference, where one can perform exact conjugate Gaussian-Gaussian updates on the final weights by treating the hidden activations of the penultimate layer as fixed features.

An *informative prior* guides the model with subjective domain knowledge, balancing the data's information and the prior. However, in modern contexts like machine learning and high-dimensional statistics, priors often serve a broader range of purposes beyond encoding prior knowledge [1]. When such knowledge is unavailable or neutrality is preferred, *uninformative priors*, such as near-flat or uniform distributions, minimise subjective influence. Similarly, *reference priors* are designed to maximise the difference between the prior and the resulting posterior, letting the data primarily drive inference. In advanced applications of BDL, priors are often selected for practical purposes, such as *convenience priors* for computational simplicity, *regularisation priors* to prevent overfitting, or *sparsity-inducing priors* to promote simpler model structures.

This chapter builds on these ideas, exploring the statistical properties of priors used in BDL. We examine their mathematical formulations, practical roles, and how they shape model behaviour, equipping the reader with tools to design effective BDL models. Each subsection discusses a specific prior type, with examples and visualisations to illustrate their impact. Accompanying figures show the prior distributions and the corresponding prior predictive distributions, which reveal the range of functions a deep learning model might produce before observing data.

28.1 Introductory overview of priors for BDL

In this section, we introduce the fundamental priors employed in Bayesian deep learning, exploring their statistical properties and their impact on model behaviour. For each prior, we provide illustrations of the *prior predictive distribution*, which reflects the range of possible model outputs before observing data. These visualisations are based on a neural network with two hidden layers, each containing two neurons, and logistic (sigmoid) activation functions. While the properties of prior predictive distributions may vary with changes in network depth, width, or activation functions, we focus on this simple architecture to build intuition.

28.1.1 Informative Priors

Informative priors encode specific domain knowledge about the parameters $\boldsymbol{\theta}$, as illustrated in the student height example (Figure 28.1), where priors for average heights of first- and third-grade students were based on past studies. In BDL, "manually specified" informative priors are rare due to the complexity of the models, yet they can still be sometimes applied to specific parameters, such as the bias term θ_0 in the output layer, to reflect expected model outputs at a baseline (e.g. all other input neurons to the last layer should be close to 0 for a chosen baseline combination of covariates). For example, in a BDL model predicting house prices in a city, we might use a one-dimensional Gaussian prior for $\theta_0^{(L)}$:

$$p(\theta_0^{(L)}) = \mathcal{N}(\theta_0^{(L)}; \mu_0, \sigma_0^2),$$

where μ_0 is the average house price (e.g., 500,000 USD) based on market data, and a small σ_0^2 (e.g., 160,000) reflects the level of confidence in this estimate. A strong informative prior (small σ_0^2) dominates the data's influence (likelihood), while a weak one (larger σ_0^2) balances data and prior information.

Another way to incorporate domain knowledge in BDL is through *spike-and-slab priors* [1190] for weights associated with specific covariates in the input layer. These priors are informative by design when we know certain inputs are more relevant than others. For a weight $\theta_{ij}^{(1)}$ in the first layer corresponding to covariate x_i, the prior takes the form:

$$p(\theta_{ij}^{(1)} \mid \gamma_i) = \gamma_i \mathcal{N}(\theta_{ij}^{(1)}; \mu_i, \sigma_i^2) + (1 - \gamma_i)\, \delta_0(\theta_{ij}^{(1)}), \quad \gamma_i \sim \text{Bernoulli}(\rho_i),$$

where $\delta_0(\cdot)$ is a point mass at zero (the "spike") that switches off covariate x_i when $\gamma_i = 0$, while the Gaussian component $\mathcal{N}(\theta_{ij}^{(1)}; \mu_i, \sigma_i^2)$ (the "slab") keeps it active when $\gamma_i = 1$. The inclusion probability ρ_i reflects our prior belief in the importance of covariate x_i. For instance, in a model predicting heart disease risk from age, blood pressure, and cholesterol, one might assign $\rho_i = 0.9$ to cholesterol and a lower $\rho_i = 0.1$ to a less relevant variable such as eye colour, as illustrated in Figure 28.2.

Specifying informative priors for all parameters in BDL is a non-trivial task, as the high-dimensional and complex nature of neural networks makes manual prior elicitation practically impossible. While informative

[1] There still is a lot of philosophical debate on whether this is acceptable from a puristic Bayesian perspective. We will not dive into this debate and rather introduce different concepts to the reader in the context of BDL.

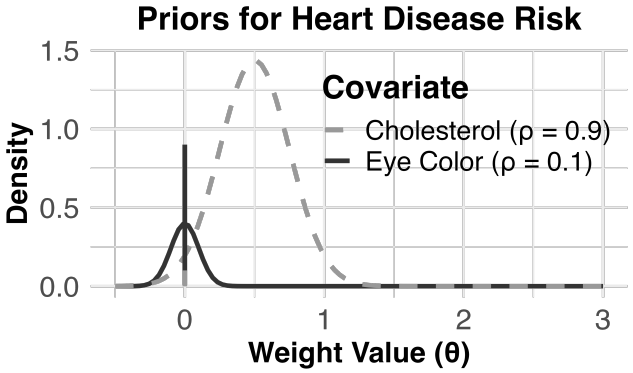


Figure 28.2: Informative spike-and-slab prior density (left) and the corresponding prior predictive distributions (right) for input layer weights under different inclusion probabilities.

priors, such as Gaussian priors for bias terms, can encode domain knowledge for specific parameters, scaling this approach to entire networks is hardly ever feasible. To address this challenge, techniques like pretraining prior hyperparameters, which leverage knowledge from pre-existing models or synthetic data (discussed further in Chapter 30), and function-space priors, which impose structure on the model's output functions (explored in detail in Chapter 31), offer promising solutions. These approaches mitigate the difficulties of manual prior specification, at least to some extent, and provide practical strategies for effective prior design in BDL.

28.1.2 Uninformative, Reference, and Objective Priors

When prior knowledge is absent or neutrality is desired, statisticians employ objective or *uninformative priors* to minimise subjective influence, letting the data dominate inference. In practice, a common choice is a uniform prior for each parameter:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \text{Uniform}(\theta_i; a, b),$$

where $[a, b]$ is a wide interval (e.g., $[-10, 10]$). This results in a joint prior that is uniform over a hyper-rectangle $[a, b]^P$, with each marginal distribution $p(\theta_i)$ being uniform, suggesting neutrality for individual parameters.

However, marginally uniform priors do not always imply an uninformative joint prior [564], especially in high-dimensional BDL models. The independence assumption can impose unintended structure, as parameters like weights in deep learning models often interact. Moreover, the joint prior can become concentrated in certain regions, undermining its uninformativeness. For instance, if $[a, b] = [-1, 1]$ is chosen to be narrow, the joint prior restricts all weights to a small hyper-rectangle, concentrating probability mass and limiting the model's ability to capture complex functions, such as non-linear patterns in a neural network. Similarly, in high-dimensional settings with large P, the product of uniform priors can lead to a prior predictive distribution (the range of possible model outputs) that concentrates around specific function classes, such as near-zero outputs if the network's activation functions (e.g., ReLU) amplify small weights. This concentration can bias inference, making the prior less uninformative than intended and resulting in quite concentrated prior predictive distributions. Additionally, there is a long-standing statistical debate regarding whether a bounded uniform prior is truly uninformative; strictly speaking, a uniform prior is only uninformative in its improper version (over the entire real line $\mathbb{R}$), which is non-normalisable, potentially causing unstable inference or improper posteriors in BDL's high-dimensional context. Furthermore, flat or uniform priors are not invariant under parameter transformations, meaning a prior that appears uniform on one scale may encode strong beliefs on another.

To address this lack of invariance, statisticians construct *objective priors*. The classic example is Jeffreys prior [840], which is defined as proportional to the square root of the determinant of the Fisher information matrix. Jeffreys prior is invariant under reparameterisation, meaning the same prior beliefs are represented regardless of how the model is parameterised. In multi-parameter settings, this idea is generalised by *reference priors* [130], which are designed to maximise the divergence (e.g., KL divergence) from the prior to the posterior as the sample size grows, ensuring the data's influence is maximised. Reference priors account for the model's structure and parameter interactions, making them more robust for complex models. For example, in a BDL model, a reference prior might adjust for the network's architecture to avoid the biases introduced by uniform priors. However, the computational complexity of computing Jeffreys or reference priors (which requires evaluating the Fisher information of a neural

network) limits their use in BDL, where models often have millions of parameters. In practice, uninformative priors like uniform distributions are rarely used since despite simplicity they require careful selection of $[a, b]$ to avoid unintended concentration or instability.

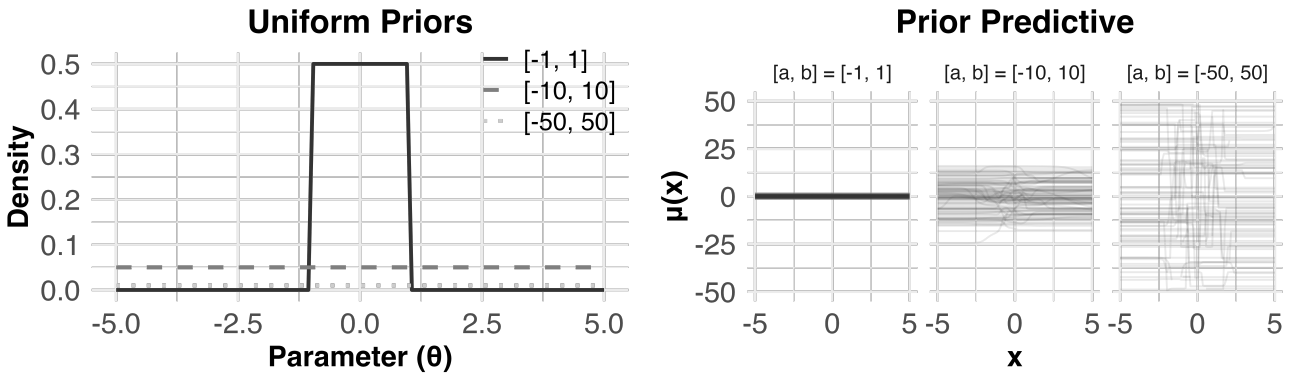


Figure 28.3: Uniform priors for single parameters (left) and the corresponding prior predictive distributions (right) under varying intervals, showing how narrow bounds can restrict the function space.

The effect of different intervals is illustrated in Figure 28.3, which shows the density of uniform priors for three intervals: $[-1, 1]$, $[-10, 10]$, and $[-50, 50]$ and their impact on the prior predictive distribution. Sample trajectories for a BDL regression model under these priors are not uniform and can be strongly concentrated in specific regions, cutting the support for the prior predictive distribution and highlighting our discussion above. This prevents the prior predictive from being non-informative on one hand, and on the other, limits its support, which makes uniform priors a poor tool both for variational methods and for regularisation in BDL. A related but distinct approach to an uninformative prior is the limiting Gaussian $\mathcal{N}(\theta_i; 0, \sigma^2)$ as $\sigma^2 \to \infty$, which behaves similarly to a flat prior over any bounded region of interest and avoids the boundary effects of uniform distributions. In the next subsection, we shall discuss other prior alternatives for BDL.

28.1.3 ℓ_2 Regularisation Priors

Regularisation priors prevent overfitting by constraining model complexity. A standard choice in BDL is the zero-centred isotropic Gaussian prior for all weights in the model:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \mathcal{N}(\theta_i; 0, \sigma^2).$$

The popularity of this prior is not just in its simplicity, it actually corresponds to ℓ_2 regularisation in maximum a posteriori (MAP) estimation, as derived below. The variance σ^2 controls regularisation strength: smaller σ^2 shrinks parameters toward zero, simplifying the model, while larger σ^2 allows more flexibility. To derive the link to ℓ_2-regularisation, consider a BDL model with parameters $\boldsymbol{\theta}$. In MAP estimation, we maximise the log-posterior:

$$\log p(\boldsymbol{\theta} \mid \mathcal{D}) = \log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) + \text{constant}.$$

For the i.i.d. Gaussian prior, the log-prior is:

$$\log p(\boldsymbol{\theta}) = \sum_{i=1}^{P} \log \mathcal{N}(\theta_i; 0, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{i=1}^{P} \theta_i^2 - \frac{P}{2} \log(2\pi\sigma^2).$$

Thus, the log-posterior is:

$$\log p(\boldsymbol{\theta} \mid \mathcal{D}) = \log p(\mathcal{D} \mid \boldsymbol{\theta}) - \frac{1}{2\sigma^2} \sum_{i=1}^{P} \theta_i^2 + \text{constant}.$$

The term $\frac{1}{2\sigma^2} \sum_{i=1}^{P} \theta_i^2$ is the ℓ_2-regularisation penalty, with $\lambda = \frac{1}{2\sigma^2}$. A smaller σ^2 increases λ, enforcing stronger regularisation.

To illustrate the role of the prior variance σ^2 in regularisation within BDL, consider three cases: $\sigma^2 = 0.02$, $\sigma^2 = 1$, and $\sigma^2 = 50$, corresponding to strong, moderate, and weak regularisation, respectively. The regularisation

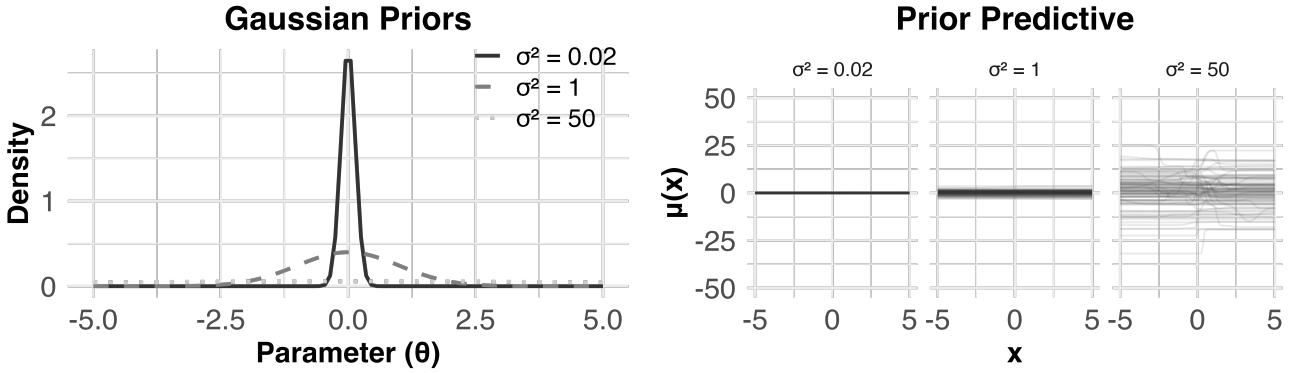


Figure 28.4: Gaussian priors for single parameters (left) and the corresponding prior predictive distributions (right) under the baseline architecture and varying σ^2.

parameter $\lambda = \frac{1}{2\sigma^2}$ governs the penalty on model weights. The small σ^2 of 0.02 yields a large $\lambda = \frac{1}{2 \cdot 0.02} = 25$, heavily penalising large weights, which constrains the model to produce simple, near-linear functions, particularly for activation functions like the logistic one, which is approximately linear near zero. Conversely, a large σ^2 of 50 results in a small $\lambda = \frac{1}{2 \cdot 50} = 0.01$, allowing larger weights and enabling complex, non-linear functions, albeit with an increased risk of overfitting. A moderate σ^2 of 1 lies between these extremes, with $\lambda = 0.5$, permitting some flexibility while controlling model complexity.

These effects manifest in the *prior predictive distribution*, which captures the range of possible model outputs before observing data. In a BDL regression model, a small σ^2 concentrates prior predictive outputs near zero, yielding smooth, constrained functions that are close to linear or even near-constant, while a large σ^2 permits highly variable, non-linear outputs, accommodating diverse patterns. Notably, the support of the prior predictive distribution remains unconstrained across all cases, making Gaussian priors versatile for robust regularisation. Figure 28.4 illustrates this effect, highlighting the critical role of σ^2 in shaping model behaviour.

The above discussion assumes a neural network with two hidden layers, each with two neurons, and a logistic activation function. However, prior predictive behaviour depends on architectural choices, particularly activation functions. While regularisation persists if the activation is approximately linear near zero, the specific functional form of outputs varies with activation properties, such as boundedness. Figure 28.5 compares prior predictive distributions for tanh and ReLU activations in the same architecture, showcasing distinct behaviours.

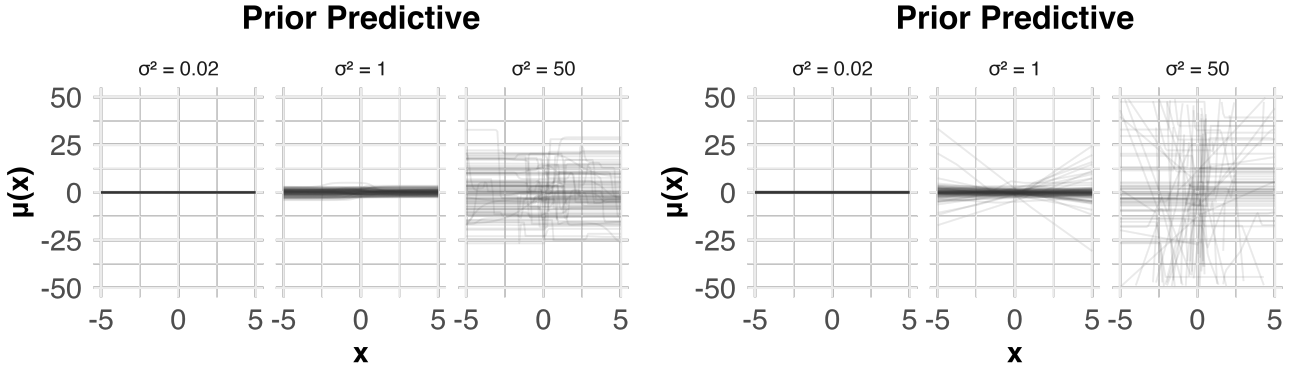


Figure 28.5: Prior predictive distributions for tanh (left) and ReLU (right) activations.

Such architectural dependencies introduce further risks of prior misspecification. To mitigate this, it is recommended to conduct prior predictive checks on simplified versions of the model.

As an example of a well-specified model and a misspecified one, consider a classification problem using our baseline BDL model with two input covariates and Bernoulli-distributed responses. Varying the prior variance σ^2 alters the decision boundary based on the posterior predictive [2], transitioning from a near-constant model

[2]Obtained with a 1000 HMC iterations on two chains, computed by Stan, yet inference details are not of central interest in this chapter, hence omitted.

(small $\sigma^2 = 0.02$), to close to linear (moderate $\sigma^2 = 1$), to non-linear boundaries (large $\sigma^2 = 50$). Figure 28.6 illustrates this property, underscoring that a naive Bayesian approach offers no inherent guarantees of optimality or robustness in the posterior predictive distribution.

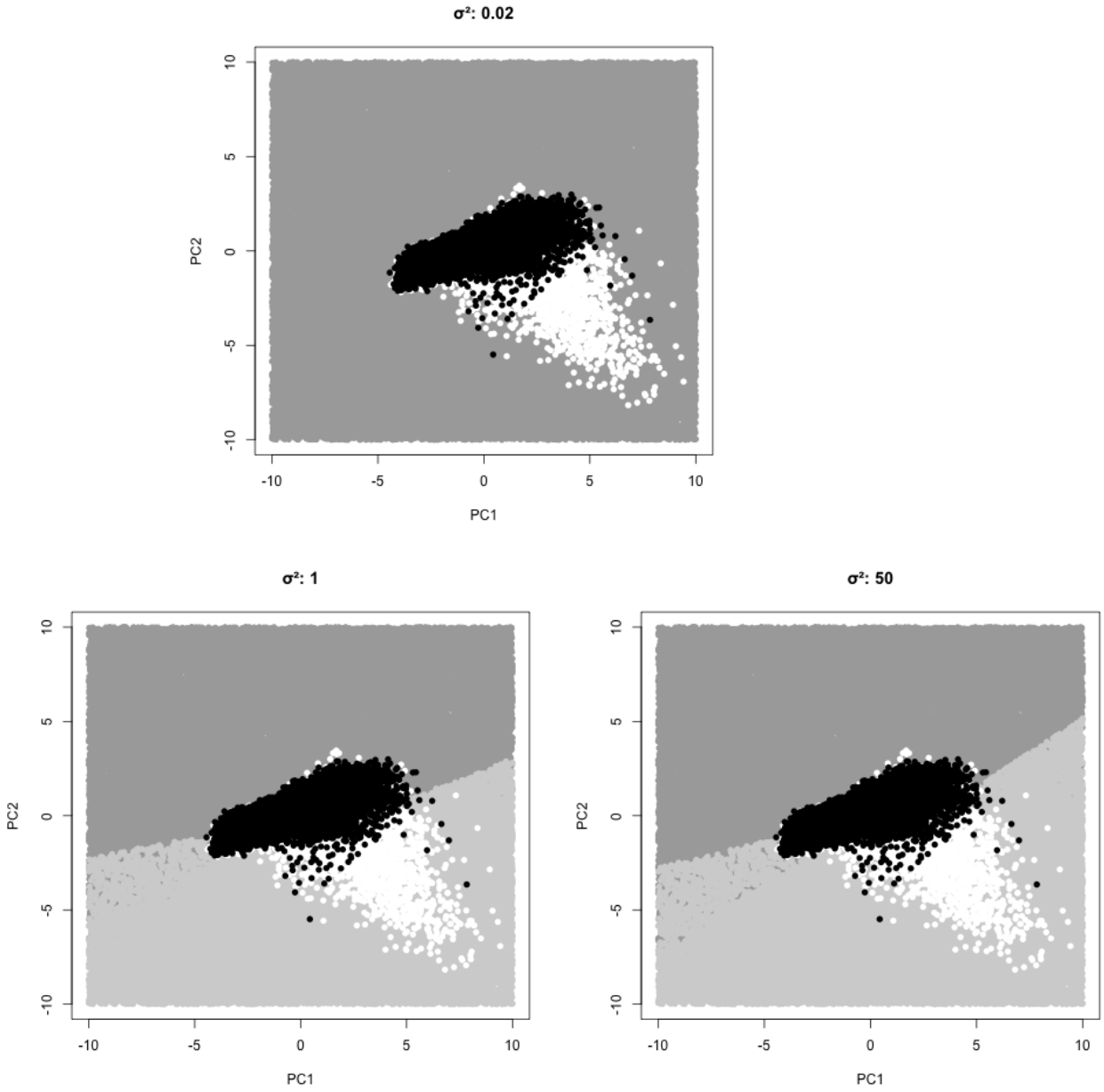


Figure 28.6: Decision boundaries based on the posterior predictive of a BDL classification model with prior variances $\sigma^2 = 0.02$, 1, and 50.

As we have seen already, poorly chosen σ^2 in Gaussian priors can lead to severe model misspecification, resulting in distorted, misspecified posteriors, a phenomenon known as the *cold posterior effect* [549, 1742]. Blindly adopting a standard Gaussian prior is ill-advised. To address this, σ^2 can be tuned via empirical Bayes or assigned a hyperprior, such as $\sigma^2 \sim \text{Inv-Gamma}(\alpha, \beta)$ [968] resulting in a robust t-distributed marginal prior. In contrast, frequentist ℓ_2-regularisation typically selects λ through cross-validation or model selection criteria.[3]

Another extension of i.i.d. Gaussian priors is the multivariate Gaussian prior, which accounts for parameter dependencies via a covariance structure:

$$p(\boldsymbol{\theta}) = \mathcal{N}_P(\boldsymbol{\theta}; \mathbf{0}, \boldsymbol{\Sigma}),$$

[3]Hyperparameter optimisation can also tune σ^2 directly, but this approach deviates from a strictly Bayesian perspective.

with the possibility to introduce an additional inverse-Wishart hyperprior on $\boldsymbol{\Sigma}$. Such priors (at least for sparse covariance structures) may be practical for large-scale BDL as they both provide regularisation and allow for handling dependencies between the parameters.

28.1.4 Sparsity-Inducing and Variable Selection Priors

Sparsity-inducing priors encourage model parameters to be zero or near-zero, which helps simplify the model structure, prevent overfitting, and improve generalisation. In BDL, two broad classes of sparsity are commonly distinguished. *Soft-sparsity*, achieved through continuous shrinkage priors such as the Laplace, Horseshoe, and scale mixtures of Gaussians, concentrates probability mass around zero but produces almost surely non-zero (non-sparse) posterior samples. These priors act as shrinkage mechanisms that pull uninformative weights toward zero while allowing large weights for genuinely important features, thanks to their heavy tails. The Laplace prior, for instance, yields exactly sparse MAP estimates (analogous to the frequentist Lasso [1607]), but posterior samples remain non-sparse. *Hard-sparsity*, in contrast, is achieved through priors that place a discrete probability mass exactly at zero, such as spike-and-slab priors [607, 1190]. These priors allow parameters to be exactly zero in the posterior, performing explicit variable selection and zeroing out weights, which is particularly valuable in BDL for weight pruning, network compression, and feature selection.

The Laplace prior is a common continuous shrinkage prior:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \text{Laplace}(\theta_i; 0, b),$$

which corresponds to ℓ_1-regularisation in MAP estimation with $\lambda = b^{-1}$ [1295, 1761]. The scale parameter b controls the degree of shrinkage, similar to σ^2 in the Gaussian prior.

Other continuous options include the scale mixture of Gaussians [174]:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \left(\pi \mathcal{N}(\theta_i; 0, \sigma_1^2) + (1-\pi)\mathcal{N}(\theta_i; 0, \sigma_2^2)\right),$$

with $\sigma_1^2 > \sigma_2^2$ and $\sigma_2^2 \ll 1$, which places a narrow Gaussian near zero to shrink small weights and a wide Gaussian to accommodate larger weights. Alternatively, one can use the horseshoe prior [243]:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \mathcal{N}(\theta_i; 0, \sigma^2 \tau_i^2),$$

where $\tau_i \sim \text{C}^+(0, 1)$ is a half-Cauchy local shrinkage parameter. Both Laplace and Horseshoe priors are heavy-tailed, allowing them to shrink noise parameters aggressively while leaving large, informative parameters virtually unaffected. This heavy-tailed property makes them robust to misspecification and helps mitigate the cold posterior effect in BDL [549].

For hard-sparsity and explicit variable selection, the spike-and-slab prior [607, 1190] can be applied to every weight $\theta_{ij}^{(l)}$ in the network:

$$p(\theta_{ij}^{(l)} \mid \gamma_{ij}^{(l)}) = \gamma_{ij}^{(l)}\, \mathcal{N}(\theta_{ij}^{(l)}; 0, \sigma_1^2) + (1 - \gamma_{ij}^{(l)})\, \delta_0(\theta_{ij}^{(l)}), \quad \gamma_{ij}^{(l)} \sim \text{Bernoulli}(\rho),$$

where $\delta_0(\cdot)$ is a Dirac delta point mass at zero (the "spike") that sets weight $\theta_{ij}^{(l)}$ to exactly zero when $\gamma_{ij}^{(l)} = 0$, while the Gaussian component $\mathcal{N}(\theta_{ij}^{(l)}; 0, \sigma_1^2)$ (the "slab") allows non-zero weights when $\gamma_{ij}^{(l)} = 1$. The inclusion probability ρ governs the overall network sparsity: a low value encourages aggressive pruning, while a high value retains most weights. When the inclusion probabilities vary by weight or layer, they can encode structural domain knowledge, as illustrated in the informative prior example in Figure 28.2. The spike-and-slab prior is closely related to the model priors discussed in Section 28.1.5.

Log-uniform priors, either improper [1196]:

$$p(\boldsymbol{\theta}) = \prod_{i=1}^{P} \text{LogU}_\infty(\theta_i) \propto \prod_{i=1}^{P} \frac{1}{\theta_i}$$

or proper [1231]:

$$p(\boldsymbol{\theta}) \propto \prod_{i=1}^{P} \text{LogU}_\infty(\theta_i) \text{I}_{[a,b]}(\log \theta_i),$$

also promote sparsity by placing heavy mass near zero, though they remain less studied in classical statistics.

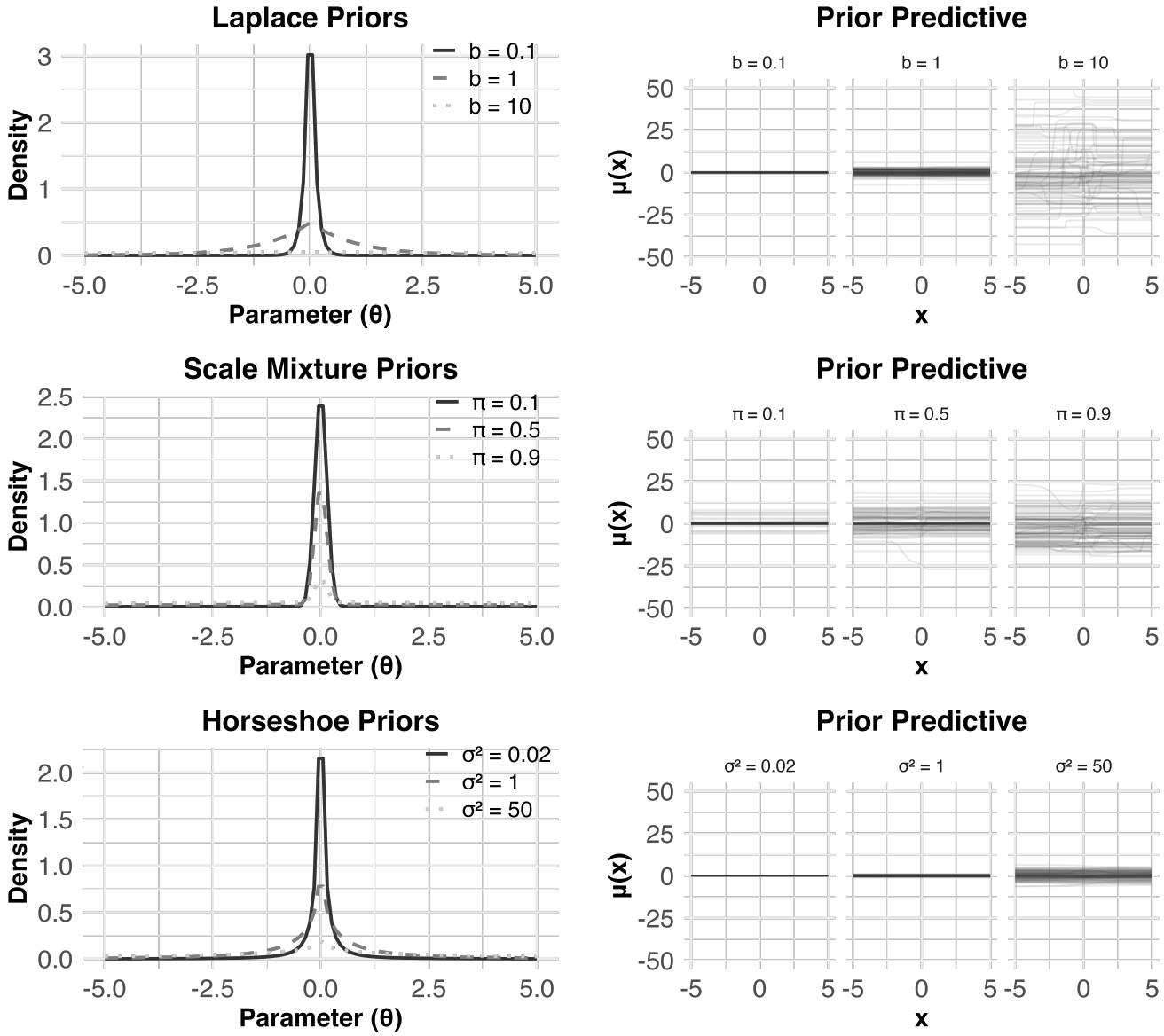


Figure 28.7: Continuous sparsity-inducing prior densities for single parameters (left column) and the corresponding prior predictive distributions (right column) under the baseline architecture: Laplace (top row), scale mixture of Gaussians (middle row), and horseshoe (bottom row) priors under varying hyperparameters.

28.1.5 Model Priors and Bayesian Model Selection and Averaging

In all previous examples, we restricted ourselves to a single BDL model. In this subsection, we allow for multiple BDL models and thus capture model uncertainty. In Bayesian model averaging (BMA), multiple models $\{\mathcal{M}_1, \ldots, \mathcal{M}_t\}$ are considered, requiring priors $p(\mathcal{M})$. We assume prior mass for all these models $p(\mathcal{M}_i), i \in \{1, \ldots, t\}$ and define parameter priors conditionally on the respective models $p(\boldsymbol{\theta}_i \mid \mathcal{M}_i), i \in \{1, \ldots, t\}$. Generally speaking, models could be competing neural networks of different types (such as dense, convolutional, or residual ones) with varying numbers of layers and hidden units, but for simplicity we will only consider models defined by different weight inclusion patterns [801, 1521]. The prior over the model indicator vector $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_P)$ takes the same structural form as the spike-and-slab inclusion prior introduced in Section 28.1.4, but is now interpreted as selecting entire model structures rather than individual weights. In this context, Bernoulli inclusion priors are commonly used:

$$p(\mathcal{M}) = \prod_{i=1}^{P} \text{Bernoulli}(\gamma_i; \rho_i),$$

where $\gamma_i \in \{0, 1\}$ indicates weight inclusion, and ρ_i is the inclusion probability. Beta-Binomial priors [802] are an alternative:

$$p(\mathcal{M}) \propto \prod_{i=1}^{P} \text{BetaBinomial}(\gamma_i; 1, a_i, b_i),$$

which correspond to a Beta-Bernoulli hierarchy where the success probabilities ρ_i are integrated out under a Beta prior, helping control the overall model size. These priors can encode domain knowledge, such as assigning higher inclusion probabilities for input weights in a BDL model for medical diagnosis. Parameter priors are then specified conditionally on the model: $p(\boldsymbol{\theta} \mid \mathcal{M})$, as discussed in the previous subsections.

Inference on the posterior $p(\mathcal{M} \mid \mathcal{D})$ allows us to perform model selection through the median probability model [95], which often leads to extreme sparsification [787], or to perform global explanations based on posterior inclusion probabilities [787].

28.2 Summary and Outlook

This chapter introduced the fundamental role of priors in Bayesian deep learning, progressing from classical informative and conjugate formulations to the regularisation, shrinkage, and variable selection priors commonly applied in modern deep networks. While parameter-space prior elicitation remains challenging, understanding how weight-space choices shape the prior predictive distribution is vital for building robust BDL models. More advanced priors designed to scale to deep structures or handle dependencies have been proposed, including priors that model weight correlations (e.g. Rudner et al. [1434]) and subnetwork priors that isolate key parameters (e.g. Sell and Singh [1488]).

The subsequent chapters in this part develop these concepts further. In Chapter 29, we explore the statistical and mathematical properties of parameter priors as they propagate through hidden units and define infinite-width limits. In Chapter 30, we study data-driven informative priors constructed to achieve desired functional properties. Finally, Chapter 31 introduces function-space priors that bypass weight-space pathology by directly defining distributions over functional outputs.

Chapter 29

Statistical properties of priors

The previous introductory Chapter 28 introduced how priors in Bayesian Deep Learning (BDL) shape model behaviour by influencing the prior and posterior distributions over parameters and functions. While informative priors (see Chapter 30) can encode domain knowledge, their use in BDL is limited due to model complexity. Instead, priors often serve practical roles such as regularisation, promoting sparsity, or enabling efficient inference. Chapter 28 emphasised the importance of understanding prior predictive distributions and illustrated how architectural choices interact with different prior types. This chapter further studies how priors over neural network parameters induce priors over hidden layer units. The main focus is on deriving distributional properties and their potential effects on neural network training. Further, Chapter 31 explores priors in function space, another way to study the influence of priors on the models. Function-space priors are often interpretable but analytically intractable. Chapter 31 highlights pathologies that arise from common parameter priors and proposes matching neural network-induced function priors to target function-space distributions such as Gaussian processes using sample-based objectives like Wasserstein distance or Maximum Mean Discrepancy.

29.1 Introduction

Priors, probability distributions that encapsulate our initial beliefs about model parameters, lie at the heart of Bayesian neural networks (BNNs). The choice of priors is not merely a theoretical consideration but has profound implications for the statistical properties of the resulting models. The theoretical study of priors in BNNs dates back to the foundational work of Neal [1226], who showed that single-hidden layer neural networks with *i.i.d.* Gaussian weights converge to a Gaussian process as the number of neurons of the hidden layer tends to infinity (the so-called infinite-width limit), see Theorem 29.2. After providing an overview of the statistical properties of priors in Bayesian inference and, more specifically, in BNNs, this chapter is organised as follows. Sections 29.2 and 29.3 examine single-hidden-layer and deep neural networks, respectively. Sections 29.4 and 29.5 consider the role of different priors and architectures. Finally, the impact of the prior on the posterior is discussed in Section 29.6.

29.1.1 Simple statistical properties of priors

The core principle of the Bayesian framework is to represent uncertainty about model parameters and predictions using probability distributions. Unlike non-Bayesian or frequentist methods, which treat parameters as fixed but unknown quantities, the Bayesian perspective considers parameters as random variables. Bayesian inference refers to the inductive process of updating beliefs in light of observed data via Bayes' rule. This choice is not arbitrary: it is grounded in rigorous mathematical foundations, notably from the works of Cox and Savage [339, 1463].

We adopt the following notation: let $\mathcal{D} = (X, Y)$ denote a training dataset of size N, assumed to be generated by an underlying *sampling model* or *likelihood* $p(\mathcal{D}|\theta)$. The parameters θ lie in a parameter space Θ, typically a subset of the Euclidean space $\mathbb{R}^P$. The *prior distribution* $p(\theta)$ encodes our beliefs about the parameters θ before observing any data, and constitutes the central theme of this chapter.

Specifying both the prior $p(\theta)$ and the likelihood $p(\mathcal{D}|\theta)$ defines the *joint distribution* of the parameters and data via the product rule: $p(\theta, \mathcal{D}) = p(\theta)p(\mathcal{D}|\theta)$. Bayes' rule combines the prior and the likelihood to produce the *posterior distribution* $p(\theta|\mathcal{D})$:

$$p(\theta|\mathcal{D}) = \frac{p(\theta)p(\mathcal{D}|\theta)}{p(\mathcal{D})}. \tag{29.1}$$

Here, the denominator $p(\mathcal{D})$, known as the model *evidence* or *marginal likelihood*, serves as the normalisation constant.

Bayes' rule (29.1) provides a principled mechanism for updating our beliefs in light of new data, but it offers no guidance on how those beliefs, encoded in the prior, should be initially specified. In practice, prior choices are often influenced by computational convenience. A notable example is the use of conjugate priors: a prior is said to be conjugate to a likelihood if the resulting posterior belongs to the same parametric family. Classic instances of conjugate prior-model pairs include the Gaussian-Gaussian, Beta-Binomial, and Gamma-Poisson.

The issue of prior choice often leads to the question: how informative is the prior? This introduces a distinction between objective and subjective priors. For example, Jeffreys' prior, defined as proportional to the square root of the determinant of the Fisher information matrix, is considered objective due to its invariance under reparameterisation. See Lee [999] and Nalisnick and Smyth [1222] for studies on Jeffreys' priors in BNNs. Objective or uninformative priors are frequently improper, meaning they do not integrate to a finite total probability (e.g., a uniform prior over an unbounded space). Surprisingly, such priors are widely used in Bayesian analysis, provided the resulting posterior is proper and thus suitable for inference. However, improper priors cannot be used to compute prior predictive distributions or Bayes factors, as these too would be improper.

In contrast, subjective priors lie at the core of the Bayesian philosophy, encoding expert beliefs or prior knowledge. The process of eliciting such priors is nuanced and challenging; see, for example, Mikkola et al. [1173] for a survey.

The challenge of specifying a prior becomes even more pronounced as models grow in complexity, when parameters lack clear interpretation, or when the parameter space is high-dimensional. In such settings, designing a prior that appropriately covers the space becomes non-trivial, and exact computation of the posterior may be infeasible. In these cases, the quality of Bayesian inference relies heavily on both the approximations made for computational tractability and the form of the prior distribution itself.

29.1.2 Statistical properties of priors in BNNs

The Gaussian distribution is a common and default choice of prior in Bayesian neural networks. Looking for a Maximum a Posteriori (MAP) estimate of such a Bayesian model is equivalent to training a standard neural network under weighted L_2 regularisation. There is currently neither theoretical nor empirical evidence that Gaussian priors are preferable to alternative choices [1213]. Yet, its well-studied mathematical properties lead to Gaussian distributions being the default prior choice in BNNs.

Arguably, the prior that matters the most from a practitioner's point of view is the prior induced in function-space, not in parameter space or weight-space [1767], as the induced prior seen at the function level can provide insight into the typical behaviour of the output functions. For reviews on priors in Bayesian deep learning models, refer to Arbel et al. [43], Fortuin [547], Nalisnick [1223], as well as Chapter 28.

29.1.3 Width and depth limits of BNNs and their implications

While inference in Bayesian neural networks is computationally limited to finite networks, the statistical properties of Bayesian neural networks vary depending on whether one considers finite networks or their infinite width or infinite depth limits.

In the finite width regime, BNNs maintain a finite number of parameters. Integrating Bayesian inference principles allows them to represent uncertainty about the output function of interest by considering a distribution over the network parameters. This approach is crucial in scenarios where overconfidence can lead to detrimental outcomes, such as in identifying rare diseases from medical images or predicting financial market trends.

In the infinite width limit, many BNNs exhibit a theoretical convergence to Gaussian processes (GPs), in the sense that, as the width of the network approaches infinity, the prior distribution over functions induced by a BNN approaches that of a GP. The infinite width perspective illuminates the behaviour of neural networks at scale, offering insights into why wide networks perform well in practice, and aids in the understanding and development of network architectures and priors that can leverage this convergence for improved generalisation and robustness.

In the context of infinite depth limits, BNNs are able to capture increasingly complex functions through deeper architectures. As networks grow deeper, they have the potential to model intricate hierarchical structures within the data. In the infinite depth regime, these networks can exhibit recursive feature transformations that resemble deep GPs [361], see Chapter 25. This allows for more expressive power and the capacity to model data with rich, layered patterns. Theoretical analysis in this regime offers insights into phenomena like the interplay between depth and nonlinearity, leading to smoother functions and better generalisation. However, maintaining computational tractability and avoiding issues such as vanishing gradients become essential challenges. Exploring infinite depth

networks provides a deeper understanding of how neural networks can effectively capture and utilise complex dependencies in data.

Furthermore, the relationship between finite BNNs and their infinite limits also simplifies theoretical analyses and provides a framework for studying the impact of architectural choices, such as depth and activation functions, through a probabilistic lens. This dual perspective makes BNN priors a rich field of study, which we are investigating in the following pages.

29.1.4 Outline

The rest of this chapter is organised as follows. Section 29.2 studies how priors propagate through shallow neural networks (with a single hidden layer), showing the transition from sub-Gaussian first-layer pre-activations to sub-exponential second-layer pre-activations, the "uncorrelated yet dependent" phenomenon among units, and the Gaussian-process limit for shallow nets. Section 29.3 develops the deep case, proving sub-Weibull tails with index increasing in depth, giving exact Meijer-G characterisations for linear/ReLU BNNs, and extending GP limits (including variants yielding Student-t processes under hyperpriors). Figure 29.1 provides a schematic illustration of the material in Section 29.2 and Section 29.3. Section 29.4 contrasts prior families and then details two examples: α-stable priors that preserve heavy tails in infinite width via stable-process limits, and trace-class priors that maintain identifiable hidden units in the infinite-width limit. Section 29.5 examines how activation assumptions (extended envelope) and architectural motifs (residual connections, convolutions, and neural ODEs) reshape unit and function-space priors, typically modulating tail behaviour and correlations. Finally, Section 29.6 analyses how these priors affect posteriors, covering contraction rates (including spike-and-slab and scalable alternatives), statistical adaptation, limits of Bernstein–von Mises phenomena in overparameterised settings, and links to deep GP asymptotics and high-dimensional regimes.

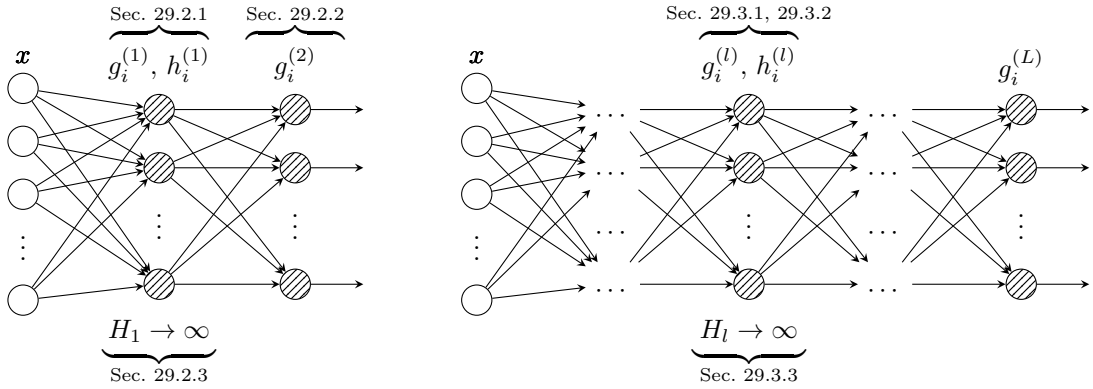


Figure 29.1: Schematic illustration of the material in Section 29.2 and Section 29.3.

29.2 Single-hidden-layer networks

Understanding the distribution of units (pre-activations and post-activations in each layer) under the prior can illuminate how information propagates in an untrained network. This is the focus of a series of works by Vladimirova et al. [1682, 1684, 1685]. The key finding is that unit distributions get progressively heavier-tailed in deeper layers when weights have light-tailed prior distributions, such as Gaussian priors.

29.2.1 First-hidden-layer outputs

Let $x \in \mathbb{R}^D$ be an input vector and assume neural network weights θ follow *i.i.d.* Gaussian distributions $\mathcal{N}(0, \sigma^2)$, and assume, for now, that we have no bias parameters. Then, the pre-activation of the i-th hidden unit in the first layer is a sum of weighted inputs:

$$g_i^{(1)} = \sum_{j=1}^{D} \theta_{i,j}^{(1)} x_j \sim \mathcal{N}(0, \sigma^2 \|x\|^2), \tag{29.2}$$

due to the linearity of expectation and the independence of weights. This result illustrates a key insight: under Gaussian priors, the input to the nonlinearity in the first layer is itself Gaussian.

However, once a nonlinearity ϕ (e. g., ReLU) is applied to obtain the activation $h_i^{(1)} = \phi(g_i^{(1)})$, the output distribution becomes non-Gaussian. For example, with ReLU, $h_i^{(1)}$ is a half-Gaussian (i. e., a Gaussian clipped below zero). The resulting distribution is sub-Gaussian, meaning that its tails decay at least as fast as Gaussian:

$$\mathbb{P}\left(|h_i^{(1)}| > t\right) \leq C\exp(-t^2/K), \quad \text{for any } t \geq 0, \tag{29.3}$$

for some constants $C, K > 0$. This sub-Gaussianity implies that, at the first layer, the prior retains relatively light tails, an important property that shapes the induced prior on functions.

[153] provides closed-form expressions for the first two moments of post-activation units in single-hidden-layer networks with Gaussian weights. These moment characterisations form the basis for bounding tail behaviour and variance growth. However, the restriction to a single hidden layer limits their analysis and does not capture the cumulative non-linear transformations seen in deeper architectures.

29.2.2 Combining the outputs of a single-hidden-layer neural network

Since the outputs of the first hidden layer are non-linear transformations of Gaussians, they are no longer Gaussian. When these are multiplied by a new set of independent Gaussian weights in the second layer (see Figure 29.1), the resulting pre-activations become products of sub-Gaussian and Gaussian variables, i. e. $g_i^{(2)} = \sum_{j=1}^{H_1} \theta_{i,j}^{(2)} h_j^{(1)}$. Vladimirova et al. [1682] shows that such products are sub-exponential with tail bounds of the form:

$$\mathbb{P}(|g_i^{(2)}| > t) \leq C\exp(-t/K), \quad \text{for any } t \geq 0, \tag{29.4}$$

for some constants $C, K > 0$. This heavier tail behaviour marks the transition from sub-Gaussian to sub-exponential distributions and is one of the earliest indicators that deep networks encode increasingly heavy-tailed beliefs under standard Gaussian weight priors.

The previous results provide marginal properties of the prior distribution of hidden units, but their joint distribution is more difficult to characterise as some dependence arises from the second hidden layer, although the units across a layer are uncorrelated [1685]. Theorem 29.1 formalises this property, showing that, under the prior distribution, any two units of any layer are uncorrelated if the weight priors are independent and centred at 0:

Theorem 29.1 (Hidden units dependence, Vladimirova et al. [1685])**.** *If the prior p on the neural network weights θ is centred, in the sense that $\mathbb{E}_p(\theta) = 0$, and independent (isotropic), then any two units of any layer are uncorrelated a priori. Also, their Kendall's tau and Spearman's rho vanish.*

The proof is left to the reader as an exercise.

For finite-width networks, these dependent and non-Gaussian distributions complicate analytic treatment. Marginal properties alone are insufficient for accurate description, as the function-space prior cannot be characterised by simple moments. Approximate representations, such as the use of sub-Weibull, generalised Weibull or Meijer G-function-based priors (see Section 29.3), aim to capture these richer properties in analytically tractable forms.

29.2.3 Gaussian process limit for shallow neural networks

As shown by Neal [1226], when the number of hidden units H_1 tends to infinity, the sum of independent (or weakly dependent) sub-exponential variables with properly scaled variance converges to a Gaussian via the Central Limit Theorem. Figure 29.2 illustrates the width limit for both the single-hidden-layer case (top row) and its generalisation to deeper networks, Theorem 29.4, due to Matthews et al. [386]. The induced prior on the function space then becomes a Gaussian process, with a covariance kernel determined by the activation function and weight variance.

Theorem 29.2 (Neal [1226])**.** *Consider a Bayesian neural network with one hidden layer with H_1 nodes, and let each weight follow a $\mathcal{N}(0, 1/H_1)$ distribution. If the post-activations have bounded variance, then the BNN output converges to a zero-mean Gaussian process as H_1 tends to ∞.*

The proof is left to the reader as an exercise.

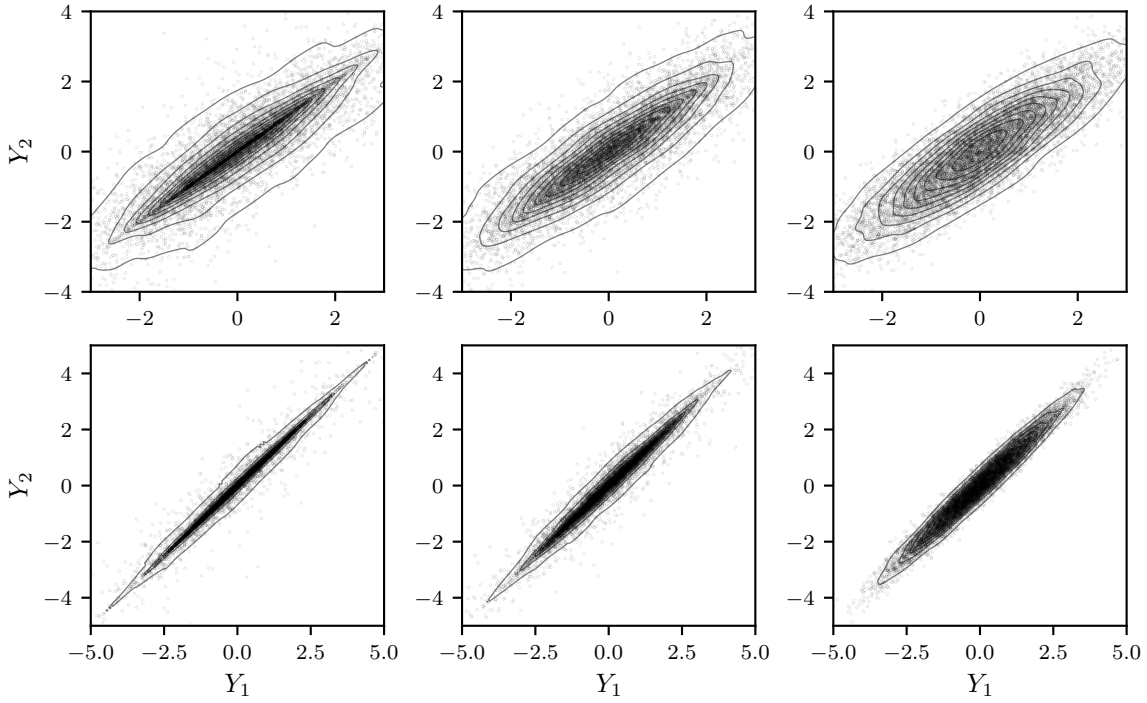


Figure 29.2: Verification of the Gaussian process limit of single-hidden-layer BNNs (Theorem 29.2) and its generalisation to deeper neural networks (Theorem 29.4). Joint distribution at two locations for BNNs of different sizes: one hidden layer (top row) and three hidden layers (bottom row) with 2 (left column), 5 (middle), and $1,000$ (right) hidden nodes per layer. Kernel density plots are shown for the respective distributions based on $10,000$ samples.

29.3 Deep neural networks

As we saw, some dependence between hidden units arises from the second hidden layer, although the units across a layer are uncorrelated. Thus, as also shown in the previous section for two hidden layers, the marginal distribution of l-th layer hidden units is difficult to characterise. Vladimirova et al. [1682, 1684] described unit priors through sub-Weibull and Weibull-tail distributions on the units using the dependence characterisation between the hidden units. Noci et al. [1246], Zavatone-Veth et al. [1833] provide a precise description of the hidden units prior distribution of both linear and ReLU BNNs. One implication of these works is that the preactivation prior is radial. More specifically, if $\boldsymbol{g}$ denotes the preactivations of some layer, then the prior $p(\boldsymbol{g})$ can be written as $p(\|\boldsymbol{g}\|)$. This property enables [1246, 1832] to obtain exact expressions for hidden units priors in terms of so-called Meijer G-functions by exploiting the relationship between the Fourier transforms of radial functions and the Hankel transform.

29.3.1 Sub-Weibull characterisation

The hidden units distributions can be characterised as sub-Weibull distributions, a generalisation of sub-Gaussian and sub-exponential characterization [938, 1683]. To describe this result, we start with the formal definition of a sub-Weibull random variable:

Definition 29.1 (Sub-Weibull random variable, Vladimirova et al. [1683])**.** *A random variable X satisfying for all $t > 0$ and for some $C, \eta > 0$*

$$\mathbb{P}(|X| \geq t) \leq C \exp\left(-t^{1/\eta}\right), \tag{29.5}$$

is called a sub-Weibull random variable with tail parameter η, which is denoted by $X \sim \text{subW}(\eta)$.

Sub-Weibull distributions are characterised by tails lighter than (or equally light as) Weibull distributions; in the same way as sub-Gaussian or sub-exponential distributions correspond to distributions with tails lighter than Gaussian and exponential distributions, respectively. Sub-Weibull distributions are parametrised by a positive tail index η and are equivalent to sub-Gaussian for $\eta = 1/2$ and sub-exponential for $\eta = 1$.

Under a mild assumption on the activation function, namely that it scales roughly linearly as x shoots off to infinity, a notion made precise in Definition 29.2 in Section 29.5.1, we have the following theorem.

Theorem 29.3 (Theorem 3.1, Vladimirova et al. [1682])**.** *Consider a Bayesian neural network with $L-1$ hidden layers, centred Gaussian priors and with activation function satisfying the extended envelope property of Definition 29.2. Then conditional on the input x, the marginal prior distribution on any unit (pre- or post-activation) of the l-th hidden layer is sub-Weibull with tail parameter $\eta = l/2$. That is for any $1 \leq l \leq L$, and for any $1 \leq m \leq H_l$,*

$$u_m^{(l)} \sim \text{subW}(l/2),$$

where $u_m^{(l)}$ is either a pre-activation $g_m^{(l)}$ or a post-activation $h_m^{(l)}$.

Thus, under certain conditions on the activation function (essentially, that it is ReLU-like and unbounded above), a Gaussian prior on weights induces a sub-Weibull($l/2$) distribution on the l-th layer units, where the tail parameter $l/2$ increases with l (larger tail parameter means heavier tail). For example, the first layer is Gaussian, the second layer is sub-exponential. Let us explain intuitively why this happens: multiplying many random matrices (even if each has nice properties) tends to produce outputs dominated by a few large random terms (due to ReLU passing through large positive sums), hence yielding fatter tails. A practical implication of this is that prior predictive distributions of a deep BNN can have high kurtosis: most of the time outputs look moderate, but occasionally they might be very large in magnitude due to a rare combination of large weights and reinforcing nonlinearities. This is a form of prior belief: namely, a deep random network prior believes that extreme activations, while rare, are more likely than a Gaussian belief would suggest. It is a departure from the infinite-width GP, which would predict output distributions to be Gaussian (no excess kurtosis).

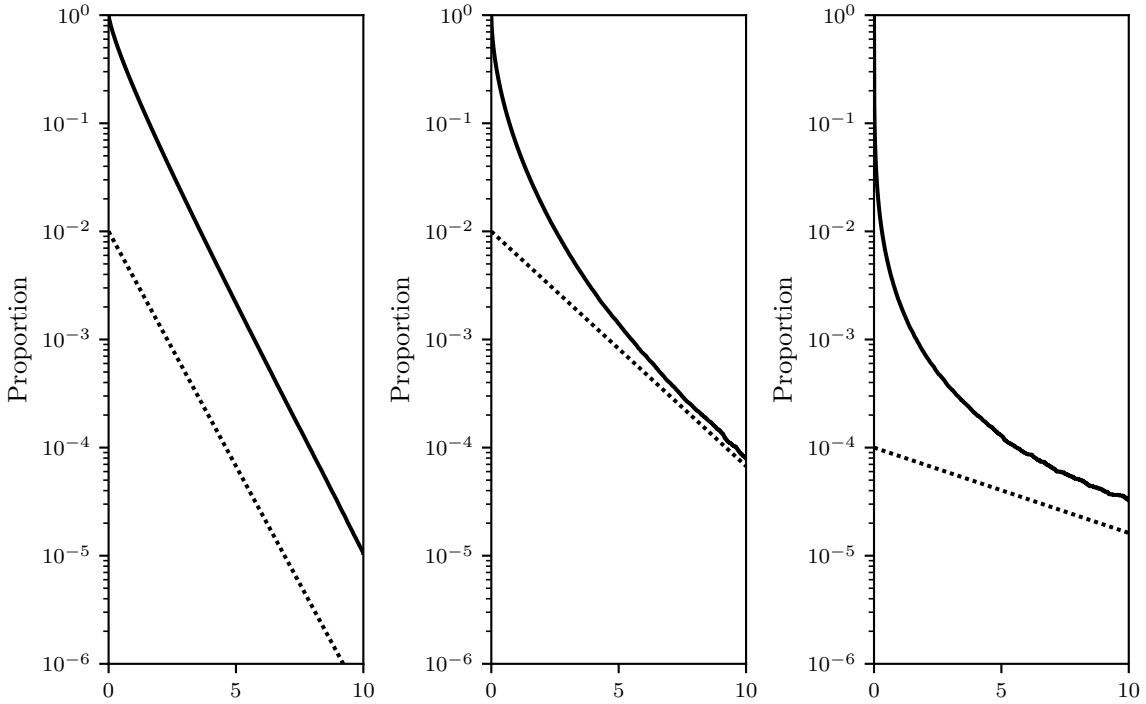


Figure 29.3: Verification of the node distribution. The plots show the tail behaviour for a node after 1 layer (left), 3 layers (middle), and 10 layers (right), both the empirical survival function (solid line) and the theoretical Weibull tail (dashed) are shown.

29.3.2 Meijer G-functions characterisation

Here, we introduce a precise characterisation of hidden units priors proposed by Noci et al. [1246], Zavatone-Veth and Pehlevan [1832] via so-called Meijer G-functions. These are very general special functions that encompass many classical functions (including hypergeometric and Bessel functions) as special cases. They are particularly useful because products of independent random variables often admit closed-form densities expressed through

Meijer G-functions Springer and Thompson [1548]. While these special functions are not commonly encountered in machine learning, they provide a compact way of expressing otherwise intractable densities and enable the derivation of analytical properties such as moments, tail behaviour, and asymptotic expansions.

With this property, Zavatone-Veth and Pehlevan [1832] and Noci et al. [1246] independently derived the density of the joint prior of the L-th layer $\mathbf{h}_L$ of a linear Bayesian neural network with Gaussian priors, with widths $H_1, \ldots, H_L$, as

$$p_L^{\text{lin}}(\mathbf{h}_L \,|\, x) = \frac{\gamma_L}{(2^L \pi \kappa_L^2)^{H_L/2}} G_{0,L}^{L,0}\left(\frac{\|\mathbf{h}_L\|^2}{2^L \kappa_L^2} \,\middle|\, \begin{matrix} - \\ 0, (H_1 - H_L)/2, \ldots, (H_{L-1} - H_L)/2 \end{matrix} \right), \tag{29.6}$$

where

$$\kappa_L \equiv \sigma_1 \cdots \sigma_L \|x\| \quad \text{and} \quad \gamma_L \equiv \prod_{l=1}^{L-1} \frac{1}{\Gamma(H_l/2)}. \tag{29.7}$$

Here, $z \mapsto G_{0,L}^{L,0}\left(z \,\middle|\, \begin{matrix} - \\ 0, (H_1 - H_L)/2, \ldots, (H_{L-1} - H_L)/2 \end{matrix} \right)$ denotes a Meijer-G function with parameters specified by Zavatone-Veth and Pehlevan [1832]. Similarly, the authors show that the density of a ReLU Bayesian neural network has the following mixture form

$$\begin{aligned} p_L^{\text{ReLU}}(\mathbf{h}_L \,|\, x) = &\left(1 - \frac{(2^{H_1} - 1)(2^{H_2} - 1) \cdots (2^{H_{L-1}} - 1)}{2^{H_1 + \cdots + H_{L-1}}}\right) \delta(\mathbf{h}_L) \\ &+ \frac{1}{2^{H_1 + \cdots + H_{L-1}}} \sum_{k_1=1}^{H_1} \cdots \sum_{k_{L-1}=1}^{H_{L-1}} \binom{H_1}{k_1} \cdots \binom{H_{L-1}}{k_{L-1}} p_L^{\text{lin}}(\mathbf{h}_L \,|\, x; k_1, \ldots, k_{L-1}, H_L), \end{aligned} \tag{29.8}$$

where $\delta(\mathbf{h}_L)$ is the H_L-dimensional Dirac distribution, and where $p_L^{\text{lin}}(\mathbf{h}_L \,|\, x; k_1, \ldots, k_{L-1}, H_L)$ denotes the linear BNN distribution of Equation (29.6) with specified hidden layer widths $(k_1, \ldots, k_{L-1}, H_L)$. Using these exact representations, Zavatone-Veth and Pehlevan [1832] derive closed-form expressions for moments, establish tail bounds, and characterize the asymptotic behaviour of deep-layer priors. These results provide theoretical insight into how depth and width shape the distributions induced by Bayesian neural network priors.

29.3.3 Gaussian process limit for deep neural networks

The foundational result 29.2 has been generalised to more than one layer by de G. Matthews et al. [386], which we recall here.

Theorem 29.4 (Theorem 4, de G. Matthews et al. [386])**.** *Consider a Bayesian neural network with a continuous activation function satisfying the extended envelope property of Definition 29.2. Let each weight of the l-th layer follow independently a $\mathcal{N}(0, \sigma^2/H_l)$ distribution. Then the output of the network converges in distribution to a zero-mean Gaussian process as all layers widths $H_1, \ldots, H_L$ tend simultaneously to ∞.*

This theorem motivates the use of BNNs in settings where the function space prior is assumed to be a Gaussian process. This approach offers computational advantages in settings where a large number of point evaluations make direct manipulation of the GP kernel matrix infeasible, see also Chapter 26.

The above theorem is appealing in its generality, but does not give explicit expressions for the mean and covariance kernel of the resulting GP. For specific choices of activation functions and network architectures, it is possible to explicitly obtain the resulting GP mean function and covariance kernel, see e. g. Dutordoir et al. [477], Yang and Salman [1796]. Practitioners may be equally interested in the reverse situation, where one aims to construct a BNN prior based on a target GP on the output function space, one such construction is the Ridgelet prior by Matsubara et al. [1145], which results in a prior that is proven to converge to the desired GP as the layer width is taken to infinity.

Notably, Theorem 29.4 requires the variance of the prior to be scaled exactly proportional to σ^2/H_l with a *fixed* constant σ^2. One may alternatively place a hyperprior on σ^2, a setting which has been investigated by Caporali et al. [232] which shows that the resulting posterior converges to a Student-t process, as is made precise in the following theorem.

Theorem 29.5 (Corollary 3.8, Caporali et al. [232])**.** *The posterior of a 1-dimensional-output BNN ($H_L = 1$) with Gaussian-Inverse-Gamma prior and Gaussian likelihood, evaluated in the input x, converges in law to a multivariate Student-t variable as all layers widths $H_1, \ldots, H_{L-1}$ tend simultaneously to ∞.*

29.4 Different priors

Several priors were introduced in Chapter 28: informative priors, such as the spike-and-slab prior, which embed expert knowledge; uninformative and reference priors, aiming to minimise subjective influence; Gaussian priors for L_2 regularisation, controlling model complexity via variance; sparsity-inducing priors such as Laplace, scale-mixture, and horseshoe distributions [622, 623], encouraging simpler models; and model priors, enabling Bayesian model selection and averaging through structured weight inclusion mechanisms. Properties of the spike-and-slab prior and its role on posterior contraction rates are detailed in Section 29.6.2, following work by Jantre et al. [833], Polson and Ročková [1335]. We present two classes of priors in more details in the rest of the section: α-stable priors and trace-class priors.

29.4.1 α-stable priors

The family of α-stable priors were alluded to by Neal [1226] as candidates for avoiding the Gaussian process limit associated to wide neural networks. The α-stable distribution is characterised by a stability parameter $\alpha \in (0, 2]$, which controls the tail behaviour and the existence of moments. These distributions are defined by their characteristic function, as they typically lack closed-form densities (except for special cases like Gaussian, Cauchy, and Lévy). For a symmetric, centred α-stable random variable X, the characteristic function is:

$$t \mapsto \mathbb{E}[e^{itX}] = \exp(-\sigma^\alpha |t|^\alpha), \tag{29.9}$$

where $\sigma > 0$ is the scale parameter. When $\alpha = 2$, this reduces to a Gaussian distribution. When $\alpha < 2$, the distribution has heavy tails and infinite variance.

[1311] extends the classical Gaussian process limit of deep neural networks by replacing Gaussian weights with symmetric stable distributions. They show that, under appropriate scaling, the outputs of fully connected DNNs converge in distribution to a stochastic process whose finite-dimensional marginals are multivariate stable distributions. This stable process generalises the GP limit stated in Theorem 29.2 and Theorem 29.4, preserving heavy-tailed behaviour in the infinite-width limit. The parameters of the limiting distribution are obtained recursively across layers, and the convergence is established jointly over inputs and layers. Favaro et al. [507] extends these results by establishing sup-norm convergence rates under both joint and sequential layer-width growth, and formalising the recursive characterisation of the spectral measure that defines the limiting process.

29.4.2 Trace-class priors

While the infinite width limit of a standard BNN may be attractive at first glance, it results in unfortunate side effects noted by Neal [1227] in the discussion of his work: *"Infinite networks drawn from these priors do not have hidden units that represent 'hidden features' of the input. The ability to find such hidden features is often seen an interesting aspect of neural network learning."* One alternative to the uniform scaling of the prior variances is given by trace-class BNN priors [1488], which retain hidden units in the infinite width limit, and aid with identifiability as each weight has a distinct variance. The trace-class BNN prior on parameters θ consisting of weights $w_{i,j}^{(l)}$ and biases $b_i^{(l)}$ is given by the following independent Gaussian distributions

$$w_{i,j}^{(1)} \sim \mathcal{N}\left(0, \frac{\sigma_{w^{(1)}}^2}{i^\alpha}\right), \quad w_{i,j}^{(l)} \sim \mathcal{N}\left(0, \frac{\sigma_{w^{(l)}}^2}{(ij)^\alpha}\right) \text{ for } l \geq 2, \quad b_i^{(l)} \sim \mathcal{N}\left(0, \frac{\sigma_{b^{(l)}}^2}{i^\alpha}\right), \tag{29.10}$$

where $\alpha > 0$, $\sigma_{w^{(l)}}^2 > 0$ and $\sigma_{b^{(l)}}^2 > 0$ are variance-related parameters to be specified.

We now follow Sell and Singh [1488] and define the infinite width limit of the network. Given an infinite sequence of weights and biases for the first layer, distributed according to the prior of Equation (29.10), i. e. $\left\{\left(b_i^{(1)}, w_{i,1}^{(1)}, \ldots, w_{i,D}^{(1)}\right) : i \in \mathbb{N}\right\}$, all the functions of the first layer, $\{g_i^{(1)} : i \in \mathbb{N}\}$, are clearly well defined. We define all the functions of the second layer corresponding to an infinite-width first layer to be the following almost sure limits, assuming they exist:

$$G_i^{(2)}(x) = \lim_{H_1 \to \infty} b_i^{(2)} + \sum_{j=1}^{H_1} w_{i,j}^{(2)} \phi(g_j^{(1)}(x)). \tag{29.11}$$

Assuming the random functions $\{G_i^{(2)} : i \in \mathbb{N}\}$ are well defined, which will be the case if $\alpha > 1/2$ as we show in Theorem 29.6, proceeding iteratively, all the functions $\{G_i^{(l)} : i \in \mathbb{N}\}$ of subsequent layers, $l = 3, \ldots, L$, are

also well defined. The functions in each layer of a finite width network are denoted with lower case to clearly distinguish them from their infinite width versions. For the output layer, the finite network gives $g^{(L)}(x)$ while the infinite network gives $G^{(L)}(x)$.

A non-centred trace-class prior can be defined but requires the mean parameters to be appropriately summable. The σ parameters are overall scaling parameters of the priors variance, while α controls the decay of the weights in the width: large values result in quickly decaying weights in each layer, while small values result in a slower decay. The constraint $\alpha > 1/2$ ensures that the prior is well-defined in the infinite width limit as shown in the theorem below.

Theorem 29.6 (Sell and Singh [1488]). *Assume the activation function ϕ is 1-Lipschitz and satisfies $\phi(0) = 0$. Let $L \in \mathbb{N}$ and $\alpha > 0$. Fix further $\sigma^2_{w^{(l)}} > 0$ and $\sigma^2_{b^{(l)}} > 0$ for each $l \in [L]$. Then, the functions of the layers of the* finite-width *neural network satisfy*

1. *For all $x \in [0,1]^D$ one has $|g_i^{(l)}(x)| < \infty$ almost surely, and $\mathbb{E}\left[g_i^{(l)}(x)\right] = 0$. Furthermore, there exists σ_l^2 such that $\mathbb{E}\left[(g_i^{(l)}(x))^2\right] < \sigma_l^2/i^\alpha$ for all $x \in [0,1]^D$. Here, both expectations are taken with respect to the prior on the parameters θ of the neural network. In particular this property holds for $g_1^{(L)}(x)$.*
2. *There exists $c_l \geq 0$ such that for all $x_1, x_2 \in [0,1]^D$, $\mathbb{E}\left[\left(g_i^{(l)}(x_1) - g_i^{(l)}(x_2)\right)^2\right] \leq c_l \|x_1 - x_2\|^2/i^\alpha$, with the expectation again taken with respect to the prior. In particular this gives $\mathbb{E}\left[\left(g_1^{(L)}(x_1) - g_1^{(L)}(x_2)\right)^2\right] \leq c_L\|x_1 - x_2\|^2$.*

In addition, if $\alpha > 1/2$, the functions on every layer of the infinite-width neural network, see Equation (29.11)*, exist almost surely and satisfy the above Properties 1 and 2, when the functions $g_i^{(l)}(x)$ therein are replaced with $G_i^{(l)}(x)$ as defined in Equation* (29.11)*.*

The trace-class prior is attractive due to its interpretability of the functions on the last hidden layers, which do not converge to 0 as one considers the infinite-width limit. It further improves model identifiability, as swapping nodes changes the prior probabilities for the resulting network. The natural ordering of the respective weights lends the trace-class prior as an obvious choice for partial or subnetwork inference [242], as well as a valuable prior for multilevel Monte Carlo, see Chapter 4. The hyperparameters can either be chosen by the user, or inferred by placing a hyperprior on each of them.

29.5 Different architectures

29.5.1 Different activation functions

As described in previous sections, the forward propagation of priors on weights induces progressively heavier-tailed distributions on the units. In this section, we examine how these properties interact with different activation functions. To this aim, the activation function ϕ is required to span at least half of the real line as follows. We introduce an extended version of the activation function assumption from Matthews et al. [386]:

Definition 29.2 (Extended envelope property, Vladimirova et al. [1684]). *An activation function $\phi : \mathbb{R} \to \mathbb{R}$ is said to obey the* extended envelope property *if there exist $c_1, c_2 \geq 0$, $d_1, d_2 > 0$ such that the following inequalities hold*

$$|\phi(u)| \geq c_1 + d_1|u| \quad \text{for all } u \in \mathbb{R}_+ \text{ or } u \in \mathbb{R}_-, \tag{29.12}$$

$$|\phi(u)| \leq c_2 + d_2|u| \quad \text{for all } u \in \mathbb{R}. \tag{29.13}$$

The interpretation of this property is that ϕ must shoot to infinity at least in one direction ($\mathbb{R}_+$ or $\mathbb{R}_-$, at least linearly (29.12), and also at most linearly (29.13). Of course, compactly supported activation functions such as sigmoid and tanh do not satisfy the extended envelope property but the majority of other activation functions do, including ReLU, ELU, PReLU, and SeLU.

29.5.2 Different layers

While fully connected networks have been the focus of much of the preceding analysis, modern architectures frequently incorporate structural innovations such as residual connections, convolutional layers, and continuous-depth models. These modifications significantly affect the prior and its induced function space behaviour.

Residual layers. Residual connections, as introduced in ResNets, can be viewed as introducing an identity mapping in addition to the learned transformation [721]. Formally, the pre-activation at layer l, $g^{(l)}$, can be written as:

$$g^{(l)} = g^{(l-1)} + r^{(l)}(g^{(l-1)}; \theta^{(l)}), \tag{29.14}$$

for some mapping $r^{(l)}$. Under Bayesian treatment, each residual block contributes a random perturbation to the input of the next layer. This modifies the prior by inducing a random walk structure over functions, leading to smoother transitions across layers. The residual architecture results in slower growth of variance with depth compared to standard feed-forward architectures, and may suppress heavy-tailed behaviour [1684].

Convolutional layers. Convolutional networks apply shared filters across spatial locations [993]. Let $\theta^{(l)}$ be a convolutional kernel and $*$ denote the convolution operator. Then the pre-activation at layer l, $g^{(l)}$, is given by:

$$g^{(l)} = \theta^{(l)} * h^{(l-1)} + b^{(l)}, \tag{29.15}$$

where $h^{(l-1)}$ denotes the post-activation at layer $l-1$. Due to weight sharing and local receptive fields, the induced function-space prior has strong spatial correlations and invariances. This structure leads to a significant reduction in the number of free parameters and typically induces smoother and more data-adaptive priors, which can outperform corresponding infinite-width GPs or neural tangent kernels [829] in practical settings.

Neural ODEs. Neural ODEs [272] generalise discrete-depth networks by considering a continuous evolution of hidden states. For a hidden state $h(t)$ and parametrised vector field $v(h, t; \theta)$, the dynamics are governed by:

$$\frac{dh(t)}{dt} = v(h(t), t; \theta), \quad h(0) = x. \tag{29.16}$$

The output of the network is then given by solving the initial value problem over some interval $[0, T]$. The induced prior on functions arises through the distribution over θ:

$$h(T) = x + \int_0^T v(h(t), t; \theta)\, dt. \tag{29.17}$$

If v is linear in h and $\theta \sim \mathcal{N}(0, I)$, then $h(t)$ behaves similarly to a Gaussian process in time [574]. More generally, the prior is implicit and can be seen as a stochastic process in function space, influenced by the ODE solver and integration method. The continuous-time formulation enables adaptive computation and smoother transitions, but complicates prior predictive analysis, as closed-form density expressions are generally unavailable.

In practice, modern architectures often combine multiple structural motifs, e. g., residual connections embedded within convolutional layers or ODE blocks used in sequence models. This results in complex, hierarchical priors that defy simple analytic characterisation. These hybrid designs further emphasise the role of architectural choices in shaping unit priors.

29.6 Influence of the prior on the posterior

29.6.1 Bayesian asymptotics

The field of Bayesian asymptotics focuses on the behaviour of the posterior distribution $p(\theta \mid \mathcal{D}^n)$ as the sample size n of the data $\mathcal{D} = \mathcal{D}^n$ tends to infinity [617–619]. In settings where a true data-generating process characterised by a true parameter θ_0 is assumed to exist, the field of Bayesian asymptotics aims to establish the following guarantees: *posterior consistency* at θ_0 refers to $p(U^c \mid \mathcal{D}^n) \to 0$ as $n \to \infty$ in P_{θ_0}-probability for all neighbourhoods U of θ_0, i. e. $p(\theta \mid \mathcal{D}^n)$ concentrates on the true θ_0; *contraction rates* describe how quickly $p(\theta \mid \mathcal{D}^n)$ concentrates around θ_0, i. e. how quickly neighbourhoods U can shrink with n [617, 619]; *Bernstein–von Mises (BvM) theorems* are types of Central Limit theorems which state that under regularity conditions, $p(\theta \mid \mathcal{D}^n)$ converges to a Gaussian centred at the maximum likelihood estimator $\hat{\theta}^{\mathrm{MLE}}$, with covariance given by the inverse Fisher information [175, 1547].

Bayesian neural networks allow us to place prior distributions over network weights and perform inference through the resulting posterior distribution. Understanding the impact of the prior on the asymptotic behaviour of the posterior is critical to establishing theoretical guarantees and bridging the gap between Bayesian learning and its frequentist counterpart. This section provides an overview of the asymptotic theory of BNNs, focusing on three key aspects: contraction rates, adaptation, and Bernstein–von Mises theorems. Many of the results below rely on regularity assumptions. Their counterparts for singular models, which frequently arise in Bayesian Deep Learning, are discussed in Chapter 52 on Singular Learning Theory.

29.6.2 Posterior contraction rates

Before stating the main result of posterior concentration for sparse ReLU networks with spike-and-slab prior [1335], we introduce the required notation. Let f_0 denote the unknown true regression function defined on $[0,1]^D$, assumed to belong to a Hölder class of smoothness $\alpha > 0$. We observe data (Y_i, x_i), where

$$Y_i = f_0(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0,1), \quad i = 1, \ldots, n,$$

the restriction to $\mathcal{N}(0,1)$ noise is for simplicity. We define a Bayesian neural network prior using deep ReLU networks of depth L, width N, and sparsity s, equipped with a spike-and-slab prior on the weights and offsets. The architecture parameters L, N, and s are chosen depending on the smoothness level α, the dimension D, and the sample size n, to achieve optimal approximation properties.

Denote by $\|\cdot\|_n$ the empirical L^2 norm on the training data, and let $\epsilon_n = n^{-\alpha/(2\alpha+D)} \log^{\delta}(n)$ for some $\delta > 1$ denote the near-minimax rate for estimating an α-smooth function in dimension D.

Theorem 29.7 (Posterior concentration, simplified from Theorem 6.1 of Polson and Ročková [1335])**.** *Assume that f_0 is α-Hölder smooth with $\alpha < D$ and uniformly bounded. Then, for a suitably chosen network architecture depending on α, D, and n, the posterior distribution concentrates around f_0 at the rate ϵ_n, i. e.,*

$$\Pi\left(f : \|f - f_0\|_n > M_n \epsilon_n \mid Y^{(n)}\right) \to 0$$

in probability as $n \to \infty$, for any sequence $M_n \to \infty$.

Significant progress has been made in establishing posterior contraction rates for BNNs in nonparametric settings. Fully connected deep neural networks have been shown to achieve (near) optimal rates in tasks such as regression [244, 911] and multiclass classification [190]. These results suggest that neural networks possess a remarkable capacity for automatic adaptation to unknown function smoothness.

While classical spike-and-slab priors, that are mixtures of a point mass and a diffuse slab, have been extensively used to achieve optimal rates [833, 1335], their computational complexity makes them less practical in deep learning contexts. Recent developments have focused on alternative priors that retain theoretical optimality while improving computational feasibility [916, 1006].

Contraction rates for variational approximations to BNN posteriors are explored by Bhattacharya and Maiti [150], Bhattacharya et al. [151], offering insights into the theoretical performance of scalable inference methods. In parallel, related results are derived for deep Gaussian process priors [361, 463], with recent advances by Bachoc and Lagnoux [72], Castillo and Randrianarisoa [245], Finocchio and Schmidt-Hieber [519] who characterise their convergence behaviour.

The high-dimensional nature of modern applications raises the question of how posterior rates scale with both sample size n and input dimension D. While most existing results assume a fixed D, Bhattacharya et al. [152] is a notable exception, deriving minimax-optimal rates that depend jointly on (n, D), and thus addressing more realistic big data scenarios.

29.6.3 Adaptation

Beyond contraction rates, recent work has shown that overparameterised networks with heavy-tailed priors can achieve automatic statistical adaptation, with posterior distributions concentrating at minimax-optimal rates without prior knowledge of function smoothness [244]. This connects with earlier findings that Gaussian priors over network weights can induce increasingly heavy-tailed behaviour through the layers of a deep network [1682].

29.6.4 Bernstein–von Mises theorems

Bernstein–von Mises (BvM) theorems describe the asymptotic normality of posterior distributions, providing a Bayesian analogue to the classical central limit theorem. In the context of BNNs, exact BvM results are difficult to obtain owing to non-identifiability, symmetries and overparameterisation of the models.

For shallow neural networks, with moderate depth and width and well-behaved priors, classical BvM behaviour can be recovered. However, as networks become deeper and more expressive, posterior distributions deviate significantly from Gaussian behaviour, and standard BvM theorems fail to hold [175].

Current research aims to extend BvM-type results to functionals of the posterior, such as predictions or feature-specific summaries, rather than the entire parameter space. This more realistic approach aligns with the practical use of BNNs, where uncertainty over predictions is often of primary interest.

29.7 Discussion and open questions

Finite-width networks induce non-Gaussian priors over the output functions, but retain certain "Bayesian" properties. For instance, by de Finetti's theorem [383], one can view the finite network's function distribution as an exchangeable random process indexed by inputs, but exact distributional forms are intractable beyond very small networks or only under highly restrictive assumptions. The non-Gaussianity means finite networks can, in principle, capture more complex prior beliefs (e. g. multimodal function distributions), but it also complicates analysis. In contrast, in the infinite-width limit, many BNN priors yield a Gaussian process over the output functions, simplifying theoretical analyses and opening exciting connections to the study of GPs.

In some cases, finite-width neural networks outperform their infinite-width counterparts [49, 477, 587, 1001]. Arora et al. [49] show that convolutional neural networks outperform their corresponding limiting neural tangent kernel (NTK, see Section 8.2, and Jacot et al. [829]). This performance gap is likely due to the finite width effect where a fixed kernel cannot fully describe the convolutional neural network dynamics. The evolution of the NTK along with training has its benefits on generalisation as shown in further works [482, 796]. However, since the performance of finite and infinite-width neural networks is different, it is still not clear when the GP framework is more amenable to describe the BNN behaviour.

Chapter 30

Informative priors

This chapter introduces a framework for constructing informative, data-driven prior distributions over the parameters of neural networks [1436, 1437]. Specifying meaningful priors for neural-network models is inherently difficult because it is rarely obvious how to map a particular parameter configuration to a desired functional behaviour, such as good calibration, high predictive uncertainty under semantic shift (for example, on inputs from classes absent from the training data), a low predictive negative log-likelihood, or robustness to other distribution shifts.

The framework uses problem-specific information and auxiliary data to assign high prior density to parameter values that induce these desired functional properties. We discuss two specific instantiations:

1. **Uncertainty-aware priors**, designed to improve uncertainty quantification by favouring models that exhibit high predictive entropy on inputs meaningfully distinct from the training distribution (Section 30.2);
2. **Group-aware priors**, designed to improve group robustness by favouring models that generalise well to data subgroups underrepresented in the training data (Section 30.3).

30.1 A Family of Informative Data-Driven Priors

We begin by formalising the model structure. Let $\theta \in \Theta$ denote a realisation of the neural-network parameter vector, and let $p(y \mid x, \theta; f)$ be the observation model induced by the network f. For classification, f may output logits that parameterise a categorical distribution; for regression, it may parameterise, for example, the mean of a Gaussian observation model. We partition $\theta = (\theta_h, \theta_L)$ into the final-layer parameters θ_L and the parameters θ_h of all preceding layers (the feature extractor). The network mapping is thus $f(\cdot; \theta) := h(\cdot; \theta_h)\theta_L$, where $h(\cdot; \theta_h)$ is the post-activation output of the penultimate layer. We assume access to pre-trained feature-extractor parameters θ_h^* and to context data encoding useful information about the downstream task.

Our framework is built on a *context inference problem*. We use a batch of context inputs $x_\mathrm{c} = \{x_1, \ldots, x_M\}$ and corresponding labels $y_\mathrm{c} = \{y_1, \ldots, y_M\}$, drawn from a joint distribution $p_{X_\mathrm{c}, Y_\mathrm{c}}$. Capital letters X_c and Y_c denote random variables, whereas x_c and y_c denote their realisations. We then introduce a Bernoulli random variable $\check{Z} \in \{0, 1\}$ indicating whether θ induces a model with a desired property; for example, $\check{Z} = 1$ may indicate high uncertainty on x_c or high accuracy on evaluation points with a specified group attribute.

We express this relationship through the context observation model $\check{p}(\check{z} \mid \theta; f, p_{X_\mathrm{c}, Y_\mathrm{c}})$, the probability mass function of the realisation $\check{z}$ conditional on θ. The quantities following the semicolon are fixed ingredients used to construct the model, not additional random conditioning variables. Given a base prior $p(\theta)$, the informative data-driven prior is the posterior distribution over θ conditional on the desired outcome $\check{z} = 1$. For notational simplicity, we suppress fixed arguments such as f where no ambiguity arises. By Bayes' theorem,

$$p(\theta \mid \check{z} = 1; p_{X_\mathrm{c}, Y_\mathrm{c}}) = \frac{\check{p}(\check{z} = 1 \mid \theta; p_{X_\mathrm{c}, Y_\mathrm{c}}) p(\theta)}{\check{p}(\check{z} = 1; p_{X_\mathrm{c}, Y_\mathrm{c}})}. \tag{30.1}$$

This conditional distribution is then used as the prior in a subsequent inference problem for the neural-network parameters, for example in maximum a posteriori (MAP) estimation or in approximate Bayesian inference such as mean-field variational inference (Chapter 13).

To specify the context observation model, we interpret $\check{Z} = 1$ as achieving the desired property (e.g., reliable

uncertainty). We link the probability of this outcome to a non-negative cost function c:

$$\begin{aligned}\check{p}(\check{z}=1 \mid \theta; p_{X_c,Y_c}) &:= \exp(-\mathbb{E}_{p_{X_c,Y_c}}[c(X_c, Y_c, \theta)]), \\ \check{p}(\check{z}=0 \mid \theta; p_{X_c,Y_c}) &:= 1 - \check{p}(\check{z}=1 \mid \theta; p_{X_c,Y_c}).\end{aligned} \tag{30.2}$$

Because c is non-negative, these two quantities define a valid Bernoulli probability mass function. Equations (30.1) and (30.2) show that the usefulness of the resulting prior depends on the cost function, context distribution, and base prior. The principal modelling challenge is therefore to choose c and p_{X_c,Y_c} so that the context observation model captures the intended functional property as faithfully as possible.

30.2 Uncertainty-Aware Data-Driven Priors

Following Rudner et al. [1438], we now instantiate this framework to construct an uncertainty-aware prior for fine-tuning pre-trained classifiers.

Base prior. We specify layer-specific base priors. For the feature-extractor parameters θ_h, we use a Gaussian prior centred on the pre-trained parameters: $p(\theta_h) = \mathcal{N}(\theta_h \mid \theta_h^*, \tau_h^{-1}\mathbf{I})$. This encourages the solution to remain close to the pre-trained representation. For the final-layer parameters, we use a standard zero-mean Gaussian prior: $p(\theta_L) = \mathcal{N}(\theta_L \mid \mathbf{0}, \tau_L^{-1}\mathbf{I})$.

Cost function and context distribution. For this classification construction, $f(x_c; \theta)$ denotes the logits evaluated on the M context inputs. For each output coordinate k, $[f(x_c;\theta)]_k \in \mathbb{R}^M$ collects the corresponding logits across the context batch. We define the cost using a squared Mahalanobis distance between these logits and a target mean:

$$c(x_c, y_c, \theta) := \tau_c \sum_{k=1}^{K} D^2_{\mathcal{M}}([f(x_c;\theta)]_k, m(x_c, y_c)_k, C(x_c)), \tag{30.3}$$

where $\tau_c > 0$ is a cost-scaling parameter, distinct from the base-prior precisions τ_h and τ_L, and K is the number of output dimensions. The squared Mahalanobis distance in Equation (30.3) is

$$D^2_{\mathcal{M}}(\mathbf{a}, \mathbf{b}, \mathbf{C}) := (\mathbf{a}-\mathbf{b})^\top \mathbf{C}^{-1}(\mathbf{a}-\mathbf{b}). \tag{30.4}$$

To encourage high uncertainty on the context inputs, we set $m(x_c, y_c)_k := [y_c]_k$ and choose the context-label distribution to be a point mass at zero, $p(y_c \mid x_c) := \delta_{\mathbf{0}}(y_c)$. This makes the target mean zero almost surely under the context-label distribution.

We define the covariance function by

$$C(\cdot) := s_1 h(\cdot; \theta_h^*) h(\cdot; \theta_h^*)^\top + s_2 \mathbf{I}, \tag{30.5}$$

where $s_1 > 0$ and $s_2 > 0$ are fixed scaling parameters. The form in Equation (30.5) reflects the structure of the pre-trained model's representations.

Finally, we take $p(x_c)$ to be an empirical distribution over inputs meaningfully distinct from the training set, such as out-of-distribution inputs. By Equations (30.2) and (30.3), a low cost—and hence a high value of $\check{p}(\check{z}=1 \mid \theta)$—is obtained when the logits are close to the zero target in the Mahalanobis metric of Equation (30.4). For a softmax classifier, logits close to zero produce a near-uniform predictive distribution, so the resulting prior favours high predictive uncertainty on the specified out-of-distribution context inputs.

30.3 Group-Aware Data-Driven Priors

Following Rudner et al. [1437], we specify a useful group-aware prior by choosing (i) a context distribution p_{X_c,Y_c} for which observing $\check{z}=1$ is informative and (ii) a cost function c that serves as a proxy for group robustness under that distribution.

Both choices are generally challenging. One could, for example, learn a tailored generative model for the context distribution or construct a more elaborate cost function. Here we use a fixed, pre-specified context distribution and a simple cost function to illustrate the framework.

Context distribution. We assume access to a small validation dataset $\mathcal{D}_{\text{val}}$ containing group information. We form the context distribution by upweighting rare groups in this dataset:

$$\begin{aligned} p_{X_c,Y_c} &= \sum_{g\in\mathcal{G}} \alpha_g p_g, \qquad \alpha_g = \frac{\widetilde{\alpha}_g}{\sum_{g'\in\mathcal{G}} \widetilde{\alpha}_{g'}}, \\ \widetilde{\alpha}_g &= \left(\frac{|\mathcal{D}_{\text{val}}|}{|\mathcal{D}^g_{\text{val}}|}\right)^{\gamma}. \end{aligned} \tag{30.6}$$

Here p_g is the empirical distribution of validation examples in group g, $|\mathcal{D}^g_{\text{val}}|$ is the number of such examples, and $\gamma \geq 1$ controls the strength of the upweighting. The construction does not require the validation-group frequencies to match the training-group frequencies; it deliberately defines a reweighted context distribution. As Equation (30.6) shows, increasing γ increases the relative emphasis on rarer groups.

Cost function. We define a cost that acts as a proxy for group robustness, inspired by sharpness-aware minimisation (SAM) [544]. The task loss $\ell(\cdot)$—typically cross-entropy for classification or mean-squared error for regression—is evaluated at a first-order worst-case parameter perturbation:

$$c(x_c, y_c, \theta) := \ell(y_c, f(x_c; \theta + \rho\,\epsilon(\theta, x_c, y_c)))\,, \tag{30.7}$$

where $\rho > 0$ is the perturbation magnitude. For a non-zero loss gradient, the perturbation direction is

$$\epsilon(\theta, x_c, y_c) := \perp\!\left(\frac{\nabla_\theta \ell(y_c, f(x_c;\theta))}{\|\nabla_\theta \ell(y_c, f(x_c;\theta))\|_2}\right), \tag{30.8}$$

where $\perp(u)$ denotes the `stop_gradient` operator: it has forward value u but derivative zero. When the loss gradient vanishes, we set $\epsilon(\theta, x_c, y_c) = \mathbf{0}$. The normalised gradient is the unit-ℓ_2 direction that maximises the first-order increase in the loss; the stop-gradient operator treats this direction as fixed when differentiating c.

Low loss is generally associated with good generalisation, but the small validation set contains few examples from rare groups, making generalisation to those groups difficult. The context distribution counteracts this imbalance by upweighting rare-group examples, while the perturbed loss favours parameter configurations that remain low-loss under a local worst-case perturbation. Such configurations are expected to generalise better to the upweighted rare-group context data.

The stop-gradient operator changes the derivative used in optimisation without changing the forward value of the perturbation. To make this explicit, let $\widetilde{\epsilon}(\theta, x_c, y_c)$ denote the same normalised gradient as in Equation (30.8), but without stop-gradient, and let θ_t denote the current parameter iterate. Using column-vector gradients, holding x_c and y_c fixed, writing $J_{\widetilde{\epsilon}}(\theta_t)$ for the Jacobian of $\widetilde{\epsilon}$ with respect to θ at θ_t, and using ϑ as a dummy parameter vector, the chain rule gives

$$\begin{aligned} &\nabla_\theta \ell(y_c, f(x_c; \theta + \rho\,\widetilde{\epsilon}(\theta, x_c, y_c)))|_{\theta=\theta_t} \\ &\quad = \left(\mathbf{I} + \rho J_{\widetilde{\epsilon}}(\theta_t)\right)^\top \nabla_\vartheta \ell(y_c, f(x_c;\vartheta))|_{\vartheta=\theta_t+\rho\,\widetilde{\epsilon}(\theta_t, x_c, y_c)}\,. \end{aligned} \tag{30.9}$$

With stop-gradient, automatic differentiation omits the Jacobian contribution in Equation (30.9) and returns

$$g_{\text{sg}}(\theta_t) := \nabla_\vartheta \ell(y_c, f(x_c;\vartheta))|_{\vartheta=\theta_t+\rho\,\epsilon(\theta_t, x_c, y_c)}\,. \tag{30.10}$$

Thus, the forward value in Equation (30.7) still defines the cost used in the context observation model, but g_{sg} is a SAM-style surrogate gradient rather than the exact gradient of that forward scalar cost. Without additional integrability conditions, this surrogate vector field need not be the gradient of a globally defined alternative scalar potential.

30.4 Summary

The framework in this chapter constructs informative priors by conditioning a base prior on the event that the induced model exhibits a desired functional property. The uncertainty-aware construction combines out-of-distribution context inputs with a Mahalanobis cost that favours near-uniform predictions, whereas the group-aware construction combines an upweighted rare-group validation distribution with a SAM-style perturbed loss. In both cases, the quality of the prior depends on how faithfully the context distribution and cost encode the intended property; developing broader classes of well-specified context observation models remains an open direction.

Chapter 31

Function-space priors

As discussed in Chapter 28, in Bayesian statistics, the choice of prior distributions over model parameters represents a fundamental part of the modelling process [601]. In many applications, the elicitation of such prior information on model parameters can be attempted through domain expertise (see, e.g., Lindley et al. [1051], O'Hagan [1255]).

However, for deep learning models, this is not straightforward, and this is the focus of this chapter. Without loss of generality, we are going to consider the case of fully connected Neural Network (NN) models; much of the discussion applies to different architectures too. Let $f(x, \theta)$ denote the output obtained by a NN with parameters θ for an input x. There are a number of challenges in choosing sensible priors for θ due to the fact that NNs are typically over-parameterised. Over-parameterisation in NNs means that the number of parameters can be very large (easily beyond a few millions) leading to statistical non-identifiability, and requiring one to pick a prior for every single parameter is a daunting task. Blocking strategies can alleviate this burden, e.g., by assigning the same prior to parameters in the same layer. However, a fundamental problem still remains: *what is the effect produced by the choice of a particular prior distribution over θ on the prior distribution over $f(x, \theta)$?*

This chapter is dedicated to the study of such prior distributions over $f(x, \theta)$. In Section 31.1, we provide some background on function-space priors defined as the priors over $f(x, \theta)$ induced by the choice of a prior over θ. After discussing the intractability of characterising these priors analytically in Section 31.1.1 and Section 31.1.2, we focus on ways in which we can obtain samples from these priors in Section 31.1.4. We then move on to discussing in Section 31.2 how the choice of convenient priors may lead to pathological priors in function-space. Indeed, astute readers with background in Bayesian Statistics may find it easy to circumvent the problem of specifying priors over NN parameters by opting for weakly informative priors over θ. Unfortunately, such a naïve and seemingly benign approach may lead to function-space priors over $f(x, \theta)$ with undesirable properties; see Tran et al. [1627] for a study of Bayesian model selection in the context of unsupervised learning problems.

We conclude this chapter with Section 31.3 where we study ways in which we can choose priors over θ inducing function-space priors exhibiting desirable properties. The majority of the focus of this latter section is on matching NN function-space priors with function-space priors of interest and for which we have easy access to samples. This is realised by casting this matching task as the optimisation of a suitable objective, and we present a number of options by drawing from the literature on divergences and integral probability metrics. All in all, this chapter should convince readers that care needs to be taken when choosing priors for Bayesian deep learning, and that there are practical strategies to adopt in order to do so.

31.1 From priors over parameters to priors over functions

For simplicity of exposition, let's focus on the case of a supervised learning problem where the aim is to establish a mapping between inputs $x \in \mathcal{X}$, with $\mathcal{X} \subset \mathbb{R}^D$, and labels $y \in \mathcal{Y}$ given a collection of pairs (x_i, y_i) with $i = 1, \dots, N$. Consider the following definition of a NN-based model with parameters θ:

$$f(x, \theta) \in \mathbb{R}^C, \qquad \text{with} \qquad \theta \in \mathbb{R}^P, \tag{31.1}$$

which we can use, together with an observation model, to define a likelihood function $p(y_1, \dots y_N \mid x_1, \dots, x_N, \theta)$ measuring the goodness of fit of the model to the given data (equivalently, one may write $p(y_{1:N} \mid x_{1:N}, \theta; f)$ to make the dependence on the network map explicit). For now we do not write a specific form for the likelihood, as it is not the focus of the discussion. In order to complete the specification of the Bayesian NN model for the given data, we need to choose a prior for θ:

$$\theta \sim p(\theta). \tag{31.2}$$

With a complete model specification, we can easily realise that the posterior over θ is intractable and that we need approximations. We will come back to this important point later in the chapter, but for now let's focus our discussion on how to choose $p(\theta)$ sensibly.

The choice of a prior over θ induces a distribution over $f(x, \theta)$, meaning that it yields a prior distribution over the NN output for any given input x. More formally, let's denote by $F_{1:n} = (F_1, \dots, F_n)$ with $F_i \in \mathbb{R}^C$ the collection of random variables representing the NN function associated with the inputs $x_1, \dots, x_n$. Note that the inputs are not necessarily the training inputs here, so we denote the number of such inputs as n instead of N. We can obtain the prior distribution over $F_{1:n}$ by marginalising out θ as follows:

$$p(F_{1:n}) = \int p(F_{1:n} \mid \theta) p(\theta) d\theta = \int \delta(F_{1:n} - (f(x_1, \theta), \dots, f(x_n, \theta))) p(\theta) d\theta. \tag{31.3}$$

The function $\delta(\cdot)$ represents the Dirac delta, and we use it to highlight the deterministic nature of the forward computations of the NN which gives the components $f(x_i, \theta)$ for a given input x_i and θ.

31.1.1 Function-space priors are desirable but intractable

Equation 31.3 expresses the distribution over the output of the NN. Why is this of any interest? And what kind of properties should we expect this distribution to have? Earlier, we discussed how in Bayesian inference it is important to include prior knowledge from experts into a modelling problem and how this principle is of little help in guiding the choice of a prior over NN parameters θ. In contrast, it is often much easier to reason about the kind of functions we may want to model *a priori*. In other words, function-space priors are more "interpretable"; it is much easier to reason about macroscopic properties that we would like to encode in the functions represented by NNs, such as, e.g., degree of smoothness, periodicity, and marginal variance, rather than dealing with individual NN parameters. Indeed, the statistical nonidentifiability and nonlinearity induced by the over-parameterisation of NNs make it challenging to study the sensitivity of the function-space prior with respect to the prior over individual parameters.

Unfortunately, the integral in Equation 31.3 is generally intractable, again due to the nonidentifiability and nonlinearity of the NN parameterisation, and reasoning about their function-space priors is not immediately possible. Considering a Gaussian prior $p(\theta)$, for example, it is easy to see that the nonlinear dependence of $f(x, \theta)$ from θ makes it intractable to calculate this integral. In Section 31.3 we are going to discuss strategies to determine prior distributions over θ so that function-space priors exhibit properties of interest. Before doing so, we present more insights into the function-space prior induced by a prior over the parameters by viewing Equation 31.3 as a change of variables operation and study it in the context of neural processes.

31.1.2 Interpretation of function-space priors as a change of variables

Another way to interpret the operation carried out in Equation 31.3 is to view it as a change of variables, where the prior measure over θ is transformed into a measure over $F_{1:n}$. Here n represents the number of input locations at which we evaluate the function-space prior. Recalling that $x_i \in \mathcal{X}$ with $\mathcal{X} \subset \mathbb{R}^D$, imagining that we are interested in analysing the behaviour of the function $f(x, \theta)$ at the n input locations, we are then interested in the distribution of $F_{1:n}$. For reference, if T were a differentiable bijection between spaces of the same dimension and $F = T(\theta)$, the usual change-of-variables formula would give

$$p_F(f) = p_\theta(T^{-1}(f)) \left| \det J_{T^{-1}}(f) \right|.$$

The map induced by a neural network at fixed inputs, $T_{x_{1:n}}(\theta) = (f(x_1, \theta), \dots, f(x_n, \theta))$, rarely satisfies these assumptions: it is typically many-to-one and often maps parameter space into an output space of different dimension. It is clear that the mapping taking θ as input and mapping these into $F_{1:n}$ is not a bijection in general, which makes the Jacobian rank deficient (rank at most the minimum between P and nC).

This view, however, reveals something interesting about the properties that we can expect to observe in the prior over $F_{1:n}$. Assuming $nC > P$, the change of variables formula indicates that the prior over $F_{1:n}$ is going to live in a low-dimensional manifold of P dimensions. We may be tempted to still characterise the change of measure over this manifold by computing the determinant of the Jacobian through the computation of its top P eigenvalues; however, this requires computing and factorising the Jacobian, which is an $(nC) \times P$ matrix, requiring $\mathcal{O}(nCP^2) + \mathcal{O}(P^3)$ computations. Thus the change-of-variables view is useful mainly as a diagnostic: it explains why evaluating the function-space density is difficult and why the induced distribution may concentrate on a low-dimensional set. The practical methods below therefore rely on samples from $p(F_{1:n})$ rather than on pointwise evaluations of this density.

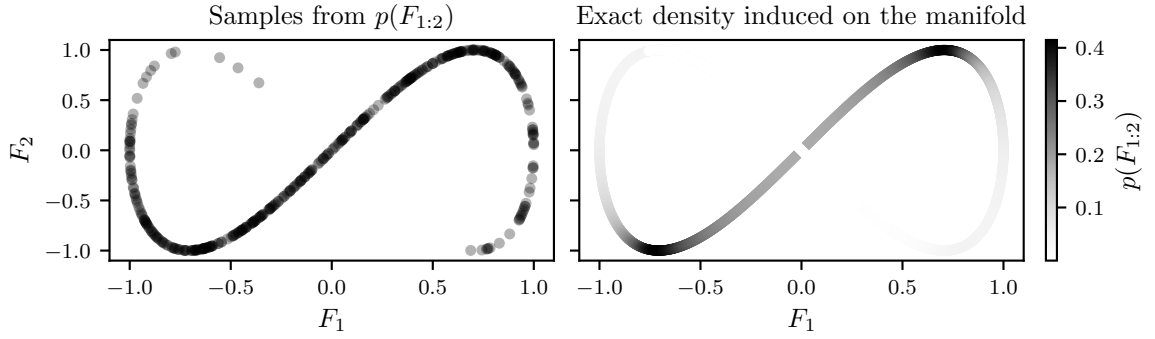


Figure 31.1: Visualisation of the mapping $(F_1, F_2) = (\sin(\theta\, x_1), \sin(\theta\, x_2))$ for $\theta \sim \mathcal{N}(0, 1)$ at input locations $x_1 = 1$ and $x_2 = 2$. (Left) Monte Carlo samples showing that all points lie on a one-dimensional sinusoidal curve in $[-1, 1]^2$; (Right) Exact induced density along this curve, computed via the density-on-manifold change-of-variables formula $p(F_{1:2}) = p(\theta)/\|dg(\theta)/d\theta\|_2$.

To visualise this phenomenon better, consider a single-parameter sine network evaluated at two input points, $x_1, x_2 \in \mathbb{R}$, that is $g : \theta \mapsto (F_1, F_2) = (\sin(\theta x_1), \sin(\theta x_2))$. Note that this is the case discussed before, where the number of input points ($n = 2$) is greater than the number of parameters ($P = 1$). Figure 31.1 shows what happens to $p(F_{1:2})$. When we sample $\theta \sim \mathcal{N}(0, 1)$ and plot the resulting pairs (F_1, F_2) in the plane, the points concentrate along a smooth, closed curve in $[-1, 1]^2$ rather than filling the square, as expected by the low-rank form of the Jacobian. In this simple case, we can also derive explicitly the induced density along the curve, using the change-of-variable formulas.

31.1.3 Connections with Neural Processes

Stochastic processes offer a natural formalism to specify priors over functions. Formally, a stochastic process indexed by $x \in \mathcal{X}$ is a collection of random variables $\{F(x)\}_{x\in\mathcal{X}}$ such that, for any finite set of indices $x_1, \ldots, x_n$, the random vector $F_{1:n} = (F(x_1), \ldots, F(x_n))$ has a well-defined joint distribution $p(F_{1:n})$. These joint distributions are the *finite-dimensional marginals* of the process, and by the Kolmogorov extension theorem a family of such marginals that is consistent and exchangeable defines a unique process over the whole index set $\mathcal{X}$. The Gaussian process of Section 31.3 is the canonical example, its finite-dimensional marginals being Gaussians fully determined by a mean and a covariance function. The question is under what conditions the construction of a function-space prior in Equation 31.3 yields a stochastic process. The literature on stochastic processes provides a complete answer to this, based on the necessity to satisfy exchangeability and consistency [862]. Exchangeability requires invariance of the joint distribution with respect to permutations of the inputs:

$$p(F_{1:n}) = p(\tilde{F}_{1:n}) \tag{31.4}$$

where we denoted by $\tilde{F}_{1:n}$ the permuted versions of $F_{1:n}$. Consistency requires some form of "closure" with respect to marginalisation; in particular, we require that:

$$p(F_{1:m}) = \int p(F_{1:n}) dF_{m+1:n} \tag{31.5}$$

For a fixed prior over θ and a measurable deterministic neural network, the finite-dimensional marginals induced by Equation 31.3 are permutation invariant and marginally consistent, and therefore define a stochastic process in this formal sense. This observation is conceptually useful, but it does not remove the main computational issue: the finite-dimensional densities are still generally unavailable in closed form. In the literature, a construction making use of NNs to learn process-like finite-dimensional distributions has been proposed in Garnelo et al. [582]. These so-called Neural Processes possess some interesting properties, as they allow for their direct use as function-space priors. Intuitively, a neural process learns to behave like a stochastic process directly: given a *context* set of input–output pairs, an encoder network summarises them into a global latent variable, and a decoder network maps this latent variable, together with a target input, to a predictive distribution over the corresponding output. Sharing the latent variable across all target inputs is what couples the predictions and lends the model its process-like, internally consistent behaviour, while training over many randomly sampled context/target splits encourages (approximate) exchangeability and consistency. The result is a flexible, learnable prior over functions whose finite-dimensional marginals are produced by a single forward pass, rather than by marginalising a prior over weights

as in Equation 31.3. In the remainder of this chapter, however, we do not need to use the full stochastic-process structure explicitly in order to determine priors over parameters.

31.1.4 Sampling from function-space priors

The intractability of the function-space prior Equation 31.3 does not prevent us from being able to obtain samples from it. Viewing Bayesian NNs as hierarchical models characterised by $p(\theta)$ and $p(F_{1:n} \mid \theta)$, it is a simple matter to set up a sampling scheme to obtain samples of $f(x, \theta)$ for a finite set of locations $x_1, \ldots, x_n$. This can be achieved by iteratively sampling $\theta' \sim p(\theta)$ and then $F_i \sim p(F_i \mid \theta')$, that is simply $f(x_i, \theta')$ for $i = 1, \ldots, n$. By repeating the sampling of θ' and by evaluating the corresponding $f(x_i, \theta')$ for $i = 1, \ldots, n$, we obtain samples from the joint distribution $(\theta, F_{1:n})$; we can then simply disregard samples from θ to be left with samples from the marginal distribution $p(F_{1:n})$.

31.2 Pathologies of function-space priors

Having established that the function-space prior density $p(F_{1:n})$ of Equation 31.3 is generally analytically intractable, so that we can sample from it but not evaluate it in closed form, we are now going to study the behaviour of function-space priors. This analysis aims at highlighting certain pathologies which emerge from making simple and seemingly benign choices for the priors over the parameters θ. In particular, we study function-space priors for various NN architectures, by varying their depth.

31.2.1 Possible priors over θ

The most common choice is an independent Gaussian prior over the weights, $\theta \sim \mathcal{N}(0, \sigma^2 I)$. More flexible alternatives include *hierarchical* Gaussian priors, which place hyper-priors on the per-layer variances and thereby induce heavier-tailed weight marginals, and *normalising-flow* priors [1628], which transform a simple base distribution through an invertible neural network to obtain a far richer family of weight distributions. We return to these last two families in Section 31.3, where their flexibility is exploited to match a target function-space prior; in the present section, which is concerned only with illustrating pathologies, we focus on the simplest Gaussian case. In all cases, we normalise by scaling the prior variance with the dimension of the preceding layer. This is inspired by the literature on initialisation of NN parameters to avoid saturation effects which induce exploding/vanishing gradients (see, e.g., He et al. [720]). The top row of Figure 31.2 illustrates the functions generated by a fully-connected network with tanh activations whose weights are drawn from a fan-in-scaled Gaussian prior with unit base variance, for increasing depth (2, 4, and 8 layers). Increasing depth results in progressively more pathological functions. Interestingly, the marginal variance of the function-space prior can be kept under control, but the covariance of the realisation of the function $f(x, \theta)$ at different inputs becomes increasingly large, leading to nearly-constant functions. This is hardly desirable in any applications where we expect to observe some interesting behaviour in the data. This prior is putting a considerable amount of emphasis on "uninteresting" nearly-constant functions; we will discuss how to tackle this problem in Section 31.3.

Increasing the width of each layer results in function-space priors resembling Gaussian processes. This can be verified by looking at finite-dimensional distributions of $f(x, \theta)$ at any fixed collection of inputs, which become increasingly Gaussian as the width increases [386, 1227]. However, this effect can only be seen in shallow architectures unless skip connections are introduced [481].

31.3 Matching neural network priors with target function-space priors

In this section, we study ways in which we can impose priors over θ so that the induced distribution over the output of the network matches some target function-space prior of interest. The only working assumption we are going to make is that we have easy access to samples from the target function-space prior. This makes the approach presented here quite general and applicable in a wide range of scenarios.

31.3.1 Candidates for function-space priors

In this section we propose a few ideas of possible function-space priors for which it is easy to obtain samples, without necessarily having access to their analytical form.

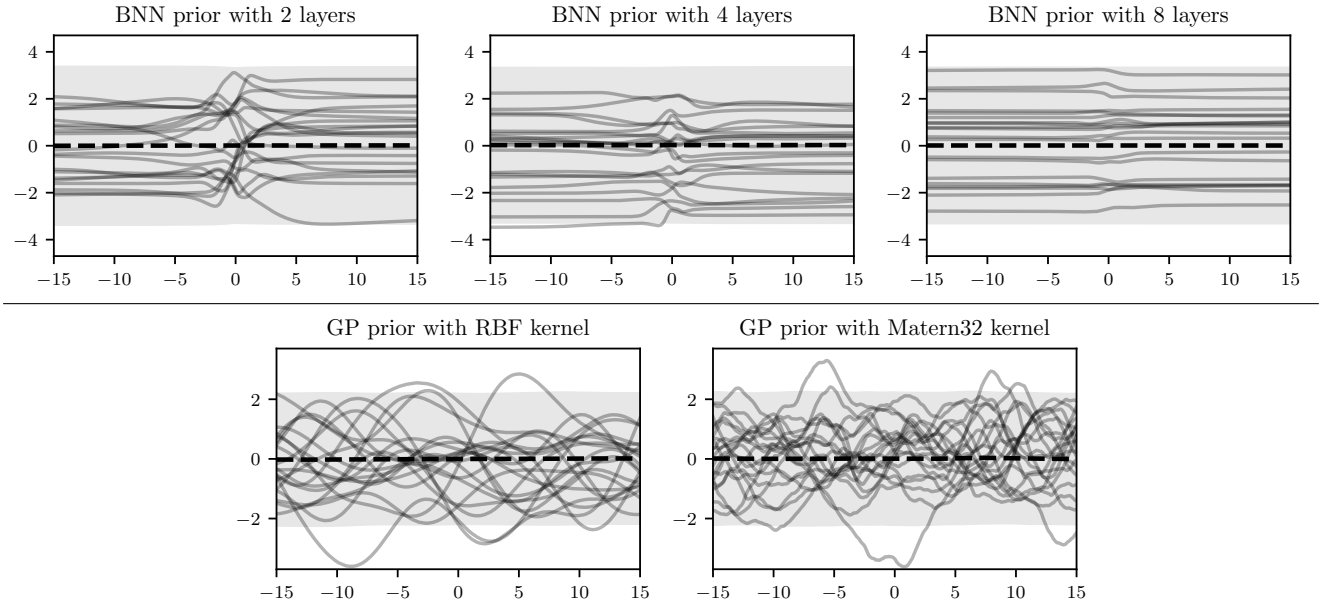


Figure 31.2: (Top) Sample functions of a fully-connected BNN with (left to right) 2, 4, and 8 layers, obtained by placing a fan-in-scaled Gaussian prior with unit base variance on the weights with tanh activations; increasing depth drives the prior towards progressively more pathological, nearly-constant functions. (Bottom) Samples from a GP prior with an RBF (left) and a Matérn-3/2 (right) kernel, which retain a controlled length-scale and marginal variance. The horizontal axis is the one-dimensional input and the vertical axis the function value. The two rows are shown together to contrast the pathologies of deep BNN priors with the well-behaved GP priors.

Gaussian process priors

As a candidate for an easy-to-interpret function-space prior we can use Gaussian processes, for which sampling is easy. In particular, considering again a set of n input points $x_i \in \mathcal{X} \subset \mathbb{R}^D$, we can define a Gaussian process through the choice of a covariance function $k(\cdot, \cdot)$ and a mean function $\mu(\cdot)$. These functions define the behaviour and properties of samples in the function space, as illustrated in the second row of Figure 31.2. In order to draw samples from the Gaussian process prior, we first calculate the covariance matrix K with entries $k_{ij} = k(x_i, x_j)$ and the mean vector with entries $\mu_i = \mu(x_i)$, and then we factorise it using the Cholesky decomposition as $K = LL^\top$. The lower triangular factor L allows us to obtain one sample from the Gaussian process prior by drawing n independent Gaussian random variables $\nu \sim \mathcal{N}(0, I)$ and then by setting $f = L\nu + \mu$. We can repeat this process as many times as needed to obtain multiple samples f from the Gaussian process prior. Once K is factorised (cost in $\mathcal{O}(n^3)$), the cost of drawing one sample is in $\mathcal{O}(n^2)$. Gaussian processes also have the added benefit of providing an analytical form for their density, but we will not use this specifically here.

Physics-based simulations

Beyond explicit function-space priors like Gaussian processes, physics-based simulations offer a sophisticated means to define function-space priors, especially when the underlying system dynamics are governed by known physical laws. These simulations can be driven by differential equations (see, e.g., Lorenzi and Filippone [1079]) and specific boundary/initial conditions, intrinsically constrain the space of admissible functions to those that are physically consistent and plausible. By running these simulations under varied, yet physically realistic, input parameters or initial states, one can generate prior samples in function space that implicitly encode the structure, symmetries, and conservation laws dictated by the physics [774, 1570]. This approach is particularly valuable when analytical forms of function-space priors are unavailable or overly simplistic, allowing for the sampling of complex functions that capture the true behaviour of the system, thus providing a rich and informative prior for neural networks [1371].

The case of unsupervised learning

To highlight the generality of this approach, we note that choosing a prior over θ from target distribution *samples* extends naturally from supervised to unsupervised learning. If we consider unsupervised learning tasks, NNs can

be used to model the true underlying probability distribution over the inputs $\pi(x)$ or to obtain a model to sample from $\pi(x)$ without explicitly modelling it. In these cases, NNs implement functions $f(x, \theta)$ aiming at reconstructing the inputs themselves. The function-space prior of interest is then $\pi(x)$ and the samples are just the given input data x_i, as studied in Tran et al. [1627] for Bayesian Auto-Encoders. The point for our purposes is that, exactly as in the supervised case, the matching objectives of Section 31.3 require only samples from the target, here the data themselves, and never an analytical form for it.

31.3.2 Matching problem and objectives

Assuming that we have access to samples from a function-space prior of interest, say $p_{\text{target}}(F_{1:n})$, we are interested in choosing a prior over θ so that the induced prior $p_{\text{nn}}(F_{1:n})$ (where the subscript "nn" stands for "neural network") matches the target function-space prior $p_{\text{target}}(F_{1:n})$ as closely as possible. If we only have access to samples from $p_{\text{target}}(F_{1:n})$, we can only hope to establish a close match conditioned on a set of input locations $x_1, \ldots, x_n$; expecting a perfect match of the two distributions when n is large is generally unrealistic.

Aiming for a "soft" match of these distributions, we now study objective functions measuring the dissimilarity between distributions over functions evaluated at n input locations. Let's denote by ψ the parameters (also known as hyper-parameters) of the prior over θ, that is $p(\theta \mid \psi)$. For example, for a Gaussian prior, ψ could include its mean and variance. The induced function-space prior in Equation 31.3 now reads:

$$p_{\text{nn}}(F_{1:n} \mid \psi) = \int p(F_{1:n} \mid \theta) p(\theta \mid \psi) d\theta. \tag{31.6}$$

Our goal is to minimise an objective measuring the dissimilarity between $p_{\text{nn}}(F_{1:n} \mid \psi)$ and the target function-space prior $p_{\text{target}}(F_{1:n})$ as follows:

$$\arg\min_{\psi} \left\{ \mathcal{D}[p_{\text{nn}}(F_{1:n} \mid \psi), p_{\text{target}}(F_{1:n})] \right\}, \tag{31.7}$$

where $\mathcal{D}(\cdot, \cdot)$ represents a dissimilarity measure between the two arguments.

We begin this section by choosing $\mathcal{D}(\cdot, \cdot)$ to be the Kullback-Leibler divergence. As we will see shortly, the difficulties stemming from the use of this divergence motivate the need to consider alternatives, and we will present two integral probability metrics [1207] for this purpose, namely the Wasserstein distance and the Maximum Mean Discrepancy.

KL Divergence

The KL divergence objective reads:

$$\mathcal{D}_{\text{KL}}[p_{\text{nn}} \| p_{\text{target}}] = \int p_{\text{nn}}(F_{1:n} \mid \psi) \log \left(\frac{p_{\text{nn}}(F_{1:n} \mid \psi)}{p_{\text{target}}(F_{1:n})} \right) dF_{1:n} \tag{31.8}$$

The calculation of this KL divergence is analytically intractable and approximating it through samples is notoriously challenging for large-dimensional distributions. In order to see this, we can rewrite the KL divergence as:

$$\mathcal{D}_{\text{KL}}[p_{\text{nn}} \| p_{\text{target}}] = \mathcal{H}[p_{\text{nn}}] - \mathcal{CH}[p_{\text{nn}} \| p_{\text{target}}] \tag{31.9}$$

where $\mathcal{H}$ and $\mathcal{CH}$ denote the entropy and cross-entropy, respectively. Estimators for the entropy of large dimensional distributions through samples generally suffer from high variance [403], making it difficult to optimise the KL objective with respect to ψ. Similar issues arise in the case of f-divergences and α-divergences. An alternative is to consider other measures of dissimilarity which are easy to estimate based on samples from the two distributions. This is what we discuss next by considering two integral probability metrics.

Wasserstein Distance

The Wasserstein distance is a popular integral probability metric frequently used in optimal transport problems [1679]. Let $\mathcal{F} \subset \mathbb{R}^C$ and $\mathcal{F}' \subset \mathbb{R}^C$ and let $p_1(f)$ and $p_2(f')$ be two (Borel) probability measures. We use the generic symbols p_1 and p_2 while introducing the distance; they will be instantiated as the BNN and target function-space priors, $p_1 = p_{\text{nn}}$ and $p_2 = p_{\text{target}}$, in Equation 31.13 below. The p-Wasserstein distance is defined as[1]

$$W_p(p_1, p_2) = \left(\inf_{\gamma \in \Gamma(p_1, p_2)} \int_{\mathcal{F} \times \mathcal{F}'} D(f, f')^p \gamma(f, f') df df' \right)^{1/p}, \tag{31.10}$$

[1]The definition of the p-Wasserstein distance holds more generally for any complete separable metric spaces $\mathcal{F}$ and $\mathcal{F}'$.

where $D(f, f')$ is a proper distance metric between $f, f' \in \mathcal{F} \times \mathcal{F}'$ and $\Gamma(p_1, p_2)$ is the set of all joint distributions (couplings) with marginals p_1 and p_2.

When $\mathcal{F} = \mathcal{F}'$ the most common formulation is the 1-Wasserstein distance with $D(\cdot, \cdot)$ taken to be the Euclidean norm:

$$W(p_1, p_2) = \inf_{\gamma \in \Gamma(p_1, p_2)} \int_{\mathcal{F} \times \mathcal{F}} \|f - f'\| \gamma(f, f') df df' \,, \tag{31.11}$$

The intuitive meaning of this distance is to measure the minimum amount of mass which should be transported in order to transform the distribution p_1 into the distribution p_2. This is why the 1-Wasserstein distance is also referred to as the Earth-Mover distance.

A convenient rewriting of the 1-Wasserstein distance leads to a dual form which lends itself to a tractable estimation [868, 869]:

$$W(p_1, p_2) = \sup_{\|g\|_L \leq 1} \int_{\mathcal{F}} g(f) p_1(f) df - \int_{\mathcal{F}} g(f') p_2(f') df' \tag{31.12}$$

where g is a 1-Lipschitz continuous function defined on $\mathcal{F} \to \mathbb{R}$. Indeed, the calculation of the 1-Wasserstein distance can now be viewed as an optimisation problem of a simple objective with respect to 1-Lipschitz continuous functions. The objective is just a difference of two expectations, which can be estimated through Monte Carlo, whereas the optimisation with respect to g can be tackled by parameterising g as a neural network.

The 1-Wasserstein distance can be used as a matching objective between $p_{\text{nn}}(F_{1:n} \mid \psi)$ and $p_{\text{target}}(F_{1:n})$:

$$W(p_{\text{nn}}, p_{\text{target}}) = \sup_{\|g\|_L \leq 1} \int_{\mathcal{F}} g(f) p_{\text{nn}}(f \mid \psi) df - \int_{\mathcal{F}} g(f') p_{\text{target}}(f') df' \tag{31.13}$$

Maximum Mean Discrepancy

Another classic example of integral probability metric is the Maximum Mean Discrepancy (MMD) [666]. The presentation of the Wasserstein distance in the previous section allows us to view MMD under a similar vein, as it results in the same objective except that the supremum is taken with respect to functions belonging to a Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$ [48] and bounded norm. The MMD between p_{target} and p_{nn} is defined as follows

$$\text{MMD}(p_{\text{nn}}, p_{\text{target}}) = \sup_{\|g\|_{\mathcal{H}} \leq 1} \int_{\mathcal{F}} g(f) p_{\text{nn}}(f \mid \psi) df - \int_{\mathcal{F}} g(f') p_{\text{target}}(f') df' \tag{31.14}$$

The Hilbert space $\mathcal{H}$ is induced by the choice of a kernel function $k(\cdot, \cdot)$, and it is desirable to consider kernels inducing infinite dimensional Hilbert spaces, which are capable of capturing discrepancies between an infinite number of moments for the two distributions. By leveraging the so-called kernel trick [48], it is possible to express the square MMD in such a way that does not require $g(f)$ directly, but just scalar products between elements of the RKHS, that is the kernel function evaluated at pairs of f, leading to:

$$\text{MMD}^2(p_{\text{nn}}, p_{\text{target}}) = \mathbb{E}_{f, f' \sim p_{\text{target}}, h, h' \sim p_{\text{nn}}(\cdot \mid \psi)} \left[k(f, f') + k(h, h') - 2k(f, h') \right]. \tag{31.15}$$

All these expectations can be estimated in an unbiased fashion through Monte Carlo by using samples from p_{target} and p_{nn}. Furthermore, this formulation lends itself to mini-batching, while preserving unbiasedness.

31.3.3 Imposing GP Priors on Bayesian Neural Networks using Wasserstein Distance

This section presents a practical implementation for imposing GP priors on Bayesian neural networks in function space, using the previously introduced 1-Wasserstein distance. We match the two stochastic processes at measurement points $Z = [z_1, \ldots, z_M]^\top$ sampled from a distribution $q(z)$. For low-dimensional inputs, Z can be drawn from a grid or uniformly sampled. In higher dimensions, samples can be taken from the training set, optionally augmented with noise. If the test input region is known, $q(z)$ can be chosen to cover it.

To match the BNN function-space prior p_{nn} to the target GP prior p_{gp} at these measurement points, we employ a sample-based approach that uses the 1-Wasserstein distance of Equation 31.13 as the objective:

$$\min_{\psi} \max_{\omega} \mathbb{E}_q \Big[\underbrace{\mathbb{E}_{p_{\text{nn}}(\cdot \mid \psi)}[g(f_M, \omega)] - \mathbb{E}_{p_{\text{gp}}}[g(f_M, \omega)]}_{\mathcal{L}(\psi, \omega)} \Big], \tag{31.16}$$

where f_M denotes the set of random variables associated with the inputs Z, and $g(\cdot, \omega)$ is a 1-Lipschitz function parameterised by a neural network with parameters ω. The dependence on the prior hyper-parameters ψ enters through the BNN function-space prior $p_{\text{nn}}(\cdot \mid \psi)$: its samples f_M are produced by first drawing $\theta \sim p(\theta \mid \psi)$ and then evaluating the network at Z, so that minimising over ψ reshapes the BNN prior to match the target.

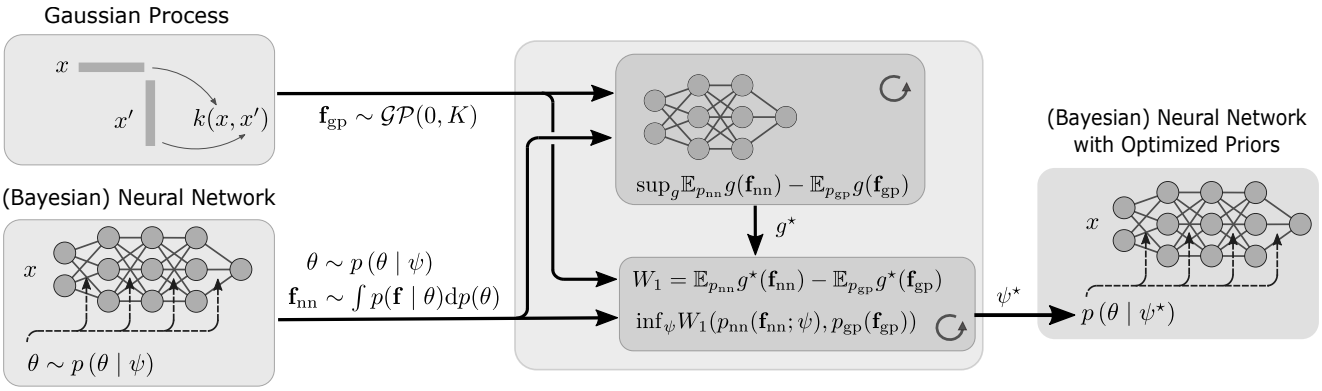


Figure 31.3: Schematic representation of the process of imposing GP priors on BNNs via Wasserstein distance minimisation.

Regarding the optimisation of the ψ and ω parameters, we alternate between $n_{\text{Lipschitz}}$ steps of maximising $\mathcal{L}$ with respect to the Lipschitz function's parameters ω and one step of minimising the Wasserstein distance with respect to the prior's parameters ψ. We therefore use two independent optimisers for ψ and ω. Figure 31.3 offers a high-level schematic representation of the proposed procedure. Given samples from two stochastic processes, the Wasserstein distance is estimated by considering the inner maximisation of Equation 31.16, resulting in an optimal g^*. This inner optimisation step is repeated for every step of the outer optimisation loop. Notice that the objective is fully sample-based. As a result, it is not necessary to know the closed-form of the marginal density $p_{\text{nn}}(\mathbf{f}; \psi)$. One may consider any stochastic process as a target prior over functions, as long as we can draw samples from it.

To enforce the Lipschitz constraint on g_ω, Arjovsky et al. [47] propose clipping the weights ω to lie within a compact set $[-c, c]$, ensuring all functions g_ω are K-Lipschitz. However, this often biases g_ω toward overly simple functions. Building on the fact that a differentiable function is 1-Lipschitz if and only if the norm of its gradient is at most one everywhere, Gulrajani et al. [686] propose penalising deviations of the gradient norm from 1. Specifically, they augment the loss with a gradient penalty term:

$$\mathcal{L}_R(\psi, \omega) = \mathcal{L}(\psi, \omega) + \lambda \, \mathbb{E}_{p_{\hat{\mathbf{f}}}} \left[\left(\left\| \nabla_{\hat{\mathbf{f}}} g(\hat{\mathbf{f}}) \right\|_2 - 1 \right)^2 \right], \tag{31.17}$$

where $p_{\hat{\mathbf{f}}}$ is the distribution of interpolated functions $\hat{\mathbf{f}} = \varepsilon \mathbf{f}_{\text{nn}} + (1 - \varepsilon) \mathbf{f}_{\text{gp}}$, with $\varepsilon \sim \mathcal{U}[0, 1]$, and $\mathbf{f}_{\text{nn}} \sim p_{\text{nn}}$, $\mathbf{f}_{\text{gp}} \sim p_{\text{gp}}$ drawn from BNN and GP priors, respectively. The coefficient λ controls the strength of the regularisation.

31.3.4 Examples on a 1D Regression Synthetic Dataset

The dataset is constructed as follows: (1) uniformly sample 64 input locations $\mathbf{x}$ in the interval $[-10, 10]$; (2) rearrange these locations to introduce a gap in the input space; (3) sample function values $\mathbf{f}$ from a GP prior with lengthscale $l = 0.6$ and amplitude $\alpha = 1$, evaluated at $\mathbf{x}$; (4) corrupt the targets with i.i.d. Gaussian noise with variance $\sigma_\epsilon^2 = 0.1$. We use a three-layer MLP as the model. Figure 31.4 presents the results, organised in five rows, with the reference Gaussian process shown in the leftmost column throughout. We consider three GP-induced (GPi) BNN priors: GPi-Gaussian, GPi-Hierarchical, and GPi-NFlow, whose prior over θ is parameterised by Gaussian, hierarchical, and normalising-flow [1628] distributions, respectively. The first row shows samples from these priors with their hyper-parameters kept *fixed* at sensible default values, that is *not* matched to the GP (labelled "Fixed"), whereas the second row shows samples after the hyper-parameters ψ have been *optimised* to match the GP prior by minimising the Wasserstein distance (labelled "Optim"). The third row reports the convergence of this prior-matching optimisation. The last two rows show the posteriors obtained with the fixed and the optimised priors, respectively.

The procedure is therefore two-stage, and the two stages should not be conflated. In the first (prior-matching) stage we solve the optimisation problem of Equation 31.16 to obtain the prior hyper-parameters $\psi^\star$; this stage

operates purely in function space and only requires *samples* from p_{nn} and p_{gp}. In the second (inference) stage we fix $\psi = \psi^\star$ and draw posterior samples with stochastic gradient Hamiltonian Monte Carlo (SGHMC; Chapter 2) [273]. It is worth emphasising why the second stage is unproblematic even though the *function-space* prior density is intractable: posterior sampling is carried out in *parameter* space, where the prior $p(\theta \mid \psi^\star)$, a Gaussian, hierarchical-Gaussian, or normalising-flow density over θ, is available in closed form and is differentiable. The sampler thus targets the unnormalised parameter-space posterior $p(y \mid x, \theta)\, p(\theta \mid \psi^\star)$ and never needs to evaluate the intractable marginal $p_{\mathrm{nn}}(F_{1:n})$; the optimisation of Equation 31.16 is performed once, beforehand, and is not part of the sampler's inner loop.

From the analysis of these plots, we clearly see the benefit of placing a prior on the functions rather than on the parameters. First, the Wasserstein distance plots show satisfactory convergence, with the normalising-flow prior closely matching the GP prior thanks to its flexibility. Second, as expected, the posteriors exhibit similar behaviour according to the possible solutions realisable from the prior: classic priors tend to yield degenerate functions resulting in overconfidence in regions without data, while our GP-induced priors retain information regarding lengthscale and amplitude.

Overall, the examples show why parameter-space convenience is not enough for Bayesian deep learning: apparently innocuous weight priors can induce highly restrictive priors over functions. Matching in function space provides a practical way to encode interpretable assumptions such as length-scale, amplitude, and smoothness while still carrying out posterior inference in parameter space.

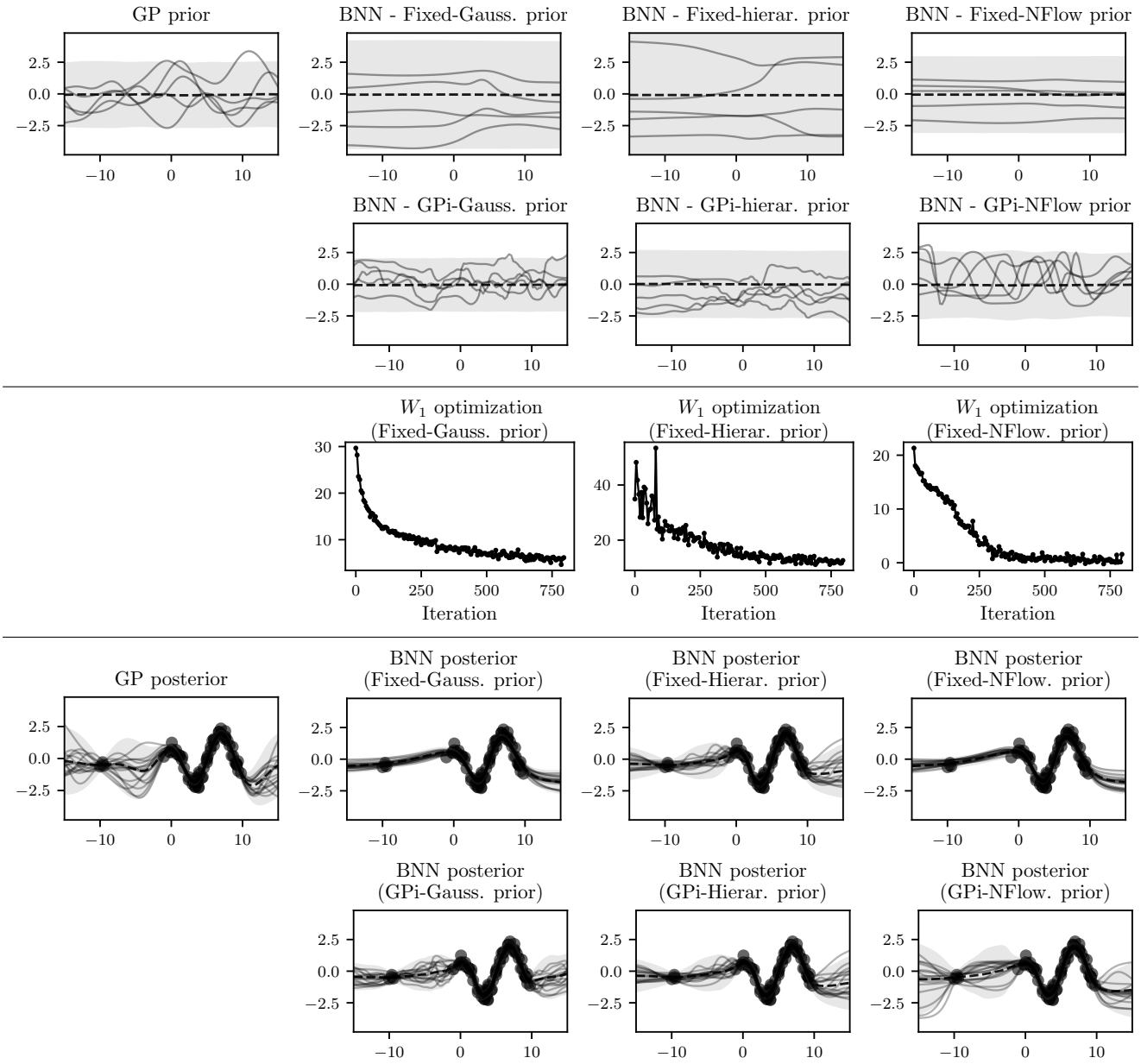


Figure 31.4: Visualisation of a one-dimensional regression example with a three-layer MLP. Throughout, the left column is the reference Gaussian process and the horizontal/vertical axes are the input and the function value, respectively. The first row shows BNN prior samples with *fixed* (default, unmatched) hyper-parameters and the second row with hyper-parameters *optimised* to match the GP prior; the middle row shows the convergence of this prior-matching optimisation. The last two rows show the posteriors obtained, respectively, with the fixed and the optimised priors, sampled with SGHMC. The dashed line and shaded areas denote the posterior mean and the 95% credible intervals.